

BOSTON UNIVERSITY
GRADUATE SCHOOL OF ARTS AND SCIENCES

Dissertation

**PROGRAM ANALYSIS FOR QUANTITATIVE-REACHABILITY
PROPERTIES**

by

JIAWEN LIU

B.A. in Department of Information Science, Central University of
Economics and Finance, 2017

Submitted in partial fulfillment of the
requirements for the degree of
Doctor of Philosophy

2023

© Copyright by
JIAWEN LIU
2023

Approved by

First Reader

Marco Gaboardi, Ph.D.
Associate Professor of Computer Science

Second Reader

Alley Stoughton, Ph.D.
Research Professor of Computer Science

Third Reader

Assaf J. Kfoury, Ph.D.
Professor of Computer Science

Fourth Reader

Hongwei Xi, Ph.D.
Associate Professor of Computer Science

DEDICATION

This dissertation is dedicated to
my loving family
my teachers

ACKNOWLEDGMENTS

Reviewing my path to Ph.D., so many people have helped me.

First and foremost, I express my great appreciation to my advisor, Professor Marco Gaboardi. His guidance on the direction of my research and the way to approach it have been crucial during my Ph.D. journey. He gave me the freedom to work on the problems that I found interesting and, at the same time, he also patiently guided me to be persistent in the direction I chose. I am indebted to him for teaching me how to ponder slowly and deeply to approach the fundamental logic and theory hidden below the observation. When I came into this process, I had minimal knowledge of formal methods and he took the time to teach me step by step. His support went beyond my expectations and his great patience and steadfastness has translated beyond simply the process of research and into my life. I also have to thank him for showing me the importance of writing scientific documents that are not only correct, but also clear and understandable. Without his support, this thesis would never have been written.

I want to also thank my committee members. Great thanks to Assaf J. Kfoury for the kindness in being the chair of my committee and for his valuable comments during my Ph.D. process. I would like to thank Alley Stoughton and Hongwei Xi for being part of my Ph.D committee. I owe my gratitude to Alley Stoughton for patiently teaching me the foundational knowledge about program language. I'm also in debit to Alley Stoughton for her careful and detail comments to improve this thesis. I'm also grateful to Hongwei Xi for his lectures and conversations with me.

I would thank many colleagues for stimulating conversations: Robin Fu and other members in the POPV group; Ludmila Glinskih and other Ph.D students in

the Algorithm & Theory group; Zongshun Zhang, another Ph.D student at Boston University; Gian Pietro Farina and other members in the Programming Language group at University at Buffalo; Ben Niu and other members in the Security and Privacy group at Chinese Academic of Science; and Sheng Gao and other members in the Information Security Department at Central University of Finance and Economics.

I would also like to thank the friends that have made my time in Boston and Buffalo pleasant. My particular thanks goes to Qiancheng Fu, Weihao Qu, Yiyun Huang, Chunyi Zhu, Rawane Issa, Lian Gao.

My gratitude goes also to all the people who are a constant presence in my everyday life. In particular, I am extremely grateful to my grandmother Xiuqin Yang, my brother Hu Liu, and the rest of my family. They are a constant source of backing and help. Finally, my special thanks goes to my lovely kitty Taro for his invaluable company.

PROGRAM ANALYSIS FOR QUANTITATIVE-REACHABILITY PROPERTIES

JIAWEN LIU

Boston University, Graduate School of Arts and Sciences, 2023

Major Professor: Marco Gaboardi, Ph.D.

ABSTRACT

Program analysis studies the execution behaviors of computer programs including programs' safety behavior, privacy behavior, resource usage, etc. The kind of program analysis on the safety behavior of a program involves analyzing if a particular line of code leaks a secret and how much secret is leaked by this line of code. When studying the resource usage of a program, certain program analysis mainly focuses on analyzing whether a piece of code consumes a certain resource and how much resource is used by this piece of code. Yet another kind of program analysis is studying the program privacy behavior by analyzing whether a specific private data is dependent on other data and how many times they are dependent during multiple executions. We notice that when studying the aforementioned behaviors, there are two dominant program properties that we are analyzing – “How Much” and “Whether”, namely *quantitative* properties and *reachability* properties. In other words, we are analyzing the kind of program property that contains two sub-properties – *quantitative* and *reachability*. A property is a hyper-property if it has two or more sub-properties. For the class of properties that has *quantitative* and *reachability* sub-properties, I refer to them as *quantitative-reachability* hyper-properties. Most existing program analysis methods can analyze only one sub-

property of a program's *quantitative-reachability* hyper-property. For example, the reachability analysis methods only tell us whether some code pieces are executed, whether the confidential data is leaked, whether certain data relies on another data, etc., which are only the *reachability* sub-properties. These analysis methods do not address how many times or how long these properties hold with respect to some particular code or data. Quantitative analysis methods, such as program complexity analysis, resource cost analysis, execution time estimation, etc., only tell us the upper bound on the overall quantity, i.e., the *quantitative* sub-property. However, these quantities are not associated with a specific piece of code, program location, private data, etc., which are related to the *reachability* sub-properties. This thesis presents new program analysis methodology for analyzing two representative *quantitative-reachability* properties. The new methodology mitigates the limitations in both reachability analysis methods and quantitative analysis methods and help to control the program's execution behaviors in higher granularity. The effectiveness of the new analysis method is validated through prototype implementations and experimental evaluations.

The first noteworthy *quantitative-reachability* property I look into is the *adaptivity* in the programs that implement certain adaptive data analyses. Data analyses are usually designed to identify some properties of the population from which the data are drawn, generalizing beyond the specific data sample. For this reason, data analyses are often designed in a way that guarantees that they produce a low generalization error. An adaptive data analysis can be seen as a process composed by multiple queries interrogating some data, where the choice of which query to run next may rely on the results of previous queries. The generalization error of each individual query/analysis can be controlled by using an array

of well-established statistical techniques. However, when queries are arbitrarily composed, the different errors can propagate through the chain of different queries and result in high generalization error. To address this issue, data analysts are designing several techniques that not only guarantee bounds on the generalization errors of single queries, but that also guarantee bounds on the generalization error of the composed analyses. The choice of which of these techniques to use, often depends on the chain of queries that an adaptive data analysis can generate, intuitively the *adaptivity* level in an adaptive data analysis. To help analysts with identifying which technique to use to control their generalization error, we consider adaptive data analyses implemented as while-like programs, and we design a program analysis framework. In this framework, we first formalize the intuitive notion of *adaptivity* as a *quantitative-reachability* property, which is a key measure of an adaptive data analysis to choose the appropriate technique. Then we design a program analysis algorithm that estimates a sound upper bound on the adaptivity of the program that implements an adaptive data analysis. We also implement my program analysis and show that it can help to analyze the adaptivity of several concrete data analyses with different adaptivity structures.

As a continuation of the previous work, to get a more precise bound on a program's *adaptivity* level, I look at another *quantitative-reachability* hyper-property – *the number of times a given location inside a procedure is visited during the program execution*. The upper bound on this hyper-property is referred to as the *reachability-bound*. It can help to improve the program analysis results when studying other different program features. For example, the *reachability-bound* on each program location can be used by some resource cost analysis techniques to compute a precise bound on a program's worst-case resource consumption. When we analyze the

adaptivity in an adaptive data analysis program as discussed above, the accuracy of my program analysis result can also be improved through a tight *reachability-bound* on every program location. Some existing program complexity analysis methods can be repurposed to analyze and estimate the *reachability-bound*. However, these methods focus only on the overall quantity and ignore the path sensitivity in the program. For this reason, the *reachability-bounds* of the locations in different sub-procedures are usually over-approximated. As far as we know, there is no general analysis algorithm that computes the reachability-bound for every program location directly and path-sensitively. To this end, I present a path-sensitive reachability-bound algorithm, which exploit the path sensitivity to compute a precise reachability-bound for every program location. We implement this path-sensitive reachability-bound algorithm in a prototype, and report on an experimental comparison with state-of-art tools over four different sets of benchmarks.

Contents

Acknowledgements	v
Abstract	vii
List of Tables	xvi
List of Figures	xvii
1 Introduction	1
1.1 Background	1
1.2 Methodology	5
1.3 Contributions	15
1.4 Dissertation Outline	16
PART i PROGRAM ANALYSIS FRAMEWORK FOR ADAPTIVE DATA	
ANALYSIS	21
2 Introduction	23
2.1 Background	23
2.2 Motivation	28
2.3 Technique Overview	34
2.4 Outline and Contributions	41

3	Previous Works on Adaptivity Analysis	44
3.1	Language Model	44
3.2	Adaptivity Formalization	48
3.3	Program Analysis for Adaptivity	51
3.4	Limitations	56
4	Language Model – QWhile	59
4.1	Syntax	60
4.2	Trace-based Operational Semantics	65
4.3	Examples	75
5	Adaptivity Formalization	77
5.1	May-dependency between Variables	77
5.2	Semantics-based Dependency Graph	86
5.3	Adaptivity	90
5.4	Examples	91
6	Program Analysis for Adaptivity	96
6.1	Algorithm Guideline	97
6.2	Graph Vertex and Annotation Estimation	99
6.3	Graph Edge and Weight Estimation	100
6.4	Dependency Graph Estimation	127
6.5	Adaptivity Upper Bound Computation	128
6.6	Examples	140
7	Examples and Evaluation Results	143
7.1	Examples	143

7.2	Evaluation Results	148
8	Related Works	157
8.1	Formalization Methods	157
8.2	Static Program Analysis	158
8.3	Generalization in Adaptive Data Analysis	160
PART ii	REACHABILITY BOUND ANALYSIS	162
9	Introduction	164
9.1	Background	164
9.2	Motivation	166
9.3	Technique Overview	171
9.4	Outline and Contributions	175
10	Language Model	178
10.1	Labeled Language	178
10.2	Trace-based Operational Semantics	180
11	The Reachability-bound	187
12	Path-sensitive Reachability-bound Analysis	190
12.1	Main Steps	190
12.2	Abstraction Transition Graph	191
12.3	Loop Refinement	200
12.4	Ranking Function Computation and Estimation	206
12.5	Path-sensitive Reachability-bound Algorithm Overview	212
12.6	Loop Bound and Local Path Reachability-bound Computation	215

12.7 Loop Reachability-bound Computation	222
12.8 Path Reachability-bound Computation	226
12.9 Reachability-bound Computation	227
13 Example And Evaluation	229
13.1 Examples	229
13.2 Empirical Evaluation	240
14 Related Works	245
PART iii Epilogue	248
15 Conclusion and Future Works	249
15.1 Program Adaptivity Analysis	249
15.2 Path-sensitive Reachability-bound Analysis	253

PART iv	Appendix	255
A	Supplementary Materials in PART i	256
A.1	Proofs of Lemmas for the Language Model	256
A.2	Proof of Adaptivity Soundness	262
A.3	Soundness of Edge Estimation	271
A.4	Soundness of The Weight Estimation	306
A.5	Soundness of AdaptSearch Algorithm	317
A.6	Conditional Completeness of AdaptSearch Algorithm	326
A.7	The Programs in Evaluation Table	329
A.8	The Codes in Evaluation Table	337
B	Supplementary Materials in PART ii	377
B.1	Proofs of Lemmas in Language Model	377
B.2	Soundness of Program Abstraction	380
B.3	Soundness of Ranking Estimation	387
B.4	Soundness of Path-sensitive Reachability-bounds Analysis	390
	Bibliography	435
	Curriculum Vitae	445

List of Tables

7.1	Experimental results of AdaptFun implementation	150
7.2	Results of the alternative AdaptFun implementation	153
13.1	The Effectiveness Evaluation Table of p_{SRB}	242
13.2	The Accuracy Evaluation Table of p_{SRB}	243
13.3	The Performance Evaluation Table of p_{SRB}	244

List of Figures

2.1	Overview of our Adaptive Data Analysis model. We have a population that we are interested in studying, and a dataset containing individual samples from this population. The adaptive data analysis we are interested in running has access to the dataset through queries of some pre-determined family (e.g., statistical or linear queries) mediated by a mechanism. This mechanism uses randomization to reduce the generalization error of the queries issued to the data.	24
2.2	The generalization errors of two adaptive data analysis examples, under different choices of mechanisms. (a) Data analysis with 2 rounds adaptivity. (b) Data analysis with 400 rounds adaptivity. (c) Same data analysis as (a) with different query numbers.	27
2.3	Architecture of the adaptivity analysis framework	35
2.4	(a) The program <code>toWRounds(k)</code> , an example with two rounds of adaptivity. (b) The corresponding semantics-based dependency graph. (c) The estimated dependency graph from <code>AdaptFun</code>	35
2.5	The overview of <code>AdaptFun</code>	40
3.1	Syntax of loop language.	45
3.2	Trace-based operational semantics of loop language.	47
3.3	The key rule of the variable estimation algorithm.	53

3.4	The key rules of the graph generating algorithm.	54
4.1	Arithmetic and boolean expression evaluation rules	69
4.2	Trace-based operational semantics rules for <code>QWhile</code> language.	72
5.1	(a) The same running example program <code>twoRounds(k)</code> as Figure 2.4(a). (b) The corresponding semantics-based dependency graph.	93
5.2	(a) The loop example with single adaptivity rounds. (b) The corresponding semantics-based dependency graph.	95
6.1	The overview of AdaptFun	97
6.2	(a) The same <code>twoRounds(k)</code> program as Figure 2.4. (b) The abstract transition graph for <code>twoRounds(k)</code> . (c) The abstract transition graph with the estimated reachability bound.	109
6.3	(a) The <code>ifQuery</code> program. (b) The abstract transition graph for <code>ifQuery</code> . (c) The estimated dependency graph for <code>ifQuery</code>	110
6.4	(a) The <code>loopQuery(k)</code> program. (b) The abstract transition graph for <code>loopQuery(k)</code> . (c) The estimated dependency graph for <code>loopQuery(k)</code>	111
6.5	(a) A simple loop program with k adaptivity rounds. (b) The estimated dependency graph by AdaptFun	131
6.6	(a) A two-level nested loop example program. (b) The estimated dependency graph generated from AdaptFun	140
6.7	(a) The same running example program <code>twoRounds(k)</code> as Figure 2.4(a). (b) The corresponding semantics-based dependency graph. (c) The estimated dependency graph from AdaptFun	142
7.1	(a) The simplified example program with multiple adaptivity rounds. (b) The estimated dependency graph by AdaptFun	145

7.2	(a) The linear regression algorithm with gradient descent optimization. (b) The estimated dependency graph by AdaptFun	146
7.3	(a) The multiple-path loop program with multiple adaptivity rounds. (b) The corresponding semantics-based dependency graph. (c) The estimated dependency graph from AdaptFun	149
7.4	The labeled program implementing the <i>multiple rounds analyst strategy</i> .156	
9.1	(a) The two paths loop example. (b) The abstract transition graph for <code>twoPathsWhile(n, m)</code> . (c) The simple transition paths. (d) The rewrote program. (e) The refined program.	167
9.2	(a) An example of three nested loops with related iterator variables. (b) The abstract transition graph, the simple transition paths and loop body. (c) The refined program.	172
9.3	(a) The program of the two paths loop with a nested Loop in one path. (b) The abstract transition graph of <code>absG(nestedOdd(n, m))</code> . (c) The refined program.	172
10.1	Evaluation rules of arithmetic expression	183
10.2	Evaluation rules of boolean expression	184
10.3	Evaluation rules of standard expression	184
10.4	Operational semantics rules	185
12.1	(a) The two paths loop example. (b) The abstract transition graph for <code>twoPathsWhile(n, m)</code> . (c) The simple transition paths. (d) The rewrote program. (e) The refined program.	205

12.2	(a) An example of three nested loops with related iterator variables. (b) The abstract transition graph, the simple transition paths and loop body. (c) The refined program.	225
13.1	(a) The example of nested loop with related iterators. (b) The ab- stract transition graph.	230
13.2	(a) The simplified example with nested loop and related iterator. (b) The corresponding abstract transition graph	231
13.3	(a) The simple loop example with two paths iterating alternatively. (b) The corresponding abstract transition graph.	233
13.4	(a) The simplified example of nested loop with related iterator. (b) The corresponding abstract transition graph	235
13.5	(a) The program of the two paths loop with a nested loop in one path. (b) The corresponding abstract transition graph	236
13.6	(a) The multiple-path loop program with a nested loop and inter- fered iterators. (b) The corresponding abstract transition graph	238

CHAPTER 1

Introduction

This chapter contains three parts. Section 1.1 introduces the background and motivation for analyzing the two *quantitative-reachability* properties. Section 1.2 gives an overview of the methodologies used in this thesis. The outline of this thesis is presented in Section 1.4.

1.1 BACKGROUND

Program analysis has a long history of analyzing and controlling a program's execution behavior. A few decades ago, computer programs were only used for research or business purposes and the only expectation of a program was to execute correctly without bugs. The most critical behavior with respect to programs has been the execution correctness behavior. This has led the program analysis community to pay a lot of attention to functional correctness properties. However, due to the popularity of mobile devices and the large-scale data usage in a variety of programs, the expectations on a program have increased. It is no longer enough for a program to only run without bugs, some *quantitative-reachability* properties have become equally important. This thesis studies two *quantitative-reachability* properties, specifically the *adaptivity* of the program implementing certain data analyses, and the *number of times that each program location is visited during the program execution*. These two properties influence the *convenience*, *enjoyment*, *efficiency*, and *privacy* when people are using mobile and computer programs.

Convenience And Enjoyment. People have a variety of experiences when using computer programs and mobile device applications. Convenience and enjoyment are closely related to some data analysis programs running on computer and mo-

bile devices. For example, weather forecasting applications use data analysis programs to predict the weather. Accurate predictions bring people convenience and enjoyment. Other applications, such as video or music players, online games and shopping use data analysis programs to provide users with personalized recommendations. The recommendation accuracy influences one's convenience and enjoyment as well. In both cases, the accuracy of the program outputs influence the user's convenience and enjoyment. This brings my attention to the program properties that can help to improve the accuracy of these data analysis programs. We look at the scenario in which the data analysis is composed of multiple queries interrogating some data. It aims to identify some properties of the population from which the data are drawn and then generalizes beyond the specific data sample. These generalized results are used by some applications to recommend games, music, clothes, etc. to the users beyond the data sample, or by the weather forecasting application to make predictions on the climate in the future. The recommendation and prediction accuracy is usually measured by the generalization error. It measures the difference between the result of a data analysis run on a sample data and the result one would achieve by running over the entire population. High generalization error in the data analysis reduces the accuracy of recommendation and prediction, bringing *in-convenience* and reduces the *enjoyment* when using these programs. When a data analysis algorithm is adaptive where the choice of the next query may rely on the results of previous queries, the error in each query can propagate through the dependency chain and brings high generalization error. In light of existing machine learning studies, some researchers found that determining the appropriate statistical technique based on the length of this dependency chain, namely the *adaptivity* level can help in controlling the gener-

alization error. In this sense, we are interested in studying this intuitive notion – the *adaptivity* level by using the principles of programming languages in **PART i** to help to reduce the generalization error of the data analysis programs. Specifically, we consider that the data analyses are implemented in a *while*-like language and formalize this intuitive notion – the *adaptivity* level as a program *quantitative-reachability* property. This *adaptivity* property captures the generalization error in terms of whether the query asked by a program is dependent on each other (the *reachability* sub-property of *adaptivity*) and how many times they are dependent (the *quantitative* sub-property of *adaptivity*). By analyzing the *adaptivity* and helping the data analyst to reduce the generalization error, we can improve the user’s experience with respect to *convenience* and *enjoyment*.

Efficiency And Privacy. When using mobile applications or computer programs, people do not only care about how enjoyable or convenient the experiences are, but also about performance and privacy. For example, in games such as racing, people care about the performance of the game. In strategy games, people also care about whether their strategies are leaked to the other parties – the privacy of the game. We observe that both performance and privacy are influenced by whether certain pieces of the program code are executed, and how many times they are executed. This brings my attention to two program properties – the reachability of certain lines of code (a *reachability* property) and the times that certain code is executed (a *quantitative* property). These two properties can be captured as two sub-properties by a *quantitative-reachability* hyper-property – *the number of times that each program location is visited during the program execution*. In Gulwani et al. (2009a), an upper bound on this property is referred to as the *reachability-bound*. Specifically, it is a worst-case upper bound on the number of times a given program location inside a

procedure is visited during the program execution. The *reachability-bound* on each program location has broad applications in analyzing other program properties beyond itself. For example, in the resource cost analysis area, it can help to provide an accurate bound on a program’s resources consumption, such as time consumption, memory consumption, network traffic, power consumption, etc. In the data analysis area, when we analyze a program’s *adaptivity* as discussed above, an accurate *reachability-bound* on each program location can improve the precision of the *adaptivity* analysis result. In this sense, analyzing and controlling the *reachability-bound* can help to improve programs reliability, security, privacy, usability, etc., and then bring the *efficiency* and *privacy* to people when they use these programs. Motivated by this and continued from the previous work, we look into this property in **PART ii** and present an algorithm that computes an accurate and sound *reachability-bound* on each program location.

To study the two conceptual *quantitative-reachability* properties from the principles of programming language, we first formalize them through specific programming language models. Then, we develop new program analysis algorithms to best exploit the benefit of these properties. However, to fully utilize the aforementioned benefits of these *quantitative-reachability* properties, appropriate implementations are inevitable. To this end, we also take one step in implementing these analysis algorithms. The evaluation results over different benchmarks show the effectiveness of our work in the real world and its potential applications beyond the program analysis area.

1.2 METHODOLOGY

1.2.1 PART i - Adaptivity Analysis

1.2.1.1 *Language Models*

Lots of languages are designed and used by people to implement data analysis algorithms. However, performing analysis on the programs implemented in existing languages is difficult. Below, I summarize the difficulties and propose the new language model.

Existing Languages Models and Limitations. Many data analysis algorithms are implemented in Python, and these programs usually use some external libraries for different purposes. For example, many programs use the Panda library for database access. Some of them also use the Numpy library for efficient computation and the TensorFlow library for data training. Another language frequently used is C with some external database access libraries as well. These languages and external libraries focus only on the efficiency and scalability of the programs, but not the generalization error of the corresponding data analysis algorithms that these programs implement. It is generally difficult to analyze the *adaptivity* and estimate generalization error of these programs based on existing programming languages.

Proposed Language Model. I focus on data analyses with linear queries that are mostly used as the core of data analysis algorithms. Based on these data analyses, I design a simple but scalable language model based on “standard while language” named `QWhile`, and then formalize the *adaptivity* property of the programs written in `QWhile`.

1.2.1.2 Program Property Formalization

Adaptivity is commonly used as an intuitive concept in the machine learning area. To study it as a program property, I need to have this concept formally defined. There are many ways to formalize a program's conceptual property. However, they all have limitations in capturing the two sub-properties, the *quantitative* and *reachability* properties. I summarize some existing formalization methods with their limitations and then introduce how I formalize the concept of *adaptivity*.

Existing Formalization Methods and Limitations. I studied three main methodologies in formalizing a program property.

Abstract Interpretation-Based. The *abstract interpretation*-based methods were developed in Cousot's seminal papers, Cousot & Halbwachs (1978) and Cousot & Cousot (1977). This category of methods is extensively developed in Miné (2006); Zanioli & Cortesi (2011); Mastroeni & Zanardini (2017); Kincaid et al. (2018); Cousot (2019a); Jana et al. (2020), etc. The majority of these methods focus on formalizing property from the *reachability* perspective, such as the dependency relation defined in Cousot (2019a), program point reachability formalized in Mastroeni & Zanardini (2008), etc. However, they are not able to capture the *quantitative* sub-property of *adaptivity*.

Program Simulation And Execution History-Based. The *execution history and program simulation*-based formalization methods by Israel (1956); Kildall (1973) are also widely used in defining and formalizing the program's properties. However, they have the same limitation as the *abstract interpretation*-based methodology. These methods are unable to capture both the *quantitative* and *reachability* sub-properties of the *adaptivity*. It is not straightforward to apply them directly for

defining the *adaptivity* involving its *quantitative* and *reachability* sub-properties.

Syntactic-Based. There are also some *syntactic*-based formalization methods, which are originated from Weiser (1984) and developed in Denning & Denning (1977). However, these methods are limited to certain languages and are not easy to generalize into other language models.

Proposed Methodology. Because of its nature as a *quantitative-reachability* property, defining *adaptivity* requires us to capture both the *reachability* and the *quantitative* sub-properties. I propose a semantics-based formalization method as follows.

1. For the *reachability* sub-property, I define it as a *data dependency* relation. The new definition captures both the control influence and value influence between data, which are never fully captured in previous dependency relation Cousot (2019a); Zanioli & Cortesi (2011); Cheney et al. (2011). It can be used to reason about the dependency between queries.
2. In terms of the *quantitative* sub-property, I define it as a quantity associated with certain data by looking into the variables that use this data. This quantity is the visiting times of each variable in the program during the execution, namely the *weight* of a variable which is inspired from Gulwani et al. (2009a).
3. Then, I construct a dependency graph by using the *data dependency* relation as the graph edge and the *weight* as the vertex weight. In this way, I encode both sub-properties of the *adaptivity* on this graph. There are also some previous works designing the weighted dependency graph. However, these methods are unable to carry both *reachability* and *quantitative* sub-properties concerning certain data in a program.

4. Based on this graph, I introduce a new concept over this graph, the *finite walk*. Then, the *adaptivity* of a program is defined as the length of the longest *finite walk* on the dependency graph of this program. To this end, I formally capture the intuitive *adaptivity* notation from the machine learning area as the *adaptivity* hyper-property of a program.

As a summarization of the four steps above, the *adaptivity* is defined as the length of the longest *finite walk* over a dependency graph. This graph contains both the *reachability* and the *quantitative* sub-properties. As far as I know, most of the existing methodologies cannot be used directly here, and none of them can capture this hyper-property as precisely as the new methodology. I present this formalization procedure in Chapter 5.

1.2.1.3 Program Analysis for Adaptivity

Then, we need program analysis techniques to estimate the *adaptivity* of a program to help programmers control the generalization error. Below, I summarize existing methodologies in the program analysis areas and their limitations, and then I introduce the design of the adaptivity analysis algorithm.

Existing Program Analysis and Limitations. To analyze the *adaptivity* of a program, I look into the following three methods.

Data Dependency / Control Flow Analysis. Some data dependency analysis techniques and control flow techniques can help to estimate the *reachability* property of a program. However, most of the existing dependency analysis considers only the value influences and the control flow analysis techniques consider only the control influences. In our setting, the *reachability* property is a sub-property of the *adaptivity*.

tivity property. We define it as the dependency relation involving both value and control influences. To analyze this sub-property, I need to consider both kinds of influences. As a result, it is difficult to apply them directly to analyze this *reachability* sub-property.

Resource Cost / Complexity Analysis. Some techniques in the complexity and resource cost analysis areas can be used to analyze the *quantitative* sub-property. However, they are focusing on estimating the overall resource costs or program execution complexity. Most of them cannot give estimations for specific data or program locations. Since I need to estimate the quantity associated with a certain program variable, reusing them in analyzing the *quantitative* sub-property of *adaptivity* is not straightforward.

Longest Path Search. Given the similarity between the longest path and the longest *finite walk*, some longest-weighted path search algorithms can help to compute the longest *finite walk*. However, since computing the longest weighted path is NP-hard and the weight on the graph is symbolic, most existing algorithms are non-terminating when searching for the longest *finite walk*. In this sense, I need to design a new algorithm to compute the longest *finite walk* and estimate the *adaptivity*.

Proposed Methodology. The new adaptivity analysis algorithm follows a similar structure as in Section 1.2.1.2. It is summarized as follows.

1. First, I need to estimate the *reachability* sub-property. Specifically, I need to look into the *data dependency* relation by considering both the value and control influences between data. The new algorithm performs both the data flow analysis and the control flow analysis, which estimates the *data dependency* re-

lation more accurately than using either single one of them.

2. Then, I propose an algorithm to analyze the *quantitative* sub-property, specifically the visiting times of each variable during the program execution. This algorithm reuses some worst-case program complexity analysis techniques and adopts them for the visiting times of every program variable. In this sense, it can be used to analyze the *adaptivity* of a program more accurate than trivially computing the worst-case program execution complexity.
3. Then, I estimate the *dependency graph* by encoding both the estimated *reachability* and *quantitative* sub-properties. This encoding complements the weakness of both lines of work in comparison to purely using single either *reachability* or *quantitative* analysis,
4. At the end, to accurately estimate the adaptivity, I compute the longest *finite walk* over the estimated graph. To estimate this longest *finite walk*, I design a new algorithm based on some longest path algorithms in the graph and complexity theory areas. This new algorithm approximates this longest *finite walk* and then computes the *adaptivity* soundly and efficiently.

In short, it is designed based on some techniques in the data flow, control flow analysis, reachability-bound analysis, and graph theory areas. It gives a sound upper bound on the *adaptivity* of a program while none of the existing techniques can compute. I present this algorithm in Chapter 6 and name it as **AdaptFun**.

1.2.2 PART ii - The Path-sensitive Reachability-bound Analysis

In **PART ii**, the property I look into is *the number of times a given location inside a procedure is visited during the program execution*. Specifically, I want to estimate an

upper bound on this property for each program location, namely the *reachability-bound*. Existing methods from different areas have different limitations in computing precise *reachability-bounds* for different program locations. I mainly studied four different methodologies. Below, I summarize the limitations of each of them and then introduce how I leverage these limitations in the proposed approach.

1.2.2.1 Program Abstraction

Existing Techniques and Limitations. The program abstraction techniques are widely used in the program analysis area to precisely summarize the key property of the program. They are usually based on some specific abstract interpretation models as introduced in Section 1.2.1. In analyzing the loop bounds, the unified lattice model in Cousot & Halbwachs (1978), polyhedra in Cousot & Cousot (1977), and octagons in Miné (2006) are used in Gulwani & Zuleger (2010) to summarize the loop transition and generate the transition system. But none of the three abstraction models can But these models cannot represent the boolean expressions in the if statement. Moreover, solving the transition system using these abstraction models is inefficient. The wedge abstract developed in Kincaid et al. (2018) for generating the non-linear loop invariant computing the *reachability-bound* can also be used. However, it only works well for the specific targeting problem and is inaccurate in summarizing the multipath loops and solving the path-sensitive *reachability-bound* problem.

Proposed Method. In the new method, I extend the difference constraint-based program abstraction algorithm in Sinn et al. (2017) and Sinn et al. (2014) with boolean expression abstraction to generate the abstract transition graph of a program. Every command in the program is abstracted into an abstract transition and

represented on this graph. By this extension, I capture the loops in a program path-sensitively. In this sense, it is more accurate than the state-of-art program abstraction models.

1.2.2.2 *Loop Summarization and Path Refinement*

Existing Techniques and Limitations. One kind of loop bound analysis through loop summarization and path refinement seeks precise loop path representation such as Manolios & Vroon (2006); Balakrishnan et al. (2009); Sharma et al. (2011); Flores-Montoya & Hähnle (2014); Humenberger et al. (2018); Cyphert et al. (2019), and compute loop bound over accurate loop summarization such as Gulwani et al. (2009a); Zuleger et al. (2011). They mainly focus on summarizing the loop path and computing the interleaving between paths, as well as computing the precise bound considering the interleaving. However, composing the bound between nested loops need to recursively unroll the nested loops and compute the interleaving between unrolled paths. These operations are usually inefficient and non-terminating.

Proposed Method. To leverage the efficiency problem, I use a lightweight path refinement algorithm adapted from Gulwani et al. (2009a). It computes the refined program over the abstract transition graph generated above.

1.2.2.3 *Amortized Complexity Analysis through Ranking Function*

Existing Techniques and Limitations. This kind of method is originated from Tarjan’s influential paper Potechin (2017). They are usually combined with *ranking function* estimation method in Bradley et al. (2005) such as the works in Zuleger

et al. (2011); Sinn et al. (2014, 2017); Lu et al. (2021); Alias et al. (2010). Indeed, using techniques like ranking functions, counter increments, difference constraints, etc. one can estimate precise bounds on loop iterations and execution time, and in turn, obtain reachability bounds for program locations. However, these techniques are designed to capture the worst-case or overall complexity, and again they may over-approximate the reachability-bounds of some locations. When there are multiple paths and the path interleaving affects the loop execution, the over-approximation is more evident.

Proposed Method. In the new approach, I use the ranking function estimation-based *amortized complexity analysis* techniques by alternatively computing the bound on each ranking function and transition edge. To overcome the over-approximation limitations in multi-paths loops, I deploy the loop path refinement technique and perform the *amortized complexity analysis* over the refined programs. In this sense, I estimate the ranking function in a path-sensitive manner. Moreover, this deployment does not reduce efficiency, since the alternation from the bound estimation into the ranking function estimation eliminating the unrolling of the nested loop recursively.

1.2.2.4 Reachability-bound Computation

Existing Techniques and Limitations. Existing static analysis techniques cannot compute the *reachability-bound* on a given program control location directly or path-sensitively. Most of them focus only on the *quantitative* sub-property, specifically the overall program execution complexity. For example, Gulwani & Zuleger (2010) give a two-step solution by combining program abstract and proof-rules-based bound computation. But it does not compute the *reachability-bound* precisely

for every control location. Instead, they only compute the complexity bounds for the overall loop and over-approximate for different locations using this bound, which is loose especially when there are multipath loops. In this sense, they only estimate the *quantitative* sub-property and lose the *reachability* perspective of the *reachability-bound*. Techniques in program complexity analysis in the work Gustafsson et al. (2005); Humenberger et al. (2018) or worst-case resource cost analysis in Brockschmidt et al. (2016); Albert et al. (2008); Alias et al. (2010); Flores-Montoya & Hähnle (2014) can be repurposed to estimate similar quantities such as the bounds on loop iterations, execution time, etc. While these quantities focus only on the overall complexity, none of them compute the *reachability-bound* on a given program control location directly or path-sensitively. For the same reason, these methods also over-approximate the *reachability-bound* by using the overall quantity.

Proposed Method. To overcome these limitations, I design our algorithm over the abstract transition graph and effectively use the *amortized complexity analysis* and *loop summarization-based multipath refinement* techniques in the first step. In this way, the new algorithm mitigates the limitations faced by each technique individually. Then, I introduce two novel quantities, the *path reachability-bound* and *loop reachability-bound*. The first one – *path reachability-bound* – bounds the evaluation times of each loop-free and interleave-free path in a refined program, and the second one – *loop reachability-bound* – bounds the number of iterations of an outer loop w.r.t. an inner loop, counting only the iterations when the inner loop is “entered”. I present algorithms for estimating these two quantities. By using them, I can then compute the *reachability-bound* for each program point in a path-sensitive way. To this end, the new methodology studies both the *reachability* and the *quantitative* perspectives and analyzes the *reachability-bound* in a higher granularity.

1.3 CONTRIBUTIONS

In comparison to those related works, this dissertation has the following contributions.

- In **PART i**, I present a programming language framework for adaptive data analyses where programs represent analysts that can query generalization-preserving mechanisms mediating the access to some data.

Based on this language framework, I give a formal definition of the intuitive notation – adaptivity – under the analyst-mechanism model. The new formalization methodology considers both the *reachability* and the *quantitative* sub-properties of this *adaptivity* program property.

Then we design a static program analysis algorithm **AdaptFun** to estimate the *adaptivity* of a program. This new analysis method estimates both the *reachability* and the *quantitative* sub-properties and computes the *adaptivity* of a program accurately. We also provide a soundness proof of the program analysis showing that the adaptivity estimated by **AdaptFun** bounds the *adaptivity* of the program given by the formal definition.

We also implement this algorithm and report the empirical evaluation results on several examples.

Designing of this framework is jointly done with Marco Gaboardi, Deepak Garg, Jonathan Ullman and Weihao Qu, and the empirical evaluation is jointly finished with Weihao Qu.

- In **PART ii**, I propose a path-sensitive reachability-bound algorithm, which computes sound bound on the evaluation times of each program point path-sensitively. This algorithm integrates the path refinement approach into the

amortized bound analysis through ranking function estimation. We introduce two novel quantities in this algorithm, the *path reachability-bound* and *loop reachability-bound*, and the corresponding estimating algorithms.

Then I implement this algorithm as a prototype and present the evaluation results over four different benchmarks.

Results of this part is done by myself with the help of writing from Marco Gaboardi.

1.4 DISSERTATION OUTLINE

There are two main parts of the work in this thesis. Each of them focus on one specific *quantitative-reachability* property, namely the *adaptivity* of an adaptive data analysis program in **PART i** and *the number of times that each program location is visited during the program execution* in **PART ii**.

1. **PART i PROGRAM ANALYSIS FRAMEWORK FOR ADAPTIVE DATA ANALYSIS** shows the work on studying and formalizing the *adaptivity* of the adaptive data analysis program.

Chapter 2 has four sections.

In Section 2.1, we first introduce the background of the adaptive data analysis in theory and algorithm research areas. Then in Section 2.2, we discuss in detail some results about *adaptivity* of the adaptive data analysis from the theory areas and the challenges and motivations in analyzing this concept in the programming language area. In Section 2.3, we show a running example and give the technical overview of our approach. Then the last part in

Section 2.4 shows the outline of **PART i**.

Chapter 3 summarizes the previous works on analyzing the *adaptivity*. It contains a short summary of the language designs in Section 3.1; the adaptivity formalization through program analysis over the trace-based operational semantics in Section 3.2; and the program analysis algorithm that is used to estimate the adaptivity of the data analysis programs in Section 3.3.

Chapter 4 gives our `Qwhile` language, which is extended from the standard while language. This chapter includes the syntax in Section 4.1 and the trace-based operational semantics in Section 4.2.

Chapter 5 shows the semantics-based definition for the *adaptivity* of a data analysis program. This formal definition is presented through the following sections.

In Section 5.1, we define a semantics-based *variable may-dependency* relation to the reason about the dependency relation between every query. In Section 5.2, we first define the quantitative sub-property as the weight for each variable based on the *variable may-dependency* relation. Then we construct a weighted dependency graph namely *semantics-based dependency graph* and encode this quantitative sub-property and with *variable may-dependency* relations on this graph.

Then in Section 5.3, we define the *adaptivity* based on the *semantics-based dependency graph* by considering both the *quantitative* and *reachability* sub-properties encoded on it.

Chapter 6 introduces a static program adaptivity analysis framework, named AdaptFun through the following sections.

In Section 6.2, 6.3 and 6.4, we construct an *estimated dependency graph* for approximating the *semantics-based dependency graph*. Specifically, in Section 6.3.2, we first use an integrated data-control flow analysis algorithm to soundly estimate the *variable may-dependency* relation and build the edges of the estimated graph. Then in Section 6.3.3, we present a reachability-bound analysis algorithm which can soundly estimate the quantitative sub-property of the *adaptivity*. Then we use it to construct the weight of the estimated graph. At the end, in Section 6.4, we build the *estimate dependency graph* using all these results estimated about. In Section 6.5, we present an efficient and accurate path search algorithm, which computes the *adaptive* from the estimated graph.

Chapter 7 has two sections. In Section 7.1, we give five examples to manually demonstrate our framework. Then we show the evaluation results over a benchmark composed of six real-world data analysis algorithms and eleven handcrafted programs based on the code pieces extracted from the C library and six synthesized programs in Section 7.2.

Chapter 8 discusses the related works we have looked in three different areas, including the works in semantics-based dependency analysis and formalization area in Section 8.1; the existing static program analysis techniques in Section 8.2; and the works about generalization error in adaptive data analysis area in Section 8.3.

2. PART ii REACHABILITY BOUND ANALYSIS

Chapter 9 has four parts.

In Section 9.1, we introduce the background of the *reachability-bound* and developments in analyzing it. Then in Section 9.2, we discuss the technical limitations and motivations in analyzing the *reachability-bound*. Section 9.3 gives the technical overview of our new approach through three running examples. The structure of this chapter is given in Section 9.4.

Chapter 10 contains the program model including the language syntax in Section 10.1 and the trace-based operational semantics in Section 10.2.

Chapter 11 gives the *reachability-bound* definition over our program model.

Chapter 12 presents the new *path-sensitive reachability-bound* analysis algorithm, named psRB through the following subsections.

Section 12.1 first shows the main steps of this algorithm as a structural section. There are five steps in this algorithm.

In Section 12.2, we give the computation steps for computing the *abstract transition graph* for the program.

Based on this graph, in Section 12.3, we refine the multi-paths loop in a program and generates a refined program.

According to the graph and the refined program, we present the computation steps for computing the ranking function for each edge on the graph and estimates the value of each ranking function in Section 12.4.

In Section 12.5 we introduce our *path-sensitive reachability-bound* algorithm, and we give the detailed computations of the *path reachability-bound* in Section 12.6 and Section 12.8, and the computation detail of the *loop reachability-bound* in Section 12.7.

Chapter 13 has two sections, including six examples to illustrate the algorithm step-by-step in Section 13.1 firstly, and then implementation and evaluation results over 270 programs from four benchmarks sets in Section 13.

Chapter 14 is the related works we looked into in program analysis area.

3. PART iii Epilogue

Chapter 15 is the conclusion of the works in **PART i** and **PART ii**, and the future plans based on the limitations through the following sections.

In Section 15.1.1, we conclude the contribution and weakness of our program analysis framework for adaptive data analysis. Then in Section 15.1.2, we propose future work on extending and improving our existing work.

Section 15.2.1 summarizes the contributions and limitations of our path-sensitive reachability-bound computation algorithm presented in **PART ii**. Then Section 15.2.2 proposes the future works, specifically the improvement of existing works and the possible applications on other fields.

4. PART iv APPENDIX This part contains the example programs, the complete evaluation table and proofs of the lemmas and theorems in **PART i** and **PART ii**.

PART I PROGRAM ANALYSIS FRAMEWORK FOR ADAPTIVE DATA ANALYSIS

Data analyses are usually designed to identify some property of the population from which the data are drawn, generalizing beyond the specific data sample. For this reason, data analyses are often designed in a way that guarantees that they produce a low generalization error. That is, they are designed so that the result of a data analysis run on sample data does not differ too much from the result one would achieve by running the analysis over the entire population.

An adaptive data analysis can be seen as a process composed by multiple queries interrogating some data, where the choice of which query to run next may rely on the results of previous queries. The generalization error of each individual query/analysis can be controlled by using an array of well-established statistical techniques. However, when queries are arbitrarily composed, the different errors can propagate through the chain of different queries and bring to high generalization error. To address this issue, data analysts are designing several techniques that not only guarantee bounds on the generalization errors of single queries, but that also guarantee bounds on the generalization error of the composed analyses. The choice of which of these techniques to use, often depends on the chain of queries that an adaptive data analysis can generate. To this end, the total number of queries and the depth of the chain of queries is of great significance to guarantee the generalization error.

In this part, we consider adaptive data analyses implemented as while-like programs. It formalizes the intuitive notion of *adaptivity* as a *quantitative-reachability*

property of programs by considering its two sub-properties, which is more precise than previous works based on the program's semantics. We do this because the adaptivity level of a data analysis is a key measure to choose the right technique. Based on this definition, I design a new program analysis framework for soundly approximating this quantity automatically. This analysis framework generates a weighted dependency graph as the representation of the data analysis program, where the weight is an upper bound on the number of times each variable can be reached during the program execution. Then it uses a path search strategy to guarantee an upper bound on the adaptivity. We implement our program analysis and show that it can help to analyze the adaptivity of several concrete data analyses with different adaptivity structures, automatically and accurately.

CHAPTER 2

Introduction

This chapter includes three sections. The background of adaptive data analysis and the significance for reasoning about this intuitive *adaptivity* notion are presented in Section 2.1. Then we discuss the motivations for analyzing this *adaptivity* in the data analysis area and the program analysis area in Section 2.2. Then in Section 2.3, we give a technical overview of this our adaptivity analysis framework. The last part in Section 2.4 is an outline of this chapter with the contributions of this work.

2.1 BACKGROUND

The adaptive data analysis designed for identifying properties for unknown populations / distributions through data samples is widely used in research and industrial areas. When generalizing the analysis result from data samples to the unknown populations, the generalization error is the key issue on which researchers are focusing to reduce. Specifically in the adaptive data analysis, which is a process composed of multiple queries interrogating some data the choice of which query to run next may rely on the results of previous queries, the generalization error is propagated through the reliance between the choices of queries. In this sense, the *adaptivity* rounds (i.e., how many queries are relied on others) in the analysis plays a key role in reducing the generalization error. Below I introduce in detail the adaptive data analysis procedure, and the significance of this *adaptivity* quantity in effecting the generalization error.

Consider a dataset X consisting of n independent samples from some unknown population P . How can we ensure that the conclusions drawn from X *generalize* to the population P ? Despite decades of research in statistics and machine learn-

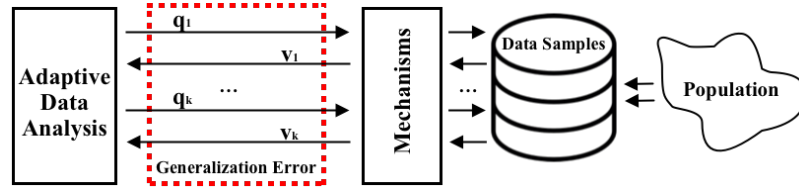


Figure 2.1: Overview of our Adaptive Data Analysis model. We have a population that we are interested in studying, and a dataset containing individual samples from this population. The adaptive data analysis we are interested in running has access to the dataset through queries of some pre-determined family (e.g., statistical or linear queries) mediated by a mechanism. This mechanism uses randomization to reduce the generalization error of the queries issued to the data.

ing on methods for ensuring generalization, there is an increased recognition that many scientific findings generalize poorly (e.g. Ioannidis (2005); Gelman & Loken (2014)). While there are many reasons a conclusion might fail to generalize, one that is receiving increasing attention is *adaptivity*, which occurs when the choice of method for analyzing the dataset depends on previous interactions with the same dataset Gelman & Loken (2014). Adaptivity can arise from many common practices, such as exploratory data analysis, using the same data set for feature selection and regression, and the re-use of datasets across research projects. Unfortunately, adaptivity invalidates traditional methods for ensuring generalization and statistical validity, which assume that the method is selected independently of the data. The misinterpretation of adaptively selected results has even been blamed for a “statistical crisis” in empirical science Gelman & Loken (2014).

A line of work initiated by Dwork et al. (2015a), Hardt & Ullman (2014) posed the question: Can we design *general-purpose* methods that ensure generalization in the presence of adaptivity, together with guarantees on their accuracy? The idea that has emerged in these works is to use randomization to help ensure generalization. Specifically, these works have proposed to mediate the access of an adaptive

data analysis to the data by means of queries from some pre-determined family (we will consider here a specific family of queries often called "statistical" or "linear" queries) that are sent to a *mechanism* which uses some randomized process to guarantee that the result of the query does not depend too much on the specific sampled dataset. This guarantees that the result of the queries generalizes well. This approach is described in Figure 2.1. This line of work has identified many new algorithmic techniques for ensuring generalization in adaptive data analysis, leading to algorithms with greater statistical power than all previous approaches. Common methods proposed by these works include, the addition of noise to the result of a query, data splitting, etc. Moreover, these works have also identified problematic strategies for adaptive analysis, showing limitations on the statistical power one can hope to achieve. Subsequent works have then further extended the methods and techniques in this approach and further extended the theoretical underpinning of this approach, e.g. Dwork et al. (2015b,a); Bassily et al. (2016); Ullman et al. (2018); Feldman & Steinke (2017); Jung et al. (2021); Steinke & Zakynthinou (2020); Rogers et al. (2020).

A key development in this line of work is that the best method for ensuring generalization in an adaptive data analysis depends to a large extent on the number of *rounds of adaptivity*, the depth of the chain of queries. As an informal example, the program $x \leftarrow q_1(D); y \leftarrow q_2(D, x); z \leftarrow q_3(D, y)$ has three rounds of adaptivity, since q_2 depends on D not only directly because it is one of its input but also via the result of q_1 , which is also run on D , and similarly, q_3 depends on D directly but also via the result of q_2 , which in turn depends on the result of q_1 . The works we discussed above showed that, not only does the analysis of the generalization error depend on the number of rounds, but knowing the number of rounds actu-

ally allows one to choose methods that lead to the smallest possible generalization error - we will discuss this further in Section 2.3.

For example, these works showed that when an adaptive data analysis uses a large number of rounds of adaptivity then a low generalization error can be achieved by a mechanism adding to the result of each query Gaussian noise scaled to the number of rounds. When instead an adaptive data analysis uses a small number of rounds of adaptivity then a low generalization error can be achieved by using more specialized methods, such as data splitting mechanism or the reusable holdout technique from Dwork et al. (2015a). To better understand this idea, we show in Figure 2.2 three experiments showcasing these situations. More precisely, in Figure 2.2(a) we show the results of a specific analysis¹ with two rounds of adaptivity. This analysis can be seen as a classifier which first runs 400 non-adaptive queries on the first 400 attributes of the data, looking for correlations between the attributes and a label, and then runs one last query which depends on all these correlations. Without any mechanism the generalization error of the last query is pretty large, and the lower generalization error is achieved when the data-splitting method is used. Figure 2.2(c) shows how this situation also change with the number of queries. Specifically, it shows the root mean square error of the last *adaptive* query when the numbers queries varies. This also highlight the fact that different mechanisms, for the same analysis, produce results with very different generalization error. In Figure 2.2(b), we show the results of a specific analysis² with four hundreds rounds of adaptivity. At each step, this analysis runs an adaptive query based on the results of the previous ones. Without any mechanism, the generaliza-

¹We will use formally a program implementing this analysis (Figure 2.4) as a running example in the rest of the paper.

²We will present this analysis formally in Section 7.1.

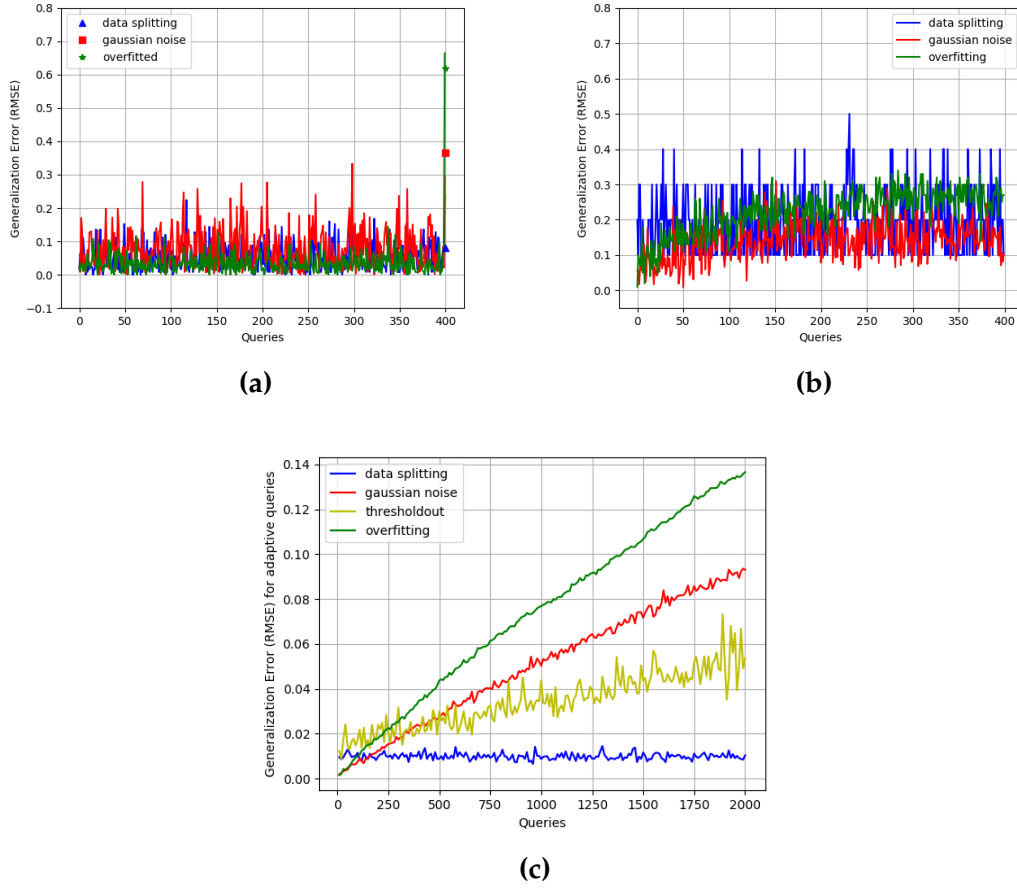


Figure 2.2: The generalization errors of two adaptive data analysis examples, under different choices of mechanisms. (a) Data analysis with 2 rounds adaptivity. (b) Data analysis with 400 rounds adaptivity. (c) Same data analysis as (a) with different query numbers.

tion error of most of the queries is pretty large, and this error can be lowered by using Gaussian noise.

This scenario motivates us to explore the design of program analysis techniques that can be used to estimate the number of *rounds of adaptivity* that a program implementing a data analysis. These techniques could be used to help a data analyst in the choice of the mechanism to use, and they could ultimately be integrated into a tool for adaptive data analysis such as the *Guess and Check* framework by Rogers

et al. (2020).

2.2 MOTIVATION

2.2.1 Some Results in Adaptive Data Analysis

In Adaptive Data Analysis an *analyst* is interested in studying some distribution P over some domain $\mathcal{X}$. Following previous works Dwork et al. (2015a); Hardt & Ullman (2014); Bassily et al. (2016), we focus on the setting where the analyst is interested in answers to *statistical queries* (also known as *linear queries*) over the distribution. A statistical query is usually defined by some function $\text{query} : \mathcal{X} \rightarrow [-1, 1]$ (often other codomains such as $[0, 1]$ or $[-R, +R]$, for some R , are considered). The analyst wants to learn the *population mean*, which (abusing notation) is defined as $\text{query}(P) = \mathbb{E}_{X \sim P}[\text{query}(X)]$. We assume that the distribution P can only be accessed via a set of *samples* $X_1, \dots, X_n$ drawn independently and identically distributed (i.i.d.) from P . These samples are held by a mechanism $M(X_1, \dots, X_n)$ who receives the query query and computes an answer $a \approx \text{query}(P)$. The naïve way to approximate the population mean is to use the *empirical mean*, which (abusing notation) is defined as $\text{query}(X_1, \dots, X_n) = \frac{1}{n} \sum_{i=1}^n \text{query}(X_i)$. However, the mechanism M can adopt some methods for improving the generalization error $|a - \text{query}(P)|$.

In this work we consider analysts that ask a sequence of k queries $\text{query}_1, \dots, \text{query}_k$. If the queries are all chosen in advance, independently of the answers of each other, then we say they are *non-adaptive*. If the choice of each query query_j depend on the prefix $\text{query}_1, a_1, \dots, \text{query}_{j-1}, a_{j-1}$ then they are *fully adaptive*. An important intermediate notion is *r-round adaptive*, where the sequence can be partitioned into r batches of non-adaptive queries. Note that non-adaptive queries are 1-round and

fully adaptive queries are k -round adaptive.

We now review what is known about the problem of answering r -round adaptive queries.

Theorem 2.2.1 (Bassily et al. (2016)). *1. For any distribution P , and any k non-adaptive statistical queries, $\max_{j=1,\dots,k} |a_j - \text{query}_j(P)| = O\left(\sqrt{\frac{\log k}{n}}\right)$.*

2. For any distribution P , and any k r -round adaptive statistical queries, with $r \geq 2$, the empirical mean (rounded to an appropriate number of bits of precision)³ satisfies:

$$\max_{j=1,\dots,k} |a_j - \text{query}_j(P)| = O\left(\sqrt{\frac{k}{n}}\right)$$

In fact, these bounds are tight (up to constant factors) which means that even allowing one extra round of adaptivity leads to an exponential increase in the generalization error, from $\log k$ to k .

Dwork et al. (2015a) and Bassily et al. (2016) showed that by using carefully calibrated Gaussian noise in order to limit the dependency of a single query on the specific data instance, one can actually achieve much stronger generalization error as a function of the number of queries, specifically.

Theorem 2.2.2 (Dwork et al. (2015a); Bassily et al. (2016)). *For any distribution P , any k , any $r \geq 2$ and any r -round adaptive statistical queries, if we answer queries with carefully calibrated Gaussian noise we have:*

$$\max_{j=1,\dots,k} |a_j - \text{query}_j(P)| = O\left(\frac{\sqrt[4]{k}}{\sqrt{n}}\right)$$

More interestingly, Dwork et al. (2015a) also gave a refined bounds that can be achieved with different mechanisms depending on the number of rounds of adaptivity.

³With infinite precision even two queries may give unbounded error, when the first query's result encodes the whole data.

Theorem 2.2.3 (Dwork et al. (2015a)). *For any r and k , there exists a mechanism such that for any distribution P , and any $r \geq 2$ any r -round adaptive statistical queries, it satisfies*

$$\max_{j=1,\dots,k} |a_j - \text{query}_j(P)| = O\left(\frac{r\sqrt{\log k}}{\sqrt{n}}\right)$$

Notice that Theorem 2.2.3 has different quantification in that the optimal choice of mechanism depends on the number of queries and number of rounds of adaptivity. This suggests that if one knows a good *a priori upper bound on the number of rounds of adaptivity*, one can choose the appropriate mechanism and get a much better guarantee in terms of generalization error. As an example, as we can see in Figure 2.2, if we know that an algorithm is two rounds adaptive, we can choose data splitting as the mechanism, while if we know that an algorithm has many rounds of adaptivity we can choose Gaussian noise. It is worth to stress that by knowing the number of rounds of adaptivity one can also compute a concrete upper bound on the generalization error of a data analysis. This information allows one to have a quantitative, *a priori*, estimation of the effectiveness of a data analysis. This motivates us to design a static program analysis aimed at giving good *a priori* upper bounds on the number of rounds of adaptivity of a program.

2.2.2 Technique Challenges

In order to analyze the *adaptivity* property of a program, there are mainly three challenges. We discuss them in detail as below.

2.2.2.1 Language Model for Adaptive Data Analysis.

The first challenge is *how to define formally* a language model for adaptive data analysis which is general enough to support the methods discussed above and

would permit to formulate the notion of adaptivity these methods use. We take the approach of designing a simple programming language for submitting queries to some *mechanism* giving access to the data, such as the mechanism of adding Gaussian noise, the mechanism that randomly selects a subset of the data, and the mechanism that uses the reusable holdout technique, etc. These mechanisms intermediate between the program and data, and answer these queries. In this approach, a program models an *analyst* asking a sequence of queries to the mechanism. The mechanism runs the queries on the data applying one of the methods discussed above and returns the result to the program. The program can then use this result to decide which query to run next. Overall, I am interested in controlling the generalization of the query results returned by the mechanism, by means of the adaptivity.

However, as discussed in Section 1.2.1.1, the existing language mainly focuses on efficiency and scalability of the programs. They are not able to model the query and mechanisms in the way that allow us to analyze and control the *adaptivity* of the corresponding algorithms that these programs implement. This makes the analysis of the *adaptivity* as a program property difficult. This motivates us to design a simple and scalable language that can be used to implement these data analysis algorithms and analyze the *adaptivity*. Previous works from the thesis by Qu (2022) develops a simple imperative language model. However, this language model is not expressive enough to support most general data analyses. I also discuss in detail this language design and limitations in Section 3.1.

Motivated by this, we present a new while-like language named `QWhile` language with extensions on query requests in Section 4.

2.2.2.2 Adaptivity Property Formalization

The second challenge is *how to define the adaptivity of a given program*. Intuitively, a query Q may depend on another query P , if there are different values that P can return, then the execution of Q is affected. For example, as shown in Dwork et al. (2015a), and as in our example in Figure 2.2(a), one can design a machine learning algorithm for constructing a classifier that first computes each feature’s correlations with the label via a sequence of queries, and then constructs the classifier based on the correlation values. If one feature’s correlation changes, the classifier depending on features is also affected. This notion of dependency builds on the execution trace as a *causal history*. In particular, I am interested in the history or provenance of a query up until this is executed, simultaneously in tracking whether the result of the query may further cause some other query. This is because I’m focusing on the generalization error which could be propagated by queries. In this sense, we need to consider both the dependency relation between queries, namely the *reachability* sub-property, and the depth of this dependency relation, namely the *quantitative* sub-property. This is the reason we claim that the *adaptivity* of a program is a specific *quantitative-reachability* property.

Existing formalization methodologies cannot capture the two sub-properties of the *adaptivity*, as in Section 1.2.1.2. Techniques in previous works by Qu (2022) on formalizing this quantity are limited in expressiveness and accuracy significantly. The *adaptivity* quantities in many general data analysis programs are unable to be defined in the previous *adaptivity* definition. For the program which is able to be expressed, they cannot give the most precise adaptivity definitions matching the intuitive *adaptivity*. These previous works and limitations on formalizing the adaptivity are introduced in more detail in Section 3.2.

In this sense, we formalize this intuitive *adaptivity* notion as a *quantitative-reachability* program property in Section 5. We use a trace semantics recording the execution history of programs on some given input and we create a dependency graph, where the dependency between different variables (queries are also assigned to variables) is explicit and track which variable is associated with a query request. We then enrich this graph with weights describing the number of times each variable is evaluated in a program evaluation starting with an initial state. The adaptivity is then defined as the length of the walk visiting most query-related variables on this graph.⁴ In other words, we define adaptivity as a *quantitative form of program dependency*.

2.2.2.3 Adaptivity Estimation

The third challenge is *how to estimate the adaptivity of a given program*. The adaptive data analysis model and the adaptivity definition suggest a program analysis based on some form of dependency analysis for *adaptivity* estimation. This analysis needs to take into consideration the following, 1) generally, a query Q is not a monolithic block, but rather it may depend on other parts of the program through the use of variables and values. Hence, the analysis needs to consider some form of data flow analysis. 2) the decision on whether to run a query or not may depend on some other value. Hence, the analysis needs to consider some form of control flow analysis. 3) I am not only interested in whether there is a dependency or not, but in the length of the chain of dependencies. Hence, the analysis needs to consider some quantitative information about the program dependencies.

Most existing techniques fail in analyzing the *adaptivity* in the sense of consid-

⁴Formally, graphs will be well-defined only for terminating programs, this will guarantee that the longest walk is finite

ering its two sub-properties. The previous works from the thesis by Qu (2022) develop a program analysis algorithm for estimating the adaptivity. However, this algorithm is low efficiency and in-precise. It is not automatic either. This analysis algorithm and limitations are introduced with more details in Section 3.2.

To address these challenges and technical limitations to estimate a sound upper bound on the adaptivity of a program, we develop a static program analysis algorithm in Section 6, named **AdaptFun**. It performs both data flow and control flow analysis, and the reachability bound analysis. Then it computes the *adaptivity* through a longest walk estimation algorithm. In this way, **AdaptFun** gives tighter bounds on the *adaptivity* of a program than the ones one would achieve by only using any analysis techniques alone. We evaluate **AdaptFun** on a number of examples showing that it is able to efficiently estimate precise upper bounds on the adaptivity of different programs. All the proofs and extended definitions are in the Appendix.

2.3 TECHNIQUE OVERVIEW

Our new adaptivity analysis framework for adaptive data analysis has three major components as shown in Figure 2.3. The first component on the top of Figure 2.3 is the `QWhile` language designed for formalizing the adaptive data analysis. Building on this, we formally define the intuitive *adaptivity* as in the second component at the middle of Figure 2.3, the “Adaptivity Formalization”, and then design a static program analysis algorithm for approximating the upper bound of the adaptivity as shown in the block at the bottom of Figure 2.3, the “Adaptivity Analysis”.

We illustrate the key technical components of our framework through a simple

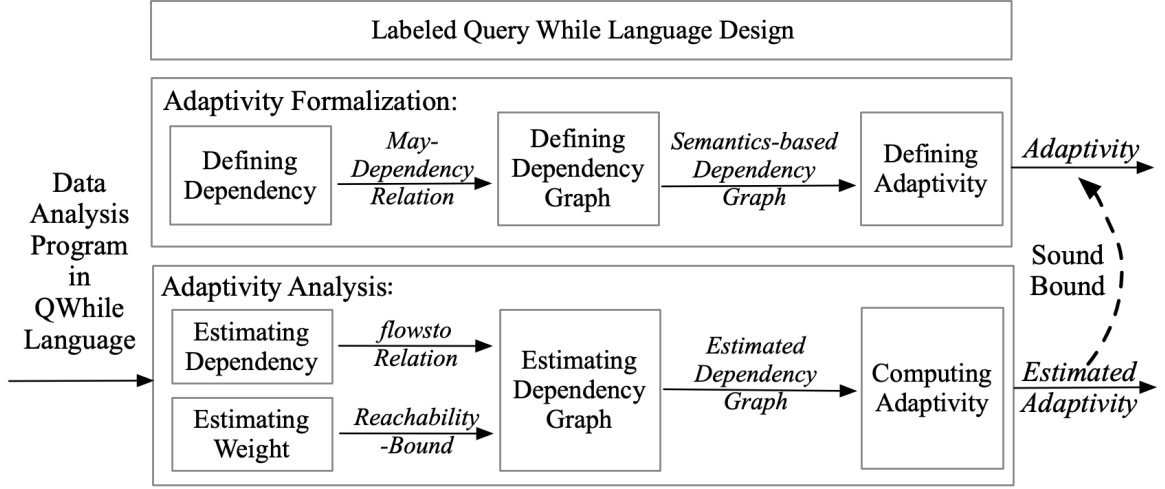
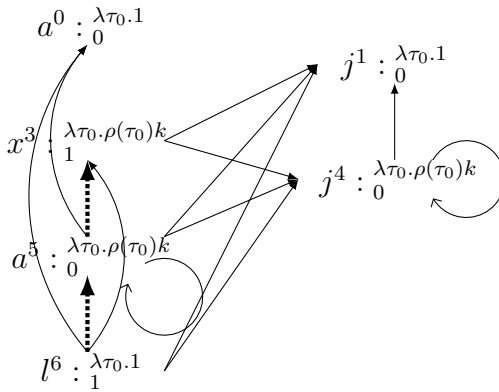


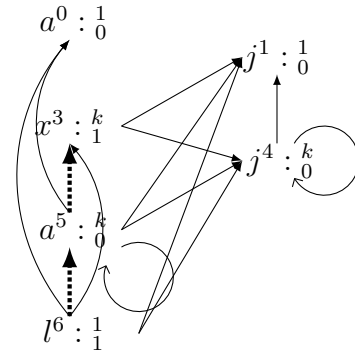
Figure 2.3: Architecture of the adaptivity analysis framework

$\text{twoRounds}(k) \triangleq$
 $[a \leftarrow 0]^0; [j \leftarrow k]^1;$
 $\text{while } [j > 0]^2 \text{ do } ([x \leftarrow \text{query}(\chi[j] \cdot \chi[k])]^3; [j \leftarrow j - 1]^4; [a \leftarrow x + a]^5);$
 $[l \leftarrow \text{query}(\chi[k] * a)]^6$

(a)



(b)



(c)

Figure 2.4: (a) The program $\text{twoRounds}(k)$, an example with two rounds of adaptivity. (b) The corresponding semantics-based dependency graph. (c) The estimated dependency graph from AdaptFun.

adaptive data analysis with two rounds of adaptivity. In this analysis, an analyst asks $k + 1$ queries to a mechanism in two phases. In the first phase, the analyst asks k queries and stores the answers that are provided by the mechanism. In the second phase, the analyst constructs a new query based on the results of the previous k queries and sends this query to the mechanism. The mechanism is abstract here and our goal is to use static analysis to provide an upper bound on adaptivity to help choose the mechanism. This data analysis assumes that the data domain $\mathcal{X}$ contains at least k numeric attributes (every query in the first phase focuses on one), which we index just by natural numbers. The implementation of this data analysis in the language of AdaptFun is presented in Figure 2.4(a).

2.3.1 Program Model

The AdaptFun language extends a standard while language⁵ with a query request constructor denoted `query`. Queries have the form `query( $\psi$ )`, where ψ is a special expression (see syntax in Section 4) representing a function $: \mathcal{X} \rightarrow U$ on rows. We use U to denote the codomain of queries and it could be $[-1, 1]$, $[0, 1]$ or $[-R, +R]$, for some R we consider. This function characterizes the linear query we are interested in running. Indeed, as we discussed in the previous section, linear queries compute the empirical mean of a function on rows — we use χ to abstract a possible row in the database. As an example, `$x \leftarrow \text{query}(\chi[j] \cdot \chi[k])$` computes an approximation, according to the used mechanism, of the empirical mean of the product of the j^{th} attribute and k^{th} attribute, identified by $\chi[j] \cdot \chi[k]$. Notice that we don't materialize the mechanism but we assume that it is implicitly run when we execute the query. In Figure 2.4(a), the queries inside the while loop corre-

⁵Programs components are labeled, so that we can uniquely identify every component.

spond to the first phase of the data analysis and compute an approximation of the product of the empirical mean of the first k attributes. The query outside the loop corresponds to the second phase and computes an approximation of the empirical mean where each record is weighted by the sum of the empirical mean of the first k attributes.

This example is intuitively 2-rounds adaptive since we have two clearly distinguished phases, and the queries that we ask in the first phase do not depend on each other (the query $\chi[j] \cdot \chi[k]$ at line 3 only relies on the counter j and input k), while the last query (at line 6) depends on the results of all the previous queries. However, capturing this concept formally is surprisingly difficult. The difficulty comes from the fact that a query can depend on the result of another query in multiple ways, by means of data flow or control flow.

2.3.2 Adaptivity Definition

The central property we are after in this work is the *adaptivity of a program*, which corresponds to the second component, the “Adaptivity Formalization” block at the middle of Figure 2.3. As shown in the flow-chart in this block, we define formally this notion in three steps, which we will describe in detail in Section 5. First, we define a notion of dependency, or better *may-dependency*, between variables. To do this we take inspiration from previous works on dependency analysis and information flow control and we say that a variable *may depend* on another one if changing the execution of the latter can affect the execution of the former. We can see in Figure 2.4(a) that the value of the variable l , which corresponds to the result of the execution of the query in the second phase (in the command with label 6), is affected by the value of the variable x , which corresponds to the result of the exe-

cution of the query at line 3 in the first phase, via the variable a . To formally define this notion of dependency, as in information flow control, we use the execution history of programs recorded by trace semantics (see Definition 17).

Second, we build an annotated weighted directed graph representing the possible dependencies between labeled variables. We call this graph *semantics-based dependency graph* to stress that this graph summarizes the dependencies we could see if we knew the overall behavior of the program. The vertices of the graph are the assigned program variables with the label of their assignments, edges are pairs of labeled variables that satisfy the dependency relations, weights are functions associated with vertexes and describing the number of times the assignment corresponding to the vertex is executed when the program is run in a given starting state⁶, and the annotations, which we call *query annotations*, are bits associated with vertexes and describe if the corresponding assignment comes from a query (1) or not (0). The *semantics-based dependency graph* of the `twoRounds(k)` program we gave in Figure 2.4(a) is described in Figure 2.4(b) (we use dashed arrows for two edges that will be highlighted in the next step, for the moment these can be considered similar to the other edges—i.e. solid arrows). We have all the variables that are assigned in the program with their labels, and edges representing dependency relations between them. For example, we have two edges (l^6, a^5) and (a^5, x^3) describing the dependency between the variables assigned by queries. The vertices l^6 and x^3 are the only ones with query annotation 1 (the subscript), since they are the only two variables that are in assignments involving queries. Notice that the graph contains cycles—in this example, it contains two self-loops. These cycles capture the fact that the variables a^5 and j^4 are updated at every iteration of the loop using

⁶In our trace semantics the state is recorded in the trace, so an initial state is actually represented by an initial trace. We will use this terminology in later sections.

their previous value. Cycles are essential to capture mutual dependencies like the ones that are generated in loops. Adaptivity is a quantitative notion, so capturing this form of dependency is not enough. This is why we also use weights. The weight of a vertex is a function that given an initial state returns a natural number representing the number of times the assignment corresponding to a vertex is visited during the program execution starting in this initial state. For example, the vertex l^6 has weight $\lambda\tau.1$ since for every initial state τ the corresponding assignment will be executed one time, the vertex a^5 on the other hand has weight $\lambda\tau.\rho(\tau)k$ since the corresponding assignment will be executed a number of times that correspond to the value of k in the initial state τ , and ρ is the operator reading the value of k from τ .

Third, we can finally define adaptivity using the semantics-based dependency graph. We actually define this notion with respect to an initial state τ , since different states can give very different adaptivity. We consider the longest walk that visits each vertex v of the semantics-based dependency graph no more than the value that the weight w_v assign to τ , and visits as many query nodes as possible. The number of query nodes visited is the adaptivity of the program with respect to τ . Looking again at Figure 2.4(b), and assuming that $\tau(k) \geq 1$, we can see that the walk along the dashed arrows, $l^6 \rightarrow a^5 \rightarrow x^3$ has two vertices with query annotation 1, and we cannot find another walk having more than 2 vertices with query annotation 1. So the adaptivity of the program in Figure 2.4(a) with respect to τ is 2. If we consider an initial state τ such that $\tau(k) = 0$ we have that the adaptivity with respect to τ is instead 1.

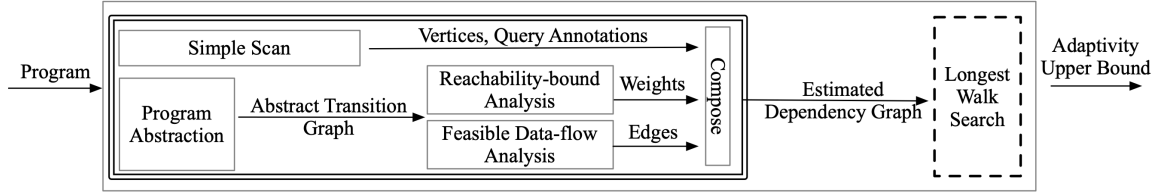


Figure 2.5: The overview of AdaptFun

2.3.3 Program Analysis

To compute statically a sound and accurate upper bound on the *adaptivity* of a program c , we design a program analysis framework named AdaptFun which we will describe formally in 6. This program analysis framework corresponds to the “Adaptivity Analysis” block at the bottom of Figure 2.3. The detailed structure of AdaptFun is illustrated in Figure 2.5, which reflects in part the definition of adaptivity we discussed in the previous section. Specifically, AdaptFun is composed of two algorithms (the ones in dashed boxes in the figure), one for building a dependency graph, which we call *estimated dependency graph*, and the other to estimate the adaptivity from this graph. The first algorithm, which we will describe formally in Section 6, generates the *estimated dependency graph* using several program analysis techniques. Specifically, AdaptFun extracts the vertices and the query annotations by looking at the assigned variables of the program, it estimates the edges by using control flow and data flow analysis, and it estimates the weights by using symbolic reachability-bound analysis—weights in this graph are symbolic expressions over input variables. The second algorithm estimates the longest walk which respects the weights and which visits as many query nodes as possible. The two algorithms together give us an upper bound on the program’s *adaptivity*.

We show in Figure 2.4(c) the estimated dependency graph that our static analysis algorithm returns for the program `twoRounds(k)` in Figure 2.4(a). Vertices and query annotations are the same as the ones in Figure 2.4(b) and they are simply

inferred by scanning the program. As we said before, the edges are estimated using control flow and data flow analysis. For the `twoRounds(k)` example, every edge in Figure 2.4(b) is precisely inferred by our combined analysis, this is why Figure 2.4(c) contains exactly the same edges. The weight of every vertex is computed using a reachability-bound estimation algorithm that outputs a symbolic expression over the input variables, in the example only k , representing an upper bound on the number of times each assignment is executed. For example, consider the vertex x^3 , its weight is k and this provides an upper bound on the values returned by the weight function $\lambda\tau.\rho(\tau)k$ associated with vertex x^3 in Figure 2.4(b) for any initial state.

The algorithm searching for the longest walk first finds a path $l^6 : \frac{1}{1} \rightarrow a^5 : \frac{k}{1} \rightarrow x^3 : \frac{k}{1}$, and then constructs a walk based on this path. Every vertex on this walk is visited once, and the number of vertices with query annotation 1 in this walk is 2, which is the upper bound we expect. It is worth noting here that x^3 and a^5 can only be visited once because there isn't an edge to go back to them, even though they both have the weight k . In this sense, instead of simply computing the weighted length of this path ($2k + 1$) as adaptivity `AdaptSearch` computes the upper bound 2. Note that 2 is not always tight, for example, when $k = 0$.

2.4 OUTLINE AND CONTRIBUTIONS

Chapter Outline The rest chapters in this part are organized as follows.

1. The previous works on analyzing the *adaptivity* of adaptive data analysis are introduced in Section 3.
2. The new program analysis framework for *adaptivity* in adaptive data analysis with three components.

- (a) A while-like language extended with query request feature, named `QWhile` Language, used to implement the adaptive data analysis in Section 4.
 - (b) A formal adaptivity model through analyzing the program's execution semantically in Section 5.
 - (c) A static program analysis algorithm, named `AdaptFun` through static adaptivity analysis in Section 6.
3. Examples demonstrating our program adaptivity analysis framework, and the evaluation results from the implementation of this analysis framework is presented in Section 7.
 4. Section 8 presents the related works in the program analysis and the data analysis areas related to our adaptivity analysis framework.

Contributions This part has the following contributions.

1. A programming framework for adaptive data analyses where programs represent analysts that can query generalization-preserving mechanisms mediating the access to some data.
2. A formal definition of the notion of adaptivity under the analyst-mechanism model. This definition is built on a variable-based dependency graph that is constructed using sets of program execution traces.
3. A static program analysis algorithm `AdaptFun` combining data flow, control flow and reachability bound analysis in order to provide tight bounds on the adaptivity of a program.

4. A soundness proof of the program analysis showing that the adaptivity estimated by **AdaptFun** bounds the *adaptivity* of the program given by the formal definition.
5. An implementation of **AdaptFun** and an experimental evaluation of the bounds this implementation provides on several examples.

CHAPTER 3

Previous Works on Adaptivity Analysis

This chapter introduces the previous works on analyzing the adaptivity property and outlines the limitations. There are mainly three parts in previous works. In Section 3.1, I introduce the previous language design, which aims to formalize the adaptive data analysis program and solve the first challenge. Section 3.2 recapitulates the previous works on defining this intuitive *adaptivity* quantity through a formal model. Then Section 3.3 includes a summary of previous works on estimating their formalized *adaptivity* and the limitations.

3.1 LANGUAGE MODEL

Previous works from the thesis by Qu (2022) formalized the adaptive data analysis into a while-like language named loop language with limited expressiveness.

3.1.1 Language Syntax

Figure 3.1 is a selection of the syntax from their loop language. The expressions can be arithmetic, boolean expressions, or query expression. The arithmetic and boolean expressions are standard. Query expressions support the query requests in the data analysis. The special variable χ represents a row of the database, and access to values at a certain column in χ , as $\chi[a]$. However, the while loop iteration times cannot depend on the program state.

Arithmetic Operators	$\oplus_a ::= + \mid - \mid \times \mid \div$
Boolean Operators	$\oplus_b ::= \vee \mid \wedge \mid \neg$
Relational Operators	$\sim ::= < \mid \leq \mid ==$
Arithmetic Expressions	$a ::= n \mid x \mid a \oplus_a a$
Boolean Expressions	$b ::= \text{true} \mid \text{false} \mid \neg b \mid b \oplus_b b \mid a \sim a$
Expressions	$e ::= a \mid b \mid [] \mid [e, \dots, e]$
Values	$v ::= n \mid \text{true} \mid \text{false} \mid [] \mid [v, \dots, v]$
Query expressions	$e_q ::= a \mid \chi \mid \chi[a] \mid e_q \oplus_a e_q$
Query Values	$v_q ::= n \mid \chi \mid \chi[n] \mid v_q \oplus_a v_q$
Commands	$c ::= \text{skip} \mid x \leftarrow e \mid x \leftarrow q(e_q) \mid \text{loop } a \text{ do } c \mid c; c \mid \text{if } (b, c, c)$

Figure 3.1: Syntax of loop language.

3.1.2 Trace-Based Operational Semantics

The previous operational semantics is defined based on labeled command and trace with some special operators. Below is a summary of their designs.

Labeled Commands Each command is labeled with a natural number l standing for the line of code where the command appears. For the if statement, the label is associated with the conditional predicate b and for the loop statement, it is associated with the loop counter a .

$$\begin{aligned} \text{Labeled commands } c ::= & [x \leftarrow e]^l \mid [x \leftarrow q(e_q)]^l \mid \text{loop } [a]^l \text{ do } c \mid c; c \\ & \mid \text{if } ([b]^l, c, c) \mid [\text{skip}]^l \end{aligned}$$

Loop Map The loop map is a map from the label l to the iteration number n . A mapping $[k \rightarrow n]$ gives accurate information on which loop a statement is in by its

key k (label at loop counter), and which iteration n the statement belongs to.

Loop Map $w \in \text{Label} \rightarrow \mathbb{N}$ Memory $m ::= [] \mid m[x \rightarrow v]$
 Annotated Query $\mathcal{AQ} ::= \{q(v_q)^{(l,w)}\}$ Trace $t ::= [] \mid q(v_q)^{(l,w)} :: t$

Annotated Query To analyze the dependency relations between queries, the previous work designs a data structure, the *annotated query*. Through this annotation, queries can be uniquely annotated by (l, w) , which considers the location of the query through the line number l and the iteration index where this query is located if the query appears in a loop statement, specified by w .

Trace A trace t is a list of annotated queries accumulated along with the execution of the program. It consists of the queries asked by the analyst during the execution of the program, collected through a trace-based small-step operational semantics based on transitions of the form $\langle m, c, t, w \rangle \rightarrow \langle m', \text{skip}, t', w' \rangle$.

Operational Semantics Rules Figure 3.2 is a selection of rules of the trace-based operational semantics from the thesis by Qu (2022). We present only the rules related to query requests and the while loop evaluations. The rule **l-query-e** evaluates the argument e_q of a query request $q(e_q)$ using the query evaluation $\Downarrow_q$. When the query expression is in the normal form, this query will be answered. The rule **l-query-v** modifies the starting memory m to $m[v_q/x]$ using the answer v of the query $q(v_q)$ from the mechanism, with the trace expanded by appending the query $q(v_q)$ with the current annotation (l, w) . The rule for assignment is standard and the trace remains unchanged. The sequence rule keeps tracking the modification of the trace, and the evaluation rule for the if command goes into one branch based on the result of the conditional predicate b . The rule **l-loop-a** first evaluates the

$$\begin{array}{c}
\frac{\langle m, a \rangle \Downarrow_a \langle m, a' \rangle}{\langle m, \text{loop } [a]^l \text{ do } c, t, w \rangle \rightarrow \langle m, \text{loop } [a']^l \text{ do } c, t, (w + l) \rangle} \text{ l-loop-a} \\
\\
\frac{v_N > 0}{\langle m, \text{loop } [v_N]^l \text{ do } c, t, w \rangle \rightarrow \langle m, c; \text{loop } [(v_N - 1)]^l \text{ do } c, t, (w + l) \rangle} \text{ l-loop} \\
\\
\frac{\langle m, e_q \rangle \Downarrow_q \langle m, e'_q \rangle}{\langle m, [x \leftarrow q(e_q)]^l, t, w \rangle \rightarrow \langle m, [x \leftarrow q(e'_q)]^l, t, w \rangle} \text{ l-query-e} \\
\\
\frac{q(v_q) = v}{\langle m, [x \leftarrow q(v_q)]^l, t, w \rangle \rightarrow \langle m[v/x], \text{skip}, t ++ [q(v_q)^{(l,w)}], w \rangle} \text{ l-query-v}
\end{array}$$

Figure 3.2: Trace-based operational semantics of loop language.

loop counter a , when a loop counter is a constant, then the evaluation will start to execute the loop body. The rules for loop modify the loop map w . In the rule, **l-loop**, the loop map w is updated by $w + l$ because the execution goes into another iteration when the condition $v_N > 0$ is satisfied.

3.1.3 Limitations

This language model is limited in its expressiveness.

The loop command in this language model can only iterate a constant of times, specifically, the guard in the loop command can only be a constant or an arithmetic expression that must be evaluated to a natural number. Below we analyze the reasons. In the rule **l-loop**, a natural number v_N is required on the premise of tracking the iteration times. This requires that the loop iteration number has to be a natural number or evaluated to a natural number in advance to execute the loop. Then in their trace and annotated query design, the trace tracks the annotated query, which requires an integer annotation explicitly indicating the iteration number of

the loop. For example, in the following program with a simple while loop,

$$x \leftarrow 20; y \leftarrow 100; \text{ while } (x < y) \text{ do } \{x \leftarrow x + 1; y \leftarrow y - 2; \}$$

the number of iterations cannot be evaluated to a natural number in advance of entering this loop. The iteration number is only able to be decided during executing this while loop body. This program represents a class of data analysis programs with non-constant loop iterations which is very common in data analysis. However, it isn't supported by their design.

3.2 ADAPTIVITY FORMALIZATION

The previous works formalized the adaptivity through a query-based dependency graph. Below is a summary of their query-based dependency graph construction and adaptivity formal definition, followed by the weaknesses of this formalization method.

3.2.1 Query-Based Dependency Graph

Dependency graph in previous work is over annotated queries. To construct the edges on this graph, they first define a query *May-dependency* relation over all queries. Below is a summary of this definition from the works by Qu (2022). The notations $\in_q$ and $\notin_q$ represent that a query belongs to the trace or not respectively.

Definition 1 (Query May-dependency). *One query $q(v_{q_1})$ may depend on another query $q(v_{q_2})$ in a program c , with a starting loop maps w , a starting memory m , hidden database D , denoted as*

$\text{DEP}(q(v_{q_1})^{(l_1, w_1)}, q(v_{q_2})^{(l_2, w_2)}, c, w, m, D)$ is defined below.

$$\forall t. \exists m_1, m_3, t_1, t_3, c_2. \left(\begin{array}{l} \langle m, c, t, w \rangle \rightarrow^* \langle m_1, [x \leftarrow q(v_{q_1})]^{l_1}; c_2, t_1, w_1 \rangle \rightarrow \\ \langle m_1[q(v_{q_1})(D)/x], c_2, t_1 + [q(v_{q_1})^{(l_1, w_1)}], w_1 \rangle \rightarrow^* \langle m_3, \text{skip}, t_3, w_3 \rangle \\ \wedge \\ \left(q(v_{q_1})^{(l_1, w_1)} \in_q (t_3 - t) \wedge q(v_{q_2})^{(l_2, w_2)} \in_q (t_3 - t_1) \right) \\ \implies \exists v \in \text{codom}(q(v_{q_1})), m'_3, t'_3, w'_3. \\ \langle m_1[v/x], c_2, t_1 + [q(v_{q_1})^{(l_1, w_1)}], w_1 \rangle \rightarrow^* \langle m'_3, \text{skip}, t'_3, w'_3 \rangle \\ \wedge (q(v_{q_2})^{(l_2, w_2)} \notin_q (t'_3 - t_1)) \\ \wedge \\ \left(q(v_{q_1})^{(l_1, w_1)} \in_q (t_3 - t) \wedge q(v_{q_2})^{(l_2, w_2)} \notin_q (t_3 - t_1) \right) \\ \implies \exists v \in \text{codom}(q(v_{q_1})), m'_3, t'_3, w'_3. \\ \langle m_1[v/x], c_2, t_1 + [q(v_{q_1})^{(l_1, w_1)}], w_1 \rangle \rightarrow^* \langle m'_3, \text{skip}, t'_3, w'_3 \rangle \\ \wedge (q(v_{q_2})^{(l_2, w_2)} \in_q (t'_3 - t_1)) \end{array} \right)$$

Based on the *may-dependency* between the two queries above, below is their formal definition of the query-based dependency graph. For every execution of a program c starting with a certain memory and while map they construct a query-based dependency graph as below.

Definition 2 (Query-based Dependency Graph). *Given a program c , a database D , a starting memory m , an initial loop map w , the query-based dependency graph $G(c, D, m, w) = (V, E)$ is defined as:*

$$V = \{q(v_q)^{l, w} \in \mathcal{AQ} \mid \forall t. \exists m', w', t'. \langle m, c, t, w \rangle \rightarrow^* \langle m', \text{skip}, t', w' \rangle \wedge q(v_q)^{l, w} \in (t' - t)\}.$$

$$E = \{(q(v_q)^{(l, w)}, q(v_q')^{(l', w')}) \in \mathcal{AQ} \times \mathcal{AQ} \mid \text{DEP}(q(v_q')^{(l', w')}, q(v_q)^{(l, w)}, c, w, m, D)\}.$$

The edge is directed, when an annotated query $q(v_q)^{(l, w)}$ may depend on its previous query $q(v_q')^{(l', w')}$, we have the directed edge $(q(v_q)^{(l, w)}, q(v_q')^{(l', w')})$, from

$q(v_q)^{(l,w)}$ to $q(v_q')^{(l',w')}$.

The query-based dependency graph only considers the newly generated annotated queries during the execution of the program c , so they see the nodes coming from the trace $t' - t$. The previous trace before the execution of c is excluded when constructing the graph.

3.2.2 Adaptivity Formalization

Below is the definition of adaptivity from works by Qu (2022), by means of the query-based dependency graph.

Definition 3 (Adaptivity in loop language). *Given a program c , and a memory m , a database D , a starting loop map w , the adaptivity of the dependency graph $G(c, D, m, w) = (V, E)$ is the length of the longest path in this graph. We denote the path from $q(v_q)^{(l,w)}$ to $q(v_q')^{(l',w')}$ as $p(q(v_q)^{(l,w)}, q(v_q')^{(l',w')})$. The adaptivity is denoted as $A(c, D, m, w)$.*

$$A(c, D, m, w) = \max_{q(v_q)^{(l,w)}, q(v_q')^{(l',w')} \in V} |p(q(v_q)^{(l,w)}, q(v_q')^{(l',w')})|$$

3.2.3 Limitations

Limited by the language design and the dependency graph construction, this adaptivity definition has the following weaknesses.

Expressiveness. Because the language model only support the program with while loop with constant number of iteration times. In this sense, this definition does not general enough to capture the adaptivity of the program with loop whose iteration times dependent on the program state. In the following example program similar to the one in Section 3.1 with two extra query request commands,

```
 $x \leftarrow 0; y \leftarrow 5; z \leftarrow q(x + y); \text{ while } (x < y) \text{ do } \{x \leftarrow x + 1; y \leftarrow y - 1; z \leftarrow q(x + y + z); \}$ 
```

the number of iterations cannot be evaluated to a natural number in advance of entering this loop for the same reason. The iteration number is only able to be decided during executing this while loop body. While this adaptivity definition cannot capture this program though the adaptivity in this program is 4, which is straightforward to observe.

Accuracy. The inaccuracy of this *adaptivity* definition is caused by both their language design and their graph construction. This dependency graph definition relies on a specific memory, while-map, and a specific execution trace. It limits the adaptivity defined for the adaptive data analysis program to be w.r.t. one specific execution. In the other words, this adaptivity definition doesn't correspond to the *adaptivity* for this program, but for the program in a certain execution.

Complexity. This definition requires the full unfolding of every iteration for the while loop. The unfolding procedure generates many unnecessary and duplicate nodes during the while iterations. There are two reasons for this unfolding operation. The first one comes from the low efficiency of their trace-based operational semantics, which requires the trace to track the loop iteration number in the annotated query. The other comes from their dependency graph generation, which requires every query request evaluated during the program execution as the graph nodes. Both two processes generate unnecessary and duplicate nodes during the loop iterations.

3.3 PROGRAM ANALYSIS FOR ADAPTIVITY

In the previous works, the adaptivity is estimated through constructing a variable-based weighted dependency graph for estimating the query-based dependence graph in Section 3.2. It first re-writes the program from the loop language into a

Static Single Assignment (SSA) version of loop language, and then estimates the adaptivity through a program analysis algorithm.

3.3.1 SSA-Language

In order to distinguish different query requests in the case where they are assigned to the same variable, they re-write the program into the SSA form. A selection of the (SSA) version of loop language syntax is shown below.

$$\begin{aligned} c ::= & [x \leftarrow e]^l \mid [x \leftarrow q(e_q)]^l \mid \text{ifvar}(\bar{x}, \bar{x}') \mid [\text{skip}]^l \mid \text{loop } [a]^l, n, [\bar{x}, \bar{x}_1, \bar{x}_2] \text{ do } c \mid \\ & c; c \mid \text{if } ([b]^l, ([\bar{x}, \bar{x}_1, \bar{x}_2], [\bar{y}, \bar{y}_1, \bar{y}_2], [\bar{z}, \bar{z}_1, \bar{z}_2])), c, c) \end{aligned}$$

In the SSA form, the if command contains an extra part, $([\bar{x}, \bar{x}_1, \bar{x}_2], [\bar{y}, \bar{y}_1, \bar{y}_2], [\bar{z}, \bar{z}_1, \bar{z}_2])$, to track the dependency of newly assigned variables in both branches $[\bar{y}, \bar{y}_1, \bar{y}_2]$ and $[\bar{z}, \bar{z}_1, \bar{z}_2]$. The SSA form of the loop command also has an extra part $[\bar{x}, \bar{x}_1, \bar{x}_2]$ to track the variables in the loop body.

The operational semantics rules in Section 3.1 are all translated into the corresponding SSA form as well. Based on the transformed language, they also re-define the formal adaptivity for the transformed program equivalent to the loop language-based definition. The complete definition is in the thesis by Qu (2022).

3.3.2 Program Analysis Algorithm

This part summarizes the adaptivity analysis algorithm that performs on the SSA version of the loop language. This algorithm consists of three auxiliary algorithms: a variable estimation algorithm VE, a matrix-vector based graph generating algorithm GG to generate the weighted variable-based dependency graph, and a path-searching algorithm PS to find the most weighted path in the graph.

$$\begin{array}{c}
\frac{}{\text{VE}(G; w; [\mathbf{x} \leftarrow q(\mathbf{e}_q)]^l) \rightarrow (G + +[\mathbf{x}^{(l,w)}]; w)} \text{ ag-query} \\
\\
\frac{\text{VE}(G; w; \mathbf{c}_1) \rightarrow (G_1; w_1) \quad \text{VE}(G_1; w; \mathbf{c}_2) \rightarrow (G_2; w_2) \quad G_3 = G_2 + +[\bar{\mathbf{x}}^{(l,w)}] + +[\bar{\mathbf{y}}^{(l,w)}] + +[\bar{\mathbf{z}}^{(l,w)}]}{\text{VE}(G; w; [\text{ if } (\mathbf{b}, [\bar{\mathbf{x}}, \bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2], [\bar{\mathbf{y}}, \bar{\mathbf{y}}_1, \bar{\mathbf{y}}_2], [\bar{\mathbf{z}}, \bar{\mathbf{z}}_1, \bar{\mathbf{z}}_2], \mathbf{c}_1, \mathbf{c}_2)]^l) \rightarrow (G_3; w)} \text{ ag-if} \\
\\
\frac{G_0 = G \quad w_0 = w \quad \forall 0 \leq z < N. \text{VE}(G_z + +[\bar{\mathbf{x}}^{(l,w_z+1)}]; (w_z + l); \mathbf{c}) \rightarrow (G_{z+1}; w_{z+1}) \quad G_f = G_N + +[\bar{\mathbf{x}}^{(l,w_N \setminus 1)}] \quad \mathbf{a} = N}{\text{VE}(G; w; [\text{ loop } \mathbf{a}, n, [\bar{\mathbf{x}}, \bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2] \text{ do } \mathbf{c}]^l) \rightarrow (G_f; w_N \setminus l)} \text{ ag-loop}
\end{array}$$

Figure 3.3: The key rule of the variable estimation algorithm.

1. Variable Estimation Algorithm

The VE specifies the nodes of their variable-based dependency graph. The result of VE is stored in a global variable list G , fed to the next step. Figure 3.3 is a selection of the three key rules from their VE algorithm. This algorithm adds all the SSA version program variables to the global variable list G .

Their variable estimation for the while loop is low-efficient. As shown in rule **ag-loop**, they unfold every iteration of the while loop and create new annotated variables for every iteration. This rule causes a major efficiency limitation in their static analysis. It also causes a critical expressiveness limitation. By the premise in the rule **ag-loop**, the arithmetic expression is required to be a natural number. This rule limits the program cannot even have a loop with an arithmetic expression like $10 + 4$ in the guard.

2. Graph Generating

The algorithm GG generates a matrix-vector-based graph for the program. The matrix M records the may-dependency between annotated variables in the global list G . It has size $|G| \times |G|$. The vector V has the same size as G and gives a weight to each variable in G . This weight is 1 when the variable

$$\begin{array}{c}
\frac{M = L(i) * (R(\mathbf{e}_q, i) + \Gamma) \quad V = L(i)}{GG(\Gamma; [\mathbf{x} \leftarrow q(\mathbf{e}_q)]^l; i) \rightarrow (M; V; i + 1)} \text{ad-query} \\
\\
\begin{array}{c}
GG(\Gamma + R(\mathbf{b}, i_1); \mathbf{c}_1; i_1) \rightarrow (M_1; V_1; i_2) \\
GG(\Gamma + R(\mathbf{b}, i_1); \mathbf{c}_2; i_2) \rightarrow (M_2; V_2; i_3) \quad GG(\Gamma; [\bar{\mathbf{x}}, \bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2]; i_3) \rightarrow (M_x; V_\emptyset; i_3 + |\bar{\mathbf{x}}|) \\
GG(\Gamma; [\bar{\mathbf{y}}, \bar{\mathbf{y}}_1, \bar{\mathbf{y}}_2]; i_3 + |\bar{\mathbf{x}}|) \rightarrow (M_y; V_\emptyset; i_3 + |\bar{\mathbf{x}}| + |\bar{\mathbf{y}}|) \\
GG(\Gamma; [\bar{\mathbf{z}}, \bar{\mathbf{z}}_1, \bar{\mathbf{z}}_2]; i_3 + |\bar{\mathbf{x}}| + |\bar{\mathbf{y}}|) \rightarrow (M_z; V_\emptyset; i_3 + |\bar{\mathbf{x}}| + |\bar{\mathbf{y}}| + |\bar{\mathbf{z}}|) \\
M = (M_1 + M_2) + M_x + M_y + M_z
\end{array} \\
\hline
GG(\Gamma; \text{if } ([\mathbf{b}]^l, [\bar{\mathbf{x}}, \bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2], [\bar{\mathbf{y}}, \bar{\mathbf{y}}_1, \bar{\mathbf{y}}_2], [\bar{\mathbf{z}}, \bar{\mathbf{z}}_1, \bar{\mathbf{z}}_2], \mathbf{c}_1, \mathbf{c}_2); i_1) \rightarrow (M; V_1 \uplus V_2; i_3 + |\bar{x}| + |\bar{y}| + |\bar{z}|) \text{ad-if} \\
\\
\begin{array}{c}
B = |\bar{\mathbf{x}}| \quad A = |\mathbf{c}| \\
\forall 0 \leq j < N. GG(\Gamma; [\bar{\mathbf{x}}, \bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2]; i + j * (B + A)) \rightarrow (M_{1j}; V_{1j}; i + B + j * (B + A)) \\
GG(\Gamma; \mathbf{c}; i + B + j * (B + A)) \rightarrow (M_{2j}; V_{2j}; i + B + A + j * (B + A)) \\
GG(\Gamma; [\bar{\mathbf{x}}, \bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2]; i + N * (B + A)) \rightarrow (M; V; i + N * (B + A) + B) \\
\mathbf{a} = N \quad M' = M + \sum_{0 \leq j < N} (M_{1j} + M_{2j}) \quad V' = V \uplus \sum_{0 \leq j < N} (V_{1j} \uplus V_{2j})
\end{array} \\
\hline
GG(\Gamma; \text{loop } [\mathbf{a}]^l, 0, [\bar{\mathbf{x}}, \bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2] \text{ do } \mathbf{c}, i) \rightarrow (M'; V'; i + N * (B + A) + B) \text{ad-loop}
\end{array}$$

Figure 3.4: The key rules of the graph generating algorithm.

is assigned with a query request and 0 otherwise.

Some key rules are selected in Figure 3.4. The rule **ad-loop** is as low-efficient and expressiveness-limited as the rule **ag-loop** in Figure 3.3 for the same reason. This will be analyzed in detail in the limitation part.

3. Longest Weighted Path Search

They use a standard longest-path search algorithm to compute the adaptivity bound over the variable-based dependency graph.

3.3.3 Limitations

Limitations of this adaptivity estimation algorithm is summarized from the same three aspects as follows.

Efficiency.

1. **SSA Transformation** In order to address the issue of re-assignment of differ-

ent queries requesting results to the same variable, they re-write the program from the loop language into SSA form. However, this rewriting is inefficient and unnecessary. The re-assignment problem can be resolved efficiently and accurately through many state-of-art static program analysis techniques, such as the variable reachable analysis, etc.

2. **Variable Estimation Algorithm** According to the rules **ag-loop** in Figure 3.3, and rule **ad-loop** in Figure 3.4, the algorithm needs to unfold every iteration of the while loop and create new annotated variables for every iteration. This operation increased the complexity of the program analysis by exponential factors, which is inefficient.
3. **Path Search and Adaptivity Estimation** Based on the unfolded loops and exponential numbers of the annotated variables, the graph generated over these variables are exponentially huge as well. However, to estimate the adaptivity over this graph, we need to search for the longest weight path over it. The exponential size of the graph increases the burden of the path search algorithm significantly.

Expressiveness. The variable estimation algorithm is unable to analyze the programs with non-constant (or non-deterministic) loop iteration numbers. This is caused by their **ag-loop** rule in Figure 3.3. This rule unfolds every iteration of the while loop and creates new annotated variables for every iteration. In order to guarantee the termination of the analysis, they have to limit the loop with a constant number of loop iterations. Specifically in rule **ag-loop** and **ad-loop**, the premise in these two rules requires the arithmetic expression to be a natural number. This rule limits the program cannot even have a loop with an arithmetic ex-

pression like $10 + 4$ in the guard. Concretely, a simple example program with a loop iterating 5 times as follows isn't allowed in their work.

$$x \leftarrow 5; z \leftarrow q(x); \text{ loop } (x) \text{ do } \{z \leftarrow q(x + z); \}$$

Accuracy. The estimated adaptivity from this framework is loose. It over-approximates in cases where there isn't semantic dependency between variables even though the variables are explicitly used in the query request. This is also because the algorithms that are used in each step are simply the standard algorithms that do not naturally compatible with each other. So the straightforward composition is not able to estimate the adaptivity property accurately when considering it as the *quantitative-reachability* property.

3.4 LIMITATIONS

To recapitulate, the *adaptivity* is only analyzed from the *quantitative* perspective in previous works. However, as we discussed before in the introduction, we consider it as a *quantitative-reachability* property that has two perspectives sub-properties, the *quantitative* and *reachability*. In the other word, previous works only analyzed it as a sub-property and lost the information from the other perspective.

Expressiveness. This language model doesn't support programs with while loop whose number of iterations not known before loop is running. It only allows the while loops in the program to iterate a fixed number of times. This is because the expression in the loop guard has to be a constant number or an arithmetic expression that is evaluated into a constant V_N before executing the loop body. However, the adaptive data analysis programs with loops of non-deterministic

iteration times are very common. The syntax doesn't support the programs with user inputs either, which are also common in the data analysis area.

Accuracy. The adaptivity definition and the estimated adaptivity are both inaccurate.

- For the *adaptivity* definition, because the dependency graph relies on a specific memory, while-map, and a specific execution trace, the *adaptivity* then is also defined w.r.t. one specific execution. In this sense, this adaptivity definition isn't the *adaptivity* of the adaptive data analysis implemented by this program, but only its *adaptivity* in a certain execution.
- The adaptivity bound computed from this framework is loose. Because the adaptivity estimation algorithm composes three standard static analysis techniques straightforwardly. The **graph generation** algorithm that uses the standard data-flow analysis technique over-approximates the data dependency relation in a large scale. And the standard path search algorithm also over-approximates the longest weighted path on this over-approximated dependency graph. In this sense, the estimated adaptivity is a loose bound on the defined adaptivity.

Efficiency.

- The operational semantics has four components in the configuration when evaluating the program. Updating this quadruple configuration (especially the while map) is inefficient.
- The adaptivity estimation algorithm transforms the loop language into SSA form, in order to address the issue of re-assignment of different queries requesting results to the same variable. This transformation is inefficient and

unnecessary. The re-assignment problem can be resolved efficiently and accurately through other static program analysis techniques, such as the variable reachable analysis, etc..

- The **variable estimation** and **graph generation** algorithms are inefficient. The **ag-loop** rule in Figure 3.3 and the **ad-loop** rule in Figure 3.4 unfold every iteration of the while loop. This operation increased the algorithms' complexity exponentially.

Given the limitations in previous work, we develop our new framework composed of three parts as well as in Figure 2.5. It is presented in the following three chapters.

CHAPTER 4

Language Model – QWhile

In this part, we introduce the program language model for implementing the data analyses. This is an extended standard while language with primitives for query requests. We first give the syntax of the language in Section 4.1 with explanation on the novelty of this language. Section 4.2 contains the trace-based operational semantics. This is the main technical ingredient to define the program's adaptivity formally. Section 4.3 in the last part presents an example manually demonstrating the language and the operational semantics rules.

4.1 SYNTAX

The syntax is shown as follows,

Mathematic Operator	$\oplus_a ::= + \mid - \mid \times \mid \div \mid \mathbf{max} \mid \mathbf{min}$	
Boolean Operator	$\oplus_b ::= \vee \mid \wedge$	
Relational Operator	$\sim ::= < \mid \leq \mid =$	
Extended Arithmetic	$a ::= n \in \mathbb{N} \cup \{\infty\} \mid x \mid a \oplus_a a$	
Expression	$\mid \log a \mid \mathbf{sign} a$	
Boolean Expression	$b ::= \mathbf{true} \mid \mathbf{false} \mid \neg b \mid b \oplus_b b \mid a \sim a$	
Expression	$e ::= v \mid a \mid b \mid [e, \dots, e]$	
Value	$v ::= n \mid \mathbf{true} \mid \mathbf{false} \mid [] \mid [v, \dots, v]$	
Query Expression	$\psi ::= \alpha \mid a \mid \psi \oplus_a \psi \mid \chi[a]$	
Query Value	$\alpha ::= n \mid \chi[n] \mid \alpha \oplus_a \alpha$	
Label	$l \in \mathbb{N} \cup \{\mathbf{in}, \mathbf{ex}\}$	
Labeled Command	$c ::= [x \leftarrow e]^l \mid [x \leftarrow \mathbf{query}(\psi)]^l \mid \mathbf{while} [b]^l \mathbf{do} (c) \mid c; c \mid \mathbf{if} ([b]^l, c, c) \mid [\mathbf{skip}]^l$	
Event	$\epsilon ::= (x, l, v, \bullet) \mid (x, l, v, \alpha) \mid (b, l, v, \bullet)$	Assignment Event Testing Event

We denote by ∞ a value s.t. $n < \infty$ for all $n \in \mathbb{N}$, and we use the following notations to represent the sets of corresponding terms:

$\mathcal{V}$	: Set of Variables
$\mathcal{L}$	: Set of Labels
$\mathcal{LV}$	: Set of Labeled Variables
$\mathcal{VAL}$	: Set of Values
$\mathcal{Q}$	: Set of Query Requests, including query expressions and values
$\mathcal{B}$	: Set of All Boolean Expressions
$\mathcal{C}$	: Set of Well-formed Commands
$\mathcal{E}$	: Set of Events
$\mathcal{E}^{\text{asn}}$	: Set of Assignment Events
$\mathcal{E}^{\text{test}}$	: Set of Testing Events
$\mathcal{DB}$	: Set of Databases
$\mathcal{T}^+$	: Set of All Finite Execution Traces
$\mathcal{T}^\infty$	: Set of Infinite Execution Traces
$\mathcal{T}^{+\infty}$	: Set of All Finite Or Infinite Execution Traces
$\mathcal{V}_{\text{in}}(c)$	: Set of Program c 's Input Variables
$\mathcal{T}_0^+(c)$	: Set of Program c 's Well-formed Initial Traces.

Each well formed initial trace $\tau_0 \in \mathcal{T}_0^+(c)$ is finite

and every input variable of the program c has an initial value in τ_0 .

Expressions include extended arithmetic expression a with value $n \in \mathbb{N} \cup \{\infty\}$; standard boolean expression b ; and the query expression ψ . A query expression ψ can be either a simple arithmetic expression a , an expression of the form $\chi[a]$ where χ represents a row of an unknown hidden database and a represents an index used

to access a specific column of the row χ , a combination of two query expressions, $\psi \oplus_a \psi$, or query value, α which is a normal form of the query expression. In the normal form, the index for accessing the column of χ is a natural number, i.e., $\chi[n]$. For example, $\chi[3] + 5$ is a query value, denotes the query request that computes the value in the 3rd column of the row χ and then add 5 to it. A query answer can be ∞ if the query request is invalid.

Basic Expression The basic expression is either an extended arithmetic expression, a boolean expression, or a list of expressions.

An extended arithmetic expression can be a constant n denoting natural number or the ∞ symbol, a variable x from some countable set $\mathcal{V}$, binary operation $\oplus_a$ including addition, product, subtraction, division over arithmetic expressions, and also $\max$, $\min$, $\log$, and sign operations.

A boolean expression can be either `true` or `false`, basic boolean connectives between boolean expressions including logical negation, logical and or denoted by $\oplus_b$, and basic comparison $\sim$ between arithmetic expressions including $\leq$, $<$ and $=$, etc.

Query Expression Our language extends the standard while language with primitives for queries, where a query request is specified by a query expression ψ . A query expression contains the necessary information for a query request, for example, $\chi[a]$ represents the values at a certain index a in a row χ of the database. Query expressions combine access to the database with other expressions through operators. For example, $\chi[3] + 5$ represents a query request that asks the value from the 3rd column of each database row χ , adds 5 to each of these values, and then computes the average of these values.

We only consider the linear query request in our program. A linear query computes the average value of a certain query expression over each row of the hidden database. Through mechanisms that intermediate between programmers and the database, the answer to a linear query is computed and returned. For example, if the mechanism computes the empirical answer to the query request $\text{query}(\chi[a])$, then it returns the average value at index a over each row of the hidden database as follows,

$$\text{query}(\chi[\psi]) = \frac{1}{n} \sum_{i=1}^n \chi_i[\psi]$$

In another example, the query expression $\chi[3]$ denotes the computation that obtains the average value in the 3^{rd} column of the hidden database.

Query expressions also combine access to the database with other expressions, for example, $\chi[3] + 5$ represents a query that asks the value from the 3^{rd} column of each database row χ , adds 5 to each of these values, and then computes the average of these values as follows, where n is database χ 's number of rows.

$$\text{query}(\chi[3] + 5) = \frac{1}{n} \sum_{i=1}^n \chi_i[3] + 5$$

Labeled Command Commands are the typical ones from while languages with an additional command $x \leftarrow \text{query}(\psi)$ for query requests which can be used to interrogate the database and compute the linear query corresponding to ψ . Each command is annotated with a label l , which is a natural number to record the location of each command, so that we can uniquely identify them. The label has to be added for every command from the first line to the last line starting from 0 by a programmer. We use the label `in` for the input variables in an initial trace and the label `ex` for annotating the exit of a program when we perform the program

analysis later.

Free Variables and Input Variables We denote by FV the operator $FV : e \rightarrow \mathcal{P}(\mathcal{V})$, which computes the set of free variables in an expression. To be precise, $FV(a)$, $FV(b)$ and $FV(\psi)$ represent the set of free variables in arithmetic expression a , boolean expression b and query expression ψ respectively. All the free variables showing up in c such that they are never defined before using, are actually the input variables of this program. We denote by $\mathcal{V}_{in}$ the universe of all input variables and $V_{in}(c)$ the set of input variables in a given program c . We compute the input variable by taking the union of the free variables in every expression and subtract the set of variables that are assigned in commands.

Labeled Variables The labeled variables and assigned variables are set of variables annotated by a label. We use $\mathcal{LV}$ represents the universe of all the labeled variables and $LV(c) \subseteq \mathcal{LV}$, represents the set of labeled variables in program c , defined in Definition 4. Labeled variable set, $LV(c)$ of a program c is the set of all the labeled variables in every assignment command and query command computed as follows.

Definition 4 (labelled variable set of a program c , $LV(c)$). *For a program c , its labeled variable set, $LV(c)$ is computed as below.*

$$LV(c) \triangleq \begin{cases} \{x^l\} & c = [x \leftarrow e]^l \\ \{x^l\} & c = [x \leftarrow \text{query } (\psi)]^l \\ LV(c_1) \cup LV(c_2) & c = c_1; c_2 \\ LV(c_1) \cup LV(c_2) & c = \text{if } ([b]^l, c_1, c_2) \\ LV(c_w) & c = \text{while } [b]^l \text{ do } (c_w) \end{cases}$$

Every labeled variable in a program is unique, Lemma 4.1.1 states this property. Proof of this lemma is in Appendix A.1.2.

Lemma 4.1.1 (Uniqueness of the Labeled Variables). *Given any program $c \in \mathcal{C}$ with its label variable set $\text{LV}(c)$, every two labeled variables in this set are different.*

$$\forall c \in \mathcal{C}, x^i, y^j \in \text{LV}(c) . i \neq j.$$

We guarantee that even if there are two labeled variables in $\text{LV}(c)$ have the same variable name, such as x^i and x^j , we still have $x^i \neq x^j$ through $i \neq j$.

4.2 TRACE-BASED OPERATIONAL SEMANTICS

We use trace-based operational semantics to track the history of program execution. This operational semantics is defined based on the event and trace with an environment operator as follows.

Event We have two kinds of events: *assignment events* and *testing events*. Each event consists of a quadruple, and we use $\mathcal{E}^{\text{asn}}$ and $\mathcal{E}^{\text{test}}$ to denote the set of all assignment events and testing events, respectively.

$$\begin{aligned} \text{Event } \epsilon ::= & (x, l, v, \bullet) \mid (x, l, v, \alpha) \quad \text{Assignment Event} \\ & \mid (b, l, v, \bullet) \quad \text{Testing Event} \end{aligned}$$

An assignment event tracks the execution of an assignment or a query request and consists of the variable name, the label of the command that generates it, the value assigned to the variable. The fourth element is the normal form of the query request, α if this command is a query request, otherwise a default value $\bullet$.

A testing event tracks the execution of if and while commands and consists of the guard of the command, the label of the command, and the result of evaluating the guard, while the last element is $\bullet$.

The event projection operators π_i projects the i^{th} element from an event: $\pi_i : \mathcal{E} \rightarrow \mathcal{V} \cup \mathcal{B} \cup \mathcal{L} \cup \mathcal{VAL} \cup \mathcal{Q} \cup \{\bullet\}$. Its computation is as follows,

$$\begin{aligned} \pi_1(x, l, v, \bullet) &= x & \pi_1(x, l, v, \alpha) &= x & \pi_1(b, l, v, \bullet) &= b \\ \pi_2(_, l, _, _) &= l & \pi_3(_, _, v, _) &= v & \pi_4(_, _, _, \alpha) &= \alpha & \pi_4(_, _, _, \bullet) &= \bullet \end{aligned}$$

Trace. A *trace* τ is a list of *events* generated when executing specific commands.

$$\text{Trace } \tau ::= [] \mid \tau :: \epsilon$$

We denote by $\mathcal{T}^+$ the set of all finite traces, by $\mathcal{T}^\infty$ the set of all infinite traces, and $\mathcal{T}^{+\infty} = \mathcal{T}^+ \cup \mathcal{T}^\infty$. We use list notation for traces, where $[]$ is the empty trace, and the operator $::$ combines an event and a trace in a new event.

The *trace concatenation* operator, $++ : \mathcal{T}^+ \rightarrow \mathcal{T}^+ \rightarrow \mathcal{T}^+$ concatenates two traces similar to the standard list concatenation operation as follows. The concatenation preserves the order of every element in the two traces.

Definition 5 (Trace Concatenation, $++ : \mathcal{T}^{+\infty} \rightarrow \mathcal{T}^{+\infty} \rightarrow \mathcal{T}^{+\infty}$). *Given two traces $\tau_1 \in \mathcal{T}^{+\infty}, \tau_2 \in \mathcal{T}^{+\infty}$, the trace concatenation operator $++$ is defined as:*

$$\tau_{1++\tau_2} \triangleq \begin{cases} \tau_1 & \tau_2 = [] \vee \tau_1 \in \mathcal{T}^\infty \\ \tau_2 & \tau_1 = [] \vee \tau_2 \in \mathcal{T}^\infty \\ (\tau_{1++\tau'_2}) :: \epsilon & \tau_1 \in \mathcal{T}^+ \wedge \tau_2 \in \mathcal{T}^+ \wedge \tau_2 = \tau'_2 :: \epsilon \end{cases}$$

Environment ρ is the environment operator such that $\rho(\tau)x$ fetches the latest value assigned to x in the trace τ . Since the trace records all the program's evalu-

ations for assignments, the value assigned to variables can be recovered from the trace directly.

Definition 6 (Environment $\rho : \mathcal{T}^{+\infty} \rightarrow \mathcal{V} \rightarrow \mathcal{VAL} \cup \{\perp\}$). *Given a trace $\tau \in \mathcal{T}^{+\infty}$ and a variable x , the environment operator $\rho(\tau)l$ fetches the latest value assigned to x on the trace τ , and $\perp$ if the trace is an infinite trace as below.*

$$\begin{aligned} \rho(\tau::(x, l, v, \bullet))x &\triangleq v & \rho(\tau::(y, l, v, \bullet))x &\triangleq \rho(\tau)x, y \neq x & \rho(\tau::(b, l, v, \bullet))x &\triangleq \rho(\tau)x \\ \rho(\tau::(x, l, v, \alpha))x &\triangleq v & \rho(\tau::(y, l, v, \alpha))x &\triangleq \rho(\tau)x, y \neq x & \rho(\square)x &\triangleq \perp \\ \rho(\tau)x &\triangleq \perp \text{ if } \tau \in \mathcal{T}^\infty \end{aligned}$$

We also define with respect to specific command c its well-formed initial trace, in which every input variable of c is assigned with an initial value.

Definition 7 (Well-formed Initial Trace). *Given a program c , τ is a well-formed initial trace of c if and only if all the input variables of c have an initial value in this trace.*

$$\forall c \in \mathcal{C}, \tau \in \mathcal{T}^+ . \tau \in \mathcal{T}_0^+(c) \iff \forall x \in \mathbf{V}_{\text{in}}(c) . \rho(\tau_0)x \neq \perp$$

We use $\mathcal{T}_0^+(c)$ to denote the set of all well-formed initial traces for program c .

For instance, the initial trace of `twoRounds(k)` example contains the initial value of the input variable k .

Operational Semantics Rules The trace-based operational semantics is described in terms of a small-step evaluation relation $\langle c, \tau \rangle \rightarrow \langle c', \tau' \rangle$ describing how a configuration program-trace evaluates to another configuration program-trace. The rules for the operational semantics are described in Figure 4.2. The rules for assignment and query generate assignment events, and the rules for the while and

if commands generate testing events. The rules for the standard while language constructs correspond to the usual rules extended to deal with traces.

The rule `assn` evaluates a standard assignment $x \leftarrow e$, the expression e is first evaluated by the expression evaluation rule $\langle \tau, e \rangle \Downarrow_e v$, presented below. The expression evaluation result v is used to construct a new event $\epsilon = (x, l, v, \bullet)$, which is attached to the previous trace. We have relations $\langle \tau, a \rangle \Downarrow_a v$ and $\langle \tau, b \rangle \Downarrow_b v$ to evaluate arithmetic expressions and boolean expressions, respectively in Figure 4.1. The $\langle \tau, e \rangle \Downarrow_e v$ is combination of arithmetic and boolean evaluation, and the evaluation of list. In these evaluation rules, $v_1 \oplus_a v_2$ denotes the corresponding semantic mathematic computation of two values and the same for $\log v$ and $\text{sign } v$.

The rules for `if` and `while` both have two cases in which the boolean expressions in the guards are evaluated to true and false, respectively. In these rules, the evaluation of the guard generates a testing event and the trace is updated as well by appending this event.

The only rule that is non-standard is the query rule. When evaluating a query, the query expression ψ is first simplified to its normal form α using an evaluation relation $\langle \tau, \psi \rangle \Downarrow_q \alpha$. This query expression evaluation relation $\langle \tau, \psi \rangle \Downarrow_q \alpha$ is defined by the following rules. I want to highlight here, these evaluation rules do not compute the query answer but only reduce a query expression to its normal form. The normal form α characterizes the linear query that is run against a hidden database through an unknown intermediate mechanism. For example, a query expression $\chi[x] - \chi[x]$ with a trace $\tau = [(x, 0, 1, \bullet)]$ is evaluated into the query value $\chi[1] - \chi[1]$, which is not the answer of this query. It is a linear query waited to be computed by an unknown intermediate mechanism on a hidden database.

$$\begin{array}{c}
 \frac{\langle \tau, a \rangle \Downarrow_a n}{\langle \tau, a \rangle \Downarrow_q n} \qquad \frac{\langle \tau, \psi_1 \rangle \Downarrow_q \alpha_1 \quad \langle \tau, \psi_2 \rangle \Downarrow_q \alpha_2}{\langle \tau, \psi_1 \oplus_a \psi_2 \rangle \Downarrow_q \alpha_1 \oplus_a \alpha_2} \qquad \frac{\langle \tau, a \rangle \Downarrow_a n}{\langle \tau, \chi[a] \rangle \Downarrow_q \chi[n]} \qquad \frac{}{\langle \tau, \alpha \rangle \Downarrow_q \alpha}
 \end{array}$$

$\langle \tau, a \rangle \Downarrow_a v : \text{Trace} \times \text{Extended Arithmetic Expression} \Downarrow_a \text{Value}$			
$\frac{}{\langle \tau, n \rangle \Downarrow_a n}$	$\frac{\rho(\tau)x = v}{\langle \tau, x \rangle \Downarrow_a v}$	$\frac{\langle \tau, a_1 \rangle \Downarrow_a v_1 \quad \langle \tau, a_2 \rangle \Downarrow_a v_2 \quad v_1 \oplus_a v_2 = v}{\langle \tau, a_1 \oplus_a a_2 \rangle \Downarrow_a v}$	
$\frac{\langle \tau, a \rangle \Downarrow_a v' \quad \log v' = v}{\langle \tau, \log a \rangle \Downarrow_a v}$		$\frac{\langle \tau, a \rangle \Downarrow_a v' \quad \text{sign } v' = v}{\langle \tau, \text{sign } a \rangle \Downarrow_a v}$	
$\langle \tau, b \rangle \Downarrow_b v : \text{Trace} \times \text{Boolean Expression} \Downarrow_b \text{Boolean Value}$			
$\frac{}{\langle \tau, \text{false} \rangle \Downarrow_b \text{false}}$	$\frac{}{\langle \tau, \text{true} \rangle \Downarrow_b \text{true}}$	$\frac{\langle \tau, b \rangle \Downarrow_b v' \quad \neg v' = v}{\langle \tau, \neg b \rangle \Downarrow_b v}$	
$\frac{\langle \tau, b_1 \rangle \Downarrow_b v_1 \quad \langle \tau, b_2 \rangle \Downarrow_b v_2 \quad v_1 \oplus_b v_2 = v}{\langle \tau, b_1 \oplus_b b_2 \rangle \Downarrow_b v}$			
$\frac{\langle \tau, a_1 \rangle \Downarrow_a v_1 \quad \langle \tau, a_2 \rangle \Downarrow_a v_2 \quad v_1 \sim v_2 = v}{\langle \tau, a_1 \sim a_2 \rangle \Downarrow_b v}$			
$\langle \tau, e \rangle \Downarrow_e v : \text{Trace} \times \text{Expression} \Downarrow_e \text{Value}$			
$\frac{\langle \tau, a \rangle \Downarrow_a v}{\langle \tau, a \rangle \Downarrow_e v}$	$\frac{\langle \tau, b \rangle \Downarrow_b v}{\langle \tau, b \rangle \Downarrow_e v}$	$\frac{\langle \tau, e_1 \rangle \Downarrow_e v_1 \cdots \langle \tau, e_n \rangle \Downarrow_e v_n}{\langle \tau, [e_1, \dots, e_n] \rangle \Downarrow_e [v_1, \dots, v_n]}$	$\frac{}{\langle \tau, v \rangle \Downarrow_e v}$

Figure 4.1: Arithmetic and boolean expression evaluation rules

The query answer v is the expected value of the function $\lambda\chi.\alpha$ applied to each row of the dataset. We summarize this process with the notation $\text{query}(\alpha) = v$ which we use in the rule `query`. $\text{query}(\alpha)$ can return different answers depending on the hidden database and the mechanism. $\text{query}(\alpha)$ returns the ∞ if the query request is invalid, such as accessing a column that is outside the column range of the hidden database. Once the answer to the query is computed, rule `query` generates an event (x, l, v, α) recording both the query answer v and the query value α . When evaluating the same query request command under the same trace, it is possible to generate events with different query answers. However, the query value α in these events is always the same given the query expression evaluation rules.

As usual, we will use $\rightarrow^*$ for the reflexive and transitive closure of $\rightarrow$. If $\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{0++\tau} \rangle$, then the program's execution terminates and produces a finite execution trace $\tau \in \mathcal{T}^+$. If a program c has some non-terminating behavior, we define its execution as follows.

Definition 8 (Non-terminating and Infinite Trace). *Given a program c and an initial trace $\tau \in \mathcal{T}_0^+(c)$, when c executes with τ , we define the execution of c under τ is non-terminating and produces an infinite trace $\tau' \in \mathcal{T}^\infty$, as $\langle c, \tau_0 \rangle \uparrow^\infty \tau' \in \lim(\uparrow)$ where the limit is defined as follows.*

$$\begin{aligned} \lim(\uparrow) \triangleq & \left\{ (\langle c, \tau \rangle, \tau') \mid c \in \mathcal{C}, \tau \in \mathcal{T}_0^+(c), \tau' \in \mathcal{T}^\infty \wedge \exists \tau_0 \in \mathcal{T}^+, c_0 \in \mathcal{C} . \langle c, \tau \rangle \rightarrow \langle c_0, \tau_0 \rangle \right. \\ & \wedge \forall i \in \mathbb{N}, \exists \tau_i, \tau_{i+1} \in \mathcal{T}^+, \tau'' \in \mathcal{T}^\infty, c_i, c_{i+1} \in \mathcal{C} . \\ & \left. \langle c_i, \tau_i \rangle \rightarrow \langle c_{i+1}, \tau_{i+1} \rangle \wedge \tau' = \tau_{i+1++\tau''} \right\} \end{aligned}$$

We consider as well-formed only the program that has only terminating executions and produces finite traces.

Definition 9 (Well-formed Program). *A program c is a well-formed program, $c \in \mathcal{C}$ if and only if its execution terminates under an arbitrary initial trace and hidden database. We use $\mathcal{C}$ to denote the set of all well-formed programs.*

$$\forall c . c \in \mathcal{C} \iff \forall \tau_0 \in \mathcal{T}_0^+(c) . \exists l' \in \mathcal{L}, \tau \in \mathcal{T}^+ . \langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle$$

Well-formedness is an undecidable property because the program halting is undecidable. For the program that has non-terminating behaviors, it does not produce an output or data analysis result. In this sense, it does not have an intuitive adaptivity, and we analyze the adaptivity only for the well-formed program. In the rest of **PART i**, we implicitly refer to “a well-formed program” when we use the words “a program”. We will extend to non-terminating programs in **PART ii**.

If we observe the operational semantics rules, we can find that no rule will shrink the trace. So we have the Lemma 4.2.1 with proof in Appendix A.1.1, which states that the trace has the property that its length never decreases during the program execution.

Lemma 4.2.1 (Trace Non-Decreasing). *For any program $c \in \mathcal{C}$ and initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, if there exists $\tau \in \mathcal{T}^{+\infty}$ and $c' \in \mathcal{C}$ such that $\langle c, \tau_0 \rangle \rightarrow \langle c', \tau \rangle$ then there exists a trace $\tau' \in \mathcal{T}^{+\infty}$ such that $\tau_{0++\tau'} = \tau$ formally as follows.*

$$\begin{aligned} \forall \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^{+\infty}, c, c' \in \mathcal{C} . \langle c, \tau_0 \rangle \rightarrow \langle c', \tau \rangle \\ \implies \exists \tau' \in \mathcal{T}^{+\infty} . \tau_{0++\tau'} = \tau \end{aligned}$$

$$\boxed{\text{Command} \times \text{Trace} \rightarrow \text{Command} \times \text{Trace}} \qquad \langle c, \tau \rangle \rightarrow \langle c', \tau' \rangle$$

$$\frac{\langle a, \tau \rangle \Downarrow_a v \quad \epsilon = (x, l, v, \bullet)}{\langle [x \leftarrow a]^l, \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau :: \epsilon \rangle} \text{ assn} \qquad \frac{\langle \tau, \psi \rangle \Downarrow_q \alpha \quad \text{query}(\alpha) = v \quad \epsilon = (x, l, v, \alpha)}{\langle [x \leftarrow \text{query}(\psi)]^l, \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau :: \epsilon \rangle} \text{ query}$$

$$\frac{\langle \tau, b \rangle \Downarrow_b \text{true} \quad \epsilon = (b, l, \text{true}, \bullet)}{\langle \text{while } [b]^l \text{ do } (c), \tau \rangle \rightarrow \langle c; \text{while } [b]^l \text{ do } (c), \tau :: \epsilon \rangle} \text{ while-t}$$

$$\frac{\langle \tau, b \rangle \Downarrow_b \text{false} \quad \epsilon = (b, l, \text{false}, \bullet)}{\langle \text{while } [b]^l \text{ do } (c), \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau :: \epsilon \rangle} \text{ while-f}$$

$$\frac{\langle c_1, \tau \rangle \rightarrow \langle c'_1, \tau' \rangle}{\langle c_1; c_2, \tau \rangle \rightarrow \langle c'_1; c_2, \tau' \rangle} \text{ seq1} \qquad \frac{\langle c_2, \tau \rangle \rightarrow \langle c'_2, \tau' \rangle}{\langle [\text{skip}]^l; c_2, \tau \rangle \rightarrow \langle c'_2, \tau' \rangle} \text{ seq2}$$

$$\frac{\langle \tau, b \rangle \Downarrow_b \text{true} \quad \epsilon = (b, l, \text{true}, \bullet)}{\langle \text{if } ([b]^l, c_1, c_2), \tau \rangle \rightarrow \langle c_1, \tau :: \epsilon \rangle} \text{ if-t} \qquad \frac{\langle \tau, b \rangle \Downarrow_b \text{false} \quad \epsilon = (b, l, \text{false}, \bullet)}{\langle \text{if } ([b]^l, c_1, c_2), \tau \rangle \rightarrow \langle c_2, \tau :: \epsilon \rangle} \text{ if-f}$$

Figure 4.2: Trace-based operational semantics rules for `QWhile` language.

Equivalence of the event and expression In order to distinguish if a query's choice is affected by previous values, we need to be able to identify whether two queries are equivalent or not. So that we can identify, when we change the result of one query, whether another query is affected. Since the answer to a query request can be different because of the hidden database and unknown mechanisms, we cannot tell their equivalence based on comparing the query answer. In this sense, we define the equivalence of queries by comparing the query expressions, ψ and quantifying over all possible values of the hidden the database.

Definition 10 (Equivalence of Query Expression). *Two query expressions ψ_1, ψ_2 are equivalent, denoted as $\psi_1 =_q \psi_2$, if and only if*

$$\begin{aligned} \forall \tau \in \mathcal{T}^+ . \exists \alpha_1, \alpha_2 . (\langle \tau, \psi_1 \rangle \Downarrow_q \alpha_1 \wedge \langle \tau, \psi_2 \rangle \Downarrow_q \alpha_2) \\ \wedge (\forall D \in \mathcal{DB}, r \in D . \exists v . \langle \tau, \alpha_1[r/\chi] \rangle \Downarrow_a v \wedge \langle \tau, \alpha_2[r/\chi] \rangle \Downarrow_a v), \end{aligned}$$

where $r \in D$ is a record (row) in the database D . As usual, we denote by $\psi_1 \neq_q \psi_2$ the negation of the equivalence.

Then, when defining the equivalence of two events generated from a query request, instead of checking the query answers in the third location, we check the fourth element, which is the query value.

Definition 11 (Event Equivalence). *Two events $\epsilon_1, \epsilon_2 \in \mathcal{E}$ are equivalent, denoted as $\epsilon_1 = \epsilon_2$ if and only if:*

$$\begin{aligned} \pi_1(\epsilon_1) = \pi_1(\epsilon_2) \wedge \pi_2(\epsilon_1) = \pi_2(\epsilon_2) \\ \wedge \left((\pi_4(\epsilon_1) = \pi_4(\epsilon_2) = \bullet \wedge \pi_3(\epsilon_1) = \pi_3(\epsilon_2)) \right. \\ \left. \vee (\pi_4(\epsilon_1) \neq \bullet \wedge \pi_4(\epsilon_2) \neq \bullet \wedge \pi_4(\epsilon_1) =_q \pi_4(\epsilon_2)) \right) \end{aligned} \tag{4.1}$$

As usual, we denote by $\epsilon_1 \neq \epsilon_2$ the negation of the equivalence.

For the events that are generated from the query request command, as in Equation 4.1(c), their equivalence is determined by the 4th element, which is the query value. Since the equivalence over two query events is defined over the query value equivalence, it is possible that two events have different values of query answer in the third position, but they are equivalent by Definition 10. For example, the events $(x, 0, 1, \chi[1])$ and $(x, 0, 2, \chi[1])$ are equivalent by this definition even though they have different query answers assigned to variable x .

So we have the following Corollaries 4.2.0.1 with proof in Appendix A.1.3. This corollary says that two events that have different values in the third location can still be equivalent if both of them are generated from query request.

Corollary 4.2.0.1. *For any event $\epsilon \in \mathcal{E}$ that is generated from a query request, there exist another event $\epsilon' \in \mathcal{E}$ such that ϵ and ϵ' are equivalent but different in their query results.*

$$\forall \epsilon \in \mathcal{E} . \pi_4(\epsilon) \neq \bullet \implies \exists \epsilon' \in \mathcal{E} . \epsilon = \epsilon' \wedge \pi_3(\epsilon) \neq \pi_3(\epsilon')$$

Then using the equivalence between events, we define the *Belongs To*, $\in$: $\mathcal{E} \rightarrow \mathcal{T}^+ \rightarrow \{\text{true}, \text{false}\}$ and its opposite $\notin$ as follows. It expresses whether an event belongs to a trace.

Definition 12. *(An Event Belongs to A Trace) An event $\epsilon \in \mathcal{E}$ belongs to a trace $\tau \in \mathcal{T}^{+\infty}$, i.e., $\epsilon \in \tau$ are defined as follows:*

$$\epsilon \in \tau \triangleq \begin{cases} \text{true} & \tau = [\epsilon]_{++}\tau' \wedge \epsilon = \epsilon' \\ \epsilon \in \tau' & \tau = [\epsilon']_{++}\tau' \wedge \epsilon \neq \epsilon' \\ \text{false} & \tau = [] \vee \tau \in \mathcal{T}^\infty \end{cases}$$

As usual, we denote by $\epsilon \notin \tau$ that the event ϵ doesn't belong to the trace τ .

Since the equivalence over two events is defined over the query value equivalence, when there is an event belonging to a trace, it is possible that the event showing up in the same trace has a different value of query result in the third position, but they are equivalent by Definition 10. So we have the following Corollaries 4.2.0.2 with proof in Appendix A.1.4.

Corollary 4.2.0.2. *For any event $\epsilon \in \mathcal{E}$ that is generated from a query request and a trace $\tau \in \mathcal{T}^+$ contains this event such that τ can be expressed in the form of $\tau_{1++}[\epsilon]_{++}\tau_2 = \tau$ for some traces $\tau_1, \tau_2 \in \mathcal{T}^+$, then there exist another event $\epsilon' \in \mathcal{E}$ such that $\pi_3(\epsilon) \neq \pi_3(\epsilon')$ and the same τ has a different form $\tau_{1++}[\epsilon']_{++}\tau_2$.*

$$\begin{aligned} \forall \epsilon \in \mathcal{E}, \tau, \tau_1, \tau_2 \in \mathcal{T}^+ . \pi_4(\epsilon) \neq \bullet \wedge \tau_{1++}[\epsilon]_{++}\tau_2 = \tau \\ \implies \exists \epsilon' \in \mathcal{E} . \tau_{1++}[\epsilon']_{++}\tau_2 = \tau \wedge \pi_3(\epsilon) \neq \pi_3(\epsilon') \end{aligned}$$

4.3 EXAMPLES

Example 4.3.1 (Evaluation Trace for The Example Program of Two Rounds Data Analysis). *In the follows, we reuse the same example program `twoRounds(k)` as Figure 2.4(a), the analyst asks in total $k + 1$ queries to the mechanism in two phases.*

```

twoRounds(k)  $\triangleq$ 
   $[a \leftarrow 0]^0; [j \leftarrow k]^1;$ 
  while  $[j > 0]^2$  do
     $\left([x \leftarrow \text{query}(\chi[j] \cdot \chi[k])]^3; [j \leftarrow j - 1]^4; [a \leftarrow x + a]^5\right);$ 
   $[l \leftarrow \text{query}(\chi[k] * a)]^6$ 

```

In the first phase, the analyst asks k queries and stores the answers that are provided by the mechanism. In the second phase, the analyst constructs a new query based on the results of the previous k queries and sends this query to the mechanism. More specifically, I assume

that, in this example, the domain $\mathcal{DB}$ contains at least k numeric attributes, which I index just by natural numbers. The queries inside the while loop correspond to the first phase and compute an approximation of the product of the empirical mean of the first k attributes. The query outside the loop corresponds to the second phase and computes an approximation of the empirical mean where each record is weighted by the sum of the empirical mean of the first k attributes. With the initial trace $\tau_0 = [(k, \text{in}, 2, \bullet)]$, the execution trace generated from this example is

$$\left[\begin{array}{l} (a, 0, 0, \bullet), (j, 1, 2, \bullet), (j > 0, 2, \text{true}, \bullet), (x, 3, v_1, \chi[2] * \chi[2]), (j, 4, 1, \bullet), (a, 5, v_1, \bullet), \\ (j > 0, 2, \text{true}, \bullet), (x, 3, v_2, \chi[1] * \chi[2]), (j, 4, 0, \bullet), (a, 5, v_1 + v_2, \bullet), (j > 0, 4, \text{false}, \bullet), \\ (l, 6, v_3, \chi[2] * (v_1 + v_2)) \end{array} \right].$$

CHAPTER 5

Adaptivity Formalization

In this section, we present the formalization of adaptivity for a given program through three steps using the new methodology proposed in Section 1.2.1.2. Each step corresponds to one of the three procedures of the flow-chart in the “Adaptivity Formalization” block at the middle of Figure 2.3. We summarize them as follows.

1. The first step in Section 5.1 defines the reachability sub-property of the *adaptivity* as a variable *may-dependency* relation based on the trace semantics.
2. In Section 5.2, we define the quantitative sub-property for a program’s *adaptivity* as the weight of each labeled variable. Then we construct a weighted dependency graph namely the *semantics-based dependency graph* of a program c , which carries both sub-properties.
3. Based on the *semantics-based dependency graph*, we define the formal *adaptivity* in definition 23 as a quantitative reachability hyper-property in Section 5.3.

Through two examples in Section 5.4, I show that the formalized adaptivity matches the program’s intuitive *adaptivity*.

5.1 MAY-DEPENDENCY BETWEEN VARIABLES

We are interested in defining a notion of dependencies between program variables since assigned variables are a good proxy to study dependencies between queries— we can recover query requests from variables associated with queries. We consider dependencies that can be generated by either data or control flow.

For example, in the program

$$c_1 = [x \leftarrow \text{query}(\chi[2])]^1; [y \leftarrow \text{query}(\chi[3] + x)]^2$$

the query $\text{query}(\chi[3] + x)$ depends on the query $\text{query}(\chi[2])$ through a *value dependency* via x^1 . Although x is not directly used as the index of χ for accessing a certain column, we still claim there is a value dependency between these two queries.

Conversely, in the program

$$c_2 = [x \leftarrow \text{query}(\chi[1])]^1; \text{if } ([x > 2])^2, [y \leftarrow \text{query}(\chi[2])]^3, [\text{skip}]^4)$$

the query $\text{query}(\chi[2])$ depends on the query $\text{query}(\chi[1])$ via the *control dependency* of the guard of the if command involving the labeled variable x^1 . Our running example does not have if control in the while body, while we have examples in Section 7.1 showing the *control dependency* between queries.

To define the dependency relation between program variables we will consider two events that are generated from the same command, hence they have the same variable name / boolean expression and label but have either different values or different query expressions, captured by the following definition.

Definition 13 (Events Different in the Value). *Two events $\epsilon_1, \epsilon_2 \in \mathcal{E}$ differ in their value, or query value, denoted as $\text{Diff}(\epsilon_1, \epsilon_2)$, if and only if:*

$$\pi_1(\epsilon_1) = \pi_1(\epsilon_2) \wedge \pi_2(\epsilon_1) = \pi_2(\epsilon_2) \tag{5.1a}$$

$$\begin{aligned} &\wedge \left((\pi_4(\epsilon_1) = \pi_4(\epsilon_2) = \bullet \wedge \pi_3(\epsilon_1) \neq \pi_3(\epsilon_2)) \right. \\ &\quad \left. \vee (\pi_4(\epsilon_1) \neq \bullet \wedge \pi_4(\epsilon_2) \neq \bullet \wedge \pi_4(\epsilon_1) \neq_q \pi_4(\epsilon_2)) \right). \end{aligned} \tag{5.1b}$$

Intuitively, the *event differ in value* is a partial negation of the *event equivalence* in Definition 11. Equation 5.1(a) is the same as the first line in Equation 4.1. In Equation 5.1(b), when the event corresponded to a query request, we require the query value in the fourth element to be different regarding the *query equivalence* definition, which is opposite to the last line in Equation 4.1.

Specifically, we first look at the first two elements of two events, as in Equation 5.1(a). The conjunction $\pi_1(\epsilon_1) = \pi_1(\epsilon_2) \wedge \pi_2(\epsilon_1) = \pi_2(\epsilon_2)$ requires that ϵ_1 and ϵ_2 have the same variable name and label. This guarantees that ϵ_1 and ϵ_2 are generated from the same labeled command. In Equation 5.1(b), two kinds of comparisons between the third and fourth elements are for the non-query assignment and query request separately. For events generated from the non-query assignments ($\pi_4(\epsilon_1) =_q \pi_4(\epsilon_2) = \bullet$), we only compare their assigned values through $\pi_3(\epsilon_1) \neq \pi_3(\epsilon_2)$. However, for the events from query requests ($\pi_4(\epsilon_1) \neq \bullet \wedge \pi_4(\epsilon_2) \neq \bullet$), we are comparing their query value in the 4th location via $\pi_4(\epsilon_1) \neq_q \pi_4(\epsilon_2)$ rather than the answer computed from the unknown database and unknown mechanism in the 3rd element. Through this way, when looking into the dependency relation between queries, we are able to check whether the “choice” of a query request is affected by others.

Example. I want to highlight that if two events are both generated from query requests, we are not comparing their query answer in the third position of the event, but the query value in the fourth position. For example, two events $\epsilon_1 = (x, 1, 0, \chi[0] + 1)$ and $\epsilon_2 = (x, 1, 0, \chi[1] + 1)$ are different by our different even though $\pi_3(\epsilon_1) = \pi_3(\epsilon_2)$. We compare only the forth element where $\chi[0] + 1 \neq_q \chi[1] + 1$ by Definition 10. The Equation 5.1(b) capture this by first checking $\pi_4(\epsilon_1) \neq \bullet \wedge \pi_4(\epsilon_2) \neq \bullet$ to guarantee both events are from query request. Then we check

again the forth element $\pi_4(\epsilon_1) \neq_1 \pi_4(\epsilon_2) \neq \bullet$ to guarantee that the two events sent different query requests.

The operator $TL : \mathcal{T}^{+\infty} \rightarrow (\mathcal{P}(\mathcal{L}) \cup \{\infty\})$ gives the set of labels in every event belonging to a trace, whose behavior is defined as follows,

$$\begin{aligned} TL(\tau :: (_, l, _, _)) &\triangleq \{l\} \cup TL(\tau) \quad \text{if } \tau \in \mathcal{T}^+ \\ TL(\tau) &\triangleq \infty \quad \text{if } \tau \in \mathcal{T}^\infty \\ TL([]) &\triangleq \{\} \end{aligned}$$

We return ∞ if the input trace is an infinite trace.

Then we need to define the dependency relation between events in order to define the *variable may-dependency* relation. To define the *event may-dependency*, we first introduce the counter notation, $\text{cnt} : \mathcal{T}^{+\infty} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ to count the occurrence of a label in a trace as below.

Definition 14 (Counter $\text{cnt} : \mathcal{T}^{+\infty} \rightarrow \mathbb{N} \rightarrow (\mathbb{N} \cup \{\infty\})$). *Given a trace $\tau \in \mathcal{T}^{+\infty}$ and a label l , the occurrence of l in τ is computed by $\text{cnt}(\tau, l)$ as follows.*

$$\begin{aligned} \text{cnt}(\tau :: (_, l, _, _), l) &\triangleq \text{cnt}(\tau, l) + 1 & \text{cnt}(\tau :: (b, l, v, \bullet), l) &\triangleq \text{cnt}(\tau, l) + 1 \\ \text{cnt}(\tau :: (x, l, v, \alpha), l) &\triangleq \text{cnt}(\tau, l) + 1 & \text{cnt}(\tau :: (x, l', v, \bullet), l) &\triangleq \text{cnt}(\tau, l), l' \neq l \\ \text{cnt}(\tau :: (b, l', v, \bullet), l) &\triangleq \text{cnt}(\tau, l), l' \neq l & \text{cnt}(\tau :: (x, l', v, \alpha), l) &\triangleq \text{cnt}(\tau, l), l' \neq l \\ \text{cnt}([], l) &\triangleq 0 & \text{cnt}(\tau, l) &\triangleq \infty \text{ if } \tau \in \mathcal{T}^\infty \end{aligned}$$

Again, we still return ∞ if the input trace is an infinite trace.

Definition 15 (Boolean Event Negation). *For a testing event, we define its negation operation as follows,*

$$\forall \epsilon \in \mathcal{E}^{\text{test}}, b \in \mathcal{B}, l \in \mathcal{L}, v \in \{\text{true}, \text{false}\} . \epsilon = (b, l, v, \bullet) \implies \neg \epsilon = (b, l, \neg v, \bullet)$$

We can now define when an event *may depend* on another one.

Definition 16 (Event May-dependency). *An event ϵ_2 is in the event may-dependency relation with an assignment event $\epsilon_1 \in \mathcal{E}^{\text{asn}}$ in a program c with a hidden database D and with a witness trace $\tau \in \mathcal{T}^+$, $\text{DEP}_e(\epsilon_1, \epsilon_2, [\epsilon_1]_{++}\tau_{++}[\epsilon_2], c, D)$ if and only if*

$$\exists \tau_0, \tau_1, \tau' \in \mathcal{T}^+, \epsilon'_1 \in \mathcal{E}^{\text{asn}}, c_1, c_2 \in \mathcal{C} . \text{Diff}(\epsilon_1, \epsilon'_1) \wedge \quad (5.2a)$$

$$(\exists \epsilon'_2 \in \mathcal{E} . \left(\begin{array}{l} \langle c, \tau_0 \rangle \rightarrow^* \langle c_1, \tau_{1++}[\epsilon_1] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon_1]_{++}\tau_{++}[\epsilon_2] \rangle \\ \wedge \langle c_1, \tau_{1++}[\epsilon'_1] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon'_1]_{++}\tau'_{++}[\epsilon'_2] \rangle \\ \wedge \text{Diff}(\epsilon_2, \epsilon'_2) \wedge \text{cnt}(\tau, \pi_2(\epsilon_2)) = \text{cnt}(\tau', \pi_2(\epsilon'_2)) \end{array} \right) \quad (5.2b)$$

$$\vee \left(\begin{array}{l} \exists \tau_3, \tau'_3 \in \mathcal{T}^+, \epsilon_b \in \mathcal{E}^{\text{test}} . \\ \langle c, \tau_0 \rangle \rightarrow^* \langle c_1, \tau_{1++}[\epsilon_1] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon_1]_{++}\tau_{++}[\epsilon_b]_{++}\tau_3 \rangle \\ \wedge \langle c_1, \tau_{1++}[\epsilon'_1] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon'_1]_{++}\tau'_{++}[(\neg \epsilon_b)]_{++}\tau'_3 \rangle \\ \wedge \text{TL}(\tau_3) \cap \text{TL}(\tau'_3) = \emptyset \wedge \text{cnt}(\tau', \pi_2(\epsilon_b)) = \text{cnt}(\tau, \pi_2(\epsilon_b)) \wedge \epsilon_2 \in \tau_3 \wedge \epsilon_2 \notin \tau'_3 \end{array} \right), \quad (5.2c)$$

The first line in Equation 5.2(a) requires that ϵ_1 comes from an assignment command. Then, $\text{Diff}(\epsilon_1, \epsilon'_1)$ guarantees that value assigned to the same variable is different in ϵ'_1 .

The following two parts in Equation 5.2(b) and (c) capture the value dependency and control dependency respectively. As in the literature on non-interference, and following Cousot (2019a), we formulate these dependencies as relational properties, i.e. in terms of two different traces of execution. I force these two traces to differ by using the event ϵ_1 in one and ϵ'_1 in the other. Both Equation 5.2(b) and (c) execute the program two times w.r.t. the different values in ϵ_1 (as line:1 in Equation 5.2(b) and line:2 in Equation 5.2(c)) and ϵ'_1 (as line:2 in Equation 5.2(b)

and line:3 in Equation 5.2(c)). However, the definition checks the difference in the newly generated traces in different ways in line:3 of Equation 5.2(b) and line:4 of Equation 5.2(c) to capture the value dependency and control dependency respectively. This idea is similar to the semantics dependency definition from Cousot (2019a).

For the value dependency, we check whether the change of ϵ_1 also creates a change in the value of ϵ_2 or not. In Equation 5.2(b) line:2, if the newly generated trace, $\tau' + +[\epsilon'_2]$ still contains ϵ_2 as ϵ'_2 , I check the difference on their value in line:3. We additionally check whether the two events appear the same times in the two traces. This is to make sure that they are in the same iteration if the two events are generated from the command of a loop body. If they only differ in their assigned values, i.e., $\text{Diff}(\epsilon_2, \epsilon'_2)$ and they are in the same loop iteration (via $\text{cnt}(\tau, \pi_2(\epsilon_2)) = \text{cnt}(\tau', \pi_2(\epsilon'_2))$), then we define there is a value *may-dependency* relation between ϵ_1 and ϵ_2 .

The Equation 5.2(c) captures the control dependency through observing whether ϵ_2 is disappeared from the second execution traces, $\tau'_{++}[(\neg\epsilon_b)]_{++}\tau'_3$ in line:3. $\epsilon_2 \in \tau_3 \wedge \epsilon_2 \notin \tau'_3$ in Equation 5.2(c) line:4 specifies this disappearance. $\text{cnt}(\tau', \pi_2(\epsilon_b)) = \text{cnt}(\tau, \pi_2(\epsilon_b))$ is used to make sure the two executions are in the same loop iteration as well. Different from Equation 5.2(b) line:3, I use a testing event, ϵ_b here because checking only ϵ_2 's occurrence and the $\text{cnt}(\tau, \pi_2(\epsilon_2)) = \text{cnt}(\tau', \pi_2(\epsilon'_2))$ causes false positives when there is no influence through control dependency. For example in the following program

$$[i \leftarrow 3]^0; [x \leftarrow \text{query}(\chi[0])]^1 \text{ while } [i \geq 0]^2 \text{ do } ([y \leftarrow \text{query}(\chi[i])]^3; [i \leftarrow i - 1]^4)$$

we execute only two iterations of the while loop as follows,

$$\begin{aligned}
& \langle [i \leftarrow 3]^0; [x \leftarrow \text{query}(\chi[0])]^1; \text{while } [i \geq 0]^2 \text{ do } ([y \leftarrow \text{query}(\chi[i])]^3; [i \leftarrow i - 1]^4), \tau_0 \rangle \\
& \rightarrow^* \langle \text{while } [i \geq 0]^2 \text{ do } ([y \leftarrow \text{query}(\chi[i])]^3; [i \leftarrow i - 1]^4), \tau_{1++}[(x, 1, v, \chi[0])] \rangle \\
& \rightarrow^* \langle \text{while } [i \geq 0]^2 \text{ do } ([y \leftarrow \text{query}(\chi[i])]^3; [i \leftarrow i - 1]^4), \\
& \quad \tau_{1++}[(x, 1, v, \chi[0]), (i \geq 0, 2, \text{true}, \bullet), (y, 3, v_0, \chi[0]), (i, 4, 1, \bullet), (i \geq 0, 2, \text{true}, \bullet)] \\
& \quad ++[(y, 3, v_1, \chi[1])] \rangle.
\end{aligned}$$

To find the dependency relation between $(x, 1, v, \chi[0])$ and $(y, 3, v_1, \chi[1])$, we modify the first event and then execute only the first iteration of the while loop as follows,

$$\begin{aligned}
& \langle \text{while } [i \geq 0]^2 \text{ do } ([y \leftarrow \text{query}(\chi[i])]^3; [i \leftarrow i - 1]^4), \tau_{1++}[(x, 1, v', \chi[0])] \rangle \\
& \rightarrow^* \langle \text{while } [i \geq 0]^2 \text{ do } ([y \leftarrow \text{query}(\chi[i])]^3; [i \leftarrow i - 1]^4), \\
& \quad \tau_{1++}[(x, 1, v', \chi[0]), (i \geq 0, 2, \text{true}, \bullet), (y, 3, v_1, \chi[0])] \\
& \quad ++[] \rangle.
\end{aligned}$$

Without looking into a testing event, we can always say $(y, 3, v_1, \chi[1]) \notin []$ and they have event may-dependency relation. However, there does not exist any dependency relation between $(x, 1, v, \chi[0])$ and $(y, 3, v_1, \chi[1])$. For this reason, we use the presence of a test event whose value is affected by the change in ϵ_1 to guarantee that the influence goes through a control flow in the first step. Because the control dependency can only be passed through the guard of if or while command, this guard must be evaluated into two different values (ϵ_b and $\neg\epsilon_b$) in the two executions.

I show the following three examples to illustrate the event may-dependency relation.

Example I. In the first example, we have two assignment commands and the

second one is a query request.

$$[x \leftarrow 1]^0; [y \leftarrow \text{query}(\chi[x]) \times \chi[0]]^1.$$

I want to highlight in this example, the dependency relation is defined by analyzing whether the “choice” of the query request is affected but not the “answer” of the query request. With an arbitrary initial trace, execution of this program generates two events $\epsilon_1 = (x, 0, 1, \bullet)$ and $\epsilon_2 = (y, 1, v, \chi[1] \times \chi[0])$ and the witness execution trace $[(x, 0, 1, \bullet), (y, 1, v, \chi[1] \times \chi[0])]$, where $v = \text{query}(\chi[1] \times \chi[0])$ is the query answer computed by an unknown mechanism using a hidden database. By Definition 16, we modify the value assigned to x into 0 and execute the second command. Then we have another execution trace $[(x, 0, 0, \bullet), (y, 1, v', \chi[0] \times \chi[0])]$. Since $\epsilon'_2 = (y, 1, v', \chi[0] \times \chi[0])$ and $\chi[0] \times \chi[0] \neq_q \chi[1] \times \chi[0]$ by Definition 10. Then there is an event may-dependency relation, $\text{DEP}_e(\epsilon_1, \epsilon_2, [(x, 0, 1, \bullet), (y, 1, 0, \chi[1] \times \chi[0])], c, D)$.

Example II. In the following example similar to the previous one, the second query request is $[y \leftarrow \text{query}(\chi[x]) \times 0]^1$.

$$[x \leftarrow 1]^0; [y \leftarrow \text{query}(\chi[x]) \times 0]^1$$

Now with a similar witness execution trace $[(x, 0, 1, \bullet), (y, 1, v, \chi[1] \times 0)]$ and two events $\epsilon_1 = (x, 0, 1, \bullet)$ and $\epsilon_2 = (y, 1, v, \chi[1] \times 0)$ where $v = \text{query}(\chi[1] \times 0)$ is the query answer computed by an unknown mechanism using a hidden database, we cannot find any ϵ'_1 that affect the choice of ϵ_2 . Because for any value v where $\epsilon'_1 = (x, 0, v, \bullet)$, we always have the execution trace $[(x, 0, v, \bullet), (y, 1, v', \chi[v] \times 0)]$ and have $\chi[v] \times 0 =_q \chi[1] \times 0$ by Definition 10. So ϵ_1 and ϵ_2 are not in this event may-dependency relation in this definition.

Example III. The last example has an if command.

$$[x \leftarrow \text{query}(\chi[1] \max 1)]^0; \text{ if } ([0 < x]^1, [y \leftarrow \text{query}(\chi[2])]^2, [\text{skip}]^3)$$

Since the mechanism and the hidden database are unknown, the answer of the first query request $[x \leftarrow \text{query}(\chi[1] \max 1)]^0$, could be different. Also, the answer of the first query in x is used in the guard of the if command, which decides whether the second query request $[y \leftarrow \text{query}(x) \times 0]^2$ is executed. In this sense, intuitively the choice of the second query request command is affected by the answer to the first one. This definition captures this dependency through a witness execution trace $[(x, 0, v, \chi[1] \max 1), (0 > x, 1, \text{true}, \bullet), (y, 2, v', \chi[2])]$ and two events $\epsilon_1 = (x, 0, v, \chi[1] \max 1)$ and $\epsilon_2 = (y, 2, v', \chi[2])$ by taking $v > 0$. Then by modifying ϵ_1 into $(x, 0, 0, \chi[1] \max 1)$, there is another execution trace $[(x, 0, v, \chi[1] \max 1), (0 > x, 1, \text{false}, \bullet)]$. Since ϵ_2 disappeared in the second execution trace, our event may-dependency definition captures this dependency relation.

We can now extend the dependency relation to variables by considering all the assignment events generated during the program's execution.

Definition 17 (Variable May-dependency). *A variable $x_2^{l_2} \in \text{LV}(c)$ may-depend on the variable $x_1^{l_1} \in \text{LV}(c)$ in a program c , $\text{DEP}_{\text{var}}(x_1^{l_1}, x_2^{l_2}, c)$, if and only if*

$$\exists \epsilon_1, \epsilon_2 \in \mathcal{E}^{\text{asn}}, \tau \in \mathcal{T}^+ . \pi_1(\epsilon_1)^{\pi_2(\epsilon_1)} = x_1^{l_1} \wedge \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)} = x_2^{l_2} \wedge \text{DEP}_{\text{e}}(\epsilon_1, \epsilon_2, \tau, c)$$

From the definition, a labeled variable $x_2^{l_2}$ may depend on another labeled variable $x_1^{l_1}$ in a program c , as long as there exist two assignment events ϵ_1 (for $x_1^{l_1}$) and ϵ_2 for $x_2^{l_2}$ satisfy the *event may-dependency* relation under a witness trace τ . Notice that in the definition above I can also have that the two variables are the same, this allows us to capture self-dependencies.

Example I. Below is an example to better understand this definition.

$$[x \leftarrow 0]^0; [y \leftarrow x]^1.$$

In this example, we have two assignment commands without query requests as follows. Intuitively, a result of the first command is always 0, and the same for the second command, and the second command depends on the constant 0 rather than variable x . However, I aim to capture the dependency relation passing through variables, so there is still a variable may-dependency relation between the two labeled variable x^0 and y^1 . Specifically, execution of this program with arbitrary initial trace generates two events $\epsilon_1 = (x, 0, 0, \bullet)$ and $\epsilon_2 = (y, 0, 0, \bullet)$. Then we can have the witness execution trace be $[(x, 0, 0, \bullet), (y, 1, 0, \bullet)]$ and modify the value assigned to x into 1. Continuously executing the second assignment command gives us another execution trace $[(x, 0, 1, \bullet), (y, 1, 1, \bullet)]$. Now we have $\epsilon'_2 = (y, 1, 1, \bullet)$ and $\text{Diff}(\epsilon_2, \epsilon'_2)$ by Definition 13. Then there is an event may-dependency relation between ϵ_1 and ϵ_2 by Definition 16.

5.2 SEMANTICS-BASED DEPENDENCY GRAPH

We can now define the *semantics-based dependency graph* for a well-formed program c . We want this graph to carry both the quantitative sub-property and the reachability sub-property (via *variable may-dependency*).

Query Variables. In order to track the adaptivity with respect to the query, we also define the set of query variables for a program c , $\text{QV}(c)$ as below. It is the set of query-associated variables (in query request assignments) that contains all labeled variables in c 's query requests, and $\text{QV}(c) \subseteq \text{LV}(c)$.

Definition 18 (Query Variables a program c , $\text{QV}(c)$). *Given a program c , its query variables $\text{QV}(c)$ is the set of variables set to the result of a query in the program. It is defined as follows:*

$$\text{QV}(c) \triangleq \begin{cases} \{\} & c = [x \leftarrow e]^l \\ \{x^l\} & c = [x \leftarrow \text{query}(\psi)]^l \\ \text{QV}(c_1) \cup \text{QV}(c_2) & c = c_1; c_2 \\ \text{QV}(c_1) \cup \text{QV}(c_2) & c = \text{if } ([b]^l, c_1, c_2) \\ \text{QV}(c_w) & c = \text{while } [b]^l \text{ do } (c_w) \end{cases}$$

Weight of Labeled Variables. Since we want to define the *adaptivity* as a *quantitative-reachability* hyper-property through constructing the semantics-based dependency graph, the *variable may-dependency* relation and the query variables are not enough. We also want to have the *quantitative* sub-property encoded on this graph. In this sense, we define the weight of every labeled variable in a program as follows.

Definition 19 (Weight of Labeled Variable). *For a labeled variable x^l in a program c , $f(c, x^l) : \mathcal{T}_0^+(c) \rightarrow (\mathbb{N})$ is a weight function of this labeled variable x^l if and only if,*

$$\forall \tau_0 \in \mathcal{T}_0^+(c), \tau' \in \mathcal{T}^+, l' . \\ \langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau'} \rangle \wedge w(\tau_0) = \text{cnt}(\tau', l).$$

Specifically, for every initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, the weight for a labeled variable x^l under this initial trace, $w(\tau_0)$ gives the number of times that the command with the label l is evaluated. In other words, it tracks for each vertex x^l its visiting times when the program c is executing under the initial trace τ_0 . The weight functions are total and with range $\mathbb{N}$. As discussed above in Definition 9, since a well-formed program c is always terminating, we have for any initial trace $\tau_0 \in \mathcal{T}_0^+(c)$ a termi-

nating execution $\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]', \tau_{0++\tau'} \rangle$ which produce a finite execution trace $\tau \in \mathcal{T}^+$. The visiting times are computed by the counter operator $\text{cnt}(\tau', l)$ by counting the occurrence of the label l in τ' .

If the program is not a well-formed program by Definition 9, the program does not terminate. As a result, every labeled variable of this program does not have a weight function.

Example. As an instance, in Figure 2.4(b), the weight of the labeled variable x^3 is $\lambda\tau_0 . \rho(\tau_0)k$, i.e. a function of type $\mathcal{T}_0^+(\text{twoRound}(k)) \rightarrow \mathbb{N}$. $\rho(\tau_0)k$ fetch the initial value assigned to variable k from the initial trace τ_0 . This function means when we execute the $\text{twoRound}(k)$ program under input τ_0 , $\langle \text{twoRound}(k), \tau_0 \rangle \rightarrow^* \langle [\text{skip}]', \tau_{0++\tau'} \rangle$, we have an execution trace τ' . Then $w_k(\tau_0)$ outputs the occurrences of the label 3 in τ' , which is the same as the iteration numbers of the loop, i.e., the value of k in the initial trace, $\rho(\tau_0)k$.

Semantics-based Dependency Graph. Then, we use the *variable may-dependency* relation, the query variables and the *weight function* of each labeled variable to define the semantics-based dependency graph for a well-formed program, and we define as below the weight of each vertex on this graph as the *quantitative* sub-property.

Definition 20 (Semantics-based Dependency Graph). *For a well-formed program $c \in \mathcal{C}$, its semantics-based dependency graph has four components as follows,*

$$\mathbf{G}_{\text{trace}}(c) \triangleq (\mathbf{V}_{\text{trace}}(c), \mathbf{E}_{\text{trace}}(c), \mathbf{W}_{\text{trace}}(c), \mathbf{Q}_{\text{trace}}(c)).$$

Each component is defined as below,

$$\begin{array}{ll}
\text{Vertices} & \mathbf{V}_{\text{trace}}(c) := \{x^l \mid x^l \in \text{LV}(c)\} \\
\text{Edges} & \mathbf{E}_{\text{trace}}(c) := \{(x^i, y^j) \mid x^i, y^j \in \text{LV}(c) \wedge \text{DEP}_{\text{var}}(x^i, y^j, c)\} \\
\text{Weights} & \mathbf{W}_{\text{trace}}(c) := \{(x^l, w) \mid w : \mathcal{T}_0^+(c) \rightarrow \mathbb{N} \wedge x^l \in \text{LV}(c) \wedge \\
& \quad \forall \tau_0 \in \mathcal{T}_0^+(c), \tau' \in \mathcal{T}^+, l' . \\
& \quad \langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau'} \rangle \wedge w(\tau_0) = \text{cnt}(\tau', l')\} \\
\text{Query} & \mathbf{Q}_{\text{trace}}(c) := \{(x^l, n) \mid x^l \in \text{LV}(c) \\
\text{Annotations} & \wedge n = 1 \Leftrightarrow x^l \in \text{QV}(c) \wedge n = 0 \Leftrightarrow x^l \notin \text{QV}(c)\},
\end{array}$$

A semantics-based dependency graph $\mathbf{G}_{\text{trace}}(c) = (\mathbf{V}_{\text{trace}}(c), \mathbf{E}_{\text{trace}}(c), \mathbf{W}_{\text{trace}}(c), \mathbf{Q}_{\text{trace}}(c))$ is well-formed if and only if $\{x^l \mid (x^l, w) \in \mathbf{W}_{\text{trace}}(c)\} = \mathbf{V}_{\text{trace}}(c)$.

The only situation where $\{x^l \mid (x^l, w) \in \mathbf{W}_{\text{trace}}(c)\} \neq \mathbf{V}_{\text{trace}}(c)$ is that the program has non-terminating execution. In this situation, by Definition 19 we know every labeled variable of this program does not have a weight function. As a result, the weight set is empty.

Below we discuss in detail the four components of the semantics-based dependency graph for a program c .

1. The vertices set, $\mathbf{V}_{\text{trace}}(c)$ contains all its labeled variables, $\text{LV}(c)$ which are statically collected.
2. $\mathbf{Q}_{\text{trace}}(c)$ contains the *query annotation* for every vertex $x^l \in \mathbf{V}_{\text{trace}}(c)$. It indicates whether x^l comes from a query request (1) or not (0) by checking if the labeled variable x^l of the vertex is in $\text{QV}(c)$.
3. Edges in $\mathbf{E}_{\text{trace}}(c)$ are all directed. They are built from the *variable may-dependency* relation, i.e. $\text{DEP}_{\text{var}}(x^i, y^j, c)$ in Definition 17 between two labeled variables.

There is an edge from y^j to x^i if and only if they are in the *variable may-dependency* relation, $\text{DEP}_{\text{var}}(x^i, y^j, c)$. This is the key definition in order to formalize the intuitive *adaptivity*. We use this to ingredient the *reachability* sub-property of *adaptivity* into this graph.

4. The weight function in $\text{W}_{\text{trace}}(c)$ for each vertex (also a labeled variable), $w : \mathcal{T}_0^+(c) \rightarrow \mathbb{N}$ maps from an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$ to a natural number as defined in Definition 19.

The main novelty of the semantics-based dependency graph is the combination of the quantitative and reachability sub-properties. It can tell both the dependency between queries via the directed edge, and the times they depend on each other via the weight. Then we can utilize them, together with the query annotation to precisely define the *adaptivity* for a program.

5.3 ADAPTIVITY

We can now define the adaptivity of a program formally. This notion is formulated in terms of an initial trace, specifying the value of the input variables, as the walk on the graph $\text{G}_{\text{trace}}(c)$, which has the largest number of query requests.

Definition 21 (Walk). *Given a well-formed program c with its semantics-based dependency graph $\text{G}_{\text{trace}}(c) = (\text{V}_{\text{trace}}, \text{E}_{\text{trace}}, \text{W}_{\text{trace}}, \text{Q}_{\text{trace}})$ of a program c , a walk k is a function that maps an initial trace τ_0 to a sequence of vertices $(v_1, \dots, v_n)$ for which there is a sequence of edges $(e_1 \dots e_{n-1})$ satisfying*

- $e_i = (v_i, v_{i+1}) \in \text{E}_{\text{trace}}$ for every $1 \leq i < n$,
- and v_i appears in $(v_1, \dots, v_n)$ at most $w_i(\tau_0)$ times for every $v_i \in \text{V}_{\text{trace}}$ and $(v_i, w_i) \in \text{W}_{\text{trace}}$.

$k(\tau_0) = (v_1, \dots, v_n)$ and we denote by $\mathcal{WK}(\mathbf{G}_{\text{trace}}(c))$ the set of all the walks k in $\mathbf{G}_{\text{trace}}(c)$.

Because for the adaptivity we are interested in the dependency between queries, we calculate a special “length” of a walk, the *query length*, by counting only the vertices corresponding to queries.

Definition 22 (Query Length). *Given the semantics-based dependency graph $\mathbf{G}_{\text{trace}}(c)$ of a well-formed program c , and a walk $k \in \mathcal{WK}(\mathbf{G}_{\text{trace}}(c))$ the query length of k is a function $\text{len}^q(k) : \mathcal{T}_0^+(c) \rightarrow \mathbb{N}$ that gives the number of vertices that correspond to query variables in the vertex sequence of $k(\tau_0)$ for any initial trace $\tau_0 \in \mathcal{T}_0^+(c)$ as follows,*

$$\text{len}^q(k) = \lambda \tau_0 . |(v \mid v \in (v_1, \dots, v_n) \wedge (v, 1) \in \mathbf{Q}_{\text{trace}}(c) \wedge k(\tau_0) = (v_1, \dots, v_n))|,$$

where the notation $|(\dots)|$ gives the number of vertices in a sequence.

Then we define for a well-formed program its adaptivity as a function as below.

Definition 23 (Adaptivity of a Program). *Given a well-formed program c , its adaptivity $A(c)$ is a function $A(c) : \mathcal{T}_0^+(c) \rightarrow \mathbb{N}$ such that for an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$,*

$$A(c) = \lambda \tau_0 . \max \{ \text{len}^q(k)(\tau_0) \mid k \in \mathcal{WK}(\mathbf{G}_{\text{trace}}(c)) \}$$

As discussed above, programs that has non-terminating behaviors do not have the intuitive adaptivity. So we define the adaptivity only for the well-formed program.

5.4 EXAMPLES

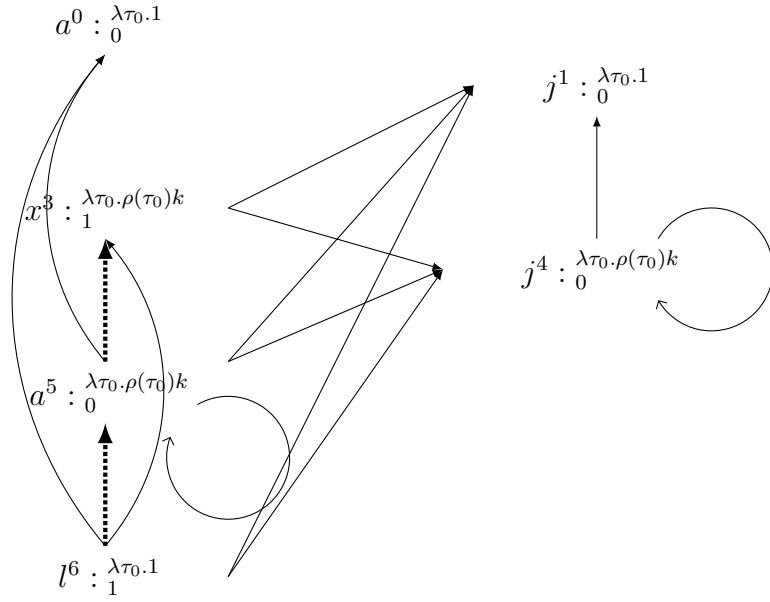
Two examples are presented as follows, illustrating the formal adaptivity definition. The first one illustrates the key concept of this definition through the same walk through example as above, the second one shows the limitation of this definition.

Example 5.4.1 (Running Example - twoRounds). *The semantics-based dependency graph of the twoRounds(k) program we gave in Figure 5.1(a) is described in Figure 5.1(b) (we use dashed arrows for two edges that will be highlighted in the next step, for the moment these can be considered similar to the other edges—i.e. solid arrows). We have all the variables that are assigned in the program with their labels, and edges representing dependency relations between them. For example, we have two edges (l^6, a^5) and (a^5, x^3) describing the dependency between the variables assigned by queries. The vertices l^6 and x^3 are the only ones with query annotation 1 (the subscript), since they are the only two variables that are in assignments involving queries. Notice that the graph contains cycles—in this example it contains two self-loops. These cycles capture the fact that the variables a^5 and j^4 are updated at every iteration of the loop using their previous value. Cycles are essential to capture mutual dependencies like the ones that are generated in loops. Adaptivity is a quantitative notion, so capturing this form of dependencies is not enough. This is why we also use weights. The weight of a vertex is a function that given an initial state returns a natural number representing the number of times the assignment corresponding to a vertex is visited during the program execution starting in this initial state. For example, the vertex l^6 has weight $\lambda\tau.1$ since for every initial state τ the corresponding assignment will be executed one time, the vertex a^5 on the other hand has weight $\lambda\tau.\rho(\tau)k$ since the corresponding assignment will be executed a number of times that correspond to the value of k in the initial state τ , and ρ is the operator reading value of k from τ .*

Example 5.4.2 (Single Adaptivity Round Example). *The program's adaptivity definition in our formal model, (in Definition 23) comes across an over-approximation when capturing the program's intuitive adaptivity rounds. It results from the difference between its weight calculation and the variable may-dependency definition. It occurs when the weight is computed over the traces different from the traces used in witnessing the variable may-dependency relation.*

$\text{towRounds}(k) \triangleq$
 $[a \leftarrow 0]^0; [j \leftarrow k]^1;$
 $\text{while } [j > 0]^2 \text{ do } \left([x \leftarrow \text{query}(\chi[j] \cdot \chi[k])]^3; [j \leftarrow j - 1]^4; [a \leftarrow x + a]^5 \right);$
 $[l \leftarrow \text{query}(\chi[k] * a)]^6$

(a)



(b)

Figure 5.1: (a) The same running example program `towRounds(k)` as Figure 2.4(a). (b) The corresponding semantics-based dependency graph.

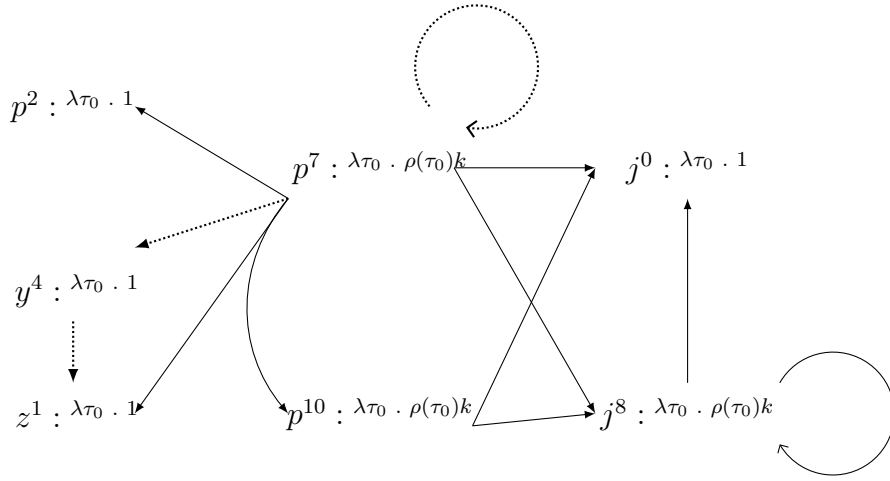
The program `multiRoundsS(k)` in Figure 5.2(a) demonstrates this over-approximation. It is a variant of the multiple rounds strategy with input k . In each iteration, the query request $[p \leftarrow \text{query}(\chi[y] + p)]^7$ is based on value stored in p and y from previous iteration. Differ from Example 7.1.1, only the query answer from the $(k - 2)^{\text{th}}$ iteration is used in the query request, $[p \leftarrow \text{query}(\chi[y] + p)]^7$ of the next $(k - 1)^{\text{th}}$ iteration. In all the other iterations, $j \neq (k - 2)$, the if-control goes to the first branch p 's value is reset by the constant 0 in command $[p \leftarrow 0]^9$. In this way, all the query answers stored in p are erased and are not used in the query request at the next iteration, except the one at the $(k - 2)^{\text{th}}$ iteration. Intuitively, when $k \geq 2$, only the $[p \leftarrow \text{query}(\chi[y] + p)]^7$ in the $(k - 1)^{\text{th}}$ iteration depends on the query in $(k - 2)^{\text{th}}$ iteration and the adaptivity round is 2. When $k = 0$, the program does not go into the loop and there is no dependency between any query request. When $k = 1$, the program goes to the first branch in the first if-control with guard $[k = 1]^3$. In this case, the second query request $[y \leftarrow \text{query}(z)]^4$ depends on the first one, $[z \leftarrow \text{query}(0)]^1$ and then the next query in the loop, $[p \leftarrow \text{query}(\chi[y] + p)]^7$ depends on the second one. Intuitively, the adaptivity is 3. However, our adaptivity definition fails to realize that there is only a dependency relation between p^7 to itself at the $(k - 2)^{\text{th}}$ iteration. As shown in the semantics-based dependency graph in Figure 5.2(b), there is a cycle on p^7 representing the existence of the Variable May-Dependency from p^7 on itself. Weight of this vertex is $\lambda_{\tau_0} \cdot \rho(\tau_0)k$, which returns the evaluation times of the command $[p \leftarrow \text{query}(\chi[y] + p)]^7$ during the program execution under the initial trace τ_0 . Since the command $[p \leftarrow \text{query}(\chi[y] + p)]^7$ will always be evaluated the same time as the loop iteration numbers, i.e. k , the weight function returns $\rho(\tau_0)k$. However, $\rho(\tau_0)k$ is the total number that this command is evaluated, rather than the number of the evaluations in which this command depends on other query requests. As a result, the walk with the longest query length is $p^7 \rightarrow \dots \rightarrow p^7 \rightarrow y^4 \rightarrow z^1$ with the vertex p^7 visited $\rho(\tau_0)k$ times, as the dotted arrows. The adaptivity based on this walk is $\lambda_{\tau_0} \cdot \rho(\tau_0)k + 2$, which is

```

multiRoundsS(k)  $\triangleq$ 
 $[j \leftarrow 0]^0; [z \leftarrow \text{query}(0)]^1; [p \leftarrow 0]^2;$ 
if  $([k = 1]^3, [y \leftarrow \text{query}(z)]^4, [\text{skip}]^5);$ 
while  $[j \neq k]^6$  do (
   $[p \leftarrow \text{query}(\chi[y] + p)]^7;$ 
  if  $([j \neq k - 2]^8, [p \leftarrow 0]^9, [\text{skip}]^{10})$ 
   $[j \leftarrow j + 1]^{11};$ )

```

(a)



(b)

Figure 5.2: (a) The loop example with single adaptivity rounds. (b) The corresponding semantics-based dependency graph.

expected to be 0 when $k = 0$ and 2 when $k \geq 2$.

CHAPTER 6

Program Analysis for Adaptivity

In this section, we introduce in detail our static program analysis algorithm, Adapt-Fun for estimating the *adaptivity* of a program, which corresponds to third part at the bottom of Figure 2.3. It is presented in the following sections.

Chapter Outline In order to give a sound and precise approximation of the formal adaptivity from Section 5 efficiently, the new static program analysis framework is developed through four steps analysis in two phases similar to the semantics-based adaptivity formalization procedure.

- The first phase estimates the *semantics-based* dependency graph in two steps.
 1. In Section 6.3.2, we first analyze the *reachability* sub-property through the data and control flow analysis, and reaching definition techniques, and estimate the *variable may-dependency* relation.
 2. In Section 6.3.3, we analyze the *quantitative* sub-property through a program complexity analysis technique to estimate the weight of each vertex on the *semantics-based dependency graph*.
 3. Then in Section 6.4, we construct the *estimated dependency graph* encoding two estimated results above, together with the statically collected vertices and query annotations.
- The second phase in Section 6.5 computes program's adaptivity. Specifically, through a newly designed algorithm based on graph theory, we compute a sound adaptivity upper bound.

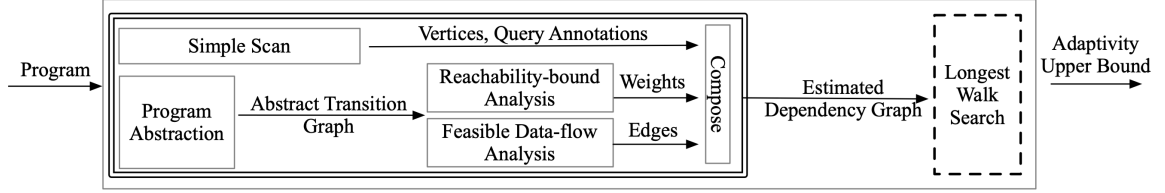


Figure 6.1: The overview of AdaptFun

6.1 ALGORITHM GUIDELINE

To have a sound and accurate upper bound on the adaptivity of a program c , we design a program analysis framework named AdaptFun. This framework composes two algorithms as shown in the double-stroke box and the dashed box in Figure 6.1. The first algorithm in the double-stroke box statically estimates the *quantitative* and *reachability* sub-properties, and produces an estimated *data-dependency graph* for a program. The second algorithm in the dashed box is a longest-walk estimation algorithm, which computes the upper bound on the program’s *adaptivity* over the estimated graph. Below is the outline of the AdaptFun.

1. **Graph Estimation** Because adaptivity is defined over a program’s *semantics-based dependency graph* (in Definition 20), this algorithm first estimates this graph in Section 6.4. Specifically, we estimate the four components of this graph in two steps and then compose them into an estimated dependency graph in the last step. The steps are summarized as follows.

(a) **Vertex and Query Annotation Estimation** Vertices and query annotations in this graph are the assigned variables with unique labels. These are extracted directly from the program as in Section 6.2.

(b) **Edge and Weight Estimation**

This step estimates the edge and weight for a quantitative dependency

graph. It combines the control, data-flow analysis algorithms, and a loop-bound inference algorithm. There are three computation steps in this algorithm.

Abstract Control Flow Graph In order to perform the dependency analysis and quantitative analysis, this step first generates an *abstract transition graph* for a program in Section 6.3.1.

Edges Estimation via Data and Control Flow Analysis The step is presented in Section 6.3.2, which performs both control flow and data flow analysis over the *abstract transition graph*. Specifically, this step first estimates the *variable may-dependency* relation between each pair of the labeled variables in a program by considering both the control flow and data flow, namely the `flowsTo` relation. Then, this `flowsTo` relation is used to construct the edge between each pair of vertices.

Weights Estimation via Quantitative Analysis. This step is presented in Section 6.3.3. It performs over the same *abstract transition graph* and computes the upper bound on the maximal visiting times of each labeled variable for a program. It estimates the reachability bound for every vertex over the *abstract transition graph*, and this reachability bound is used to estimate the maximal visiting times of each labeled variable in a program and the weight of the corresponding vertex.

- (c) **Graph Estimation.** In Section 6.4, we construct the final approximated graph, named *estimated dependency graph* by simply composing the four estimated ingredients. Overall, this *estimated dependency graph* has a similar topology structure as the *semantics-based dependency graph*. It has the same vertices and query annotations but approximated edges and

weights.

2. **Adaptivity Computation.** Likewise, the adaptivity in Definition 23, the static estimation on the *adaptivity* also relies on finding a walk in the *estimated dependency graph*. We discuss some challenges in finding the ‘appropriate’ walk in the graph, and how our algorithm responds to these challenges in Section 6.5.

6.2 GRAPH VERTEX AND ANNOTATION ESTIMATION

Given a program c , the set of vertices $V_{\text{est}}(c)$ and query annotations $Q_{\text{est}}(c)$ of the *estimated dependency graph* can be computed by simply scanning the program c . These sets can be computed precisely and correspond to the same sets in the semantics-based dependency graph. This means that $G_{\text{est}}(c)$ has the same underlying vertex structure as the semantics-based graph $G_{\text{trace}}(c)$.

Vertex Estimation The first component of the *estimated dependency graph* is the vertex set, which is identical to the *semantics-based dependency graph*. Every vertex is an assigned variable in the program, which comes from an assignment command or query request with a unique label. These vertices are collected by statically scanning the program, like what we do for vertices of the *semantics-based dependency graph*, as follows.

$$V_{\text{est}}(c) \triangleq LV(c)$$

Query Annotation Estimation The static scanning of the programs also tells us whether a vertex is a variable assigned by the query request. Identically to the *semantics-based dependency graph*, $Q_{\text{est}}(c)$ is a set of pairs $Q_{\text{est}}(c) \in \mathcal{P}(LV(c) \times \{0, 1\})$ mapping each $x^l \in V_{\text{est}}(c)$ to either 0 or 1. 1 denotes x^l is a member of $QV(c)$, which

is the set of program's variables assigned with query requests, and 0 means x^l not in this set. It is defined formally below as identical to the $Q_{\text{trace}}(c)$.

$$Q_{\text{est}}(c) = \left\{ (x^l, n) \in \text{LV}(c) \times \{0, 1\} \left| \begin{array}{l} x^l \in \text{LV}(c) \\ \wedge n = 1 \Leftrightarrow x^l \in \text{QV}(c) \wedge n = 0 \Leftrightarrow x^l \notin \text{QV}(c) \end{array} \right. \right\}.$$

6.3 GRAPH EDGE AND WEIGHT ESTIMATION

The edges and weight are estimated through a combined control, data flow, and loop bound analysis. Because these analyses are all performed on basis of the *Abstract Transition Graph* of the program, we first introduce how to generate this *abstract transition graph* in Section 6.3.1. Then Section 6.3.2 presents the edge estimation based on a combined control and data flow analysis algorithm, and Section 6.3.3 computes the weight through a loop bound analysis.

6.3.1 Abstract Transition Graph

This section shows how to generate the abstract transition graph $\text{absG}(c)$ of a program c through constructing its vertices and edges.

An *Abstract Transition Graph*, $\text{absG}(c)$ for a program c is composed of a vertex set $\text{absV}(c)$ and an edge set $\text{absE}(c)$, $\text{absG}(c) \triangleq (\text{absV}(c), \text{absE}(c))$. Specifically, every vertex $l \in \text{absV}(c)$ is the label of a labeled command in c , which is unique. We also call the unique label a program point. Each edge $(l \xrightarrow{dc} l') \in \text{absE}(c)$ is an abstract transition between two program points l, l' . There is an edge from l to l' if and only if the command with label l' can execute right after the execution of the command with label l . Each edge is annotated by a constraint $dc \in \mathcal{DC}^\top$, which is generated from the command with label l . This constraint describes the abstract execution

of the command with l . Below we present in detail how to compute them and construct this graph.

Abstract Control Flow Graph Vertices Construction Every vertex $l \in \text{absV}(c)$ corresponds to a program point l , which is a unique label of command in this program. Concretely, the vertices of this graph is the set of c 's labels with the exit label ex formally as follows,

$$\text{absV}(c) = \bigcup_{x^l \in \text{LV}(c)} \{l\} \cup \{\text{ex}\}$$

Abstract Control Flow Graph Edge Construction Each edge $(l \xrightarrow{dc} l') \in \text{absE}(c)$ is an abstract transition between two program points l, l' . The edge is constructed by the standard control flow analysis satisfying there is an edge from l to l' if and only if the command with label l' can execute right after the execution of the command with label l .

Each edge is annotated by a constraint dc generated from the command with label l . This constraint describes the abstract execution of the command with l . It is either the symbol $\top$, a boolean expression or a *difference constraint* computed according to the algorithm in Sinn et al. (2017). Below we summarize how to compute the constraints and generate the edges in three steps,

1. In the first step, we generate the *constraint* of the expression for every program's labeled command, which is used as the annotation of an edge.
2. In the second step, we compute the *initial and continuation state* for each command. The initial state is a set that contains the program point where this command starts executing, and the final state is a set that contains the con-

straint of this command and the continuation program points after the execution of this command.

3. In the third step, we compute the set of edges for the program, $\text{abstrace}(c)$. Each edge is an *abstract event*, which is a pair of the initial and final states.

Constraint Computation Given a program c , every expression in an assignment command or in the guard of a `if` or `while` command is transformed into a constraint as follows.

Definition 24 (Difference Constraint, Symbolic Constant). *The difference constraints $DC(\mathcal{V} \cup SC)$ is the set of all the inequality of form $x' \leq y + v$ or $x' \leq v$ where $x \in \mathcal{V}$, $y \in \mathcal{V}$ and $v \in SC$. The Symbolic Constant can be a natural number, ∞ , the input variable of a program, $\mathcal{V}_{\text{in}}$ or Q_m representing the maximum value returned as the query answer. We use $SC \subseteq \mathbb{N} \cup \mathcal{V} \cup \{\infty, Q_m\}$ to denote the universe of all Symbolic Constant, which is the set of all natural numbers with ∞ and programs' input variables.*

An inequality $x' \leq y + v$ describes that the value of x in the current state is at most the value of y in the previous state plus the symbolic constant v . An inequality $x' \leq v$ describes that the value of x in the current state is at most the value v .

Definition 25 (Symbolic Expression). *The symbolic expression of a program $A_{\text{sc}}(c) \subseteq \mathcal{A}$ is the set of all the arithmetic expressions over $\mathbb{N} \cup \mathcal{V}_{\text{in}}(c) \cup \{\infty, Q_m\}$ for the program c .*

Definition 26 (Constraint). *The constraint set, $DC^\top$ is composed of the Difference Constraints $DC(\mathcal{V} \cup SC)$, the Boolean Expressions, $\mathcal{B}$ and $\top$.*

When a difference constrain shows up as an edge annotation, $l \xrightarrow{x' \leq y + v} l'$, it denotes that the value of variable x after executing the command at l is at most the

value of variable y plus v before the execution, and $l \xrightarrow{x' \leq v} l'$ respectively denotes value of variable x after executing the command at l is at most the value of the symbolic constant v before the execution. We have $l \xrightarrow{x' \leq Q_m} l'$ in the case that command l is a query request and the query answer is assigned to variable x .

The Boolean Expressions b from the set $\mathcal{B}$. b on an edge $l \xrightarrow{b} l'$ describes that after evaluating the guard with label l , b holds and the command with label l will execute right after.

The top constraint, $\top$ denotes true. It is preserved for skip command or commands that don't interfere with any counter variable.

For every expression in each of the label commands, we abstract it into a constraint in three steps via the program abstraction method as below. It is a context-free computation skeleton overall the variable and value domain. In a specific computation with a given program c , these domains will associate with the program c .

Definition 27 (Constraint Computation). *With a program c , a boolean expression b in the guard of a `if` or `while` command or an expression e and a variable x in an assignment command $x \leftarrow e$ as input.*

We first initialize $V_{\text{guard}} = \{\}$ as the set of the variable used in the expression of every `while` or `if` guard in the program c .¹

¹We omit the input c in the rest of **PART i** if the context is clear.

Then the constraint $\text{absexpr}(b, _)$ or $\text{absexpr}(x - v, x)$ is computed as follows,

$$\begin{array}{ll}
\text{absexpr}(x - v, x) = x' \leq x - v & x \in V_{\text{guard}} \wedge v \in \mathcal{SC} \\
\text{absexpr}(y + v, x) = x' \leq y + v & x, y \in V_{\text{guard}} \wedge v \in \mathcal{SC} \\
\text{absexpr}(v, x) = x' \leq v & x \in V_{\text{guard}} \wedge v \in (V_{\text{guard}} \cup \mathcal{SC}) \\
\text{absexpr}(y + v, x) = x' \leq y + v, V_{\text{guard}} = V_{\text{guard}} \cup \{y\} & x \in V_{\text{guard}} \wedge y \notin V_{\text{guard}} \wedge v \in \mathcal{SC} \\
\text{absexpr}(\psi, x) = x' \leq Q_m & x \in V_{\text{guard}} \wedge \psi \text{ is a query expression} \\
\text{absexpr}(b, _) = b, V_{\text{guard}} = V_{\text{guard}} \cup \text{FV}(b) & x \in V_{\text{guard}} \wedge b \text{ is a boolean expression} \\
\text{absexpr}(e, x) = \top & x \notin V_{\text{guard}}
\end{array}$$

$\text{absexpr}(e, x) \ V_{\text{guard}}$ is iteratively updating until stabilized over every guard and assignment command in c , and $\text{absexpr}(e, x)$ denotes the stabilized result in the following paper.

$V_{\text{guard}} \subseteq \mathcal{V}$ is the domain for all variables used in the guard expression of every while command overall programs. In the fourth case, if a variable x , belonging to the set V_{guard} is updated by a variable y , which isn't in this set, we add y into the set V_{guard} and repeat above procedure until V_{guard} and $\text{absexpr}(e, x)$ is stabilized. Specifically, we handle a normalized expression, $x > 0$ in guards of while loop headers, and the counter variable x only increases, decreases, or is reset by simple arithmetic expression (mainly multiplication, division, minus, and plus (able to extend to max and min)). The counter variable x is generalized into the norm when the boolean expression $x > 0$ in `while` doesn't have the form $x > 0$. The way of normalizing the guards and computing the norms is adopted from the computation step 1 in Section 6.1 in paper Sinn et al. (2017).

In the 5th case where the expression comes from a query request, we abstract this query expression into $x \leq Q_m$. By doing this, we ignore the answer of this query in the static analysis just to capture the possible dependency relation pass

through query requests.

Definition 28 (Universe of The Symbolic Expression ($\mathcal{A}_{sc}$)). $\mathcal{A}_{sc}$ is the universe of all the arithmetic expressions over $\mathcal{SC}$, $\mathcal{A}_{sc} \subseteq \mathcal{A}$

Intuitively, the symbolic expression set is a subset of arithmetic expressions over $\mathbb{N}$ by restricting the variables into the input variables.

Abstract Initial and Final State Computation The initial state is the program points before executing this command, which is computed by the standard initial state generation method from control flow analysis. The final state is a set that contains the constraint of this command and the program points after the execution of this command. This set is enriched from the standard control flow analysis.

- The *abstract initial state*, $\text{absinit}(c) \in \mathcal{P}(\mathcal{L})$ for a command, c is the initial program point corresponds to the unique program label of the first executed command in c , computed as follows,

$$\begin{aligned}
 \text{absinit}([x \leftarrow e]^l) &= l \\
 \text{absinit}([x \leftarrow \text{query}(\psi)]^l) &= l \\
 \text{absinit}([\text{skip}]^l) &= l \\
 \text{absinit}(\text{ if } [b]^l \text{ then } c_1 \text{ else } c_2) &= l \\
 \text{absinit}(\text{ while } [b]^l \text{ do } (c_w)) &= l \\
 \text{absinit}(c_1; c_2) &= \text{absinit}(c_1)
 \end{aligned}$$

- The *abstract final state* of the program c , $\text{absfinal}(c) \in \mathcal{P}(\mathcal{L} \times \mathcal{DC}^\top)$ is a set of pairs, (l, dc) with a program point (i.e., a label), l as the first component and a constraint, dc as the second component. The program point l corresponds

to the labeled command after the execution of c , and the constraint dc in this pair is computed by absexpr for the expression in c .

Given a program c , its final state, $\text{absfinal}(c)$ is computed as follows,

$$\begin{aligned}
\text{absfinal}([x \leftarrow e]^l) &= \{(l, \text{absexpr}(e, x))\} \\
\text{absfinal}([x \leftarrow \text{query}(\psi)]^l) &= \{(l, x' \leq 0 + Q_m)\} \\
\text{absfinal}([\text{skip}]^l) &= \{(l, \top)\} \\
\text{absfinal}(\text{if } [b]^l \text{ then } c_1 \text{ else } c_2) &= \text{absfinal}(c_1) \cup \text{absfinal}(c_2) \\
\text{absfinal}(\text{while } [b]^l \text{ do } (c_w)) &= \{(l, \text{absexpr}(b, \top))\} \\
\text{absfinal}(c_1; c_2) &= \text{absfinal}(c_2)
\end{aligned}$$

Abstract Event Computation The set of abstract events for a program c is generated by computing the initial state and final state interactively and recursively.

Definition 29 (Abstract Event). *An abstract event, $\hat{e} \in \mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L}$ is a triple where the first and third components are labels, second component is a constraint from $\mathcal{DC}^\top$.*

In an abstract event (l, dc, l') of a program c , the first label $l \in \mathcal{L}$ corresponds to an initial state of c , and the second label $l' \in \mathcal{L}$ with the constraint $dc \in \mathcal{DC}^\top$ correspond to an abstract continuation state of c . The abstract initial state is a label from $\mathcal{L}$. We abuse the notation $\mathcal{P}(\hat{e})$ for the power set of all abstract events.

The set of the abstract events $\text{abstrace}(c)$ for a program c is computed as follows in Definition 30.

Definition 30 (Abstract Event Computation). $\text{abstrace} \in \mathcal{C} \rightarrow \mathcal{P}(\mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L})$

We first append a `skip` command with the label `ex`, i.e., $[\text{skip}]^{\text{ex}}$ at the end of the program c , and construct the program $c' = c; [\text{skip}]^{\text{ex}}$. Then, we compute the

$\text{abstrace}(c) = \text{abstrace}'(c')$ for c' as follows,

$$\begin{aligned}
\text{abstrace}'([x \leftarrow e]^l) &= \emptyset \\
\text{abstrace}'([x \leftarrow \text{query}(\psi)]^l) &= \emptyset \\
\text{abstrace}'([\text{skip}]^l) &= \emptyset \\
\text{abstrace}'(\text{ if } [b]^l \text{ then } c_t \text{ else } c_f) &= \text{abstrace}'(c_t) \cup \text{abstrace}'(c_f) \\
&\quad \cup \{(l, \text{absexpr}(b, \top), \text{absinit}(c_t))\} \\
&\quad \cup \{(l, \text{absexpr}(\neg b, \top), \text{absinit}(c_f))\} \\
\text{abstrace}'(\text{ while } [b]^l \text{ do } (c_w)) &= \text{abstrace}'(c_w) \cup \{(l, \text{absexpr}(b, \top), \text{absinit}(c_w))\} \\
&\quad \cup \{(l', dc, l) \mid (l', dc) \in \text{absfinal}(c_w)\} \\
\text{abstrace}'(c_1; c_2) &= \text{abstrace}'(c_1) \cup \text{abstrace}'(c_2) \\
&\quad \cup \{(l, dc, \text{absinit}(c_2)) \mid (l, dc) \in \text{absfinal}(c_1)\}
\end{aligned}$$

Notice $\text{abstrace}'([x := e]^l)$, $\text{abstrace}'([x := \text{query}(\psi)]^l)$ and $\text{abstrace}'([\text{skip}]^l)$ are all empty set.

For every event ϵ with label l , if it is in an execution trace τ of the program c , there is an abstract event in the program's abstract execution trace of form $(l, _, _)$, formally below with the proof in [Appendix A.4.1](#)

Lemma 6.3.1 (Soundness of the Abstract Events). *For every program c and an execution trace $\tau \in \mathcal{T}^+$ that is generated w.r.t. an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, there is an abstract event $\hat{\epsilon} = (l, _, _) \in \text{abstrace}(c)$ for every event $\epsilon \in \tau$ having the same label l , i.e., $\epsilon = (_, l, _, _)$.*

$$\begin{aligned}
&\forall c \in \mathcal{C}, \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, l, l' \in \text{absV}(c), \epsilon = (_, l, _, _) \in \mathcal{E} . \\
&\quad \langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \wedge \epsilon \in \tau \\
&\quad \implies \exists \hat{\epsilon} = (l, _, _) \in (\mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L}) . \hat{\epsilon} \in \text{abstrace}(c)
\end{aligned}$$

For every program point l , if it is the label of an assignment command in a program c , there is a unique abstract event in the program's abstract events set $\hat{e} \in \text{abstrace}(c)$ of form $(l, _, _)$.

Lemma 6.3.2 (Uniqueness of the Abstract Event). *For every program c and an execution trace $\tau \in \mathcal{T}^+$ that is generated w.r.t. an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, there is a unique abstract event $\hat{e} = (l, _, _) \in \text{abstrace}(c)$ for every assignment event $\epsilon \in \mathcal{E}^{\text{asn}}$ in the execution trace having the label l , i.e., $\epsilon = (_, l, _, _)$ and $\epsilon \in \tau$.*

$$\begin{aligned} \forall c \in \mathcal{C}, \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, l, l' \in \text{absV}(c), \epsilon = (_, l, _) \in \mathcal{E}^{\text{asn}}. \\ \langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \wedge \epsilon \in \tau \\ \implies \exists! \hat{e} = (l, _, _) \in (\mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L}) . \hat{e} \in \text{abstrace}(c) \end{aligned}$$

This lemma is proved in Appendix [A.4.2](#).

Edge Construction For a program c , the edges on its abstract transition graph are constructed by computing the set of all its abstract events, $\text{abstrace}(c)$ as follows,

$$\text{absE}(c) = \{(l_1, dc, l_2) \mid (l_1, dc, l_2) \in \text{abstrace}(c)\}$$

Abstract Transition Graph Construction With the vertices $\text{absV}(c)$ and edges $\text{absE}(c)$ ready, we construct the abstract transition graph, formally in Definition [31](#).

Definition 31 (Abstract Transition Graph). *Given a program c , its abstract transition graph $\text{absG}(c) = (\text{absV}(c), \text{absE}(c))$ is computed as follows,*

$$\begin{aligned} \text{absE}(c) &= \{(l_1, dc, l_2) \mid (l_1, dc, l_2) \in \text{abstrace}(c)\}, \\ \text{absV}(c) &= \text{LV}(c) \cup \{\text{ex}\} \end{aligned}$$

The three examples as below are presented to illustrate the construction of the abstraction transition graphs.


```

twoRounds(k)  $\triangleq$ 
 $[a \leftarrow 0]^0; [j \leftarrow k]^1;$ 
while  $[j > 0]^2$  do  $\left( [x \leftarrow \text{query}(\chi[j] \cdot \chi[k])]^3; [j \leftarrow j - 1]^4; [a \leftarrow x + a]^5 \right);$ 
 $[l \leftarrow \text{query}(\chi[k] * a)]^6$ 

```

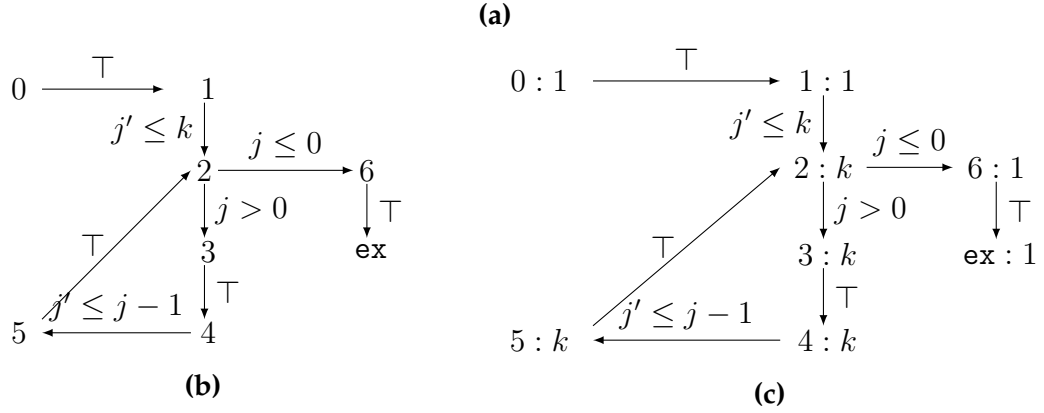


Figure 6.2: (a) The same twoRounds(k) program as Figure 2.4. (b) The abstract transition graph for twoRounds(k). (c) The abstract transition graph with the estimated reachability bound.

Example I. The first example is our running through example, its abstract transition graph is generated in Figure 6.2. The edge $(1 \xrightarrow{j' \leq k} 2)$ on the top tells us the command $[j \leftarrow k]^1$ is executed with a continuation point 2 such that the guard $[j > 0]^2$ will be evaluated next. The annotation $j' \leq k$ is a difference constraint computed for the expression k from the assignment command $j \leftarrow k$. It represents that the value of j is less than or equal to value of input variable k after the execution of $[a \leftarrow 0]^0$ and before executing the loop. The boolean constraint $j \leq 0$ on the edge $2 \xrightarrow{j \leq 0} 6$ represents the negation of the testing guard $j > 0$ of the while command with header at label 2.

Example II. In another example in Figure 6.3(a), the answer of a query request is used in the guard of the if control. The query request in the first branch, $[y \leftarrow \text{query}(\chi[2])]^2$ may-depend on the first query $[x \leftarrow \text{query}(\chi[1])]^0$ through the control dependency.

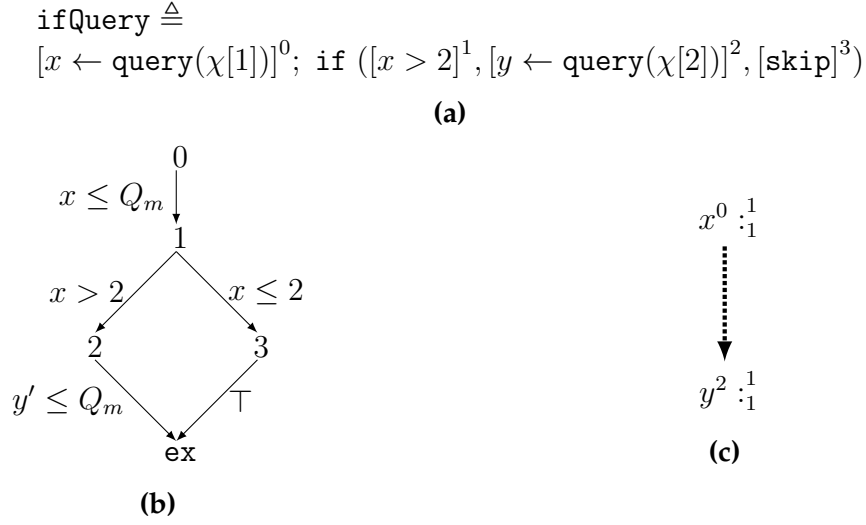


Figure 6.3: (a) The `ifQuery` program. (b) The abstract transition graph for `ifQuery`. (c) The estimated dependency graph for `ifQuery`.

Adaptivity in this graph is 2, which is correctly captured by Definition 23. Its abstract transition graph is generated as in Figure 6.3(b). The result of the first query request is used in the guard of the `if` command, the graph captures this relation through two branches $1 \xrightarrow{x > 2} 2$ and $1 \xrightarrow{x \leq 2} 3$. Constraints on the two edges are summarized from the guard $[x > 2]^1$ through Definition 54. In this sense, the following static analysis based on this graph is able to analyze the dependency relation passing through control flow.

Example III. In the following example, the answer of a query request is used in the guard of the loop. Similar to the `twoRounds` example, query requests in each loop iteration are independent to each other and the last query request after the loop, $[l \leftarrow \text{query}(\chi[k] * a)]^6$ depends on all the queries in loop iterations. The only difference is that, the queries in loop iterations all depend on the first query, $[x \leftarrow \text{query}(\chi[0] \cdot \chi[k])]^0$ through control flow. In this sense, adaptivity is 3, which is correctly captured by Definition 23 as well. Its abstract transition graph is gen-

```

loopQuery(k)  $\triangleq$ 
 $[x \leftarrow \text{query}(\chi[0] \cdot \chi[k])]^0; [a \leftarrow 0]^1;$ 
  while  $[x > 0]^2$  do  $\left( [y \leftarrow \text{query}(\chi[j] \cdot \chi[k])]^3; [x \leftarrow x - 1]^4; [a \leftarrow y + a]^5 \right);$ 
 $[l \leftarrow \text{query}(\chi[k] * a)]^6$ 

```

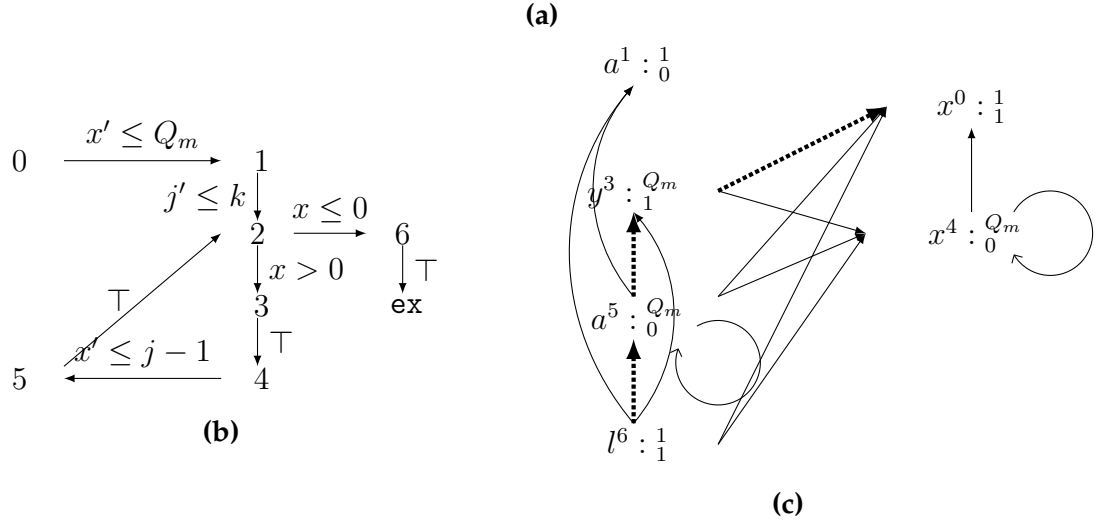


Figure 6.4: (a) The `loopQuery(k)` program. (b) The abstract transition graph for `loopQuery(k)`. (c) The estimated dependency graph for `loopQuery(k)`.

erated as in Figure 6.4(b) according to Definition 31. The result of the first query request, $[x \leftarrow \text{query}(\chi[0] \cdot \chi[k])]^0$ is used in the maximum iteration number of the loop. According to the Definition 17, the query request, $[y \leftarrow \text{query}(\chi[j] \cdot \chi[k])]^3$ depend on the first query through the control flow. On the edge, $0 \xrightarrow{x' \leq Q_m} 1$ describe the value of x after command 0 is no more than Q_m , where Q_m is a symbol describing the maximum value of query answer. Weight of every vertex in Figure 6.4(b) is Q_m , which is the estimated upper bound on the iteration number of the loop by Section 6.3.3. Based on this graph, the reset of the analysis algorithm is able to analyze the dependency relation passing through control flow.

6.3.2 Edge Estimation

The set of edges $E_{\text{est}}(c)$ is estimated through a combined data and control flow analysis with three components.

Since the edges of the semantics-based graph of a program rely on the dependency relation, it contains both control flow and data flow. In this sense, We first compute a *feasible data-flow* relation to estimate the *variable may-dependency* relation, which catches these two flows. Then we construct the edges for $G_{\text{est}}(c)$ based on this *feasible data-flow* relation. It considers both the control flow and data flow and is a sound approximation of the edges in the semantics-based dependency graph. This algorithm is summarized into three steps as follows,

1. We first perform the **reaching definition** analysis and compute a set of labeled variables that is reachable for every label l in c , $RD(l, c)$. For example, in the following program

$$c = [x \leftarrow \text{query}(\chi[1])]^0; [x \leftarrow \text{query}(\chi[0])]^1; [y \leftarrow \text{query}(\chi[x])]^2,$$

only x^1 is reachable when executing $[y \leftarrow \text{query}(\chi[x])]^2$. So we want to have $RD(2, c) = \{1\}$. The computation performs the standard reaching definition analysis and working-list algorithm over the abstract control flow graph, $\text{absG}(c)$. $RD(l, c)$ contains all the labeled variables which are reachable at program point l . For each label l , the analysis generates two initial sets of labeled variables, *in* and *out*, containing all the labeled variables x^l that are newly generated but not yet reassigned before and after executing the command l . The analysis iterates over $\text{absG}(c)$, and updates $\text{in}(l)$ and $\text{out}(l)$ until they are stable. The final $\text{in}(l)$ is the set of reaching definitions $RD(l, c)$ for l .

2. Then we compute the **feasible data-flow** relation for every pair of labeled variables in a program c . Specifically, the *feasible data-flow* relation denoted by $\text{flowsTo}(x^i, y^j, c)$ for each pair of labeled variables the c in a program c , $x^i, y^j \in \text{LV}(c)$. It combines the $\text{RD}(l, c)$, $\text{absG}(c)$, and the data flow analysis, which gives a more precise approximation on the *data may-dependency* than the result from a single analysis. We also prove it is a sound approximation of the *variable may-dependency* relation, $\text{DEP}_{\text{var}}(x^i, y^j, c)$ for every $x^i, y^j \in \text{LV}(c)$.
3. Then using the $\text{flowsTo}(x^i, y^j, c)$ relation, we define the estimated directed edges as set of pairs of vertices $x^i, y^j \in \text{V}_{\text{est}}(c)$, $\text{E}_{\text{est}}(c) \in \mathcal{P}(\mathcal{LV} \times \mathcal{LV})$, by computing transitive closure (through other variables) of the flowsTo relation.

The details are as follows.

Reaching definition analysis This part performs the standard reaching definition analysis given a program c , on every label in $\text{absV}(c)$. This step generates that the set contains all the reachable variables at location l in the program c . The $\text{RD}(l, c)$ represents the analysis result, which is the set of reachable labeled variables in program c at the location of label l . For every labeled variable x^l in this set, the value assigned to that variable in the assignment command associated with that label is reachable at the point of executing the command of label l . It is computed in five steps as follows,

1. The block, is either the command of the form of assignment, skip, or a test of the form of $[b]^l$, denoted by $\text{blocks}(c)$ the set of all the blocks in program c , where $\text{blocks} : \mathcal{C} \rightarrow \mathcal{P}(\mathcal{C} \cup [b]^l)$.

A block is either the command of the form of assignment, skip, or test of the form of $[b]^l$.

The operator $\text{blk} : \mathcal{C} \rightarrow \text{blocks}$ gives all the blocks in program c .

Set $?$ to be undefined.

2. The operator $\text{kill} : \text{blocks} \rightarrow \mathcal{P}(\mathcal{V} \times \mathcal{L} \cup \{?\})$ produces the set of labeled variables of assignment destroyed by the block.
3. The operator $\text{gen} : \text{blocks} \rightarrow \mathcal{P}(\mathcal{V} \times \mathcal{L} \cup \{?\})$ generates the set of labeled variables generated by the block.
4. The operator $\text{in}(l), \text{out}(l) : \mathcal{L} \rightarrow \mathcal{L}\mathcal{V} \cup \{?\}$ for every block in program c is defined as follows,

$$\begin{aligned} \text{in}(l) &\triangleq \{x^? \mid x^? \in \text{LV}(c) \wedge l = \text{absinit}(c)\} \\ &\quad \cup \{\text{out}(l') \mid (l', _, l) \in \text{absE}(c) \wedge l \neq \text{absinit}(c)\} \\ \text{out}(l) &\triangleq \text{gen}(B^l) \cup \{\text{in}(l) \setminus \text{kill}(B^l)\} \end{aligned}$$

computing $\text{in}(l)$ and $\text{out}(l)$ for every $B^l \in \text{blocks}(c)$, and repeating these two steps until the $\text{in}(l)$ and $\text{out}(l)$ are stabilized for every $B^l \in \text{blocks}(c)$. We use $\text{RD}(l, c)$ to represent denote the stabilized result of $\text{in}(l)$ at label l in program c .

5. The stabilized $\text{in}(l)$ and $\text{out}(l)$ for program c , as well as $\text{RD}(l, c)$, is computed by the standard work-list algorithm with detail as below.

Feasible Data-Flow Computation This part presents the computation of the *feasible data-flow* relation between each pair of labeled variables in a program c , formally in Definition 32,

Algorithm 1 Working List Algorithm ($\text{wklst}(c, \text{absG}(c))$)

Require: The program c , the abstract control flow graph $\text{absG}(c)$.

- 1: **initialize** $\text{in}[l]=\text{out}[l]=\emptyset$; $\text{in}[0] = \emptyset$
 a work queue W , contains all the blocks in c
 - 2: **while** $|W| \neq 0$
 - 3: **pop** l in W
 - 4: $\text{old} = \text{out}[l]$
 - 5: $\text{in}(l) = \text{out}(l')$ where $(l', _, l) \in \text{absE}(c)$
 - 6: $\text{out}(l) = \text{gen}(b^l) \cup (\text{in}(l) - \text{kill}(b^l))$, where $b^l \in \text{blk}(c)$
 - 7: **if** $(\text{old} \neq \text{out}(l))$ $W = W \cup \{l' \mid (l', _, l) \in \text{absE}(c)\}$
-

Definition 32 (Feasible Data-Flow). *Given a program c and two labeled variables x^i, y^j in this program, $\text{flowsTo}(x^i, y^j, c)$ is*

$$\begin{aligned}
 &\text{flowsTo}(x^i, y^j, [y \leftarrow e]^l) \\
 &\quad \triangleq (x^i, y^j) \in \{(x^i, y^l) \mid x \in \text{FV}(e) \wedge x^i \in \text{RD}(l, [y \leftarrow e]^l)\} \\
 &\text{flowsTo}(x^i, y^j, [y \leftarrow \text{query}(\psi)]^l) \\
 &\quad \triangleq (x^i, y^j) \in \{(x^i, y^l) \mid x \in \text{FV}(\psi) \wedge x^i \in \text{RD}(l, [y \leftarrow \text{query}(\psi)]^l)\} \\
 &\text{flowsTo}(x^i, y^j, [\text{skip}]^l) \triangleq \emptyset \\
 &\text{flowsTo}(x^i, y^j, \text{if } ([b]^l, c_1, c_2)) \\
 &\quad \triangleq \text{flowsTo}(x^i, y^j, c_1) \vee \text{flowsTo}(x^i, y^j, c_2) \\
 &\quad \vee (x^i, y^j) \in \{(x^i, y^j) \mid x \in \text{FV}(b) \wedge x^i \in \text{RD}(l, \text{if } ([b]^l, c_1, c_2)) \wedge y^j \in \text{LV}(c_1) \cup \text{LV}(c_2)\} \\
 &\text{flowsTo}(x^i, y^j, \text{while } [b]^l \text{ do } (c_w)) \\
 &\quad \triangleq \text{flowsTo}(x^i, y^j, c_w) \\
 &\quad \vee (x^i, y^j) \in \{(x^i, y^j) \mid x \in \text{FV}(b) \wedge x^i \in \text{RD}(l, \text{while } [b]^l \text{ do } (c_w)) \wedge y^j \in \text{LV}(c_w)\} \\
 &\text{flowsTo}(x^i, y^j, c_1; c_2) \triangleq \text{flowsTo}(x^i, y^j, c_1) \vee \text{flowsTo}(x^i, y^j, c_2)
 \end{aligned}$$

We prove that the transitive closure of the *Feasible Data-Flow* relation is a sound approximation of the *Variable May-dependency* relation over labeled variables for every program, in [Appendix A.3](#).

The combinations of the result of *reaching definition*, $RD(l, c)$ with the *abstract control flow graph*, $absG(c)$ with control flow analysis into the feasible data-flow generation improves the data-dependency relation approximation accuracy.

Example I. In an example program $[x \leftarrow 0]^0; [x \leftarrow 2]^1; [y \leftarrow x + 1]^2$. The standard data flow analysis tells us that both the labeled variable x^0 and x^1 may flow to y^2 , which will result in an unnecessary edge (x^0, y^2) . The result of reaching definition can help us eliminate this kind of edge by telling us, at line 2, only variable x^1 is reachable.

Example II. This analysis focuses on guaranteeing soundness, there is an over-approximation in the operation $FV(e)$ by looking into only the syntactic dependency. For example, in the program $[x \leftarrow \text{query}(\chi[0])]^0; [y \leftarrow x - x]^1$, y^1 does not depend on x^0 by Definition 17. However, the Definition 32 still produces the $\text{flowsTo}(x^0, y^1, x)$, which over-approximates the variable may-dependency relation.

Edge Estimation The **Edge Estimation** is based on the $\text{flowsTo}(x^i, y^j, c)$ relation. For each pair of vertices x^i, y^j in $V_{\text{est}}(c)$, There is a directed edge from x^i to y^j if and only if there is a chain of variables in the flowsTo relation between x^i and y^j . Using the $\text{flowsTo}(x^i, y^j, c)$ relation, we define the estimated directed edges as a set that contains all pair of vertices x^i, y^j in $V_{\text{est}}(c)$, $E_{\text{est}}(c) \in \mathcal{P}(\mathcal{LV} \times \mathcal{LV})$ satisfying in the transitive closure of flowsTo as follows,

$$E_{\text{est}}(c) \triangleq \{ (y^j, x^i) \mid y^j, x^i \in V_{\text{est}}(c) \wedge \exists n, z_1^{r_1}, \dots, z_n^{r_n} \in \mathcal{LV}(c) . \\ n \geq 0 \wedge \text{flowsTo}(x^i, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, y^j, c) \}$$

We prove that this estimated directed edge set $E_{\text{est}}(c)$ is a sound approximation

of the edge set in c 's semantics-based dependency graph in Appendix A.2.

Lemma 6.3.3 (Mapping from Edges of G_{trace} to G_{est}). *For every program c we have:*

$$\forall e = (v_1, v_2) \in E_{\text{trace}}(c) . \exists e' \in E_{\text{est}}(c) . e' = (v_1, v_2)$$

Example I. As in the Figure 2.4(c), the edge $l^6 \rightarrow a^5$ is built by $\text{flowsTo}(l^6, a^5, c)$ relation because a is used directly in the query expression $\chi[k] * a$ in the command $[l \leftarrow \text{query}(\chi[k] * a)]^6$, i.e., $a \in \text{FV}(\chi[k] * a)$. And we also have $a^5 \in \text{RD}(6, \text{twoRounds}(k))$ from the reaching definition analysis. Another edge $x^3 \rightarrow j^5$ in the same graph represents the control flow from j^5 to x^3 , which is soundly caught by our flowsTo relation.

Example II. In the example program in Figure 6.3(a), there is only a variable may-dependency relation from y^2 to x^0 , which is pass through control flow. The 4th case of the analysis algorithm in Definition 32 constructs this edge as in Figure 6.3(c).

Example III. In the third example program from Section 6.3.1 in Figure 6.4(a), there are data dependency relations through both control flow and value dependency. For the dependency relations through control flow between x^0 and the variables inside the loop body. The 5th case in Definition 32 constructs these edges including $y^3 \rightarrow x^0$ and $a^5 \rightarrow x^0$ in Figure 6.4(c). The edges $l^6 \rightarrow y^3$, $l^6 \rightarrow x^0$ and $l^6 \rightarrow x^4$ are constructed by the transitive closure of the *feasible data-flow* relation.

6.3.3 Weight Estimation

This section presents the quantitative analysis algorithm, which performs over the same *abstract control flow graph*, $\text{absG}(c)$ of a program c as well. As the $W_{\text{trace}}(c)$

defined in Definition 20, the weight of every $x^l \in V_{\text{est}}(c)$ is the execution times of the command with label l . In this sense, to estimate weight of x^l , this step first computes an upper bound, the *reachability-bound* Gulwani & Zuleger (2010) for every $l \in \text{absV}(c)$ on the execution times of the command with the label l . Then, the *reachability-bound* is used as the estimated weight of the vertex $x^l \in V_{\text{est}}(c)$. The two computation steps are summarized as follows,

1. **Reachability Bound Analysis** This analysis estimate for each program point $l \in \text{absV}(c)$ a symbolic upper bounds on the execution times of the command with label l during the program execution. These symbolic upper bounds are symbolic expressions, $A_{\text{sc}}(c)$ containing the input variables, hence they can soundly approximate the weight functions in the semantics-based dependency graphs. Our reachability-bound algorithm adapts to our setting ideas from previous work Zuleger et al. (2011); Sinn et al. (2014, 2017). Specifically, it provides an upper bound on the number of times every command can be executed.
2. **Weight Estimation** Because the vertex in program's $\text{absG}(c)$ shares the same unique label with the vertex in $V_{\text{est}}(c)$, we use the *reachability-bound* on the vertex $l \in \text{absV}(c)$ directly as the weight of the vertex x^l in $V_{\text{est}}(c)$.

Step-1: Reachability Bound Analysis This symbolic reachability bound analysis performs over the same *abstract control flow graph*, $\text{absG}(c)$ of a program c . It first computes a *reachability bound* for every edge $l \xrightarrow{dc} l' \in \text{absE}(c)$, which is a symbolic bound on the maximum execution times of the command with label l of c . Then the *reachability bound* for edge $l \xrightarrow{dc} l'$ is used as the bound on the maximum visiting times of the vertex $l \in \text{absV}(c)$. It is a sound upper bound on the visiting times

of every label $l \in \text{absV}(c)$, named *reachability-bound*. The computation steps are summarized as follows,

1. It first collects three edge sets for each variable, in which the variable increases, decreases, and is reset respectively.
2. Then, for each edge $l \xrightarrow{dc} l' \in \text{absE}(c)$, it assigns a variable x (or a symbolic constant $v \in \mathcal{SC}$) if x (or v) decreases in dc , as this edge's local bound.
3. It then computes the bound on the maximum value of the local bound for each edge, and the *reachability-bound* on the execution times of the corresponding edge recursively.
4. The last step uses *reachability-bound* w for edge $l \xrightarrow{dc} l'$ as the bound on the maximum visiting times of the vertex $l \in \text{absV}(c)$ and generates a set $\text{absW}(c)$ contains a pair (l, w) for every $l \in \text{absV}(c)$.

The algorithm in this step is inspired from the Algorithm 2 in paper Sinn et al. (2014), the Algorithm 3 in paper Zuleger et al. (2011), and the Definition 25 in Section 4 of paper Sinn et al. (2017).

- Algorithm 3 in paper Zuleger et al. (2011) assigns a set of variables to each transition in which these variables decrease as the local bound and estimates the maximum value of each variable in this set.
- Algorithm 2 in paper Sinn et al. (2014) assigns a variable to each edge on which this variable decreases as its ranking function and then estimates the maximum value for the ranking function.
- The Definition 25 in paper Sinn et al. (2017) assigns each transition with a

variable that decreases in this transition, as the local bound and computes the bound similarly.

The computation steps are as follows,

1. **Variable Modifications.** For each variable x in a program c , this step computes three edge sets, $\text{inc}(c, x)$, $\text{dec}(c, x)$, and $\text{re}(c, x)$ for x .

Definition 33 (Variable Modification). *The Variable Modification sets for a variable x in a program c are three sets of edges. $\text{inc}(c, x)$, $\text{dec}(c, x)$, and $\text{re}(c, x) \in \mathcal{P}(\hat{e})$ Every edge in a set corresponds to a transition in which x is increased, decreased or reset respectively as follows.*

$\text{inc} : \mathcal{C} \rightarrow \mathcal{V} \rightarrow \mathcal{P}(\hat{e})$ is the set of edges where the variable increase,

$$\text{inc}(c, x) = \{\hat{e} | \hat{e} = (l, x' \leq x + v, l') \wedge \hat{e} \in \text{abstrace}(c)\}$$

$\text{dec} : \mathcal{V} \rightarrow \mathcal{P}(\hat{e})$ is the set of abstract events where the variable decrease,

$$\text{dec}(c, x) = \{\hat{e} | \hat{e} = (l, x' \leq x - v, l') \wedge \hat{e} \in \text{abstrace}(c)\}$$

$\text{re} : \mathcal{C} \rightarrow \mathcal{V} \rightarrow \mathcal{P}(\hat{e})$ is the set of the abstract events where the variable is reset,

$$\text{re}(c, x) = \{\hat{e} | \hat{e} = (l, x' \leq y - v, l') \wedge x \neq y \wedge \hat{e} \in \text{abstrace}(c)\}$$

In the following steps, c is omitted in $\text{inc}(x)$, $\text{dec}(x)$ and $\text{re}(x)$ for concise when the reference of a program c is clear in the context.

2. **Local Bound Computation.** This step adopts the local bound computation method in Section 4 of Sinn et al. (2017). It assigns to each edge $l \xrightarrow{dc} l' \in \text{absE}(c)$ a *local bound* as follows.

Definition 34 (Local Bound Generation). *For every edge $\hat{e}$ in the transition graph $\text{absG}(c)$ of a program c , its local bound, $\text{locb}(\hat{e}, c)$ is the variable that decreases on this edge, computed as follows,*

$$\begin{aligned} \text{locb}(\hat{e}, c) &\triangleq 1 && \hat{e} \notin \text{SCC}(\text{absG}(c)) \\ \text{locb}(\hat{e}, c) &\triangleq x && \hat{e} \in \text{SCC}(\text{absG}(c)) \wedge \hat{e} \in \text{dec}(x) \wedge \hat{e} = (_, _, x' \leq x - v) \\ \text{locb}(\hat{e}, c) &\triangleq x && \hat{e} \in \text{SCC}(\text{absG}(c)) \wedge \hat{e} \notin \bigcup_{x \in \mathcal{V}} \text{dec}(x) \wedge \hat{e} \notin \text{SCC}(\text{absG}(c) \setminus \text{dec}(x)). \\ \text{locb}(\hat{e}, c) &\triangleq \infty && \text{o.w.} \end{aligned}$$

$\text{SCC}(\text{absG}(c))$ is the set of all the non-trivial strongly connected components of $\text{absG}(c)$.

We look at the non-trivial strongly connected components of $\text{absG}(c)$, which contains at least one edge. If the edge does not belong to any strongly connected components, then the local bound is 1, representing the fact that the edge is not in a loop and so it gets executed at most once. If the edge belongs to a strongly connected component and one of the variables x in dc decreases, then the local bound is x . Otherwise, if the edge belongs to a strongly connected component and there is a variable y that decreases in the difference constraint of some other edge, and if by removing this other edge, the original edge does not belong anymore to the strongly connected components of $\text{absG}(c)$, then the local bound is y . Otherwise, the local bound is ∞ . Notice that the output is either a symbolic constant in $\mathcal{SC}$ or a variable that is not an input variable.

Soundness Informal The first case is straightforward. For the label l which is not in any while loop, the labeled command with the label l will be evaluated at most once. The second and third cases are guaranteed by the *Discussion on Soundness* in Section 4 in Sinn et al. (2017). We formalized the soundness and

proof by Lemma B.3.1 in Appendix A.4.

3. Reachability-bound Estimation.

This step aims at determining the *reachability-bound* $TB(e, c)$ of every edge $e \in E_{\text{est}}(c)$. Every bound is a symbolic expression built out of symbols in $\mathcal{SC}$ and the operations $+$, $*$, $\max$. For every edge, if the local bound of this edge computed at the previous step is a symbol in $\mathcal{SC}$ then this is already the reachability-bound. If instead, the local bound of the edge is a variable y which is not an input variable, this step will eliminate it and replace it with a symbolic expression. In order to do this, this steps will compute two quantities: first, it will recursively sum the reachability-bounds of all the edges whose difference constraint may increment the variable y , plus the corresponding increment; second, it will recursively sum the reachability-bounds of all the edges whose difference constraint may reset the variable y to a (symbolic) expression that doesn't depend on it, multiplied by the maximal value of this symbolic expression. The sum of these two quantities provides the symbolic expression that is an upper bound on the number of times the edge can be reached.

To compute these two quantities we use two mutually recursive procedures in a path-insensitive manner in Definition 63 and Definition 36.

Definition 35 (Upper Bound Invariant of Local Bound). *For a program c and an edge $\hat{e} \in \text{absE}(c)$, the local bound, $VB(\text{locb}(\hat{e}, c), c)$ for the local bound $\text{locb}(\hat{e}, c)$*

of this edge is computed as follows,

$$\begin{aligned} \text{VB}(x, c) &\triangleq x && \text{if } x \in \mathcal{SC} \\ \text{VB}(x, c) &\triangleq \text{increase}(x, c) + \max(\{\text{VB}(y, c) + v \mid (l, x' \leq y + v, l') \in \text{re}(x)\}) \\ &&& \text{if } x \notin \mathcal{SC} \end{aligned}$$

$$\text{increase}(x, c) \triangleq \sum_{\hat{e} \in \text{inc}(x)} \{\text{TB}(\hat{e}, c) \times v \mid \hat{e} = (l, x' \leq x + v, l')\}$$

Definition 63 estimates the upper bound invariant, $\text{VB}(x, c) \in \mathcal{A}_{\text{in}}$ on the maximum value of each local bound $x \in \mathcal{V} \cup \mathcal{SC}$. This computation involves mutual recursion of computing the *transition bound* $\text{TB}(\hat{e}, c)$ of an edge $\hat{e}$ as below.

Definition 36 (Transition Bound²). *For a program c and an edge $\hat{e} \in \text{absE}(c)$, the path-insensitive transition bound, $\text{TB}(\hat{e}, c) \in \mathcal{A}_{\text{in}}$ for this edge is computed as follows,*

$$\begin{aligned} \text{TB}(\hat{e}, c) &\triangleq \text{VB}(\text{locb}(\hat{e}, c), c) && \text{if } \text{locb}(\hat{e}, c) \in \mathcal{SC} \\ \text{TB}(\hat{e}, c) &\triangleq \text{increase}(x, c) + \sum_{\hat{e}' \in \text{re}(x, c) \wedge \hat{e}' = (l, x \leq y + v, l')} \left(\text{TB}(\hat{e}', c) \times (\text{VB}(y, c) + v) \right) \\ &&& \text{if } \text{locb}(\hat{e}, c) = x \wedge x \notin \mathcal{SC} \end{aligned} ,$$

Definition 36 computes ∞ for the program that have non-terminating behavior. However, by the well-formedness of the program in Definition 9, we consider in **PART i** only the programs that are terminating with any initial trace. For a program c and an edge $\hat{e} \in \text{absE}(c)$, $\text{TB}(\hat{e})$ is a sound upper bound on the execution times of this transition by Sinn et al. (2017). The soundness is formally stated in Theorem 6.3.1, and the proof is in Appendix A.4.

Theorem 6.3.1 (Soundness of the Transition Bound). *For a program c with its*

abstract transition graph $\text{absG}(c)$, if l is the label of an assignment command and $\hat{e} = (l, _, _) \in \text{absG}(c)$ has the same label, then $\text{TB}(\hat{e}, c)$ is a sound upper bound on the numbers that the assignment command with label l is evaluated during the execution of c with any initial trace $\tau_0 \in \mathcal{T}_0^+(c)$.

$$\forall c \in \mathcal{C}, l, l' \in c, \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^{+\infty}, v \in \mathbb{N} .$$

$$\left(\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \wedge \langle \text{TB}(\hat{e}, c), \tau_0 \rangle \Downarrow_a v \wedge \text{cnt}(\tau, l) \leq v \right)$$

Example We perform the symbolic reachability bound analysis on the abstract control flow graph in Figure 6.2(b) and compute the result in Figure 6.2(c). We would like to generate the closure of every edge, which is an equality relation between variables. Solving this closure gives us the reachability bound for this edge. With all the bound for all the edges in the abstract control flow graph, we can calculate the weight for every vertex in this graph. For example, we show the closure generated for the edge $4 \xrightarrow{j' \leq j-1} 5$, $\text{TB}(4 \xrightarrow{j' \leq j-1} 5) = \text{VB}(j)$. The invariant for variable j , $\text{VB}(j)$ used here is $\text{VB}(j) = k * \text{TB}(1 \xrightarrow{i' \leq k} 2)$, which is generated by all the different constraints involving j in the graph. Notice the k in $\text{VB}(j)$ comes from considering both difference constraint $j' \leq k$ from edge $1 \xrightarrow{j' \leq k} 2$ and $j' \leq j - 1$ from $4 \xrightarrow{j' \leq j-1} 5$, which intuitively reflects the while loop whose counter is set to k at the beginning and decreases by 1 at each iteration. With all the closures for all the edges of the abstract control flow graph, we can solve them to obtain the reachability bound of every edge. We decide the weight for every vertex in the abstract control flow graph by using the bound of the edges which head out from this vertex, by taking the max of the bound from these involving edges. For instance, By the constraint on the edge $4 \xrightarrow{j' \leq j-1} 5$, we get bound k for this edge. Then, we assign vertex 4 by reachability bound k , as in Figure 6.2(c). Another

interesting vertex is 2, which has more than one edge heading out from it, $2 \xrightarrow{j \geq 0} 3$ and $2 \xrightarrow{j \leq 0} 6$. For the weight for vertex 2, we choose the max between the bound k from $2 \xrightarrow{j \geq 0} 3$ and 1 from $2 \xrightarrow{j \leq 0} 6$. The same way for the rest weights' computation. We use $\text{absW}(c)$ for the set of weights we just computed for each label in the abstract control flow graph of c . The same way for the rest weights' computation.

Vertex Weight Computation Using the reachability-bound $\text{TB}(\hat{e}, c)$ for every edge $\hat{e} = (l, dc, l')$ we can provide a bound on the visiting times of each vertex $x^l \in \text{absV}(c)$.

Definition 37 (Weight Estimation). *The weight set $\text{W}_{\text{est}}(c)$ of program c is a set of pairs $\text{W}_{\text{est}}(c) \in \mathcal{P}(\mathcal{LV} \times \mathcal{A}_{\text{sc}})$. Each pair maps a vertex $x^l \in \text{LV}(c)$ to a symbolic expression over $\mathcal{SC}$, as follows.*

$$\text{W}_{\text{est}}(c) \triangleq \left\{ (x^l, \hat{w}) \mid x^l \in \text{V}_{\text{est}}(c) \wedge \hat{w} = \sum \{ \text{TB}(\hat{e}, c) \mid \hat{e} \in \text{abstrace}(c) \wedge \hat{e} = (l, -, -) \} \right\}.$$

Notice that $\hat{w} \in \mathcal{A}_{\text{sc}}$ is an expression over symbols in $\mathcal{SC}$. In particular, it may contain the input variables and so it may effectively be used as a function of the input - and capture loop bounds in terms of these inputs.

We guarantee that $\hat{w}$ for $x^l \in \text{V}_{\text{est}}(c)$ is a sound upper bound of the weight for the same vertex x^l in the program's semantics-based dependency graph in Theorem 6.3.2. The proof is in Appendix A.4.3, which is based on the Lemma A.2.3 in Appendix A.2, and Theorem 6.3.1.

Theorem 6.3.2 (Soundness of the Weight Estimation). *Let c be a program and $\text{W}_{\text{est}}(c)$ be its estimated weight set. Then, for every $(x^l, w) \in \text{W}_{\text{trace}}(c)$, we have an estimated weight $(x^l, \hat{w}) \in \text{W}_{\text{est}}(c)$ and for every possible $\tau_0 \in \mathcal{T}_0^+(c)$, $v \in \mathbb{N}$, if $\langle \tau_0, \hat{w} \rangle \Downarrow_e v$, then v*

is an upper bound on $w(\tau_0)$.

$$\begin{aligned} \forall c \in \mathcal{C}, x^l \in \text{LV}(c), \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, v \in \mathbb{N}, l', (x^l, w) \in \mathcal{W}_{\text{trace}}(c) . \\ \exists (x^l, \hat{w}) \in \mathcal{W}_{\text{est}}(c) \wedge (\langle \tau_0, \hat{w} \rangle \Downarrow_e v \implies w(\tau_0) \leq v) \end{aligned}$$

Notice that in this theorem, the evaluation $\langle \tau_0, w \rangle \Downarrow_e v$ is needed in order to obtain a concrete value v from the symbolic weight w by specifying a value for the input variables through τ_0 .

Example I. Going back to the estimated dependency graph for the two-round example in Figure 6.5(c), which we aim to estimate. Every vertex from $V_{\text{est}}(c)$ in this graph corresponds to a labeled variable, for example, a^5 , and this label 5 is also a vertex 5 in the abstract control flow graph in Figure 6.2(b). Then, it is straightforward, that the reachability bound for the label 5, is also the maximum visiting times bound of the labeled variable a^5 . So, we estimate the visiting time for labeled variable a^5 in the estimated dependency graph in Figure 6.2(c) as k as well. In the same way, we compute the weights of the other vertices on this graph.

Example II. In the `ifQuery` program in Figure 6.3(a), there is not a loop. Every vertex is visited at most once, and we estimate its weight as in the superscript of each vertex in Figure 6.3(c).

Example III. In the `loopQuery` program in Figure 6.4(a), the while loop will iterate the number of Q_m times, where Q_m is a symbol representing the maximal value of a query answer. In this sense, we estimate the weight for vertices in the loop body as Q_m in the superscript of each vertex in Figure 6.3(c). This Q_m is indeed similar to the ∞ since we do not have a finite bound on the possible answer

returned from the query requests. We construct this weight in order to have a sound approximation on the semantics-based dependency graph.

6.4 DEPENDENCY GRAPH ESTIMATION

With the four components $V_{\text{est}}(c)$, $E_{\text{est}}(c)$, $W_{\text{est}}(c)$, and $Q_{\text{est}}(c)$ computed in each step above, this step simply combines the four components into the quantitative dependency graph for program c as follows,

$$G_{\text{est}}(c) = (V_{\text{est}}(c), E_{\text{est}}(c), W_{\text{est}}(c), Q_{\text{est}}(c)).$$

We prove that this graph is a sound approximation of the program's semantics-based dependency graph by the soundness of each component formally in the Appendix.

This estimated graph has a similar topology structure as the Semantics-based Dependency Graph. It has the same vertices but approximated edges and weights. This graph is a sound approximation of the quantitative dependency graph for a program c .

It is formally defined in Definition 38 as follows.

Definition 38 (Estimated Dependency Graph). *Given a program c with the feasible data flow relation $\text{flowsTo}(x^i, y^j, c)$ for every $x^i, y^j \in \text{LV}(c)$, and the $\text{absW}(c)$ as the set of estimated weight for every program label l , its estimated dependency graph is generated as follows.*

$$G_{\text{est}}(c) = (V_{\text{est}}(c), E_{\text{est}}(c), W_{\text{est}}(c), Q_{\text{est}}(c))$$

Each of the four components is defined as follows.

$$\begin{aligned}
V_{\text{est}} &:= \{x^l \mid x^l \in \text{LV}(c)\} \\
E_{\text{est}} &:= \left\{ (x^i, y^j) \mid \begin{array}{l} x^i, y^j \in V \wedge \exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . n \geq 0 \wedge \\ \text{flowsTo}(y^j, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, x^i, c) \end{array} \right\} \\
W_{\text{est}} &:= \{(x^l, \hat{w}) \mid x^l \in V_{\text{est}}(c) \wedge \hat{w} = \sum \{\text{TB}(\hat{e}, c) \mid \hat{e} \in \text{abstrace}(c) \wedge \hat{e} = (l, _, _)\}\} \\
Q_{\text{est}} &:= \left\{ (x^l, n) \mid \begin{array}{l} x^l \in \text{LV}(c) \wedge n = 1 \iff x^l \in \text{QV}(c) \\ \wedge n = 0 \iff x^l \in \text{QV}(c). \end{array} \right\}
\end{aligned}$$

The construction of the static analysis dependency graph is of great value in showing some useful properties of the target program, such as dependency between variables, the execution upper bound of a certain command, while the key novelty is our path-searching algorithm, which connects all the information we need in the static analysis dependency graph and provides us with a sound estimation of adaptivity.

6.5 ADAPTIVITY UPPER BOUND COMPUTATION

Based on the estimated dependency graph of a program c , we compute a sound upper bound on c 's adaptivity in this section.

Different from the weight on the semantics-based dependency graph, the weight on the estimated graph is a symbolic expression. This symbolic expression cannot be used directly to construct a finite walk by definition 21. In this sense, we first introduce the path on the estimated dependency graph as follows to estimate a finite walk.

Definition 39 (Path). *Given a program c with its estimated dependency graph $G_{\text{est}}(c) = (V_{\text{est}}(c), E_{\text{est}}(c), W_{\text{est}}(c), Q_{\text{est}}(c))$, a path p in $G_{\text{trace}}(c)$ is a sequence of edges $(e_1 \dots e_{n-1})$*

for which there is a sequence of vertices $(v_1, \dots, v_n)$ such that:

- $e_i = (v_i, v_{i+1}) \in E_{\text{est}}(c)$ for every $1 \leq i < n$,
- and $v_i \neq v_j$ for every $i, j = 1, \dots, n$ and $i \neq j$.

We use $\mathcal{PATH}(G_{\text{est}}(c))$ to denote the set of all paths on $G_{\text{est}}(c)$. Following Definition 22, we compute the *query length* of a path, $p \in \mathcal{PATH}(G_{\text{est}}(c))$ on the estimated graph as an arithmetic expression formally as follows.

Definition 40 (Query Length Estimation). *Given a program c and its estimated dependency graph $G_{\text{est}}(c) = (V_{\text{est}}(c), E_{\text{est}}(c), W_{\text{est}}(c), Q_{\text{est}}(c))$, let $p \in \mathcal{PATH}(G_{\text{est}}(c))$ be a path with edge sequence $(e_1 \dots e_{n-1})$ a sequence of vertices $(v_1, \dots, v_n)$, the estimated query length of p , $\hat{\text{len}}^q(p)$ is computed as follows.*

$$\sum \{v \times \hat{w} \mid v \in (v_1, \dots, v_n) \wedge (v, 1) \in Q_{\text{est}}(c) \wedge (v, \hat{w}) \in W_{\text{est}}(c)\}$$

We estimate the adaptivity upper bound, $A_{\text{est}}(c)$ for a program c as the maximum query length over all paths in its *estimated dependency graph*, $G_{\text{est}}(c)$.

Definition 41 (Adaptivity Computation). *Given a program c with its estimated dependency graph $G_{\text{est}}(c)$ we compute its adaptivity $A_{\text{est}}(c) \in \mathcal{A}_{\text{in}}$ as*

$$\max\{\hat{\text{len}}^q(p) \mid p \in \mathcal{PATH}(G_{\text{est}}(c))\}.$$

Similarly, the adaptivity bound $A_{\text{est}}(c)$ will also be a symbolic arithmetic expression over the input variables. With this symbolic expression, we prove that Definition 41 computes a sound upper bound sound for the *adaptivity* w.r.t the Definition 23.

Theorem 6.5.1 (Soundness of AdaptFun). *For every well-formed program c , its estimated adaptivity $A_{\text{est}}(c)$ is a sound upper bound of its adaptivity.*

$$\forall \tau_0 \in \mathcal{T}_0^+(c), v \in \mathbb{N} \cup \{\infty\} . \langle A_{\text{est}}(c), \tau_0 \rangle \Downarrow_e v \implies A(c)(\tau_0) \leq v$$

The proof is in Appendix [A.2](#).

Symbolic expressions as used in the weight are great to express symbolic bounds, but make the computation of the longest path non-terminating. In this sense, the computation by Definition [41](#) has to over-approximate the adaptivity on a large scale in order to address the non-terminating problem. To compute $A_{\text{est}}(c)$ accurately and soundly, we develop an adaptivity approximation algorithm named AdaptSearch. It combines the depth-first search and breath-first search strategies and computes a sound upper bound on $A_{\text{est}}(c)$. AdaptSearch also involves another algorithm $\text{AdaptSearch}_{\text{SCC}}$ in [4](#) recursively, which finds the longest walk for a strongly connected component (SCC) (SCC is the maximal strongly connected subgraph) of $G_{\text{est}}(c)$. Theorem [6.5.2](#) below formally describes the soundness of this algorithm with proof in Appendix [A.5](#).

Theorem 6.5.2 (Soundness of AdaptSearch). *For every well-formed program c with its estimated dependency graph G_{est} ,*

$$\forall \tau_0 \in \mathcal{T}_0^+(c), v \in (\mathbb{N} \cup \{\infty\}) . \langle \text{AdaptSearch}(c), \tau_0 \rangle \Downarrow_e v \implies A(c)(\tau_0) \leq v.$$

By Definition [41](#), the key point is to find the walks in the estimated dependency graph. We first discuss two challenges when we try to find the walks, and then show how we solve them using our algorithms.

Non-Termination Challenge: One naive walk-finding method is to simply tra-

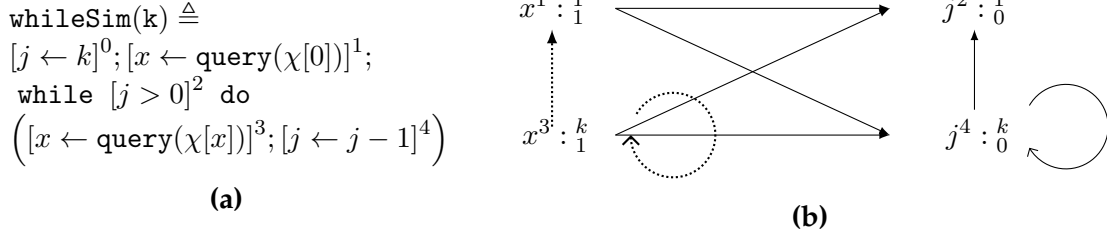


Figure 6.5: (a) A simple loop program with k adaptivity rounds. (b) The estimated dependency graph by AdaptFun.

verse this graph and decrease the weight of every node by one after every visit. However, this simple traversing strategy leads to a non-termination dilemma for most programs in which we are interested. Because the weight of each vertex in a program's estimated dependency graph, which is an arithmetic expression containing input variables. In this sense, the simple traversing could never terminate when the domain of the input variables isn't finite. However, it is very common that the domain of the program's input variables is infinite such as natural number $\mathbb{N}$, real number $\mathbb{R}$, etc. As the simple while loop example program in Figure 6.5 with k adaptivity rounds, the input variable k has domain $\mathbb{N}$. If we traverse the estimated dependency graph and decrease the weight of x^3 (the weight k is symbolic) by one after every visit, we will never terminate because we only know $k \in \mathbb{N}$.

To solve this non-termination challenge, we switch to another walk-finding approach: finding the longest path in the estimated dependency graph via depth-first search and then using this path as the longest walk. We show this naive longest walk searching algorithm in Algorithm 2. Through a simple depth-first search algorithm, we find the longest weighted path as the dotted arrow in Figure 6.5(c), $x^3 : \frac{k}{1} \rightarrow x^1 : \frac{1}{1}$. Then, by summing up the weights on this path where the vertices have query annotations 1, the depth-first search algorithm gives the adaptivity bound k . This is a tight bound for this simple k adaptivity rounds example pro-

gram.

Algorithm 2 Naive Adaptivity Computation $\text{AdaptSearch}_{\text{naive}}(c, G_{\text{est}}(c))$

Require: A program c ,

its estimated dependency graph: $G_{\text{est}}(c) = (V, E, W, Q)$

```

1: init
   visited = [0] * |V| #a list of length |V|, initialize with 0 for every vertex.
   result = 0
2: def dfs(G, s, visited, cur):
3:     longstw = cur
4:     for every vertex  $v$  connected by a directed edge from  $s$ :
5:         if visited[ $v$ ] = false:
6:             visited[ $v$ ] = true;
7:             longstw = max{longstw, dfs( $v$ , visited, cur + W( $v$ ) × Q( $v$ ))}
8:     return longstw
9: for every vertex  $v$  in  $V_i$ :
10:    initialize the visited list
11:    result = max(result, dfs( $v$ , visited, W( $v$ ) × Q( $v$ )));
12: return result

```

Approximation Challenge: However, this naive approximation via depth-first searching over-approximates the adaptivity rounds largely in many cases. It computes ∞ adaptivity upper bound for our twoRounds example program in Figure 2.4, which has only 2 adaptivity rounds. More specifically, the depth-first searching finds the longest weighted path, $x^3 : \frac{k}{1} \rightarrow a^5 : \frac{k}{0} \rightarrow l^6 : \frac{1}{0}$. Then, it computes the weighted length, $1 + k$. If we use this path to approximate the longest finite walk, and the weight of each vertex as its visiting time, then we have a walk, $x^3 \rightarrow \dots \rightarrow x^3 \rightarrow a^5 \rightarrow \dots \rightarrow a^5 \rightarrow l^6$. However, this isn't a qualified walk by our Definition 21. Because l^6 has weight 1, it can only be visited at most once. However, x^3 is visited k times in this approximated walk. As a result, the weighted length of this path is $1 + k$, which over approximates the two-round example program's adaptivity rounds, which is expected to be 2.

To this end, we combine the depth-first search and breath-first search strate-

gies in our longest walk estimation algorithm. Our algorithm reduces the task of computing the longest walk into the computation of local adaptivity and the composition of local adaptivity into global adaptivity. We exploit the structure of the estimated dependency graph $G_{\text{est}}(c)$ for a program c : 1). Partitioning the graph of programs into its maximal strongly connected components (SCCs) 2). Then, for each SCC, we compute an adaptivity bound 3). In the last, we compose these local bounds to an overall adaptivity bound. $\text{AdaptSearch}(c, G_{\text{est}}(c))$ algorithm in Algorithm 3 arranges the estimated dependency graph $G_{\text{est}}(c)$ into SCCs $(G^{\text{SCC}}_1, \dots, G^{\text{SCC}}_n)$ and obtains the adaptivity local bound of each SCC from $\text{AdaptSearch}_{\text{SCC}}(c, G^{\text{SCC}}_i)$ algorithm in Algorithm 4. Then AdaptSearch shrinks the estimated dependency graph into a directed acyclic graph (DAG) by reducing each SCC into a vertex with the weight equal to its adaptivity local bound. In this way, it simply computes the length of the longest path over this DAG.

We discuss in detail this adaptivity computation algorithm.

The Adaptivity Computation Algorithm ($\text{AdaptSearch}(c, G_{\text{est}}(c))$) At Line:3, this algorithm first finds all the SCCs of $G_{\text{est}}(c)$, $G^{\text{SCC}}_1, \dots, G^{\text{SCC}}_n$ where $0 \leq n \leq |V|$ by the standard Kosaraju's algorithm, where each $G^{\text{SCC}}_i = (V_i, E_i, W_i, Q_i)$. Then, it computes the local adaptivity upper bound for every G^{SCC}_i in line:4-5 by $\text{AdaptSearch}_{\text{SCC}}(c, G^{\text{SCC}}_i)$. We guarantee that $\text{AdaptSearch}_{\text{SCC}}(c, G^{\text{SCC}}_i)$ computes a sound upper bound on the adaptivity of an SCC by Lemma A.5.1 with formal proof in Appendix A.5. The $G_{\text{est}}(c)$ is then shrunk into a directed acyclic graph (DAG) where $G^{\text{SCC}}_1, \dots, G^{\text{SCC}}_n$ are all the vertices and the adaptivity local bounds are their weights. There is an edge $s_i \rightarrow s_j$ in this shrank graph, as long as we can find an edge $v_i \rightarrow v_j \in E_{\text{est}}(c)$ such that $v_1 \in V_i, v_j \in V_j$ and $i \neq j$. Then, we use the standard breath-first search strategy to find the longest-weighted path on this DAG and return this length as

Algorithm 3 Adaptivity Computation Algorithm ($\text{AdaptSearch}(c, G_{\text{est}}(c))$)

Require: A program c and its estimated dependency graph: $G_{\text{est}}(c) = (V, E, W, Q)$

```

1: init
    $q$ : empty queue.
    $\text{adapt}$ : the adaptivity of this graph initialize with 0.
2: Find all strongly connected components in  $G$ :  $G^{\text{SCC}}_1, \dots, G^{\text{SCC}}_n, 0 \leq n \leq |V|$ ,
3: for every component  $G^{\text{SCC}}_i$  compute its local adaptivity:
4:    $\text{adapt}_{\text{scc}}[G^{\text{SCC}}_i] = \text{AdaptSearch}_{\text{scc}}(c, G^{\text{SCC}}_i)$ ;
5: for every  $G^{\text{SCC}}_i$ :
6:    $q.\text{append}(G^{\text{SCC}}_i)$ ;
7:    $\text{adapt}_{\text{tmp}} = 0$ ;
8:   while  $q$  isn't empty:
9:      $s = q.\text{pop}()$ ; #{take the top component from head of queue}
10:     $\text{adapt}_{\text{tmp}_0} = \text{adapt}_{\text{tmp}}$ ; #{record the adaptivity of last level}
11:     $G^{\text{SCC}}_{\text{max}}$ ; #{record the SCC with longest walk in this level}
12:    for every component  $s' \neq s$  and connected by  $s$ :
13:      if ( $\text{adapt}_{\text{tmp}} < \text{adapt}_{\text{tmp}_0} + \text{adapt}_{\text{scc}}[s']$ ):
14:         $\text{adapt}_{\text{tmp}} = \text{adapt}_{\text{tmp}_0} + \text{adapt}_{\text{scc}}[s']$ ;
15:         $G^{\text{SCC}}_{\text{max}} = s'$ ; #{record the SCC with the longest walk}
16:      if ( $G^{\text{SCC}}_{\text{max}} \neq s'$ ):
17:         $q.\text{append}(G^{\text{SCC}}_{\text{max}})$ ;
18:     $\text{adapt} = \max(\text{adapt}, \text{adapt}_{\text{tmp}})$ ;
19: return  $\text{adapt}$ .

```

the adaptivity upper bound.

We guarantee that the length of this longest weighted path is a sound computation of the adaptivity for program c and this longest weighted path is a sound computation of the finite walk having the longest query length on c 's estimated dependency graph by Theorem 6.5.2 in Appendix A.5. If a program c 's estimated dependency graph $G_{\text{est}}(c)$ is a DAG, then we prove that the adaptivity upper bound by Algorithm 3 is tight by Theorem A.6.1 in Appendix A.6.

Adaptivity Computation for An Strongly Connected Component of $G_{\text{est}}(c)$. This algorithm takes the program and a strongly connected component, G^{SCC}_i of a program's estimated dependency graph $G_{\text{est}}(c)$ as input and outputs the adaptivity local bound of G^{SCC}_i . For a strongly connected component that contains only one vertex without any edge, it returns the query annotation of this vertex as adaptivity. For a strongly connected component containing at least one edge, there are three steps in this algorithm: 1. It first collects all the finite paths in the input graph 2. Then it calculates the adaptivity of every path by a novel adaptivity computation method. 3. The maximal adaptivity among all paths is the adaptivity of this component in the end. Because the input graph is SCC, when the algorithm starts to traverse from a vertex, it finally goes back to the same vertex. In this sense, the paths collected in step 1 are all simple cycles with the same starting and ending vertex. The most interesting part is step 2. It recursively computes the adaptivity upper bound on the fly of paths collecting through a depth-first search procedure dfs from the line: 5-15. It designs a novel adaptivity computation method, which guarantees the visiting times of each vertex by its weight and addresses the **Approximation Challenge**. The guarantee is achieved by two special parameters flowcapacity and querynum and the updating operations in line:7 and line:10.

Algorithm 4 Adaptivity Computation on An SCC ($\text{AdaptSearch}_{\text{scc}}(c, G^{\text{scc}}_i)$)

Require: The program c ,

An strongly connected component of $G_{\text{est}}(c)$: $G^{\text{scc}}_i = (V_i, E_i, W_i, Q_i)$

```

1: init
    $r_{\text{scc}}$ :  $A_{\text{scc}}(c)$ , initialized 0, the Adaptivity of this SCC
2:   init
       $\text{visited} = [0] * |V_i|$  #{a list of length  $|V|$ , initialize with 0 for every vertex.}
       $r = [Q(v)] * |V_i|$  :  $A_{\text{scc}}(c)$  List,
      #{length  $|V_i|$ , initialize with  $Q(v)$  for every vertex, recording the adaptivity
      reaching each vertex.}
       $\text{flowcapacity} = [\infty] * |V_i|$  :  $A_{\text{scc}}(c)$  List,
      #{length  $|V_i|$ , initialize with  $\infty$  for every vertex, recording the minimum
      weight when the walk reaching that vertex, inside a cycle}
       $\text{querynum} = [Q(v)] * |V_i|$  :  $\mathbb{N}$  List,
      #{length  $|V_i|$ , initialize with  $Q(v)$  for every vertex, recording the query num-
      bers when the path reaching that vertex, inside a cycle}
3:   if  $|V_i| = 1$  and  $|E_i| = 0$ :
4:     return  $Q(v)$ 
5:   def  $\text{dfs}(G, s, \text{visited})$ :
6:     for every vertex  $v$  connected by a directed edge from  $s$ :
7:       if  $\text{visited}[v] = \text{false}$ :
8:          $\text{flowcapacity}[v] = \min(W_i(v), \text{flowcapacity}[s]);$ 
9:          $\text{querynum}[v] = \text{querynum}[s] + Q_i(v);$ 
10:         $r[v] = \max(r[v], \text{flowcapacity}[v] \times \text{querynum}[v]);$ 
11:         $\text{visited}[v] = 1;$ 
12:         $\text{dfs}(G, v, \text{visited});$ 
13:       else: #{There is a cycle finished}
14:          $r[v] = \max(r[v], r[s]$ 
            $+ \min(W_i(v), \text{flowcapacity}[s]) * (\text{querynum}[s] + Q_i(v)));$ 
           #{update the length of the longest walk reaching this vertex on this cycle}
15:       return  $r[c]$ 
16:   for every vertex  $v$  in  $V_i$ :
17:     initialize the  $\text{visited}, r, \text{flowcapacity}, \text{querynum}$  with the same value at
     line:2.
18:    $r_{\text{scc}} = \max(r_{\text{scc}}, \text{dfs}(G^{\text{scc}}_i, v, \text{visited}));$ 
19: return  $r_{\text{scc}}$ 

```

- `flowcapacity` is a list of arithmetic expression $\mathcal{A}_{\text{in}}$. It tracks the minimum weight along the path during the searching procedure. For each vertex, it updates the minimum weight when the path reaches that vertex with ∞ as the initial value.
- `querynum` is a list of integer initialized by query annotation $Q_i(v)$ for every vertex. It tracks the total number of vertices with query annotation 1 along the path.
- The updating operation during the traversal (line: 7) and at the end of the traverse (line: 10) is $\text{flowcapacity}[v] \times \text{querynum}[v]$. Because `querynum[v]` is the # of the vertices with query annotation 1 and `flowcapacity[v]` is the minimum weight over this path, this number is the accurate query length of this path. It guarantees the visiting times of each vertex on the path reaching a vertex v is no more than the maximum visiting times it can be on a qualified walk by `flowcapacity[v]`, and at the same time compute the query length instead of the weighted length through `querynum[v]`.

In this way, we resolve the **Approximation Challenge** without losing the soundness, formally in Appendix A.5. This step also guarantees the termination through a boolean list, `visited` in line:7 and line:13.

Algorithm Detail Steps The detail steps of dfs from the line: 2-15 in Algorithm 4 is described as follows.

Line:2 initialize parameters:

1. `flowcapacity` is a list of arithmetic expressions with length $|Q_i(c)|$ and the initial value ∞ for every element. For every vertex, it records the minimum weight when the path traverses to this vertex.

2. `querynum` is a list of integer with length $|V_i(c)|$ and the initial value $Q_i(v)$ for every element. For every vertex, it records the total query numbers when the path traverses to this vertex.

3. The `visited` is initialized by 0 for every element and has length $|V_{\text{est}}(c)|$ as well. It is used to guarantee termination during recursion.

4. `r` is a list of $A_{\text{sc}}(c)$ initialized with query annotation for each vertex. For each vertex, it maintains the longest query length when the recursion reaches it.

Line:7-12 updates the parameters and recursively traverses for every unvisited vertex heading out from v . In each recursion, Line:8 maintains the minimum weight for the flowcapacity and Line:9 updates the number of query vertices `querynum` so far when the traversing reaches v . Line:10 updates the longest query length `r` along the path when the traverse arrives at vertex v by the operation $\text{flowcapacity}[v] \times \text{querynum}[v]$. This computation guarantees: 1. The visiting times of each vertex on the walk reaching v is no more than the maximum visiting it can be on this walk; 2. Only the vertices have annotation 1 are counted in adaptivity. In this way, we accurately approximate a walk using this path and compute the query length of this walk safely. This addresses the **Approximation Challenge** and at the same time without losing the soundness.

At line: 14, if this vertex v is visited, i.e., the traverse of this path goes back to its starting point, we only update the longest query length $r[v]$ for v in the same way as Line:11. However, we do not update `querynum` and `flowcapacity` in this case. This improves the accuracy and still guarantees the soundness, formally by Lemma 6.5.2. We also discuss how these computations guarantee soundness and improve accuracy in the following example.

Example The example program in Figure 6.6 illustrates how these special oper-

ations compute accurate and sound adaptivity for the program. `AdaptSearch` first find the SCC contains vertices y^6 and x^9 , $\text{SCC} = (V, E)$ where $V = \{y^6, x^9\}$ and $E = \{(y^6, y^6), (x^9, x^9), (x^9, y^6), (y^6, x^9)\}$. Then `AdaptSearchscc(SCC, nestedW)` takes this SCC as input. When starting from vertex y^6 , it first finds the path $y^6 \rightarrow y^6$. By updating parameters through Line:10 and 14, it computes the longest query length for this path as k . As highlighted in Line:14, we do not update `querynum` and `flowcapacity` when we identify the simple cycle $y^6 \rightarrow y^6$. This improves the accuracy and still guarantees soundness. Because in the following recursions, we continuously search for walks heading out from y^6 , the `flowcapacity` of this simple cycle does not restrict the walks going out of this vertex that does not interleave with the cycle $y^6 \rightarrow y^6$. However, if we keep updating the minimum weight, then we restrict the visiting times of vertices on a walk by using the minimum weight of vertices that do not on this walk. This leads to the unsoundness in computing adaptivity. Concretely, if we update the `flowcapacity`[y^6] as k after visiting y^6 the second time on this walk, and continuously visit x^9 , then the `flowcapacity`[k] is updated as $\min(k, k^2)$. So the visiting times of x^9 is restricted by k on the walk $y^6 \rightarrow y^6 \rightarrow x^9$. This restriction excludes the finite walk $y^6 \rightarrow y^6 \rightarrow x^9 \rightarrow x^9$ where y^6 and x^9 visited by k^2 times in the computation. However, the finite walk $y^6 \rightarrow y^6 \rightarrow x^9 \rightarrow x^9$ where y^6 is visited k times and x^9 k^2 times is a qualified walk, and exactly the longest walk we aim to find. So, by Non-updating the `flowcapacity` after visiting y again, we guarantee that the visiting times of vertices on every searched walk will not be restricted by weights not on this walk, i.e., the soundness. Line: 15 returns the adaptivity heading out from its input vertex. Line 16-18 applies `dfs` on every vertex of this SCC and computes the adaptivity of this SCC by taking the maximum value. The soundness is formally guaranteed in

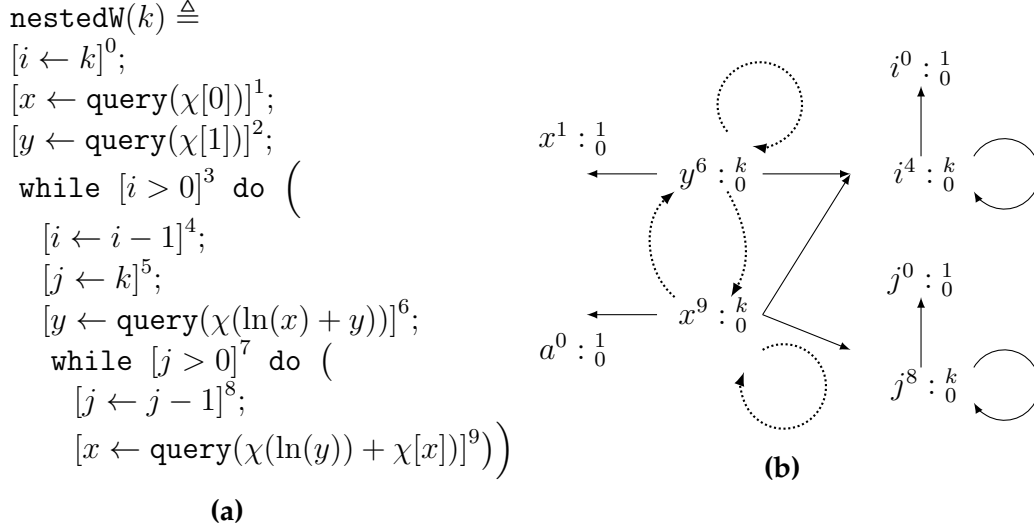


Figure 6.6: (a) A two-level nested loop example program. (b) The estimated dependency graph generated from AdaptFun.

Lemma A.5.1 in Appendix A.5.

6.6 EXAMPLES

We use the same walk through example illustrates the static analysis algorithm.

Example I. Below in Figure 6.7(c) we show again the program and the graph, which is the same walk-through example as Figure 2.4(a). The estimated dependency graph that our static analysis algorithm returns for the program `twoRounds(k)` is in Figure 6.7(a). Vertices and query annotations are the same as the ones in Figure 6.7(b) and they are simply inferred by scanning the program. As we said before, the edges are estimated using control flow and data flow analysis. For the `twoRounds(k)` example, every edge in Figure 6.7(b) is precisely inferred by our combined analysis, this is why Figure 6.7(c) contains exactly the same edges. The weight of every vertex is computed using a reachability-bound estimation algo-

rithm that outputs a symbolic expression over the input variables, in the example only k , representing an upper bound on the number of times each assignment is executed. For example, consider the vertex x^3 , its weight is k and this provides an upper bound on the values returned by the weight function $\lambda\tau.\rho(\tau)k$ associated with vertex x^3 in Figure 6.7(b) for any initial state.

The algorithm searching for the longest walk first finds a path $l^6 : \frac{1}{1} \rightarrow a^5 : \frac{k}{1} \rightarrow x^3 : \frac{k}{1}$, and then constructs a walk based on this path. Every vertex on this walk is visited once, and the number of vertices with query annotation 1 in this walk is 2, which is the upper bound we expect. It is worth noting here that x^3 and a^5 can only be visited once because there isn't an edge to go back to them, even though they both have the weight k . In this sense, instead of simply computing the weighted length of this path ($2k + 1$) as adaptivity AdaptSearch computes the upper bound 2. Note that 2 is not always tight, for example, when $k = 0$.

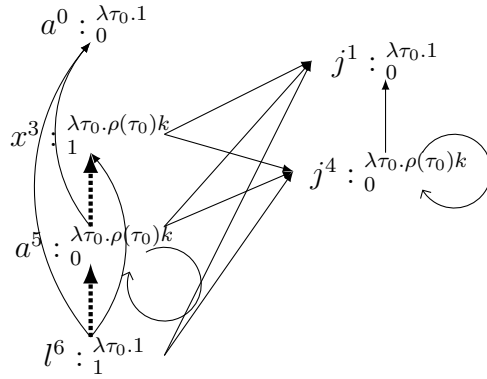
Example II. The example program from Figure 6.4(a) has similar estimated dependency graph as the twoRounds example. In the same way, we find its longest walk as $l^6 \rightarrow a^5 \rightarrow y^3 \rightarrow x^0$. Different from the twoRounds example, there is a dependency relation from the first query request, $[x \leftarrow \text{query}(\chi[0])]^0$ to the commands inside the loop body passing through control flow. In this sense, it has adaptivity 3, and we find its longest walk as $l^6 \rightarrow a^5 \rightarrow y^3 \rightarrow x^0$. The query length of this walk is 3 given that only the three vertices l^6, y^3 and x^0 correspond to query requests.

```

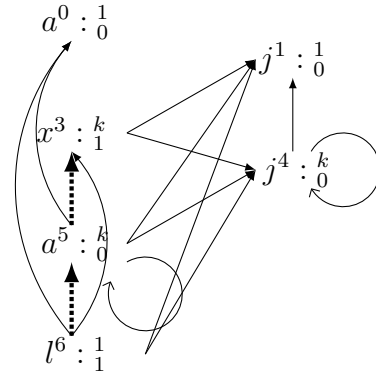
towRounds(k)  $\triangleq$ 
 $[a \leftarrow 0]^0; [j \leftarrow k]^1;$ 
  while  $[j > 0]^2$  do  $\left( [x \leftarrow \text{query}(\chi[j] \cdot \chi[k])]^3; [j \leftarrow j - 1]^4; [a \leftarrow x + a]^5 \right);$ 
 $[l \leftarrow \text{query}(\chi[k] * a)]^6$ 

```

(a)



(b)



(c)

Figure 6.7: (a) The same running example program `towRounds(k)` as Figure 2.4(a). (b) The corresponding semantics-based dependency graph. (c) The estimated dependency graph from AdaptFun.

CHAPTER 7

Examples and Evaluation Results

7.1 EXAMPLES

We illustrate here how the **AdaptFun** works in three examples.

Example 7.1.1 (Multiple Rounds Algorithm). *Our first example, `multipleRounds` in Figure 7.1(a), is an advanced adaptive data analysis algorithm. This is a simplified version of the Monitor Augment from Rogers et al. (2020) with the complete program in the Appendix. It takes the user input k which decides the the number of iterations. It starts from an initialized empty tracking list I , goes k rounds, and at every round, tracking list I is updated by a query result of $\text{query}(\chi[I])$. After r rounds, the algorithm returns the columns of the hidden database D not specified in the tracking list I . The functions $\text{updnscore}(p, a)$, $\text{updcscore}(p, a)$, $\text{update}(I, ns, cs)$ simplify the computations of updating ns , cs and I .*

Different from $\text{twoRounds}(k)$ in Figure 6.7, the query request, $[a \leftarrow \text{query}(I)]^6$ in each loop iteration is not independent. $\text{query}(I)$ in each iteration depends on the tracking list I from all the previous iterations, and I is updated by all the query results in the previous iterations as well. In this sense, all these k queries are adaptively chosen according to our discussion in the overview. The program-based dependency graph is presented in Figure 7.1(b). We omitted its execution-based dependency graph $G_{\text{trace}}(\text{multipleRounds}(k))$ because they have the same graph topology and only differ in weight. For each vertex v in $G_{\text{est}}(\text{multipleRounds}(k))$ in Figure 7.1(b), we use w_v to denote its weight function in $G_{\text{trace}}(\text{multipleRounds}(k))$.

As the adaptivity definition in our formal adaptivity model in Definition 23, there is a finite walk along the dashed arrows, $a^6 \rightarrow I^9 \rightarrow ns^7 \rightarrow \dots \rightarrow ns^7$, where the vertices a^6 , I^9 and ns^7 are visited $w_{a^6}(\tau_0)$, $w_{I^9}(\tau_0)$ and $w_{ns^7}(\tau_0)$ times respectively with input τ_0 .

The vertex a^6 has query annotation 1, and it is visited $w_{a^6}(\tau_0)$ times. In this sense, the adaptivity of this program is $w_{a^6}(\tau_0)$ given input τ_0 , i.e., $A(\text{multipleRounds}(\mathbf{k}))(\tau_0) = w_{a^6}(\tau_0)$. Since $w_{a^6}(\tau_0)$ counts the execution times of command $[a \leftarrow \text{query}(I)]^6$; this count is at most the loop iteration numbers, i.e., k 's initial value, $\rho(\tau_0)k$ from the initial trace τ_0 . Next, we show that **AdaptFun** provides a tight upper bound for this example by $\text{AdaptSearch}(\text{multipleRounds}(\mathbf{k}))$. It first finds a path on $G_{\text{est}}(\text{multipleRounds}(\mathbf{k}))$ $a^6 : \frac{k}{1} \rightarrow I^9 : \frac{k}{0} \rightarrow ns^7 : \frac{k}{0}$ with three weighted vertices. Then AdaptSearch approximates this path to a walk $a^6 : \frac{k}{1} \rightarrow I^9 : \frac{k}{0} \rightarrow ns^7 : \frac{k}{0} \rightarrow a^6 : \frac{k}{1} \dots$. In this walk, a^6, I^9, ns^7 are all visited k times respectively. So $A_{\text{est}}(\text{multipleRounds}(\mathbf{k})) = k$. We know for any initial trace τ_0 , $\langle \tau_0, k \rangle \Downarrow_e \rho(\tau_0)$ So we guarantee $A(\text{multipleRounds}(\mathbf{k}))(\tau_0) \leq \rho(\tau_0)$ for any τ_0 and k is a sound bound.

```

multipleRounds(k)  $\triangleq$ 
 $[j \leftarrow k]^0; [I \leftarrow []]^1; [ns \leftarrow 0]^2; [cs \leftarrow 0]^3;$ 
  while  $[j > 0]^4$  do
     $([j \leftarrow j - 1]^5; [a \leftarrow \text{query}(I)]^6; [ns \leftarrow \text{updnscore}(ns, a)]^7;$ 
     $[cs \leftarrow \text{updcscore}(cs, a)]^8; [I \leftarrow \text{updI}(I, ns, cs)]^9)$ 

```

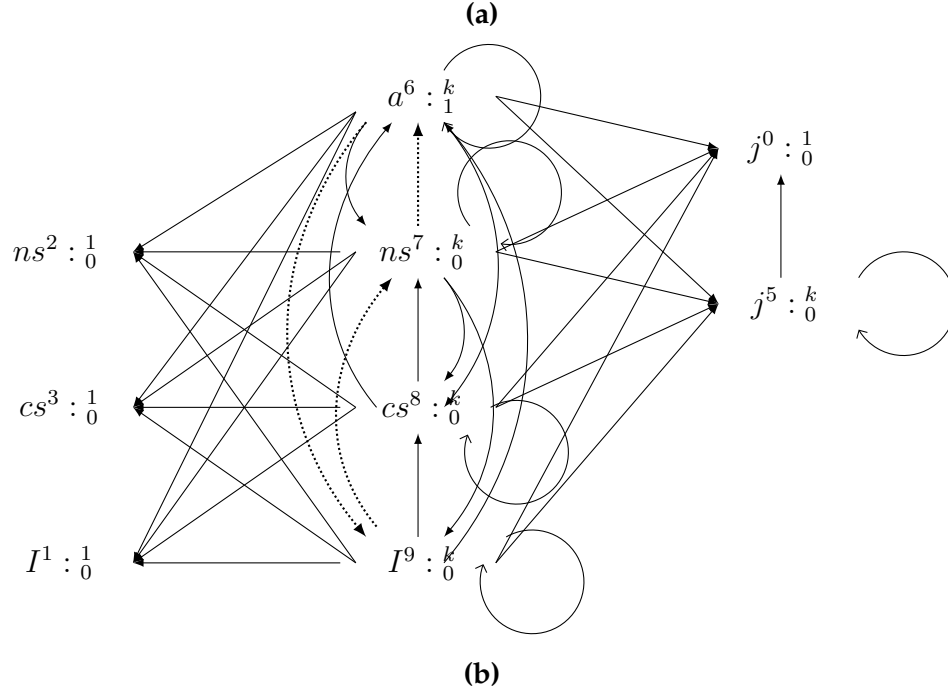


Figure 7.1: (a) The simplified example program with multiple adaptivity rounds. (b) The estimated dependency graph by AdaptFun

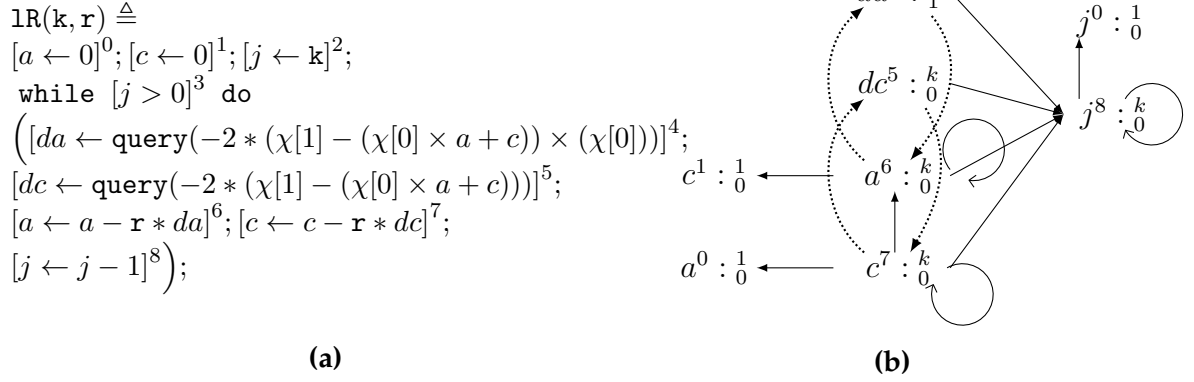


Figure 7.2: (a) The linear regression algorithm with gradient descent optimization. (b) The estimated dependency graph by AdaptFun

Example 7.1.2 (Linear Regression Algorithm with Gradient Decent Optimization). Our *AdaptFun* works well also over the linear regression algorithm with a gradient descent optimization method. It computes $A_{\text{est}}(1R(k, r)) = k$.

This linear regression algorithm aims to find a linear relationship, $y = a \times x + c$ between a dependent variable y and an independent variable x , by approximating the model parameter a and c . In order to have a good approximation of the model parameter a and c , it sends a query to a training data set adaptively in each iteration. This training data set contains two columns (which can extend to higher dimensional data sets), the first column contains the observed values of the independent variable x and the second column for the dependent variable y . $1R(k, r)$ is the program of this example written in our language model in Figure 7.2(a) with input variables k and r .

$1R(k, r)$ starts from initializing the linear model parameters and the counter variable in commands 0, 1, 2, and then goes into the while iterations. In each iteration, it computes the differential value w.r.t. parameter a and c , through two query requests, $\text{query}(-2 * (\chi[1] - (\chi[0] \times a + c)) \times (\chi[0]))$ and $\text{query}(-2 * (\chi[1] - (\chi[0] \times a + c)))$

in commands of label 4 and 5. Then, it uses these two differential values stored in variables da and dc to update the model parameters a and c . In this sense, the two query requests in each iteration depend on queries in the previous iterations, and the intuitive adaptivity rounds are k . Its program-based dependency graph, $G_{\text{est}}(\text{1R}(k, r))$ is shown in Figure 7.2(b), and $G_{\text{trace}}(\text{1R}(k, r))$ is omitted for the same reason as Example 7.1.1. In Figure 7.2(b), we omit the edges which are constructed by the transition of `flowsTo` relation for concise, but these edges exist in $G_{\text{trace}}(\text{1R}(k, r))$ because they can be constructed directly by DEP_{var} relation. Given an input τ_0 , there are multiple walks having the same longest query length in $G_{\text{trace}}(\text{1R}(k, r))$, such as the walks $c^7 \rightarrow dc^5 \rightarrow c^7 \rightarrow \dots \rightarrow dc^5$ and $a^6 \rightarrow da^4 \rightarrow a^6 \rightarrow \dots \rightarrow da^4$ along the dotted arrows. The vertices c^7, dc^5, a^6, da^4 in the two walks are visited $w_{c^7}, w_{dc^5}, w_{a^6}, w_{da^4}$ respectively. Since all of them are in the same path of the loop, their weight functions are expected to be the same, i.e., $\lambda\tau_0 \cdot \rho(\tau_0)k$. Specifically the value of the input variable in the initial trace, which is also the loop iterations. In this sense, $A(\text{1R}(k, r)) = w_{dc^5} = w_{da^4} = \lambda\tau_0 \cdot \rho(\tau_0)k$. Similar to Example 7.1.1, `AdaptFun` estimates the tight adaptivity bound, k for this example.

Example 7.1.3 (Multiple Rounds Odds Algorithm). *The **AdaptFun** comes across an over-approximation due to its path-insensitive nature. It occurs when the control flow can be decided in a particular way in front of conditional branches, while the static analysis fails to witness. As in Figure 7.3(a), `multiRounds0(k)` is an example program with $1 + k$ adaptivity rounds and two paths while loop. In each iteration, the query $[y \leftarrow \text{query}(\chi[x])]^5$ and $[p \leftarrow \text{query}(\chi[x])]^6$ are based on previous query results stored in x , which is similar to Example 7.1.1. The difference is that, only the query answer from $[y \leftarrow \text{query}(\chi[x])]^5$ in the first branch is used in the query in command 7, $\text{query}(\chi[\ln(y)])$, and the first branch is only executed in even iterations ($i = 0, 2, \dots$). From the Semantics-based dependency graph in Figure 7.3(b),¹ the weight $w_{y^5}(\tau_0)$ for the vertex y^5 will count the precise evaluation times of $[y \leftarrow \text{query}(\chi[x])]^5$, i.e., half of the iteration numbers. This number is expected to be half of the initial value of input k from τ_0 . However, **AdaptFun** fails to realize that all the odd iterations only execute the first branch and only even iterations execute the second branch. So it considers both branches for every iteration when estimating the adaptivity. In this sense, the weight estimated for y^5 and p^6 are both k as in Figure 7.3(c). As a result, **AdaptFun** computes $y^5 \rightarrow x^7 \rightarrow y^5 \rightarrow \dots \rightarrow x^7$ as the longest walk in Figure 7.3(c) where each vertex is visited k times. In this sense, the estimated adaptivity is $1 + 2 * k$, instead of $1 + k$.*

7.2 EVALUATION RESULTS

We implemented **AdaptFun** as a tool that takes a labeled command as input and outputs two upper bounds on the program adaptivity and the number of query requests respectively. This implementation consists of an abstract control flow graph

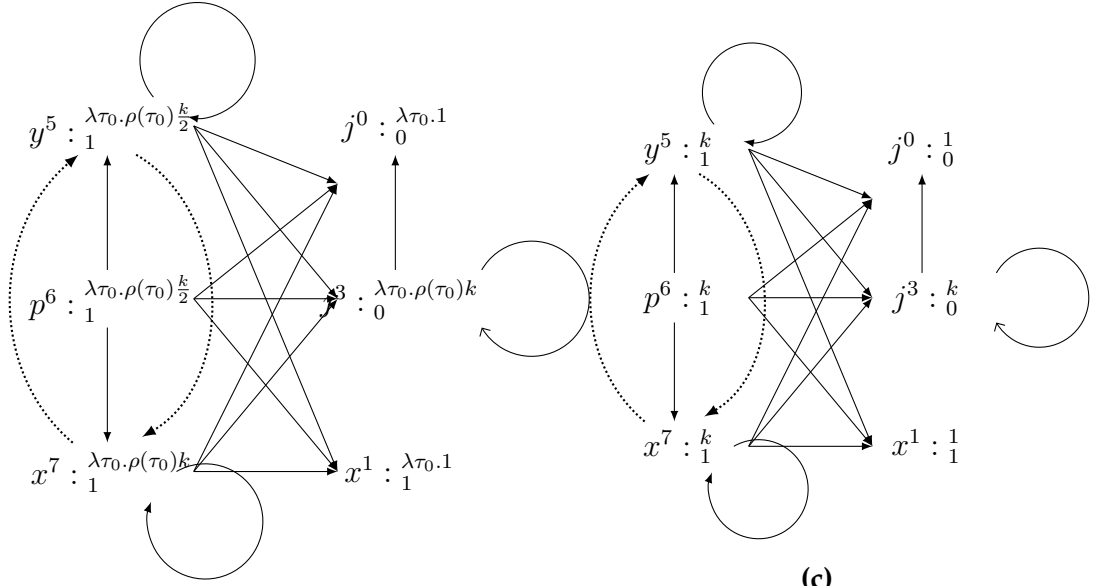
¹Similar to the Figure 7.2(b), we omit some edges which are constructed by the transition of `flowsTo` relation for concise, but these edges exist in $G_{\text{trace}}(\text{multiRounds0}(k))$ because they can be constructed directly by DEP_{var} relation. These omissions don't affect the illustration and adaptivity computation.


```

multiRounds0( $k$ )  $\triangleq$ 
 $[j \leftarrow k]^0; [x \leftarrow \text{query}(\chi[0])]^1;$ 
  while  $[j > 0]^2$  do
    ( $[j \leftarrow j - 1]^3; \text{if } ([j \% 2 == 0])^4, [y \leftarrow \text{query}(\chi[x])]^5, [p \leftarrow \text{query}(\chi[x])]^6;$ 
     $[x \leftarrow \text{query}(\chi(\log y))]^7)$ 

```

(a)



(b)

(c)

Figure 7.3: (a) The multiple-path loop program with multiple adaptivity rounds. (b) The corresponding semantics-based dependency graph. (c) The estimated dependency graph from AdaptFun.

generation, edge estimation (as presented in Section 6.3.2), and weight estimation (as presented in Section 6.3.3) in Ocaml, and the adaptivity computation algorithm shown in Section 6.5 in Python. The OCaml program takes the labeled command as input and outputs the program-based dependency graph and the abstract transition graph, feeds into the python program and the python program provides the adaptivity upper bound and the query number as the final output.

We evaluated this implementation on 23 example programs with the evaluation results shown in Table 7.1.

Table 7.1: Experimental results of AdaptFun implementation

Program c	$adaptivity$	AdaptFun		L.O.C	running time (second)		
		A_{est}	query#		graph	weight	AdaptSearch
twoRound(k)	2	2	$k + 1$	8	0.0005	0.0017	0.0003
mR(k)	k	k	k	10	0.0012	0.0017	0.0002
lRGD(k, r)	k	k	$2k$	10	0.0015	0.0072	0.0002
mROdd(k)	$1 + k$	$2 + \max(1, 2k)$	$1 + 3k$	10	0.0015	0.0061	0.0002
mRSingle(k)	2	$1 + \max(1, k)$	$1 + k$	9	0.0011	0.0075	0.0002
ifCD()	3	3	3	5	0.0005	0.0003	0.0001
loop(k)	$1 + \frac{k}{2}$	$1 + \max(1, \frac{k}{2})$	$1 + \frac{k}{2}$	7	0.0021	0.0015	0.0001
loopRV(k)	$1 + 2k$	$1 + 2k$	$2 + 3k$	9	0.0016	0.0056	0.0001
loopVCD(k)	$1 + 2Q_m$	$Q_m + \max(1, 2Q_m)$	$2 + 2Q_m$	6	0.0016	0.0007	0.0001
loopMPVCD(k)	$2 + Q_m$	$2 + Q_m$	$2 + 2Q_m$	9	0.0017	0.0043	0.0001
loop2VD(k)	$2 + k^2$	$3 + k^2$	$1 + k + k^2$	10	0.0018	0.0126	0.0001
loop2RV(k)	$1 + k + k^2$	$2 + k + k^2$	$2 + k + k^2$	10	0.0017	0.0186	0.0001
loop2MV(k)	$1 + 2k$	$1 + \max(1, 2k)$	$1 + k + k^2$	10	0.0016	0.0071	0.0001
loop2MPRV(k)	$1 + k + k^2$	$3 + k + k^2$	$2 + 2k + k^2$	10	0.019	0.0999	0.0002
loopMP(k)	$1 + k$	$2 + \max(1, 2k)$	$1 + 3k$	9	0.0017	0.0062	0.0001
loop2MP(k)	$1 + k$	$2 + k$	$1 + 3k$	9	0.0017	0.0062	0.0001
loop2RV(k)	$1 + 3k$	$1 + 3k$	$1 + 3k + k^2$	11	0.019	0.2669	0.0007
mRCom(k, c, N)	k	k	k	27	0.0026	85.9017	0.0004
mRCop(k)	$2k$	$2k$	$2k$	46	0.0036	5104	0.0013
seqC(k)	12	12	326	502	0.0426	1.2743	0.0223
tRC(k)	2	*	*	42	0.0026	*	0.0005
jumbo(k)	$\max(20, 8 + k^2)$	*	*	71	0.0035	*	0.0085
big(k)	$22 + k + k * k$	*	*	214	0.0175	*	0.002

In this table, the first column is the name of each program. For each program c , the second column is its intuitive adaptivity rounds, and the third column is the output of the AdaptFun implementation, which consists of two expressions. The first one is the upper bound for adaptivity and the second one is the upper bound

for the total number of query requests in the program. And the last column is the performance evaluation w.r.t. the program size.

The last column is the performance evaluation. The time contains three parts. The first part is the running time of the Ocaml code, which parses the program and generates the $G_{\text{est}}(c)$. The second and third parts are the running times of the reachability bound analysis algorithm and the adaptivity computation algorithm, $\text{AdaptSearch}(c)$.

The first six programs are adapted from real-world data analysis algorithms. The first two programs $\text{twoRounds}(k)$, $\text{multiRounds}(k)$ are the same as Figure 2.4(a) and Figure 7.1(a). AdaptFun computes tight adaptivity bound for the first three examples. For the fourth program $\text{multiRounds0}(k)$, AdaptFun outputs an over-approximated upper bound $1 + 2 * k$ as discussed in Example 7.1.3. The fifth program is the evaluation results for the example in Example 5.4.2, where AdaptFun outputs the tightly bound for $A(c)$ but $A(c)$ is a loose definition of the program's actual adaptivity rounds.

The next eleven programs from Table 7.1 rows 7 to 18 are handcrafted programs based on the code pieces extracted from the C library. They all have small sizes but complex structures in order to test the programs under different situations including data, control dependency, the multiple paths nested loop with related counters, etc. The names of these programs obey the convention that, *if* means there is if control in the program; *loop* means there is while loop and *loop2* represents two levels nested loop in the program; *C* denotes Control; *D* for Dependency; *V* for Variable; *M* for Multiple; *P* for Path and *R* for Related.

The algorithm computes the tight bound for examples from line six, $\text{ifCD}()$ to line fourteen, $\text{loop2MPRV}(k)$ and over-approximate the *adaptivity* for the 15th and

16th examples in the table due to path-insensitivity.

The last six programs are synthesized programs composed of the previous programs in order to test the performance limitation when the input program is large. From the evaluation results, the performance bottleneck is the reachability bound analysis algorithm.

By implementing the bound analysis algorithm in Section 6.3.3 (adapted from Sinn et al. (2017)), we are unable to evaluate the Jumbo in a reasonable time period. Alternatively, we implement another light reachability bound analysis algorithm and compute the *adaptivity* for jumboS, jumbo and big effectively. We also show our alternative implement evaluation result in Table 7.2. The alternative implementation computes the tight bound for all the examples from line:1 to line:14 and over-approximate the *adaptivity* for 15th and 16th due to path-insensitivity similar to the AdaptFun. For the 17th example (loop2RC), AdaptFun gives a tight bound while the alternative implementation gives a loose bound, so we keep both implementations.

Overall, AdaptFun gives the accurate estimated adaptivity upper bound and our adaptivity formalization also gives precise adaptivity definition w.r.t. the intuitive *adaptivity rounds*.

Table 7.2: Results of the alternative AdaptFun implementation

Program c	AdaptFun-I		running time (second)
	A_{est}	query#	
twoRound(k)	2	$k + 1$	0.0010
mR(k)	$\max(1, k)$	k	0.0016
lRGD(k, r)	$\max(1, k)$	$2k$	0.0019
mROdd(k)	$2 + \max(1, 2k)$	$1 + 3k$	0.0019
mRSingle(k)	$1 + \max(1, k)$	$1 + k$	0.0015
ifCD()	4	4	0.0007
loop(k)	$1 + \max(1, \frac{k}{2})$	$1 + \frac{k}{2}$	0.0023
loopRV(k)	$1 + \max(1, 2k)$	$2 + 3k$	0.0019
loopVCD(k)	$Q_m + \max(1, 2Q_m)$	$2 + 2Q_m$	0.0019
loopMPVCD(k)	$2 + Q_m$	$2 + 2Q_m$	0.0020
loop2VD(k)	$3 + k^2$	$1 + k + k^2$	0.0021
loop2RV(k)	$2 + k + k^2$	$2 + k + k^2$	0.0021
loop2MV(k)	$1 + \max(1, 2k)$	$1 + k + k^2$	0.0019
loop2MPRV(k)	$3 + k + k^2$	$2 + 2k + k^2$	0.0194
loopMP(k)	$2 + \max(1, 2k)$	$1 + 3k$	0.0021
loop2MP(k)	$2 + k$	$1 + 3k$	0.0021
loop2RV(k)	$2 + 3k + k^2$	$1 + k + k^2$	0.0199
mRCom(k, N)	k	k	0.0033
mRCop(k)	$2k$	$2k$	0.0052
seqC(k)	12	326	0.0652
tRC(k)	2	$1 + 5k + 2k^2$	0.0034
jumbo(k)	$\max(20, 6 + k + k^2)$	$44 + k + k^2$	0.0123
big(k)	$28 + k + k^2$	$121 + 11k + 4k^2$	0.0181

The programs we evaluated in the Appendix A.7 and Appendix A.8.

Below we discuss in detail the real-world adaptive data analyses that we evaluated. The first one is the two rounds adaptivity data analysis which we present as the walk-through example. The second one is the complete multiple rounds adaptive data analysis for the simplified version that we discussed in Example 7.1.1.

The Two Rounds Adaptive Data Analysis Algorithm, trComplete

Example 7.2.1 (The Two Rounds Analyst Strategy). *Below is the complete two rounds analyst strategy for random data algorithm. This is instantiated from the Custom Adaptive Analyst Strategy, the Algorithm 5 in Rogers et al. (2020) by setting the adaptive queries indices parameter, S as the last column $\{k\}$.*

We also have the complete implementation of the algorithm above in our language be-

Algorithm 5 The complete *two rounds analyst strategy* for random data

Require: Mechanism $\mathcal{M}$ with a hidden data set $D \in \{-1, +1\}^{n \times (k+1)} \subset \mathcal{DB}$.

for $j \in [k]$ **do**.

define $q_j(d) = d(j) \cdot d(k)$ where $d \in \{D(i) \mid i = 0, \dots, n\} \subseteq \{-1, +1\}^{k+1}$.

let $a_j = \mathcal{M}(q_j)$

 {In the line above, $\mathcal{M}$ computes approx. the exp. value of q_j over D . So, $a_j \in [-1, +1]$.}

define $q_k(d) = d(k) \cdot \text{sign}(\sum_{i \in [k]} x(i) \cdot \ln \frac{1+a_i}{1-a_i})$ where $x \in \{-1, +1\}^{k+1}$.

 {In the line above, $\text{sign}(y) = \begin{cases} +1 & \text{if } y \geq 0 \\ -1 & \text{otherwise} \end{cases}$.}

let $a_{k+1} = \mathcal{M}(q_{k+1})$

 {In the line above, $\mathcal{M}$ computes approx. the exp. value of q_{k+1} over X . So, $a_{k+1} \in [-1, +1]$.}

return a_{k+1} .

Ensure: $a_{k+1} \in [-1, +1]$

low, which is evaluated in the first line in Table 7.1.

$$\begin{aligned}
 & [a \leftarrow []]^1; \\
 & [j \leftarrow k]^2; \\
 & \text{while } [j > 0]^3 \text{ do} \\
 \text{twoRounds}(k) \triangleq & \left([x \leftarrow \text{query}(\chi[k-j] \cdot \chi[k])]^4; \right. \\
 & [j \leftarrow j-1]^5; \\
 & [a \leftarrow x :: a]^6); \\
 & \left. [l \leftarrow (\text{sign}(\sum_{i \in [k]} \chi[i] \times \ln \frac{1+a[i]}{1-a[i]}))]^7 \right)
 \end{aligned}$$

The evaluation result in the 1st line in Table 7.1 shows that our **AdaptFun** works well for this complete implementation.

The Multiple Rounds Adaptive Data Analysis Algorithm, mRComplete

Example 7.2.2 (The Multiple Rounds Analyst Strategy). Below is the multiple rounds analyst strategy. This is instantiated from the Monitor, the Algorithm 2 in Rogers et al.

(2020). We instantiate this algorithm to trace the possible population shown in a database. We monitor the score of each person in the database. We also have the complete imple-

Algorithm 6 The multiple rounds analyst strategy

Require: Mechanism $\mathcal{M}$ with a hidden state $X \in [N]^n$ sampled u.a.r., control set size c
 Define control dataset $C = \{0, 1, \dots, c-1\}$
 Initialize $Nscore(i) = 0$ for $i \in [N]$, $I = \emptyset$ and $Cscore(C(i)) = 0$ for $i \in [c]$
for $j \in [k]$ **do**
 let $p = \text{uniform}(0, 1)$
 define $q(x) = \text{bernoulli}(p)$.
 define $qc(x) = \text{bernoulli}(p)$.
 let $a = \mathcal{M}(q)$
 for $i \in [N]$ **do**
 $Nscore(i) = Nscore(i) + (a - p) * (q(i) - p)$ **if** $i \notin I$
 for $i \in [c]$ **do**
 $Cscore(C(i)) = Cscore(C(i)) + (a - p) * (qc(i) - p)$
 let $I = \{i | i \in [N] \wedge Nscore(i) > \max(Cscore)\}$
 let $D = D \setminus I$
return D .

mentation of this strategy in `QWhile` below. The evaluation result is in the 18th line in Table 7.1, which shows that our *AdaptFun* works well for this complete implementation.

```

mRCom(k, c, N)  $\triangleq$ 
 $[j \leftarrow N]^0; [cs \leftarrow 0]^1; [ns \leftarrow 0]^2; [I \leftarrow 0]^3; [w \leftarrow k]^4;$ 
  while  $[j > 0]^5$  do
     $([j \leftarrow j - 1]^6; [cs \leftarrow 0 + cs]^7; [ns \leftarrow 0 + ns]^8);$ 
    while  $[w > 0]^9$  do
       $([w \leftarrow w - 1]^{10}; [p \leftarrow c]^{11}; [q \leftarrow c]^{12}; [a \leftarrow \text{query}(\chi[I])]^{13};$ 
       $[i \leftarrow N]^{14}; \text{while } [i > 0]^{15} \text{ do}$ 
         $([i \leftarrow i - 1]^{16}; [cs(i) \leftarrow cs(i) + (a - p) * (q - p)]^{17};$ 
         $\text{if } ([I < i]^{18}, [ns(i) \leftarrow ns(i) + (a - p) * (q - p)]^{19}, [ns \leftarrow ns(i)]^{20});$ 
       $[i2 \leftarrow N]^{21};$ 
      while  $[i2 > 0]^{22}$  do
         $([i2 \leftarrow i2 - 1]^{23}; \text{if } ([ns(i2) > \max(cs)]^{24}, [I \leftarrow i + I]^{25}, [I \leftarrow I]^{26}))$ 

```

Figure 7.4: The labeled program implementing the *multiple rounds analyst strategy*.

CHAPTER 8

Related Works

8.1 FORMALIZATION METHODS

Our adaptivity analysis framework constructs a semantics-based dependency graph based on the execution traces of a program. I define semantic dependence on this graph by considering (intraprocedural) data and control dependency, e.g. Bilardi & Pingali (1996); Cytron et al. (1991); Pollock & Soffa (1989); Burke & Ryder (1990). One related work in Austin & Sohi (1992) presents a methodology to construct a dynamic dependency graph (DDG) based on the dynamic execution of a program in an imperative language, where edges represent dependency between instructions. Data dependency, control dependency, storage dependency, and resource dependency between instructions are all considered. My semantics-based dependency graph only needs data dependency and control dependency between variable assignment results.

Dynamic dependency graphs have been used in many other domains. For example, Nagar & Jagannathan (2018) uses dynamic dependency graphs to find serializability violations. Hammer et al. (2006) proposes similar ways of constructing different kinds of program slices based on the algorithm presented in Ferrante et al. (1987), by choosing different program dependency graphs.

They actually use a combination of static and dynamic dependency graphs but in a manner that is different from how we use the two. Their slicing uses both static and dynamic dependency graphs, while we use the dynamic dependency graph as the basis of a definition, which is then soundly approximated by an analysis based on the static dependency graph.

My semantics-based data dependency relation definition over variables is inspired by the method in Cousot (2019a), where the dependency relation is also identified by looking into the differences between two execution traces. However, Cousot excludes timing channels Sabelfeld & Myers (2003) and empty observation, which are also not considered as a form of dependency in traditional dependency analysis such as Denning & Denning (1977). My definition includes timing channels and empty observation by observing both the disappearance and value variation.

8.2 STATIC PROGRAM ANALYSIS

My design of AdaptFun in this section is influenced by many areas of static program analysis such as effect systems, control-flow analysis, and data-flow analysis e.g. Burke & Ryder (1990). The idea of statically estimating a sound upper bound for the adaptivity from the semantics is indirectly inspired by prior work on cost analysis via effect systems in Çiçek et al. (2017); Radicek et al. (2018); Qu et al. (2019). The idea of defining adaptivity using data flow is inspired by the work of grading Hoare logic Gaboardi et al. (2021), which reasons about the data flows as a resource. One of the most important ingredients of my work is the estimation of the program-based dependency graph. There are many ways to construct a dependency graph statically. Some of the most related work focuses on the testing of graphical user interfaces (GUIs) Belli (2001); Arlt et al. (2012); Memon (2007), using an event graph. For example, Memon (2007) proposes an event-flow model using an algorithm to construct an event-flow graph, representing all the possible event interactions. This event-flow graph has a vertex for every GUI event such as click-to-paste and an edge between pairs of events that can be performed immedi-

ately after the other. My program-based dependency graph uses the edge to track the may-dependence of one variable with respect to another variable. The main difference is in the way the graph is constructed. **AdaptFun** relies on the structure of the target program, while the event-flow model only considers the event type. Another work in Arlt et al. (2012) constructs a weighted event-dependency graph, capturing data dependencies between events by analyzing bytecode. Every weighted edge indicates a dependency between two events, meaning one event possibly reads data written by the other event, with the weight showing the intensity of the dependency (the quantity of data involved). My approach to generating the program-based dependency graph shares the idea of tracking data dependency via static analysis on the smycc code. However, because of the different domains, we care about assigned variables, and we use the weight differently to find a finite walk in the graph.

Moreover, the state-of-art data-flow analysis techniques do not consider the quantitative information on how many times each variable is dependent on the other. My weight estimation is inspired by works in program complexity analysis and worst-case execution time analysis areas, focusing on analyzing the cost of the entire program. The techniques are based on type system (e.g. Çiçek et al. (2017); Rajani et al. (2021)), Hoare logic (e.g. Carbonneaux et al. (2015)), abstract interpretation (e.g. Gustafsson et al. (2005); Humenberger et al. (2018)), invariant generation through cost equations or ranking functions such as Brockschmidt et al. (2016); Albert et al. (2008); Alias et al. (2010); Flores-Montoya & Hähnle (2014) or a combination of program abstraction and invariant inferring e.g. Gulwani & Zuleger (2010); Sinn et al. (2017); Gulwani et al. (2009a). In general, these techniques give the approximated upper bound of the program's total running time or resource

cost. However, they failed to consider the case where the cost – the adaptivity – could decrease when there isn’t a dependency relation between variables.

8.3 GENERALIZATION IN ADAPTIVE DATA ANALYSIS

Starting from the works by Dwork et al. (2015a) and Hardt & Ullman (2014), several works have designed methods that ensure generalization for adaptive data analyses. Some examples are: Dwork et al. (2015b,a); Bassily et al. (2016); Ullman et al. (2018); Feldman & Steinke (2017); Jung et al. (2021); Steinke & Zakyntinou (2020); Rogers et al. (2020). Several of these works drew inspiration from the idea of using methods designed to ensure differential privacy, a notion of formal data privacy, in order to guarantee generalization for adaptive data analyses. By limiting the influence that an individual can have on the result of a data analysis, even in adaptive settings, differential privacy can also be used to limit the influence that a specific data sample can have on the statistical validity of a data analysis. This connection is actually in two directions, as discussed for example by Yeom et al. (2018).

Considering this connection between generalization and privacy, it is not surprising that some works on programming language techniques for privacy-preserving data analysis are related to our work. Adaptive Fuzz by Winograd-Cort et al. (2017) is a programming framework for differential privacy that is designed around the concept of adaptivity. This framework is based on a typed functional language that distinguishes between several forms of adaptive and non-adaptive composition theorem with the goal of achieving better upper bounds on the privacy cost. Adaptive Fuzz uses a type system and some partial evaluation to guarantee that the programs respect differential privacy. However, it does not include any tech-

nique to estimate the upper bound on the number of rounds of adaptivity. Vesga et al. (2021) propose a language for differential privacy where one can reason about the accuracy of programs in terms of confidence intervals on the error that the use of differential privacy can generate. These are akin to bounds on the generalization error. This language is based on a static analysis which however cannot handle adaptivity. The way we formalize the access to the data mediated by a mechanism is a reminiscence of how the interaction with an oracle is modeled in the verification of security properties. As an example, the recent works by Barbosa et al. (2021) and Aguirre et al. (2021) use different techniques to track the number of accesses to an oracle. However, reasoning about the number of accesses is easier than estimating the adaptivity of these calls, as we do instead here.

PART II REACHABILITY BOUND ANALYSIS

As a continuation, we look at the program’s *reachability-bound* as the second quantitative reachability property in order to get a precise bound when estimating the *adaptivity*. It is the upper bound on this execution number for a given control location introduced by Gulwani & Zuleger (2010) in the first place. A tight *reachability-bound* can help to improve the analysis result on other program features. For example, in the resource cost analysis, this quantitative reachability property can help to provide a tighter bound on the resources consumed by a program such as time, memory, network traffic, power, etc. In the data analysis area, the *reachability-bound* on each program control location can help to improve the precision of the data training result. In security, this *reachability-bound* helps to improve the accuracy in estimating the program’s information leakage or uncertainty propagation, etc. However, computing a tight *reachability-bound* for a given program control location is challenging. Many works in the program analysis area develop algorithms estimating the program’s loop bound or overall complexity. But there isn’t a general analysis algorithm that computes the *reachability-bound* directly or path-sensitively. To leverage these limitations, we present a path-sensitive *reachability-bound* algorithm aiming to find a precise symbolic worst-case *reachability-bound* on the program’s every control location in this paper. Our algorithm brings together two techniques: 1. ranking function based amortized bound analysis through program abstraction, and 2. loop summarization-based multiple loop paths refinement and combines them effectively. The effective combination improves the precision of the estimated *reachability-bound* compared to adopting either one of them. It allows us

to compute precise *reachability-bound* on each program point path-sensitively. We implement a prototype and report on an experimental comparison with state-of-art tools over four different benchmarks.

CHAPTER 9

Introduction

9.1 BACKGROUND

Gulwani & Zuleger (2010) first introduced *reachability-bound analysis* as a way to provide upper bounds on the number of times a given program location is visited during program execution. Obtaining tight reachability-bounds is useful in many applications. For example from a privacy and security perspective, how much secret information is leaked by a program may depend on the number of times a certain operation that leaks the data is executed Malacaria (2007). From an efficiency perspective, when different program locations consume different resources, a precise reachability-bound for each location can help to estimate the resource cost more accurately than just computing the overall complexity. Gulwani & Zuleger (2010) give a two-step solution combining program abstraction with a set of proof rules that are used to compute the actual reachability-bounds. This combination is effective at computing reachability-bounds for a rich class of programs, however, it also has limitations. This method computes complexity bounds for loops and over-approximate the reachability-bounds of the different locations inside the loop by using this bound. Then, it results in an over-approximation of the reachability-bounds of several locations, especially when there are loops with multiple paths. A similar approach to reachability-bounds can be also computed, in principle, using techniques from *amortized program complexity analysis* such as Bradley et al. (2005); Cook et al. (2013); Zuleger (2018); Sinn et al. (2014, 2017); Lu et al. (2021); Alias et al. (2010). Indeed, using techniques like ranking functions, counter increments, difference constraints, etc. one can estimate precise bounds on loop iterations and ex-

ecution time, and in turn obtain reachability bounds for complex programs. However, these techniques are designed to capture the worst-case or overall complexity, and again they may over-approximate the reachability-bounds of certain locations. This is again particularly evident when a loop contains multiple paths, since in this situation, one usually obtains an overall complexity bound that is greater than the reachability-bounds on some locations of some paths.

To overcome these limitations, we introduce a new path-sensitive reachability-bound analysis that combines ideas from amortized complexity analysis with ideas from *path refinement complexity analysis* including Gustafsson et al. (2005); Manolios & Vroon (2006); Balakrishnan et al. (2009); Sharma et al. (2011); Flores-Montoya & Hähnle (2014); Humenberger et al. (2018); Cyphert et al. (2019); Gulwani et al. (2009a); Zuleger et al. (2011). Specifically, we propose a new algorithm that by combining amortized analysis with loop summarization-based multipath refinement mitigates the limitations faced by each technique individually. More in detail, our algorithm first uses the difference constraints program abstraction model in Sinn et al. (2017, 2014) enriched with boolean expressions to generate an abstract transition graph. Then the algorithm performs both path refinement and amortized complexity analysis through ranking estimation over this graph. By computing the ranking function, we effectively alternate computation for the loop bound with the bound invariant of the ranking function. We also leverage the path-insensitivity of ranking function estimation through a lightweight path refinement algorithm adopted from Gulwani et al. (2009a). Building on this combination, we then focus on two useful quantities which we call *path reachability-bound* and *loop reachability-bound*. The first one bounds the evaluation times of each loop-free and interleave-free path in a refined program, and the second one bounds the number

of iterations of an outer loop w.r.t. an inner loop, counting only the iterations when the inner loop is “reached”. We present algorithms for estimating these two quantities. By using them we can then compute the *reachability-bound* for each program point in a path-sensitive way. We have implemented our algorithm in a prototype and evaluated it on 290 programs extracted from different benchmarks. Our evaluation shows that we can accurately estimate different bounds for different locations in programs with loops containing multiple paths. Based on this, we can also compute a tighter overall complexity compared to the state-of-art worst-case complexity analysis tools.

9.2 MOTIVATION

In this section, we discuss two representative examples illustrating the challenges of analyzing the precise *reachability-bound* on every location.

9.2.1 Multiple-Path Loop

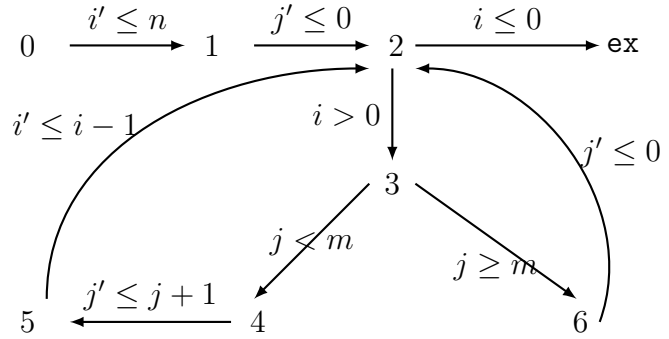
Figure 9.1(a) shows an example of a two paths loops with different *reachability-bounds* on the locations in different paths. This example is adopted from the example in Gulwani & Zuleger (2010), which is a skeleton code from the .Net base-class library.

In this example, given $n \geq m$, the expected *reachability-bounds* for locations 4 and 5 are both $m \times \lfloor \frac{n}{m} \rfloor$, for location 2 and 3 are $(m+1) \times \lfloor \frac{n}{m} \rfloor + 1$, and 1 for locations 0, 1 and ex. Notice here, though within the same loop L_2 , the bounds for locations 4 and 5 on the first branch, and 6 on the second branch are different. State-of-art

¹We use the notation $(l_0 \rightarrow \dots \rightarrow l_n)$ to denote a path with vertices sequence $(l_0, \dots, l_n)$, and the constraint on each edge is omitted for concise.

$\text{twoPathsWhile}(n, m) \triangleq$
 $[i \leftarrow n]^0; [j \leftarrow 0]^1;$
 $L_2 : \text{while } [i > 0]^2 \text{ do}$
 $\quad (\text{if } [j < m]^3,$
 $\quad \text{then } [j \leftarrow j + 1]^4;$
 $\quad \quad [i \leftarrow i - 1]^5,$
 $\quad \text{else } [j \leftarrow 0]^6);)$

(a)



(b)

$\text{tp}_0 = 0 \rightarrow 1 \rightarrow 2, \text{tp}_2 = 2 \rightarrow 3 \rightarrow 6 \rightarrow 2,$
 $\text{tp}_1 = 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 2, \text{tp}_3 = 2 \rightarrow \text{ex}$

(c)

$\text{tp}_0; \text{choose}\{2 : \text{tp}_1, 2 : \text{tp}_2\}; \text{tp}_3.$

(d)

$\text{tp}_0; \text{choose}\{2 : \text{repeat}(\text{repeat}(\text{tp}_1); \text{tp}_2), 2 : \text{repeat}(\text{tp}_1)\}; \text{tp}_3.$

(e)

Figure 9.1: (a) The two paths loop example. (b) The abstract transition graph for $\text{twoPathsWhile}(n, m)$. (c) The simple transition paths. (d) The rewrote program. (e) The refined program.

reachability-bound analysis Gulwani & Zuleger (2010) gives the same *reachability-bound*, $n + \lfloor \frac{n}{m} \rfloor$ for all the locations within the loop L_2 , which is tight w.r.t. L_2 's iteration times but not for different locations inside L_2 without considering multiple paths. Among works on program complexity, cost, and loop bound analysis, the algorithm in Gulwani et al. (2009a) computes a tight bound on the loop iteration times but not precise *reachability-bounds* on different locations. Though we can use this loop iteration bound as the *reachability-bounds* for locations 1 and 2, the bounds for locations 4, 5 and 6 are still unclear.

Path reachability-bound To compute the bounds for locations on different paths of a loop, we compute a new quantity – *path reachability-bound*. This quantity bounds the evaluation times of each *simple transition path* during the program execution, where a *simple transition path* is loop-free and interleaving-free.

For this example, we first generate the abstract transition graph for the program as in Figure 9.1(b), using the difference constraint-based program abstraction method. Then we refine the loop in this program by checking the execution orders of each simple transition path and generates a refined program rp as below,

$$tp_0; \text{choose}\{\text{repeat}(\text{repeat}(tp_1); tp_2), \text{repeat}(tp_1)\}; tp_3.$$

In this refined program, each tp_i is a *simple transition path* we computed in a pre-procedure, which is loop-free and not interleaving with the other paths, $tp_j, j \neq i$ as in Figure 9.1(c). $\text{repeat}(tp_1)$ represents that execution of this program contains iteration on the path tp_1 . $\text{repeat}(tp_1); tp_2$ denotes that after the iterations of tp_1 is done, tp_2 will execute right after. $\text{choose}\{\text{repeat}(\text{repeat}(tp_1); tp_2), \text{repeat}(tp_1)\}$ means that there are two possible execution orders for the original loop L_2 . The ex-

ecution orders of these *simple transition paths* are explicated in the refined program. Then we compute the *path reachability-bound* for every tp_i , $pathRB(rp, tp_i)$ during the execution of rp through the ranking function estimation in Section 12.4 and the algorithm in Section 12.6 to compute the result as $pathRB(rp, tp_1) = \max\{m, m \times \frac{n}{m}\}$, $pathRB(rp, tp_2) = \frac{n}{m}$, and $pathRB(rp, tp_0) = pathRB(rp, tp_3) = 1$. Since there isn't nested loop in this example, we simply sum up $pathRB(rp, tp)$ over the tp where a certain location shows up and as the *reachability-bound* of this location. Then we get the precise *reachability-bound* for every location in program `twoPathsWhile` as $psRB(0) = psRB(1) = psRB(ex) = 1$, $psRB(4) = psRB(5) = \max\{m, m \times \lfloor \frac{n}{m} \rfloor\}$, $psRB(3) = psRB(2) = \max\{m, m \times \lfloor \frac{n}{m} \rfloor\} + \lfloor \frac{n}{m} \rfloor + 1$, and $psRB(6) = \lfloor \frac{n}{m} \rfloor$.

9.2.2 Nested Loops with Related Iterator

However, when there exists a nested loop, computing the *reachability-bound* for each location encounters another challenge. Figure 9.2(a) shows an example of the nested loops with related iterators. This example is adopted from the example in Gulwani et al. (2009a), which is common in product code.

In line 8, i is reset by w , and w is reset by j in line 5. So the while L_6 is only executed in the first iteration of while loop L_1 and L_3 . The while loop L_3 at line 3 is executed only in the first $m - N$ iterations of the L_1 because j is reset by i in line 2. So the total iterations of all three loops are $n + m^2 - m \times N$, and the precise *reachability-bound* for location 7 inside the L_6 is N , for locations 4, 5 and 8 between the L_3 and L_6 are $(n - N) \times (m - N)$, and $n - N$ for locations 2 and 9. Notice here, the *reachability-bounds* for the locations inside the loop L_6 is the same as its innermost loop iteration bound. However, for the locations between L_3 and L_6 , the *reachability-bounds* are the multiplication of the inner and outer loop iteration

bounds. To the best of our knowledge, the loop bound analysis method in Gulwani et al. (2009a) can only give a loose upper bound $n + (m \times n) + N$ for the entire loop complexity, and the difference constraint-based algorithm in Sinn et al. (2017) is able to compute a better but still loose bound, $n + m^2 - m \times N$ on total iteration times. None of them can give the precise *reachability-bound* for every location in these nested loops, which is non-trivial to compute even though knowing the loop bound, especially for the locations similar to 7 in `threeNestedWhile`.

In order to precisely compute how many times the location 7 is reached, the local *path reachability-bound* we discussed in the first example is not precise enough. We need to know the numbers of iterations of the outer loop L_3 and L_1 such that, during these iterations, the inner loop L_6 is “reached”. We call the upper bound on this number, the *loop reachability-bound* of the location within loop L_6 w.r.t the outer loops L_3 and L_1 . Then by multiplying the loop iteration bound of the L_6 with its *loop reachability-bound* w.r.t the L_3 and L_1 , we can compute the precise *reachability-bounds* for location 7.

The method in Gulwani et al. (2009a) based on the *progress invariant* computes a similar quantity, but it only computes the bound on iteration numbers of the inner loop L_6 in each iteration of L_3 and L_1 , which are both N . So they have to over-approximate the reachability-bound for locations inside L_6 with the overall program complexity by multiplication, i.e., $n + m^2 - m \times N$. The difference constraint-based algorithm in Sinn et al. (2017) through ranking function estimation is only able to compute the combined loop bound and the local bound of each loop separately as well.

Loop reachability-bound. In this sense, we introduce the *loop reachability-bound*. For each *simple transition path* w.r.t each of its outer loop, *loop reachability-bound*

is a bound on the iterations of the outer loop such that during these outer loop iterations, the inner loop is “reached”. This is distinguished from the traditional methods, which only estimate the bound on the inner loop’s iteration number in one iteration of the outside loop.

Similar to the `twoPathsWhile` example, we generate its abstract transition graph as in Figure 9.2(a), and compute the refined program, $rp = tp_0; 1 : \text{repeat}(tp_1; 3 : \text{repeat}(tp_2; 6 : \text{repeat}(tp_3); tp_4); tp_5); tp_6$, where the $tp_0, \dots$ are shown in the middle part of Figure 9.2(b). We use rp_1 and rp_3 denote the body of the loop L_1 and L_3 respectively as in the bottom part of Figure 9.2(b). In the first step, we still compute the *path reachability-bound* locally for each tp_i but only w.r.t. the innermost loop in which tp_i is nested. Then differently from `twoPathsWhile`, we compute *loop reachability-bound*, $lpRB(rp_j, tp_i)$ for each tp_i w.r.t. each of its outer loop by the algorithm in Section 12.7. For tp_3 we compute $lpRB(1 : rp_1, tp_3) = 1$ and $lpRB(3 : rp_3, tp_3) = 1$. Both are tight because loop L_6 will only be entered once among all iterations of L_1 and L_3 , and in all the rest iterations, the body of loop L_6 isn’t executed at all. Then for each tp_i , the multiplication of its local *path reachability-bound* with all its *loop reachability-bounds* is the accurate *path reachability-bound* globally. By summing up the reachability-bound of the path where each location shows up, we compute N as the *reachability-bound* of location 7, which is tight.

9.3 TECHNIQUE OVERVIEW

We give the technique overview of our algorithm through the running example in Figure 9.3.1.

Example 9.3.1 (The running example with two paths and a nested loop in one

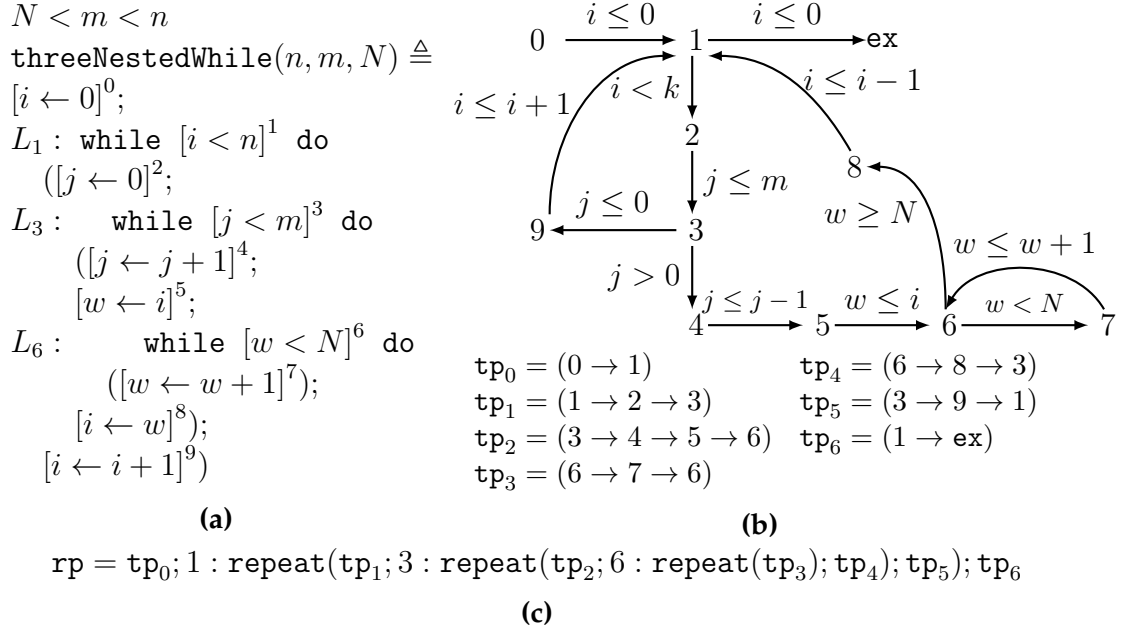


Figure 9.2: (a) An example of three nested loops with related iterator variables. (b) The abstract transition graph, the simple transition paths and loop body. (c) The refined program.

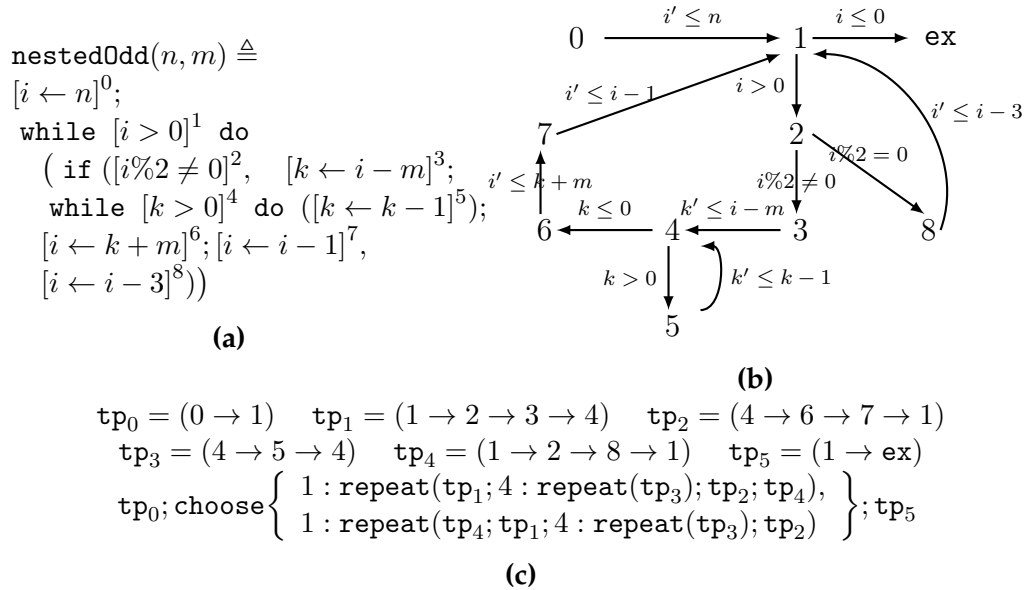


Figure 9.3: (a) The program of the two paths loop with a nested Loop in one path. (b) The abstract transition graph of $\text{absG}(\text{nestedOdd}(n, m))$. (c) The refined program.

path).

The running example program in Figure 9.3.1(a) has two level nested loops, outer loop L_1 and inner loop L_4 where L_4 is in the first path of L_1 . It has both challenges in the two previous examples. The first challenge comes with the if command in the outer loop. The bounds for different locations in the same loop L_1 are different. Given $n \geq m \geq 0$, the overall loop iteration times of this outer loop are expected to be $\lfloor \frac{m}{2} \rfloor$. However, the expected *reachability-bounds* for locations 3, 6, 7 and 8 in L_1 are all $\lfloor \frac{m}{4} \rfloor$, for location 2 is $\lfloor \frac{m}{2} \rfloor$ and $\lfloor \frac{m}{2} \rfloor + 1$ for location 1. These *reachability-bounds* for different locations in a loop are different from the bound on loop iterations. Though within the same loop L_1 , the bounds for different locations are different. Amortized analysis tools such as Loopus Sinn et al. (2017), KoAT Brockschmidt et al. (2014); Falke et al. (2012, 2011), C4B Carboneaux et al. (2015), etc. ignore path sensitivity and approximate the bound n as the bound for loop L_1 . While some path-refinement-based methods such as Gulwani & Zuleger (2010); Gulwani et al. (2009a), CoFloCo Flores-Montoya (2017, 2016); Flores-Montoya & Hähnle (2014) etc. give the same bound for all the locations within the loop L_1 . Although we can reuse their results i.e., loop bounds as the *reachability-bound* for location 1 and 2, it is still unclear for locations 3, 6, 7, and 8. This motivates our first key novelty – the *path reachability-bound* for a loop-free and interleaving-free path tp . It bounds the times that each path is evaluated in a path-refined program.

The second challenge occurs in the nested loop. In line 6, i is reset by $k + m$, and k is reset by $i - m$ at line 3. So the loop L_4 is only executed in the first iteration of while loop L_1 . We say the nested loop, L_4 is only “reached” in 1 iteration of the outer loop, and the precise *reachability-bound* for location 5 inside the L_4 is m . This

motivates us to consider our second novel quantity – the numbers of iterations of the outer loop L_1 such that, during these iterations, the inner loop L_4 is “reached”. We refer to this quantity as the *loop reachability* of the inner loop L_4 w.r.t the outer loop L_1 . By multiplying the *path reachability-bound* of tp_3 within L_4 with its *loop reachability-bound* w.r.t L_1 , we can obtain an accurate *reachability-bound* for location 5.

Our algorithm computes the *reachability-bound* for locations in this example through the following steps.

Step 1 in Section 12.2 first generates the *Abstract Transition Graph* as in Figure 9.3(b). Each edge $l \xrightarrow{dc} l'$ is an abstract transition $\hat{e} = (l, dc, l')$ annotated with a constraint dc corresponding to the command of label l .

Step 2 in Section 12.3 computes the *Refined Program*, rp for a program c based on its abstraction transition graph and transforms the multiple-paths loops into multiple loops where the interleaving of paths is explicit as in the bottom of Figure 9.3(c). It has two interleaving patterns in the loop L_1 .

Step 3 computes the *Ranking Function*, $\text{rank}(\hat{e}, c)$ for every edge $\hat{e}$ and estimates an upper bound invariant, which is presented in Section 12.4.

Step 4 computes the *Reachability-bound*, $\text{psRB}(l, c)$ for every program point l using the rp and the upper bound invariant of the $\text{rank}(\hat{e}, c)$ where $\hat{e} = (l, dc, l')$. It requires computing the *path reachability-bound*, $\text{pathRB}(\text{rp}, \text{tp}, c)$ for every tp . If there are nested loops, we need to compute the *loop reachability-bound*, $\text{lpRB}(l : \text{rp}, \text{tp}, c)$ w.r.t. every level of tp 's outer loop. Section 12.5 introduces this algorithm and the following sections describe the computations. The soundness and proof of each algorithm are given in the Appendices and the input c will be omitted in the rest of the paper if the context is clear.

In Figure 9.3, for tp_3 in the first interleaving pattern, $rp_1^1 = 1 : \text{repeat}(tp_1; 4 : \text{repeat}(tp_3); tp_2; tp_4)$, we first compute a local *path reachability-bound*, $\text{pathLocRB}(4 : \text{repeat}(tp_3), tp_3, c) = n - m$ w.r.t. its innermost loop L_4 by Section 12.6.

Since tp_3 is nested in two level loops, we need to compute its *loop reachability-bound* w.r.t. the outer loop L_1 . For the outer loop L_1 , the inner loop L_4 is reached only in the first iteration of the outer loop. By computing the initial state, next state of visiting L_4 during the execution of loop L_1 , the algorithm in Section 12.7 computes a tight *loop reachability-bound*, $\text{lpRB}(rp_1^1, tp_3, c) = 1$. We also compute in the second interleaving pattern rp_1^2 the same number and then Section 12.8 computes the *path reachability-bound* globally for tp_3 by taking the maximal value over all interleaving patterns, $\text{pathRB}(rp, tp_3, c) = \max\{1 \times (n - m), 1 \times (n - m - 3)\} = n - m$.

Now for every program point l , we sum up the $\text{pathRB}(rp, tp)$ over all tp that contains l and get $\text{psRB}(l, c)$. Since point 5 only shows up on tp_3 , we compute $\text{psRB}(5, c) = n - m$. The points 0 and ex are not in any loop, so we have $\text{psRB}(0, c) = \text{psRB}(\text{ex}, c) = 1$. The points 3, 6, 7 and 8 which only show up once on tp_2 and tp_4 are all equal to $\lfloor \frac{m}{4} \rfloor$ the same as their pathRB . For the loop headers 1 and 4, we only count the tp where they show up as a start-point. So $\text{psRB}(4, c) = \lfloor \frac{m}{4} \rfloor + n - m + 1$ and $\text{psRB}(1, c) = 2 \times \lfloor \frac{m}{4} \rfloor + 1$ all as expected.

9.4 OUTLINE AND CONTRIBUTIONS

Chapter Structure

- In Chapter 10, we first give the language and program model formally similar to our language mode in **PART I**.
- Chapter 11 gives the *reachability-bound* definition.

- Chapter 12 presents the new *path-sensitive reachability-bound* analysis algorithm, named psRB through the following sections. Section 12.1 first gives the main steps of this algorithm as a structural section. There are five steps in this algorithm. In Section 12.2, the first step abstracted the program into an *abstract transition graph*. Based on this graph, Section 12.3 then refines the multi-paths loop in this program and generates a refined program. According to the graph and the refined program, Section 12.4 computes the ranking function for each edge on the graph and estimates the value of each ranking function. Section 12.5 gives the *path-sensitive reachability-bound* algorithm and the rest of the sections give the computation algorithm for each step. Specifically, Section 12.6 gives the local *path reachability-bound* estimation algorithm, Section 12.7 presents the *loop reachability-bound* estimation algorithm, and Section 12.8 gives the global *path reachability-bound* estimation algorithm.
- Chapter 13 presents four step-by-step examples to illustrate the algorithm clearly in Section 13.1 firstly, and then shows the implementation and evaluation results over five real world benchmarks in Section 13.
- Chapter 14 discusses the related works in this area.

Contributions The contributions of this work can be summarized as follows,

- We introduce a path-sensitive reachability-bound algorithm combining *amortized bound analysis* and *path refinement*. The algorithm computes tight bounds on the times that each program point is evaluated.
- We identify two quantities, *path reachability-bound* and *loop reachability-bound*,

which are useful to measure precise information about the program execution. We also propose two algorithms for soundly estimating these two quantities.

- We develop a prototype implementation with evaluation over 290 programs. The evaluation shows improvement in precision compared to other tools for reachability-bound analysis and the worst-case complexity analysis.

CHAPTER 10

Language Model

10.1 LABELED LANGUAGE

Arithmetic Operators	$\oplus_a ::= + \mid - \mid \times \mid \div$
Arithmetic Expression	$a ::= n \mid x \mid a \oplus_a a \mid \max(a, a) \mid \min(a, a)$
Boolean Expression	$b ::= \text{true} \mid \text{false} \mid \neg b \mid b \wedge b \mid b \vee b$ $\mid a \leq a \mid a < a \mid a = a$
Expression	$e ::= v \mid a \mid b \mid [e, \dots, e]$
Value	$v ::= n \in \mathbb{N} \cup \{\infty\} \mid \text{true} \mid \text{false} \mid [] \mid [v, \dots, v]$
Label	$l \in (\mathbb{N} \cup \{\text{in}, \text{ex}\})$
Labeled Command	$c ::= [x \leftarrow e]^l \mid [\text{skip}]^l$ $\mid \text{while } [b]^l \text{ do } (c) \mid \text{if } ([b]^l, c, c) \mid c; c$
Event	$\epsilon ::= (x, l, v) \text{ Assignment Event} \mid (b, l, v) \text{ Testing Event}$
Trace	$\tau ::= [] \mid \tau :: \epsilon$

Likewise, we denote by ∞ a value s.t. $n < \infty$ for all $n \in \mathbb{N}$, and we use the

following notations to represent the sets of corresponding terms:

$\mathcal{V}$	: Set of Variables
$\mathcal{B}$	: Set of Boolean Expressions
$\mathcal{C}$	: Set of All Labeled Commands
$\mathcal{E}$	: Set of Events
$\mathcal{E}^{\text{asn}}$	: Set of Assignment Events
$\mathcal{E}^{\text{test}}$	: Set of Testing Events
$\mathcal{L}$	: Set of Labels
$\mathcal{T}^+$	: Set of All Finite Execution Traces
$\mathcal{T}^\infty$	: Set of Infinite Execution Traces
$\mathcal{T}^{+\infty}$	: Set of All Finite Or Infinite Execution Traces
$\mathcal{V}_{\text{in}}(c)$	: Set of Program c 's Input Variables
$\mathcal{T}_0^+(c)$	: Set of Program c 's Well-formed Initial Traces.

Each well formed initial trace $\tau_0 \in \mathcal{T}_0^+(c)$ is finite

and every input variable of the program c has an initial value in τ_0 .

Expression Expressions include standard arithmetic expression, $a \in \mathcal{A}$ (with the value $n \in \mathbb{N} \cup \{\infty\}$ and the variable x from a finite universe $\mathcal{V}$) and boolean expression, $b \in \mathcal{B}$. We denote by ∞ a value s.t. $n < \infty$ for all $n \in \mathbb{N}$.

An arithmetic expression can be a constant n denoting integer, a variable x from some countable set $\mathcal{V}$, arithmetic binary operation $\oplus_a$ including addition, product, subtraction, division, over arithmetic expressions, or $\max$, $\min$ of two arithmetic expressions.

A boolean expression can be either `true` or `false`, basic boolean connectives including logical negation, $\neg b$, logical and, $b \wedge b$ and logic or $b \vee b$ and basic com-

parison between arithmetic expressions, including $a \leq a$, $a < a$ and $a = a$. Additionally, we also have list in expression.

Labeled Command The commands are the typical ones from while languages. Each command is annotated with a label $l \in (\mathbb{N} \cup \{\text{in}, \text{ex}\})$. The program is a labeled command c having the following syntax.

$$c ::= [x \leftarrow e]^l \mid [\text{skip}]^l \mid \text{while } [b]^l \text{ do } (c) \mid \text{if } ([b]^l, c, c) \mid c; c$$

We use label to record the location of each command, so that we can uniquely identify each program point. We denote by $\mathcal{C}$ the universe of all labeled commands and $\mathcal{L}$ for all labels.

Free Variables and Input Variables We denote by FV the operator $\text{FV} : e \rightarrow \mathcal{P}(\mathcal{V})$, which computes the set of free variables in an expression. For example, $\text{FV}(a)$ and $\text{FV}(b)$ represent the set of free variables in arithmetic expression a and boolean expression b respectively. The union of all the free variables showing up in c that are never defined before using, are actually the input variables of this program. We use the same notation as **PART i**, where $\mathcal{V}_{\text{in}}$ denotes the universe of all input variables and $V_{\text{in}}(c)$ the set of input variables in a given program c .

10.2 TRACE-BASED OPERATIONAL SEMANTICS

Event An event is a triple. Its first element is the variable name x or a boolean expression (from the guard of if or while command), following by the label, l associated to this command and the value assigned to the variable.

We have two kinds of events: *assignment events* and *testing events*, and we use

$\mathcal{E}^{\text{asn}}$ and $\mathcal{E}^{\text{test}}$ to denote the set of all assignment events and testing events, respectively.

An *assignment event* tracks the execution of an assignment and consists of the assigned variable, the label of the command that generates it, the value assigned to the variable.

A *testing event* tracks the execution of if and while commands, specifically the evaluation of the boolean expression b in the guard of a `if` $([b]^l, c_1, c_2)$ command or `while` $[b]^l$ do (c) . It consists of the boolean expression b in the guard of the command, the label of the guard, the result of evaluating the guard.

$$\text{Event } \epsilon ::= (x, l, v) \text{ Assignment Event} \mid (b, l, v) \text{ Testing Event}$$

Event projection operators π_i projects the i^{th} element from an event: $\pi_i : \mathcal{E} \rightarrow \mathcal{V} \cup \mathcal{B} \cup \mathcal{L}$

Trace. A trace $\tau \in \mathcal{T}^{+\infty}$ is a list of events, collecting the events generated during a specific program execution.

$$\text{Trace } \tau ::= [] \mid \tau :: \epsilon$$

A trace can be regarded as the program history, which records all the evaluations for assignment commands and guards in `if` and `while` command.

A trace can be finite ($\tau \in \mathcal{T}^+$) or infinite $\tau \in \mathcal{T}^\infty$. If a program doesn't terminate when executing under some initial trace, it produces the infinite trace from $\mathcal{T}^\infty$, which records a non-terminating computation. So we denote by $\mathcal{T}^{+\infty}$ the set of all traces, and $\mathcal{T}^{+\infty} = \mathcal{T}^+ \cup \mathcal{T}^\infty$. The trace-based semantics with non-terminating ex-

ecution is defined below following the maximal trace semantics in Cousot (2019b).

We use list notation for traces, where $\square$ is the empty trace, the operator $::$ combines an event and a trace in a new event, and the operator $++$ concatenates two traces formally defined as follows.

Definition 42 (Trace Concatenation, $++ : \mathcal{T}^{+\infty} \rightarrow \mathcal{T}^{+\infty} \rightarrow \mathcal{T}^{+\infty}$). *Given two traces $\tau_1 \in \mathcal{T}^{+\infty}, \tau_2 \in \mathcal{T}^{+\infty}$, the trace concatenation operator $++$ is defined as:*

$$\tau_1 ++ \tau_2 \triangleq \begin{cases} \tau_1 & \tau_2 = \square \vee \tau_1 \in \mathcal{T}^\infty \\ \tau_2 & \tau_1 = \square \vee \tau_2 \in \mathcal{T}^\infty \\ (\tau_1 ++ \tau'_2) :: \epsilon & \tau_1 \in \mathcal{T}^+ \wedge \tau_2 = \tau'_2 :: \epsilon \end{cases}$$

Environment. $\rho : \mathcal{T}^{+\infty} \rightarrow \mathcal{V} \rightarrow (\mathbb{N} \cup \{\perp, \infty\})$. Similar to **PART i**, we don't use separate data structure as the environment to record each variable's value or state. Instead, we use the operator $\rho(\tau)x$ to fetch the latest value assigned to x in the trace τ .

$$\begin{aligned} \rho(\tau :: (x, l, v))x &\triangleq v & \rho(\tau :: (y, l, v))x &\triangleq \rho(\tau)x, y \neq x & \rho(\tau :: (b, l, v))x &\triangleq \rho(\tau)x \\ \rho(\square)x &\triangleq \perp & \rho(\tau)x &\triangleq \perp \text{ if } \tau \in \mathcal{T}^\infty \end{aligned}$$

Definition 43 (Well-formed Initial Traces). *Given a program c , τ is a well-formed initial trace of c if and only if all the input variables of c has an initial value in this trace.*

$$\forall c \in \mathcal{C}, \tau \in \mathcal{T}^+ . \tau \in \mathcal{T}_0^+(c) \iff \forall x \in \mathcal{V}_{\text{in}}(c) . \rho(\tau_0)x \neq \perp$$

Again, $\mathcal{T}_0^+(c)$ denotes the set of all well-formed initial traces for program c .

Expression Semantics The evaluation notation for arithmetic expression is $\llbracket a \rrbracket_a : \mathcal{A} \rightarrow \mathcal{T}^{+\infty} \rightarrow \mathcal{V}$. The $\llbracket a \rrbracket_a(\tau)$ evaluates an arithmetic expression a under trace τ

$\llbracket \cdot \rrbracket_a : \text{Arithmetic Expression} \rightarrow \text{Trace} \rightarrow \text{Value}$	
--	--

$$\begin{array}{c}
\frac{}{\llbracket n \rrbracket_a(\tau) = n} \qquad \frac{\rho(\tau)x = v}{\llbracket x \rrbracket_a(\tau) = v} \qquad \frac{\llbracket a_1 \rrbracket_a(\tau) = v_1 \quad \llbracket a_2 \rrbracket_a(\tau) = v_2 \quad v_1 \oplus_a v_2 = v}{\llbracket a_1 \oplus_a a_2 \rrbracket_a(\tau) = v} \\
\\
\frac{\llbracket a_1 \rrbracket_a(\tau) = v_1 \quad \llbracket a_2 \rrbracket_a(\tau) = v_2 \quad v_1 < v_2}{\llbracket \mathbf{max} (a_1, a_2) \rrbracket_a(\tau) = v_2} \\
\\
\frac{\llbracket a_1 \rrbracket_a(\tau) = v_1 \quad \llbracket a_2 \rrbracket_a(\tau) = v_2 \quad v_1 \geq v_2}{\llbracket \mathbf{max} (a_1, a_2) \rrbracket_a(\tau) = v_1} \\
\\
\frac{\llbracket a_1 \rrbracket_a(\tau) = v_1 \quad \llbracket a_2 \rrbracket_a(\tau) = v_2 \quad v_1 < v_2}{\llbracket \mathbf{min} (a_1, a_2) \rrbracket_a(\tau) = v_1} \\
\\
\frac{\llbracket a_1 \rrbracket_a(\tau) = v_1 \quad \llbracket a_2 \rrbracket_a(\tau) = v_2 \quad v_1 \geq v_2}{\llbracket \mathbf{min} (a_1, a_2) \rrbracket_a(\tau) = v_2}
\end{array}$$

Figure 10.1: Evaluation rules of arithmetic expression

following the arithmetic expression evaluation rules in Figure 10.1. We also define for ∞ the following mathematic computation with arbitrary $\tau \in \mathcal{T}^{+\infty}$.

$$\begin{array}{l}
\llbracket a + \infty \rrbracket_a(\tau) = \infty \quad \llbracket a - \infty \rrbracket_a(\tau) = 0 \quad \llbracket \infty - a \rrbracket_a(\tau) = \infty \quad \llbracket a \div \infty \rrbracket_a(\tau) = 0 \\
\llbracket \mathbf{min} (a, \infty) \rrbracket_a(\tau) = a \quad \llbracket \mathbf{max} (a, \infty) \rrbracket_a(\tau) = \infty
\end{array}$$

The evaluation rules for boolean expression, $\llbracket \cdot \rrbracket_b : \mathcal{B} \rightarrow \mathcal{T}^{+\infty} \rightarrow \mathcal{V}$ are in Figure 10.2. Figure 10.3 shows the standard expression evaluation, $\llbracket \cdot \rrbracket : (\mathcal{A} \cup \mathcal{B}) \rightarrow \mathcal{T}^{+\infty} \rightarrow \mathcal{V}$, which includes the evaluation rules of both the boolean and arithmetic expressions.

$\llbracket \cdot \rrbracket_b : \text{Boolean Expression} \times \text{Trace} \rightarrow \text{Boolean Value}$		
$\frac{}{\llbracket \text{false} \rrbracket_b(\tau) = \text{false}}$	$\frac{}{\llbracket \text{true} \rrbracket_b(\tau) = \text{true}}$	$\frac{\llbracket b \rrbracket_b(\tau) = v' \quad \neg v' = v}{\llbracket \neg b \rrbracket_b(\tau) = v}$
$\frac{\llbracket b_1 \rrbracket_b(\tau) = v_1 \quad \llbracket b_2 \rrbracket_b(\tau) = v_2 \quad v_1 \wedge v_2 = v}{\llbracket b \wedge b_2 \rrbracket_b(\tau) = v}$		
$\frac{\llbracket b_1 \rrbracket_b(\tau) = v_1 \quad \llbracket b_2 \rrbracket_b(\tau) = v_2 \quad v_1 \vee v_2 = v}{\llbracket b_1 \vee b_2 \rrbracket_b(\tau) = v}$		

Figure 10.2: Evaluation rules of boolean expression

$\llbracket \cdot \rrbracket : \text{Expression} \times \text{Trace} \rightarrow \text{Value}$			
$\frac{\llbracket a \rrbracket_a(\tau) = v}{\llbracket a \rrbracket(\tau) = v}$	$\frac{\llbracket b \rrbracket_b(\tau) = v}{\llbracket b \rrbracket_b(\tau) = v}$	$\frac{\llbracket e_1 \rrbracket(\tau) = v_1 \quad \dots \quad \llbracket e_n \rrbracket(\tau) = v_n}{\llbracket [e_1, \dots, e_n] \rrbracket(\tau) = [v_1, \dots, v_n]}$	$\frac{}{\llbracket v \rrbracket(\tau) = v}$

Figure 10.3: Evaluation rules of standard expression

$\text{Command} \times \text{Trace} \rightarrow \text{Command} \times \text{Trace}$	$\langle c, \tau \rangle \rightarrow \langle c', \tau' \rangle$
$\frac{\llbracket e \rrbracket(\tau) = v \quad \epsilon = (x, l, v)}{\langle [x \leftarrow e]^l, \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau :: \epsilon \rangle} \text{ assn}$	
$\frac{\llbracket b \rrbracket_b(\tau) = \text{true} \quad \epsilon = (b, l, \text{true})}{\langle \text{while } [b]^l \text{ do } (c), \tau \rangle \rightarrow \langle c; \text{while } [b]^l \text{ do } (c), \tau :: \epsilon \rangle} \text{ while-t}$	
$\frac{\llbracket b \rrbracket_b(\tau) = \text{false} \quad \epsilon = (b, l, \text{false})}{\langle \text{while } [b]^l \text{ do } (c), \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau :: \epsilon \rangle} \text{ while-f}$	$\frac{\langle c_1, \tau \rangle \rightarrow \langle c'_1, \tau' \rangle}{\langle c_1; c_2, \tau \rangle \rightarrow \langle c'_1; c_2, \tau' \rangle} \text{ seq1}$
$\frac{\langle c_2, \tau \rangle \rightarrow \langle c'_2, \tau' \rangle}{\langle [\text{skip}]^l; c_2, \tau \rangle \rightarrow \langle c'_2, \tau' \rangle} \text{ seq2}$	$\frac{\llbracket b \rrbracket_b(\tau) = \text{true} \quad \epsilon = (b, l, \text{true})}{\langle \text{if } ([b]^l, c_1, c_2), \tau \rangle \rightarrow \langle c_1, \tau :: \epsilon \rangle} \text{ if-t}$
$\frac{\llbracket b \rrbracket_b(\tau) = \text{false} \quad \epsilon = (b, l, \text{false})}{\langle \text{if } ([b]^l, c_1, c_2), \tau \rangle \rightarrow \langle c_2, \tau :: \epsilon \rangle} \text{ if-f}$	

Figure 10.4: Operational semantics rules

Operational Semantics Rules The trace based operational semantics is described in terms of a small step evaluation relation $\langle c, \tau \rangle \rightarrow \langle c', \tau' \rangle$ describing how a configuration program-trace evaluates to another configuration program-state. The rules are presented in Figure 10.4.

Given an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$ of the program c , we use $\rightarrow^*$ for the reflexive and transitive closure of $\rightarrow$. If $\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{0++}\tau \rangle$, then the program's execution terminates and produces a finite execution trace $\tau \in \mathcal{T}^+$.

Definition 44 (Non-terminating and Infinite Trace). *Given a program c and an initial trace $\tau \in \mathcal{T}_0^+(c)$, when c executes with τ , we define the execution of c under τ is non-terminating and produces an infinite trace $\tau' \in \mathcal{T}^\infty$, as $\langle c, \tau \rangle \uparrow^\infty \tau' \in \lim(\uparrow)$ where the*

limit is defined as follows.

$$\begin{aligned} \lim(\uparrow) \triangleq & \left\{ (\langle c, \tau \rangle, \tau') \mid c \in \mathcal{C}, \tau \in \mathcal{T}_0^+(c), \tau' \in \mathcal{T}^\infty \wedge \exists \tau_0 \in \mathcal{T}^+, c_0 \in \mathcal{C} . \langle c, \tau \rangle \rightarrow \langle c_0, \tau_0 \rangle \right. \\ & \wedge \forall i \in \mathbb{N}, \exists \tau_i, \tau_{i+1} \in \mathcal{T}^+, \tau'' \in \mathcal{T}^\infty, c_i, c_{i+1} \in \mathcal{C} . \\ & \left. \langle c_i, \tau_i \rangle \rightarrow \langle c_{i+1}, \tau_{i+1} \rangle \wedge \tau' = \tau_{i+1} ++ \tau'' \right\} \end{aligned}$$

This definition has the similar intuition as the maximal trace semantics in Equation (12) from Section 2.5 of paper Cousot (2019a), and the semantics of the program model in Sinn et al. (2017).

We also have the trace non-decreasing property in this language model as below. It is similar to the Lemma 4.2.1 with proof in Appendix B.1.

Lemma 10.2.1 (Trace Non-Decreasing). *For any program $c \in \mathcal{C}$ and initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, if there exists $\tau \in \mathcal{T}^{+\infty}$ and $c' \in \mathcal{C}$ such that $\langle c, \tau_0 \rangle \rightarrow^* \langle c', \tau \rangle$ or $\langle c, \tau_0 \rangle \uparrow^\infty \tau$ then there exists a trace $\tau' \in \mathcal{T}^{+\infty}$ such that $\tau_0 ++ \tau' = \tau$ formally as follows.*

$$\begin{aligned} & \forall \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^{+\infty}, c, c' \in \mathcal{C} . \left(\langle c, \tau_0 \rangle \rightarrow^* \langle c', \tau \rangle \vee \langle c, \tau_0 \rangle \uparrow^\infty \tau \right) \\ & \implies \exists \tau' \in \mathcal{T}^{+\infty} . \tau_0 ++ \tau' = \tau \end{aligned}$$

Specifically the trace has the property that its length never decreases during the program execution.

CHAPTER 11

The Reachability-bound

The operator $L : \mathcal{C} \rightarrow \mathcal{P}(\mathcal{L})$, computes the set of all labels in a labeled command c defined as follows, we call it program points. Every program point is unique.

Definition 45 (Program Points ($L : \mathcal{C} \rightarrow \mathcal{P}(\mathcal{L})$)).

$$L(c) \triangleq \begin{cases} \{l\} & c = [x \leftarrow e]^l \\ L(c_1) \cup L(c_2) & c = c_1; c_2 \\ L(c_1) \cup L(c_2) \cup \{l\} & c = \text{if } ([b]^l, c_1, c_2) \\ L(c_w) \cup \{l\} & c = \text{while } [b]^l \text{ do } (c_w) \end{cases}$$

Every program point corresponds to a labeled command in a program (either a guard of if or while command, or an assignment command), and it is unique. The Lemma below formally describes the uniqueness property of the program points with proof in Appendix B.1.

Lemma 11.0.1 (Uniqueness of The Program Point). *For every program $c \in \mathcal{C}$ and every two labels such that $i, j \in L(c)$, then $i \neq j$.*

$$\forall c \in \mathcal{C}, i, j \in \mathcal{L} . i, j \in L(c) \implies i \neq j.$$

Definition 46. (An Event Belongs to A Trace) *An event $\epsilon \in \mathcal{E}$ belongs to a trace $\tau \in \mathcal{T}^{+\infty}$, i.e., $\epsilon \in \tau$ are defined as follows:*

$$\epsilon \in \tau \triangleq \begin{cases} \text{true} & \tau = [\epsilon]_{++}\tau' \wedge \epsilon = \epsilon' \\ \epsilon \in \tau' & \tau = [\epsilon']_{++}\tau' \wedge \epsilon \neq \epsilon' \\ \text{false} & \tau = [] \vee \tau \in \mathcal{T}^\infty \end{cases}$$

As usual, we denote by $\epsilon \notin \tau$ that the event ϵ doesn't belong to the trace τ .

In the rest of the paper, we denote by $\perp$ a value s.t. $\perp < n$ for all $n \in \mathbb{N}$.

Definition 47 (Counter Notation for Program Point). *The counting operator $\text{cnt} : \mathcal{T}^{+\infty} \rightarrow \mathcal{L} \rightarrow (\mathbb{N} \cup \{\perp\})$ counts the appearance of a label in a trace.*

$$\begin{aligned} \text{cnt}([], l) &\triangleq 0 \\ \text{cnt}([(_, l, _)_{++\tau'}, l) &\triangleq \text{cnt}(\tau', l) + 1 \quad \text{if } l = l \\ \text{cnt}([(_, l', _)_{++\tau'}, l) &\triangleq \text{cnt}(\tau', l) \quad \text{if } l' \neq l \\ \text{cnt}(\tau, l) &\triangleq \perp \quad \text{if } \tau \in \mathcal{T}^\infty \end{aligned}$$

When the input trace is infinite, $\text{cnt}(\tau, l)$ returns $\perp$ for any program label l .

Definition 48 (Counter Notation for List of Program Point). *The counting operator $\text{lcnt} : \mathcal{T}^{+\infty} \rightarrow \mathcal{P}(\mathcal{L}) \rightarrow (\mathbb{N} \cup \{\perp\})$ counts the appearance of a list of labels $[l_1, \dots, l_n]$ as follows.*

$$\begin{aligned} \text{lcnt}([], [l_1, \dots, l_n]) &\triangleq 0 \\ \text{lcnt}(\tau''_{++\tau'}, [l_1, \dots, l_n]) &\triangleq \text{lcnt}(\tau', [l_1, \dots, l_n]) + 1 \quad \text{if } \pi_2(\tau''[i]) = l_i, \forall i = 1, \dots, n \\ \text{lcnt}([(_, l, _)_{++\tau'}, [l_1, \dots, l_n]) &\triangleq \text{lcnt}(\tau', [l_1, \dots, l_n]) \quad \text{if } l \neq l_1 \\ \text{lcnt}(\tau, [l_1, \dots, l_n]) &\triangleq \perp \quad \text{if } \tau \in \mathcal{T}^\infty \end{aligned}$$

When the input trace is infinite, $\text{lcnt}(\tau, L)$ returns $\perp$ for any list of labels as well.

We define the operator $\text{L}_{\text{trace}} : \mathcal{T}^{+\infty} \rightarrow \mathcal{P}(\mathcal{L})$ projects the label from every event in a trace as a list of program points, defined as follows,

$$\text{L}_{\text{trace}}([(_, l, _)_{++\tau'}) \triangleq [l]_{++\text{L}_{\text{trace}}(\tau')} \quad \text{L}_{\text{trace}}([]) \triangleq []$$

Definition 49 (Reachability-bound). *For a program c and a location $l \in \mathbf{L}(c)$ in this program, a function $f(c, l) : \mathcal{T}_0^+(c) \rightarrow (\mathbb{N} \cup \{\infty\})$ is a reachability-bound for l if and only if*

$$\begin{aligned} & \forall \tau_0 \in \mathcal{T}_0^+(c), c' \in \mathcal{C}, l, l' \in \mathbf{L}(c), \tau \in \mathcal{T}^{+\infty} . \\ & \left(\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \vee \langle c, \tau_0 \rangle \uparrow^\infty \tau_{0++\tau} \right) \implies f(c, l)(\tau_0) \geq \text{cnt}(\tau, l) \end{aligned}$$

Given a program point l in c , our algorithm (introduced in the following sections) computes a reachability-bound for l in c . It is easy to compute a trivial reachability-bound $f(c, l) : \mathcal{T}_0^+(c) \rightarrow \infty$, but it is not interesting to us. Ideally, we aim to compute a precise reachability-bound as below.

Definition 50 (Precise Reachability-bound). *For a program c and a location $l \in \mathbf{L}(c)$ in this program, $\text{RB}_{\text{exe}}(c, l) : \mathcal{T}_0^+(c) \rightarrow (\mathbb{N} \cup \{\infty\})$ is a Precise Reachability-bound if and only if,*

$$\begin{aligned} & \forall \tau_0 \in \mathcal{T}_0^+(c), c' \in \mathcal{C}, l, l' \in \mathbf{L}(c), \tau \in \mathcal{T}^{+\infty} . \\ & \left(\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \vee \langle c, \tau_0 \rangle \uparrow^\infty \tau_{0++\tau} \right) \implies f(c, l)(\tau_0) = \text{cnt}(\tau, l) \end{aligned}$$

CHAPTER 12

Path-sensitive Reachability-bound Analysis

12.1 MAIN STEPS

Our static program analysis algorithm computes a *reachability-bound* for every program point l in a program c in a path-sensitive manner. The algorithm is summarized into the following three steps,

1. The Section 12.2 first computes the *Abstract Transition Graph*, $\text{absG}(c)$ for a program c through program abstraction. It computes the abstract transition for every labeled command in c and each abstract transition is an edge in $\text{absG}(c)$.
2. The second step in Section 12.3 computes the *Refined Program*, rp for a program c based on its abstraction transition graph by the program refinement algorithm in Gulwani et al. (2009a). This algorithm transforms the multiple-path loops into multiple loops where the interleaving of paths is explicit.
3. At the same time, we also compute the *Ranking Function* in Section 12.4¹ for every transition edge and estimate the upper bound invariant w.r.t, the input variables on every ranking function's maximum value.
4. The *path-sensitive reachability-bound* algorithm computes the *reachability-bound* for every program point based on the *refined program* and the upper bound invariant of the *ranking function* computed in previous stages. It also requires computing several quantities to obtain the final result. Section 12.5 intro-

¹**ranking function** is the name used in Sinn et al. (2014) and **local bound** is the name used in Zuleger et al. (2011); Sinn et al. (2017). We refer to the two names as having the same meaning in this paper.

duces these quantities that are needed to be computed in this algorithm and the following sections describe the computations of each quantity.

5. Section 12.6 gives the computation of the first quantity – the local *path reachability-bound*, $\text{pathLocRB}(\text{rp}, \text{tp})$ for every *simple transition path* tp in Section 12.6. This is the local bound on the numbers that tp is evaluated when executing the closet loop rp in which tp is nested. This local bound considers only the execution of the innermost loop where tp is located.
6. Then in Section 12.7, we compute the second quantity – *loop reachability-bound*, $\text{lpRB}(l : \text{rp}, \text{tp})$ for every *simple transition path* tp w.r.t its outer loop $l : \text{rp}$. This is the bound on the iteration numbers of the outer loop l , such that during these iterations, the inner loop where tp is located is reached.
7. Then using the local *path reachability-bound* and the *loop reachability-bound*, we compute the *path reachability-bound* globally. The *path reachability-bound*, $\text{pathRB}(\text{rp}, \text{tp})$ is a global upper bound on the execution times of a *simple transition path* tp computed in Section 12.8.
8. With the global reachability-bound of every simple transition path, now we can sum up all the *path reachability-bounds*, $\text{pathRB}(\text{rp}, \text{tp})$ over the tp which contains the program point l , and compute the final result – *reachability-bound*, $\text{psRB}(c, l)$ for every program point $l \in \text{LV}(c)$ as in Definition 68.

12.2 ABSTRACTION TRANSITION GRAPH

This path-sensitive reachability-bound algorithm is performed on basis of an *Abstract Transition Graph* for the program c . This step shows how to generate the

abstract transition graph $\text{absG}(c)$ of a program c by constructing its vertices and edges.

12.2.1 Vertices Construction

Every vertex corresponds to a program execution point, and the vertices set is all the c 's program execution points with an extra exit point ex ,

$$\text{absV}(c) = L(c) \cup \{\text{ex}\}$$

12.2.2 Edges Construction

Each edge $(l \xrightarrow{dc} l') \in \text{absE}(c)$ is an abstract transition between two program points l, l' . The edge is constructed by the standard control flow analysis satisfying there is an edge from l to l' if and only if the command with label l' can execute right after the execution of the command with label l . We use the same algorithm as in Section 6.3.1 to compute the constraint and edges. Specifically we compute the *constraint*, the *initial and continuation state* for each command and then generate the set of edges for the program, $\text{abstrace}(c)$ in which each edge is an *abstract event*.

Constraint Computation A constraint describes the abstract execution of the assignment command with the label l , or abstract evaluation of the boolean expression in the guard with label l .

Again we first introduce some components of the constraint. Most of them are the same as the definitions in Section 6.3.1. We present them in detail in our simplified program model.

Definition 51 (Symbolic Constant, Symbolic Expression). *The universe of the symbolic constant is denoted by $SC \subseteq \mathbb{N} \cup \mathcal{V} \cup \{\infty\}$, in which a symbolic constant can be*

a natural number, ∞ , or a program's input variable. The symbolic expression of a program $A_{sc}(c) \subseteq \mathcal{A}$ is the set of all the arithmetic expressions over $\mathbb{N} \cup V_{in}(c) \cup \{\infty\}$ for the program c .

Definition 52 (Difference Constraints). *The difference constraints $DC(\mathcal{V} \cup SC)$ is the set of all the inequality of form $x' \leq y + v$ or $x' \leq v$ where $x, y \in \mathcal{V}$ and $v \in SC$. An inequality $x' \leq y + v$ describes that the value of x in the current state is at most the value of y in the previous state plus the symbolic constant v , and $x' \leq v$ describes that the value of x in the current state is at most v .*

Definition 53 (Constraints). *A constraints $dc \in \mathcal{DC}^\top = DC(\mathcal{V} \cup SC) \cup \mathcal{B} \cup \{\top\}$ can be either a difference constraint $d \in DC(\mathcal{V} \cup SC)$, a boolean expression $b \in \mathcal{B}$ or $\top$ denotes always true.*

When the constraint in a transition is a difference constrain, $l \xrightarrow{x' \leq y + v} l'$, it denotes that the value of variable x after executing the command at l is at most the value of variable y plus v before the execution, and $l \xrightarrow{x' \leq v} l'$ respectively denotes x has at most the value of the symbolic constant v . For example in Figure 9.3(b), constraint $i' \leq i - 1$ on the edge $7 \xrightarrow{i' \leq i - 1} 1$ describes the execution of the command at line 7, $[i \leftarrow i - 1]^7$.

The boolean constraint, b on an edge $l \xrightarrow{b} l'$ describes that after evaluating the guard with label l , b holds and the command with label l will execute right after. In Figure 9.3, $i \leq 0$ on the edge $1 \xrightarrow{i \leq 0} ex$, represents the negation of the testing guard $[i > 0]^1$ in the `while` command, and $i \leq 0$ must hold in order to perform this transition from program point 1 to the program exit. We also have $\top$, which is preserved for `skip` command or the commands that don't interfere with any guard variable.

Definition 54 (Constraint Computation). *For a program c , a boolean expression b in the guard of a `if` or `while` command or an expression e and a variable x in an assignment command $x \leftarrow e$, the constraint $\text{absexpr}(b, _, c)$ or $\text{absexpr}(x - v, x, c)$ is computed as follows.*

1. Compute the symbolic constant set of program c , $\text{SC}(c)$ of program c as follows.

$$\text{SC}(c) = \mathbb{N} \cup \text{V}_{\text{in}}(c) \cup \{\infty\} \subseteq \mathcal{SC}$$

2. Initialize $\text{V}_{\text{guard}}(c) = \{\}$ as the set of the variable used in the expression of every `while` or `if` guard in the program c .
3. Then we compute $\text{absexpr}(b, _, c)$ and $\text{absexpr}(e, x, c)$.

$$\begin{array}{ll}
 \text{absexpr}(x - v, x, c) = x' \leq x - v & x \in \text{V}_{\text{guard}}(c) \wedge v \in \text{SC}(c) \\
 \text{absexpr}(y + v, x, c) = x' \leq y + v & x, y \in \text{V}_{\text{guard}}(c) \wedge v \in \text{SC}(c) \\
 \text{absexpr}(v, x, c) = x' \leq v & x \in \text{V}_{\text{guard}}(c) \wedge v \in \text{SC}(c) \\
 \text{absexpr}(y + v, x, c) = x' \leq y + v, & x \in \text{V}_{\text{guard}}(c) \wedge v \in \text{SC}(c) \\
 \text{V}_{\text{guard}}(c) = \text{V}_{\text{guard}}(c) \cup \{y\} & \wedge y \notin \text{V}_{\text{guard}}(c) \\
 \text{absexpr}(b, _, c) = b, \text{V}_{\text{guard}}(c) = \text{V}_{\text{guard}}(c) \cup \text{FV}(b) & b \in \text{SC}(c) \\
 \text{absexpr}(e, x, c) = \top & o.w.
 \end{array}$$

$\text{absexpr}(e, x, c)$ and $\text{V}_{\text{guard}}(c)$ are iteratively updating until stabilized over every guard and assignment command in c . In the following part of the paper, we use $\text{absexpr}(e, x, c)$ to denote the stabilized computation result.

In case 4, if a variable x , belonging to the set $\text{V}_{\text{guard}}(c)$ is updated by a variable y , which isn't in this set, we add y into the set $\text{V}_{\text{guard}}(c)$ and repeat above procedure until $\text{V}_{\text{guard}}(c)$ and $\text{absexpr}(e, x, c)$ is stabilized.

Specifically we handle a normalized boolean expression, $x > 0$ in guards of if and while loop headers, and the guard variable x only increases, decreases, or is reset by simple arithmetic expression (mainly multiplication, division, minus, and plus and the extended operations max and min). The guard variable x is generalized into the *norm* when the guard in if or while has the form $e > 0$, where e is a norm and V_{guard} is generalized into the set of norms. Then in the last clause in Definition 54, i.e., $\text{absexpr}(e, x, c)$, we will check if e is a norm or contains norms, and extend the $V_{\text{guard}}(c) = V_{\text{guard}}(c) \cup \{e\}$ if neither. The constraint computation over the *norm* follows the computation steps 1, 2, and 3 in Section 6.1 in paper Sinn et al. (2017).

Initial and Continuation State Computation This step computes two sets for each command. The initial state is a set that contains the program points before executing this command, which is computed by the standard initial state generation method from control flow analysis. The continuation state is a set that contains the constraint of this command and the program points after the execution of this command. This set is enriched by the standard control flow analysis.

- The *initial state*, $\text{absinit}(c) \in \mathcal{L}$ for a command c . $\text{absinit}(c)$ is a unique program label that corresponds to the command before executing this command.

Given a program c , its abstract initial state, $\text{absinit}(c)$ is computed as fol-

lows,

$$\begin{aligned}
\text{absinit}([x \leftarrow e]^l) &= l \\
\text{absinit}([\text{skip}]^l) &= l \\
\text{absinit}(\text{ if } ([b]^l, c_t, c_f)) &= l \\
\text{absinit}(\text{ while } [b]^l \text{ do } (c_w)) &= l \\
\text{absinit}(c_1; c_2) &= \text{absinit}(c_1)
\end{aligned}$$

- The *continuation state*, $\text{absfinal}(c)$ of program c , $\text{absfinal}(c) \in \mathcal{P}(\mathcal{L} \times \mathcal{DC}^\top)$ is a set of pairs, (l, dc) with a program point (i.e., a label), l as the first component and a constraint, dc as the second component. The program point l corresponds to the labeled command after the execution of c , and the constraint dc in this pair is computed by absexpr for the expression in c .

Given a program c , its continuation state, $\text{absfinal}(c)$ is computed as follows,

$$\begin{aligned}
\text{absfinal}([x \leftarrow e]^l) &= \{(l, \text{absexpr}(e, x, c))\} \\
\text{absfinal}([\text{skip}]^l) &= \{(l, \top)\} \\
\text{absfinal}(\text{ if } ([b]^l, c_t, c_f)) &= \text{absfinal}(c_t) \cup \text{absfinal}(c_f) \\
\text{absfinal}(\text{ while } [b]^l \text{ do } (c_w)) &= \{(l, \text{absexpr}(b, \top, c_w))\} \\
\text{absfinal}(c_1; c_2) &= \text{absfinal}(c_2)
\end{aligned}$$

Abstract Event Computation Each abstract event is an edge between two vertices in the abstract transition graph. It is generated by computing the initial and continuation execution state interactively and recursively for a program c .

Definition 55 (Abstract Event). *Abstract Event*: $\hat{e} \in \mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L}$ is a triple where the first and third components are labels, second component is a constraint from $\mathcal{DC}^\top$.

In an abstract event (l, dc, l') of a program c , the first label $l \in \mathcal{L}$ is the *initial*

state $\text{absinit}(c)$ of c , and the second label $l' \in \mathcal{L}$ with the constraint $dc \in \mathcal{DC}^\top$ correspond to a *continuation state* of c .

The set of the abstract events $\text{abstrace}(c)$ for a program c is computed as follows in Definition 30.

Definition 56 (Abstract Execution Trace Computation). *The abstract trace, $\text{abstrace}(c) \in \mathcal{P}(\mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L})$ of a program c is computed as follows.*

We first append a skip command with the label ex , i.e., $[\text{skip}]^{\text{ex}}$ at the end of the program c , and construct the program $c' = c; [\text{skip}]^{\text{ex}}$. Then, we compute the $\text{abstrace}(c) = \text{abstrace}'(c')$ for c' as follows,

$$\begin{aligned}
\text{abstrace}'([x \leftarrow e]^l) &= \emptyset \\
\text{abstrace}'([\text{skip}]^l) &= \emptyset \\
\text{abstrace}'(\text{if } ([b]^l, c_t, c_f)) &= \text{abstrace}'(c_t) \cup \text{abstrace}'(c_f) \\
&\quad \cup \{(l, \text{absexpr}(b, \top, c), \text{absinit}(c_t))\}, \\
&\quad \cup \{(l, \text{absexpr}(\neg b, \top, c), \text{absinit}(c_f))\} \\
\text{abstrace}'(\text{while } [b]^l \text{ do } (c_w)) &= \text{abstrace}'(c_w) \cup \{(l, \text{absexpr}(b, \top, c), \text{absinit}(c_w))\} \\
&\quad \cup \{(l', dc, l) \mid (l', dc) \in \text{absfinal}(c_w)\} \\
\text{abstrace}'(c_1; c_2) &= \text{abstrace}'(c_1) \cup \text{abstrace}'(c_2) \\
&\quad \cup \{(l, dc, \text{absinit}(c_2)) \mid (l, dc) \in \text{absfinal}(c_1)\}
\end{aligned}$$

Notice $\text{abstrace}'([x \leftarrow e]^l)$ and $\text{abstrace}'([\text{skip}]^l)$ are both empty set. For every event ϵ with label l in an execution trace τ of program c , there is an abstract event in the program's abstract execution trace of the form $(l, _, _)$. We also show the soundness of the *abstract event computation* as Theorem 6.3.1 below with proof in Appendix B.2.

Lemma 12.2.1 (Soundness of the Abstract Execution Trace). *For any program c with*

an arbitrary initial trace $\tau_0 \in \mathcal{T}_0^+(c)$ and a finite execution trace $\tau \in \mathcal{T}^+$ that is generated when c is executing under τ_0 , then for every event $\epsilon \in \tau$ such that $\epsilon = (_, l, _)$ for some label l in program c , there is an abstract event $\hat{\epsilon} = (l, _, _) \in \text{abstrace}(c)$ and $\hat{\epsilon} = (l, _, _)$.

$$\forall c, c' \in \mathcal{C}, \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, \epsilon = (_, l, _) \in \mathcal{E}, l \in \mathcal{L}(c) .$$

$$(\langle c, \tau_0 \rangle \rightarrow^* \langle c', \tau_0 + \tau \rangle) \wedge \epsilon \in \tau \implies \exists \hat{\epsilon} \in \text{abstrace}(c) \wedge \hat{\epsilon} = (l, _, _)$$

If l is the label of an assignment command in a program c , then there is a unique abstract event in the program's abstract events set $\hat{\epsilon} \in \text{abstrace}(c)$ of form $(l, _, _)$. The following Lemma 12.2.2 states this uniqueness property and the proof is in Appendix B.2.

Lemma 12.2.2 (Uniqueness of the Abstract Events). *For every program c and a finite execution trace $\tau \in \mathcal{T}^+$ which is generated w.r.t. an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, there is a unique abstract event $\hat{\epsilon} = (l, _, _) \in \text{abstrace}(c)$ for every assignment event $\epsilon \in \mathcal{E}^{\text{asn}}$ that has the same label l in the execution trace, specifically, $\epsilon = (_, l, _) \in \mathcal{E}^{\text{asn}}$ and $\epsilon \in \tau$.*

$$\forall c \in \mathcal{C}, \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, \epsilon = (_, l, _) \in \mathcal{E}^{\text{asn}} . (\langle c, \tau_0 \rangle \rightarrow^* \langle c', \tau_0 + \tau \rangle) \wedge \epsilon \in \tau$$

$$\implies \exists! \hat{\epsilon} = (l, _, _) \in (\mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L}) . \hat{\epsilon} \in \text{abstrace}(c)$$

Edge Construction The edge for c 's abstract transition graph is constructed simply by computing the program's abstract events set, $\text{abstrace}(c)$ as follows,

$$\text{absE}(c) = \{(l_1, dc, l_2) \mid (l_1, dc, l_2) \in \text{abstrace}(c)\}$$

For each edge $(l, dc, l') \in \text{absE}(c)$, dc describes either an abstract execution of the assignment command with label l , or the evaluation of the guard with label l .

Again in Figure 9.3(b), the edge $(0 \xrightarrow{i' \leq n} 1)$ on the top tells us the command

$[i \leftarrow n]^0$ is executed with an abstract transition, $i' \leq n$ and a continuation point 1. According to the continuation point 1, the while loop with header at location 1, $\text{while } [i > 0]^1 \text{ do } \{\dots\}$ will be executed right after.

12.2.3 Abstract Transition Graph Construction

With the vertices $\text{absV}(c)$ and edges $\text{absE}(c)$ ready, we construct the abstract transition graph, formally in Definition 31.

Definition 57 (Abstract Transition Graph). *The abstract transition graph of a program c is $\text{absG}(c) \triangleq (\text{absV}(c), \text{absE}(c))$, where $\text{absV}(c) \triangleq L(c) \cup \{\text{ex}\}$ and each edge $(l, dc, l') \in \text{absE}(c)$ is an abstract transition $\text{absE}(c) \triangleq \{(l_1, dc, l_2) \mid (l_1, dc, l_2) \in \text{abstrace}(c)\}$.*

Definition 58 (Path). *A path on $\text{absG}(c)$ is a sequence, $l_0 \xrightarrow{dc_0} l_1 \xrightarrow{dc_1} \dots$ with*

- *the vertices sequence $(l_0, \dots)$, where $l_i \in \text{absV}(c)$ for every $i = 0, 1, \dots$ and*
- *the edge sequence $(e_0, \dots)$, where $e_i = (l_i, dc_i, l_{i+1}) \in \text{absE}(c)$ for every $i = 0, 1, \dots$*

A path is cyclic, if it has the same start- and end-point. A path is simple, if it does not visit a location twice except for the start- and end-location. We use $\mathcal{PATH}(\text{absG}(c))$ to denote the set of all the paths on $\text{absG}(c)$, and $L_{\text{path}}(p)$ to represent the list of program points corresponding to the vertices sequence of a path $p \in \mathcal{PATH}(\text{absG}(c))$.

For example in Figure 9.3(b), $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 4$ is a *path*, but it is not simple (the program points 4 is visited twice). The path $4 \rightarrow 5 \rightarrow 4$ is both cyclic and simple.

Example. In Figure 9.3(b), the edge $(0 \xrightarrow{i' \leq n} 1)$ on the top tells us the command $[i \leftarrow n]^0$ is executed with a continuation point 1, and the command $\text{while } [i > 0]^1 \text{ do } \{\dots\}$ will be executed next. The annotation $i' \leq n$ is a difference constraint computed by

absexpr over the expression n in the assignment command $i \leftarrow n$. It represents that the value of i is less than or equal to the value of n after the execution of $[i \leftarrow n]^0$ and before evaluating the guard $[i > 0]^1$. Another example constraint $i' \leq i - 1$ on the edge $7 \xrightarrow{i' \leq i - 1} 1$ describes the execution of the command at line 7, $[i \leftarrow i - 1]^7$. The i' on the left side of $i' \leq i - 1$ represents the value of i after the assignment operation, and the right-hand side i stores the value before the assignment. The boolean constraint $i \leq 0$ on the edge $1 \xrightarrow{i \leq 0} \text{ex}$, represents the negation of the testing guard $[i > 0]^1$ in the `while` command. $1 \xrightarrow{i \leq 0} \text{ex}$ denotes that $i \leq 0$ must hold to perform this transition from program point 1 to the program exit.

12.3 LOOP REFINEMENT

There are three steps as follows.

1. **Collecting** all *simple transition paths*. Every *simple transition path* is loop free and interleaving free. It contains only the edges of atomic assignment or guard transitions without interleaving other paths.
2. **Rewriting** the program in the form of combinations of *simple transition paths*.
3. **Refining** the program by explicating the execution orders of the simple transition paths.

12.3.1 The Simple Transition Path

We first collect the loop headers $L_{\text{loop}}(c) \subseteq L(c)$ from a program c , which is the set of all program points corresponding to the loop headers in program c .

Definition 59 (Loop Headers ($L_{\text{loop}} : \mathcal{C} \rightarrow \mathcal{P}(\mathcal{L})$)).

$$L_{\text{loop}}(c) \triangleq \begin{cases} \{\} & c = [x \leftarrow e]^l \\ L_{\text{loop}}(c_1) \cup L_{\text{loop}}(c_2) & c = c_1; c_2 \\ L_{\text{loop}}(c_t) \cup L_{\text{loop}}(c_f) & c = \text{if } ([b]^l, c_t, c_f) \\ L_{\text{loop}}(c_w) \cup \{l\}, & c = \text{while } [b]^l \text{ do } (c_w) \end{cases}$$

Definition 60 (Simple Transition Path). A simple transition path $\text{tp} \in \mathcal{PATH}(\text{absG}(c))$ for the program c , is either a simple cyclic path, which has the same start- and end-point or a simple path has either different while loop headers, the program entrance or exit as its start- and end-point without visiting any loop header inside the path.

Specifically, a path $l_0 \xrightarrow{dc_0} l_1 \xrightarrow{dc_1} \dots l_n \in \mathcal{PATH}(\text{absG}(c))$ with the vertices sequence $(l_0, \dots, l_n)$, where $l_i \in \text{absV}(c)$ for every $i = 0, \dots, n$ and the edge sequence $(e_1, \dots, e_n)$, where $e_i = (l_{i-1}, dc_i, l_i) \in \text{absE}(c)$ for every $i = 1, \dots, n$, is a simple transition path if and only if it satisfies,

- $l_i \neq l_j$ for every $i = 0, \dots, n$ and $j = 1, \dots, n - 1$,
- l_0 is either the program point of a loop header or the program entrance ($l_0 = 0$), i.e., $l_0 \in L_{\text{loop}}(c) \cup \{0\}$
- and l_n is either the program point of a loop header or the program exit ($l_n = \text{ex}$), i.e., $l_n \in L_{\text{loop}}(c) \cup \{\text{ex}\}$,
- and $l_i \notin L_{\text{loop}}(c) \cup \{0, \text{ex}\}$ for every $i = 1, \dots, n - 1$.

Example. For example in Figure 9.3(b), the $1 \rightarrow 2 \rightarrow 8 \rightarrow 1$ is a simple transition path. However, $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 1$ is not a simple transition path because it is not simple. Also, $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 1$ is not a simple transition path because it visits a loop header 4 inside the path. $1 \rightarrow 2 \rightarrow 3$ isn't either because

its end-point isn't a loop header. In Figure 9.2(b), $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$ is not a *simple transition path* on $\text{absG}(\text{threeNestedWhile}(n, m, N))$ because it visits a loop header 3 inside the path.

12.3.2 The Program Rewriting

Syntax.

$$\text{rp} := \text{tp} \mid \text{repeat}(\text{rp}) \mid l : \text{repeat}(\text{rp}) \mid \text{rp}; \text{rp} \mid \text{choose}\{\text{rp}, \dots\}$$

where $l \in L_{\text{loop}}(c)$ is a loop header.

In the new syntax, $\text{repeat}(\text{rp})$ is a loop statement iterating over the transitions in rp . $l : \text{repeat}(\text{rp})$ represents that the loop $\text{repeat}(\text{rp})$ corresponds to a loop of the source program with loop header l . The multiple-paths statement $\text{choose}\{\text{rp}, \dots\}$ contains all the execution paths of a loop from the source program.

Algorithm. Algorithm 7 transforms the syntax of the while language program into the new syntax defined above.

In this algorithm, we first use a simple depth-first search strategy to collect all the *simple transition paths* satisfying the Definition 60. In line:2, we initialize each candidate c_i with a *simple transition path* tp_i . New candidates generated in line:4, 5, and 6 correspond to the multiple-paths statement $\text{choose}\{\dots\}$, loop $\text{repeat}(c_i)$, or sequence statements $c_i; c_j$ respectively.

- Line:5 for every candidate c' , if it starts and ends with the same vertex, then it is a loop path, and we rewrite it into a loop statement $\text{repeat}(c)$.
- Line:4 for all the candidates $c_1, \dots, c_m$ having the same starting and ending

Algorithm 7 Program Rewriting Algorithm. $\text{Rewrite}(c)$

Require: program c

- 1: collects all c 's simple transition paths from $\text{absG}(c)$, $\text{tp}_1, \dots, \text{tp}_n \in \mathcal{PATH}(\text{absG}(c))$.
 - 2: **init:** candidate set $W = \{c_1, \dots, c_n\}$, where $c_i = \text{tp}_i$ and $i = 1, \dots, n$
 - 3: **while** $W.\text{size}() > 1$:
 - 4: create $c' = \text{choose}\{c_1, \dots, c_m\}$ s.t. $c_i \in W \wedge c_i[0] = c_j[0] \wedge c_i[-1] = c_j[-1], i, j = 1, \dots, m$.
 $W.\text{add}(c')$ $W.\text{remove}(c_1, \dots, c_m)$
 - 5: create $c' = \text{repeat}(c)$ s.t. $c_i \in W \wedge c[0] = c[-1] \wedge c[0] \in L_{\text{loop}}(c)$
 $W.\text{add}(c[0] : c')$, $W.\text{remove}(c)$
 - 6: create $c' = c_1; c_2$ s.t. $c_1, c_2 \in W \wedge c_1[-1] = c_2[0]$
 $W.\text{add}(c')$ $W.\text{remove}(c_1, c_2)$
 - 7: **Endwhile**
 $c^T = W[0]$
 - 8: **return** c^T .
-

vertices, then they are multiple paths of a loop, and we rewrite them into $\text{choose}\{c_1, \dots, c_m\}$.

- Line:6 for every two candidates c_1, c_2 , if c_1 ends with the same vertex as c_2 's starting label, rewrite them into sequence statement $c_1; c_2$.

12.3.3 The Program Refinement

To compute the refined program rp , we are inspired from the algorithm **REFINE** in paper Gulwani et al. (2009a).

We first compute the execution orders of the simple transition paths that come from the same loop. For example, two simple transition paths $\text{tp}_1 = (1 \rightarrow 2 \rightarrow 3 \rightarrow 4)$ and $\text{tp}_4 = (1 \rightarrow 2 \rightarrow 8 \rightarrow 1)$ in the same loop Figure 9.3(b) have two possible execution orders: either tp_1 executes after the execution of tp_4 , or vice verse.

Then we construct the refined program by explicating the execution orders of simple transition paths. Simple transition paths can iterate on themselves, so an

execution order could contain iterations of simple transition paths, and sequence of iterations. For example, the simple transition path $tp_3 = (4 \rightarrow 5 \rightarrow 4)$ the loop L_4 in Figure 9.3(b) has only one possible execution order, which is the iteration on itself, $repeat(tp_3)$.

Each loop in a refined program contains several execution orders over simple transition paths. For example, the loop L_1 in Figure 9.3(b) has two execution orders. In the first one, $repeat(tp_4; tp_1; 4 : repeat(tp_3); tp_2)$ tp_1 will execute right after the execution of tp_4 . Then following the execution of tp_1 , loop L_4 finishes full iterations and followed by the execution of tp_2 . In the second execution order, $repeat(tp_1; 4 : repeat(tp_3); tp_2; tp_4)$, tp_4 is executed after the execution of $tp_1; 4 : repeat(tp_3); tp_2$. Iteration of one execution order is equivalent to one possible full iterations of the source loop program. In this sense, we construct $1 : repeat(tp_1; 4 : repeat(tp_3); tp_2; tp_4)$ as an execution path of the loop L_1 in the source program with loop header 1. The refined loop program for L_1 is

$$\text{choose} \left\{ \begin{array}{l} 1 : repeat(tp_1; 4 : repeat(tp_3); tp_2; tp_4), \\ 1 : repeat(tp_4; tp_1; 4 : repeat(tp_3); tp_2) \end{array} \right\}.$$

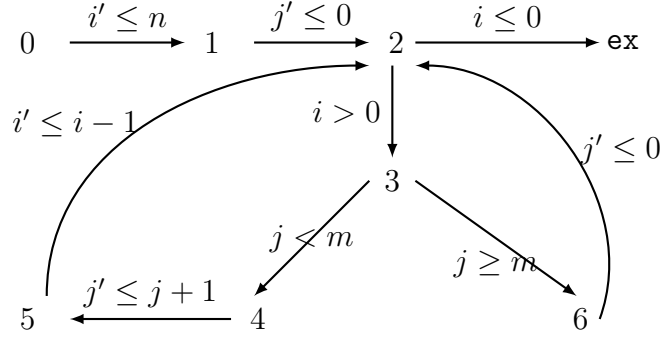
Then we construct the refined program for program $nestedOdd(n, m)$ in Figure 9.3(a) as follows, which is also presented in Figure 9.3(c).

$$tp_0; \text{choose} \left\{ \begin{array}{l} 1 : repeat(tp_1; 4 : repeat(tp_3); tp_2; tp_4), \\ 1 : repeat(tp_4; tp_1; 4 : repeat(tp_3); tp_2) \end{array} \right\}; tp_5.$$

Example. We present again the motivating example program from Figure 9.1(a) below in Figure 12.1(a) to show the necessity of the *path refinement*. It has a two paths loop with different *reachability-bounds* on the locations of different paths.

$\text{twoPathsWhile}(n, m) \triangleq$
 $[i \leftarrow n]^0; [j \leftarrow 0]^1;$
 $L_2 : \text{while } [i > 0]^2 \text{ do}$
 $(\text{if } ([j < m]^3,$
 $\text{then } [j \leftarrow j + 1]^4;$
 $\quad [i \leftarrow i - 1]^5,$
 $\text{else } [j \leftarrow 0]^6);)$

(a)



(b)

$\text{tp}_0 = 0 \rightarrow 1 \rightarrow 2, \text{tp}_2 = 2 \rightarrow 3 \rightarrow 6 \rightarrow 2,$
 $\text{tp}_1 = 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 2, \text{tp}_3 = 2 \rightarrow \text{ex}^2$

(c)

$\text{tp}_0; \text{choose}\{2 : \text{tp}_1, 2 : \text{tp}_2\}; \text{tp}_3.$

(d)

$\text{tp}_0; \text{choose}\{2 : \text{repeat}(\text{repeat}(\text{tp}_1); \text{tp}_2), 2 : \text{repeat}(\text{tp}_1)\}; \text{tp}_3.$

(e)

Figure 12.1: (a) The two paths loop example. (b) The abstract transition graph for $\text{twoPathsWhile}(n, m)$. (c) The simple transition paths. (d) The rewrote program. (e) The refined program.

Over its abstract transition graph in Figure 12.1(b), we first collect all its simple transition path as in Figure ??(c). Then we transform the multiple-path loop by computing the interleaving between paths explicitly and generates a refined program rp as the bottom part of Figure 12.1.

We first compute that tp_1 can iterate on itself, so there is one possible execution sub-order, $\text{repeat}(\text{tp}_1)$. We recursively compute that after tp_1 done with the iteration on itself, either the iteration of the original loop is finished or tp_2 can execute right after. For the first case, we compute a complete execution order $\text{repeat}(\text{tp}_1)$. In the second case, we again have a possible execution sub-order, $\text{repeat}(\text{tp}_1); \text{tp}_2$. After execution of tp_2 , we compute that tp_1 iterates on itself again and goes back to the same execution sub-order. The refinement algorithm finds this repetition

and generate a complete execution order $\text{repeat}(\text{repeat}(\text{tp}_1); \text{tp}_2)$. Then the algorithm finish the refinement process since there is no more sub-orders. The refined program then is constructed below as in Figure 12.1(e).

$$\text{tp}_0; \text{choose}\{2 : \text{repeat}(\text{repeat}(\text{tp}_1); \text{tp}_2), 2 : \text{repeat}(\text{tp}_1)\}; \text{tp}_3.$$

From this refined program, we can explicitly tell two possible path interleaving patterns. With the explicit interleaving patterns, the following computation based on this refined program can compute more accurate reachability-bounds.

12.4 RANKING FUNCTION COMPUTATION AND ESTIMATION

This section is summarized into three steps as follows,

1. It first collects three edge sets for each variable, in which the variable increases, decreases, and is reset respectively.
2. Then, it assigns a variable (or a normalized counter expression that is viewed as variable) x to the edge on which this symbol decreases as this edge's ranking function.
3. In the last step, it estimates the upper bound invariant on the maximum value of each ranking function recursively. In the meantime, it also computes an upper bound on the iteration times of each loop in a path-insensitive manner.

The algorithm in this step is inspired by the Algorithm.2 in paper Sinn et al. (2014), the Algorithm.3 in paper Zuleger et al. (2011), and the Definition.25 in Section 4 of paper Sinn et al. (2017). Algorithm.3 in paper Zuleger et al. (2011) assigns a set of variables to each transition in which these variables decrease as the

local bound and estimates the maximum value of each variable in this set. Algorithm.2 in paper Sinn et al. (2014) assigns a variable to each edge on which this variable decreases as its ranking function and then estimates the maximum value for the ranking function. The Definition.25 in paper Sinn et al. (2017) assigns each transition with a variable that decreases in this transition, as the local bound and computes the bound similarly.

12.4.1 Collecting Variable Modifications

For each variable x in a program c , this step computes three edge sets, $\text{inc}(x, c)$, $\text{dec}(x, c)$, and $\text{re}(x, c)$ for x . Every edge in a set corresponds to a transition in which x is increased, decreased or reset respectively. The computation follows the same way as the Definition 33, we present them again as follows.

- $\text{inc} : \mathcal{C} \rightarrow \mathcal{V} \rightarrow \mathcal{P}(\hat{e})$ is the set of edges where the variable increase,

$$\text{inc}(x, c) = \{\hat{e} | \hat{e} = (l, x' \leq x + v, l') \wedge \hat{e} \in \text{abstrace}(c)\}$$

- $\text{dec} : \mathcal{V} \rightarrow \mathcal{P}(\hat{e})$ is the set of abstract events where the variable decrease,

$$\text{dec}(x, c) = \{\hat{e} | \hat{e} = (l, x' \leq x - v, l') \wedge \hat{e} \in \text{abstrace}(c)\}$$

- $\text{re} : \mathcal{C} \rightarrow \mathcal{V} \rightarrow \mathcal{P}(\hat{e})$ is the set of the abstract events where the variable is reset,

$$\text{re}(x, c) = \{\hat{e} | \hat{e} = (l, x' \leq y \pm v, l') \wedge x \neq y \wedge \hat{e} \in \text{abstrace}(c)\}$$

- Additionally, we also compute the reset graph $\text{reG}(c)$ and the reset chain, $\text{rechain}(x, c) \in \mathcal{P}(\mathcal{P}(\hat{e}))$ for every rank x . The $\text{rechain}(x, c)$ for every rank x

contains all the paths in $\text{reG}(c)$ that end at x . The computation of $\text{reG}(c)$ and $\text{rechain}(x, c)$ follows the Definition 20 in Sinn et al. (2017).

Definition 61. *For a program c , its reset graph $\text{reG}(c) = (\text{reV}(c), \text{reE}(c))$ has two components, each of them is computed as follows,*

$$\text{reE}(c) = \{(x, \hat{e}, y) \mid \hat{e} \in \text{re}(x, c) \wedge \hat{e} = (l, x' \leq y \pm v, l')\}$$

$$\text{reV}(c) = \{x \mid (x, _, _) \in \text{reE}(c) \vee (_, _, x) \in \text{reE}(c)\}$$

In a variable x 's reset chain set, $\text{rechain}(x, c)$, in each chain $(e_0, \dots, e_m) \in \text{rechain}(x, c)$ a variable x_i is reset by another variable x_{i+1} on edge e_i and x_{i+1} is reset on edge e_{i+1} recursively for every $i = 0, \dots, m - 1$. x is reset on the first edge e_0 of every sequence in $\text{rechain}(x, c)$. Each edge e_i in a sequence $(e_0, \dots, e_m) \in \text{rechain}(x, c)$ resets a variable x_i by another variable x_{i+1} such that x_{i+1} is reset on edge e_{i+1} recursively. The first edge e_0 of each sequence resets the variable x .

In the following steps, c is omitted in $\text{inc}(x, c)$, $\text{dec}(x, c)$ and $\text{re}(x, c)$ for concise when the reference of a program c is clear in the context.

12.4.2 Assigning The Ranking Function to An Edge

For each edge in the transition graph $\text{absG}(c)$ of a program c , this step assigns the variable that decreases on this edge as the ranking function of this edge. This step adopts the local bound computation method in Section 4 of Sinn et al. (2017) to assign the local bound to each edge, formally as follows.

Definition 62 (Ranking Function Generation). *For every edge $\hat{e}$ in the transition*

graph $\text{absG}(c)$ of a program c , its ranking function/local bound, $\text{rank}(\hat{e}, c)$ is the variable that decreases on this edge, computed as follows,

$$\begin{aligned}
\text{rank}(\hat{e}, c) &\triangleq 1 && \hat{e} \notin \text{SCC}(\text{absG}(c)) \\
\text{rank}(\hat{e}, c) &\triangleq x && \hat{e} \in \text{SCC}(\text{absG}(c)) \wedge \hat{e} \in \text{dec}(x, c) \wedge \hat{e} = (_, _, x' \leq x - v) \\
\text{rank}(\hat{e}, c) &\triangleq \text{rank}(\hat{e}', c) && \exists \hat{e}' \in \text{absG}(c), \text{tp} \in \mathcal{PATH}(\text{absG}(c)) . \hat{e}, \hat{e}' \in \text{tp} \\
\text{rank}(\hat{e}, c) &\triangleq x && \hat{e} \in \text{SCC}(\text{absG}(c)) \wedge \hat{e} \notin \bigcup_{x \in \mathcal{V}} \text{dec}(x, c) \\
&&& \wedge \hat{e} \notin \text{SCC}(\text{absG}(c) \setminus \text{dec}(x, c)) \\
\text{rank}(\hat{e}, c) &\triangleq \infty && o.w..
\end{aligned}$$

$\text{SCC}(\text{absG}(c))$ is the set of all the non-trivial strongly connected components of $\text{absG}(c)$.

The first case is straightforward. For the label l which is not in any while loop, the labeled command with the label l will be evaluated at most once. The second and third cases are guaranteed by the *Discussion on Soundness* in Section 4 in Sinn et al. (2017). The soundness is formalized in Lemma B.3.1 with proof in Appendix B.3.

Example In Figure 9.3(b), we assign variable i to edge $7 \xrightarrow{i' \leq i-1} 1$ as ranking. If we remove this edge, the edges on tp_2 and tp_1 are not in any SCC, so they also have the same ranking. In the same way, ranking for edges on tp_4 is i as well as k for them on tp_3 .

12.4.3 Ranking Function Estimation

This step estimates the upper bound, $\text{VB}(x, c) \in \mathbf{A}_{\text{sc}}(c)$ on the maximum value for each ranking function $x \in \mathcal{V} \cup \text{SC}(c)$.

For a program c , the upper bound invariant, $\text{VB}(\text{rank}(\hat{e}, c), c)$ for a ranking function

$\text{rank}(\hat{e}, c)$ is an invariant of the maximum value of $\text{rank}(\hat{e}, c)$ during the program execution.

In order to estimate the invariant of the maximum value of the ranking function $\text{rank}(\hat{e}, c)$ of the edge $\hat{e} \in \text{absE}(c)$, we use a mutual recursion and recursively compute the bound on the iteration times of the edge, $\text{TB}(\hat{e}, c)$ in the meantime.

Definition 63 (Ranking Function Estimation). *For a program c and an edge $\hat{e} \in \text{absE}(c)$, the ranking function bound, $\text{VB}(\text{rank}(\hat{e}, c), c)$ for the ranking function $x = \text{rank}(\hat{e}, c)$ of this edge is computed through a mutual recursion between $\text{VB}(x, c)$ and $\text{TB}(\hat{e}, c)$ as follows,*

$$\begin{aligned} \text{VB}(x, c) &\triangleq x && \text{if } x \in \text{SC}(c) \\ \text{VB}(x, c) &\triangleq \text{increase}(x, c) && \text{if } x \notin \text{SC}(c) \\ &\quad + \max\{\text{VB}(y, c) + v \mid (l, x' \leq y + v, l') \in \text{re}(x, c)\} \end{aligned}$$

where $\text{increase}(x, c) \triangleq \sum_{\hat{e} \in \text{inc}(x, c)} \{\text{TB}(\hat{e}, c) \times v \mid \hat{e} = (l, x' \leq x + v, l')\}$. $\text{TB}(\hat{e}, c) \in \mathbf{A}_{\text{sc}}(c)$ is an upper bound on the numbers that the transition $\hat{e}$ is evaluated, which is mutually recursively computed in a path-insensitive manner as below,

$$\begin{aligned} \text{TB}(\hat{e}, c) &\triangleq \text{VB}(\text{rank}(\hat{e}, c), c) \\ &\quad \text{if } \text{rank}(\hat{e}, c) \in \text{SC}(c) \\ \text{TB}(\hat{e}, c) &\triangleq \sum\{\text{increase}(y, c) \mid ch \in \text{rechain}(x, c) \wedge y \in ch\} \\ &\quad + \sum_{ch \in \text{rechain}(x, c)} \min\{\text{TB}(\hat{e}', c) \mid \hat{e}' \in ch\} \times (\text{VB}(\text{in}(ch), c) + \sum_{(_, (_, x' \leq y + v, _), _) \in ch} v) \\ &\quad \text{if } \text{rank}(\hat{e}, c) = x \wedge x \notin \text{SC}(c) \end{aligned} ,$$

where $\text{in}(ch)$ is the first vertex of the reset chain ch .

For every edge, if the ranking function of this edge computed at the previous step is a symbol, specifically an input variable, then this is already the upper bound invariant.

If instead, the ranking function of the edge is a variable y which is not an input variable, this step will eliminate it and replace it with a symbolic expression. In order to do this, these steps will compute two quantities: first, it will recursively sum over the TB of all the edges whose constraint may increment the variable y , multiplying the corresponding increment; second, it will recursively compute the upper bound invariant of all the other edges whose difference constraint may reset the variable y and take the maximum value. The sum of these two quantities provides the symbolic expression that is an upper-bound invariant on the maximum value of this ranking function. To compute these two quantities we use two mutually recursive procedures.

If a ranking function is increased in an edge of its nested loop, then this mutual recursion is non-terminating. In this case, we identify a cycle and produce infinity as the bound.

We also have the soundness of this path-insensitive transition bound. For a program c and an edge $\hat{e} \in \text{absE}(c)$, $\text{TB}(\hat{e})$ is a sound upper bound on the execution times of this transition by paper Sinn et al. (2017), formally below in Theorem 12.4.1 with proof in Appendix B.3.

Theorem 12.4.1 (Soundness of the Path-insensitive Transition Bound). *For each program c and an edge $\hat{e} = (l, _, _) \in \text{absG}(c)$, if l is the label of an assignment command, then its path-insensitive transition bound $\text{TB}(\hat{e}, c)$ is a sound upper bound on the execution times of this assignment command in c .*

$$\begin{aligned}
& \forall \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^{+\infty}, c \in \mathcal{C}, l, l' \in \text{LV}(c) . \\
& \left(\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \vee \langle c, \tau_0 \rangle \uparrow^\infty \tau_{0++\tau} \right) \\
& \implies \exists \hat{e} = (_, l, _) \in \text{abstrace}(c) \wedge \text{cnt}(\tau, l) \leq \llbracket \text{TB}(\hat{e}, c) \rrbracket(\tau_0)
\end{aligned}$$

Example. Using this ranking function-based approach, we first find the ranks for every transition, $\text{rank}(0 \rightarrow 1) = 1$, $\text{rank}(1 \rightarrow \text{ex}) = 1$, $\text{rank}(1 \rightarrow 2) = i$, $\text{rank}(2 \rightarrow 3) = i$, $\text{rank}(3 \rightarrow 4) = i$, $\text{rank}(4 \rightarrow 5) = k$, $\text{rank}(5 \rightarrow 4) = k$, $\text{rank}(4 \rightarrow 6) = \text{rank}(6 \rightarrow 7) = \text{rank}(7 \rightarrow 1) = i$, and $\text{rank}(2 \rightarrow 8) = \text{rank}(8 \rightarrow 1) = i$. Then we estimate the upper bound invariant for the ranking function i and k by computing the transition bound at the same time. We get $\text{VB}(i, c) = n$ and $\text{VB}(k, c) = m$. While the transition bound for each transition is the same as the upper bound invariant of its ranking function. They are loose and path-insensitive.

Specifically, in Example 9.3.1, the ranking i is reset by n at edge $0 \rightarrow 1$ and $k + m$ at edge $6 \rightarrow 7$ and k is assigned value $i - m$ at edge $3 \rightarrow 4$. i is reset by $k + m$, and k is reset by $i - m$. We compute a variable reset graph, and we use a variable renaming method to unify the variable name of k and i into the same new variable z . Then this new variable z is the ranking function of all edges where k and i were, and z is only reset on the edge $0 \rightarrow 1$ by input variable n . $\text{VB}(n) = n$ because n is an input variable. So we compute the upper bound invariant for z is n , and the upper bound invariant for both i and k is n as well.

12.5 PATH-SENSITIVE REACHABILITY-BOUND ALGORITHM OVERVIEW

Now for a program c , we use $\text{absG}(c)$, its refined program rp , and all the ranking functions to compute a *reachability-bound* for each of its locations. We first introduce some quantities which are required to compute ahead.

As pre-procedures, we first introduce the *loop bound* for every loop rp_l in rp , and the local *path reachability-bound* for each tp w.r.t. its enclosed(rp , tp).

Definition 64 (Loop Bound). *For any program c with it refined program rp , the loop bound for a subprogram, rp_l in rp is an upper bound on the times that rp_l is executed*

from its entry to the exit point with arbitrary initial trace.

We presented two algorithms in Section 12.6 that both compute a sound loop bound for every loop program rp_l in rp .

Definition 65 (Local Path Reachability-bound). *Given program c with its refined program rp and a simple transition path tp in this program, let $rp_l = \text{enclosed}(rp, tp)$ be the innermost loop program in rp where tp is located, then tp 's path local reachability-bound w.r.t. rp_l , is an upper bound on the times that tp is executed when executing program rp_l with any initial trace of c .*

Intuitively, it bounds the execution times of tp when only its innermost loop program is executing. For example in the first interleaving pattern rp_1^1 in Figure 9.3, $4 : \text{repeat}(tp_3)$ is the innermost loop program of tp_3 . So we first will compute $m-n$ as its local path reachability-bound.

In Section 12.6, we give the computation algorithm that computes a sound path local reachability-bound for a simple transition path w.r.t. the innermost loop where the simple transition path is located.

If there isn't a nested loop in a program, we will jump to the Definition 75. Otherwise, we need to compute loop reachability-bound for each tp w.r.t. every level of its outer nested loop.

Definition 66 (Loop Reachability-bound). *For the program with nested loops, given its refined program rp , a simple transition path tp , and an outer loop program rp_l where tp is nested in, $f(rp_l, tp, c)$ is a loop reachability-bound for tp w.r.t. rp_l if and only if for every $\tau \in \mathcal{T}_0^+(c)$, $f(rp_l, tp, c)(\tau)$ greater than or equal to the iteration numbers of rp_l , such that, during these iterations, the innermost loop $\text{enclosed}(tp)$ is entered.*

In Figure 9.3, tp_3 has an outer loop rp_1^1 . Since L_4 will be "entered" only in the

first iteration of rp_1^1 , we aim to compute 1 as the *loop reachability-bound* for tp_3 w.r.t. rp_1^1 .

Our algorithm in Section 12.7 computes a sound *loop reachability-bound* for every simple transition path with respect to each level of its outer loop.

Now we want to bound for each tp the times it is executed globally.

Definition 67 (Path Reachability-bound). *For a program c with its refined program rp and a simple transition path tp in this program, the path reachability-bound, is the upper bound on the times that tp is executed when executing c with arbitrary initial trace.*

In Figure 9.3, since there isn't a nested loop for tp_1 and tp_2 , their local bound $\frac{m}{4}$ is indeed the global *path reachability-bound*. tp_3 is nested in two loops, but the inner loop L_4 is only entered in the first iteration of the outer loop, we aim to compute $\frac{m}{4}$ using the *loop reachability-bound*.

Section 12.8 gives the algorithm that computes a sound *path reachability-bound*, $pathRB(rp, tp, c)$ for each tp in rp . Then we use it to compute for each program point, $l \in LV(c)$ in a program c , its *reachability-bound*, $psRB(l, c)$ by summing up all the path reachability-bounds over all tp that contains l .

Definition 68 (The Reachability-bound Computation). *Given a program c with its abstract transition graph, $absG(c)$ and refined program rp , the reachability-bound of each program point $l \in LV(c)$, $psRB(l, c)$ is computed as follows,*

$$psRB(l, c) = \sum \{pathRB(rp, tp, c) \mid tp \in rp \wedge l \in L_{path}(tp)\}$$

$tp \in rp$ denotes tp is a simple transition path from $PAT\mathcal{H}(absG(c))$.

Theorem 12.5.1 (Soundness). *For every program c and every label l in this program,*

$\llbracket \text{psRB}(l, c) \rrbracket$ is a reachability-bound for l in c .

$$\begin{aligned} & \forall c, c_r \in \mathcal{C}, \text{tp} \in \text{absG}(c), \tau_0 \in \mathcal{T}_0^+(c), \tau_r \in \mathcal{T}_0^+(c_r), \tau \in \mathcal{T}^{+\infty}, l, l' \in \mathcal{L}, \text{rp} . \\ & \text{rp} = \text{REFINE}(\text{Rewrite}(c)) \wedge \text{rp} = \text{Rewrite}(c_r) \wedge \\ & \wedge \left(\langle c_r, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \vee \langle c_r, \tau_0 \rangle \uparrow^\infty \tau_{0++\tau} \right) \implies \llbracket \text{psRB}(l, c) \rrbracket(\tau_0) \geq \text{cnt}(\tau, l) \end{aligned}$$

12.6 LOOP BOUND AND LOCAL PATH REACHABILITY-BOUND COMPUTATION

12.6.1 Loop Bound Computation

There are three methods for computing the loop bound, $BD(\text{repeat}(\text{rp}'), c)$ for every sub program, specifically loop program $\text{repeat}(\text{rp}')$ in a program c 's refined program rp :

1. Estimated by $BD(\text{repeat}(\text{rp}'), c) = \text{BOUND}(\text{repeat}(\text{rp}'))$ from paper Gulwani et al. (2009a), which is precise for simple loop without nested loops but not for nested loops. We are not giving the computation detail in this paper, which can be found in paper Gulwani et al. (2009a).
2. The second method is a *ranking function-based* method in Section 12.6.1.1, which is based on the estimated ranking function from previous stages. This bound is precise when there is only one path in this loop but not for loops with multiple paths.
3. We provide a new loop bound computation method in Section 12.6.1.2 using the upper bound invariant estimation idea from Sinn et al. (2017) and the progress invariant idea from Gulwani et al. (2009a). We compute three abstract states for each ranking function, $\text{rInit}(\text{rp}, c)$, $\text{rFinal}(\text{rp}, c)$, $\text{rNext}(\text{rp}, c)$

and $\text{varGD}(\text{rp}, c)$ as in Definition 69. Then we solve $BD(\text{repeat}(\text{rp}'), c)$ by Definition 70 using SMT solver.

12.6.1.1 Ranking Function-based Loop Bound Computation

We show in this section the *ranking function-based* loop bound computation method. Based on the estimated ranking function from Section 12.4, the $BD(\text{rp}, c)$ for the refined program rp can be estimated by $TB(\hat{e}, c)$ by taking the minimum value over all $\hat{e} \in \text{rp}$. We use $\hat{e} \in \text{rp}$ to denote $\hat{e}$ is an edge on path rp .

$$BD(\text{rp}, c) = \min\{TB(\hat{e}, c) \mid \hat{e} \in \text{rp}\}. \quad (12.1)$$

Example. For the running example in Figure 9.3(a), Equation 12.1 computes $BD(\text{rp}_1^1) = n$ and $BD(\text{rp}_1^2) = n$ for the two interleaving patterns of loop L_1 . But both of their iteration bounds are expected to be $\lfloor \frac{m}{4} \rfloor$. The inaccuracy is resulted by the nature of the path-insensitivity in the ranking function invariant estimation in Definition 63.

12.6.1.2 Proposed Loop Bound Computation

Our approach uses the ranking function invariant to compute a more precise progress invariant inspired from Gulwani et al. (2009a). We first compute three abstract states for each ranking function in different subprograms of a refined program as below.

Definition 69 (Ranking Function Abstract States Computation). *Given a program c with its refined program rp_c and a sub-program rp in rp_c , we compute three abstract states, $\text{rInit}(\text{rp}, c)$, $\text{rFinal}(\text{rp}, c)$ and $\text{rNext}(\text{rp}, c) \in \mathbf{A}_{\text{sc}}(c)$ w.r.t. the ranking functions in this sub program as follows.*

1. We compute the initial execution point and continuation execution points of rp denoted by $\text{absinit}(\text{rp})$ and $\text{absfinal}(\text{rp})$ using standard control flow method.

2. The Initial State, $\text{rInit}(\text{rp}, c)$ is a set of equations $x = e$, where $e \in \mathbf{A}_{\text{sc}}(c)$ is a symbolic expression bounding the initial value of x before executing rp . Each x is a ranking function of a transition edge in this program, and $\text{rInit}(\text{rp}, c)$ contains the equations for every ranking function in rp , $\text{rInit}(\text{rp}, c) \triangleq$

$$\bigcup_{x \in \{\text{rank}(\hat{e}, c) \mid \text{tp} \in \text{rp} \wedge \hat{e} \in \text{tp}\}} \left\{ x = \max_{l_1} \left\{ \text{VB}(y, c) + v \mid \begin{array}{l} (l_1, x' \leq y + v, l_2) \in \text{re}(x, c) \\ \wedge l_1 \leq \text{absinit}(\text{rp}, c) \end{array} \right\} \right\}$$

$\text{rInit}(\text{tp}, c)$ can also be estimated by $\text{INIT}(c, 0, \text{absinit}(\text{rp}))$ in Gulwani et al. (2009a).

3. The Final State, $\text{rFinal}(\text{rp}, c)$ computes the set of minimum values for every ranking functions after executing rp .

$$\bigcup_{x \in \{\text{rank}(\hat{e}, c) \mid \text{tp} \in \text{rp} \wedge \hat{e} \in \text{tp}\}} \left\{ x = \min_{\{l_1 \mid l_1 \geq \text{absfinal}(\text{rp}, c)\}} \max \left\{ v \mid \left((x = v) \wedge \bigwedge_{b \in \mathbf{B}(\text{rp}, c)} \neg b \right) \neq \text{false} \right\} \right\}$$

$\mathbf{B}(\text{rp}, c)$ is the set of all the boolean constraints in rp .

4. The Next State, $\text{rNext}(\text{rp}, c) \in \mathbf{A}_{\text{sc}}(c)$ computes for the ranking functions on rp the amount that each of them is changed after the first execution of rp and before the second visit. This quantity is only required for tp .

$$\text{rNext}(\text{tp}, c) \triangleq \bigcup_{x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}} \left\{ x = \begin{array}{l} \sum_{\hat{e} \in \text{inc}(x, c)} \{v \mid \hat{e} = (l, x' \leq x + v, _) \wedge l \in \text{tp}\} \\ + \max_{l'} \{ \text{VB}(y, c) + v \mid (l, x' \leq y + v, l') \in \text{re}(x, c) \wedge l \in \text{tp} \} \\ - \sum_{\hat{e} \in \text{dec}(x, c)} \{v \mid \hat{e} = (l, x' \leq x - v, _) \wedge l \in \text{tp}\} \end{array} \right\}$$

Now we compute $BD(\text{rp}, c)$ through a mutual recursion procedure involving the computation of another quantity, the *Variable Gradient Decent* as follows.

Definition 70 (Loop Bound Computation). *Given a program c with its refined program rp and a subprogram $\text{rp}' = \text{repeat}(\text{rp}_l) \in \text{rp}$, we compute a loop bound, $BD(\text{repeat}(\text{rp}_l), c)$ through a mutual recursion between $BD(\text{repeat}(\text{rp}_l), c)$ and $\text{varGD}(\text{rp}', c)$. $\text{varGD}(\text{rp}', c)$ is the Variable Gradient Decent—an invariant of the minimum amount decreased in one execution of rp_l for the ranking functions in rp .*

$$BD(\text{repeat}(\text{rp}_l), c) \triangleq$$

$$\left\{ \begin{array}{ll} \max_{x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}} \left\{ \frac{a-b}{\text{varGD}(\text{tp}, c)} \right. & \text{rp}_l = \text{tp} \\ \quad \left. \mid x = a \in \text{rInit}(\text{tp}, c) \wedge x = b \in \text{rFinal}(\text{tp}, c) \right\} & \\ \max_{x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}} \left\{ \frac{\min\{a, a - \text{varGD}(\text{rp}_1, c)\} - b}{\text{varGD}(\text{rp}_1; \text{rp}_2, c)} \right. & \text{rp}_l = \text{rp}_1; \text{rp}_2 \\ \quad \left. \mid x = a \in \text{rInit}(\text{rp}_1, c) \wedge x = b \in \text{rFinal}(\text{rp}_2; \text{rp}_2, c) \right\} & \\ 0 & \text{o.w.,} \end{array} \right.$$

where the variable gradient decent, $\text{varGD}(\text{rp}, c)$ for the ranking functions of rp is computed as follows,

$$\text{varGD}(\text{rp}, c) \triangleq$$

$$\left\{ \begin{array}{ll} BD(\text{rp}, c) \times \text{varGD}(\text{rp}'_l, c) & \text{rp} = \text{repeat}(\text{rp}'_l) \\ \text{varGD}(\text{rp}_1, c) + \text{varGD}(\text{rp}_2, c) & \text{rp} = \text{rp}_1; \text{rp}_2 \\ \min\{v \mid x = v \in \text{rNext}(\text{tp}, c)\} & \text{rp} = \text{tp}. \end{array} \right.$$

We show that Definition 70 computes a sound loop bound for every loop in a refined program below in Lemma 12.6.1 with proof in Appendix B.4.1.

Lemma 12.6.1 (Soundness of Loop Bound). *For every program c with it refined program rp , $BD(\text{rp}, c)$ is a sound upper bound on the iterating times of this program.*

Example. Now with the new approach, we compute a more precise loop bound $BD(rp)$ for each subprogram in Figure 9.3(a). We first compute $rInit = \{k = n - m\}$, $rFinal = \{k = 0\}$ and $rNext = 1$ for subprogram $repeat(tp_3)$ and get $BD(repeat(tp_3)) = n - m$ tighter than n by Equation 12.1. For the first iteration pattern rp_1^1 in Figure 9.3(b), we compute $rInit = \{k = n - m, i = m - 1\}$, $rFinal = \{k = 0, i = 0\}$, $varGD(rp_1^1) = 4$, and get $BD(rp_1^1) = \lfloor \frac{m}{4} \rfloor$ as expected.

For the first iteration pattern rp_1^1 in Figure 9.3(b), we also compute $rInit = \{k = n - m, i = m\}$, $rFinal = \{k = 0, i = 0\}$ and $varGD(rp_1^1) = 4$. So we compute $BD(rp_1^1) = \lfloor \frac{m}{4} \rfloor$ as expected.

Using the BD computed by our new approach, we compute the *path reachability-bound* locally for each tp w.r.t. the **innermost loop**, $enclosed(rp, tp)$ where tp is located.

12.6.2 Path Local Reachability-bound Computation

We first compute $enclosed(rp, tp)$, which is the **innermost loop** inside rp where tp is located. Then by using the loop bound computed by our new approach, we compute the *path local reachability-bound* of the tp in its closest nested loop as follows in Definition 71.

Definition 71 (Path Local Reachability-bound Computation). *Given program c with its refined program rp and a simple transition path tp in this program, let $l : rp_l = enclosed(rp, tp)$ be a sub loop program in rp , then tp 's local reachability-bound w.r.t. $l : rp_l$, $pathLocRB(rp_l, tp, c)$ is computed inductively as follows.*

$$\text{pathLocRB}(\text{rp}_l, \text{tp}, c) \triangleq \begin{cases} 1 & \text{rp} = \text{tp} \\ 0 & \text{rp} = \text{tp}' \neq \text{tp} \\ \text{pathLocRB}(\text{rp}_1, \text{tp}, c) + \text{pathLocRB}(\text{rp}_2, \text{tp}, c) & \text{rp} = \text{rp}_1; \text{rp}_2 \\ \max\{\text{pathLocRB}(\text{rp}_1, \text{tp}, c), \dots, \text{pathLocRB}(\text{rp}_m, \text{tp}, c)\} & \text{rp} = \text{choose}\{\text{rp}_1, \dots, \text{rp}_m\} \\ BD(\text{repeat}(\text{rp}'), c) \times \text{pathLocRB}(\text{rp}', \text{tp}, c) & \text{rp} = \text{repeat}(\text{rp}'). \end{cases}$$

We show the *path local reachability-bound* of the simple transition path tp in a refined program rp is a sound upper bound of its execution times when executing its enclosed loop $\text{enclosed}(\text{rp}, \text{tp})$ in Lemma 12.6.2, with the proof in Appendix B.4.2.

Lemma 12.6.2 (Soundness of the Path Local Reachability-bound). *For any program c with its refined program rp and a simple transition path tp in rp , if $l : \text{rp}_l = \text{enclosed}(\text{rp}, \text{tp})$ is the closest loop where tp is nested in this program, then the execution times of tp when executing the rp_l under initial trace $\tau_l \in \mathcal{T}_0^+(c_l)$ is bounded by $\llbracket \text{pathLocRB}(\text{rp}_l, \text{tp}) \rrbracket(\tau_0)$ with any initial trace $\tau_0 \in \mathcal{T}_0^+(c)$.*

$$\begin{aligned} & \forall c, c_l \in \mathcal{C}, \text{tp} \in \text{absG}(c), \tau_l \in \mathcal{T}_0^+(c_l), \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^{+\infty}, l, l' \in \mathcal{L}, \text{rp} . \\ & \text{rp} = \text{REFINE}(c) \wedge l : \text{rp}_l = \text{enclosed}(\text{rp}, \text{tp}) \wedge \text{rp}_l = \text{Rewrite}(c_l) \\ & \wedge \left(\langle c_l, \tau_l \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{l++\tau} \rangle \vee \langle c_l, \tau_l \rangle \uparrow^\infty \tau_{l++\tau} \right) \\ & \implies \llbracket \text{pathLocRB}(\text{rp}_l, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp})). \end{aligned}$$

Example. Now based on the tighter loop bound, we compute local reachability-bound for every transition path w.r.t. its innermost loop as $\text{pathLocRB}(\text{rp}, \text{tp}_0) = 1$, $\text{pathLocRB}(\text{rp}, \text{tp}_4) = 1$, $\text{pathLocRB}(1 : \text{rp}_1^1, \text{tp}_1) = \frac{m}{4}$, $\text{pathLocRB}(1 : \text{rp}_1^1, \text{tp}_2) = \frac{m}{4}$, $\text{pathLocRB}(1 : \text{rp}_1^1, \text{tp}_4) = \frac{m}{4}$, $\text{pathLocRB}(1 : \text{rp}_1^2, \text{tp}_1) = \frac{m}{4}$, $\text{pathLocRB}(1 : \text{rp}_1^2, \text{tp}_2) = \frac{m}{4}$, $\text{pathLocRB}(1 : \text{rp}_1^2, \text{tp}_4) = \frac{m}{4}$, and $\text{pathLocRB}(4 : \text{repeat}(\text{tp}_3), \text{tp}_3) =$

$n - m$

Specifically, for the program in the first interleaving pattern rp_1^1 in Example 9.3.1, rp_1^1 is the innermost loop for tp_1 , tp_2 and tp_4 , so compute $\text{pathLocRB}(rp_1^1, tp_1, c) = BD(rp_1^1, c) \times \text{pathLocRB}(tp_1; 4 : \text{repeat}(tp_3); tp_2; tp_4, tp_1, c) = BD(rp_1^1, c) \times (1+0) = \lfloor \frac{m}{4} \rfloor$ and the same for tp_2 and tp_4 .

For the program in the first interleaving pattern rp_1^1 in Figure 9.3, rp_1^1 is the innermost loop for tp_1 , tp_2 and tp_4 , so compute $\text{pathLocRB}(rp_1^1, tp_1, c) = BD(rp_1^1, c) \times \text{pathLocRB}(tp_1; 4 : \text{repeat}(tp_3); tp_2; tp_4, tp_1, c) = BD(rp_1^1, c) \times (1 + 0) = \lfloor \frac{m}{4} \rfloor$ and the same for tp_2 and tp_4 .

As discussed in Section 12.5, if there isn't a nested loop, we will skip the Section 12.7 and use this local bound directly as the global *path reachability-bound*, $\text{pathRB}(rp, tp, c)$ as in Section 12.8. In this sense, we have a reachability bound computation subroutine as below for every program point $l \in LV(c)$ by replacing the equation in Definition 68 with the follows.

Definition 72 (Reachability-bound Computation Sub-routine). *Given a program c with its abstract transition graph, $\text{absG}(c)$ and refined program rp , if the program does not have a nested loop, then the reachability-bound of each program point $l \in LV(c)$, $\text{psRB}(l, c)$ is computed as follows,*

$$\text{psRB}(l, c) = \sum \{\text{pathLocRB}(rp, tp, c) \mid tp \in rp \wedge l \in tp\}. \quad (12.2)$$

However, if there are nested loops, by replacing the restriction, rp_l in Definition 71 with the complete program rp , $\text{pathLocRB}(rp, tp, c)$ is not precise w.r.t. the reaching time of tp during the execution of the complete program. Although we can still compute a loose reachability-bound, psRB just using $\text{pathLocRB}(rp, tp, c)$, it is not tight enough. For the motivating example in Figure 9.3 that contains three

levels nested loop, Equation 12.2 computes $\text{pathLocRB}(\text{rp}, \text{tp}_3, c) = \frac{m}{4} \times (n - m)$. It is loose because Equation 12.2 assumes the inner loop L_4 is reached in every iteration of the outer loop L_1 . In this sense, $\text{psRB}(4, c) = \frac{m}{4} \times (n - m)$ is loose as well. While the inner loop L_4 is only reached in the first iteration of the outer loop, so we will compute a more accurate quantity, the *loop reachability-bound* in the next section.

12.7 LOOP REACHABILITY-BOUND COMPUTATION

Similar to the *loop bound* computation, we first compute two abstract states for each simple transition path w.r.t. every level of its outer loops, $\text{lpInit}(\text{rp}, \text{tp}, c)$, and $\text{lpNext}(\text{rp}, \text{tp}, c)$ based on the refined program below.

Definition 73 (Loop Abstract States). *For a program c with its refined program rp_c , a tp and a loop rp in rp_c where tp is located, we compute $\text{lpInit}(\text{rp}, \text{tp}, c)$ and $\text{lpNext}(\text{rp}, \text{tp}, c)$ w.r.t. the ranking functions as follows.*

1. *The loop's initial state $\text{lpInit}(\text{rp}, \text{tp}, c) \in \mathbf{A}_{\text{sc}}(c)$ computes a symbolic expression, which is an upper bound for the initial value of tp 's ranking function before any visit of tp and during the first execution of rp .*

$$\text{lpInit}(\text{rp}, \text{tp}, c) \triangleq \max_{l_1} \left\{ \text{VB}(y, c) + v \mid \begin{array}{l} (l_1, x' \leq y + v, l_2) \in \text{re}(x, c) \\ \wedge \text{absinit}(\text{rp}) \leq l_1 \leq \text{absinit}(\text{tp}) \wedge x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\} \end{array} \right\}$$

2. *The loop next state $\text{lpNext}(\text{rp}, \text{tp}, c) \in \mathbf{A}_{\text{sc}}(c)$ estimates the amount for tp 's ranking function that is modified before the second visit of tp but during the second execution of*

$$\text{rp. lpNext}(\text{rp}, \text{tp}, c) \triangleq \max_{x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}} \left\{ \begin{array}{l} \sum_{\hat{e} \in \text{inc}(x, c)} \{v \mid \hat{e} = (l', x' \leq x + v, _) \wedge l' \in \text{rp} \wedge l' \notin \text{tp}\} \\ + \max_{l_2} \{\text{VB}(y, c) + v \mid (l_1, x' \leq y + v, l_2) \in \text{re}(x, c) \wedge l_1 \in \text{rp} \wedge l_1 \notin \text{tp}\} \\ - \sum_{\hat{e} \in \text{dec}(x, c)} \{v \mid \hat{e} = (l', x' \leq x + v, _) \wedge l' \in \text{rp} \wedge l' \notin \text{tp}\} \\ + BD(\text{enclosed}(\text{tp}), c) \times \text{rNext}(\text{tp}, c) \end{array} \right\}$$

Then we compute the *loop reachability-bound* as below.

Definition 74 (Loop Reachability-bound Computation). *Given a refined program rp and a simple transition path tp in this program, let rp_l be a loop program in rp , then rp_l 's loop reachability-bound w.r.t. tp , $\text{lpRB}(\text{rp}, \text{tp}, c)$ is computed interactively with the abstract loop states as follows.*

$$\text{lpRB}(\text{rp}, \text{tp}, c) \triangleq \max_{x=a \in \text{rFinal}(\text{tp}, c)} \frac{\text{lpInit}(\text{rp}, \text{tp}, c) - a}{\text{lpNext}(\text{rp}, \text{tp}, c)}$$

Definition 74 computes a sound *loop reachability-bound*, and the proof of the soundness is in Appendix B.4.3.

Lemma 12.7.1 (Soundness of the Loop Reachability-Bound). *For any loop $l : \text{rp}$ and a simple transition path tp in a refined program, if $l_t : \text{rp}_t$ is the closest loop enclosing the tp , then the entering times of $l_t : \text{rp}_t$ when executing the $l : \text{rp}$ under initial trace*

$\tau_0 \in \mathcal{T}_0^+(c)$, is bounded by $\llbracket \text{lpRB}(l : \text{rp}, \text{tp}, c) \rrbracket(\tau_0)$.

$$\begin{aligned}
& \forall c, c_l \in \mathcal{C}, \text{tp} \in \text{absG}(c), \tau_0 \in \mathcal{T}_0^+(c_l), \tau \in \mathcal{T}^{+\infty}, l, l' \in \mathcal{L}, \text{rp} \cdot \text{rp} = \text{REFINE}(\text{Rewrite}(c)) \\
& \wedge l_t : \text{rp}_t = \text{enclosed}(\text{rp}, \text{tp}) \wedge \text{rp} = \text{Rewrite}(c_l) \\
& \wedge \left(\langle c_l, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \vee \langle c_l, \tau_0 \rangle \uparrow^\infty \tau_{0++\tau} \right) \\
& \implies \llbracket \text{lpRB}(l : \text{rp}, \text{tp}, c) \rrbracket(\tau_0) \\
& \geq \sum \left\{ \text{lcnt}(\tau, \text{L}_{\text{trace}}(\tau_{l \rightarrow l_t})) \mid \tau_{l \rightarrow l_t} \in \left\{ \tau_{l \rightarrow l_t} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau' \in \mathcal{T}^+, \tau'' \in \mathcal{T}^{+\infty} . \right. \right. \\
& \quad \left. \left. \tau = \tau_{l \rightarrow l_t++\tau''} \wedge \tau_{l \rightarrow l_t} = [(_, l, \text{true})]_{++\tau'} :: (_, l_t, \text{true}) \right. \right. \\
& \quad \left. \left. \wedge \text{cnt}(\tau', l) = \text{cnt}(\tau', l_t) = 0 \right\} \right\}.
\end{aligned}$$

The $\text{lpRB}(l : \text{rp}, \text{tp})$ can also be over-approximated by $I(l, l')$, specifically the function $\text{BOUNDFINDERD}(\text{INIT}(c, i, \text{absinit}(\text{tp})), \text{NEXT}(c, i, \text{absinit}(\text{tp})), V_{\text{in}})$ in paper Gulwani et al. (2009a). $I(l, l')$ over-approximates this value to a large extend by estimating the bound on iteration numbers of l' in one iteration of l .

Example. The running example in Figure 9.3(a) has only one nested loop, so $\text{lpRB}(\text{rp}_1^1, \text{tp}_3, c)$ is the only interesting case. We compute $\text{lpInit}(\text{rp}_1^1, \text{tp}_3, c) = n - m$, $\text{lpNext}(\text{rp}_1^1, \text{tp}_3, c) = n - m - 1$ and then derive $\text{lpRB}(\text{rp}_1^1, \text{tp}_3, c) = 1$ as expected.

We show again here the motivating example from Figure 9.2(a) with three nested loops to show the importance of the *loop reachability-bound* as in Figure 12.2(a). This example is similar to the loop L_4 nested in the second branch in Figure 9.3. w is reset by j in command 5 and i is then reset by w , so L_6 is only executed in the first iteration of loops L_1 and L_3 . Then the total iterations times are $n + m^2 - m \times N$, and the expected *reachability-bound* for location 7 inside the L_6 is N , for locations 4, 5 and 8 between the L_3 and L_6 are $(n - N) \times (m - N)$, and $n - N$ for locations 2 and 9. The interesting case here is that the locations inside the loop L_6 have the

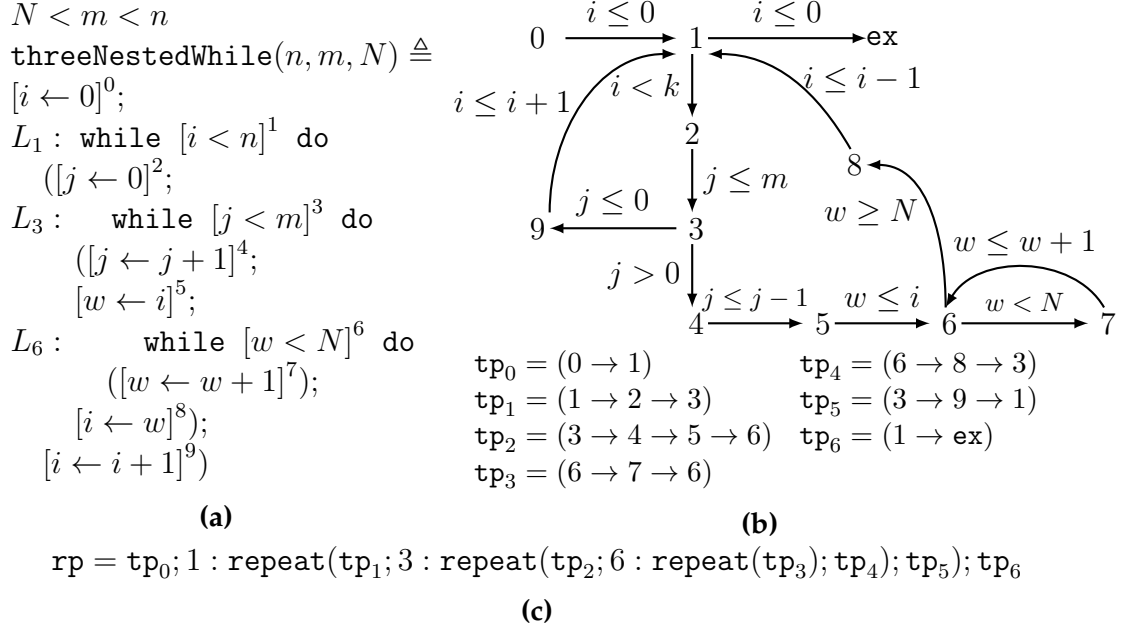


Figure 12.2: (a) An example of three nested loops with related iterator variables. (b) The abstract transition graph, the simple transition paths and loop body. (c) The refined program.

same *reachability-bound* as the local iteration times of L_6 . While for the locations between L_3 and L_6 , their bounds are the product of the inner and outer loop iteration bounds. To compute the precise bound, it is critical to know the numbers of iterations of the outer loops L_3 and L_1 such that, during these iterations, the loop L_6 is “reached”, in the other words, the *loop reachability-bound* of L_6 w.r.t. L_3 and L_1 . Figure 12.2(b) shows its abstract transition graph, and we compute its refined program at the bottom of this graph. rp_1 , rp_3 and rp_6 denote the body of the loop L_1 , L_3 and L_6 respectively. We first compute the $\text{pathLocRB}(\text{rp}_6, \text{tp}_3, c) = N$ for tp_3 w.r.t. its innermost loop. Since tp_3 is nested in two levels of outer loops, L_1 and L_3 , we compute two *loop reachability-bounds* for tp_3 . For $\text{lpRB}(\text{rp}_1, \text{tp}_3, c)$, we compute $\text{lpInit}(\text{rp}_1, \text{tp}_3, c) = 0$, $\text{lpNext}(\text{rp}_1, \text{tp}_3, c) = N + 1$, and $\text{rFinal}(\text{rp}_1, c) = \{i = n, k = N\}$ and obtain $\text{lpRB}(\text{rp}_1, \text{tp}_3, c) = \max\{\frac{n-0}{n-N-1}, \frac{N-0}{N+1}\} = 1$, as expected.

Then for $\text{lpRB}(\text{rp}_3, \text{tp}_3, c)$, we compute $\text{lpInit}(\text{rp}_3, \text{tp}_3, c) = 0$, $\text{lpNext}(\text{rp}_3, \text{tp}_3, c) = N+1$, and $\text{rFinal}(\text{rp}_3, c) = \{i = n, k = N\}$ and obtain $\text{lpRB}(\text{rp}_3, \text{tp}_3, c) = \max\{\frac{n-0}{n-N-1}, \frac{N-0}{N+1}\} = 1$ as well.

12.8 PATH REACHABILITY-BOUND COMPUTATION

Using the *loop reachability-bound* and *path local reachability-bound*, we compute the *path reachability-bound* first in Section 12.8, then the final path-sensitive reachability-bound for each program point in Section 12.5.

For the program c with its refined program rp , we compute for each of its *simple transition paths*, tp the *path reachability-bound*, $\text{pathRB}(\text{rp}, \text{tp})$ during the execution of the program c as in Definition 75.

Definition 75 (Path Reachability-bound Computation). *Given a refined program rp and a simple transition path tp in this program, tp 's reachability-bound, $\text{pathRB}(\text{rp}, \text{tp}, c)$ is computed recursively as follows. The input c is omitted in the following equations for concise given the context is clear.*

$$\left\{ \begin{array}{ll} 1 & \text{rp} = \text{tp} \\ 0 & \text{rp} = \text{tp}' \neq \text{tp} \\ \text{pathRB}(\text{rp}_1, \text{tp}, c) + \text{pathRB}(\text{rp}_2, \text{tp}, c) & \text{rp} = \text{rp}_1; \text{rp}_2 \\ \max\{\text{pathRB}(\text{rp}_1, \text{tp}, c), \dots, \text{pathRB}(\text{rp}_m, \text{tp}, c)\} & \text{rp} = \text{choose}\{\text{rp}_1, \dots, \text{rp}_m\} \\ \text{pathLocRB}(\text{rp}_l, \text{tp}, c), & \\ \quad \text{rp} = l : \text{rp}_l \wedge l : \text{rp}_l = \text{enclosed}(\text{tp}) & \\ \text{lpRB}(\text{rp}_l, \text{tp}, c) \times \max_{l : \text{rp}_l = \text{enclosed}(l' : \text{rp}')} \{\text{pathRB}(l' : \text{rp}', \text{tp}, c)\}, & \\ \quad \text{rp} = l : \text{rp}_l \wedge l : \text{rp}_l \neq \text{enclosed}(\text{tp}) & \end{array} \right.$$

We show the *path reachability-bound* of the simple transition path tp in a refined

program rp is a sound upper bound of its execution times when executing the program in Lemma 12.8.1, with the proof in Appendix B.4.4.

Lemma 12.8.1 (Soundness of the Path Reachability-bound). *For any program with its refined program rp and a simple transition path tp in this program, the execution times of tp when executing the rp under initial trace $\tau_0 \in \mathcal{T}_0^+(c)$ is bounded by $\llbracket \text{pathRB}(\text{rp}, \text{tp}) \rrbracket(\tau_0)$.*

$$\begin{aligned} & \forall c, c_r \in \mathcal{C}, \text{tp} \in \text{absG}(c), \tau_0 \in \mathcal{T}_0^+(c), \tau_r \in \mathcal{T}_0^+(c_r), \tau \in \mathcal{T}^{+\infty}, l \in \mathcal{L}, \text{rp} . \\ & \text{REFINE}(\text{Rewrite}(c)) = \text{rp} \wedge \text{Rewrite}(c_r) = \text{rp} \wedge \\ & \wedge \left(\langle c_r, \tau_r \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++\tau} \rangle \vee \langle c_r, \tau_r \rangle \uparrow^\infty \tau_{r++\tau} \right) \\ & \implies \llbracket \text{pathRB}(\text{rp}, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, \text{L}_{\text{path}}(\text{tp})). \end{aligned}$$

Example. In Example 9.3.1, we computed $\text{lpRB}(\text{rp}_1^1, \text{tp}_3, c) = 1$ and $\text{pathLocRB}(4 : \text{repeat}(\text{tp}_3), \text{tp}_3) = n - m$ for tp_3 in previous stages. Now we obtain $\text{pathRB}(\text{rp}, \text{tp}) = 1 \times (n - m)$ as tight as expected.

In comparison with the substituted computation in Section 71, now we use the *loop reachability-bound* and compute a more precise path *global* reachability-bound for the path tp_3 as $\frac{m}{4} \times 1$. The bounds for the other paths are the same. Then by summing up all the $\text{pathRB}(\text{rp}, \text{tp})$ where each program point l shows up on the tp , we finally compute the *reachability-bound* for every point path-sensitively as $\text{psRB}(0) = \text{psRB}(\text{ex}) = 1$, $\text{psRB}(1) = \frac{m}{2} + 1$, $\text{psRB}(2) = \frac{m}{2}$, $\text{psRB}(3) = \text{psRB}(6) = \text{psRB}(7) = \text{psRB}(8) = \frac{m}{4}$, $\text{psRB}(5) = \frac{m}{4} \times 1$, and $\text{psRB}(4) = \frac{m}{4} + n - m + 1$. As expected, this bound is tight for every program point.

12.9 REACHABILITY-BOUND COMPUTATION

With the global reachability-bound of every simple transition path, now we can sum up all the *path reachability-bounds*, $\text{pathRB}(\text{rp}, \text{tp})$ over the tp which contains

the program point l , and compute the final result – *reachability-bound*, $\text{psRB}(c, l)$ for every program point $l \in \text{LV}(c)$ as in Definition 68. Below we present again the computation and the soundness theorem. The computation below is the same as Definition 68 and the soundness theorem is the same as Theorem 12.5.1 as well.

Definition (Path-Sensitive Reachability-bound Computation). *Given a program c with its Abstract Transition Graph, $\text{absG}(c)$ and refined program rp , the reachability bound of each program point $l \in \text{LV}(c)$, $\text{psRB}(c, l)$ sums up all the path reachability bounds, $\text{pathRB}(\text{rp}, \text{tp})$ over all simple transition paths tp that contains the program point l .*

$$\text{psRB}(c, l) = \sum \{\text{pathRB}(\text{rp}, \text{tp}) \mid \text{tp} \in \text{rp} \wedge l \in \text{tp}\}$$

$l \in \text{tp}$ denotes that the program point l is a vertex on tp and $\text{tp} \in \text{rp}$ denotes tp is a simple transition path of program c .

We prove that $\llbracket \text{psRB}(c, l) \rrbracket$ is a *Reachability-bound* for every program point l in a program c with proof in Appendix B.4.5.

Theorem (Soundness of the Path-sensitive Reachability-bound Estimation). *For every program c and every label l in this program, $\llbracket \text{psRB}(l, c) \rrbracket$ is a reachability-bound for l in c .*

$$\forall c, c_r \in \mathcal{C}, \text{tp} \in \text{absG}(c), \tau_0 \in \mathcal{T}_0^+(c), \tau_r \in \mathcal{T}_0^+(c_r), \tau \in \mathcal{T}^{+\infty}, l, l' \in \mathcal{L}, \text{rp} .$$

$$\text{rp} = \text{REFINE}(\text{Rewrite}(c)) \wedge \text{rp} = \text{Rewrite}(c_r) \wedge$$

$$\wedge \left(\langle c_r, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \vee \langle c_r, \tau_0 \rangle \uparrow^\infty \tau_{0++\tau} \right) \implies \llbracket \text{psRB}(l, c) \rrbracket(\tau_0) \geq \text{cnt}(\tau, l)$$

CHAPTER 13

Example And Evaluation

13.1 EXAMPLES

We show six examples in this section to illustrate the path-sensitive reachability-bound algorithm step by step.

Example 13.1.1 (Nested Loop with Related Iterators). *This example program is the same as the motivating program in Figure 9.2(a) that has three nested loops, and they contain related iterators. The iterators of the inner loop is reset by the outer loop. At the meantime, the inner loop modifies the counters of the outer loop. We illustrate step-by-step here how we compute the expected reachability-bound of each program point.*

1. **The Abstract Transition Graph:** Figure 13.1(b).

2. **Program Refinement**

Simple Transition Paths:

$$\begin{aligned} \text{tp}_0 &= (0 \rightarrow 1) & \text{tp}_1 &= (1 \rightarrow 2 \rightarrow 3) & \text{tp}_2 &= (3 \rightarrow 4 \rightarrow 5 \rightarrow 6) & \text{tp}_3 &= (6 \rightarrow 7 \rightarrow 6) \\ \text{tp}_6 &= (1 \rightarrow \text{ex}) & \text{tp}_4 &= (6 \rightarrow 8 \rightarrow 3) & \text{tp}_5 &= (3 \rightarrow 9 \rightarrow 1) \end{aligned}$$

Refined Program:

$$\text{rp} = \text{tp}_0; 1 : \text{repeat}(\text{tp}_1; 3 : \text{repeat}(\text{tp}_2; 6 : \text{repeat}(\text{tp}_3); \text{tp}_4); \text{tp}_5); \text{tp}_6$$

Define sub loop programs as follows,

$$\begin{aligned} \text{rp}_1 &= \text{repeat}(\text{tp}_1; 3 : \text{repeat}(\text{tp}_2; 6 : \text{repeat}(\text{tp}_3); \text{tp}_4); \text{tp}_5) \\ \text{rp}_3 &= \text{repeat}(\text{tp}_2; 6 : \text{repeat}(\text{tp}_3); \text{tp}_4) \\ \text{rp}_6 &= \text{repeat}(\text{tp}_3) \end{aligned}$$

3. **Path Local Reachability-bound:**

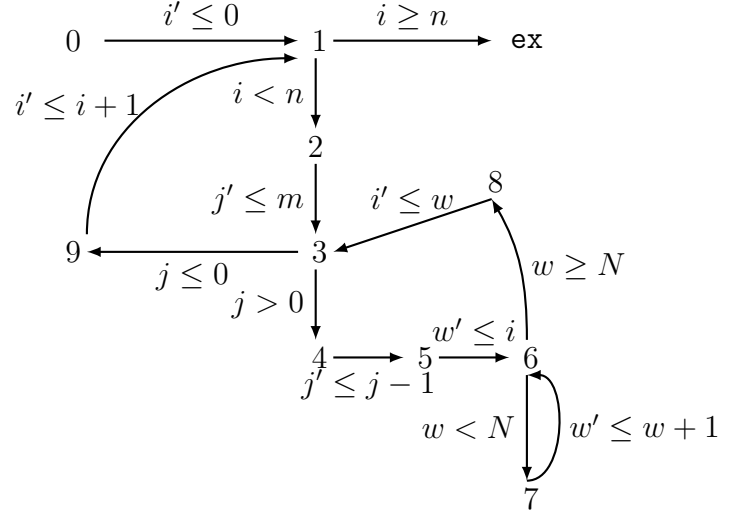
$$\text{pathLocRB}(1 : \text{rp}_1, \text{tp}_1) = n - N \quad \text{pathLocRB}(1 : \text{rp}_1, \text{tp}_5) = n - N$$

```

relatedNestedWhile( $n, m, N$ )  $\triangleq$ 
 $[i \leftarrow 0]^0$ ;
while  $[i < n]^1$  do
  ( $[j \leftarrow m]^2$ ;
  while  $[j > 0]^3$  do
    ( $[j \leftarrow j - 1]^4$ ;  $[w \leftarrow i]^5$ ;
    while  $[w < N]^6$  do
      ( $[w \leftarrow w + 1]^7$ ;
       $[i \leftarrow w]^8$ );
     $[i \leftarrow i + 1]^9$ )
  )

```

(a)



(b)

Figure 13.1: (a) The example of nested loop with related iterators. (b) The abstract transition graph.

$$\text{pathLocRB}(3 : \text{rp}_3, \text{tp}_2) = m \quad \text{pathLocRB}(3 : \text{rp}_3, \text{tp}_4) = m$$

$$\text{pathLocRB}(6 : \text{rp}_6, \text{tp}_3) = N$$

Loop Bounds:

$$BD(\text{tp}_0) = 1 \quad BD(\text{tp}_6) = 1 \quad BD(\text{rp}_6) = N$$

$$BD(\text{rp}_3) = m \quad BD(\text{rp}_1) = n - N$$

4. Loop Reachability-bound:

$$\text{lpRB}(1 : \text{rp}_1, \text{tp}_2) = n - N \quad \text{lpRB}(1 : \text{rp}_1, \text{tp}_4) = n - N$$

$$\text{lpRB}(1 : \text{rp}_1, \text{tp}_3) = 1 \quad \text{lpRB}(3 : \text{rp}_3, \text{tp}_3) = 1$$

5. Path Global Reachability-bound:

$$\text{pathRB}(\text{rp}, \text{tp}_1) = n - N \quad \text{pathRB}(\text{rp}, \text{tp}_5) = n - N$$

$$\text{pathRB}(\text{rp}, \text{tp}_2) = (n - N) \times m \quad \text{pathRB}(\text{rp}, \text{tp}_4) = (n - N) \times m$$

$$\text{pathRB}(\text{rp}, \text{tp}_3) = N$$

$$\text{pathRB}(\text{rp}, \text{tp}_6) = 1 \quad \text{pathRB}(\text{rp}, \text{tp}_0) = 1$$

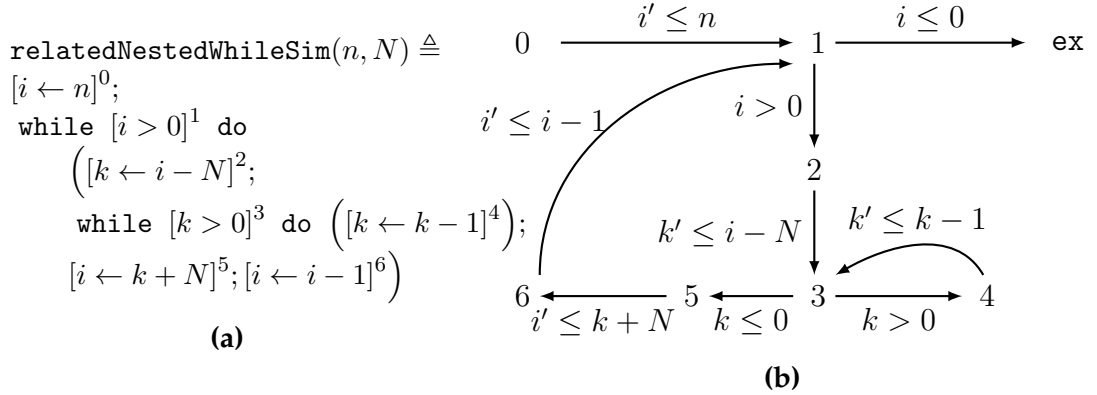


Figure 13.2: (a) The simplified example with nested loop and related iterator. (b) The corresponding abstract transition graph

6. The Reachability-bound:

$$\begin{aligned}
 \text{psRB}(0) &= \text{psRB}(\text{ex}) = 1 & \text{psRB}(1) &= n - N + 1 & \text{psRB}(2) &= \text{psRB}(9) = n - N \\
 \text{psRB}(3) &= n - N + (n - N) \times m \\
 \text{psRB}(4) &= \text{psRB}(5) = \text{psRB}(8) = (n - N) \times m \\
 \text{psRB}(6) &= N + (n - N) \times m & \text{psRB}(7) &= N
 \end{aligned}$$

Example 13.1.2 (Nested Loop with Related Iterator). *This is an example with two nested loops, and the counters of the two loops are modified by each other. It is the simplified version of Example 13.1.1. The iterator of the outer loop is reset at the 5th command, $[i \leftarrow k + N]^5$ by the inner loop iterator k and k is reset by i at the second command, $[k \leftarrow i - N]^2$. The expected reachability-bound for program point 4 in the inner loop is $n - N$. For program points 2, 5 and 6 in the outer loop, their reachability-bounds are expected to be N .*

1. **Abstract Transition Graph** is generated in Figure 13.2(b).

2. Program Refinement

The loop free simple transition paths: $\text{tp}_0 = (0 \rightarrow 1)$ $\text{tp}_1 = (1 \rightarrow 2 \rightarrow 3)$

$$\text{tp}_2 = (3 \rightarrow 4 \rightarrow 3)$$

$$\text{tp}_3 = (3 \rightarrow 5 \rightarrow 6 \rightarrow 1) \quad \text{tp}_4 = (1 \rightarrow \text{ex})$$

Refined Program:

$$\text{rp} = \text{tp}_0; 1 : \text{repeat}(\text{tp}_1; 3 : \text{repeat}(\text{tp}_2); \text{tp}_3); \text{tp}_4$$

Define sub loop programs as follows,

$$\text{rp}_1 = \text{repeat}(\text{tp}_1; 3 : \text{repeat}(\text{tp}_2); \text{tp}_3)$$

3. Path Local Reachability-bound:

$$\text{pathLocRB}(\text{rp}, \text{tp}_0) = 1 \quad \text{pathLocRB}(\text{rp}, \text{tp}_4) = 1$$

$$\text{pathLocRB}(1 : \text{rp}_1, \text{tp}_1) = N$$

$$\text{pathLocRB}(1 : \text{rp}_1, \text{tp}_3) = N$$

$$\text{pathLocRB}(3 : \text{repeat}(\text{tp}_2), \text{tp}_2) = n - N$$

Loop Bounds:

$$BD(\text{tp}_0) = 1 \quad BD(\text{tp}_4) = 1 \quad BD(\text{repeat}(\text{tp}_2)) = n - N$$

$$BD(\text{repeat}(\text{tp}_1; 3 : \text{repeat}(\text{tp}_2); \text{tp}_3)) = N$$

4. Loop Reachability-bound:

$$\text{lpRB}(1 : \text{rp}_1, \text{tp}_1) = N \quad \text{lpRB}(1 : \text{rp}_1, \text{tp}_3) = N$$

$$\text{lpRB}(1 : \text{rp}_1, \text{tp}_2) = 1 \quad \text{lpRB}(3 : \text{repeat}(\text{tp}_2), \text{tp}_2) = n - N$$

5. Path Global Reachability-bound:

$$\text{pathRB}(\text{rp}, \text{tp}_0) = 1 \quad \text{pathRB}(\text{rp}, \text{tp}_1) = N \quad \text{pathRB}(\text{rp}, \text{tp}_2) = 1 \times (n - N)$$

$$\text{pathRB}(\text{rp}, \text{tp}_3) = N \quad \text{pathRB}(\text{rp}, \text{tp}_4) = 1$$

6. The Reachability-bound:

$$\text{psRB}(0) = \text{psRB}(\text{ex}) = 1 \quad \text{psRB}(1) = N + 1 \quad \text{psRB}(4) = n - N$$

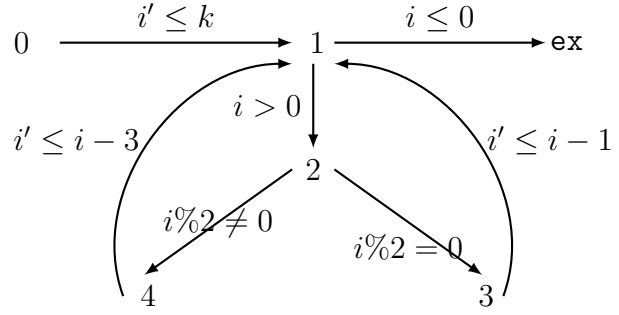
$$\text{psRB}(2) = \text{psRB}(5) = \text{psRB}(6) = N \quad \text{psRB}(\{3\}) = N + n - N = n$$

```

whileOdd( $k$ )  $\triangleq$ 
 $[i \leftarrow k]^0$ ;
while  $[i > 0]^1$  do
( if  $([i \% 2 = 0]^2,$ 
     $[i \leftarrow i - 1]^3,$ 
     $[i \leftarrow i - 3]^4);$  )

```

(a)



(b)

Figure 13.3: (a) The simple loop example with two paths iterating alternatively. (b) The corresponding abstract transition graph.

Example 13.1.3 (While Loop Example with Two Paths Iterating Alternately). *This example program has a loop and two paths in this loop. The two paths are iterating alternatively with each other. Expected reachability-bound for program points on two branches are both $\frac{m}{4}$. For the point that is in the loop but not in the branches are $\frac{m}{2}$.*

1. **The Abstract Transition Graph** is generated in Figure 13.3(b).

2. Program Refinement

The loop free simple transition paths are computed as follows,

$$\text{tp}_0 = (0 \rightarrow 1) \quad \text{tp}_1 = (1 \rightarrow 2 \rightarrow 3 \rightarrow 1)$$

$$\text{tp}_2 = (1 \rightarrow 2 \rightarrow 4 \rightarrow 1) \quad \text{tp}_3 = (1 \rightarrow \text{ex})$$

Refined Program:

$$\text{tp}_0; \text{choose}\{1 : \text{repeat}(\text{tp}_1; \text{tp}_2), 1 : \text{repeat}(\text{tp}_2; \text{tp}_1)\}; \text{tp}_3$$

3. **Path Local Reachability-bound** for every simple transition path tp .

$$\text{pathLocRB}(1, \text{tp}_1) = \frac{n}{4} \quad \text{pathLocRB}(1, \text{tp}_2) = \frac{n}{4}$$

Loop bounds for every sub program:

$$\begin{aligned}
BD(\text{tp}_0) &= 1 & BD(\text{tp}_3) &= 1 & BD(\text{repeat}(\text{tp}_1; \text{tp}_2)) &= \frac{n}{4} \\
BD(\text{tp}_1) &= 1 & BD(\text{tp}_2) &= 1 & BD(\text{repeat}(\text{tp}_2; \text{tp}_1)) &= \frac{n}{4}
\end{aligned}$$

4. Loop Reachability-bound.

There isn't a nested loop, so we do not need to compute loop reachability-bounds in this example.

5. Path Global Reachability-bound for every simple transition path tp

$$\begin{aligned}
\text{pathRB}(\text{tp}_1) &= \frac{n}{4} & \text{pathRB}(\text{tp}_2) &= \frac{n}{4} \\
\text{pathRB}(\text{tp}_0) &= 1 & \text{pathRB}(\text{tp}_3) &= 1
\end{aligned}$$

6. Program Point Path-sensitive Reachability-Bound on every program control location

$$\begin{aligned}
\text{psRB}(0) &= \text{psRB}(\text{ex}) = 1 & \text{psRB}(3) &= \text{psRB}(4) = \frac{n}{4} \\
\text{psRB}(2) &= \frac{n}{2} & \text{psRB}(1) &= \frac{n}{2} + 1
\end{aligned}$$

Example 13.1.4 (Nested Loop with Variable Reset Chain). *This is an example program with two level nested loops. For the inner loop L_6 , the loop iterator k is reset by variable r through variable reset chain. Variable r is reset by the inner loop iterator k after the loop in the command $[r \leftarrow k]^7$ and k is reset by r before going to the loop in the command $[k \leftarrow r]^4$. The expected reachability-bound for program point 6 in the inner loop is $2 \times n$.*

1. **Abstract Transition Graph** is generated in Figure 13.4(b).

2. **Program Refinement** The loop free simple transition paths:

$$\begin{aligned}
\text{tp}_0 &= (0 \rightarrow 1 \rightarrow 2) & \text{tp}_1 &= (2 \rightarrow 3 \rightarrow 4 \rightarrow 5) & \text{tp}_2 &= (5 \rightarrow 6 \rightarrow 5) \\
\text{tp}_3 &= (5 \rightarrow 7 \rightarrow 8 \rightarrow 2) & \text{tp}_4 &= (2 \rightarrow \text{ex})
\end{aligned}$$

Refined Program:

$$\text{rp} = \text{tp}_0; 2 : \text{repeat}(\text{tp}_1; 5 : \text{repeat}(\text{tp}_2); \text{tp}_3); \text{tp}_4$$

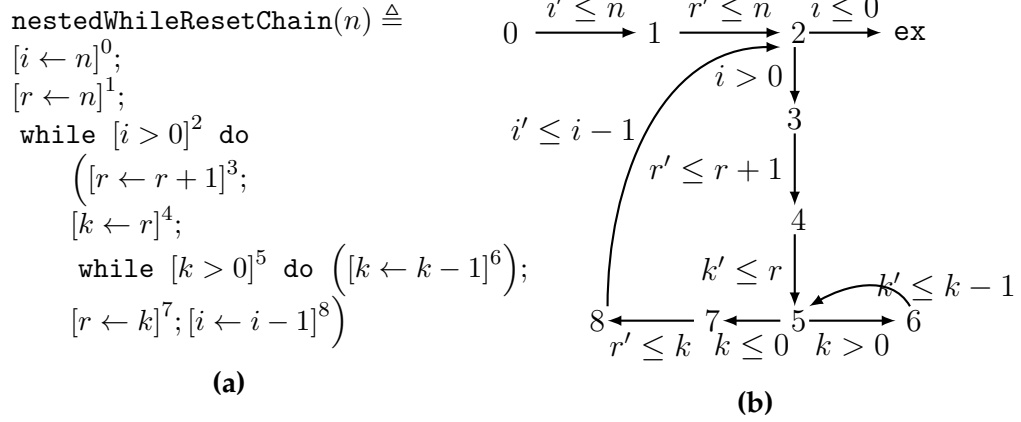


Figure 13.4: (a) The simplified example of nested loop with related iterator. (b) The corresponding abstract transition graph

Define the sub-loop program as follows,

`rp2 = repeat(tp1; 5 : repeat(tp2); tp3)`

3. Path Local Reachability-bound:

`pathLocRB(rp, tp0) = 1` `pathLocRB(rp, tp4) = 1`

`pathLocRB(2 : rp2, tp1) = n` `pathLocRB(2 : rp2, tp3) = n`

`pathLocRB(5 : repeat(tp2), tp2) = n + 1`

Loop Bounds:

`BD(tp0) = 1` `BD(tp4) = 1` `BD(repeat(tp2)) = n + 1`

`BD(repeat(tp1; 5 : repeat(tp2); tp3)) = n`

4. Loop Reachability-bound:

`lpRB(2 : rp2, tp2) =  $\frac{(n+1)}{n}$`

5. Path Global Reachability-bound:

`pathRB(rp, tp0) = 1` `pathRB(rp, tp1) = n` `pathRB(rp, tp2) =  $\frac{(n+1)^2}{n}$`

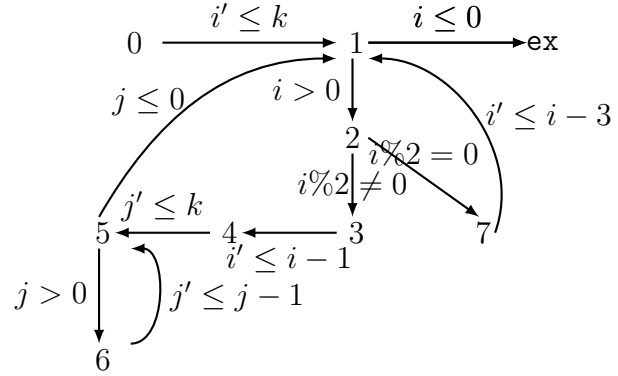
`pathRB(rp, tp3) = n` `pathRB(rp, tp4) = 1`

```

nestedWhileOdd( $k$ )  $\triangleq$ 
 $[i \leftarrow k]^0$ ;
while  $[i > 0]^1$  do
  ( if  $([i \% 2 = 0]^2$ ,
     $[i \leftarrow i - 1]^3$ ;  $[j \leftarrow k]^4$ ;
    while  $[j > 0]^5$  do
      ( $[j \leftarrow j - 1]^6$ ),
       $[i \leftarrow i - 3]^7$ ); )

```

(a)



(b)

Figure 13.5: (a) The program of the two paths loop with a nested loop in one path. (b) The corresponding abstract transition graph

6. The Reachability-bound:

$$\text{psRB}(0) = \text{psRB}(1) = \text{psRB}(\text{ex}) = 1 \quad \text{psRB}(2) = n + 1$$

$$\text{psRB}(3) = \text{psRB}(4) = \text{psRB}(7) = \text{psRB}(8) = n$$

$$\text{psRB}(6) = \frac{(n+1)^2}{n} \quad \text{psRB}(\{5\}) = n + \frac{(n+1)^2}{n}$$

Example 13.1.5 (Example with Two Paths Iterating Alternatively and Nested Loop in One Path). *This example program is similar to our running example in the Figure 9.3. It has two level nested loops, and the outer level loop has two paths and the nested loop is in one path. Different from the running example, the iterator of the inner loop does not interface with the outer loop iterator. In this sense, the reachability-bound of the program point 6 in the inner loop is $\frac{k^2}{4}$.*

1. **The Abstract Transition Graph** is generated in Figure 13.5(b).

2. Program Refinement

The loop free simple transition paths are computed as follows,

$$\text{tp}_0 = (0 \rightarrow 1) \quad \text{tp}_1 = (1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5) \quad \text{tp}_2 = (5 \rightarrow 6 \rightarrow 5)$$

$$\text{tp}_3 = (1 \rightarrow 7 \rightarrow 2 \rightarrow 1) \quad \text{tp}_4 = (1 \rightarrow \text{ex}) \quad \text{tp}_5 = (5 \rightarrow 1)$$

Refined Program:

$$\text{tp}_0; \text{choose} \left\{ \begin{array}{l} 1 : \text{repeat}(\text{tp}_1; 5 : \text{repeat}(\text{tp}_2); \text{tp}_3), \\ 1 : \text{repeat}(\text{tp}_3; \text{tp}_1; 5 : \text{repeat}(\text{tp}_2); \text{tp}_5) \end{array} \right\}; \text{tp}_4$$

Define the sub-loop program as follows,

$$\text{rp}_1^1 = \text{repeat}(\text{tp}_1; 5 : \text{repeat}(\text{tp}_2); \text{tp}_3)$$

$$\text{rp}_1^2 = \text{repeat}(\text{tp}_3; \text{tp}_1; 5 : \text{repeat}(\text{tp}_2); \text{tp}_5)$$

3. Path Local Reachability-bound:

$$\text{pathLocRB}(\text{rp}, \text{tp}_0) = 1$$

$$\text{pathLocRB}(\text{rp}, \text{tp}_4) = 1$$

$$\text{pathLocRB}(1 : \text{rp}_1^1, \text{tp}_1) = \frac{k}{4}$$

$$\text{pathLocRB}(1 : \text{rp}_1^1, \text{tp}_3) = \frac{k}{4}$$

$$\text{pathLocRB}(1 : \text{rp}_1^1, \text{tp}_5) = \frac{k}{4}$$

$$\text{pathLocRB}(1 : \text{rp}_1^2, \text{tp}_1) = \frac{k}{4}$$

$$\text{pathLocRB}(1 : \text{rp}_1^2, \text{tp}_3) = \frac{k}{4}$$

$$\text{pathLocRB}(1 : \text{rp}_1^2, \text{tp}_5) = \frac{k}{4}$$

$$\text{pathLocRB}(5 : \text{repeat}(\text{tp}_2), \text{tp}_2) = k$$

Loop Bounds:

$$BD(\text{tp}_0) = 1 \quad BD(\text{tp}_4) = 1 \quad BD(\text{repeat}(\text{tp}_2)) = k$$

$$BD(\text{rp}_1^1) = \frac{k}{4} \quad BD(\text{rp}_1^2) = \frac{k}{4}$$

4. Loop Reachability-bound:

$$\text{lpRB}(1 : \text{rp}_1^1, \text{tp}_2) = \frac{k}{4} \quad \text{lpRB}(1 : \text{rp}_1^2, \text{tp}_2) = \frac{k}{4}$$

5. Path Global Reachability-bound:

$$\text{pathRB}(\text{rp}, \text{tp}_0) = 1 \quad \text{pathRB}(\text{rp}, \text{tp}_1) = \frac{k}{4} \quad \text{pathRB}(\text{rp}, \text{tp}_2) = \frac{k}{4} \times k$$

$$\text{pathRB}(\text{rp}, \text{tp}_3) = \frac{k}{4} \quad \text{pathRB}(\text{rp}, \text{tp}_5) = \frac{k}{4} \quad \text{pathRB}(\text{rp}, \text{tp}_4) = 1$$

```

relatedNestedWhileOdd( $n, m$ )  $\triangleq$ 
 $[i \leftarrow n]^0$ ;
while  $[i > 0]^1$  do
  ( if  $[i \% 2 = 0]^2$ ,
     $[k \leftarrow i - m]^3$ ;
    while  $[k > 0]^4$  do  $([j \leftarrow j - 1]^5)$ ;
     $[i \leftarrow k + m]^6$ ;  $[i \leftarrow i - 1]^7$ ,
     $[i \leftarrow i - 3]^8$ ); )

```

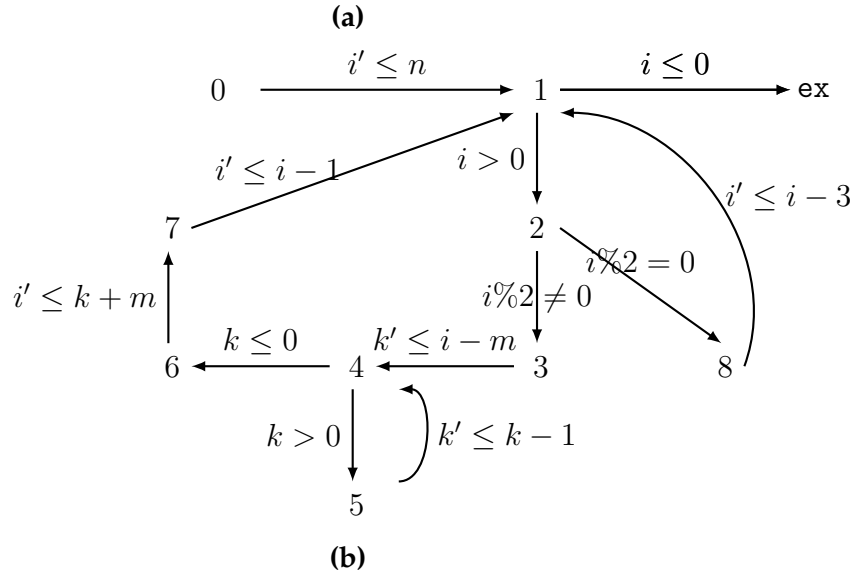


Figure 13.6: (a) The multiple-path loop program with a nested loop and interfered iterators. (b) The corresponding abstract transition graph

6. The Reachability-bound:

$$\text{psRB}(0) = \text{psRB}(\text{ex}) = 1 \quad \text{psRB}(1) = \frac{k}{2} + 1 \quad \text{psRB}(2) = \frac{k}{2}$$

$$\text{psRB}(3) = \text{psRB}(4) = \text{psRB}(7) = \frac{k}{4} \quad \text{psRB}(6) = \frac{k}{4} \times k \quad \text{psRB}(5) = \frac{k}{4} \times (k + 1)$$

Example 13.1.6 (Example with Two Paths Iterating Alternatively and Nested Loop with Related Counters). *This example is the same as our running example, we give the details of every computation results from every sub-step below.*

1. The Abstract Transition Graph is generated in Figure 13.6(b).

2. Program Refinement

The loop free simple transition paths are computed as follows,

$$\begin{aligned} \text{tp}_0 &= (0 \rightarrow 1) & \text{tp}_1 &= (1 \rightarrow 2 \rightarrow 3 \rightarrow 4) & \text{tp}_2 &= (4 \rightarrow 6 \rightarrow 7 \rightarrow 1) \\ \text{tp}_3 &= (4 \rightarrow 5 \rightarrow 4) & \text{tp}_4 &= (1 \rightarrow 2 \rightarrow 8 \rightarrow 1) & \text{tp}_5 &= (1 \rightarrow \text{ex}) \end{aligned}$$

Refined Program:

$$\text{tp}_0; \text{choose} \left\{ \begin{array}{l} 1 : \text{repeat}(\text{tp}_1; 4 : \text{repeat}(\text{tp}_3); \text{tp}_2; \text{tp}_4), \\ 1 : \text{repeat}(\text{tp}_4; \text{tp}_1; 4 : \text{repeat}(\text{tp}_3); \text{tp}_2) \end{array} \right\}; \text{tp}_5$$

Define the sub-loop program as follows,

$$\begin{aligned} \text{rp}_1^1 &= \text{repeat}(\text{tp}_1; 4 : \text{repeat}(\text{tp}_3); \text{tp}_2; \text{tp}_4) \\ \text{rp}_1^2 &= \text{repeat}(\text{tp}_4; \text{tp}_1; 4 : \text{repeat}(\text{tp}_3); \text{tp}_2) \end{aligned}$$

3. Path Local Reachability-bound:

$$\text{pathLocRB}(\text{rp}, \text{tp}_0) = 1$$

$$\text{pathLocRB}(\text{rp}, \text{tp}_4) = 1$$

$$\text{pathLocRB}(1 : \text{rp}_1^1, \text{tp}_1) = \frac{m}{4}$$

$$\text{pathLocRB}(1 : \text{rp}_1^1, \text{tp}_2) = \frac{m}{4}$$

$$\text{pathLocRB}(1 : \text{rp}_1^1, \text{tp}_4) = \frac{m}{4}$$

$$\text{pathLocRB}(1 : \text{rp}_1^2, \text{tp}_1) = \frac{m}{4}$$

$$\text{pathLocRB}(1 : \text{rp}_1^2, \text{tp}_2) = \frac{m}{4}$$

$$\text{pathLocRB}(1 : \text{rp}_1^2, \text{tp}_4) = \frac{m}{4}$$

$$\text{pathLocRB}(4 : \text{repeat}(\text{tp}_3), \text{tp}_3) = n - m$$

Loop Bounds:

$$BD(\text{tp}_0) = 1 \quad BD(\text{tp}_5) = 1 \quad BD(\text{repeat}(\text{tp}_3)) = n - m$$

$$BD(\text{rp}_1^1) = \frac{m}{4} \quad BD(\text{rp}_1^2) = \frac{m}{4}$$

4. Loop Reachability-bound:

$$\text{lpRB}(1 : \text{rp}_1^1, \text{tp}_3) = 1 \quad \text{lpRB}(1 : \text{rp}_1^2, \text{tp}_3) = 1$$

5. Path Global Reachability-bound:

$$\begin{aligned} \text{pathRB}(\text{rp}, \text{tp}_0) &= 1 & \text{pathRB}(\text{rp}, \text{tp}_1) &= \frac{m}{4} & \text{pathRB}(\text{rp}, \text{tp}_2) &= \frac{m}{4} \\ \text{pathRB}(\text{rp}, \text{tp}_3) &= n - m \times 1 & \text{pathRB}(\text{rp}, \text{tp}_4) &= \frac{m}{4} & \text{pathRB}(\text{rp}, \text{tp}_5) &= 1 \end{aligned}$$

6. The Reachability-bound:

$$\begin{aligned} \text{psRB}(0) &= \text{psRB}(\text{ex}) = 1 & \text{psRB}(1) &= \frac{m}{2} + 1 & \text{psRB}(2) &= \frac{m}{2} \\ \text{psRB}(3) &= \text{psRB}(6) = \text{psRB}(7) = \text{psRB}(8) = \frac{m}{4} & \text{psRB}(5) &= n - m \\ \text{psRB}(4) &= \frac{m}{4} + n - m \end{aligned}$$

13.2 EMPIRICAL EVALUATION

Set up We implemented a prototype `psRB` that takes a program as input and outputs the reachability-bound on each program point. The program abstraction is implemented in OCaml and the program refinement, ranking function estimation, and the path-sensitive reachability-bound algorithm are implemented in Python. We evaluate the prototype on 290 programs: 20 hand-crafted programs and 270 programs extracted from different sets of benchmarks. These programs include 37 `c` programs from the testing set of the [LoopusJAR](#) Benchmark (2017) tool, 37 `java` programs from the testing set of the tool [TianhanLu](#) (2021); Lu et al. (2021) and 118 `c` programs from the testing set of [Icra](#) Benchmark (2020); Kincaid et al. (2019); Cyphert et al. (2019), and 110 representative `c` programs from the testing sets of 7 other tools¹. We translate these programs into our syntax. For comparison, we

¹ABC Benchmark (2009): 15, C4B Benchmark (2015): 15, KoAT Benchmark (2018): 3, Loopus Benchmark (2017): 10, Rank10 Benchmark (2010): 3, SPEED Gulwani et al. (2009a); Gulwani & Zuleger (2010); Gulwani et al. (2009b): 27, WTC Benchmark (2017): 37

also report the results of four bound analysis tools, [LoopusJAR](#) Sinn et al. (2017), [CoFloCo](#) Benchmark (2021), Lu (2021) and our own SPEED Gulwani et al. (2009a) implementation.

When comparing with tools for worst-case complexity analysis we aim to answer the following two questions: is our prototype able to compute precise reachability-bound for different program points (*Effectiveness*)? If so, can we also have better overall complexity estimation (*Accuracy*)?

Effectiveness We summarize the results of the effectiveness evaluation in Table 13.1. On each row, a theoretical worst-case complexity is listed in the 2nd column and the number of programs (P. #) with this particular complexity is given in the 3rd column. In the “worst bound (P. #)” column, each row provides the number of programs that the corresponding tool identify of the corresponding “theory worst” complexity. For example, the number 1 in the column of our tool for the row $O(1)$ says that our tool infer the right complexity for only one of the programs, for the other three programs it infers a higher complexity. To show *effectiveness*, we present in the “reachability-bound (L. #)” column our results w.r.t. the program locations. Each sub-column is the number of locations (L. #) with certain reachability-bound. These locations in the same row are from the programs of the same worst-case complexity. The results show that although these locations are from the programs of the same worst-case complexity, they have significantly different reachability-bounds. This is because of the existence of multipaths loops, where locations on different paths have different reachability-bound. We identify a number of loop patterns with large worst-case complexity but only reachability-bounds of $O(1)$ on points inside the loop. For example in the row of $O(n)$ worst complexity in Icra benchmark, we observe that almost $\frac{1}{3}$ of the program points

inside the loops are evaluated only once. Given this, if we use the worst-case bound to estimate the reachability-bounds on different locations, these locations that are only evaluated once will be over-approximated significantly. While our tool computes for almost all the multipath loops precise *reachability-bounds* on locations in different paths. Even for the unbounded loops, we can still compute finite *reachability-bounds* on certain program points that are not involved with the unbounded loop paths. None of other tools compute bounds for these programs.

Table 13.1: The Effectiveness Evaluation Table of pSRB

Bench- marks	theory worst	P. #	L. #	reachability-bound (L. #)						worst bound (P. #)				
				1	n	n^2	n^3	n^4	∞	pSRB	Loopus	Cofloco	SPEED	Lu (2021)
Represen- tatives	$O(1)$	4	82	32	0	0	0	0	0	1	2	3	2	1
	$O(n)$	59	373	112	261	0	0	0	0	48	51	45	46	40
	$O(n^2)$	31	414	70	191	153	0	0	0	37	29	34	37	49
	$n \log(n)$	6	81	0	0	0	0	0	0	0	0	0	0	0
	$O(n^3)$	2	91	6	16	13	13	0	0	4	3	2	5	7
	$O(n^4)$	3	36	4	9	9	12	11	0	4	4	3	5	5
Loopus	$O(1)$	1	154	0	0	0	0	0	0	0	1	0	0	0
	$O(n)$	13	176	49	105	0	0	0	0	12	13	14	14	11
	$O(n^2)$	4	64	4	13	15	0	0	0	3	2	5	2	6
	$O(n^3)$	4	193	2	7	10	4	0	0	2	1	2	2	3
	$O(n^4)$	1	21	1	3	6	8	2	0	1	1	1	1	0
Icra	$O(1)$	4	93	41	0	0	0	0	0	1	3	2	2	0
	$\log(n)$	2	16	0	0	0	0	0	0	0	0	0	0	0
	$O(n)$	79	988	267	698	0	0	0	0	78	80	82	78	77
	$O(n^2)$	12	151	26	110	26	0	0	0	18	14	11	16	17
	$O(n^3)$	1	9	1	3	3	2	0	0	1	1	4	2	4
	∞	20	295	50	0	0	0	0	245	9	0	0	0	0
Tianhan	$O(n)$	35	188	68	120	0	0	0	0	35	35	35	35	35
	$O(n^2)$	2	12	4	8	4	0	0	0	2	2	2	2	2

Accuracy Based on the worst-case bound and reachability-bound, we present the accuracy evaluation result in Table 13.2. In the last column we show the “overall complexity bound” estimation. In the 1st sub-column, formula in each row is the sum of different reachability-bounds times the L. # that has certain reachability-bound. For comparison, we show the worst-case bound times $\sum(L.\#)$ in the 2nd sub-column by other tools. Given the precise reachability-bounds on every pro-

gram locations, we can compute tighter overall complexity bound than the best worst-case analysis tools.

Table 13.2: The Accuracy Evaluation Table of pSRB

Bench- marks	theory worst	P. #	L. #	overall complexity bound	
				pSRB	worst
Representatives	$O(1)$	4	82	32	64
	$O(n)$	59	373	$112 + 261n$	$373n$
	$O(n^2)$	31	414	$70 + 191n + 153n^2$	$414n^2$
	$n \log(n)$	6	81	0	0
	$O(n^3)$	2	91	$6 + 16n + 13n^2 + 13n^3$	$48n^3$
	$O(n^4)$	3	36	$4 + 9n + 9n^2 + 12n^3 + 11n^4$	$45n^4$
Loopus	$O(1)$	1	154	0	-
	$O(n)$	13	176	$49 + 105n$	$162n$
	$O(n^2)$	4	64	$4 + 13n + 15n^2$	$32n^2$
	$O(n^3)$	4	193	$2 + 7n + 10n^2 + 4n^3$	$23n^3$
	$O(n^4)$	1	21	$1 + 3n + 6n^2 + 8n^3 + 2n^4$	$20n^4$
Icra	$O(1)$	4	93	6	64
	$\log(n)$	2	16	0	0
	$O(n)$	79	988	$267 + 698n$	$965n$
	$O(n^2)$	12	151	$26 + 110n + 26n^2$	$162n^2$
	$O(n^3)$	1	9	$1 + 3n + 3n^2 + 2n^3$	$9n^3$
	∞	20	295	50	∞
Tianhan	$O(n)$	35	188	$68 + 120n$	$188n$
	$O(n^2)$	2	12	$4 + 8n + 4n^2$	$16n^2$

Performance The performance evaluation is in Table 13.3. The columns from 2 to 6 are the numbers of the programs, program with multipath loops, program with nested loops, and program with bounded loops and the lines of code in each benchmark set respectively. The 7 to 9 columns are our success rate, the number of programs we failed and time outs. The last column is the total runtime w.r.t. the programs that we successfully computed bounds. We are not doing well in terms of efficiency. Because of the possible non-terminating in program refining and the ranking function estimating, our time outs are higher than other tools. Since our prototype consists of the implementations in OCaml and Python, the inter-procedures calls also reduce the efficiency.

Table 13.3: The Performance Evaluation Table of psRB

Benchmarks	# P	M.P.L #	Nested Loop #	Bounded #	L.O.C	Success Rate	Failed	Time Outs	Total Runtime
Representatives	110	53	52	107	954	85.5%	4	12	7min42sec
Challenges	23	20	20	20	454	78.2%	1	4	12min39sec
Icra	118	111	21	98	1434	90.7%	1	10	4min48sec
Tianhan	37	2	3	37	204	100.0%	0	0	1min03sec

CHAPTER 14

Related Works

Program Abstraction. The program abstraction techniques are widely used in program analysis area to precisely summarize the key property of the program. In analyzing the loop bounds, the unified lattice model by Cousot & Halbwachs (1978), polyhedra by Cousot & Cousot (1977) and octagons by Miné (2006) are used in Gulwani & Zuleger (2010) for summarize the loop transition and generating the transition system. But none of the three abstraction models can represent the if guard in multipath loop, and solving the transition system using these abstraction models is low-efficient. The wedge abstract developed by Kincaid et al. (2018) for generating the non-linear loop invariant computing the reachability-bound can also be used. While it only works well for the specific targeting problem, and still low-efficient or inaccuracy in summarizing the multipath loops and solving the path-sensitive reachability-bound problem. Compare to them, we choose to use the difference constrain based program abstraction model combined with boolean expression abstraction to generate our abstract transition graph. It can represent the program loops path-sensitively more is more accurate than the state-of-art program abstraction models mentioned above. While our program abstraction model is based on the difference constrain model in Sinn et al. (2017) and the size change abstraction in Sinn et al. (2014), which is more efficient comparing to the models above. The efficiency also benefit from our semantics model, which is inspired from Cousot (2019a) and Cousot (2019b) and has similar intuition with the program model in Sinn et al. (2017).

Program Cost Analysis. As already introduced in the introduction section, many works in cost analysis area focus on analyzing the loop iteration bound. Though our motivation is mainly inspired from the SPEED tool in paper Gulwani & Zuleger (2010), they only infer overall loop bounds path-insensitively. We also implement part of the algorithm of Loopus based on the transition bound algorithm by Sinn et al. (2017) in terms of program abstraction and variable invariant generation, and the path refinement algorithm from paper Gulwani et al. (2009a), but both of them suffer from the same weakness as Gulwani & Zuleger (2010). The works in analyzing the program complexity including Gustafsson et al. (2005); Humenberger et al. (2018) only focus on estimating the overall complexity by inferring the bounds on the loop iteration numbers. Similarly, the algorithm of computing a program’s worst-case resource cost such as Albert et al. (2008); Alias et al. (2010) reason the worst-case running time and resource cost of the program’s entire execution as well. For the same reason, the tools CofloCo from works in Flores-Montoya (2017); Flores-Montoya & Hähnle (2014); Flores-Montoya (2016), KoAT by Brockschmidt et al. (2016, 2014); Falke et al. (2012, 2011), and the amortized algorithm in Lu et al. (2021) which we evaluated over our benchmarks all give only the entire loop bound path-insensitively.

Crary & Weirich (2000) presents a type system for the certification of resource bounds (once they are provided by the programmer). In contrast, we infer bounds for different program location. Jost et al. (2010) use linear programming to infer bounds for functional programs, but they are restricted to computing only linear bounds. Other techniques that are based on type system (e.g. Çiçek et al. (2017); Rajani et al. (2021)), the cost analysis via effect systems (e.g. Çiçek et al. (2017); Radicek et al. (2018); Qu et al. (2019)), and Hoare logic (e.g. Carbonneaux et al.

(2015)) give the approximated upper bound of the program’s total resource cost as well but still failed to consider the path sensitivity and the resource cost on different program points.

Loop Summarization and Termination Analysis. There are numerous works studying the loop termination, invariant generation. Algorithms and techniques in this area are closely related to the bound analysis and loop path refinement. For example, the termination analysis through ranking functions originated from paper Bradley et al. (2005) and followed up papers such as Falke et al. (2011, 2012); Cook et al. (2013); Zuleger (2018) are widely developed into bound analysis including Alias et al. (2010); Sinn et al. (2014); Brockschmidt et al. (2016, 2014). Loop summarization and invariant generate through path refinement Manolios & Vroon (2006); Balakrishnan et al. (2009); Sharma et al. (2011); Flores-Montoya & Hähnle (2014); Humenberger et al. (2018); Cyphert et al. (2019) techniques are also extended into precise resource cost and loop bound estimation based on multiple loop path refinement e.g. Gulwani et al. (2009a); Zuleger et al. (2011). Kincaid et al. (2019) introduce a precise closed-form loop summarization techniques which can help to improve the accuracy of the path refinement, as well as the non-linear loop summarization techniques in Kincaid et al. (2018) and the invariant generating algorithm considering recurrence in Breck et al. (2020). They are all heavy compare to our light-weight path refinement implementation adapted from Gulwani et al. (2009a). The contextualization path refinement algorithm, based on the loop termination techniques in Manolios & Vroon (2006) are used in both Zuleger et al. (2011); Sinn et al. (2014). While this refinement only computes the multiple paths interleaving for one iteration, which is loose comparing to our path refinement implementation limited to at most 100 iterations.

PART III EPILOGUE

We review the works on studying two *quantitative-reachability* properties in this dissertation in this part, and the future directions are also discussed.

CHAPTER 15

Conclusion and Future Works

This dissertation presents the works on studying two conceptual *quantitative-reachability* properties from the principles of programming language in two parts respectively. We formalize the two properties through specific programming language models and then develop new program analysis algorithms to best exploit the benefit. We also take one step in implementing these analysis frameworks and evaluate them over real-world benchmarks showing its effectiveness potential applications beyond the program analysis area. Below we conclude the two parts in detail and then present the plans for the future works in Section [15.1.2](#) and [15.2.2](#) respectively for the two parts.

15.1 PROGRAM ADAPTIVITY ANALYSIS

15.1.1 Conclusion

In **PART i**, we presented **AdaptFun**, a program analysis useful to provide an upper bound on the adaptivity of a data analysis, as well as on the total number of queries asked. This estimation can help data analysts to control the generalization errors of their analyses by choosing different algorithmic techniques based on the adaptivity. Besides, a key contribution of our works is the formalization of the notion of adaptivity for adaptive data analysis. We showed the applicability of our approach by implementing and experimentally evaluating our program analysis.

15.1.2 Future Works

We would focus on two aspects in the future, the potential practical applications of **AdaptFun** and how to improve the accuracy of **AdaptFun**.

15.1.2.1 Practical Applications

We plan to investigate the potential integration of **AdaptFun** in an adaptive data analysis framework like Guess and check by Rogers et al. (2020). As we discussed, this framework is designed to support adaptive data analyses with limited generalization error. As our experiments show, this framework could benefit from the information provided by **AdaptFun** to provide a more precise estimation and improved confidence intervals.

15.1.2.2 Accuracy

Another direction we will explore is to make the upper bounds provided by **AdaptFun** are more precise by integrating our algorithm with a path-sensitive approach. Specifically, we plan to improve the accuracy from the following two aspects.

The Adaptivity Definition. The adaptivity definition we give in Definition 23 is not precise enough with respect to the intuitive *adaptivity* notion. There are some cases in which the definition over-approximate the intuitive *adaptivity* of the programs. The Example 5.4.2 in Section 7.1 shows one example of this over-approximation. In this example, the adaptivity by semantics-based definition in Section 5 is $\lambda\tau_0 . \rho(\tau_0)k$, i.e., the initial variable of the input variable. However, the intuitive *adaptivity* is only 2 given whatever initial trace, i.e., $\lambda\tau_0 . 2$. This over-approximation is resulted from the irrelevance between the weight and the *variable*

may-dependency relation. Specifically the irrelevance between the vertex weight and the edge on the *semantics-based dependency* graph. When the weight of a vertex is counted from the execution trace that is different from the witness traces used in the *variable may-dependency* relation of that vertex, there is an inaccuracy.

Based on the *adaptivity* formalization in Section 5, the improvement on the accuracy of *adaptivity* definition is planned to develop in the following steps.

1. In the first stage of formalization, I will give an alternative *variable may-dependency* definition by referring to the methodology in Cousot (2019a). Specifically, I will define the variables' dependency relation over two witness traces and an initial trace. Comparing to the existing *variable may-dependency* definition, which quantifies overall possible execution traces, the alternative the definition explicitly relies on two specific witness traces from the program executions. This externalization helps in analyzing the *dependency quantity* based on the same witness traces as the *variable may-dependency* relation. In this way, the over-approximation as illustrated above can be reduced.
2. Then based on the new *variable may-dependency* definition, I will add the weight of every edge constructed from *variable may-dependency* relation in the *semantics-based dependency* graph, with respect to the witness traces.
3. Then in the third stage, the *adaptivity* is still defined as the length of the longest finite walk. Differently, the finite walk is added with extra restriction. The occurrence of every edge in a finite walk is restricted to be no more than its weight as well.

Through these steps above, ideally, we will give a more accurate definition of the intuitive *adaptivity*.

Adaptivity Static Analysis. In the adaptivity analysis algorithm **AdaptFun**, we use the reachability bound analysis technique to estimate the weight for each labeled variable. However, it comes across an over-approximation due to its path-insensitive nature. Specifically, it is not precise enough with respect to the semantics-based weight for the labeled variables in different loop paths. The Example 7.1.3 shows this limitation. In this example, the estimated adaptivity by static analysis in Section 6 is $1 + 2 * k$ where k is the input variable. However, the formal *adaptivity* is only $1 + k$ from the semantics-based analysis. This is because that the weight estimation algorithm in Section 6.3.3 is path-insensitive. It occurs when the control flow can be decided in a particular way in front of conditional branches, while the static analysis fails to witness.

The accurate static adaptivity analysis is planned to be improved by using our *path-sensitive reachability-bound* algorithm from **PART ii** as follows.

- New **AdaptFun** will first construct the abstract control flow graphs with extended constraints according to the *path-sensitive reachability bound* algorithm.
- We will still perform the same algorithm for analyzing the dependency relations.
- Then in the second step, we improve the **AdaptFun** by performing the newly-designed *path-sensitive reachability bound* algorithm on each program location for estimating its visiting times during the program execution. The result from the new algorithm gives us a tighter approximation on the weight of each labeled variable.
- At the end, we still construct the same *estimated dependency graph* but using more accurate estimated result, and compute a tighter *adaptivity* for the pro-

grams.

Again as shown in Figure 7.3(b), ideally we will compute $\frac{k}{2}$ as the weight for vertices p^6 and x^7 through the newly-designed path-sensitive reachability bound analysis algorithm! This also gives the motivation for the work in **PART ii** in this thesis.

15.2 PATH-SENSITIVE REACHABILITY-BOUND ANALYSIS

15.2.1 Conclusion

We presented a path-sensitive reachability-bound computation algorithm. This algorithm can compute the evaluation times of each program point accurately and path-sensitively. We also implemented this algorithm as a prototype and evaluated it over 5 different benchmarks. The evaluation results show that we can compute tight bound on the evaluation times of each program point in a program. For program with multi-path loop, we computed different bounds for the points on different paths.

15.2.2 Future Works

The future works of our path-sensitive reachability-bound analysis mainly focus on improving our algorithm and the implementation from the three aspects, the *efficiency*, *accuracy* and the *generality*.

15.2.2.1 The Efficiency

In our algorithm, there are six steps in order to precisely estimate the reaching times of each location. Each step involves at least one iteration of the program, which results in low efficiency. The major bottleneck comes from the recursion

and potential non-termination in the path-refinement algorithm and computing the invariants for the ranking functions. So in the next step, we will explore new path refinement Cyphert et al. (2019) and loop summarization techniques Breck et al. (2020); Kincaid et al. (2019, 2018) and design a more efficient path refinement algorithm. We also aim to design a more efficient ranking estimation in replace of the existing one.

15.2.2.2 *The Accuracy*

The accuracy of the path refinement algorithm and program abstraction implementation. We sacrificed accuracy in order to improve the efficiency of the implementation. For the path refinement algorithm, we limited its maximal iteration times to 100, and the same for computing program constraints. In the future, we plan to improve the accuracy of the implementation by extending the implementation with the new efficient algorithm. Cyphert et al. (2019) introduced a precise path refinement technique over regular expression. Its experiments show that they are accurate in eliminating infeasible loop paths, which can be a potential technique for improvement.

15.2.2.3 *The Generality*

In our existing implementation, the prototype is limited in terms of expressiveness and scalability. So far we only support the program with integer counters and linear counter transformation. While there are some advanced works reasoning about the non-linear loop summarization and invariant generation. Against this background, we plan to support more complex computations in the next step.

PART IV APPENDIX

APPENDIX A

Supplementary Materials in PART i

A.1 PROOFS OF LEMMAS FOR THE LANGUAGE MODEL

A.1.1 Proof of Lemma 4.2.1

Lemma (Trace Non-Decreasing). *For any program $c \in \mathcal{C}$ and initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, if there exists $\tau \in \mathcal{T}^{+\infty}$ and $c' \in \mathcal{C}$ such that $\langle c, \tau_0 \rangle \rightarrow \langle c', \tau \rangle$ then there exists a trace $\tau' \in \mathcal{T}^{+\infty}$ such that $\tau_{0++}\tau' = \tau$ formally as follows.*

$$\begin{aligned} \forall \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^{+\infty}, c, c' \in \mathcal{C} . \langle c, \tau_0 \rangle \rightarrow \langle c', \tau \rangle \\ \implies \exists \tau' \in \mathcal{T}^{+\infty} . \tau_{0++}\tau' = \tau \end{aligned}$$

Proof. Taking arbitrary program c , initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, let $\tau \in \mathcal{T}^+$ be the execution trace and c' be a program satisfying

$$\langle c, \tau_0 \rangle \rightarrow \langle c', \tau \rangle,$$

then we have the following cases by induction on c .

case: $c = [x \leftarrow e]^l$

By the evaluation rule `assn`, we have $\langle [x \leftarrow a]^l, \tau_0 \rangle \rightarrow \langle \text{skip}, \tau :: (x, l, v, \bullet) \rangle$, for some $v \in \mathbb{N}$.

Picking $\tau = \tau :: (x, l, v, \bullet)$ and $\tau' = [(x, l, v, \bullet)]$, it is obvious that $\tau_{0++}\tau' = \tau$.

This case is proved.

case: $c = [x \leftarrow \text{query}(\psi)]^l$

By the evaluation rule query

$$\frac{\langle \tau, \psi \rangle \Downarrow_q \alpha \quad \text{query}(\alpha) = v \quad \epsilon = (x, l, v, \alpha)}{\langle [x \leftarrow \text{query}(\psi)]^l, \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau :: \epsilon \rangle},$$

let $v \in \mathbb{N}$ be the result of query request and α be the query value. Then we have

$$\langle [x \leftarrow \text{query}(\psi)]^l, \tau_0 \rangle \rightarrow \langle [\text{skip}]^l, \tau_0 :: (x, l, v, \alpha) \rangle.$$

Let $\tau = \tau_0 :: (x, l, v, \alpha)$ and $\tau' = [(x, l, v, \alpha)]$, then we have

$$\tau_{0++\tau'} = \tau,$$

and this case is proved.

case: $c = c_1; c_2$

By the first rule applied to c in operational semantics, there are two cases as follows.

sub-case: seq1

If the first rule applied to c is seq1, we have

$$\frac{\langle c_1, \tau_0 \rangle \rightarrow \langle c'_1, \tau_1 \rangle}{\langle c_1; c_2, \tau_0 \rangle \rightarrow \langle c'_1; c_2, \tau_1 \rangle}$$

By induction hypothesis on c_1 , we know

$$\exists \tau'_1 \in \mathcal{T}^{+\infty} . \tau_{0++\tau'_1} = \tau_1$$

Let $\tau = \tau_1$ and $\tau' = \tau'_1$, then we have $\tau_{0++\tau'} = \tau_{0++\tau'_1} = \tau_1 = \tau$ and this case is

proved.

sub-case: seq2

If the first rule applied to c is **seq2**, we have

$$\frac{\langle c_2, \tau_0 \rangle \rightarrow \langle c'_2, \tau_2 \rangle}{\langle [\text{skip}]^l; c_2, \tau_0 \rangle \rightarrow \langle c_2, \tau_2 \rangle}$$

By induction hypothesis on c_2 , we know

$$\exists \tau'_2 \in \mathcal{T}^{+\infty} . \tau_{0++}\tau'_2 = \tau_2$$

Let $\tau = \tau_2$ and $\tau' = \tau'_2$, then we have $\tau_{0++}\tau' = \tau$ and this case is proved.

case: **while** $[b]^{l_w}$ **do** (c_w)

By the first rule applied to c in operational semantics, there are two cases as follows.

sub-case: while-t

If the first rule applied to c is **while-t**, we have

$$\langle \text{while } [b]^{l_w} \text{ do } (c_w), \tau_0 \rangle \rightarrow \langle c_w; \text{while } [b]^{l_w} \text{ do } (c_w), \tau_0 :: (b, l_w, \text{true}, \bullet) \rangle.$$

Let $\tau = \tau_0 :: (b, l_w, \text{true})$ and $\tau' = [(b, l_w, \text{true})]$, then we have $\tau_{0++}\tau' = \tau$ and this case is proved.

sub-case: while-f

If the first rule applied to c is **while-f**, we have

$$\langle \text{while } [b]^{l_w} \text{ do } (c_w), \tau_0 \rangle \rightarrow^{\text{while-f}} \langle \text{skip}, \tau_0 :: (b, l_w, \text{false}, \bullet) \rangle,$$

Let $\tau = \tau_0 :: (b, l_w, \text{false}, \bullet)$ and $\tau' = [(b, l_w, \text{false}, \bullet)]$, then we have $\tau_{0++\tau'} = \tau$ and this case is proved.

case: $\text{if } ([b]^l, c_t, c_f)$

By the first rule applied to $\text{if } ([b]^l, c_t, c_f)$, we have two subcases if-t and if-f.

Both of the two subcases will generate event of the evaluation result on the guard expression $[b]^l$.

In the two subcases, the event ϵ is either $(b, l, \text{true}, \bullet)$ or $(b, l, \text{false}, \bullet)$ respectively.

Again by rule if-t and if-f, ϵ is appended to the initial trace.

Let $\tau' = [(b, l, \text{false}, \bullet)]$ and $\tau' = [(b, l, \text{false}, \bullet)]$ in each subcases respectively, we have $\tau_{0++\tau'} = \tau$ in both subcases and this case is proved. $\square$

A.1.2 Proof of Lemma 4.1.1

Lemma (Uniqueness of the Labeled Variables). *Given any program $c \in \mathcal{C}$ with its label variable set $\text{LV}(c)$, every two labeled variables in this set are different.*

$$\forall c \in \mathcal{C}, x^i, y^j \in \text{LV}(c) . i \neq j.$$

Proof. This is proved directly by the consistency property of the command label. $\square$

A.1.3 Proof of Corollary 4.2.0.1

Corollary. *For any event $\epsilon \in \mathcal{E}$ that is generated from a query request, there exist another event $\epsilon' \in \mathcal{E}$ such that ϵ and ϵ' are equivalent but different in their query results.*

$$\forall \epsilon \in \mathcal{E} . \pi_4(\epsilon) \neq \bullet \implies \exists \epsilon' \in \mathcal{E} . \epsilon = \epsilon' \wedge \pi_3(\epsilon) \neq \pi_3(\epsilon')$$

Proof. Let $\epsilon = (x, l, v, \alpha) \in \mathcal{E}$ be arbitrary event that is generated from a query request for some variable x , label l and a query result v .

By the query evaluation rule `query`, we know $v \in \mathbb{N}$.

Let $v' = v + 1$, we construct another event $\epsilon' = (x, l, v', \alpha) \in \mathcal{E}$.

By definition 10, we know $\epsilon =_q \epsilon'$.

We also know $\pi_3(\epsilon) \neq \pi_3(\epsilon')$ since $v' = v + 1 \neq v$.

So we have this corollary proved. □

A.1.4 Proof of Corollary 4.2.0.2

Corollary. *For any event $\epsilon \in \mathcal{E}$ that is generated from a query request and a trace $\tau \in \mathcal{T}^+$ contains this event such that τ can be expressed in the form of $\tau_{1++}[\epsilon]_{++}\tau_2 = \tau$ for some traces $\tau_1, \tau_2 \in \mathcal{T}^+$, then there exist another event $\epsilon' \in \mathcal{E}$ such that $\pi_3(\epsilon) \neq \pi_3(\epsilon')$ and the same τ has a different form $\tau_{1++}[\epsilon']_{++}\tau_2$.*

$$\begin{aligned} \forall \epsilon \in \mathcal{E}, \tau, \tau_1, \tau_2 \in \mathcal{T}^+ . \pi_4(\epsilon) \neq \bullet \wedge \tau_{1++}[\epsilon]_{++}\tau_2 = \tau \\ \implies \exists \epsilon' \in \mathcal{E} . \tau_{1++}[\epsilon']_{++}\tau_2 = \tau \wedge \pi_3(\epsilon) \neq \pi_3(\epsilon') \end{aligned}$$

Proof. Let $\epsilon = (x, l, v, \alpha) \in \mathcal{E}$ be arbitrary event that is generated from a query request command for some variable x , label l , query result v and query value α .

Let $\tau \in \mathcal{T}^+$ be the trace that contains ϵ and two finite traces τ_1, τ_2 satisfying

$$\tau = \tau_{1++}[\epsilon]_{++}\tau_2.$$

Then it is sufficient to show

$$\exists \epsilon' \in \mathcal{E} . \tau_{1++}[\epsilon']_{++}\tau_2 = \tau \wedge \pi_3(\epsilon) \neq \pi_3(\epsilon')$$

By Corollary 4.2.0.1, let ϵ' be the event such that

$$\epsilon =_q \epsilon' \wedge \pi_3(\epsilon) \neq \pi_3(\epsilon').$$

Then by definition 12, we know $\epsilon' \in \tau$ and

$$\tau_{1++}[\epsilon']_{++}\tau_2 = \tau$$

Then we have this corollary proved. □

A.2 PROOF OF ADAPTIVITY SOUNDNESS

Theorem (Soundness of AdaptFun). *For every well-formed program c , its estimated adaptivity is a sound upper bound of its adaptivity.*

$$\forall \tau_0 \in \mathcal{T}_0^+(c), v \in \mathbb{N} \cup \{\infty\} . \langle \mathbf{A}_{\text{est}}(c), \tau_0 \rangle \Downarrow_e v \implies A(c)(\tau_0) \leq v$$

This proof is summarized informally as below.

Constructing the estimated dependency graph $\mathbf{G}_{\text{est}}(c) = (\mathbf{V}_{\text{est}}, \mathbf{E}_{\text{est}}, \mathbf{W}_{\text{est}}, \mathbf{Q}_{\text{est}})$

and semantics-based dependency graph $\mathbf{G}_{\text{trace}}(c) = (\mathbf{V}_{\text{trace}}, \mathbf{E}_{\text{trace}}, \mathbf{W}_{\text{trace}}, \mathbf{Q}_{\text{trace}})$

1. prove the one-on-one mapping from $\mathbf{V}_{\text{est}}$ to $\mathbf{V}_{\text{trace}}$, in Lemma A.2.1;
2. prove the total map from $\mathbf{E}_{\text{trace}}$ to $\mathbf{E}_{\text{est}}$, in Lemma A.2.2;
3. prove that the weight of every vertex in $\mathbf{G}_{\text{trace}}$ is bounded by the weight of the same vertex in $\mathbf{G}_{\text{est}}$, in Lemma A.2.3;
4. prove the one-on-one mapping from $\mathbf{Q}_{\text{est}}$ to $\mathbf{Q}_{\text{trace}}$, in Lemma A.2.4;
5. show query length of every walk in $\mathcal{WK}(\mathbf{G}_{\text{trace}})$ is bounded by the query length of a path in $\mathcal{PATH}(\mathbf{G}_{\text{est}})$.
6. get the conclusion that $A(c)$ is bounded by the $\mathbf{A}_{\text{est}}(c)$.

Proof. Given a program c , let $\mathbf{G}_{\text{trace}}(c) = (\mathbf{V}_{\text{trace}}, \mathbf{E}_{\text{trace}}, \mathbf{W}_{\text{trace}}, \mathbf{Q}_{\text{trace}})$ be its semantics-based dependency graph by Definition 20 and $\mathbf{G}_{\text{est}}(c) = (\mathbf{V}_{\text{est}}, \mathbf{E}_{\text{est}}, \mathbf{W}_{\text{est}}, \mathbf{Q}_{\text{est}})$ be its estimated-dependency graph by Definition 38.

According to the Definition 41 and Definition 23, it is sufficient to show:

$$\begin{aligned} \forall \tau_0 \in \mathcal{T}_0^+(c), m \in \mathbb{N} \cup \{\infty\} . \langle \max\{\mathbf{len}^q(p) \mid p \in \mathcal{PATH}(\mathbf{G}_{\text{est}}(c))\}, \tau_0 \rangle \Downarrow_e m \\ \implies \lambda \tau_0 . \max\{\mathbf{len}^q(k)(\tau_0) \mid k \in \mathcal{WK}(\mathbf{G}_{\text{trace}}(c))\}(\tau_0) \leq m \end{aligned}$$

Let $\tau_0 \in \mathcal{T}_0^+(c)$ be an arbitrary initial trace of c , then it is sufficient to show that:

$$\forall k \in \mathcal{WK}(\mathcal{G}_{\text{trace}}(c)), m \in \mathbb{N} \cup \{\infty\} . \exists p \in \mathcal{PATH}(\mathcal{G}_{\text{est}}(c)) \\ \wedge \left(\langle \hat{\text{len}}^q(p), \tau_0 \rangle \Downarrow_e m \implies \text{len}^q(k)(\tau_0) \leq m \right)$$

Let $k \in \mathcal{WK}(\mathcal{G}_{\text{trace}}(c))$ be an arbitrary walk in $\mathcal{G}_{\text{trace}}(c)$, $(e_1, \dots, e_{(n-1)})$ and $(v_1, \dots, v_n)$ be the edges and vertices sequence of $k(\tau_0)$.

sub-case:

To show $\exists p \in \mathcal{PATH}(\mathcal{G}_{\text{est}}(c))$, we construct a path p with an edge sequence $(e'_1, \dots, e'_{(n-1)})$ be all the unique edges in $(e_1, \dots, e_{(n-1)})$ and $(v'_1, \dots, v'_n)$ be the corresponding vertices sequence such that

$$\forall e'_i, e'_j \in (e'_1, \dots, e'_{(n-1)}), e_{i'}, e_{j'} \in (e_1, \dots, e_{(n-1)}) . \\ e'_i = (v'_i, v'_{i+1}) \wedge e'_j = (v'_j, v'_{j+1}) \wedge e_{i'} = (v'_{i'}, v'_{i'+1}) \wedge e_{j'} = (v'_{j'}, v'_{j'+1}) \wedge i \leq j \\ \implies i' \leq j'$$

To show $p \in \mathcal{G}_{\text{est}}(c)$, By Lemma A.2.1 and Lemma A.2.2, we know

$$\forall e_i \in k . e_i = (v_i, v_{i+1}) \\ \implies \exists e'_i . e'_i = (v_i, v_{i+1}) \wedge e'_i \in \mathcal{E}_{\text{est}}.$$

Since we construct $(e'_1, \dots, e'_{(n-1)})$ be all the unique edges in $(e_1, \dots, e_{(n-1)})$, then we know

$$\forall i, j = 1, \dots, n . i \neq j \implies v'_i \neq v'_j$$

So we have $p \in \mathcal{PATH}(\mathcal{G}_{\text{est}})$ proved.

sub-case:

To show

$$\forall m \in \mathbb{N} \cup \{\infty\} . \langle \mathbf{len}^q(p), \tau_0 \rangle \Downarrow_e m \implies \mathbf{len}^q(k)(\tau_0) \leq m,$$

by Definition 40, we know

$$\mathbf{len}^q(p) = \sum_{\{v'_i \in (v'_1, \dots, v'_n) \mid (v_i, 1) \in \mathbf{Q}_{\text{est}}(c) \wedge (v'_i, \hat{w}_i) \in \mathbf{W}_{\text{est}}(c)\}} \hat{w}_i.$$

By Definition 22, we know

$$\mathbf{len}^q(k)(\tau_0) = |\{(v_i \mid v_i \in (v_1, \dots, v_n) \wedge (v_i, 1) \in \mathbf{Q}_{\text{trace}}(c) \wedge k(\tau_0) = (v_1, \dots, v_n))\}|.$$

Since $(v'_1, \dots, v'_n)$ are all the unique vertices from the vertices sequence $k(\tau_0) = (v_1, \dots, v_n)$, by the finite walk definition in Definition 21, we know

$$\begin{aligned} \mathbf{len}^q(k)(\tau_0) &= |\{(v_i \mid v_i \in (v_1, \dots, v_n) \wedge (v_i, 1) \in \mathbf{Q}_{\text{trace}}(c) \wedge k(\tau_0) = (v_1, \dots, v_n))\}| \\ &\leq \sum \{w_i(\tau_0) \mid v'_i \in (v'_1, \dots, v'_n) \wedge (v'_i, 1) \in \mathbf{Q}_{\text{trace}}(c) \wedge (v'_i, w_i) \in \mathbf{W}_{\text{trace}}(c)\} \end{aligned}$$

Then, it is sufficient to show

$$\begin{aligned} &\forall m \in \mathbb{N} \cup \{\infty\} . \langle \sum_{\{v'_i \in (v'_1, \dots, v'_n) \mid (v_i, 1) \in \mathbf{Q}_{\text{est}}(c) \wedge (v'_i, \hat{w}_i) \in \mathbf{W}_{\text{est}}(c)\}} \hat{w}_i, \tau_0 \rangle \Downarrow_e m \\ &\implies \sum_{\{v'_i \in (v'_1, \dots, v'_n) \mid (v'_i, 1) \in \mathbf{Q}_{\text{trace}}(c) \wedge (v'_i, w_i) \in \mathbf{W}_{\text{trace}}(c)\}} w_i(\tau_0) \leq m \end{aligned}$$

Let $m \in \mathbb{N} \cup \{\infty\}$ be the evaluated result satisfying

$$\langle \sum_{\{v'_i \in (v'_1, \dots, v'_n) \mid (v_i, 1) \in \mathbf{Q}_{\text{est}}(c) \wedge (v'_i, \hat{w}_i) \in \mathbf{W}_{\text{est}}(c)\}} \hat{w}_i, \tau_0 \rangle \Downarrow_e m.$$

Let $m_1, \dots, m_n$ be the evaluation results of $\hat{w}_i$ where $(v'_i, \hat{w}_i) \in \mathbf{W}_{\text{est}}(c)$ for every

$v'_i \in (v'_1, \dots, v'_n)$ and $(v'_i, 1) \in \mathbf{Q}_{\text{est}}(c)$ such that

$$\langle \hat{w}_i, \tau_0 \rangle \Downarrow_e m_i.$$

Then by operational semantics rules, we know

$$m = \sum_{\{v'_i \in (v'_1, \dots, v'_n) \mid (v'_i, 1) \in \mathbf{Q}_{\text{est}}(c)\}} m_i.$$

Then it is sufficient to show

$$\begin{aligned} & \sum_{\{v'_i \in (v'_1, \dots, v'_n) \mid (v'_i, 1) \in \mathbf{Q}_{\text{trace}}(c) \wedge (v'_i, w_i) \in \mathbf{W}_{\text{trace}}(c)\}} w_i(\tau_0) \\ & \leq \sum_{\{v'_i \in (v'_1, \dots, v'_n) \mid (v'_i, 1) \in \mathbf{Q}_{\text{est}}(c)\}} m_i \end{aligned}$$

By Lemma A.2.4, we know

$$\forall v'_i \in (v'_1, \dots, v'_n) . (v'_i, 1) \in \mathbf{Q}_{\text{trace}}(c) \iff (v'_i, 1) \in \mathbf{Q}_{\text{est}}(c)$$

We also know by Definition 20 and 38

$$(v'_i, 1) \in \mathbf{Q}_{\text{est}}(c) \iff v'_i \in \mathbf{QV}(c) \wedge (v'_i, 1) \in \mathbf{Q}_{\text{trace}}(c) \iff v'_i \in \mathbf{QV}(c)$$

Then it is sufficient to show

$$\begin{aligned} & \sum_{\{v'_i \in (v'_1, \dots, v'_n) \mid v'_i \in \mathbf{QV}(c) \wedge (v'_i, w_i) \in \mathbf{W}_{\text{trace}}(c)\}} w_i(\tau_0) \\ & \leq \sum_{\{v'_i \in (v'_1, \dots, v'_n) \mid v'_i \in \mathbf{QV}(c)\}} m_i \end{aligned}$$

By Lemma A.2.3, we know

$$\begin{aligned} \forall v'_i \in (v'_1, \dots, v'_n), (v'_i, w_i) \in \mathbf{W}_{\text{trace}}(c), m_i \in \mathbb{N} \cup \{\infty\} . \exists (v'_i, \hat{w}_i) \in \mathbf{W}_{\text{est}}(c) . \\ (\langle \hat{w}_i, \tau_0 \rangle \Downarrow_e m_i \implies w_i(\tau_0) \leq m_i) \end{aligned}$$

Let $m_1, \dots, m_n$ be the evaluation results of each $(v'_i, \hat{w}_i)$ for every $v'_i \in (v'_1, \dots, v'_n)$, then we know

$$\begin{aligned} & \sum_{\{v'_i \in (v'_1, \dots, v'_n) \mid (v_i, 1) \in \mathbf{Q}_{\text{trace}}(c) \wedge (v'_i, w_i) \in \mathbf{W}_{\text{trace}}(c)\}} w_i(\tau_0) \\ & \leq \sum_{\{v'_i \in (v'_1, \dots, v'_n) \mid v'_i \in \mathbf{QV}(c) \wedge (v'_i, \hat{w}_i) \in \mathbf{W}_{\text{est}}(c)\}} m_i \\ & = \sum_{\{v'_i \in (v'_1, \dots, v'_n) \mid v'_i \in \mathbf{QV}(c)\}} m_i \\ & = m \end{aligned}$$

and this sub goal is proved. □

The following are four helper lemmas used above, showing the correspondence between each component on the *estimated dependency graph* and *semantics-based dependency graph*.

Lemma A.2.1 (Vertex One-to-one Correspondence). *Given a program c with its estimated dependency graph $\mathbf{G}_{\text{est}}(c) = (\mathbf{V}_{\text{est}}, \mathbf{E}_{\text{est}}, \mathbf{W}_{\text{est}}, \mathbf{Q}_{\text{est}})$ and semantics-based dependency graph $\mathbf{G}_{\text{trace}}(c) = (\mathbf{V}_{\text{trace}}, \mathbf{E}_{\text{trace}}, \mathbf{W}_{\text{trace}}, \mathbf{Q}_{\text{trace}})$, then for every labeled variable $v \in \mathcal{LV}$, $v \in \mathbf{V}_{\text{est}}$ if and only if $v \in \mathbf{V}_{\text{trace}}$.*

$$\begin{aligned} \forall c \in \mathcal{C}, v \in \mathcal{LV} . \mathbf{G}_{\text{est}}(c) &= (\mathbf{V}_{\text{est}}, \mathbf{E}_{\text{est}}, \mathbf{W}_{\text{est}}, \mathbf{Q}_{\text{est}}) \\ \wedge \mathbf{G}_{\text{trace}}(c) &= (\mathbf{V}_{\text{trace}}, \mathbf{E}_{\text{trace}}, \mathbf{W}_{\text{trace}}, \mathbf{Q}_{\text{trace}}) \\ \implies v \in \mathbf{V}_{\text{est}} &\iff v \in \mathbf{V}_{\text{trace}} \end{aligned}$$

Proof. Given a program c , let $\mathbf{G}_{\text{trace}}(c) = (\mathbf{V}_{\text{trace}}, \mathbf{E}_{\text{trace}}, \mathbf{W}_{\text{trace}}, \mathbf{Q}_{\text{trace}})$ be its semantics-

based dependency graph by Definition 20 and $G_{\text{est}}(c) = (V_{\text{est}}, E_{\text{est}}, W_{\text{est}}, Q_{\text{est}})$ be its estimated-dependency graph by Definition 38.

By the two definitions, we also know $V_{\text{trace}} = LV(c)$ and $V_{\text{est}} = LV(c)$.

Then we know $V_{\text{trace}} = V_{\text{est}}$, i.e.,

$$v \in \mathcal{LV} \cdot v \in V_{\text{est}} \iff v \in V_{\text{trace}},$$

and this case is proved □

Lemma A.2.2 (Edge Correspondence). *Given a program c with its estimated dependency graph $G_{\text{est}}(c) = (V_{\text{est}}, E_{\text{est}}, W_{\text{est}}, Q_{\text{est}})$ and semantics-based dependency graph $G_{\text{trace}}(c) = (V_{\text{trace}}, E_{\text{trace}}, W_{\text{trace}}, Q_{\text{trace}})$, then for every $e = (v_1, v_2) \in E_{\text{trace}}$, there exists an edge $e' = (v'_1, v'_2) \in E_{\text{est}}$ with $v_1 = v'_1 \wedge v_2 = v'_2$.*

$$\begin{aligned} \forall c \in \mathcal{C} \cdot G_{\text{est}}(c) &= (V_{\text{est}}, E_{\text{est}}, W_{\text{est}}, Q_{\text{est}}) \wedge G_{\text{trace}}(c) = (V_{\text{trace}}, E_{\text{trace}}, W_{\text{trace}}, Q_{\text{trace}}) \\ \implies \forall e = (v_1, v_2) \in E_{\text{trace}} \cdot \exists e' \in E_{\text{est}} \cdot e' &= (v_1, v_2) \end{aligned}$$

Proof. Taking arbitrary program c . By Definition 38 and Definition 20, we construct its estimated dependency graph

$$G_{\text{est}}(c) = (V_{\text{est}}, E_{\text{est}}, W_{\text{est}}, Q_{\text{est}})$$

and semantics-based dependency graph

$$G_{\text{trace}}(c) = (V_{\text{trace}}, E_{\text{trace}}, W_{\text{trace}}, Q_{\text{trace}}).$$

Taking an arbitrary edge $e = (x^i, y^j) \in E_{\text{trace}}$, it is sufficient to show $(x^i, y^j) \in E_{\text{est}}$.

By Lemma A.2.1, we know $x^i, y^j \in V_{\text{est}}$.

By definition of E_{trace} in Definition 20, we know $\text{DEP}_{\text{var}}(x^i, y^j, c)$.

By Theorem A.3.1, we know

$$\exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . n \geq 0 \wedge \text{flowsTo}(x^i, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, y^j, c).$$

Then by definition of E_{est} in Definition 38, we know $(x^i, y^j) \in E_{\text{est}}$. This Lemma is proved. $\square$

Lemma A.2.3 (Weight Correspondence). *Given a program c with the weight set W_{trace} in its semantics-based dependency graph and the estimated weight set W_{est} in its estimated dependency graph, for every pair of weight $(x^l, w) \in W_{\text{trace}}$, there is a pair of estimated weight $(x^l, \hat{w}) \in W_{\text{est}}$ such that the estimated weight $\hat{w}$ is an upper bound on w for any arbitrary initial trace $\tau_0 \in \mathcal{T}_0^+(c)$.*

$$\forall c \in \mathcal{C}, \tau_0 \in \mathcal{T}_0^+(c) .$$

$$G_{\text{est}}(c) = (V_{\text{est}}, E_{\text{est}}, W_{\text{est}}, Q_{\text{est}}) \wedge G_{\text{trace}}(c) = (V_{\text{trace}}, E_{\text{trace}}, W_{\text{trace}}, Q_{\text{trace}})$$

$$\implies \forall (x^l, w) \in W_{\text{trace}}, v \in \mathbb{N} \cup \{\infty\} . \exists (x^l, \hat{w}) \in W_{\text{est}} .$$

$$(\langle \hat{w}, \tau_0 \rangle \Downarrow_e v \implies w(\tau_0) \leq v)$$

Proof. Taking arbitrary program c and an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$. By Definition 38 and Definition 20, we construct its estimated dependency graph

$$G_{\text{est}}(c) = (V_{\text{est}}, E_{\text{est}}, W_{\text{est}}, Q_{\text{est}})$$

and semantics-based dependency graph

$$G_{\text{trace}}(c) = (V_{\text{trace}}, E_{\text{trace}}, W_{\text{trace}}, Q_{\text{trace}}).$$

By the well-formed program in Definition 9, we know c terminates when exe-

cutting under τ_0 . Let $\tau \in \mathcal{T}^+$ be the execution trace satisfying:

$$\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle$$

Taking arbitrary $(x^l, w) \in W_{\text{trace}}$, by Lemma 6.3.1, we know there exists $\hat{e} = (l, _, _) \in \text{abstrace}(c)$.

Then by definition 37, we know

$$\hat{w} = \sum \{ \text{TB}(\hat{e}, c) \mid \hat{e} \in \text{abstrace}(c) \wedge \hat{e} = (l, _, _) \}.$$

Let $v \in \mathbb{N} \cup \{\infty\}$ be the evaluation result of $\hat{w}$, i.e.,

$$\langle \hat{w}, \tau_0 \rangle \Downarrow_e v$$

By soundness of reachability bound analysis in Theorem 6.3.1, we know

$$\text{cnt}(\tau, l) \leq v.$$

By definition 20, we know

$$w(\tau_0) = \text{cnt}(\tau, l).$$

Then we have $w(\tau_0) \leq v$ and this is proved. $\square$

Lemma A.2.4 (Query Annotaion One-to-one Correspondence). *Given a program c with its estimated dependency graph $G_{\text{est}}(c) = (V_{\text{est}}, E_{\text{est}}, W_{\text{est}}, Q_{\text{est}})$ and semantics-based dependency graph $G_{\text{trace}}(c) = (V_{\text{trace}}, E_{\text{trace}}, W_{\text{trace}}, Q_{\text{trace}})$, then for every $(x^i, n) \in \mathcal{LV} \times$*

$\{0, 1\}, (x^i, n) \in \mathbf{Q}_{\text{trace}}$ if and only if $(x^i, n) \in \mathbf{Q}_{\text{est}}$.

$$\forall c \in \mathcal{C}, x^i \in \text{LV}(c), n \in \{0, 1\} .$$

$$\begin{aligned} \mathbf{G}_{\text{est}}(c) &= (\mathbf{V}_{\text{est}}, \mathbf{E}_{\text{est}}, \mathbf{W}_{\text{est}}, \mathbf{Q}_{\text{est}}) \wedge \mathbf{G}_{\text{trace}}(c) = (\mathbf{V}_{\text{trace}}, \mathbf{E}_{\text{trace}}, \mathbf{W}_{\text{trace}}, \mathbf{Q}_{\text{trace}}) \\ \implies (x^i, n) \in \mathbf{Q}_{\text{trace}} &\iff (x^i, n) \in \mathbf{Q}_{\text{est}} \end{aligned}$$

Proof. Given a program c , let $\mathbf{G}_{\text{trace}}(c) = (\mathbf{V}_{\text{trace}}, \mathbf{E}_{\text{trace}}, \mathbf{W}_{\text{trace}}, \mathbf{Q}_{\text{trace}})$ be its semantics-based dependency graph by Definition 20 and $\mathbf{G}_{\text{est}}(c) = (\mathbf{V}_{\text{est}}, \mathbf{E}_{\text{est}}, \mathbf{W}_{\text{est}}, \mathbf{Q}_{\text{est}})$ be its estimated-dependency graph by Definition 38.

By the two definitions, we also know

$$\begin{aligned} \forall x^i \in \text{LV}(c), n \in \{0, 1\} . \\ (x^i, n) \in \mathbf{Q}_{\text{est}}(c) &\iff x^i \in \mathbf{QV}(c) \wedge (x^i, m) \in \mathbf{Q}_{\text{trace}}(c) \iff x^i \in \mathbf{QV}(c) \end{aligned}$$

Then we have

$$\forall x^i \in \text{LV}(c), n \in \{0, 1\} . (x^i, n) \in \mathbf{Q}_{\text{est}}(c) \iff (x^i, m) \in \mathbf{Q}_{\text{trace}}(c).$$

This lemma is proved. ■

A.3 SOUNDNESS OF EDGE ESTIMATION

A.3.1 Main Theorem

Theorem A.3.1 (flowsTo Soundness). *Given a program c , for all $x^i, y^j \in \text{LV}(c)$, if $\text{DEP}_{\text{var}}(x^i, y^j, c)$, then there exist $z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c)$ with $n \geq 0$ such that $\text{flowsTo}(x^i, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, y^j, c)$*

$$\begin{aligned} & \forall x^i, y^j \in \text{LV}(c) . \text{DEP}_{\text{var}}(x^i, y^j, c) \\ & \implies \left(\exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . n \geq 0 \right. \\ & \quad \left. \wedge \text{flowsTo}(x^i, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, y^j, c) \right) \end{aligned}$$

The proof is summarized informally as below.

For arbitrary two $x^i, y^j \in \text{LV}(c)$, with *Variable May-Dependency* relation, in order to show there exists a "flows-to chain" relation from the static analysis results from x^i to y^j , it is sufficient to show:

1. **One-step Soundness.** x^i is directly used in the expression of the assignment command associated to y^j , or a boolean expression of the guard for a *if* or *while* command with the assignment command associated to y^j showing up in the body of that command, we call it, x^i directly flows to y^j , i.e.,

$$\text{flowsTo}(x^i, y^j, c)$$

2. **Multiple-step Soundness.**, there exists another labelled variable z^l with *variable May-Dependency* relation on x^i and z^l directly flows to y^j , where the *variable May-Dependency* relation between x^i and z^l implies a "sub flowsto-chain" from z^i to z^l , i.e.,

$$\exists z^l \in \text{LV}(c). \text{DEP}_{\text{var}}(x^i, z^l, c) \wedge \text{flowsTo}(z^l, y^j, c)$$

By definition of $\text{DEP}_{\text{var}}(x^i, y^j, c)$, let $D \in \mathcal{DB}$ be the dataset, and $\tau \in \mathcal{T}^+$, ϵ_x, ϵ_y be the trace and two events with $\pi_1(\epsilon_x) = x$ and $\pi_1(\epsilon_y) = y$, such that

$$\text{DEP}_{\mathbf{e}}(\epsilon_x, \epsilon_y, \tau, c, D).$$

Then we show the soundness w.r.t. the *Event May-dependency* relation through the **One-step Soundness** and the **Multiple-step Soundness** cases.

$$\begin{aligned} \text{DEP}_{\mathbf{e}}(\epsilon_x, \epsilon_y, \tau, c, D) &\implies \left(\text{flowsTo}(x^i, y^j, c) \right) \\ &\vee \left(\exists \epsilon \in \tau . \epsilon \in \mathcal{E}^{\text{asn}} \wedge \text{DEP}_{\mathbf{e}}(\epsilon_x, \epsilon, \tau[\epsilon_x : \epsilon], c, D) \wedge \text{flowsTo}(\pi_1(\epsilon)^{\pi_2(\epsilon)}, y^j, c) \right) \end{aligned}$$

Proof. Taking arbitrary $x^i, y^j \in \text{LV}(c)$, by definition of $\text{DEP}_{\text{var}}(x^i, y^j, c)$, let $D \in \mathcal{DB}$ be the dataset, $\tau \in \mathcal{T}^+$, ϵ_x, ϵ_y be the trace and two events such that,

$$\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)} = x^i \wedge \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)} = y^j \wedge \text{DEP}_{\mathbf{e}}(\epsilon_x, \epsilon_y, \tau, c, D).$$

Then, it is sufficient to show,

$$\begin{aligned} &\text{DEP}_{\mathbf{e}}(\epsilon_x, \epsilon_y, \tau, c, D) \\ &\implies \exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . \\ &n \geq 0 \wedge \text{flowsTo}(x^i, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, y^j, c) \end{aligned}$$

By Theorem [A.3.2](#), we have this theorem proved. □

A.3.2 Soundness w.r.t. Event May-dependency

We first introduce some conventional operators as follows, which will be used in the proof.

Definition 76 (Subtrace). *Subtrace*: $[\cdot] : \mathcal{T} \rightarrow \mathcal{E} \rightarrow \mathcal{E} \rightarrow \mathcal{T}^+$

$$\tau[\epsilon_x : \epsilon_y] \triangleq \begin{cases} \tau'[\epsilon_x : \epsilon_y] & \tau = \epsilon :: \tau' \wedge \epsilon \neq_{\mathbf{e}} \epsilon_x \\ \epsilon_x :: \tau'[\epsilon_y] & \tau = \epsilon :: \tau' \wedge \epsilon =_{\mathbf{e}} \epsilon_x \\ [] & \tau = [] \end{cases}$$

For any trace τ and two events $\epsilon_x, \epsilon_y \in \mathcal{E}$, $\tau[\epsilon_x : \epsilon_y]$ takes the subtrace of τ starting with ϵ_x and ending with ϵ_y including ϵ_x and ϵ_y .

We use $\tau[: \epsilon_y]$ as the shorthand of subtrace starting from head and ending with ϵ_y , and similarly for $\tau[\epsilon_x :]$.

$$\tau[: \epsilon] \triangleq \begin{cases} \epsilon' :: \tau'[: \epsilon] & \tau = \epsilon' :: \tau' \wedge \epsilon' \neq_{\mathbf{e}} \epsilon \\ \epsilon' & \tau = \epsilon' :: \tau' \wedge \epsilon' =_{\mathbf{e}} \epsilon \\ [] & \tau = [] \end{cases} \quad \tau[\epsilon :] \triangleq \begin{cases} \tau'[\epsilon :] & \tau = \epsilon' :: \tau' \wedge \epsilon \neq_{\mathbf{e}} \epsilon' \\ \epsilon' :: \tau' & \tau = \epsilon' :: \tau' \wedge \epsilon =_{\mathbf{e}} \epsilon' \\ [] & \tau = [] \end{cases}$$

Program Entry Point: $\text{entry}(c) : \text{Command} \rightarrow \mathbb{N}$

$$\text{entry}(c) \triangleq \begin{cases} l & c = [\text{skip}]^l \\ l & c = [x \leftarrow e_1]^l \\ l & c = [x \leftarrow \text{query}(\psi_1)]^l \\ l & c_1 = \text{if } ([b]^l, c_t, c_f) \\ l & c = \text{while } [b]^l \text{ do } (c_w) \\ \text{entry}(c1) & c = c1; c2 \end{cases}$$

The *Latest Label* operator $\iota : \mathcal{T}^+ \rightarrow \mathcal{V} \rightarrow \mathbb{N} \cup \{\perp\}$, takes a trace and a variable as

input and returns the label of the latest assignment event which assigns value to that variable. Its behavior is defined as follows,

Definition 77 (Latest Label). *Given the trace $\tau \in \mathcal{T}^{+\infty}$ and a variable $x \in \mathcal{VAR}$, the latest label of x is computed as follows.*

$$\begin{aligned} \iota(\tau :: (x, l, _, _))x &\triangleq l & \iota(\tau :: (y, l, _, _))x &\triangleq \iota(\tau)x, y \neq x \\ \iota(\tau :: (b, l, v, \bullet))x &\triangleq \iota(\tau)x & \iota(\square)x &\triangleq \perp \end{aligned}$$

Theorem A.3.2 (DEP_e implies flowsTo). *For every $D \in \mathcal{DB}, c \in \mathcal{C}, \tau \in \mathcal{T}^+ . \forall \epsilon_x, \epsilon_y \in \mathcal{E} . \epsilon_x, \epsilon_y \in \mathcal{E}^{\text{asn}}$, if $\exists \tau' \in \mathcal{T}^+ . \tau = [\epsilon_x]_{++}\tau'_{++}[\epsilon_y]$ and DEP_e($\epsilon_x, \epsilon_y, \tau, c, D$), then there exists $z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c)$ with $n \geq 0$ such that $\text{flowsTo}(x^i, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, y^j, c)$*

$$\begin{aligned} \forall D \in \mathcal{DB}, c \in \mathcal{C}, \tau \in \mathcal{T}^+, \epsilon_x, \epsilon_y \in \mathcal{E} . \text{DEP}_e(\epsilon_x, \epsilon_y, \tau, c, D) \\ \implies \exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . \\ n \geq 0 \wedge \text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c) \end{aligned}$$

Proof. Taking arbitrary $D \in \mathcal{DB}, c \in \mathcal{C}, \tau \in \mathcal{T}^+, \epsilon_x, \epsilon_y \in \mathcal{E}$ such that

$$\text{DEP}_e(\epsilon_x, \epsilon_y, \tau, c, D).$$

Then it is sufficient to show

$$\begin{aligned} \exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . \\ n \geq 0 \wedge \text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c). \end{aligned}$$

By induction on the trace τ we have the following cases:

Case 1. ($\tau = \square$)

In this case, we know $\nexists \tau' \in \mathcal{T}^+$ satisfies $\square = [\epsilon_x]_{++}\tau'_{++}[\epsilon_y]$.

By Definition 16, we know $\neg \text{DEP}_e(\epsilon_x, \epsilon_y, \tau, c, D)$.

By the assumption, we also know $\text{DEP}_e(\epsilon_x, \epsilon_y, \tau, c, D)$ and this case is proved by contradiction.

Case 2. $(\exists \epsilon \in \mathcal{E} . \tau = [\epsilon])$

For any $\epsilon \in \mathcal{E}$, we know $\nexists \tau' \in \mathcal{T}^+$ satisfies $[\epsilon] = [\epsilon_x]_{++}\tau'_{++}[\epsilon_y]$.

By Definition 16, we know $\neg \text{DEP}_e(\epsilon_x, \epsilon_y, \tau, c, D)$.

By the assumption, we also know $\text{DEP}_e(\epsilon_x, \epsilon_y, \tau, c, D)$ and this case is proved by contradiction.

Case 3. $[(\exists \epsilon'_x, \epsilon'_y \in \mathcal{E} . \tau = [\epsilon'_x; \epsilon'_y])]$

In this case, it is sufficient to show

$$\text{DEP}_e(\epsilon_x, \epsilon_y, [\epsilon'_x, \epsilon'_y], c, D) \implies \text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c).$$

Taking arbitrary $\epsilon'_x, \epsilon'_y \in \mathcal{E}$ satisfying $\tau = [\epsilon'_x; \epsilon'_y]$, by law of excluded middle, there are two subcases,

$$\epsilon_x = \epsilon'_x \wedge \epsilon_y = \epsilon'_y \text{ or } \neg(\epsilon_x = \epsilon'_x \wedge \epsilon_y = \epsilon'_y)$$

sub-case: $\neg(\epsilon_x = \epsilon'_x \wedge \epsilon_y = \epsilon'_y)$

In this subcase, we know $\neg \text{DEP}_e(\epsilon_x, \epsilon_y, [\epsilon'_x, \epsilon'_y], c, D)$ by Definition 16.

Then we have this subcase proved by contradiction.

sub-case: $\epsilon_x = \epsilon'_x \wedge \epsilon_y = \epsilon'_y$

In this subcase, it is sufficient to show:

$$\text{DEP}_e(\epsilon_x, \epsilon_y, [\epsilon_x, \epsilon_y], c, D) \implies \text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c)$$

By Lemma A.3.1, we have this case proved.

Case 4. $(\exists \epsilon'_x, \epsilon'_y \in \mathcal{E}, \tau_{ih} \in \mathcal{T}^+ . \tau = [\epsilon'_x]_{++}\tau_{ih}^{++}[\epsilon'_y] \wedge \tau_{ih} \neq [])$

In this case, it is sufficient to show:

$$\begin{aligned}
& \text{DEP}_e(\epsilon_x, \epsilon_y, [\epsilon'_x]_{++}\tau_{ih}++[\epsilon'_y], c, D) \\
& \implies \exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . n \geq 0 \\
& \wedge \text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c)
\end{aligned}$$

Taking arbitrary $\epsilon'_x, \epsilon'_y \in \mathcal{E}$ and trace τ_{ih} satisfying $\tau = [\epsilon'_x]_{++}\tau_{ih}++[\epsilon'_y]$, by law of excluded middle, there are two subcases,

$$\epsilon_x = \epsilon'_x \wedge \epsilon_y = \epsilon'_y \text{ or } \neg(\epsilon_x = \epsilon'_x \wedge \epsilon_y = \epsilon'_y)$$

sub-case: $\neg(\epsilon_x = \epsilon'_x \wedge \epsilon_y = \epsilon'_y)$

In this subcase, we know $\neg \text{DEP}_e(\epsilon_x, \epsilon_y, [\epsilon'_x, \epsilon'_y], c, D)$ by Definition 16.

Then we have this subcase proved by contradiction.

sub-case: $\epsilon_x = \epsilon'_x \wedge \epsilon_y = \epsilon'_y$

In this subcase, it is sufficient to show,

$$\begin{aligned}
& \text{DEP}_e(\epsilon_x, \epsilon_y, [\epsilon_x]_{++}\tau_{ih}++[\epsilon_y], c, D) \\
& \implies \exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . n \geq 0 \\
& \wedge \text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c)
\end{aligned}$$

By $\text{DEP}_e(\epsilon_x, \epsilon_y, [\epsilon_x]_{++}\tau_{ih}++[\epsilon_y], c, D)$ and Definition 16, we have two cases.

case: $\text{DEP}_e(\epsilon_x, \epsilon_y, [\epsilon_x]_{++}\tau_{ih}++[\epsilon_y], c, D)$ satisfies the Equation 5.2(b)

In this case, by Lemma A.3.2, we know:

$$\begin{aligned}
& \left(\text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c) \right) \\
& \vee \left(\exists \epsilon \in \tau_{ih} . \epsilon \in \mathcal{E}^{\text{asn}} \wedge \text{DEP}_e(\epsilon_x, \epsilon, \tau[\epsilon_x : \epsilon], c, D) \wedge \text{flowsTo}(\pi_1(\epsilon)^{\pi_2(\epsilon)}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c) \right)
\end{aligned}$$

Splitting the formula above, we have two subcases.

sub-case: $\text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c)$

In this case, taking $n = 0 \in \mathbb{N}$, we have $\text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c)$ proved directly.

sub-case: $\exists \epsilon \in \tau_{ih} . \epsilon \in \mathcal{E}^{\text{asn}} \wedge \text{DEP}_e(\epsilon_x, \epsilon, \tau[\epsilon_x : \epsilon], c, D) \wedge \text{flowsTo}(\pi_1(\epsilon)^{\pi_2(\epsilon)}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c)$

In this case, let $\epsilon_{ih} \in \tau_{ih}$ be the assignment event such that

$$\text{DEP}_e(\epsilon_x, \epsilon_{ih}, \tau[\epsilon_x : \epsilon_{ih}], c, D) \wedge \text{flowsTo}(\pi_1(\epsilon_{ih})^{\pi_2(\epsilon_{ih})}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c).$$

By induction hypothesis on $\text{DEP}_e(\epsilon_x, \epsilon_{ih}, \tau[\epsilon_x : \epsilon_{ih}], c, D)$, we know:

$$\begin{aligned} \exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . n \geq 0 \\ \wedge \text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, \pi_1(\epsilon_{ih})^{\pi_2(\epsilon_{ih})}, c) \end{aligned}$$

Let $m \geq 0 \in \mathbb{N}$ and $z_1^{r_1}, \dots, z_m^{r_m} \in \text{LV}(c)$ be m labeled variables such that

$$\text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_m^{r_m}, \pi_1(\epsilon_{ih})^{\pi_2(\epsilon_{ih})}, c).$$

Then we have the following by the assumption.

$$\begin{aligned} \text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_m^{r_m}, \pi_1(\epsilon_{ih})^{\pi_2(\epsilon_{ih})}, c) \\ \wedge \text{flowsTo}(\pi_1(\epsilon_{ih})^{\pi_2(\epsilon_{ih})}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c) \end{aligned}$$

Then let $n = m + 1$, and $z_{m+1}^{r_{m+1}} = (\epsilon_{ih})^{\pi_2(\epsilon_{ih})}$, we know

$$\text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_{m+1}^{r_{m+1}}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c)$$

Then we know

$$\begin{aligned} \exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . n \geq 1 \\ \wedge \text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c) \end{aligned}$$

This case is proved.

case: $\text{DEP}_e(\epsilon_x, \epsilon_y, \tau, c, D)$ satisfies the Equation 5.2(c)

By Definition 16, let $\tau_0, \tau_1, \tau', \tau_3, \tau'_3 \in \mathcal{T}, \epsilon'_y \in \mathcal{E}, \epsilon'_x \in \mathcal{E}^{\text{asn}}, \epsilon_b, c_1, c_2$ be the traces, events and commands satisfying

$$\begin{aligned} & \text{Diff}(\epsilon_x, \epsilon'_x) \wedge \left(\langle c, \tau_0 \rangle \rightarrow^* \langle c_1, \tau_{1++}[\epsilon_x] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon_x; \epsilon_b]_{++\tau_3} \rangle \right. \\ & \quad \wedge \langle c_1, \tau_{1++}[\epsilon'_x] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon'_x]_{++\tau'_{++}[(\neg \epsilon_b)]_{++\tau'_3}} \rangle \\ & \quad \left. \wedge \text{TL}(\tau_3) \cap \text{TL}(\tau'_3) = \emptyset \wedge \text{cnt}(\tau') \pi_2(\epsilon_b) = \text{cnt}(\tau) \pi_2(\epsilon_b) \wedge \epsilon_y \in_e \tau_3 \wedge \epsilon_y \notin_e \tau'_3 \right) \end{aligned}$$

Then by Lemma A.3.4 we know

$$\forall z \in \text{FV}(\pi_1(\epsilon_b)) . \exists l \in \mathbb{N} . \text{flowsTo}(z^l, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c) \quad (\star)$$

By Lemma A.3.3, we know

$$\begin{aligned} & (\pi_1(\epsilon_x) \in \text{FV}(\pi_1(\epsilon_b)) \wedge \pi_2(\epsilon_x) = \iota(\tau)) \\ & \vee (\exists \epsilon \in \tau_{ih} . \epsilon \in \mathcal{E}^{\text{asn}} \wedge \pi_1(\epsilon) \in \text{FV}(\pi_1(\epsilon_b)) \wedge \text{DEP}(\epsilon_x, \epsilon, \tau_{ih}[\epsilon_x : \epsilon], c)) \end{aligned}$$

Then we have two subcases.

sub-case: $\pi_1(\epsilon_x) \in \text{FV}(\pi_1(\epsilon_b)) \wedge \pi_2(\epsilon_x) = \iota(\tau)$

In this case, by Definition 32 and $(\star)$ we know

$$\text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c)$$

sub-case: $\exists \epsilon \in \tau_{ih} . \epsilon \in \mathcal{E}^{\text{asn}} \wedge \pi_1(\epsilon) \in \text{FV}(\pi_1(\epsilon_b)) \wedge \text{DEP}(\epsilon_x, \epsilon, \tau_{ih}[\epsilon_x : \epsilon], c)$

In this case, let $\epsilon_{ih} \in \tau_{ih}$ be the assignment event such that

$$\pi_1(\epsilon_{ih}) \in \text{FV}(\pi_1(\epsilon_b)) \wedge \text{DEP}(\epsilon_x, \epsilon, \tau_{ih}[\epsilon_x : \epsilon_{ih}], c)$$

By induction hypothesis on $\text{DEP}(\epsilon_x, \epsilon, \tau_{ih}[\epsilon_x : \epsilon_{ih}], c)$ we know

$$\begin{aligned} \exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . n \geq 0 \\ \wedge \text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, \pi_1(\epsilon_{ih})^{\pi_2(\epsilon_{ih})}, c) \end{aligned}$$

Let $m \geq 0 \in \mathbb{N}$ and $z_1^{r_1}, \dots, z_m^{r_m} \in \text{LV}(c)$ be m labeled variables such that

$$\text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_m^{r_m}, \pi_1(\epsilon_{ih})^{\pi_2(\epsilon_{ih})}, c).$$

By $\pi_1(\epsilon_{ih}) \in \text{FV}(\pi_1(\epsilon_b))$ and $(\star)$ and Definition 32, we also know

$$\text{flowsTo}(\pi_1(\epsilon_{ih})^{\pi_2(\epsilon_{ih})}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c)$$

Then we have the following by the assumption.

$$\begin{aligned} \text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_m^{r_m}, \pi_1(\epsilon_{ih})^{\pi_2(\epsilon_{ih})}, c) \\ \wedge \text{flowsTo}(\pi_1(\epsilon_{ih})^{\pi_2(\epsilon_{ih})}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c) \end{aligned}$$

Then let $n = m + 1$, and $z_{m+1}^{r_{m+1}} = (\epsilon_{ih})^{\pi_2(\epsilon_{ih})}$, we know

$$\text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_{m+1}^{r_{m+1}}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c)$$

Then we know

$$\begin{aligned} \exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . n \geq 1 \\ \wedge \text{flowsTo}(\pi_1(\epsilon_x)^{\pi_2(\epsilon_x)}, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c) \end{aligned}$$

This case is proved.

□

A.3.3 One-step Soundness Lemma

Lemma A.3.1 (One-step `flowsTo` Soundness). *For every $c \in \mathcal{C}$, $D \in \mathcal{DB}$ and two assignment events $\epsilon_1, \epsilon_2 \in \mathcal{E}^{\text{asn}}$, if $\text{DEP}_e(\epsilon_1, \epsilon_2, [\epsilon_1; \epsilon_2], c, D)$, then there is a `flowsTo` relation, $\text{flowsTo}(\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c)$.*

$$\begin{aligned} & \forall \epsilon_1, \epsilon_2 \in \mathcal{E}^{\text{asn}}, c \in \mathcal{C}, D \in \mathcal{DB} . \text{DEP}_e(\epsilon_1, \epsilon_2, [\epsilon_1; \epsilon_2], c, D) \\ & \implies \text{flowsTo}(\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c) \end{aligned}$$

Proof Summary:

1. In the case of the value dependency – the labelled unique assignment command associated to the ϵ_2 is executed but the value assigned to the variable in this event is changed in second execution, we show x directly used by the assignment of the second event
2. In the case of the control dependency – (the labelled unique assignment command associated to the ϵ_2 isn't executed in second execution, we show x is directly used by the boolean expression for a conditional command and second event shows in the body of that conditional command.

Proof. By the Definition 16 for $\text{DEP}_e(\epsilon_1, \epsilon_2, [\epsilon_1; \epsilon_2], c, D)$, we know there are 2 cases:

case: $\text{DEP}_e(\epsilon_1, \epsilon_2, [\epsilon_1; \epsilon_2], c, D)$ satisfies Equation 5.2(b)

Proof. By the definition $\text{DEP}_e(\epsilon_1, \epsilon_2, [\epsilon_1; \epsilon_2], c, D)$, we know

$$\begin{aligned} & \exists \tau_0, \tau_1, \tau' \in \mathcal{T}, \epsilon'_1 \in \mathcal{E}^{\text{asn}}, \epsilon'_2 \in \mathcal{E}, c_1, c_2 \in \mathcal{C} . \text{Diff}(\epsilon_1, \epsilon'_1) \wedge \\ & \left(\begin{aligned} & \langle c, \tau_0 \rangle \rightarrow^* \langle c_1, \tau_{1++}[\epsilon_1] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon_1; \epsilon_2] \rangle \\ & \wedge \langle c_1, \tau_{1++}[\epsilon'_1] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon'_1]_{++\tau'_{++}}[\epsilon'_2] \rangle \\ & \wedge \text{Diff}(\epsilon_2, \epsilon'_2) \wedge \text{cnt}(\tau) \pi_2(\epsilon_2) = \text{cnt}(\tau') \pi_2(\epsilon_2) \end{aligned} \right) \end{aligned} \quad (\text{A.1})$$

Let $\tau_0, \tau_1, \tau' \in \mathcal{T}, \epsilon'_2 \in \mathcal{E}, \epsilon'_1 \in \mathcal{E}^{\text{asn}}, c_1, c_2$ be the traces, events and commands satisfying the executions, by Inversion Lemma A.3.7 on ϵ_1, ϵ_2 , we have the following instance of the first execution in Eq. A.1,

$$\begin{aligned} \langle c, \tau_0 \rangle &\rightarrow^* \langle [x_1 \leftarrow e_1 / \text{query}(\psi_1)]^{\pi_2(\epsilon_1)}; c_1, \tau_1 \rangle \xrightarrow{\text{assn/query}} \langle c_1, \tau_{1++}[\epsilon_1] \rangle \\ &\rightarrow^* \langle [x_2 \leftarrow e_2 / \text{query}(\psi_2)]^{l_2}; c_2, \tau_{1++}[\epsilon_1] \rangle \xrightarrow{\text{assn/query}} \langle c_2, \tau_{1++}[\epsilon_1; \epsilon_2] \rangle, \end{aligned} \quad 1 \quad (\text{A.2})$$

where $x_1 = \pi_1(\epsilon_1)$, $l_1 = \pi_2(\epsilon_1)$, $x_2 = \pi_1(\epsilon_2)$, $l_2 = \pi_2(\epsilon_2)$, and e_1/ψ_1 , e_2/ψ_2 are the expressions of the assignment commands associated to the ϵ_1 and ϵ_2 from Lemma A.3.7.

By $\text{Diff}(\epsilon_2, \epsilon'_2)$ and the command label consistency, we also have the instance of second execution in Eq. A.1 as follows:

$$\langle c_1, \tau_{1++}[\epsilon'_1] \rangle \rightarrow^* \langle [x_2 \leftarrow e_2 / \text{query}(\psi_2)]^{l_2}; c_2, \tau_{1++}[\epsilon'_1] \cdot \tau'_2 \rangle \xrightarrow{\text{assn/query}} \langle c_2, \tau_{1++}[\epsilon'_1] \cdot \tau'_{2++}[\epsilon'_2] \rangle \quad (\text{A.3})$$

From Eq. A.1, we also have

$$\text{cnt}(\tau')_{l_2} = \text{cnt}([\]_{l_2}) = 0 \quad (\text{A.4})$$

By the execution in Eq. A.2, we know:

$$c_1 =_c [x_2 \leftarrow e_2 / \text{query}(\psi_2)]^{l_2}; c_2^2$$

By substituting c_1 in Eq. A.3, the following subproof shows there is only 1 qualified instance of the execution in Eq. A.3.

Subproof. There are two possibilities by the law of excluded middle:

¹ $x \leftarrow e / \text{query}(\psi)$ denotes variable x is assigned by either an expression e or query $\text{query}(\psi)$
² $([\text{skip}];)^*$ denotes a sequence command only composed of $[\text{skip}]$ commands.

$[x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2} \in_c c_2$

or $[x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2} \notin_c c_2$.

1. $[x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2} \notin_c c_2$

In this case, we have the following execution instance

$$\begin{aligned} & \langle [x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2}; c_2, \tau_{1++}[\epsilon'_1] \rangle \\ & \rightarrow_{\text{assn/query}} \langle c_2, \tau_{0++\tau_{1++}}[\epsilon'_1; \epsilon'_2] \rangle \end{aligned}$$

2. $[x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2} \in_c c_2$

By Inversion Lemma A.3.9, we know there is a `while` conditional command (`while` $[b_w]^{l_w}$ `do` (c_w)) in c_2 and $[x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2} \in_c (c_w)$. Then, we have the following possible execution instances

$$\begin{aligned} & \langle [x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2}; c_2, \tau_{1++}[\epsilon'_1] \rangle \\ & \rightarrow_{\text{assn/query}} \langle c_2, \tau_{1++}[\epsilon'_1]_{++}[\epsilon'_2] \rangle \\ \\ & \langle [x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2}; c_2, \tau_{1++}[\epsilon'_1] \rangle \\ & \rightarrow_{\text{assn/query}} \langle c_2, \tau_{1++}[\epsilon'_1]_{++}[(x_2, l_2, v'_2)] \rangle \\ & \rightarrow^* \langle [x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2}; c_2, \tau_{1++}[\epsilon'_1]_{++}[(x_2, l_2, v'_2)]_{++\tau_3} \rangle \\ & \rightarrow_{\text{assn/query}} \langle c_2, \tau_{1++}[\epsilon'_1]_{++}[(x_2, l_2, v'_2)]_{++\tau_3++}[\epsilon'_2] \rangle \\ & \dots, \end{aligned}$$

where each execution instance iterates the conditional command (`while` $[b_w]^{l_w}$ `do` (c_w)) in c_2 , 0, 1 or more times.

For each execution instance, we have the corresponding instance of τ' as follows:

$$\tau' = []$$

$$\tau' = [(x_2, l_2, v'_2)]_{++\tau_3}$$

...

By Eq. A.4 where $\text{cnt}(\tau')l_2 = 0$, we know only the first execution instance with 0 iteration of `while` command in c_2 satisfies this restriction, i.e., $\tau' = []$.

In conclusion, we have the only qualified execution instance as follows where $\tau' = []$.

$$\langle [x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2}; c_2, \tau_{1++}[\epsilon'_1] \rangle \rightarrow^{\text{assn/query}} \langle c_2, \tau_{1++}[\epsilon'_1]_{++}[\epsilon'_2] \rangle$$

■

Then we know by the environment definition, ρ obtains different values only for variable x_1 from trace $\tau_{1++}[\epsilon_1]$ and $\tau_{1++}[\epsilon'_1]$, i.e.,

$$\forall z^r \in \text{LV}(c) \setminus \{x_1^{l_1}\}, \rho(\tau_{1++}[\epsilon_1])(z) = \rho(\tau_{1++}[\epsilon'_1])(z)$$

By Inversion Lemma A.3.5 of arithmetic expression evaluation, we have

$$x_1 \in \text{FV}(e_2/\psi_2)$$

Since $\iota(\tau_{1++}[\epsilon_1])x_1 = l_1$, by Inversion Lemma A.3.8 we know $x_1^{l_1} \in \text{RD}(l_2, c)$.

By `flowsTo` definition, we have:

$$\text{flowsTo}(x_1^{l_1}, x_2^{l_2}, c)$$

i.e.,

$$\text{flowsTo}(\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c)$$

This case is proved. ■

case: $\text{DEP}_e(\epsilon_1, \epsilon_2, [\epsilon_1; \epsilon_2], c, D)$ satisfies Equation 5.2(c)

Proof Summary:

1. Let ϵ_b be the testing event, in the same way of case 1, we get: $\pi_1(\epsilon_1) \in \text{FV}(\pi_1(\epsilon_b)) \wedge \pi_1(\epsilon_1)^{l_1} \in \text{RD}(l_b, c)$
2. By Lemma A.3.4, we know: $\forall z \in \text{FV}(\pi_1(\epsilon_b)) . \exists i \in \mathbb{N} . \text{flowsTo}(z^i, \pi_1(\epsilon)^{\pi_2(\epsilon)}, c)$
3. By `flowsTo` definition we have: $\text{flowsTo}(\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c)$

Proof. By definition $\text{DEP}_e(\epsilon_1, \epsilon_2, [\epsilon_1; \epsilon_2], c, D)$ of case 2, we know

$$\begin{aligned}
& \exists \tau_0, \tau_1, \tau', \tau_3, \tau'_3 \in \mathcal{T}, \epsilon'_1 \in \mathcal{E}^{\text{asn}}, c_1, c_2 \in \mathcal{C}, \epsilon_b \in \mathcal{E}^{\text{test}} . \\
& \text{Diff}(\epsilon_1, \epsilon'_1) \wedge \left(\langle c, \tau_0 \rangle \rightarrow^* \langle c_1, \tau_{1++}[\epsilon_1] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon_1; \epsilon_b]_{++\tau_3} \rangle \right. \\
& \quad \wedge \langle c_1, \tau_{1++}[\epsilon'_1] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon'_1]_{++\tau'_{++}}[(\neg \epsilon_b)]_{++\tau'_3} \rangle \\
& \quad \left. \wedge \text{TL}(\tau_3) \cap \text{TL}(\tau'_3) = \emptyset \wedge \text{cnt}(\tau') \pi_2(\epsilon_b) = \text{cnt}(\tau) \pi_2(\epsilon_b) \wedge \epsilon_2 \in_e \tau_3 \wedge \epsilon_2 \notin_e \tau'_3 \right)
\end{aligned} \tag{A.5}$$

Let $\tau_0, \tau_1, \tau', \tau_3, \tau'_3 \in \mathcal{T}, \epsilon'_2 \in \mathcal{E}, \epsilon'_1 \in \mathcal{E}^{\text{asn}}, \epsilon_b, c_1, c_2$ be the traces, events and commands satisfying the executions, by Inversion Lemma A.3.7 on ϵ_1, ϵ_2 , and ϵ_b , we have the following instance of the first execution in Eq. A.5,

$$\begin{aligned}
& \langle c, \tau_0 \rangle \rightarrow^* \langle [x_1 \leftarrow e_1 / \text{query}(\psi_1)]^{l_1}; c_1, \tau_1 \rangle \xrightarrow{\text{asn/query}} \langle c_1, \tau_{1++}[\epsilon_1] \rangle \\
& \quad \rightarrow^* \langle \text{if } ([b]^{l_b}, c_t, c_f) / \text{while } [b]^{l_b} \text{ do } c_w; c'_3, \tau_{1++}[\epsilon_1] \rangle \\
& \quad \rightarrow \text{if-b / while-b} \langle (c_t; c'_3 / c_f; c'_3) / (c'_3 / c_w; \text{while } [b]^{l_b} \text{ do } c_w; c'_3), \tau_{1++}[\epsilon_1; \epsilon_b] \rangle \\
& \quad \rightarrow^* \langle c_3, \tau_{1++}[\epsilon_1; \epsilon_b]_{++\tau_3} \rangle
\end{aligned} \tag{A.6}$$

, where $x_1 = \pi_1(\epsilon_1)$, $l_1 = \pi_2(\epsilon_1)$, and $\text{if } ([b]^{l_b}, c_t, c_f) / \text{while } [b]^{l_b} \text{ do } c_w$ is the conditional command of the assignment commands associated to the ϵ_b from Inversion Lemma A.3.7 of testing event.

By the command label consistency, we also have the instance of second execution

in Eq. A.5 as follows:

$$\begin{aligned}
\langle c, \tau_0 \rangle &\rightarrow^* \langle [x_1 \leftarrow e_1 / \text{query}(\psi_1)]^{l_1}; c_1, \tau_1 \rangle \xrightarrow{\text{assn/query}} \langle c_1, \tau_{1++}[\epsilon_1] \rangle \\
&\rightarrow^* \langle \text{if } ([b]^{l_b}, c_t, c_f) / \text{while } [b]^{l_b} \text{ do } c_w; c'_3, \tau_{1++}[\epsilon_1]_{++}\tau' \rangle \\
&\rightarrow \text{if-b / while-b } \langle (c_f; c'_3/c_t; c'_3) / (c_w; \text{while } [b]^{l_b} \text{ do } c_w; c'_3/c'_3), \tau_{1++}[\epsilon_1]_{++}\tau'_{++}[\neg\epsilon_b] \rangle \\
&\rightarrow^* \langle c_3, \tau_{1++}[\epsilon_1]_{++}\tau'_{++}[\neg\epsilon_b]_{++}\tau'_3 \rangle
\end{aligned} \tag{A.7}$$

From Eq. A.5, we also have $\text{cnt}(\tau')l_b = \text{cnt}(\square)l_b = 0$.

By the same proof steps from case 1 in Subproof A.3.3, we have

$$x_1 \in \text{FV}(b) \wedge x_1^{l_1} \in \text{RD}(l_b, c)$$

By Lemma A.3.4, we also know:

$$\forall z \in \text{FV}(\pi_1(\epsilon_b)) . \exists i \in \mathbb{N} . \text{flowsTo}(z^i, \pi_1(\epsilon)^{\pi_2(\epsilon)}, c)$$

Then by `flowsTo` definition, we have `flowsTo`($x_1^{l_1}, x_2^{l_2}, c$) i.e.,

$$\text{flowsTo}(\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c)$$

This case is proved. ■

□

A.3.4 Multiple-step Soundness Lemma

Lemma A.3.2 (Multi-step Soundness through Assignment Event). *For every $D \in \mathcal{DB}$, $c \in \mathcal{C}$, $\tau \in \mathcal{T}^+$, one assignment events $\epsilon_1 \in \mathcal{E}^{\text{asn}}$, and another event $\epsilon_2 \in \mathcal{E}$, if the trace τ has the form $\tau = [\epsilon_1]_{++}\tau'_{++}[\epsilon_2]$ with $\tau' \in \mathcal{T}^+$ and $\text{DEP}_e(\epsilon_1, \epsilon_2, \tau, c, D)$ where ϵ_1 and ϵ_2 satisfying the Equation 5.2(b) in this relation, then either there is a direct feasible data-flow relation between $\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}$ and $\pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}$, i.e., $\text{flowsTo}(\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c)$ or there exists an assignment event $\epsilon \in \tau'$ such that $\text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon], c, D)$ and ϵ and ϵ_2 are in the feasible flows to relation, $\text{flowsTo}(\pi_1(\epsilon)^{\pi_2(\epsilon)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c)$.*

$$\forall D \in \mathcal{DB}, c \in \mathcal{C}, \tau \in \mathcal{T}^+ . \forall \epsilon_1, \epsilon_2 \in \mathcal{E}^{\text{asn}} . \exists \tau' \in \mathcal{T}^+ .$$

$$\tau = [\epsilon_1]_{++}\tau'_{++}[\epsilon_2] \wedge \text{DEP}_e(\epsilon_1, \epsilon_2, \tau, c, D) \wedge \text{Equation 5.2(b)} = \text{true}$$

$$\implies \text{flowsTo}(\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c)$$

$$\vee \left(\exists \epsilon \in \tau' . \epsilon \in \mathcal{E}^{\text{asn}} \wedge \text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon], c, D) \wedge \text{flowsTo}(\pi_1(\epsilon)^{\pi_2(\epsilon)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c) \right)$$

Proof. Taking arbitrary $D \in \mathcal{DB}$, $c \in \mathcal{C}$, and an assignment events $\epsilon_1 \in \mathcal{E}^{\text{asn}}$ and another event $\epsilon_2 \in \mathcal{E}$.

Without loss of generalization, taking arbitrary trace has the form $\tau = [\epsilon_1, \dots, \epsilon_2]$ and $\tau_2 \in \mathcal{T}^+$ such that

$$\tau = [\epsilon_1]_{++}\tau_2_{++}[\epsilon_2] \wedge \text{DEP}_e(\epsilon_1, \epsilon_2, \tau, c, D)$$

By the definition of $\text{DEP}_e(\epsilon_1, \epsilon_2, \tau, c, D)$ in Definition 16 and Equation 5.2(b), taking

$\epsilon'_1, \epsilon'_2 \in \mathcal{E}^{\text{asn}}, \tau'_2 \in \mathcal{T}^+, c_1, c_2 \in \mathcal{C}$ as the events, traces and commands such that

$$\begin{aligned} & \langle c, \tau_0 \rangle \rightarrow^* \langle c_1, \tau_{1++}[\epsilon_1] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon_1]_{++}\tau_{2++}[\epsilon_2] \rangle \\ & \wedge \langle c_1, \tau_{1++}[\epsilon'_1] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon'_1]_{++}\tau'_{2++}[\epsilon'_2] \rangle \\ & \wedge \text{Diff}(\epsilon_2, \epsilon'_2) (\star) \\ & \wedge \text{cnt}(\tau_2, \pi_2(\epsilon_2)) = \text{cnt}(\tau'_2, \pi_2(\epsilon'_2)) \end{aligned}$$

By inversion Lemma. A.3.7 on ϵ_2 and ϵ'_2 in the two executions and $\text{Diff}(\epsilon_2, \epsilon'_2)$, we have the following two execution instances:

$$\begin{aligned} & \langle c_1, \tau_{1++}[\epsilon_1] \rangle \rightarrow^* \langle [\pi_1(\epsilon_2) \leftarrow e_2/\text{query}(\psi_2)]^{\pi_2(\epsilon_2)}; c_2, \tau_{1++}[\epsilon_1]_{++}\tau_2 \rangle \\ & \rightarrow \text{asn / query} \langle c_2, \tau_{1++}[\epsilon_1]_{++}\tau_{2++}[\epsilon_2] \rangle \\ & \langle c_1, \tau_{1++}[\epsilon'_1] \rangle \rightarrow^* \langle [\pi_1(\epsilon_2) \leftarrow e_2/\text{query}(\psi_2)]^{\pi_2(\epsilon_2)}; c_2, \tau_{1++}[\epsilon'_1]_{++}\tau'_2 \rangle \\ & \rightarrow \text{asn / query} \langle c_2, \tau_{1++}[\epsilon'_1]_{++}\tau'_{2++}[\epsilon'_2] \rangle \end{aligned}$$

, where e_2/ψ_2 is the expression of the assignment command associated to the ϵ_2 and ϵ'_2 by the Inversion Lemma. A.3.7.

To show

$$\begin{aligned} & \text{flowsTo}(\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c) \\ & \vee \left(\exists \epsilon \in \tau' . \epsilon \in \mathcal{E}^{\text{asn}} \wedge \text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon], c, D) \wedge \text{flowsTo}(\pi_1(\epsilon)^{\pi_2(\epsilon)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c) \right) \end{aligned}$$

It is sufficient to show

$$\begin{aligned} & \left(\forall \epsilon \in \tau' . \neg \epsilon \in \mathcal{E}^{\text{asn}} \vee \neg \text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon], c, D) \vee \neg \text{flowsTo}(\pi_1(\epsilon)^{\pi_2(\epsilon)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c) \right) \\ & \implies \text{flowsTo}(\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c) \end{aligned}$$

Taking arbitrary $\epsilon_z \in \tau_2$, let $\epsilon_z \in \mathcal{E}^{\text{asn}}$, then it is sufficient to show

$$\neg \text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon_z], c, D) \vee \neg \text{flowsTo}(\pi_1(\epsilon_z)^{\pi_2(\epsilon_z)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c) \\ \implies \text{flowsTo}(\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c)$$

Splitting the precondition, we have the following two subcases.

sub-case: $\neg \text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon_z], c, D)$

In case of $\neg \text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon_z], c, D)$, by Definition 16, we know ϵ_z shows up identically on τ'_2 on the same position, i.e., Then by Definition 6, we know

$$\rho(\tau_{1++}[\epsilon_1]_{++}\tau_2[: \epsilon_z])\pi_1(\epsilon_z) = \rho(\tau_{1++}[\epsilon'_1]_{++}\tau'_2[: \epsilon_z])\pi_1(\epsilon_z)$$

Then we have

$$\forall \epsilon_z \in \tau_2 . \rho(\tau_{1++}\tau[\epsilon_1 : \epsilon_z])\pi_1(\epsilon_z) = \rho(\tau_{1++}[\epsilon'_1]_{++}\tau'_2[: \epsilon_z])\pi_1(\epsilon_z)$$

Then by Definition 6, we know

$$\forall \epsilon_z \in \tau_2 . \rho(\tau_{1++}[\epsilon_1]_{++}\tau_2)\pi_1(\epsilon_z) = \rho(\tau_{1++}[\epsilon_1]_{++}\tau_2)\pi_1(\epsilon_z)$$

Then by the determinism of the execution, we know $\epsilon_2 = \epsilon'_2$. Then we have the following by Definition 11.

$$\left((\pi_4(\epsilon_2) = \bullet \wedge \pi_4(\epsilon'_2) = \bullet) \wedge \pi_3(\epsilon_2) = \pi_3(\epsilon'_2) \right) \vee \left((\pi_4(\epsilon_2) \neq \bullet \wedge \pi_4(\epsilon'_2) \neq \bullet) \wedge \pi_4(\epsilon_2) = \pi_4(\epsilon'_2) \right)$$

Then by Definition 13, we know

$$\neg \text{Diff}(\epsilon_2, \epsilon'_2).$$

Since we know $\text{Diff}(\epsilon_2, \epsilon'_2)$ by $(\star)$, then we have this case proved by the contradiction.

sub-case: $\text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon_z], c, D) \wedge \neg \text{flowsTo}(\pi_1(\epsilon_z)^{\pi_2(\epsilon_z)}, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c)$

In this case, by Definition 32, we know

$$\pi_1(\epsilon_z) \notin \text{FV}(e_2/\psi_2).$$

By Inversion Lemma A.3.5 of arithmetic and query expression cases, we know:

$$\begin{aligned} & \forall x^i \in \mathcal{LV}, \tau, \tau' \in \mathcal{T}^+, v, v' . \\ & \quad \left(\forall z^j \in \mathcal{LV} \setminus \{\pi_1(\epsilon_z)^{\pi_2(\epsilon_z)}\} . \right. \\ & \quad \left. \rho(\tau)z = \rho(\tau')z \right) \wedge \langle \tau, e_2/\psi_2 \rangle \Downarrow_a v \wedge \langle \tau', e_2 \rangle \Downarrow_a v' \\ & \implies v = v' \end{aligned}$$

$$\begin{aligned} & \forall x^i \in \mathcal{LV}, \tau, \tau' \in \mathcal{T}^+, \alpha, \alpha' . \\ & \quad \left(\forall z^j \in \mathcal{LV} \setminus \{\pi_1(\epsilon_z)^{\pi_2(\epsilon_z)}\} . \right. \\ & \quad \left. \rho(\tau)z = \rho(\tau')z \right) \wedge \langle \tau, \psi_2 \rangle \Downarrow_q \alpha \wedge \langle \tau', \psi_2 \rangle \Downarrow_q \alpha' \\ & \implies \alpha =_q \alpha' \end{aligned}$$

for e_2 or ψ_2 respectively.

Let use_{τ_2} a subset of the events in τ_2 , satisfying:

$$\forall \epsilon \in \mathcal{E}^{\text{asn}} . \epsilon \in \text{use}_{\tau_2} \iff \epsilon \in \tau_2 \wedge \pi_1(\epsilon) \in \text{FV}(e_2/\psi_2)$$

Let $\text{LV}(\tau_2)$ denotes the set of all labeled variables in τ_2 and $\text{LV}(\text{use}_{\tau_2})$ the set of all labeled variables in use_{τ_2} .

Then we also known for every $\epsilon_z \in \text{use}_{\tau_2}$, $\neg \text{DEP}_e(\epsilon_1, \epsilon_z, \tau[\epsilon_1 : \epsilon_z], c, D)$ as fol-

lows.

$$\forall z^l \in \text{LV}(c) \setminus ((\text{LV}(\tau_2) \setminus \text{LV}(\text{use}_{\tau_2})) \cup \{\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}\}) . \rho(\tau_{1++}[\epsilon_1]_{++}\tau_2)z = \rho(\tau_{1++}[\epsilon'_1]_{++}\tau'_2)z \quad (1)$$

and

$$\forall z^l \in \text{LV}(c) \setminus (\text{LV}(\tau_2) \setminus \text{LV}(\text{use}_{\tau_2})), \tau, \tau' \in \mathcal{T}^+, v, v' .$$

$$\rho(\tau)z = \rho(\tau')z \wedge \langle \tau, e_2 \rangle \Downarrow_a v \wedge \langle \tau', e_2 \rangle \Downarrow_a v' \implies v = v' \quad (2a)$$

and

$$\forall z^l \in \text{LV}(c) \setminus (\text{LV}(\tau_2) \setminus \text{LV}(\text{use}_{\tau_2})), \tau, \tau' \in \mathcal{T}^+, \alpha, \alpha' .$$

$$\rho(\tau)z = \rho(\tau')z \wedge \langle \tau, \psi_2 \rangle \Downarrow_q \alpha \wedge \langle \tau', \psi_2 \rangle \Downarrow_q \alpha' \implies \alpha =_q \alpha' \quad (2q)$$

Since $\text{Diff}(\epsilon_2, \epsilon'_2)$, we also know:

$$\langle \tau_{1++}[\epsilon_1]_{++}\tau_2, e_2 \rangle \Downarrow_a \pi_3(\epsilon_2) \wedge \langle \tau_{1++}[\epsilon'_1]_{++}\tau'_2, e_2 \rangle \Downarrow_a \pi_3(\epsilon'_2) \wedge \pi_3(\epsilon_2) \neq \pi_3(\epsilon'_2)$$

We know ϵ_1 is the only cause of the difference in ϵ_y and ϵ'_y when evaluating the command $[\pi_1(\epsilon_2) \leftarrow e_2/\text{query}(\psi_2)]^{\pi_2(\epsilon_2)}$.

By inversion Lemma. A.3.6 of arithmetic and query expression cases, given the two traces $\tau_{1++}[\epsilon'_1]_{++}\tau'_2$ and $\tau_{1++}[\epsilon_1]_{++}\tau_2$ satisfying this lemma by (1), (2a) and (2q), we know

$$\pi_1(\epsilon_1) \in \text{FV}(e_2/\psi_2) \wedge \pi_2(\epsilon_1) = \iota(\tau_{1++}[\epsilon_1]_{++}\tau_2)\pi_1(\epsilon_1)$$

By `flowsTo` definition:

$$\text{flowsTo}(\pi_1(\epsilon_1)^{\pi_2(\epsilon_1)}, \pi_1(\epsilon_y)^{\pi_2(\epsilon_y)}, c)$$

This case is proved. □

Lemma A.3.3 (Multi-step Soundness through Testing Event). *For every $D \in \mathcal{DB}$, $c \in$*

$\mathcal{C}, \tau \in \mathcal{T}^+$, one assignment events $\epsilon_1 \in \mathcal{E}^{\text{asn}}$, and another testing event $\epsilon_2 \in \mathcal{E}^{\text{test}}$, if the trace τ has the form $\tau = [\epsilon_1]_{++}\tau'_{++}[\epsilon_2]$ with $\tau' \in \mathcal{T}^+$, and $\text{DEP}_e(\epsilon_1, \epsilon_2, \tau, c, D)$ where ϵ_1, ϵ_2 satisfy Equation 5.2(b), then either variable in ϵ_1 directly shows up in the boolean expression in ϵ_2 , or there exists an assignment event $\epsilon \in \tau'$ such that the variable assigned in ϵ shows up in the boolean expression in ϵ_2 and $\text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon], c, D)$.

$$\begin{aligned} & \forall D \in \mathcal{DB}, c \in \mathcal{C}, \tau \in \mathcal{T}^+ . \forall \epsilon_1 \in \mathcal{E}^{\text{asn}}, \epsilon_2 \in \mathcal{E}^{\text{test}} . \exists \tau' \in \mathcal{T}^+ . \\ & \tau = [\epsilon_1]_{++}\tau'_{++}[\epsilon_2] \wedge \text{DEP}_e(\epsilon_1, \epsilon_2, \tau, c, D) \wedge \text{Equation 5.2(b)} = \text{true} \\ & \implies \pi_1(\epsilon_1) \in \text{FV}(\pi_1(\epsilon_2)) \wedge \pi_2(\epsilon_1) = \iota(\tau) \\ & \vee \left(\exists \epsilon \in \tau' . \epsilon \in \mathcal{E}^{\text{asn}} \wedge \pi_1(\epsilon) \in \text{FV}(\pi_1(\epsilon_2)) \wedge \text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon], c, D) \right) \end{aligned}$$

Proof. Taking arbitrary $D \in \mathcal{DB}, c \in \mathcal{C}$, and an assignment events $\epsilon_1 \in \mathcal{E}^{\text{asn}}$ and another event $\epsilon_2 \in \mathcal{E}$.

Without loss of generalization, taking arbitrary trace has the form $\tau = [\epsilon_1, \dots, \epsilon_2]$ and $\tau_2 \in \mathcal{T}^+$ such that

$$\tau = [\epsilon_1]_{++}\tau_2_{++}[\epsilon_2] \wedge \text{DEP}_e(\epsilon_1, \epsilon_2, \tau, c, D)$$

By the definition of $\text{DEP}_e(\epsilon_1, \epsilon_2, \tau, c, D)$, taking $\epsilon'_1 \in \mathcal{E}^{\text{asn}}, \tau'_2 \in \mathcal{T}^+, c_1, c_2 \in \mathcal{C}$ and $\epsilon'_2 \in \mathcal{E}^{\text{test}}$ as the events, traces and commands satisfying the definition, we have following two executions:

$$\begin{aligned} & \langle c, \tau_0 \rangle \rightarrow^* \langle c_1, \tau_1_{++}[\epsilon_1] \rangle \rightarrow^* \langle c_2, \tau_1_{++}[\epsilon_1]_{++}\tau_2_{++}[\epsilon_2] \rangle \\ & \wedge \langle c_1, \tau_1_{++}[\epsilon'_1] \rangle \rightarrow^* \langle c_2, \tau_1_{++}[\epsilon'_1]_{++}\tau'_2_{++}[\epsilon'_2] \rangle \\ & \wedge \text{Diff}(\epsilon_2, \epsilon'_2) (\star) \\ & \wedge \text{cnt}(\tau_2, \pi_2(\epsilon_2)) = \text{cnt}(\tau'_2, \pi_2(\epsilon'_2)) \end{aligned}$$

To show

$$\pi_1(\epsilon_1) \in \mathbf{FV}(\pi_1(\epsilon_2)) \wedge \pi_2(\epsilon_1) = \iota(\tau) \\ \vee \left(\exists \epsilon \in \tau' . \epsilon \in \mathcal{E}^{\text{asn}} \wedge \pi_1(\epsilon) \in \mathbf{FV}(\pi_1(\epsilon_2)) \wedge \text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon], c, D) \right)$$

It is sufficient to show

$$\forall \epsilon \in \tau' . \neg \epsilon \in \mathcal{E}^{\text{asn}} \vee \neg \text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon], c, D) \vee \pi_1(\epsilon) \notin \mathbf{FV}(\pi_1(\epsilon_2)) \\ \implies \pi_1(\epsilon_1) \in \mathbf{FV}(\pi_1(\epsilon_2)) \wedge \pi_2(\epsilon_1) = \iota(\tau)$$

Taking arbitrary $\epsilon_z \in \tau_2$, let $\epsilon_z \in \mathcal{E}^{\text{asn}}$, it is sufficient to show

$$\neg \text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon], c, D) \vee \pi_1(\epsilon) \notin \mathbf{FV}(\pi_1(\epsilon_2)) \\ \implies \pi_1(\epsilon_1) \in \mathbf{FV}(\pi_1(\epsilon_2)) \wedge \pi_2(\epsilon_1) = \iota(\tau)$$

Splitting the precondition in the same way as the proof of Lemma A.3.2, we have the same two subcases.

sub-case: $\neg \text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon], c, D)$

In this case, by the same proof in the first case of Lemma A.3.2, we have $\neg \text{Diff}(\epsilon_2, \epsilon'_2)$.

Then we have this case proved by the contradiction between $(\star)$ and $\neg \text{Diff}(\epsilon_2, \epsilon'_2)$.

sub-case: $\text{DEP}_e(\epsilon_1, \epsilon, \tau[\epsilon_1 : \epsilon_z], c, D) \wedge \pi_1(\epsilon) \notin \mathbf{FV}(\pi_1(\epsilon_2))$

In this case, by the Inversion Lemma A.3.5 and A.3.6 of the boolean expression case, and the same proof in the second case of Lemma A.3.2, we have

$$\pi_1(\epsilon_1) \in \mathbf{FV}(\pi_1(\epsilon_2)) \wedge \pi_2(\epsilon_1) = \iota(\tau)$$

This case is proved. □

Lemma A.3.4 (Equation 5.2(c) Inversion). *For every $c \in \mathcal{C}$, $D \in \mathcal{DB}$, $\tau \in \mathcal{T}^+$ and*

two assignment events $\epsilon_1, \epsilon_2 \in \mathcal{E}^{\text{asn}}$, if $\text{DEP}_e(\epsilon_1, \epsilon_2, c, \tau, D)$ where ϵ_1 and ϵ_2 satisfy the Equation 5.2(c) in Definition 16 and $\epsilon_b \in \mathcal{E}^{\text{test}}$ is the testing event witness, then for all $z \in \text{FV}(\pi_1(\epsilon_b))$ there exists a label $i \in \mathbb{N}$ such that $\text{flowsTo}(z^i, \pi_1(\epsilon)^{\pi_2(\epsilon)}, c)$

$$\begin{aligned}
& \forall D \in \mathcal{DB}, c \in \mathcal{C}, \tau \in \mathcal{T}^+, \epsilon_1, \epsilon_2 \in \mathcal{E}^{\text{asn}}. \\
& \exists \tau_0, \tau_1, \tau', \tau_3, \tau'_3 \in \mathcal{T}^+, \epsilon'_1 \in \mathcal{E}^{\text{asn}}, c_1, c_2 \in \mathcal{C}, \epsilon_b \in \mathcal{E}^{\text{test}}, \tau_{ih} \in \mathcal{T}^+ . \tau = [\epsilon_1]_{++} \tau_{ih} [\epsilon_2] \\
& \wedge \langle c, \tau_0 \rangle \rightarrow^* \langle c_1, \tau_{1++} [\epsilon_1] \rangle \rightarrow^* \langle c_2, \tau_{1++} [\epsilon_1]_{++} \tau_{++} [\epsilon_b]_{++} \tau_3 \rangle \\
& , \\
& \wedge \langle c_1, \tau_{1++} [\epsilon'_1] \rangle \rightarrow^* \langle c_2, \tau_{1++} [\epsilon'_1]_{++} \tau'_{++} [(\neg \epsilon_b)]_{++} \tau'_3 \rangle \\
& \wedge \text{TL}(\tau_3) \cap \text{TL}(\tau'_3) = \emptyset \wedge \text{cnt}(\tau') \pi_2(\epsilon_b) = \text{cnt}(\tau) \pi_2(\epsilon_b) \wedge \epsilon_2 \in_e \tau_3 \wedge \epsilon_2 \notin_e \tau'_3 \\
& \implies \forall z \in \text{FV}(\pi_1(\epsilon_b)) . \exists l \in \mathbb{N} . \text{flowsTo}(z^l, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c)
\end{aligned} \tag{A.8}$$

Proof Summary:

Proving by using the Inversion Lemmas A.3.5, A.3.6, A.3.7, and A.3.8, and the Event May-dependency definition of the second case.

Proof. Take arbitrary $D \in \mathcal{DB}, c \in \mathcal{C}, \tau \in \mathcal{T}^+, \epsilon_1, \epsilon_2 \in \mathcal{E}^{\text{asn}}$, let $\tau_0, \tau_1, \tau', \tau_3, \tau'_3 \in \mathcal{T}^+, \epsilon'_2 \in \mathcal{E}, \epsilon'_1 \in \mathcal{E}^{\text{asn}}, \epsilon_b, c_1, c_2$ be the traces, events and commands satisfying the executions, by Inversion Lemma A.3.7 on ϵ_2 , and ϵ_b , we have the following instance of the first execution in Eq. A.8,

$$\begin{aligned}
& \langle c, \tau_0 \rangle \rightarrow^* \langle \text{if } ([b]^{l_b}, c_t, c_f) / \text{while } [b]^{l_b} \text{ do } (c_w); c'_3, \tau_{1++} [\epsilon_1]_{++} \tau \rangle \\
& \rightarrow \text{if-b / while-b} \langle (c_t; c'_3 / c_f; c'_3) / (c_w; \text{while } [b]^{l_b} \text{ do } (c_w); c'_3 / [\text{skip}]^{l'}; c'_3), \tau_{1++} [\epsilon_1]_{++} \tau_{++} [\epsilon_b] \rangle \\
& \rightarrow^* \langle [x_2 \leftarrow e_2 / \text{query}(\psi_2)]^{l_2}; c'_{3b}, \tau_{1++} [\epsilon_1]_{++} \tau_{++} [\epsilon_b]_{++} \tau_{3a} \rangle \\
& \rightarrow \text{assn/query} \langle c'_{3b}, \tau_{1++} [\epsilon_1]_{++} \tau_{++} [\epsilon_b]_{++} \tau_{3a} [\epsilon_2] \rangle \\
& \rightarrow^* \langle c_3, \tau_{1++} [\epsilon_1]_{++} \tau_{++} [\epsilon_b]_{++} \tau_{3a} [\epsilon_2]_{++} \tau_{3b} \rangle
\end{aligned} \tag{A.9}$$

, where $\tau_3 = \tau_{3a} [\epsilon_2]_{++} \tau_{3b}$, $x_2 = \pi_1(\epsilon_2)$, $l_2 = \pi_2(\epsilon_2)$, and $\text{if } ([b]^{l_b}, c_t, c_f) / \text{while } [b]^{l_b} \text{ do } (c_w)$ is the conditional command of the assignment commands associated to the ϵ_b from

Inversion Lemma A.3.7 of testing event.

The notation $(c_t; c'_3/c_f; c'_3)/(c_w; \text{while } [b]^{l_b} \text{ do } (c_w); c'_3/[\text{skip}]^{l'}; c'_3)$ represents:

In case of $\text{if } ([b]^{l_b}, c_t, c_f), \text{ if } \pi_3(\epsilon_b) = \text{true}$, we have the evaluation:

$$\langle \text{if } ([b]^{l_b}, c_t, c_f); c'_3, \tau_{1++}[\epsilon_1]_{++}\tau \rangle \xrightarrow{\text{if-b}} \langle c_t; c'_3\tau_{1++}[\epsilon_1]_{++}\tau_{++}[\epsilon_b] \rangle$$

The same for case of $\text{if } ([b]^{l_b}, c_t, c_f)$ with $\pi_3(\epsilon_b) = \text{false}$, and case of $\text{while } [b]^{l_b} \text{ do } (c_w)$ with $\pi_3(\epsilon_b) = \text{true}$ and $\pi_3(\epsilon_b) = \text{false}$.

By the command label consistency, we also have the instance of second execution as follows:

$$\begin{aligned} \langle c, \tau_0 \rangle &\xrightarrow{*} \langle \text{if } ([b]^{l_b}, c_t, c_f) / \text{while } [b]^{l_b} \text{ do } (c_w); c'_3, \tau_{1++}[\epsilon_1]_{++}\tau' \rangle \\ &\xrightarrow{\text{if-b} / \text{while-b}} \langle (c_f; c'_3/c_t; c'_3)/([\text{skip}]^{l'}; c'_3/c_w; \text{while } [b]^{l_b} \text{ do } (c_w); c'_3), \\ &\quad \tau_{1++}[\epsilon_1]_{++}\tau'_{++}[\neg\epsilon_b] \rangle \\ &\xrightarrow{*} \langle c_3, \tau_{1++}[\epsilon_1]_{++}\tau'_{++}[\neg\epsilon_b]_{++}\tau'_3 \rangle \end{aligned} \quad (\text{A.10})$$

By the label consistency, and $\text{TL}(\tau_3) \cap \text{TL}(\tau'_3) = \emptyset$, we know τ_3 and τ'_3 doesn't contain any event of evaluating the commands in c'_3 . Otherwise, $\text{TL}(\tau_3) \cap \text{TL}(\tau'_3) \neq \emptyset$, which is a contradiction.

Since $\tau_3 = \tau_{3a++}[\epsilon_2]_{++}\tau_{3b}$, we know ϵ_2 doesn't comes from evaluating of c'_3 , i.e.:

In the case of $\text{if } ([b]^{l_b}, c_t, c_f)$, ϵ_2 comes from the evaluation of c_t or c_f , Then we know

$$[x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2} \in_c c_t \vee [x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2} \in_c c_f;$$

and in the case of $\text{while } [b]^{l_b} \text{ do } (c_w)$, ϵ_2 comes from the evaluation of c_w , i.e.,

$$[x_2 \leftarrow e_2/\text{query}(\psi_2)]^{l_2} \in_c c_w.$$

In both of the two cases, we know $\forall z \in \text{FV}(\pi_1(\epsilon_b))$ there is a label $l \in \mathbb{N}$ for this variable, and by the `flowsTo` definition, $\text{flowsTo}(z^l, \pi_1(\epsilon_2)^{\pi_2(\epsilon_2)}, c)$.

This lemma is proved.



A.3.5 Inversion Lemmas and Helper Lemmas

The following are the inversion lemmas and helper lemmas used in the proof of Theorem A.3.2 above, showing the correspondence properties between the trace based semantics and the program analysis results.

Lemma A.3.5 (Expression Inversion). *For all $x^i \in \mathcal{LV}$, and $\tau, \tau' \in \mathcal{T}^{+\infty}$, and an expression e if $\forall z^j \in \mathcal{LV} \setminus \{x^i\} . \rho(\tau)z = \rho(\tau')z$, and if*

- *e is an arithmetic expression a , and $\langle \tau, a \rangle \Downarrow_a v$ and $\langle \tau', a \rangle \Downarrow_a v'$ with $v' \neq v$, then x is in the free variables of a and i is the latest label for x in τ , i.e., $x \in \text{FV}(a)$ and $i = \iota(\tau)x$.*
- *e is a boolean expression b , and $\langle \tau, b \rangle \Downarrow_b v$ and $\langle \tau', b \rangle \Downarrow_b v'$ with $v' \neq v$, then x is in the free variables of b and i is the latest label for x in τ , i.e., $x \in \text{FV}(b)$ and $i = \iota(\tau)x$.*
- *e is a query expression ψ , and $\langle \tau, \psi \rangle \Downarrow_q \alpha$ and $\langle \tau', \psi \rangle \Downarrow_q \alpha'$ with $\alpha \neq_q \alpha'$, then x is in the free variables of ψ and i is the latest label for x in τ , i.e., $x \in \text{FV}(\psi)$ and $i = \iota(\tau)x$.*

Proof Summary:

To show $x \in \text{FV}(a)$, by showing contradiction ($\forall \tau, \tau'$ in second hypothesis $v = v'$) if $x \notin \text{FV}(a)$.

To show $i = \iota(\tau)$, by showing contradiction ($\forall \tau, \tau'$ in second hypothesis $v = v'$) if $j = \iota(\tau)x$ and $i \neq j$.

Proof. Take two arbitrary traces $\tau, \tau' \in \mathcal{T}^{+\infty}$, and an expression e satisfying $\forall z^j \in \mathcal{LV} \setminus \{x^i\} . \rho(\tau)z = \rho(\tau')z$, we have the following three cases.

case: e is an arithmetic expression a

We have $\langle \tau, a \rangle \Downarrow_a v$ and $\langle \tau', a \rangle \Downarrow_a v'$ with $v' \neq v$ from the lemma hypothesis.

To show $x \in \text{FV}(\psi)$ and $i = \iota(\tau)x$:

Assuming $x \notin \text{FV}(a)$, since $\forall z^j \in \mathcal{LV} \setminus \{x^i\} . \rho(\tau)z = \rho(\tau')z$, we know $v = v'$, which is contradicted to $v' \neq v$.

Then we know $x \in \text{FV}(\psi)$.

Assuming $j = \iota(\tau)x \wedge i \neq j$, by $\forall z^j \in \mathcal{LV} \setminus \{x^i\} . \rho(\tau)z = \rho(\tau')z$, we know $\rho(\tau)x = \rho(\tau')x$, i.e.,

$$\forall z^j \in \mathcal{LV} . \rho(\tau)z = \rho(\tau')z.$$

Then by the determination of the evaluation, we know $v = v'$, which is contradicted to $v' \neq v$.

Then we know $i = \iota(\tau)x$.

case: e is a boolean expression b

This case is proved trivially in the same way as the case of the arithmetic expression.

case: e is a query expression ψ

This case is proved trivially in the same way as the case of the arithmetic expression. □

Lemma A.3.6 (Expression Inversion Generalization). *For all subset of the labelled variables $\mathcal{DF} \subset \mathcal{LV}$, and $x^i \in (\mathcal{LV} \setminus \mathcal{DF})$, and an expression e , if*

- *e is an arithmetic expression a , and for all $z^j \in \mathcal{LV} \setminus \mathcal{DF}, \tau, \tau' \in \mathcal{T}^+, v, v'$ such that $\rho(\tau)z = \rho(\tau')z$, and $\langle \tau, a \rangle \Downarrow_a v$, and $\langle \tau', a \rangle \Downarrow_a v'$ with $v = v'$; and for all $z^j \in \mathcal{LV} \setminus (\mathcal{DF} \cup \{x^i\})$ there exist $\tau, \tau' \in \mathcal{T}^+, v, v'$ such that $\rho(\tau)z = \rho(\tau')z$, and*

$\langle \tau, a \rangle \Downarrow_a v$, and $\langle \tau', a \rangle \Downarrow_a v'$ with $v \neq v'$, then $x \in \mathbf{FV}(a)$ and $i = \iota(\tau)x$.

$$\forall \mathcal{DF} \subset \mathcal{LV}, x^i \in (\mathcal{LV} \setminus \mathcal{DF}), a .$$

$$\forall z^j \in \mathcal{LV} \setminus \mathcal{DF}, \tau, \tau' \in \mathcal{T}^+, v, v' .$$

$$\rho(\tau)z = \rho(\tau')z \wedge \langle \tau, a \rangle \Downarrow_a v \wedge \langle \tau', a \rangle \Downarrow_a v' \wedge v = v'$$

$$\implies \forall z^j \in \mathcal{LV} \setminus (\mathcal{DF} \cup \{x^i\}) . \exists \tau, \tau' \in \mathcal{T}^+, v, v' .$$

$$\rho(\tau)z = \rho(\tau')z \wedge \langle \tau, a \rangle \Downarrow_a v \wedge \langle \tau', a \rangle \Downarrow_a v' \wedge v \neq v'$$

$$\implies x \in \mathbf{FV}(a) \wedge i = \iota(\tau)x$$

- e is a boolean expression b , and for all $z^j \in \mathcal{LV} \setminus \mathcal{DF}, \tau, \tau' \in \mathcal{T}^+, v, v'$ such that $\rho(\tau)z = \rho(\tau')z \wedge \langle \tau, b \rangle \Downarrow_b v \wedge \langle \tau', b \rangle \Downarrow_b v' \wedge v = v'$; and for all $z^j \in \mathcal{LV} \setminus (\mathcal{DF} \cup \{x^i\}) . \exists \tau, \tau' \in \mathcal{T}^+, v, v' . \rho(\tau)z = \rho(\tau')z \wedge \langle \tau, b \rangle \Downarrow_b v \wedge \langle \tau', b \rangle \Downarrow_b v' \wedge v \neq v'$ then $x \in \mathbf{FV}(b) \wedge i = \iota(\tau)x$

$$\forall \mathcal{DF} \subset \mathcal{LV}, x^i \in (\mathcal{LV} \setminus \mathcal{DF}), b .$$

$$\forall z^j \in \mathcal{LV} \setminus \mathcal{DF}, \tau, \tau' \in \mathcal{T}^+, v, v' .$$

$$\rho(\tau)z = \rho(\tau')z \wedge \langle \tau, b \rangle \Downarrow_b v \wedge \langle \tau', b \rangle \Downarrow_b v' \wedge v = v'$$

$$\implies \forall z^j \in \mathcal{LV} \setminus (\mathcal{DF} \cup \{x^i\}) . \exists \tau, \tau' \in \mathcal{T}^+, v, v' .$$

$$\rho(\tau)z = \rho(\tau')z \wedge \langle \tau, b \rangle \Downarrow_b v \wedge \langle \tau', b \rangle \Downarrow_b v' \wedge v \neq v'$$

$$\implies x \in \mathbf{FV}(b) \wedge i = \iota(\tau)x$$

- e is a query expression ψ , and for all $\mathcal{DF} \subset \mathcal{LV}, x^i \in (\mathcal{LV} \setminus \mathcal{DF}), \psi$ such that for all $z^j \in \mathcal{LV} \setminus \mathcal{DF}, \tau, \tau' \in \mathcal{T}^+, \alpha, \alpha' . \rho(\tau)z = \rho(\tau')z \wedge \langle \tau, \psi \rangle \Downarrow_q \alpha \wedge \langle \tau', \psi \rangle \Downarrow_q \alpha' \wedge \alpha =_q \alpha'$; and for all $z^j \in \mathcal{LV} \setminus (\mathcal{DF} \cup \{x^i\}) . \exists \tau, \tau' \in \mathcal{T}^+, \alpha, \alpha' . \rho(\tau)z =$

$\rho(\tau')z \wedge \langle \tau, \psi \rangle \Downarrow_q \alpha \wedge \langle \tau', \psi \rangle \Downarrow_q \alpha' \wedge \alpha \neq_q \alpha'$, then $x \in \text{FV}(\psi) \wedge i = \iota(\tau)x$.

$$\forall \mathcal{DF} \subset \mathcal{LV}, x^i \in (\mathcal{LV} \setminus \mathcal{DF}), \psi.$$

$$\forall z^j \in \mathcal{LV} \setminus \mathcal{DF}, \tau, \tau' \in \mathcal{T}^+, \alpha, \alpha'.$$

$$\rho(\tau)z = \rho(\tau')z \wedge \langle \tau, \psi \rangle \Downarrow_q \alpha \wedge \langle \tau', \psi \rangle \Downarrow_q \alpha' \wedge \alpha =_q \alpha'$$

$$\implies \forall z^j \in \mathcal{LV} \setminus (\mathcal{DF} \cup \{x^i\}). \exists \tau, \tau' \in \mathcal{T}^+, \alpha, \alpha'.$$

$$\rho(\tau)z = \rho(\tau')z \wedge \langle \tau, \psi \rangle \Downarrow_q \alpha \wedge \langle \tau', \psi \rangle \Downarrow_q \alpha' \wedge \alpha \neq_q \alpha'$$

$$\implies x \in \text{FV}(\psi) \wedge i = \iota(\tau)x$$

Proof Summary:

To show $x \in \text{FV}(a)$, by showing contradiction ($\forall \tau, \tau'$ in second hypothesis $v = v'$) if $x \notin \text{FV}(a)$.

To show $i = \iota(\tau)$, by showing contradiction ($\forall \tau, \tau'$ in second hypothesis $v = v'$) if $j = \iota(\tau)x$ and $i \neq j$.

Proof. Taking an arbitrary expression e , we have the following three cases.

case: e is an arithmetic expression a

Taking an arbitrary set of labelled variables $\mathcal{DF} \subset \mathcal{LV}$, $x^i \in (\mathcal{LV} \setminus \mathcal{DF})$ satisfies:

$$\forall z^j \in \mathcal{LV} \setminus \mathcal{DF}, \tau, \tau' \in \mathcal{T}^+, v, v'. \rho(\tau)z = \rho(\tau')z \wedge \langle \tau, a \rangle \Downarrow_a v \wedge \langle \tau', a \rangle \Downarrow_a v' \wedge v = v' \quad (1)$$

$$\text{and } \forall z^j \in \mathcal{LV} \setminus (\mathcal{DF} \cup \{x^i\}). \exists \tau, \tau' \in \mathcal{T}^+, v, v'. \rho(\tau)z = \rho(\tau')z \wedge \langle \tau, a \rangle \Downarrow_a v \wedge \langle \tau', a \rangle \Downarrow_a v' \wedge v \neq v' \quad (2),$$

Let $\tau, \tau' \in \mathcal{T}^+, v, v'$ be the two traces and values satisfies hypothesis (2).

To show: $x \in \text{FV}(a) \wedge i = \iota(\tau)x$:

Assuming $x \notin \text{FV}(a)$, we know from the Inversion Lemma [A.3.5](#) of the arithmetic expression case,

$$\forall z^j \in \mathcal{LV} \setminus \{x^i\}, \tau, \tau' \in \mathcal{T}^+, v, v'. \rho(\tau)z = \rho(\tau')z \wedge \langle \tau, a \rangle \Downarrow_a v \wedge \langle \tau', a \rangle \Downarrow_a v' \wedge v = v'.$$

Then with the hypothesis (1), we know:

$$\forall z^j \in \mathcal{LV} \setminus (\mathcal{DF} \cup \{x^i\}), \tau, \tau' \in \mathcal{T}^+, v, v' . \rho(\tau)z = \rho(\tau')z \wedge \langle \tau, a \rangle \Downarrow_a v \wedge \langle \tau', a \rangle \Downarrow_a v' \wedge v = v'$$

This is contradicted to the hypothesis (2).

Then we know $x \in \text{FV}(e)$.

Assuming $j = \iota(\tau)x \wedge i \neq j$, by hypothesis (2) where $\forall z^j \in \mathcal{LV} \setminus (\mathcal{DF} \cup \{x^i\}) . \rho(\tau)z = \rho(\tau')z$, we know $\rho(\tau)x = \rho(\tau')x$, i.e.,

$$\forall z^j \in \mathcal{LV} \setminus (\mathcal{DF}) . \rho(\tau)z = \rho(\tau')z.$$

Then we have $v' = v$ by hypothesis (1), which is contradicted to $v' \neq v$.

Then we know $i = \iota(\tau)x$.

case: e is a boolean expression b

This case is proved trivially in the same way as the case of the arithmetic expression.

case: e is a query expression ψ

This case is proved trivially in the same way as the case of the arithmetic expression. □

Lemma A.3.7 (Event Inversion). *For all $c \in \mathcal{C}, \tau_0 \in \mathcal{T}^+, \epsilon \in \mathcal{E}$ such that $\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]'', \tau_{0++\tau_1} \rangle$, and $\epsilon \in_e \tau_1$, if*

- $\epsilon \in \mathcal{E}^{\text{asn}}$, then either

- there exists $\tau'_1 \in \mathcal{T}^+, c' \in \mathcal{C}, e$ such that

$$\langle c, \tau_0 \rangle \rightarrow^* \langle [x \leftarrow e]^l; c', \tau_{0++\tau'} \rangle \xrightarrow{\text{asn}} \langle c', \tau_{0++\tau'_1++[\epsilon]} \rangle \rightarrow^* \langle [\text{skip}]'', \tau_{0++\tau_1} \rangle$$

- or there exists $\tau'_1 \in \mathcal{T}^+, c' \in \mathcal{C}, \psi$ such that

$$\begin{aligned} \langle c, \tau_0 \rangle &\rightarrow^* \langle [x \leftarrow \text{query}(\psi)]^l; c', \tau_{0++\tau'_1} \rangle \\ &\xrightarrow{\text{query}} \langle c', \tau_{0++\tau'_1++[\epsilon]} \rangle \rightarrow^* \langle \text{skip}, \tau_{0++\tau_1} \rangle \end{aligned}$$

- $\epsilon \in \mathcal{E}^{\text{test}}$ then either

– there exists $\tau'_1 \in \mathcal{T}^+$, $c', c_t, c_f, c'' \in \mathcal{C}$, b such that

$$\begin{aligned} \langle c, \tau_0 \rangle &\rightarrow^* \langle \text{if } ([b]^l, c_t, c_f); c', \tau_{0++\tau'_1} \rangle \\ &\rightarrow^{\text{if-}b} \langle c'', \tau_{0++\tau'_1++[\epsilon]} \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau_1} \rangle \end{aligned}$$

– or there exists $\tau'_1 \in \mathcal{T}^+$, $c', c_w, c'' \in \mathcal{C}$, b such that

$$\begin{aligned} \langle c, \tau_0 \rangle &\rightarrow^* \langle \text{while } [b]^l \text{ do } (c_w); c', \tau_{0++\tau'_1} \rangle \\ &\rightarrow^{\text{while-}b} \langle c'', \tau_{0++\tau'_1++[\epsilon]} \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau_1} \rangle \end{aligned}$$

Proof Summary: trivially by induction on c and enumerate all operational semantic rules.

Proof. Take arbitrary $\tau_0 \in \mathcal{T}^{+\infty}$, by induction on c , we have following cases:

case: $c = [x \leftarrow e]^l$

By the evaluation rule `assn`, we have $\langle [x \leftarrow a]^l, \tau \rangle \rightarrow \langle [\text{skip}]^{l'}, \tau_{++}[(x, l, v)] \rangle$.

Then we know $\tau_1 = [(x, l, v)]$ and there is only 1 event $(x, l, v) \in \tau_1$.

Then we have $\tau'_1 = []$ and $c' = \text{skip}$ satisfying

$$\langle c, \tau_0 \rangle \rightarrow^* \langle [x \leftarrow e]^l; c', \tau_{0++\tau'} \rangle \rightarrow^{\text{assn}} \langle c', \tau_{0++\tau'_1++[\epsilon]} \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau_1} \rangle.$$

This case is proved.

case: $c = [x \leftarrow \text{query}(\psi)]^l$

This case is proved trivially in the same way as **case:** $c = [x \leftarrow e]^l$.

case: $c = c_{s1}; c_{s2}$

This case is proved trivially by the induction hypothesis on c_{s1} and c_{s2} separately, we have this case proved.

case: while $[b]^l$ do (c_w)

If the rule applied to is while-t, we have:

$$\langle \text{while } [b]^l \text{ do } (c_w), \tau \rangle \rightarrow \langle c_w; \text{while } [b]^l \text{ do } (c_w), \tau_{++}[(b, l, \text{true})] \rangle \xrightarrow{*} \langle \text{skip}, \tau_{++}\tau_1 \rangle, \\ (b, l, \text{true}) \in \epsilon^{\text{test}} \text{ and } (b, l, \text{true}) \in \tau_1.$$

Let $\tau' = []$, $c' = \text{skip}$ and $c'' = c_w$; while $[b]^l$ do (c_w) , we know that they satisfy

$$\langle c, \tau_0 \rangle \rightarrow^* \langle \text{while } [b]^l \text{ do } (c_w); c', \tau_{0++}\tau'_1 \rangle \xrightarrow{\text{while-b}} \langle c'', \tau_{0++}\tau'_1++[\epsilon] \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++}\tau_1 \rangle$$

This case is proved.

If the rule applied to is while-f, we have

$$\langle \text{while } [b]^l \text{ do } (c_w), \tau \rangle \xrightarrow{\text{while-f}} \langle \text{skip}, \tau_{++}[(b, l, \text{false})] \rangle.$$

We also know $(b, l, \text{true}) \in \mathcal{E}^{\text{test}}$ and $(b, l, \text{true}) \in \tau_1$.

Let $\tau' = []$, $c' = \text{skip}$ and $c'' = \text{skip}$, we know that they satisfy

$$\langle c, \tau_0 \rangle \rightarrow^* \langle \text{while } [b]^l \text{ do } (c_w); c', \tau_{0++}\tau'_1 \rangle \xrightarrow{\text{while-f}} \langle c'', \tau_{0++}\tau'_1++[(b, l, \text{false})] \rangle \rightarrow^* \\ \langle [\text{skip}]^{l'}, \tau_{0++}\tau_1 \rangle$$

This case is proved.

case: if $([b]^l, c_t, c_f)$

This case is proved in the same way as **case:** $c = [x \leftarrow \text{query}(\psi)]^l$. □

Lemma A.3.8 (Reachable Variable Inversion). *For a well-formed program $c \in \mathcal{C}$, an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, an execution trace $\tau \in \mathcal{T}^{+\infty}$, and a labeled variable $x^l \in \text{LV}(c)$, if $\langle c, \tau_0 \rangle \rightarrow^* \langle c', \tau \rangle$ and $\iota(\tau)x = l$, then $x^l \in \text{RD}(\text{absinit}(c), c)$.*

$$\forall c \in \mathcal{C}, \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, x^l \in \text{LV}(c) . \langle c, \tau_0 \rangle \rightarrow^* \langle c', \tau \rangle \wedge \iota(\tau)x = l \\ \implies x^l \in \text{RD}(\text{absinit}(c), c)$$

Proof Summary: If a variable with the label which is the latest one in the trace, Then by the environment definition, the value associated to this labelled variable is read from the trace.

Then this labelled variable must be reachable at the point of $\text{entry}_{c'}$, i.e., $x^l \in \text{RD}(\text{absinit}(c), c)$.

Proof. Take arbitrary $c \in \mathcal{C}$, an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, an execution trace $\tau \in \mathcal{T}^{+\infty}$ satisfying $\langle c, \tau_0 \rangle \rightarrow^* \langle c', \tau \rangle$, and an arbitrary $x^l \in \text{LV}(c)$ satisfying $\iota(\tau)x = l$.

By definition of ι , we know τ has the form $\tau_{a++}[(x, l, v)]_{++}\tau'_b$ for some $\tau_a, \tau'_b \in \mathcal{T}^{+\infty}$ and v .

And the variable x doesn't show up in all the events in τ'_b .

Then, by the environment definition, we know: $\rho(\tau)x = v$, i.e., x^l is reachable at the point of $\text{absinit}(c)$.

By the $\text{in}(l)$ operator define in Section 6.3.2, we know x^l is in the $\text{in}(\text{absinit}(c))$ for program c .

Since $\text{RD}(\text{absinit}(c), c)$ is a stabilized closure of $\text{in}(l)$ for c , we know $x^l \in \text{RD}(\text{absinit}(c), c)$.

This lemma is proved. $\square$

Lemma A.3.9 (While Loop Inversion). *For every $\tau, \tau' \in \mathcal{T}^+, c, c_1, c_2 \in \mathcal{C}$ if $\langle c, \tau \rangle \rightarrow^* \langle c_1; c_2, \tau' \rangle$ and $c_1 \in_c c_2$, then there must exist a `while` command in c_2 and c_1 must shows up in the body of that `while` command, i.e., $\exists l \in \mathbb{N}, b \in \mathcal{B}, c_w \in \mathcal{C} . (\text{while } [b]^l \text{ do } (c_w)) \in_c c_2 \wedge c_1 \in_c c_w$.*

$$\forall \tau, \tau' \in \mathcal{T}^+, c, c_1, c_2 \in \mathcal{C} .$$

$$\langle c, \tau \rangle \rightarrow^* \langle c_1; c_2, \tau' \rangle \implies$$

$$(c_1 \in_c c_2 \wedge \exists l \in \mathbb{N}, b \in \mathcal{B}, c_w \in \mathcal{C} . (\text{while } [b]^l \text{ do } (c_w)) \in_c c_2 \wedge c_1 \in_c c_w)$$

Proof Summary: trivially by induction on c and enumerate all operational semantic rules.

Proof. Take arbitrary $\tau \in \mathcal{T}^+$, by induction on c , we have following cases:

case: $c = [x \leftarrow e]^l$

By the evaluation rule `assn`, we have $\langle [x \leftarrow a]^l, \tau \rangle \rightarrow \langle [\text{skip}]^{l'}, \tau_{++}[(x, l, v)] \rangle$.

Since there doesn't exist $c_1, c_2 \in \mathcal{C}$ satisfying $\text{skip} = c_1; c_2$, this theorem is vacuously true.

case: $c = [x \leftarrow \text{query}(\psi)]^l$

By the evaluation rule `query`, we have $\langle [x \leftarrow \text{query}(\psi)]^l, \tau \rangle \rightarrow \langle [\text{skip}]^{l'}, \tau_{++}[(x, l, \alpha, v)] \rangle$.

Since there doesn't exist $c_1, c_2 \in \mathcal{C}$ satisfying $\text{skip} = c_1; c_2$, this theorem is vacuously true.

case: $c = \text{if } ([b]^l, c_1, c_2)$

By the evaluation rule `query` and `if-f`, and the label consistency, we know:

for all possible c_{t1} and c_{t2} such that c_t has the form $c_t = c_{t1}; c_{t2}$;

all possible c_{f1} and c_{f2} such that c_f has the form $c_f = c_{f1}; c_{f2}$,

$c_{t1} \notin c_{t1}$ and $c_{f1} \notin c_{f2}$.

Then this theorem is vacuously true.

case: $c = c_{s1}; c_{s2}$

By label consistency, we know for every $c'_1 \in_c c_{s1}$, $c'_1 \notin c_{s2}$.

Then by the induction hypothesis on c_{s1} and c_{s2} separately, we have this case proved.

case: $\text{while } [b]^l \text{ do } c$

By rule `while-t`, we have:

$$\langle \text{while } [b]^l \text{ do } (c_w), \tau \rangle \rightarrow \langle c_w; \text{while } [b]^l \text{ do } (c_w), \tau_{++}[\epsilon] \rangle$$

If c_w is a sequence command, let $c_1 = c_{w1}$ be the any possible command in this sequence, for all possible c_{w1} and c_{w2} such that c_w has the form $c_w = c_{w1}; c_{w2}$.

Then we have $c_2 = c_{w2}; \text{while } [b]^l \text{ do } (c_w)$ and $c_1 \in_c c_2$.

And we also have the existence of $l = l_b, b$ and c_w , and $\text{while } [b]^l \text{ do } (c_w) \in_c c_2$ and $c_1 \in c_w$.

If c_w isn't a sequence command, let $c_1 = c_w$, then we have $c_2 = \text{while } [b]^l \text{ do } (c_w)$ and $c_1 \in_c c_2$.

And we also have the existence of $l = l_b, b$ and c_w , and $\text{while } [b]^l \text{ do } (c_w) \in_c c_2$ and $c_1 \in c_w$.

This case is proved.

By the evaluation rule `while-f`, we have

$$\langle \text{while } [b]^l \text{ do } (c_w), \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau_{++}[(b, l, \text{false})] \rangle.$$

Since there doesn't exist $c_1, c_2 \in \mathcal{C}$ satisfying $\text{skip} = c_1; c_2$, this theorem is vacuously true. □

A.4 SOUNDNESS OF THE WEIGHT ESTIMATION

A.4.1 Proof of Lemma 6.3.1

Lemma (Soundness of the Abstract Events Computation). *For any program c with an arbitrary initial trace $\tau_0 \in \mathcal{T}_0^+(c)$ and a finite execution trace $\tau \in \mathcal{T}^+$ that is generated when c is executing under τ_0 , then for every event $\epsilon \in \tau$ such that $\epsilon = (_, l, _, _)$ for some label l in program c , there is an abstract event $\hat{\epsilon} = (l, _, _) \in \text{abstrace}(c)$ and $\hat{\epsilon} = (l, _, _)$.*

$$\begin{aligned} \forall c, c' \in \mathcal{C}, \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, \epsilon = (_, l, _, _) \in \mathcal{E}, l \in \mathbf{L}(c) . \\ (\langle c, \tau_0 \rangle \rightarrow^* \langle c', \tau_{0++\tau} \rangle) \wedge \epsilon \in \tau \implies \exists \hat{\epsilon} \in \text{abstrace}(c) \wedge \hat{\epsilon} = (l, _, _) \end{aligned}$$

Proof. Taking an arbitrary program c , an intermediate program $c' \in \mathcal{C}$, an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, a finite execution trace $\tau \in \mathcal{T}^+$, a label $l \in \mathbf{L}(c)$ and an event $\epsilon = (_, l, _, _) \in \mathcal{E}$ such that

$$\langle c, \tau_0 \rangle \rightarrow^* \langle c', \tau_{0++\tau} \rangle$$

Then it is sufficient to show,

$$\epsilon \in \tau \implies \exists \hat{\epsilon} \in \text{abstrace}(c) \wedge \hat{\epsilon} = (l, _, _)$$

By induction on program c , we have the following cases:

case: $c = [x \leftarrow e]^{l'}$

By the evaluation rule `assn`, we have

$$\langle [x \leftarrow a]^{l'}, \tau_0 \rangle \rightarrow \langle [\text{skip}]^{l'}, \tau_0 :: (x, l', v, \bullet) \rangle$$

, for some $v \in \mathbb{N}$.

There are 2 cases, where $l' = l$ and $l' \neq l$.

In case of $l' \neq l$, we know $\epsilon \notin [(x, l', v, \bullet)]$, then this Lemma is vacuously true.

In case of $l' = l$, by the abstract execution trace computation in Definition 30, we have

$$\text{abstrace}(c) = \text{abstrace}'([x \leftarrow a]^{l'}; [\text{skip}]^{\text{ex}}) = \{(l', \text{absexpr}(e), \text{ex})\}$$

Let $\hat{e} = (l', \text{absexpr}(e), \text{ex})$, then we have $\hat{e} \in \text{abstrace}(c)$ and this case is proved.

case: $c = [x \leftarrow \text{query}(\psi)]^{l'}$

By the evaluation rule query

$$\frac{\langle \tau, \psi \rangle \Downarrow_q \alpha \quad \text{query}(\alpha) = v \quad \epsilon = (x, l', v, \alpha)}{\langle [x \leftarrow \text{query}(\psi)]^{l'}, \tau \rangle \rightarrow \langle [\text{skip}]^{l'}, \tau :: \epsilon \rangle},$$

let $v \in \mathbb{N}$ be the result of query request and α be the query value. Then we have

$$\langle [x \leftarrow \text{query}(\psi)]^{l'}, \tau_0 \rangle \rightarrow \langle [\text{skip}]^{l'}, \tau_0 :: [(x, l', v, \alpha)] \rangle.$$

There are 2 cases, where $l' = l$ and $l' \neq l$.

In case of $l' \neq l$, we know $\epsilon \notin [(x, l', v, \alpha)]$, then this Lemma is vacuously true.

In case of $l' = l$, by the abstract execution trace computation in Definition 30, we have

$$\text{abstrace}(c) = \text{abstrace}'([x \leftarrow \text{query}(\psi)]^{l'}; [\text{skip}]^{\text{ex}}) = \{(l', \text{absexpr}(e), \text{ex})\}$$

Let $\hat{e} = (l', \text{absexpr}(e), \text{ex})$, then we have $\hat{e} \in \text{abstrace}(c)$ and this case is proved.

case: while $[b]^{l_w}$ do (c_w)

By the first rule applied to c in operational semantics, there are two cases as follows.

sub-case: while-t

In this case, we have

$$\langle \text{while } [b]^{l_w} \text{ do } (c_w), \tau_0 \rangle \rightarrow \langle c_w; \text{while } [b]^{l_w} \text{ do } (c_w), \tau_0 :: (b, l_w, \text{true}, \bullet) \rangle.$$

In case of $l \neq l_w$, we know $\epsilon \notin [(b, l_w, \text{true}, \bullet)]$, and this Lemma is vacuously true.

In case of $l = l_w$, let $\epsilon = (b, l_w, \text{true}, \bullet)$ and $\tau' = [(b, l_w, \text{true}, \bullet)]$, then it is sufficient to show.

$$\exists \hat{\epsilon} \in \text{abstrace}(c) \wedge \hat{\epsilon} = (l, _, _)$$

By abstract trace computation in Definition 30, we have

$$\text{abstrace}(c) = \text{abstrace}'(c; [\text{skip}]^{\text{ex}}) = \{(l_w, \top, \text{init}(c_w))\} \cup \dots$$

Let $\hat{\epsilon} = (l_w, \top, \text{init}(c_w))$, then we have this case is proved.

sub-case: while-f

In this case, we have

$$\langle \text{while } [b]^{l_w} \text{ do } (c_w), \tau_0 \rangle \xrightarrow{\text{while-f}} \langle [\text{skip}]^{l_w}, \tau_0 :: (b, l_w, \text{false}, \bullet) \rangle.$$

In case of $l \neq l_w$, we know $\epsilon \notin [(b, l_w, \text{false}, \bullet)]$, and this Lemma is vacuously true.

In the case where $l = l_w$, let $\tau = [(b, l_w, \text{false}, \bullet)]$ and $\epsilon = (b, l_w, \text{false}, \bullet)$, then it is sufficient to show

$$\epsilon \in [(b, l_w, \text{false}, \bullet)] \implies \exists \hat{\epsilon} = (l, _, _) \in (\mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L}) . \hat{\epsilon} \in \text{abstrace}(c).$$

By abstract trace computation in Definition 30, we have

$$\text{abstrace}(c) = \text{abstrace}'(c; [\text{skip}]^{\text{ex}}) = \{(l_w, \top, \text{init}(c_w))\} \cup \dots$$

Let $\hat{e} = (l_w, \top, \text{init}(c_w))$, then we have this case is proved.

case: $\text{if } ([b]^l, c_t, c_f)$

By the first rule applied to $\text{if } ([b]^l, c_t, c_f)$ in operational semantics, we have two subcases if-t and if-f.

Then both of the two cases will append the evaluation result of the guard $[b]^l$, i.e., $(b, l, \text{true}, \bullet)$ and $(b, l, \text{false}, \bullet)$ respectively.

By abstract trace computation in Definition 30, we have

$$\begin{aligned} \text{abstrace}(c) &= \text{abstrace}'(c; [\text{skip}]^{\text{ex}}) \\ &= \text{abstrace}'(\text{if } [b]^l \text{ then } c_t \text{ else } c_f) \cup \text{abstrace}'([\text{skip}]^{\text{ex}}) \\ &\quad \cup \{(l, dc, \text{absinit}(c_2)) \mid (l, dc) \in \text{absfinal}(c_1)\} \\ &= \{(l, \text{absexpr}(b, \top), \text{absinit}(c_t)), (l, \text{absexpr}(\neg b, \top), \text{absinit}(c_f))\} \\ &\quad \cup \text{abstrace}'(c_t) \cup \text{abstrace}'(c_f) \cup \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1)\} \end{aligned}$$

Let $\hat{e} = (l, \text{absexpr}(b, \top), \text{absinit}(c_t))$ or $(l, \text{absexpr}(\neg b, \top), \text{absinit}(c_f))$ respectively in the two subcases.

Then we have both of the two subcases proved.

case: $c = c_1; c_2$

By the first rule applied to c in operational semantics, there are two cases as follows.

sub-case: seq1

In this case, we have the following where $\tau = \tau_1$.

$$\frac{\langle c_1, \tau_0 \rangle \rightarrow \langle c'_1, \tau_{0++\tau_1} \rangle}{\langle c_1; c_2, \tau_0 \rangle \rightarrow \langle c'_1; c_2, \tau_{0++\tau_1} \rangle}$$

By induction hypothesis on c_1 , we know

$$\epsilon \in \tau_1 \implies \exists \hat{\epsilon} \in \text{abstrace}(c_1) \wedge \hat{\epsilon} = (l, _, _)$$

Let $\epsilon = (_, l_1, _, _) \in \tau_1$ and $\hat{\epsilon}_1 = (l_1, _, _) \in \text{abstrace}(c_1)$, we know $\epsilon \in \tau$ as well.

Then it is sufficient to show

$$\exists \hat{\epsilon} \in \text{abstrace}(c) \wedge \hat{\epsilon} = (l_1, _, _)$$

By abstract trace computation in Definition 30, we have

$$\begin{aligned} \text{abstrace}(c_1) &= \text{abstrace}'(c_1; [\text{skip}]^{\text{ex}}) \\ &= \text{abstrace}'(c_1) \cup \text{abstrace}'([\text{skip}]^{\text{ex}}) \\ &\quad \cup \{(l, dc, \text{absinit}([\text{skip}]^{\text{ex}})) \mid (l, dc) \in \text{absfinal}(c_1)\} \\ &= \text{abstrace}'(c_1) \cup \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1)\} \end{aligned}$$

and

$$\begin{aligned} \text{abstrace}(c) &= \text{abstrace}'(c; [\text{skip}]^{\text{ex}}) \\ &= \text{abstrace}'(c_1; c_2) \cup \text{abstrace}'([\text{skip}]^{\text{ex}}) \\ &\quad \cup \{(l, dc, \text{absinit}([\text{skip}]^{\text{ex}})) \mid (l, dc) \in \text{absfinal}(c_1; c_2)\} \\ &= \text{abstrace}'(c_1) \cup \text{abstrace}'(c_2) \\ &\quad \cup \{(l, dc, \text{absinit}(c_2)) \mid (l, dc) \in \text{absfinal}(c_1)\} \cup \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1; c_2)\} \end{aligned}$$

Then we have two subcases where $\hat{e}_1 \in \text{abstrace}'(c_1)$ or $\hat{e}_1 \in \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1)\}$.

sub-case: $\hat{e}_1 \in \text{abstrace}'(c_1)$

Since $\text{abstrace}'(c_1) \subseteq \text{abstrace}(c)$, then we know $\hat{e}_1 \in \text{abstrace}(c)$.

Let $\hat{e} = \hat{e}_1$, then we have $\hat{e} \in \text{abstrace}(c)$ and this subcase is proved.

sub-case: $\hat{e}_1 \in \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1)\}$

In this case, we know $(l_1, _) \in \text{absfinal}(c_1)$.

Let $\hat{e} = (l_1, _, \text{absinit}(c_2))$, then we know

$$\hat{e} \in \{(l, dc, \text{absinit}(c_2)) \mid (l, dc) \in \text{absfinal}(c_1)\}.$$

Since $\{(l, dc, \text{absinit}(c_2)) \mid (l, dc) \in \text{absfinal}(c_1)\} \subset \text{abstrace}(c)$, then we have $\hat{e} \in \text{abstrace}(c)$ and this subcase is proved.

sub-case: seq2

In this case, we have the following where $\tau = \tau_2$.

$$\frac{\langle c_2, \tau_0 \rangle \rightarrow \langle c'_2, \tau_{0++\tau_2} \rangle}{\langle [\text{skip}]^l; c_2, \tau_0 \rangle \rightarrow \langle c_2, \tau_{0++\tau_2} \rangle}$$

By induction hypothesis on c_2 , we know

$$\epsilon \in \tau_2 \implies \exists \hat{e} \in \text{abstrace}(c_2) \wedge \hat{e} = (l, _, _)$$

Let $\epsilon = (_, l_2, _, _) \in \tau_2$ and $\hat{e}_2 = (l_2, _, _) \in \text{abstrace}(c_2)$, we know $\epsilon \in \tau$ as well.

Then it is sufficient to show

$$\exists \hat{e} \in \text{abstrace}(c) \wedge \hat{e} = (l_2, _, _)$$

By abstract trace computation in Definition 30, we have

$$\begin{aligned}
\text{abstrace}(c_2) &= \text{abstrace}'(c_2; [\text{skip}]^{\text{ex}}) \\
&= \text{abstrace}'(c_2) \cup \text{abstrace}'([\text{skip}]^{\text{ex}}) \\
&\quad \cup \{(l, dc, \text{absinit}([\text{skip}]^{\text{ex}})) \mid (l, dc) \in \text{absfinal}(c_2)\} \\
&= \text{abstrace}'(c_2) \cup \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_2)\}
\end{aligned}$$

and

$$\begin{aligned}
\text{abstrace}(c) &= \text{abstrace}'(c; [\text{skip}]^{\text{ex}}) \\
&= \text{abstrace}'(c_1; c_2) \cup \text{abstrace}'([\text{skip}]^{\text{ex}}) \\
&\quad \cup \{(l, dc, \text{absinit}([\text{skip}]^{\text{ex}})) \mid (l, dc) \in \text{absfinal}(c_1; c_2)\} \\
&= \text{abstrace}'(c_1) \cup \text{abstrace}'(c_2) \\
&\quad \cup \{(l, dc, \text{absinit}(c_2)) \mid (l, dc) \in \text{absfinal}(c_1)\} \cup \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1; c_2)\}
\end{aligned}$$

Then we have two subcases where $\hat{e}_2 \in \text{abstrace}'(c_2)$ or $\hat{e}_2 \in \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_2)\}$.

sub-case: $\hat{e}_2 \in \text{abstrace}'(c_2)$

Since $\text{abstrace}'(c_2) \subseteq \text{abstrace}(c)$, then we know $\hat{e}_2 \in \text{abstrace}(c)$.

Let $\hat{e} = \hat{e}_2$, then we have $\hat{e} \in \text{abstrace}(c)$ and this subcase is proved.

sub-case: $\hat{e}_2 \in \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_2)\}$

In this case, we know $(l_2, _) \in \text{absfinal}(c_2)$. By abstract continuation state computation in Section 12.2.2, we know

$$\text{absfinal}(c_1; c_2) = \text{absfinal}(c_2)$$

Then we know $(l_2, _) \in \text{absfinal}(c_1; c_2)$. Let $\hat{e} = \hat{e}_2$, then we know

$$\hat{e} = \hat{e}_2 \in \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1; c_2)\} \subset \text{abstrace}(c).$$

Then we have $\hat{e} \in \text{abstrace}(c)$ and this subcase is proved. $\square$

A.4.2 Proof of Lemma 6.3.2

Lemma (Uniqueness of the Abstract Events Computation). *For every program c and an execution trace $\tau \in \mathcal{T}^+$ that is generated w.r.t. an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, there is a unique abstract event $\hat{e} = (l, _, _) \in \text{abstrace}(c)$ for every assignment event $\epsilon \in \mathcal{E}^{\text{asn}}$ in the execution trace having the label l , i.e., $\epsilon = (_, l, _, _) \text{ and } \epsilon \in \tau$.*

$$\begin{aligned} \forall c \in \mathcal{C}, \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, l, l' \in \text{absV}(c), \epsilon = (_, l, _, _) \in \mathcal{E}^{\text{asn}}. \\ \langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \wedge \epsilon \in \tau \\ \implies \exists! \hat{e} = (l, _, _) \in (\mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L}). \hat{e} \in \text{abstrace}(c) \end{aligned}$$

Proof. This is proved trivially by induction on the program c . $\square$

A.4.3 Proof of Theorem 6.3.1

Preliminary Theorem from paper Sinn et al. (2017).

Theorem (Soundness of the Transition Bound). *For each program c and an edge $\hat{e} = (l, _, _) \in \text{absG}(c)$, if l is the label of an assignment command, then its path-insensitive transition bound $\text{TB}(\hat{e}, c)$ is a sound upper bound on the execution times of this assignment command in c .*

$$\begin{aligned} \forall c \in \mathcal{C}, l, l' \in c, \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^{+\infty}, v \in \mathbb{N}. \\ \left(\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \wedge \langle \text{TB}(\hat{e}, c), \tau_0 \rangle \Downarrow_a v \wedge \text{cnt}(\tau, l) \leq v \right) \end{aligned}$$

Theorem (Soundness of the Weight Estimation). *Let c be a program and $W_{\text{est}}(c)$ be its estimated weight set. Then, for every $(x^l, \hat{w}) \in W_{\text{est}}(c)$ and $(x^l, w) \in W_{\text{trace}}(c)$, we have for every possible $\tau_0 \in \mathcal{T}_0^+(c)$, $v \in \mathbb{N}$, if $\langle \tau_0, \hat{w} \rangle \Downarrow_e v$, then v is an upper bound on $w(\tau_0)$.*

$$\begin{aligned} \forall c \in \mathcal{C}, x^l \in \text{LV}(c), \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, v \in \mathbb{N}, l', (x^l, w) \in W_{\text{trace}}(c) . \\ \exists (x^l, \hat{w}) \in W_{\text{est}}(c) \wedge (\langle \tau_0, \hat{w} \rangle \Downarrow_e v \implies w(\tau_0) \leq v) \end{aligned}$$

Proof. Taking an arbitrary a program c with its program-based dependency graph $G_{\text{est}} = (V, E, W, Q)$, and an arbitrary pair of labeled variable and weights $(x^l, w) \in W_{\text{est}}$, and arbitrary $\tau, \tau' \in \mathbb{T}, v \in \mathbb{N}$ satisfying $\langle c, \tau \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{++\tau'} \rangle \wedge \langle \tau, w \rangle \Downarrow_e v$

By Definition of W_{est} in $G_{\text{est}}(c)$, we know $w = \text{absW}(l) = \max\{\text{TB}(\hat{e}) \mid \hat{e} = (l, _, _)\}$.

By Lemma 6.3.1, there exists an abstract event in $\text{abstrace}(c)$ of form $(\hat{e}) = (l, _, _)$, corresponding to the assignment command associated to labeled variable x^l .

Let $(\hat{e}) = (l, dc, l') \in \text{abstrace}(c)$ be this event for some dc and l' such that $(\hat{e}) = (l, dc, l') \in \text{abstrace}(c)$, by the last step of phase 2, we know $W_{\text{est}}(x^l) \triangleq \text{TB}(\hat{e})$. Then, it is sufficient to show:

$$\forall v \in \mathbb{N} . \langle \text{TB}(\hat{e}), \tau \rangle \Downarrow_e \text{cnt}(\tau', l) \leq v \text{TB}(\hat{e})$$

By definition of $\text{TB}(\hat{e})$:

$$\text{locb}(\hat{e})$$

$$\text{locb}(\hat{e}) \in \mathcal{SC}$$

$$\text{Incr}(\text{locb}(\hat{e})) + \sum \{\text{TB}(\hat{e}') \times \max(\text{VB}(a) + c, 0) \mid (\hat{e}', a, c) \in \text{re}(\text{locb}(\hat{e}))\} \quad \text{locb}(\hat{e}) \notin \mathcal{SC}$$

case: $\text{locb}(\hat{e}) \in \mathcal{SC}$

Proved by the soundness of Local bound in Lemma B.3.1.

case: $\text{locb}(\hat{e}) \notin \mathcal{SC}$

To show:

$$\begin{aligned} & \max \left\{ \text{cnt}(\tau')l \mid \forall \tau \in \mathcal{T}^+ . \langle c, \tau \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{++}\tau' \rangle \right\} \\ & \leq \text{Incr}(\text{locb}(\hat{e})) + \sum \{ \text{TB}(\hat{e}') \times \max(\text{VB}(a) + c, 0) \mid (\hat{e}', a, c) \in \text{re}(\text{locb}(\hat{e})) \} \end{aligned}$$

Taking an arbitrary initial trace $\tau_0 \in \mathcal{T}^+$, executing c with τ_0 , let τ be the trace after evaluation, i.e., $\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau \rangle$, it is sufficient to show:

$$\text{cnt}(\tau')l \leq \text{Incr}(\text{locb}(\hat{e})) + \sum \{ \text{TB}(\hat{e}') \times \max(\text{VB}(a) + c, 0) \mid (\hat{e}', a, c) \in \text{re}(\text{locb}(\hat{e})) \}$$

By the soundness of the (1) Transition Bound and (2) Variable Bound Invariant in Theorem 1 by Sinn et al. (2017) (attached above in Theorem 6.3.1), This case is proved. $\square$

Lemma A.4.1 (Soundness of the Local Bound). *Given a program c , we have:*

$$\forall \hat{e} = (l, dc, l') . \max \left\{ \text{cnt}(\tau')l \mid \forall \tau \in \mathcal{T}^+ . \langle c, \tau \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{++}\tau' \rangle \right\} \leq \text{locb}(\hat{e})$$

Proof.

sub-case: $l \notin \text{SCC}(\text{absG}(c))$

In this case, we know variable x^l isn't involved in the body of any `while` command.

Taking an arbitrary $\tau_0 \in \mathcal{T}^+$, let $\tau \in \mathcal{T}^+$ be of resulting trace of executing c with τ , i.e., $\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau \rangle$,

we know the assignment command at line l associated with the abstract event $\hat{e}$ will be executed at most once, i.e., $\text{cnt}(\tau)l \leq 1$

By locb definition, we know $\text{locb}(\hat{e}) = 1$.

This case is proved.

sub-case: $l \in SCC(\text{absG}(c)) \wedge \hat{e} \in \text{dec}(x)$

in this case, we know $\text{locb}(\hat{e}) \triangleq x$.

sub-case: $l \in SCC(\text{absG}(c)) \wedge \hat{e} \notin \bigcup_{x \in VAR} \text{dec}(x) \wedge \hat{e} \notin SCC(\text{absG}(c)/\text{dec}(x))$

in this case, we know $\text{locb}(\hat{e}) \triangleq x$.

In the two cases above, the soundness is discussed by Sinn et al. (2017) in the Paragraph *Discussion on Soundness* in Section 4, Page 25. □

A.5 SOUNDNESS OF ADAPTSEARCH ALGORITHM

Theorem (Soundness of AdaptSearch). *For every well-formed program c , $\text{AdaptSearch}(c)$ computes an upper bound on the adaptivity of program c .*

$$\forall \tau_0 \in \mathcal{T}_0^+(c), v \in (\mathbb{N} \cup \{\infty\}) . \langle \text{AdaptSearch}(c), \tau_0 \rangle \Downarrow_e v \implies A(c)(\tau_0) \leq v.$$

Proof. For an arbitrary well-formed program c , let $\mathbf{G}_{\text{trace}}(c) = (\mathbf{V}_{\text{trace}}, \mathbf{E}_{\text{trace}}, \mathbf{W}_{\text{trace}}, \mathbf{Q}_{\text{trace}})$ be its semantics-based dependency graph by Definition 20 and

$$\mathbf{G}_{\text{est}}(c) = (\mathbf{V}_{\text{est}}, \mathbf{E}_{\text{est}}, \mathbf{W}_{\text{est}}, \mathbf{Q}_{\text{est}})$$

be its estimated-dependency graph by Definition 38.

According to the Definition 41 and Definition 23, it is sufficient to show:

$$\begin{aligned} & \forall \tau_0 \in \mathcal{T}_0^+(c), v \in \mathbb{N} \cup \{\infty\} . \langle \text{AdaptSearch}(c), \tau_0 \rangle \Downarrow_e v \\ & \implies \lambda \tau_0 . \max \{ \text{len}^q(k)(\tau_0) \mid k \in \mathcal{WK}(\mathbf{G}_{\text{trace}}(c)) \}(\tau_0) \leq v \end{aligned}$$

Let $\tau_0 \in \mathcal{T}_0^+(c)$ be an arbitrary initial trace of c , then it is sufficient to show that:

$$\begin{aligned} & \forall v \in \mathbb{N} \cup \{\infty\} . \langle \text{AdaptSearch}(c), \tau_0 \rangle \Downarrow_e v \\ & \implies \max \{ \text{len}^q(k)(\tau_0) \mid k \in \mathcal{WK}(\mathbf{G}_{\text{trace}}(c)) \} \leq v \end{aligned}$$

Equally, it is sufficient to show

$$\begin{aligned} & \forall v \in \mathbb{N} \cup \{\infty\}, k \in \mathcal{WK}(\mathbf{G}_{\text{trace}}(c)) . \\ & \langle \text{AdaptSearch}(c), \tau_0 \rangle \Downarrow_e v \implies \text{len}^q(k)(\tau_0) \leq v \end{aligned}$$

Let $k_{x,y} \in \mathcal{WK}(\mathbf{G}_{\text{trace}}(c))$ be an arbitrary finite walk in $\mathbf{G}_{\text{trace}}(c)$ and $k_{x,y}(\tau_0)$ has the

edge sequence

$$(e_1, \dots, e_{(n-1)}),$$

and the vertices sequence

$$(x, v_1, \dots, v_{n-1}, y).$$

Then it is sufficient to show

$$\forall v \in \mathbb{N} \cup \{\infty\}, \langle \text{AdaptSearch}(c), \tau_0 \rangle \Downarrow_e v \implies \text{len}^q(k)(\tau_0) \leq v$$

By line:3 of $\text{AdaptSearch}(G_{\text{est}})$ in Algorithm 3, let $G^{\text{SCC}}_1, \dots, G^{\text{SCC}}_m$ be all the strongly connected components on G_{est} with $0 \leq n \leq |V|$, where

$$G^{\text{SCC}}_i = (V_i, E_i, W_i, Q_i),$$

for every $i = 1, \dots, m$.

By line:5-6 of $\text{AdaptSearch}(G_{\text{est}})$ in Algorithm 3, let $\text{adapt}_{\text{scc}}[G^{\text{SCC}}_i]$ be the result of $\text{AdaptSearch}_{\text{scc}}(G^{\text{SCC}}_i)$ for each G^{SCC}_i .

There are 2 cases:

case: vertices x, y are on the same strongly connected component

Let G^{SCC}_j be this component for some $j \in \{1, \dots, m\}$ and

$$\text{AdaptSearch}_{\text{scc}}(G^{\text{SCC}}_j)(c) = \text{adapt}_{\text{scc}}[G^{\text{SCC}}_j].$$

By line:15 and line:18 in $\text{AdaptSearch}(G_{\text{est}})$ algorithm in Algorithm 3, let adapt be the output value, we know

$$\text{AdaptSearch}(G_{\text{est}}(c)) \geq \text{adapt}_{\text{scc}}[G^{\text{SCC}}_j] \wedge \text{AdaptSearch}_{\text{scc}}(G^{\text{SCC}}_j)(c) = \text{adapt}_{\text{scc}}[G^{\text{SCC}}_j].$$

Then it is sufficient to show

$$\begin{aligned} \forall v \in \mathbb{N} \cup \{\infty\} . \\ \langle \text{AdaptSearch}_{\text{scc}}(\mathbf{G}^{\text{scc}}_j)(c), \tau_0 \rangle \Downarrow_e v \implies \text{len}^q(k)(\tau_0) \leq v \end{aligned}$$

By Lemma A.5.1, we know

$$\begin{aligned} \forall v \in \mathbb{N} \cup \{\infty\} . \langle \text{AdaptSearch}_{\text{scc}}(\mathbf{G}^{\text{scc}})(c), \tau_0 \rangle \Downarrow_e v \\ \implies \max\{\text{len}^q(k)(\tau_0) | k \in \mathcal{WK}(\mathbf{G}^{\text{scc}})\} \leq v \end{aligned}$$

Since $\mathbf{G}^{\text{scc}}_j$ is we know

$$\begin{aligned} \forall v \in \mathbb{N} \cup \{\infty\} . \\ \langle \text{AdaptSearch}_{\text{scc}}(\mathbf{G}^{\text{scc}}_j)(c), \tau_0 \rangle \Downarrow_e v \implies \max\{\text{len}^q(k)(\tau_0) | k \in \mathcal{WK}(\mathbf{G}^{\text{scc}})\} \leq v \end{aligned}$$

Since we know

$$\text{len}^q(k)(\tau_0) \leq \max\{\text{len}^q(k)(\tau_0) | k \in \mathcal{WK}(\mathbf{G}^{\text{scc}})\}$$

Then we know

$$\begin{aligned} \forall v \in \mathbb{N} \cup \{\infty\} . \\ \langle \text{AdaptSearch}_{\text{scc}}(\mathbf{G}^{\text{scc}}_j)(c), \tau_0 \rangle \Downarrow_e v \implies \text{len}^q(k)(\tau_0) \leq v \end{aligned}$$

and we have this case proved.

case: Vertices x, y are on different strongly connected components

Let $\mathbf{G}^{\text{scc}}_x, \mathbf{G}^{\text{scc}}_1, \dots, \mathbf{G}^{\text{scc}}_m, \mathbf{G}^{\text{scc}}_y, 0 \leq m$ be all the strongly connected components that the walk $k_{x,y}$ is passed by.

Since each $\mathbf{G}^{\text{scc}}_i$ is strongly connected, we also know every two $\mathbf{G}^{\text{scc}}_i$ and $\mathbf{G}^{\text{scc}}_j$ where $i \neq j$ are single direct connected.

Then we can divide this walk into $m + 2$ sub-walks:

$$k_x = (x, s_1, \dots, s_{scc_x});$$

$$k_1 = (s_{scc_x}, \dots, s_{scc_1});$$

...

$$k_y = (s_{scc_m}, \dots, s_y);$$

where $k_x \in \mathcal{WK}(\mathcal{G}^{scc}_x), \dots, k_y \in \mathcal{WK}(\mathcal{G}^{scc}_y)$.

Let m_i be the evaluation results of $\text{AdaptSearch}_{scc}(\mathcal{G}^{scc}_i)$ for every $\mathcal{G}^{scc}_i$ as follows,

$$\langle \text{AdaptSearch}_{scc}(\mathcal{G}^{scc}_i), \tau_0 \rangle \Downarrow_e m_i$$

By Lemma A.5.1, we know for each walk k_i :

$$\text{len}^q(k_i)(\tau_0) \leq \max\{\text{len}^q(k_i)(\tau_0) | k_i \in \mathcal{WK}(\mathcal{G}^{scc}_i)\} \leq m_i$$

By Definition 22, we also have,

$$\text{len}^q(k_{x,y})(\tau_0) = \sum_{k_i \in \{k_x, \dots, k_y\}} \text{len}^q(k_i)(\tau_0) \leq \sum_{k_i \in \{k_x, \dots, k_y\}} m_i$$

Then it is sufficient to show

$$\forall v \in \mathbb{N} \cup \{\infty\} . \langle \text{AdaptSearch}(c), \tau_0 \rangle \Downarrow_e v \implies \sum_{k_i \in \{k_x, \dots, k_y\}} m_i \leq v$$

By Algorithm, we know

$$\begin{aligned} \forall i = 1, \dots, m . \text{AdaptSearch}_{scc}(\mathcal{G}^{scc}_i)(c) &= \text{adapt}_{scc}[\mathcal{G}^{scc}_i] \\ \wedge \text{AdaptSearch}(c) &\geq \sum_{k_i} \text{adapt}_{scc}[\mathcal{G}^{scc}_i] \end{aligned}$$

Then we know

$$\forall v \in \mathbb{N} \cup \{\infty\} . \langle \text{AdaptSearch}(c), \tau_0 \rangle \Downarrow_e v \implies v \geq \sum_{k_i} m_i$$

This case is proved. □

Lemma A.5.1 (Soundness of AdaptSearch_{scc}). *For any well-formed program c with its estimated dependency graph G_{est} , for any strongly connected component, G^{SCC} on this graph, $\text{AdaptSearch}_{scc}(G^{\text{SCC}})$ computes a sound upper bound on the length of the longest walk on G^{SCC} .*

$$\forall c \in \mathcal{C}, G^{\text{SCC}} \subseteq_{\text{graph}} G_{\text{est}}(c), \tau_0 \in \mathcal{T}_0^+(c), m \in \mathbb{N} \cup \{\infty\} .$$

$$\langle \text{AdaptSearch}_{scc}(G^{\text{SCC}})(c), \tau_0 \rangle \Downarrow_e m \implies \max\{\text{len}^q(k)(\tau_0) | k \in \mathcal{WK}(G^{\text{SCC}})\} \leq m$$

The proof summary informally:

1. (1) for each node x on SCC, by property of SCC, for every walk on SCC $k_{x,x} = (x, s_1, \dots, x)$, with set of unique vertex $\{v_1, \dots, x\}$ there are $\mathcal{PATH}(p_{x,x})$ on G^{SCC} .
2. (2) For every path $p_{x,x}^i = (x, v_1, \dots, x) \in \mathcal{PATH}(p_{x,x})$, $\text{flowcapacity}(p_{x,x}^i)$ is the maximum visiting times for every $v \in (x, v_1, \dots, x)$. So we know $\text{visit}(s)(s_1, \dots, x) \leq \text{flowcapacity}(p_{x,x}^i)$.
3. (3) $\text{querynum}(p_{x,x}^i) * \text{flowcapacity}(p_{x,x}^i) \geq \text{len}(s | s \in (s_1, \dots, x) \wedge Q(s) = 1) = \text{len}^q(k)$,
4. (4) Then, the $\max_{p_{x,x}^i \in \mathcal{PATH}(p_{x,x})} \geq \max\{\text{len}^q(k_{x,x}) | k_{x,x} \in \mathcal{WK}(k_{x,x})\}$
5. (5) Then, $\max\{\text{querynum}(p_{x,x}^i) * \text{flowcapacity}(p_{x,x}^i) | x \in G^{\text{SCC}} \wedge p_{x,x}^i \in \mathcal{PATH}(p_{x,x})\} \geq \max\{\text{len}^q(k_{x,x}^i) | x \in G^{\text{SCC}} \wedge k_{x,x}^i \in \mathcal{WK}(k_{x,x})\}$

6. (6) We also know by the property of SCC, $\forall x, y \in G^{\text{SCC}}$, let $k_{x,y}$ be arbitrary walk on G^{SCC} , $\text{len}^q(k_{x,y}) \leq \max\{\text{len}^q(k_{x,x}^i) | k_{x,x}^i \in \mathcal{WK}(k_{x,x})\}$.
7. (7) Then, $\max\{\text{len}^q(k_{x,x}^i) | x \in G^{\text{SCC}} \wedge k_{x,x}^i \in \mathcal{WK}(k_{x,x})\} \geq \max\{\text{len}^q(k_{x,y}^i) | x, y \in G^{\text{SCC}} \wedge k_{x,y}^i \in \mathcal{WK}(k_{x,y})\}$ i.e., $\max\{\text{len}^q(k_{x,x}^i) | x \in G^{\text{SCC}} \wedge k_{x,x}^i \in \mathcal{WK}(k_{x,x})\} \geq \max\{\text{len}^q(k) | k \in \mathcal{WK}(G^{\text{SCC}})\} = A_{\text{est}}(G^{\text{SCC}})$.
8. (8) We also know $\text{AdaptSearch}_{\text{scc}}(G^{\text{SCC}}) = \max\{\text{querynum}(p_{x,x}^i) * \text{flowcapacity}(p_{x,x}^i) | x \in G^{\text{SCC}} \wedge p_{x,x}^i \in \mathcal{PATH}(p_{x,x})\}$ by the $\text{AdaptSearch}_{\text{scc}}$ algorithm.

Then we have $\text{AdaptSearch}_{\text{scc}}(G^{\text{SCC}}) \geq A_{\text{est}}(G^{\text{SCC}})$

Proof. Taking arbitrary program $c \in \mathcal{C}$, let $G_{\text{est}}(c) = (V, E, W, Q)$ be its estimated dependency graph and $G^{\text{SCC}} = (V_{\text{scc}}, E_{\text{scc}}, W_{\text{scc}}, Q_{\text{scc}})$ be an arbitrary strongly connected component of G_{est} .

There are 2 cases:

case: G^{SCC} contains no edge and only 1 vertex $v \in V_{\text{scc}}$, i.e., $|E_{\text{scc}}| = 0 \wedge |V_{\text{scc}}| = 1$

In this case there is no walk in this graph, i.e., $\mathcal{WK}(G^{\text{SCC}}) = \emptyset$. The adaptivity is at most 1 if $(v, 1) \in Q_{\text{est}}(c)$.

This case is proved.

case: G^{SCC} contains at least 1 edge and at least 1 vertex v , i.e., $1 \leq |E| \wedge 1 \leq |V|$

Taking arbitrary walk $k_{x,y} \in \mathcal{WK}(G^{\text{SCC}})$, and an arbitrary initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, such that $k_{x,y}(\tau_0)$ has the vertices sequence $k_{x,y}(\tau_0) = (x, s_1, \dots, y)$. By Definition 22, we know

$$\text{len}^q(k_{x,y})(\tau_0) = |(s | s \in (x, s_1, \dots, y) \wedge (s, 1) \in Q_{\text{trace}}(c))|$$

Then, it is sufficient to show:

$$\begin{aligned} \forall m \in \mathbb{N} \cup \{\infty\} . \langle \text{AdaptSearch}_{scc}(\mathbf{G}^{\text{SCC}})(c), \tau_0 \rangle \Downarrow_e m \\ \implies |(s | s \in (x, s_1, \dots, y) \wedge (s, 1) \in \mathbf{Q}_{\text{trace}}(c))| \leq m \end{aligned}$$

Let $M_{\text{AdaptSearch}_{scc}(\mathbf{G}^{\text{SCC}})(c), \tau_0} \in \mathbb{N} \cup \{\infty\}$ be the evaluation result of $\langle \text{AdaptSearch}_{scc}(\mathbf{G}^{\text{SCC}})(c), \tau_0 \rangle$, i.e.,

$$\langle \text{AdaptSearch}_{scc}(\mathbf{G}^{\text{SCC}})(c), \tau_0 \rangle \Downarrow_e M_{\text{AdaptSearch}_{scc}(\mathbf{G}^{\text{SCC}})(c), \tau_0}$$

Then, it is sufficient to show:

$$|(s | s \in (x, s_1, \dots, y) \wedge (s, 1) \in \mathbf{Q}_{\text{trace}}(c))| \leq M_{\text{AdaptSearch}_{scc}(\mathbf{G}^{\text{SCC}})(c), \tau_0}$$

By $\text{AdaptSearch}_{scc}(\mathbf{G}^{\text{SCC}})$ algorithm line 19, in the iteration where x is the starting vertex, we know $\text{AdaptSearch}_{scc}(\mathbf{G}^{\text{SCC}}) = \mathbf{r}_{scc} = \max(\mathbf{r}_{scc}, \text{dfs}(\mathbf{G}^{\text{SCC}}, \mathbf{x}, \text{visited}))$, then it is sufficient to show:

$$|(s | s \in (x, s_1, \dots, y) \wedge (s, 1) \in \mathbf{Q}_{\text{trace}}(c))| \leq M_{(\text{dfs}(\mathbf{G}^{\text{SCC}}, \mathbf{x}, \text{visited}), \tau_0)}.$$

where $M_{(\text{dfs}(\mathbf{G}^{\text{SCC}}, \mathbf{x}, \text{visited}), \tau_0)}$ is the evaluation result of $\langle \text{dfs}(\mathbf{G}^{\text{SCC}}, \mathbf{x}, \text{visited}), \tau_0 \rangle$ as well.

For convenience, in the following proof, we use notation $M_{(e, \tau_0)}$ to represent the evaluation result of e under trace τ_0 .

Let $\{v_1, \dots, x\}$ be the set of all the distinct vertices of $k_{x,y}$'s vertices sequence $(x, s_1, \dots, y)$, and $(v_1, \dots, x)$ be a subsequence containing all the vertices in $\{x, v_1, \dots, y\}$. By the definition of walk, there is a path $p_{x,y}$ from x to y with this vertices sequence: $(x, v_1, \dots, y)$.

By line:13 of the $\text{dfs}(\mathbf{G}^{\text{SCC}}, \mathbf{x}, \text{visited})$ in Algorithm 4,

we know $\text{dfs}(\mathcal{G}^{\text{SC}}, x, \text{visited}) = r[x]$ and $r[x] = \max\{\text{flowcapacity}(p) \times \text{querynum}(p) \mid p \in \mathcal{PATH}_{x,x}(\mathcal{G}^{\text{SC}})\}$, where $\mathcal{PATH}_{x,x}(\mathcal{G}^{\text{SC}})$ is a subset of $\mathcal{PATH}_{x,x}(\mathcal{G}^{\text{SC}})$, in which every path starts from x and goes back to x .

By the property of strong connected graph, we know in this case $\mathcal{PATH}_{x,x}(\mathcal{G}^{\text{SC}}) \neq \emptyset$ and there are 2 cases, $x = y$ and $x \neq y$.

case: $x = y$

In this case, we know $p_{x,y} \in p \in \mathcal{PATH}_{x,x}(\mathcal{G}^{\text{SC}})$, then it is sufficient to show:

$$|(s \mid s \in (x, s_1, \dots, y) \wedge (s, 1) \in \mathcal{Q}_{\text{trace}}(c))| \leq M_{(\text{flowcapacity}(p_{x,y}) \times \text{querynum}(p_{x,y}))}$$

By line:7 and line:13 in Algorithm 4, we know $\text{flowcapacity}(p_{x,y})$ is the maximum visiting times for every $v \in (x, v_1, \dots, y)$, we know for every s in the vertices sequence of walk $k_{x,y}$, $\text{visit}(s)(x, s_1, \dots, y) \leq \text{flowcapacity}(p_{x,y})$

By line:8 and line:13 in Algorithm 4, we know $\text{querynum}(p_{x,y})$ is the number of vertices with $\mathcal{Q}$ equal to 1. Then we know

$$|(s \mid s \in (x, s_1, \dots, y) \wedge (s, 1) \in \mathcal{Q}_{\text{trace}}(c))| \leq M_{(\text{flowcapacity}(p_{x,y}) \times \text{querynum}(p_{x,y}))}.$$

This case is proved.

case: $x \neq y$

we also have a path start from y and go back to x .

Let $p_{y,x}$ be this path with vertices sequence $(y, v'_1, \dots, x)$, we have a path $p_{x,x}$ which is the path $p_{x,y}$ concatenated by path $p_{y,x}$ with vertices sequence $(x, v_1, \dots, y, v'_1, \dots, v'_m, x)$, where $m \geq 0$.

Then in this case, it is sufficient to show:

$$|(s \mid s \in (x, s_1, \dots, y) \wedge (s, 1) \in \mathcal{Q}_{\text{trace}}(c))| \leq M_{(\text{flowcapacity}(p_{x,x}) \times \text{querynum}(p_{x,x}, \tau_0))} = M_{(r[x], \tau_0)}$$

Since $\text{flowcapacity}(p_{x,y} + p_{y,x})$ is the maximum visiting times for every v such that $v \in (x, v_1, \dots, y, v'_1, \dots, x)$,

By line:7 in Algorithm 4, we know $\text{flowcapacity}(p_{x,y})$ is the maximum visiting times for every $v \in (x, v_1, \dots, y)$,

we know for every s in the vertices sequence of walk $k_{x,y}$, $\text{visit}(s)(x, s_1, \dots, y) \leq \text{flowcapacity}(p_{x,y})$

Also by line:8 in Algorithm 4, we know $\text{querynum}(p_{x,y})$ is the number of vertices with Q equal to 1,

Then we know

$$|(s | s \in (x, s_1, \dots, y) \wedge (s, 1) \in Q_{\text{trace}}(c))| \leq M_{(\text{flowcapacity}(p_{x,y}) \times \text{querynum}(p_{x,y}), \tau_0)} = M_{(r[y], \tau_0)}$$

By line:13, we also know $r[x] = \max(r[x], r[v'_m] + \text{flowcapacity}(p_{x,x}) \times \text{querynum}(p_{x,x}))$, and $r[y] \leq r[v'_m]$ then we know $M_{(r[y], \tau_0)} \leq M_{(r[x], \tau_0)}$, i.e.,

$$|(s | s \in (x, s_1, \dots, y) \wedge (s, 1) \in Q_{\text{trace}}(c))| \leq M_{(r[x], \tau_0)}$$

This case is proved. □

A.6 CONDITIONAL COMPLETENESS OF ADAPTSEARCH ALGORITHM

Theorem A.6.1 (Conditional Completeness of AdaptSearch). *For any well-formed program c with its estimated dependency graph $G_{\text{est}}(c)$, if $G_{\text{est}}(c)$ is acyclic directed, then $\text{AdaptSearch}(c)$ computes a tight bound on the length of the longest walk on $G_{\text{est}}(c)$.*

$$\forall c \in \mathcal{C}, G^{\text{SCC}} \subseteq_{\text{graph}} G_{\text{est}}(c), \tau_0 \in \mathcal{T}_0^+(c), m \in \mathbb{N} \cup \{\infty\} .$$

$$\langle \text{AdaptSearch}(c), \tau_0 \rangle \Downarrow_e m \implies \max\{\text{len}^q(k)(\tau_0) \mid k \in \mathcal{WK}(G_{\text{est}})\} = m$$

Proof. Taking arbitrary program $c \in \mathcal{C}$, let $G_{\text{est}}(c) = (V_{\text{est}}, E_{\text{est}}, W_{\text{est}}, Q_{\text{est}})$ be its estimated dependency graph.

Taking an arbitrary initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, let the walk $k_{\text{max}} \in \mathcal{WK}(G_{\text{est}}(c))$ be the finite walk with the longest query length, and the vertices sequence $k_{\text{max}}(\tau_0) = (s_1, \dots, s_n)$, it is sufficient to show:

$$\text{len}^q(k_{\text{max}})(\tau_0) = |(s \mid s \in (s_1, \dots, s_n) \wedge (s, 1) \in Q_{\text{est}}(c))| = \text{AdaptSearch}(G_{\text{est}}(c))$$

We denote by $M_{\langle e, \tau_0 \rangle}$ the evaluation result of $\langle e, \tau_0 \rangle \Downarrow_a$ for arbitrary expression e under trace τ_0 .

In order to show the completeness, it is sufficient to show two following two sub goals,

1. By line: 15, $\text{AdaptSearch}(G_{\text{est}}(c))$ can find a path p_{max} such that

$$M_{\langle \text{adapt}_{\text{pmax}}, \tau_0 \rangle} = \text{len}^q(k_{\text{max}})(\tau_0)$$

2. $\text{AdaptSearch}(G_{\text{est}}(c))$ finds this longest path $p_{x,y}$ and $\text{adapt}_{\text{pmax}}$ is returned as the final output.

By the property of an acyclic directed graph, we know every $s_i \in (s_1, \dots, s_n)$

shows up exactly once. Then we know this walk is a path and

$$\text{len}^q(k_{max})(\tau_0) = |(s|s \in (s_1, \dots, s_n) \wedge (s, 1) \in Q_{\text{est}}(c))|$$

By line: 13, through searching on all the vertices connected on $G_{\text{est}}(c)$ from the starting node s_i , by the soundness of the breath first search strategy, we know that $\text{AdaptSearch}(G_{\text{est}}(c))$ finds this path $p_{max} = (s_1, \dots, s_n)$ and

$$\text{AdaptSearch}(G_{\text{est}}(c)) = \text{adapt}_{p_{max}}$$

So we have (2) proved.

Then, to show (1), it is sufficient to show

$$M_{\langle \text{adapt}_{p_{max}}, \tau_0 \rangle} = |(s|s \in (s_1, \dots, s_n) \wedge (s, 1) \in Q_{\text{est}}(c))|.$$

By line: 15, let $G^{\text{SCC}}_1, \dots, G^{\text{SCC}}_m$ be all the strongly connected components, where each vertex in $(s_1, \dots, s_n)$ belongs to, we know

$$\text{adapt}_{p_{max}} = \sum_{G^{\text{SCC}}_i \in (G^{\text{SCC}}_1, \dots, G^{\text{SCC}}_m)} \text{adapt}_{\text{scc}}[G^{\text{SCC}}_i].$$

By line:3 in Algorithm 3, let $G^{\text{SCC}}_i = (V_i, E_i, W_i, Q_i)$ for $G^{\text{SCC}}_i \in (G^{\text{SCC}}_1, \dots, G^{\text{SCC}}_m)$ be the SCC found by the standard Algorithm.,

Then, by the property of ACG, we know every G^{SCC}_i is a single vertex v_i without edge.

Then we know $\text{adapt}_{\text{scc}}[G^{\text{SCC}}_i] = 1$ if $(s_i, 1) \in Q_{\text{est}}(c)$ o.w. 0.

So we know

$$\begin{aligned}
& \sum_{\mathbf{g}^{\text{SCC}}_i \in (\mathbf{g}^{\text{SCC}}_1, \dots, \mathbf{g}^{\text{SCC}}_m)} \text{adapt}_{\text{SCC}}[\mathbf{g}^{\text{SCC}}_i] \\
&= \sum_{s_i \mid s_i \in (s_1, \dots, s_n) \wedge (s, 1) \in \mathbf{Q}_{\text{est}}(c)} 1 \\
&= |(s \mid s \in (s_1, \dots, s_n) \wedge (s, 1) \in \mathbf{Q}_{\text{est}}(c))|
\end{aligned}$$

Then we can conclude:

$$\begin{aligned}
& M_{\langle \text{adapt}_{\text{pmax}}, \tau_0 \rangle} \\
&= M_{\langle \sum_{\mathbf{g}^{\text{SCC}}_i \in (\mathbf{g}^{\text{SCC}}_1, \dots, \mathbf{g}^{\text{SCC}}_m)} \text{adapt}_{\text{SCC}}[\mathbf{g}^{\text{SCC}}_i], \tau_0 \rangle} \\
&= |(s \mid s \in (s_1, \dots, s_n) \wedge (s, 1) \in \mathbf{Q}_{\text{est}}(c))|
\end{aligned}$$

Then, we have (1) proved. □

A.7 THE PROGRAMS IN EVALUATION TABLE

Below are the example programs in the evaluation table ?? from line 6 to line 15.

Example A.7.1 (The Complete Gradient Decent Optimization Algorithm). *This example is the gradient decent algorithm example is a generalization of the linear regression on a higher degree data relation. It uses gradient decent algorithm to minimize the mean square loss function for a two-degree relation $y = a_1 \times x_1^2 + a_2 \times x_2 + c$ on the dataset of two feature columns and one indicator column.*

```

gradientDecent(step, rate, t, n)  $\triangleq$ 
   $[a_1 \leftarrow 0]^0$ ;
   $[a_2 \leftarrow 0]^1$ ;
   $[c \leftarrow 0]^2$ ;
   $[j \leftarrow \text{step}]^3$ ;
  while  $[j > 0]^4$  do
    ( $[da1 \leftarrow \text{query}(-2 * (\chi[2] - (\chi[0]^2 \times a_1 + \chi[1] \times a_2 + c)) \times (\chi[0]))]^5$ ;
     $[da2 \leftarrow \text{query}(-2 * (\chi[2] - (\chi[0]^2 \times a_1 + \chi[1] \times a_2 + c)) \times (\chi[1]))]^6$ ;
     $[dc \leftarrow \text{query}(-2 * (\chi[2] - (\chi[0]^2 \times a_1 + \chi[1] \times a_2 + c)))]^5$ ;
     $[a_1 \leftarrow a_1 - \text{rate} * da1]^7$ ;
     $[a_2 \leftarrow a_2 - \text{rate} * da2]^8$ ;
     $[c \leftarrow c - \text{rate} * dc]^9$ ;
     $[j \leftarrow j - 1]^{10}$ );

```

This approach can be generalized to the regression of a variety of relations in machine learning area.

Example A.7.2 (Sequence with Linear Query Dependency). *This example algorithm contains only sequence of four query commands. Each of them depends on a previous query.*

The longest dependency depth, i.e., the adaptivity is expectation to be 4.

$$\text{seq}() \triangleq \begin{array}{l} [x \leftarrow \chi[0]]^0; [y \leftarrow \chi[x+1]]^1; \\ [z \leftarrow \chi[y+1]]^2; [w \leftarrow \chi[z+1]]^3 \end{array}$$

Evaluation Result: $A_{\text{est}}(\text{seq}()) = 4$

Example A.7.3 (Sequence with Query Dependency between Related Variables).

This example algorithm contains a sequence of four query commands. Each of them depends on one or more of the previous queries. The longest dependency depth, i.e., the adaptivity is expectation to be 4.

$$\text{seqRV}() \triangleq \begin{array}{l} [x \leftarrow \chi[0]]^0; [y \leftarrow \chi[x+1]]^1; \\ [z \leftarrow \chi[y+x]]^2; [w \leftarrow \chi[z+1] \cdot \chi[y]]^3 \end{array}$$

Evaluation Result: $A_{\text{est}}(\text{seqMultiVar}()) = 4$

Example A.7.4 (If with Data-Value Dependency Separated). This example algorithm contains a `if` command and a query requests in each branch. Only the query in the first branch depend on the query in the command 0, and the variable in the guard is not assigned by a query request. The longest dependency depth, i.e., the adaptivity is expectation to be 3.

$$\text{ifVD}(k) \triangleq \begin{array}{l} [z \leftarrow \text{query}(\chi[0])]^0; [x \leftarrow k/2]^1; \\ \text{if } ([x < 0])^2, [y \leftarrow \text{query}(\chi[z])]^3, [y \leftarrow \text{query}(\chi[0])]^4) \end{array}$$

Evaluation Result: $A_{\text{est}}(\text{ifVD}()) = 3$

Example A.7.5 (If with Data-Control Dependency Overlapped). This example algorithm contains a `if` command and a query requests in each branch. The variable in the guard is assigned by a query request in command 1. The two queries in the branches de-

pend on the second query in command 1 but not depend on the query in the command 0. Even though the variable x isn't used in the query expression in the query 3 and 4, there are still dependency relation because x is in the guard. The longest dependency depth, i.e., the adaptivity is expectation to be 3.

$$\text{ifCD}() \triangleq [z \leftarrow \text{query}(\chi[0])]^0; [x \leftarrow \text{query}(\chi[z])]^1; \\ \text{if } ([x < 0])^2, [y \leftarrow \text{query}(\chi[0] + \chi[1])]^3, [y \leftarrow \text{query}(\chi[0])]^4)$$

Evaluation Result: $A_{\text{est}}(\text{ifCD}()) = 3$

Example A.7.6 (While with Nested Query Dependency). *This example algorithm contains a simple while loop. There is one query requests in the loop body at command 3. In each iteration, the query request depend on the query result from previous iteration. The longest dependency depth, i.e., the adaptivity is expectation to be k .*

$$[j \leftarrow k]^0; [a \leftarrow \text{query}(\chi[0])]^1; \\ \text{whileNested}(k) \triangleq \text{while } [j > 0]^2 \text{ do} \\ \left([x \leftarrow \text{query}(\chi[a])]^3; [a \leftarrow x + a]^4; [j \leftarrow j - 1]^5 \right)$$

The Evaluation Result: $A_{\text{est}}(\text{whileRec}(k)) = 1 + k$

Example A.7.7 (While with Multi-Path Query Dependency). *This example algorithm contains a simple while loop and a if command in the loop body. Each branch has a query request (in the commands 5 and 6) depend on the query at command 1 and the query at command 7. Among the $\frac{k}{2}$ iterations, result from previous iteration. The longest*

dependency depth, i.e., the adaptivity is expectation to be $1 + 2 * \lfloor \frac{k}{2} \rfloor$.

```

[j ← k]0; [x ← query(χ[0])]1;
while [j > 0]2 do
whileM(k) ≜ ([j ← j - 1]3;
if ([j%2 == 0]4, [y ← χ[x]]5, [w ← χ[x]]6);
[x ← query(χ(ln(y)))]7)

```

The Evaluation Result: $A_{\text{est}}(\text{whileM}(k)) = 1 + 2 * \lfloor \frac{k}{2} \rfloor$

Example A.7.8 (While with Query Dependency through Related Variables). *This example algorithm contains a simple while loop and a sequence of three query requests in the loop body. In each iteration, every query request depend on one or more query results from previous iteration. The longest dependency depth, i.e., the adaptivity is expectation to be $1 + 2 * k$.*

```

[j ← k]0; [x ← query(χ[0])]1; [y ← query(χ[1])]2;
while [j > 0]3 do
whileRV(k) ≜ ([j ← j - 1]4; [z ← query(χ(x + ln(y)))]5;
[x ← query(χ[z])]6; [y ← query(χ[z])]7)

```

The Evaluation Result: $A_{\text{est}}(\text{whileRV}(k)) = 1 + 2 * k$

Example A.7.9 (While with Query Dependency through Control Flow and Data Flow). *This example algorithm contains a simple while loop and a sequence of three query requests in the loop body. The variable in the guard is assigned by a query request in command 0. In each iteration, the query at 3 depends on either the query at line 1, and the query result at line 4 from the previous iteration. In each iteration, the query at 4 depends on either the query at line 0 and the query at line 3 in the same iteration. The longest*

dependency depth, i.e., the adaptivity is expectation to be $1 + 2 * k$.

$$\begin{aligned} & [x \leftarrow \text{query}(\chi[0])]^0; [z \leftarrow \text{query}(\chi[0])]^1; \\ \text{whileVCD}() \triangleq & \text{ while } [x > 0]^2 \text{ do} \\ & \left([x \leftarrow \text{query}(\chi(z))]^3; [z \leftarrow \text{query}(\chi(x))]^4 \right) \end{aligned}$$

The Evaluation Result: $A_{\text{est}}(\text{whileVCD}(k)) = 1 + 2 * k$

Example A.7.10 (While with Multiple Path Query Dependency Dependency). This example algorithm contains a simple while loop and a `if` command in the loop body. Each branch has a query request (in the commands 5 and 6) depend on either the query at command 1 or the query at command 7. The longest dependency depth, i.e., the adaptivity is expectation to be $2 + k$.

$$\begin{aligned} & [x \leftarrow \text{query}(k)]^0; [y \leftarrow 0]^1; \text{ while } [x > 0]^2 \text{ do} \\ \text{whileMPVCD}(k) \triangleq & \left(\text{ if } ([y > 0]^3, [y \leftarrow \text{query}(\chi[12])]^4, [w \leftarrow \text{query}(\chi[9])]^5); \right. \\ & [x \leftarrow x - 1]^6 \left. \right); \\ & [y \leftarrow \text{query}(\chi(\ln(y)))]^7 \end{aligned}$$

The Evaluation Result: $A_{\text{est}}(\text{whileMPVCD}(k)) = 2 + k$

Example A.7.11 (Nested While with Nested Query Dependency). This example algorithm contains two nested while loops. The query in the outer loop at line 5 depends on either the query at line 1 or the query results at line 8 from the previous iteration of the inner loop. The longest dependency depth, i.e., the adaptivity is expectation to be $2 + k^2$.

$$\begin{aligned} & [i \leftarrow k]^0; [x \leftarrow \text{query}(\chi[0])]^1; \\ \text{nestWhileVD}(k) \triangleq & \text{ while } [i > 0]^2 \text{ do } \left([i \leftarrow i - 1]^3; [j \leftarrow k]^4; [y \leftarrow \text{query}(\chi(\ln(x)))]^5; \right. \\ & \text{ while } [j > 0]^6 \text{ do } \left([j \leftarrow j - 1]^7; [x \leftarrow \text{query}(\chi(\ln(x)))]^8 \right) \end{aligned}$$

The Evaluation Result: $A_{\text{est}}(\text{nestWhileVD}(k)) = 2 + k^2$

Example A.7.12 (Nested While with Query Dependency through Related Variables). *This example algorithm contains two nested while loops, one query in the outer loop, and one query in the inner loop. The query in the outer loop at line 8 depends on only the query result at line 7 from the last iteration of the inner loop. However, the query at line 7 depends on either the query at line 1 the query results at line 8 from the previous iteration. The longest dependency depth, i.e., the adaptivity is expectation to be $1 + 2 * k$.*

$$\begin{aligned} & [i \leftarrow k]^0; [x \leftarrow \text{query}(\chi[0])]^1; \\ \text{nestWhileRV}(k) \triangleq & \quad \text{while } [i > 0]^2 \text{ do } \left([i \leftarrow i - 1]^3; [j \leftarrow k]^4; \right. \\ & \quad \text{while } [j > 0]^5 \text{ do } \left([j \leftarrow j - 1]^6; [y \leftarrow \text{query}(\chi(x) + \chi(1))]^7 \right); \\ & \quad \left. [x \leftarrow \text{query}(\chi(\ln(y)))]^8 \right) \end{aligned}$$

*The Evaluation Result: $A_{\text{est}}(\text{nestWhileRV}(k)) = 1 + 2 * k$*

Example A.7.13 (Nested While with Nest Query Dependency and Related Variable Accross Outer and Inner Loop). *This example algorithm contains two nested while loops, one query in the outer loop, and one query in the inner loop as well. The two queries depend on both the query results assigned to themselves in previous iteration. The longest dependency depth, i.e., the adaptivity is expectation to be $1 + k + k^2$.*

$$\begin{aligned} & [i \leftarrow k]^0; [x \leftarrow \text{query}(\chi[0])]^1; [y \leftarrow \text{query}(\chi[1])]^2; \\ & \quad \text{while } [i > 0]^3 \text{ do} \\ \text{nestWhileMR}(k) \triangleq & \quad \left([i \leftarrow i - 1]^4; [j \leftarrow k]^5; [y \leftarrow \text{query}(\chi(\ln(x) + y))]^6; \right. \\ & \quad \text{while } [j > 0]^7 \text{ do} \\ & \quad \left. \left([j \leftarrow j - 1]^8; [x \leftarrow \text{query}(\chi(\ln(y)) + \chi[x])]^9 \right) \right) \end{aligned}$$

The Evaluation Result: $A_{\text{est}}(\text{nestWhileMR}(k)) = 1 + k + k^2$

Reachability Bound The Evaluation Result:

*weight for Variable: j of label 6 is: $0 + 0 + 1 * k * k$*

*weight for Variable: y of label 7 is: $0 + 0 + 1 * k * k$*

*weight for Variable: j of label 4 is: $0 + 1 * k$*

*weight for Variable: i of label 3 is: $0 + 1 * k$*

*weight for Variable: x of label 8 is: $0 + 1 * k$*

weight for Variable: x of label 1 is: 1

weight for Variable: i of label 0 is: 1

Example A.7.14 (Nested While with MultiplePath and Nested Recursive Multiple Variable Data-Value Dependency Across Outer and Inner Loop). *We then show a more complex example with nested while command and nested data-flow across the outer and inner while loop through multiple variables. This example also contains the if command with data dependency occurred through the if guard. The longest dependency depth, i.e., the adaptivity is expectation to be $1 + k + k^2$.*

$$\begin{aligned}
 & [i \leftarrow k]^0; [x \leftarrow \text{query}(\chi[0])]^1; [y \leftarrow \text{query}(\chi[1])]^2; \\
 & \text{while } [i > 0]^3 \text{ do } \left([i \leftarrow i - 1]^4; [j \leftarrow k]^5; \right. \\
 \text{nestWhileMPRV}(k) \triangleq & \text{if } ([x > 0]^6, [y \leftarrow \text{query}(\chi(\ln(x) + y))]^7, [y \leftarrow \text{query}(\chi(x))]^8); \\
 & \text{while } [j > 0]^9 \text{ do} \\
 & \left. \left([j \leftarrow j - 1]^{10}; [x \leftarrow \text{query}(\chi(\ln(y)) + \chi[x])]^{11} \right) \right)
 \end{aligned}$$

The Adaptivity Evaluation Result: $A_{\text{est}}(\text{nestWhileMPRV}(k)) = 1 + k + k^2$

The Reachability Bound Evaluation Result:

*weight for Variable: j of label 10 is: $0 + 0 + 1 * k * k$*

*weight for Variable: x of label 11 is: $0 + 0 + 1 * k * k$*

*weight for Variable: y of label 7 is: $0 + 1 * k$*

*weight for Variable: y of label 8 is: $0 + 1 * k$*

*weight for Variable: j of label 5 is: $0 + 1 * k$*

*weight for Variable: i of label 4 is: $0 + 1 * k$*

weight for Variable: y of label 2 is: 1

weight for Variable: x of label 1 is: 1

weight for Variable: i of label 0 is: 1

A.8 THE CODES IN EVALUATION TABLE

Below are the example programs in evaluation table ?? from line 16 to 20.

Example A.8.1 (mRCompose). *The composed multiple rounds program:*

```

[ j <- N ] 0 ; 1
[ l <- 0 ] 1 ; 2
[ cs <- -1 ] 2 ; 3
[ ns <- -1 ] 3 ; 4
while [ < (0, j) ] 4 do { 5
  [ j <- - ( j, 1 ) ] 5; 6
  [ cs <- + ( cs, 0 ) ] 6; 7
  [ ns <- + ( ns, 0 ) ] 7 }; 8
[w <- k] 8; 9
while [ < (0, w)] 9 do { 10
  [ w <- - ( w, 1 ) ] 10; 11
  [ p <- c ] 11; 12
  [ q <- c ] 12; 13
  [ a <- query ( l ) ] 13 ; 14
  [ i <- N ] 14; 15
  while [ < (0, i) ] 15 do { 16
    [ i <- - (i, 1) ] 16; 17
    [ csi <- + (csi, * (- (a, p), - (q, p))) ] 17; 18
    if [ > (i , 1) ] 18 19
      then { [ nsi <- + (nsi, * (- (a, p), - (q, p)) ) ] 19 } 20
      else { [ nsi <- nsi ] 20 } 21
    }; 22
  [ i2 <- N ] 21; 23
  while [ < (0, i2) ] 22 do { 24
    [ i2 <- - (i2, 1) ] 23; 25
    if [ > (nsi , 1) ] 24 26

```

```

        then { [ l <- + ( l , i2 ) ] 25 }
        else { [ l <- l ] 26 }
    }
};
[w <- k] 27;
while [ < (0, w) ] 28 do {
    [ w <- - ( w, 1 ) ] 29;
    [ p <- c ] 30;
    [ q <- c ] 31;
    [ a <- query ( l ) ] 32 ;
    [ i <- N ] 33;
    while [ < (0, i) ] 34 do {
        [ i <- - (i, 1) ] 35;
        [ csi <- + (csi, * (- (a, p), - (q, p))) ] 36;
        if [ > (i , 1) ] 37
            then { [ nsi <- + (nsi, * (- (a, p), - (q, p)) ) ] 38 }
            else { [ nsi <- nsi ] 39 }
        };
    [ i2 <- N ] 40;
    while [ < (0, i2) ] 41 do {
        [ i2 <- - (i2, 1) ] 42;
        if [ > (nsi , 1) ] 43
            then { [ l <- + ( l , i2 ) ] 44 }
            else { [ l <- l ] 45 }
        }
    }
}

```

Example A.8.2 (tRCompose). *The composed two rounds program:*

```

[ i <- k ] 0 ;
[ l <- 0 ] 1 ;

```

```

while [ > (i, 0) ] 2 do {                                     3
[ i <- - ( i, 1 ) ] 3 ;                                       4
[ a <- query ( - (k, i) ) ] 4 ;                               5
[ l <- + ( l, a ) ] 5                                         6
    } ;                                                       7
[ y <- query ( l ) ] 6;                                       8
[ j <- k ] 7 ;                                               9
[ l <- 0 ] 8 ;                                              10
while [ > (j, 0) ] 9 do {                                     11
[ j <- - ( j, 1 ) ] 10 ;                                       12
[ a <- query ( - (k, j) ) ] 11 ;                               13
[ l <- + ( l, a ) ] 12;                                       14
[ i <- k ] 13 ;                                              15
[ l <- 0 ] 14 ;                                              16
while [ > (i, 0) ] 15 do {                                     17
[ i <- - ( i, 1 ) ] 16 ;                                       18
[ a <- query ( - (k, i) ) ] 17 ;                               19
[ l <- + ( l, a ) ] 18                                         20
    }                                                         21
};                                                            22
[ i <- k ] 19 ;                                              23
[ l <- 0 ] 20 ;                                              24
while [ > (i, 0) ] 21 do {                                     25
[ i <- - ( i, 1 ) ] 22 ;                                       26
[ a <- query ( - (k, i) ) ] 23 ;                               27
[ l <- + ( l, a ) ] 24                                         28
    } ;                                                       29
    [ i <- k ] 25 ;                                           30
[ l <- 0 ] 26 ;                                              31
while [ > (i, 0) ] 27 do {                                     32
[ i <- - ( i, 1 ) ] 28 ;                                       33

```

```

[ a <- query ( - (k, i) ) ] 29 ; 34
[ l <- + ( l, a ) ] 30 35
}; 36
while [ > (j, 0) ] 31 do { 37
[ j <- - ( j, 1 ) ] 32 ; 38
[ a <- query ( - (k, j) ) ] 33 ; 39
[ l <- + ( l, a ) ] 34; 40
[ i <- k ] 35 ; 41
[ l <- 0 ] 36 ; 42
while [ > (i, 0) ] 37 do { 43
[ i <- - ( i, 1 ) ] 38 ; 44
[ a <- query ( - (k, i) ) ] 39 ; 45
[ l <- + ( l, a ) ] 40 46
} 47
}; 48
[ y <- query ( l ) ] 41 49

```

Example A.8.3 (seqCompose). *The composed two rounds program:*

```

[ x <- query ( 0 ) ] 0 ; 1
[ y <- query ( x ) ] 1 ; 2
[ z <- query ( y ) ] 2 ; 3
[ a <- + ( x, 0 ) ] 3 ; 4
[ b <- + ( a, z ) ] 4 ; 5
[ c <- + ( a, b ) ] 5 ; 6
[ w <- query ( a ) ] 6 ; 7
[ x <- query ( b ) ] 7 ; 8
[ y <- query ( c ) ] 8 ; 9
[ z <- query ( z ) ] 9 ; 10
[ w <- query ( z ) ] 10 ; 11
[ d <- + ( x, w ) ] 11 ; 12

```



```

[ e <- + ( c, z ) ] 12 ; 13
[ f <- + ( a, b ) ] 13 ; 14
[ x <- query ( 0 ) ] 14 ; 15
[ y <- query ( x ) ] 15 ; 16
[ z <- query ( y ) ] 16 ; 17
[ x <- query ( z ) ] 17 ; 18
[ g <- + ( f, w ) ] 18 ; 19
[ h <- + ( c, x ) ] 19 ; 20
[ i <- + ( w, e ) ] 20 ; 21
[ z <- query ( x ) ] 21 ; 22
if [ > ( x , 0 ) ] 22 23
then { [ y <- query ( 0 ) ] 23 } 24
else { [ w <- query ( 0 ) ] 24 }; 25
[ x <- - ( y, w ) ] 25 ; 26
[ j <- 5 ] 26 ; 27
[ x <- query ( chi : x : ) ] 27 ; 28
[ y <- query ( chi : x :) ] 28 ; 29
[ j <- - ( j, 1 ) ] 29 ; 30
if [ < ( j , 5 ) ] 30 31
then { [ y <- query ( chi : x :) ] 31 } 32
else { [ w <- query ( chi : x : ) ] 32 } ; 33
[ x <- query ( chi : y : ) ] 33 ; 34
[ y <- query ( x ) ] 34 ; 35
[ z <- query ( + ( x, y ) ) ] 35 ; 36
[ w <- query ( * ( chi : y : , chi : z :) ) ] 36 ; 37
[ z <- query ( w ) ] 37 ; 38
[ g <- + ( f, z ) ] 38 ; 39
[ h <- + ( c, x ) ] 39 ; 40
[ i <- + ( w, g ) ] 40 ; 41
[ z <- query ( x ) ] 41 ; 42
[ e <- + ( c, z ) ] 42 ; 43

```

```

[ f <- + ( a, i ) ] 43 ; 44
[ x <- query ( 0 ) ] 44 ; 45
[ y <- query ( x ) ] 45 ; 46
[ z <- query ( y ) ] 46 ; 47
[ z <- query ( z ) ] 47; 48
[ x <- / (k, 2 ) ] 48 ; 49
if [ > (x , 0) ] 49 50
then { [ y <- query ( z ) ] 50 } 51
else { [ y <- query ( 0 ) ] 51 } ; 52
[ i <- k ] 52 ; 53
[ x <- query ( y ) ] 53 ; 54
[ y <- query ( x ) ] 54 ; 55
[ w <- - ( w, 1 ) ] 55 ; 56
[ i <- - (i, 1) ] 56 ; 57
[ y <- query ( + (z, y) ) ] 57 ; 58
[ j <- k ] 58 ; 59
[ y <- query ( chi : x :) ] 59; 60
[ j <- - (j, 1) ] 60 ; 61
[ x <- query ( + ( x, y) ) ] 61; 62
[ j <- N ] 62 ; 63
[ l <- x ] 63 ; 64
[ cs <- -1 ] 64 ; 65
[ ns <- -1 ] 65 ; 66
[ y <- query ( chi : x :) ] 66 ; 67
[ j <- - ( j, 1 ) ] 67; 68
[ cs <- + ( cs, 0 ) ] 68 ; 69
[ ns <- + ( ns, 0 ) ] 69 ; 70
[w <- k] 70; 71
[w <- - ( w, 1 )] 71 ; 72
[ w <- - ( w, 1 ) ] 72 ; 73
[ p <- c ] 73; 74

```

```

[ q <- c ] 74; 75
[ a <- query ( l ) ] 75 ; 76
[ i <- N ] 76; 77
[ w <- - ( w, l ) ] 77 ; 78
[ i <- - (i, l) ] 78; 79
[ csi <- + (csi, * (- (a, p), - (q, p))) ] 79; 80
if [ > (i , I) ] 80 81
    then { [ nsi <- + (nsi, * (- (a, p), - (q, p)) ) ] 81 } 82
    else { [ nsi <- nsi ] 82 }; 83
[ i2 <- N ] 83; 84
[ y <- query ( chi : x :) ] 84 ; 85
[ i2 <- - (i2, l) ] 85; 86
if [ > (ns , I) ] 86 87
    then { [ l <- + ( l , i2 ) ] 87 } 88
    else { [ l <- l ] 88 }; 89
[ x <- query ( cs ) ] 89 ; 90
[ y <- query ( x ) ] 90 ; 91
[ z <- query ( ns ) ] 91 ; 92
[ w <- query ( z ) ] 92 ; 93
[ a <- x ] 93 ; 94
[ c <- z ] 94 ; 95
[ j <- k ] 95 ; 96
[ y <- query ( chi : x :) ] 96 ; 97
[ da <- query ( * ( a , c ) ) ] 97 ; 98
[ dc <- query ( * ( a , c ) ) ] 98 ; 99
[ a <- - (a, da) ] 99 ; 100
[ c <- - (c, dc) ] 100 ; 101
[ j <- - (j, l ) ] 101 ; 102
[ x <- query ( 0 ) ] 102 ; 103
[ y <- query ( cs ) ] 103 ; 104
[ z <- query ( c ) ] 104 ; 105

```

```

[ w <- query ( z ) ] 105 ; 106
[ x <- query ( 0 ) ] 106 ; 107
[ y <- query ( x ) ] 107 ; 108
[ z <- query ( y ) ] 108 ; 109
[ w <- query ( z ) ] 109 ; 110
[ x <- query ( 0 ) ] 110 ; 111
[ y <- query ( x ) ] 111 ; 112
[ z <- query ( + (x, y) ) ] 112 ; 113
[ w <- query ( * (chi : y : , chi : z :) ) ] 113 ; 114
[ z <- query ( w ) ] 114 ; 115
[ x <- query ( e ) ] 115 ; 116
if [ > (x , 0) ] 116 117
then { [ y <- query ( 0 ) ] 117 } 118
else { [ w <- query ( 0 ) ] 118 } ; 119
[ a <- x ] 119 ; 120
[ c <- z ] 120 ; 121
[ j <- k ] 121 ; 122
[ cs <- + ( cs, 0 ) ] 122 ; 123
[ ns <- + ( ns, 0 ) ] 123 ; 124
[ i <- k ] 124 ; 125
[ x <- query ( 0 ) ] 125 ; 126
[ y <- query ( cs ) ] 126 ; 127
[ w <- - ( w, 1 ) ] 127 ; 128
[ i <- - (i, 1) ] 128 ; 129
[ j <- k ] 129 ; 130
if [ > (x , 0) ] 130 131
then { [ y <- query ( + ( chi : x : , chi : y : ) ) ] 131 } 132
else { [ y <- query ( chi : x : ) ] 132 } ; 133
[ y <- query ( chi : x :) ] 133 ; 134
[ j <- - (j, 1) ] 134 ; 135
[ x <- query ( + ( x, y) ) ] 135; 136

```

```

[ x <- query ( z ) ] 136 ; 137
[ y <- query ( cs ) ] 137 ; 138
[ z <- query ( c ) ] 138 ; 139
[ w <- query ( z ) ] 139 ; 140
[ y <- query ( x ) ] 140 ; 141
[ z <- query ( + (x, y) ) ] 141 ; 142
[ w <- query ( * (chi : y : , chi : z :) ) ] 142 ; 143
[ z <- query ( 0 ) ] 143 ; 144
if [ > (x , 0) ] 144 145
then { [ y <- query ( x ) ] 145 } 146
else { [ w <- query ( z ) ] 146 } ; 147
[ y <- query ( cs ) ] 147 ; 148
[ z <- query ( c ) ] 148 ; 149
[ w <- query ( z ) ] 149 ; 150
[ x <- query ( w ) ] 150 ; 151
[ y <- query ( x ) ] 151 ; 152
[ z <- query ( y ) ] 152 ; 153
[ w <- query ( z ) ] 153 ; 154
[ x <- query ( 0 ) ] 154 ; 155
[ y <- query ( x ) ] 155 ; 156
[ z <- query ( + (x, y) ) ] 156 ; 157
[ w <- query ( * (chi : y : , chi : z :) ) ] 157 ; 158
[ z <- query ( 0 ) ] 158 ; 159
[ x <- query ( w ) ] 159 ; 160
[ i <- k ] 160 ; 161
[ x <- query ( chi : cs : ) ] 161 ; 162
[ w <- - ( w, 1 ) ] 162 ; 163
[ i <- - (i, 1) ] 163 ; 164
[ j <- k ] 164 ; 165
[ y <- query ( chi : x : ) ] 165 ; 166
[ y <- query ( chi : x :) ] 166; 167

```

```

[ j <- - (j, 1) ] 167 ; 168
[ x <- query ( chi : x : ) ] 168 ; 169
[ x <- query ( cs ) ] 169 ; 170
[ y <- query ( x ) ] 170 ; 171
[ z <- query ( ns ) ] 171 ; 172
[ w <- query ( z ) ] 172 ; 173
[ a <- x ] 173 ; 174
[ c <- z ] 174 ; 175
[ j <- k ] 175 ; 176
[ y <- query ( chi : x :) ] 176 ; 177
[ da <- query ( * ( a , c ) ) ] 177 ; 178
[ dc <- query ( * ( a , c ) ) ] 178 ; 179
[ a <- - (a, da) ] 179 ; 180
[ c <- - (c, dc) ] 180 ; 181
[ j <- - (j, 1 ) ] 181 ; 182
[ x <- query ( 0 ) ] 182 ; 183
[ y <- query ( cs ) ] 183 ; 184
[ z <- query ( c ) ] 184 ; 185
[ w <- query ( z ) ] 185 ; 186
[ x <- query ( 0 ) ] 186 ; 187
[ y <- query ( x ) ] 187 ; 188
[ z <- query ( y ) ] 188 ; 189
[ w <- query ( z ) ] 189 ; 190
[ x <- query ( 0 ) ] 190 ; 191
[ y <- query ( x ) ] 191 ; 192
[ z <- query ( + (x, y) ) ] 192 ; 193
[ w <- query ( * (chi : y : , chi : z :) ) ] 193 ; 194
[ b <- + ( a, z ) ] 194 ; 195
[ c <- + ( a, b ) ] 195 ; 196
[ w <- query ( a ) ] 196 ; 197
[ x <- query ( b ) ] 197 ; 198

```

```

[ y <- query ( c ) ] 198 ; 199
[ z <- query ( z ) ] 199 ; 200
[ w <- query ( z ) ] 200 ; 201
[ j <- - ( w , 1 ) ] 201 ; 202
[ x <- query ( 0 ) ] 202 ; 203
[ y <- query ( cs ) ] 203 ; 204
[ z <- query ( c ) ] 204 ; 205
[ w <- query ( z ) ] 205 ; 206
[ x <- query ( 0 ) ] 206 ; 207
[ y <- query ( x ) ] 207 ; 208
[ z <- query ( y ) ] 208 ; 209
[ w <- query ( z ) ] 209 ; 210
[ x <- query ( 0 ) ] 210 ; 211
[ d <- + ( x , w ) ] 211 ; 212
[ e <- + ( c , z ) ] 212 ; 213
[ f <- + ( a , b ) ] 213 ; 214
[ x <- query ( w ) ] 214 ; 215
[ y <- query ( x ) ] 215 ; 216
[ z <- query ( y ) ] 216 ; 217
[ x <- query ( z ) ] 217 ; 218
[ g <- + ( f , w ) ] 218 ; 219
[ h <- + ( c , x ) ] 219 ; 220
[ i <- + ( w , e ) ] 220 ; 221
[ z <- query ( x ) ] 221 ; 222
[ cs <- + ( cs , 0 ) ] 222 ; 223
[ ns <- + ( ns , 0 ) ] 223 ; 224
[ i <- k ] 224 ; 225
[ x <- query ( z ) ] 225 ; 226
[ y <- query ( cs ) ] 226 ; 227
[ w <- - ( w , 1 ) ] 227 ; 228
[ i <- - ( i , 1 ) ] 228 ; 229

```

```

[ j <- k ] 229 ; 230
if [ > (x , 0) ] 230 231
then { [ y <- query ( + ( chi : x : , chi : y : ) ) ] 231 } 232
else { [ y <- query ( chi : x : ) ] 232 } ; 233
[ y <- query ( chi : x :) ] 233; 234
[ j <- - (j, 1) ] 234 ; 235
[ x <- query ( + ( x, y) ) ] 235; 236
[ x <- query ( z ) ] 236 ; 237
[ y <- query ( cs ) ] 237 ; 238
[ z <- query ( c ) ] 238 ; 239
[ w <- query ( z ) ] 239 ; 240
[ y <- query ( x ) ] 240 ; 241
[ z <- query ( + (x, y) ) ] 241 ; 242
[ w <- query ( * (chi : y : , chi : z :) ) ] 242 ; 243
[ z <- query ( 0 ) ] 243 ; 244
if [ > (x , 0) ] 244 245
then { [ y <- query ( x ) ] 245 } 246
else { [ w <- query ( z ) ] 246 } ; 247
[ y <- query ( cs ) ] 247 ; 248
[ z <- query ( c ) ] 248 ; 249
[ w <- query ( z ) ] 249 ; 250
[ x <- query ( w ) ] 250 ; 251
[ y <- query ( x ) ] 251 ; 252
[ z <- query ( w ) ] 252 ; 253
[ w <- query ( z ) ] 253 ; 254
[ x <- query ( 0 ) ] 254 ; 255
[ y <- query ( x ) ] 255 ; 256
[ z <- query ( + (x, y) ) ] 256 ; 257
[ w <- query ( * (chi : y : , chi : z :) ) ] 257 ; 258
[ z <- query ( 0 ) ] 258 ; 259
[ x <- query ( w ) ] 259 ; 260

```



```

[ i <- k ] 260 ; 261
[ x <- query ( chi : cs : ) ] 261 ; 262
[ w <- - ( w, 1 ) ] 262 ; 263
[ i <- - (i, 1) ] 263 ; 264
[ j <- k ] 264 ; 265
[ y <- query ( chi : x : ) ] 265 ; 266
[ w <- - ( w, 1 ) ] 266 ; 267
[ j <- - (j, 1) ] 267 ; 268
[ x <- query ( chi : x : ) ] 268 ; 269
[ x <- query ( cs ) ] 269 ; 270
[ y <- query ( x ) ] 270 ; 271
[ z <- query ( ns ) ] 271 ; 272
[ w <- query ( z ) ] 272 ; 273
[ a <- x ] 273 ; 274
[ c <- z ] 274 ; 275
[ j <- k ] 275 ; 276
[ y <- query ( chi : x :) ] 276 ; 277
[ da <- query ( * ( a , c ) ) ] 277 ; 278
[ dc <- query ( * ( a , c ) ) ] 278 ; 279
[ a <- - (a, da) ] 279 ; 280
[ c <- - (c, dc) ] 280 ; 281
[ j <- - (j, 1 ) ] 281 ; 282
[ x <- query ( a ) ] 282 ; 283
[ y <- query ( cs ) ] 283 ; 284
[ z <- query ( c ) ] 284 ; 285
[ w <- query ( z ) ] 285 ; 286
[ x <- query ( w ) ] 286 ; 287
[ y <- query ( x ) ] 287 ; 288
[ z <- query ( y ) ] 288 ; 289
[ w <- query ( z ) ] 289 ; 290
[ x <- query ( 0 ) ] 290 ; 291

```

```

[ y <- query ( x ) ] 291 ; 292
[ z <- query ( + (x, y) ) ] 292 ; 293
[ w <- query ( * (chi : y : , chi : z :) ) ] 293 ; 294
[ z <- query ( 0 ) ] 294 ; 295
[ x <- query ( w ) ] 295 ; 296
if [ > (x , 0) ] 296 297
then { [ y <- query ( 0 ) ] 297 } 298
else { [ w <- query ( 0 ) ] 298 } ; 299
[ a <- x ] 299 ; 300
[ c <- z ] 300 ; 301
[ j <- k ] 301 ; 302
[ cs <- + ( cs, 0 ) ] 302 ; 303
[ ns <- + ( ns, 0 ) ] 303 ; 304
[ i <- k ] 304 ; 305
[ x <- query ( 0 ) ] 305 ; 306
[ y <- query ( cs ) ] 306 ; 307
[ w <- - ( w, 1 ) ] 307 ; 308
[ i <- - (i, 1) ] 308 ; 309
[ j <- k ] 309 ; 310
if [ > (x , 0) ] 310 311
then { [ y <- query ( + ( chi : x : , chi : y : ) ) ] 311 } 312
else { [ y <- query ( chi : x : ) ] 312 } ; 313
[ y <- query ( chi : x :) ] 313; 314
[ j <- - (j, 1) ] 314 ; 315
[ x <- query ( + ( x, y) ) ] 315; 316
[ x <- query ( c ) ] 316 ; 317
[ y <- query ( cs ) ] 317 ; 318
[ z <- query ( c ) ] 318 ; 319
[ w <- query ( z ) ] 319 ; 320
[ y <- query ( x ) ] 320 ; 321
[ z <- query ( + (x, y) ) ] 321 ; 322

```

```

[ w <- query ( * (chi : y : , chi : z :) ) ] 322 ; 323
[ z <- query ( 0 ) ] 323 ; 324
if [ > (x , 0) ] 324 325
then { [ y <- query ( w ) ] 325 } 326
else { [ w <- query ( z ) ] 326 } ; 327
[ y <- query ( cs ) ] 327 ; 328
[ z <- query ( c ) ] 328 ; 329
[ w <- query ( z ) ] 329 ; 330
[ x <- query ( 0 ) ] 330 ; 331
[ y <- query ( x ) ] 331 ; 332
[ z <- query ( y ) ] 332 ; 333
[ w <- query ( z ) ] 333 ; 334
[ x <- query ( y ) ] 334 ; 335
[ y <- query ( x ) ] 335 ; 336
[ z <- query ( + (x, y) ) ] 336 ; 337
[ w <- query ( * (chi : y : , chi : z :) ) ] 337 ; 338
[ z <- query ( 0 ) ] 338 ; 339
[ x <- query ( w ) ] 339 ; 340
[ i <- k ] 340 ; 341
[ x <- query ( chi : cs : ) ] 341 ; 342
[ w <- - ( w, 1 ) ] 342 ; 343
[ i <- - (i, 1) ] 343 ; 344
[ j <- k ] 344 ; 345
[ y <- query ( chi : x : ) ] 345 ; 346
[ y <- query ( chi : x :) ] 346 ; 347
[ j <- - (j, 1) ] 347 ; 348
[ x <- query ( chi : x : ) ] 348 ; 349
[ x <- query ( cs ) ] 349 ; 350
[ y <- query ( x ) ] 350 ; 351
[ z <- query ( ns ) ] 351 ; 352
[ w <- query ( z ) ] 352 ; 353

```

```

[ a <- x ] 353 ; 354
[ c <- z ] 354 ; 355
[ j <- k ] 355 ; 356
[ y <- query ( chi : x :) ] 356 ; 357
[ da <- query ( * ( a , c ) ) ] 357 ; 358
[ dc <- query ( * ( a , c ) ) ] 358 ; 359
[ a <- - (a, da) ] 359 ; 360
[ c <- - (c, dc) ] 360 ; 361
[ j <- - (j, 1 ) ] 361 ; 362
[ x <- query ( 0 ) ] 362 ; 363
[ y <- query ( cs ) ] 363 ; 364
[ z <- query ( c ) ] 364 ; 365
[ w <- query ( z ) ] 365 ; 366
[ x <- query ( 0 ) ] 366 ; 367
[ y <- query ( x ) ] 367 ; 368
[ z <- query ( y ) ] 368 ; 369
[ w <- query ( z ) ] 369 ; 370
[ x <- query ( 0 ) ] 370 ; 371
[ y <- query ( x ) ] 371 ; 372
[ z <- query ( + (x, y) ) ] 372 ; 373
[ w <- query ( * (chi : y : , chi : z :) ) ] 373 ; 374
[ z <- query ( 0 ) ] 374 ; 375
[ x <- query ( w ) ] 375 ; 376
if [ > (x , 0) ] 376 377
then { [ y <- query ( x ) ] 377 } 378
else { [ w <- query ( 0 ) ] 378 } ; 379
[ a <- x ] 379 ; 380
[ c <- z ] 380 ; 381
[ j <- k ] 381 ; 382
[ cs <- + ( cs, 0 ) ] 382 ; 383
[ ns <- + ( ns, 0 ) ] 383 ; 384

```

```

[ i <- k ] 384 ; 385
[ x <- query ( 0 ) ] 385 ; 386
[ y <- query ( cs ) ] 386 ; 387
[ w <- - ( w, 1 ) ] 387 ; 388
[ i <- - (i, 1) ] 388 ; 389
[ j <- k ] 389 ; 390
if [ > (x , 0) ] 390 391
then { [ y <- query ( + ( chi : x : , chi : y : ) ) ] 391 } 392
else { [ y <- query ( chi : x : ) ] 392 } ; 393
[ y <- query ( chi : x :) ] 393; 394
[ j <- - (j, 1) ] 394 ; 395
[ x <- query ( + ( x, y) ) ] 395; 396
[ x <- query ( 0 ) ] 396 ; 397
[ y <- query ( cs ) ] 397 ; 398
[ z <- query ( c ) ] 398 ; 399
[ w <- query ( z ) ] 399 ; 400
[ y <- query ( x ) ] 400 ; 401
[ z <- query ( + (x, y) ) ] 401 ; 402
[ w <- query ( * (chi : y : , chi : z :) ) ] 402 ; 403
[ z <- query ( 0 ) ] 403 ; 404
if [ > (x , 0) ] 404 405
then { [ y <- query ( w ) ] 405 } 406
else { [ w <- query ( z ) ] 406 } ; 407
[ y <- query ( cs ) ] 407 ; 408
[ z <- query ( c ) ] 408 ; 409
[ w <- query ( z ) ] 409 ; 410
[ x <- query ( 0 ) ] 410 ; 411
if [ > (w , 0) ] 411 412
then { [ y <- w ] 412 } 413
else { [ w <- query ( z ) ] 413 } ; 414
[ x <- query ( w ) ] 414 ; 415

```

```

[ y <- query ( x ) ] 415 ; 416
[ z <- query ( y ) ] 416 ; 417
[ x <- query ( z ) ] 417 ; 418
[ g <- + ( f, w ) ] 418 ; 419
[ h <- + ( c, x ) ] 419 ; 420
[ i <- + ( w, e ) ] 420 ; 421
[ z <- query ( x ) ] 421 ; 422
[ cs <- + ( cs, 0 ) ] 422 ; 423
[ ns <- + ( ns, 0 ) ] 423 ; 424
  [ i <- k ] 424 ; 425
[ x <- query ( z ) ] 425 ; 426
[ y <- query ( cs ) ] 426 ; 427
  [ w <- - ( w, 1 ) ] 427 ; 428
[ i <- - ( i, 1 ) ] 428 ; 429
[ j <- k ] 429 ; 430
if [ > ( x , 0 ) ] 430 431
then { [ y <- query ( + ( chi : x : , chi : y : ) ) ] 431 } 432
else { [ y <- query ( chi : x : ) ] 432 } ; 433
  [ y <- query ( chi : x : ) ] 433 ; 434
  [ j <- - ( j, 1 ) ] 434 ; 435
[ x <- query ( + ( x, y ) ) ] 435 ; 436
[ x <- query ( z ) ] 436 ; 437
[ y <- query ( cs ) ] 437 ; 438
[ z <- query ( c ) ] 438 ; 439
[ w <- query ( z ) ] 439 ; 440
[ y <- query ( w ) ] 440 ; 441
[ z <- query ( + ( x, y ) ) ] 441 ; 442
[ w <- query ( * ( chi : y : , chi : z : ) ) ] 442 ; 443
[ z <- query ( 0 ) ] 443 ; 444
if [ > ( x , 0 ) ] 444 445
then { [ y <- query ( x ) ] 445 } 446

```

```

else { [ w <- query ( z ) ] 446 } ; 447
[ y <- query ( cs ) ] 447 ; 448
[ z <- query ( c ) ] 448 ; 449
[ w <- query ( z ) ] 449 ; 450
[ x <- query ( w ) ] 450 ; 451
[ y <- query ( x ) ] 451 ; 452
[ z <- query ( w ) ] 452 ; 453
[ w <- query ( z ) ] 453 ; 454
[ x <- query ( 0 ) ] 454 ; 455
[ y <- query ( x ) ] 455 ; 456
[ z <- query ( + (x, y) ) ] 456 ; 457
[ w <- query ( * (chi : y : , chi : z :) ) ] 457 ; 458
[ z <- query ( 0 ) ] 458 ; 459
[ x <- query ( w ) ] 459 ; 460
[ i <- k ] 460 ; 461
[ x <- query ( chi : cs : ) ] 461 ; 462
[ i <- - (i, 1) ] 462 ; 463
[ i <- - (i, 1) ] 463 ; 464
[ j <- k ] 464 ; 465
[ y <- query ( chi : x : ) ] 465 ; 466
[ j <- - (j, 1) ] 466 ; 467
[ j <- - (j, 1) ] 467 ; 468
[ x <- query ( chi : x : ) ] 468 ; 469
[ x <- query ( cs ) ] 469 ; 470
[ y <- query ( x ) ] 470 ; 471
[ z <- query ( ns ) ] 471 ; 472
[ w <- query ( z ) ] 472 ; 473
[ a <- x ] 473 ; 474
[ c <- z ] 474 ; 475
[ j <- k ] 475 ; 476
[ da <- query ( * ( a , c ) ) ] 476 ; 477

```

```

[ da <- query ( * ( a , c ) ) ] 477 ; 478
[ dc <- query ( * ( a , c ) ) ] 478 ; 479
[ a <- - (a, da) ] 479 ; 480
[ c <- - (c, dc) ] 480 ; 481
[ j <- - (j, 1 ) ] 481 ; 482
[ x <- query ( a ) ] 482 ; 483
[ y <- query ( cs ) ] 483 ; 484
[ z <- query ( c ) ] 484 ; 485
[ w <- query ( z ) ] 485 ; 486
[ x <- query ( w ) ] 486 ; 487
[ y <- query ( x ) ] 487 ; 488
[ z <- query ( y ) ] 488 ; 489
[ w <- query ( z ) ] 489 ; 490
[ x <- query ( 0 ) ] 490 ; 491
[ y <- query ( x ) ] 491 ; 492
[ z <- query ( + (x, y) ) ] 492 ; 493
[ w <- query ( * (chi : y : , chi : z :) ) ] 493 ; 494
[ z <- query ( 0 ) ] 494 ; 495
[ x <- query ( w ) ] 495 ; 496
if [ > (x , 0) ] 496 497
then { [ y <- query ( 0 ) ] 497 } 498
else { [ w <- query ( 0 ) ] 498 } ; 499
[ a <- x ] 499 ; 500
[ c <- z ] 500 ; 501
[ r <- query ( c ) ] 501 502

```

Example A.8.4 (jumboS). *The composed program with nested loops.*

```

[ x <- query ( 0 ) ] 0 ; 1
[ y <- query ( x ) ] 1 ; 2
[ z <- query ( y ) ] 2 ; 3

```



```

[ a <- + ( x, 0 ) ] 3 ; 4
[ b <- + ( a, z ) ] 4 ; 5
[ c <- + ( a, b ) ] 5 ; 6
[ w <- query ( a ) ] 6 ; 7
[ x <- query ( b ) ] 7 ; 8
[ y <- query ( c ) ] 8 ; 9
[ z <- query ( z ) ] 9 ; 10
[ w <- query ( z ) ] 10 ; 11
[ d <- + ( x, w ) ] 11 ; 12
[ e <- + ( c, z ) ] 12 ; 13
[ f <- + ( a, b ) ] 13 ; 14
[ x <- query ( 0 ) ] 14 ; 15
[ y <- query ( x ) ] 15 ; 16
[ z <- query ( y ) ] 16 ; 17
[ x <- query ( z ) ] 17 ; 18
[ g <- + ( f, w ) ] 18 ; 19
[ h <- + ( c, x ) ] 19 ; 20
[ i <- + ( w, e ) ] 20 ; 21
[ z <- query ( x ) ] 21 ; 22
if [ > (x , 0) ] 22 23
then { [ y <- query ( 0 ) ] 23 } 24
else { [ w <- query ( 0 ) ] 24 }; 25
[ x <- - (y, w) ] 25 ; 26
[ j <- 5 ] 26 ; 27
[ x <- query ( chi : x : ) ] 27 ; 28
while [ > (j, 0) ] 28 do { 29
[ j <- - ( j, 1 ) ] 29 ; 30
if [ < (j , 5) ] 30 31
then { [ y <- query ( chi : x :) ] 31 } 32
else { [ w <- query ( chi : x : ) ] 32 } ; 33
[ x <- query ( chi : y : ) ] 33 34

```

```

};
[ y <- query ( x ) ] 34 ;
[ z <- query ( + (x, y) ) ] 35 ;
[ w <- query ( * (chi : y : , chi : z :) ) ] 36;
[ z <- query ( w ) ] 37 ;
[ g <- + ( f, z ) ] 38 ;
[ h <- + ( c, x ) ] 39 ;
[ i <- + ( w, g ) ] 40 ;
[ z <- query ( x ) ] 41 ;
[ e <- + ( c, z ) ] 42 ;
[ f <- + ( a, i ) ] 43 ;
[ x <- query ( 0 ) ] 44 ;
[ y <- query ( x ) ] 45 ;
[ z <- query ( y ) ] 46 ;
[ z <- query ( z ) ] 47;
[ x <- / (k, 2 ) ] 48 ;
if [ > (x , 0) ] 49
then { [ y <- query ( z ) ] 50 }
else { [ y <- query ( 0 ) ] 51 } ;
[ i <- k ] 52 ;
[ x <- query ( y ) ] 53 ;
[ y <- query ( x ) ] 54 ;
while [ > (i , 0) ] 55 do {
[ i <- - (i, 1) ] 56 ;
[ y <- query ( + (z, y) ) ] 57 ;
[ j <- k ] 58 ;
while [ > (j , 0) ] 59 do
{ [ j <- - (j, 1) ] 60 ;
[ x <- query ( + ( x, y) ) ] 61 }
} ;
[ j <- N ] 62 ;

```

```

[ l <- x ] 63 ; 66
[ cs <- -1 ] 64 ; 67
[ ns <- -1 ] 65 ; 68
while [ < (0, j) ] 66 do { 69
[ j <- - ( j, 1 ) ] 67; 70
[ cs <- + ( cs, 0 ) ] 68 ; 71
[ ns <- + ( ns, 0 ) ] 69 }; 72
[w <- k] 70 73

```

Example A.8.5 (jumbo). *The composed program with multiple paths nested loops.*

```

[ x <- query ( 0 ) ] 0 ; 1
[ y <- query ( x ) ] 1 ; 2
[ z <- query ( y ) ] 2 ; 3
[ a <- + ( x, 0 ) ] 3 ; 4
[ b <- + ( a, z ) ] 4 ; 5
[ c <- + ( a, b ) ] 5 ; 6
[ w <- query ( a ) ] 6 ; 7
[ x <- query ( b ) ] 7 ; 8
[ y <- query ( c ) ] 8 ; 9
[ z <- query ( z ) ] 9 ; 10
[ w <- query ( z ) ] 10 ; 11
[ d <- + ( x, w ) ] 11 ; 12
[ e <- + ( c, z ) ] 12 ; 13
[ f <- + ( a, b ) ] 13 ; 14
[ x <- query ( 0 ) ] 14 ; 15
[ y <- query ( x ) ] 15 ; 16
[ z <- query ( y ) ] 16 ; 17
[ x <- query ( z ) ] 17; 18
[ g <- + ( f, w ) ] 18 ; 19
[ h <- + ( c, x ) ] 19 ; 20

```

```

[ i <- + ( w, e ) ] 20 ; 21
[ z <- query ( x ) ] 21 ; 22
if [ > (x , 0) ] 22 23
then { [ y <- query ( 0 ) ] 23 } 24
else { [ w <- query ( 0 ) ] 24 }; 25
[ x <- - (y, w) ] 25 ; 26
[ j <- 5 ] 26 ; 27
[ x <- query ( chi : x : ) ] 27 ; 28
while [ > (j, 0) ] 28 do { 29
[ j <- - ( j, 1 ) ] 29 ; 30
if [ < (j , 5) ] 30 31
then { [ y <- query ( chi : x :) ] 31 } 32
else { [ w <- query ( chi : x : ) ] 32 } ; 33
[ x <- query ( chi : y : ) ] 33 34
}; 35
[ y <- query ( x ) ] 34 ; 36
[ z <- query ( + (x, y) ) ] 35 ; 37
[ w <- query ( * (chi : y : , chi : z :) ) ] 36; 38
[ z <- query ( w ) ] 37 ; 39
[ g <- + ( f, z ) ] 38 ; 40
[ h <- + ( c, x ) ] 39 ; 41
[ i <- + ( w, g ) ] 40 ; 42
[ z <- query ( x ) ] 41; 43
[ e <- + ( c, z ) ] 42 ; 44
[ f <- + ( a, i ) ] 43 ; 45
[ x <- query ( 0 ) ] 44 ; 46
[ y <- query ( x ) ] 45 ; 47
[ z <- query ( y ) ] 46 ; 48
[ z <- query ( z ) ] 47; 49
[ x <- / (k, 2 ) ] 48 ; 50
if [ > (x , 0) ] 49 51

```

```

then { [ y <- query ( z ) ] 50 } 52
else { [ y <- query ( 0 ) ] 51 } ; 53
[ i <- k ] 52 ; 54
[ x <- query ( y ) ] 53 ; 55
[ y <- query ( x ) ] 54 ; 56
while [ > (i , 0) ] 55 do { 57
[ i <- - (i, 1) ] 56 ; 58
[ y <- query ( + (z, y) ) ] 57 ; 59
[ j <- k ] 58 ; 60
while [ > (j , 0) ] 59 do 61
{ [ j <- - (j, 1) ] 60 ; 62
[ x <- query ( + ( x, y) ) ] 61 } 63
} ; 64
[ j <- N ] 62 ; 65
[ l <- x ] 63 ; 66
[ cs <- -1 ] 64 ; 67
[ ns <- -1 ] 65 ; 68
while [ < (0, j) ] 66 do { 69
[ j <- - ( j, 1 ) ] 67; 70
[ cs <- + ( cs, 0 ) ] 68 ; 71
[ ns <- + ( ns, 0 ) ] 69 }; 72
[w <- k] 70; 73
while [ < (0, w)] 71 do { 74
[ w <- - ( w, 1 ) ] 72 ; 75
[ p <- c ] 73; 76
[ q <- c ] 74; 77
[ a <- query ( l ) ] 75 ; 78
[ i <- N ] 76; 79
while [ < (0, i) ] 77 do { 80
[ i <- - (i, 1) ] 78; 81
[ csi <- + (csi, * (- (a, p), - (q, p))) ] 79; 82

```

```

    if [ > (i , I) ] 80
        then { [ nsi <- + (nsi, * (- (a, p), - (q, p)) ) ] 81 }
        else { [ nsi <- nsi ] 82 }
    };
[ i2 <- N ] 83;
while [ < (0, i2) ] 84 do {
    [ i2 <- - (i2, 1) ] 85;
    if [ > (ns , I) ] 86
        then { [ l <- + ( l , i2 ) ] 87 }
        else { [ l <- l ] 88 }
    }
};
[ x <- query ( cs ) ] 89 ;
[ y <- query ( x ) ] 90 ;
[ z <- query ( ns ) ] 91 ;
[ w <- query ( z ) ] 92 ;
[ a <- x ] 93 ;
[ c <- z ] 94 ;
[ j <- k ] 95 ;
while [ < (0, j) ] 96 do {
    [ da <- query ( * ( a , c ) ) ] 97 ;
    [ dc <- query ( * ( a , c ) ) ] 98 ;
    [ a <- - (a, da) ] 99 ;
    [ c <- - (c, dc) ] 100 ;
    [ j <- - (j, 1 ) ] 101
    };
[ x <- query ( 0 ) ] 102 ;
[ y <- query ( cs ) ] 103 ;
[ z <- query ( c ) ] 104 ;
[ w <- query ( z ) ] 105 ;
[ x <- query ( 0 ) ] 106 ;

```

```

[ y <- query ( x ) ] 107 ; 114
[ z <- query ( y ) ] 108 ; 115
[ w <- query ( z ) ] 109 ; 116
[ x <- query ( 0 ) ] 110 ; 117
[ y <- query ( x ) ] 111 ; 118
[ z <- query ( + (x, y) ) ] 112 ; 119
[ w <- query ( * (chi : y : , chi : z :) ) ] 113 ; 120
[ z <- query ( w ) ] 114 ; 121
[ x <- query ( e ) ] 115 ; 122
if [ > (x , 0) ] 116 123
then { [ y <- query ( 0 ) ] 117 } 124
else { [ w <- query ( 0 ) ] 118 } ; 125
[ a <- x ] 119 ; 126
[ c <- z ] 120 ; 127
[ j <- k ] 121 ; 128
[ cs <- + ( cs, 0 ) ] 122 ; 129
[ ns <- + ( ns, 0 ) ] 123 ; 130
[ i <- k ] 124 ; 131
[ x <- query ( 0 ) ] 125 ; 132
[ y <- query ( cs ) ] 126 ; 133
while [ > (i , 0) ] 127 do { 134
[ i <- - (i, 1) ] 128 ; 135
[ j <- k ] 129 ; 136
if [ > (x , 0) ] 130 137
then { [ y <- query ( + ( chi : x : , chi : y : ) ) ] 131 } 138
else { [ y <- query ( chi : x : ) ] 132 } ; 139
while [ > (j , 0) ] 133 do 140
{ [ j <- - (j, 1) ] 134 ; 141
[ x <- query ( + ( x, y) ) ] 135 } 142
}; 143
[ x <- query ( z ) ] 136 ; 144

```

```

[ y <- query ( cs ) ] 137 ; 145
[ z <- query ( c ) ] 138 ; 146
[ w <- query ( z ) ] 139 ; 147
[ y <- query ( x ) ] 140 ; 148
[ z <- query ( + (x, y) ) ] 141 ; 149
[ w <- query ( * (chi : y : , chi : z :) ) ] 142 ; 150
[ z <- query ( 0 ) ] 143 ; 151
if [ > (x , 0) ] 144 152
then { [ y <- query ( x ) ] 145 } 153
else { [ w <- query ( z ) ] 146 } ; 154
[ y <- query ( cs ) ] 147 ; 155
[ z <- query ( c ) ] 148 ; 156
[ w <- query ( z ) ] 149 ; 157
[ x <- query ( w ) ] 150 ; 158
[ y <- query ( x ) ] 151 ; 159
[ z <- query ( y ) ] 152 ; 160
[ w <- query ( z ) ] 153 ; 161
[ x <- query ( 0 ) ] 154 ; 162
[ y <- query ( x ) ] 155 ; 163
[ z <- query ( + (x, y) ) ] 156 ; 164
[ w <- query ( * (chi : y : , chi : z :) ) ] 157 ; 165
[ z <- query ( 0 ) ] 158 ; 166
[ x <- query ( w ) ] 159 ; 167
[ i <- k ] 160 ; 168
[ x <- query ( chi : cs : ) ] 161 ; 169
while [ > (i , 0) ] 162 do { 170
[ i <- - (i, 1) ] 163 ; 171
[ j <- k ] 164 ; 172
[ y <- query ( chi : x : ) ] 165 ; 173
while [ > (j , 0) ] 166 do 174
{ [ j <- - (j, 1) ] 167 ; 175

```



```

[ x <- query ( chi : x : ) ] 168 }                                176
};                                                                    177
[ x <- query ( cs ) ] 169 ;                                          178
[ y <- query ( x ) ] 170 ;                                          179
[ z <- query ( ns ) ] 171 ;                                          180
[ w <- query ( z ) ] 172 ;                                          181
[ a <- x ] 173 ;                                                    182
[ c <- z ] 174 ;                                                    183
[ j <- k ] 175 ;                                                    184
while [ < (0, j) ] 176 do {                                         185
[ da <- query ( * ( a , c ) ) ] 177 ;                               186
[ dc <- query ( * ( a , c ) ) ] 178 ;                               187
[ a <- - (a, da) ] 179 ;                                            188
[ c <- - (c, dc) ] 180 ;                                            189
[ j <- - (j, 1 ) ] 181                                             190
};                                                                    191
[ x <- query ( 0 ) ] 182 ;                                          192
[ y <- query ( cs ) ] 183 ;                                          193
[ z <- query ( c ) ] 184 ;                                          194
[ w <- query ( z ) ] 185 ;                                          195
[ x <- query ( 0 ) ] 186 ;                                          196
[ y <- query ( x ) ] 187 ;                                          197
[ z <- query ( y ) ] 188 ;                                          198
[ w <- query ( z ) ] 189 ;                                          199
[ x <- query ( 0 ) ] 190 ;                                          200
[ y <- query ( x ) ] 191 ;                                          201
[ z <- query ( + (x, y) ) ] 192 ;                                    202
[ w <- query ( * (chi : y : , chi : z :) ) ] 193 ;                203
[ b <- + ( a, z ) ] 194 ;                                          204
[ c <- + ( a, b ) ] 195 ;                                          205
[ w <- query ( a ) ] 196 ;                                          206

```

```

[ x <- query ( b ) ] 197 ; 207
[ y <- query ( c ) ] 198 ; 208
[ z <- query ( z ) ] 199 ; 209
[ w <- query ( z ) ] 200 ; 210
[ j <- - ( w , 1 ) ] 201 ; 211
[ x <- query ( 0 ) ] 202 ; 212
[ y <- query ( cs ) ] 203 ; 213
[ z <- query ( c ) ] 204 ; 214
[ w <- query ( z ) ] 205 ; 215
[ x <- query ( 0 ) ] 206 ; 216
[ y <- query ( x ) ] 207 ; 217
[ z <- query ( y ) ] 208 ; 218
[ w <- query ( z ) ] 209 ; 219
[ x <- query ( 0 ) ] 210 ; 220
[ d <- + ( x , w ) ] 211 ; 221
[ e <- + ( c , z ) ] 212 ; 222
[ f <- + ( a , b ) ] 213 ; 223
[ x <- query ( w ) ] 214 ; 224
[ y <- query ( x ) ] 215 ; 225
[ z <- query ( y ) ] 216 ; 226
[ x <- query ( z ) ] 217 ; 227
[ g <- + ( f , w ) ] 218 ; 228
[ h <- + ( c , x ) ] 219 ; 229
[ i <- + ( w , e ) ] 220 ; 230
[ z <- query ( x ) ] 221 ; 231
[ cs <- + ( cs , 0 ) ] 222 ; 232
[ ns <- + ( ns , 0 ) ] 223 ; 233
[ i <- k ] 224 ; 234
[ x <- query ( z ) ] 225 ; 235
[ y <- query ( cs ) ] 226 ; 236
while [ > ( i , 0 ) ] 227 do { 237

```

```

[ i <- - (i, 1) ] 228 ; 238
[ j <- k ] 229 ; 239
if [ > (x , 0) ] 230 240
then { [ y <- query ( + ( chi : x : , chi : y : ) ) ] 231 } 241
else { [ y <- query ( chi : x : ) ] 232 } ; 242
while [ > (j , 0) ] 233 do 243
{ [ j <- - (j, 1) ] 234 ; 244
[ x <- query ( + ( x, y) ) ] 235 } 245
}; 246
[ x <- query ( z ) ] 236 ; 247
[ y <- query ( cs ) ] 237 ; 248
[ z <- query ( c ) ] 238 ; 249
[ w <- query ( z ) ] 239 ; 250
[ y <- query ( x ) ] 240 ; 251
[ z <- query ( + (x, y) ) ] 241 ; 252
[ w <- query ( * (chi : y : , chi : z :) ) ] 242 ; 253
[ z <- query ( 0 ) ] 243 ; 254
if [ > (x , 0) ] 244 255
then { [ y <- query ( x ) ] 245 } 256
else { [ w <- query ( z ) ] 246 } ; 257
[ y <- query ( cs ) ] 247 ; 258
[ z <- query ( c ) ] 248 ; 259
[ w <- query ( z ) ] 249 ; 260
[ x <- query ( w ) ] 250 ; 261
[ y <- query ( x ) ] 251 ; 262
[ z <- query ( w ) ] 252 ; 263
[ w <- query ( z ) ] 253 ; 264
[ x <- query ( 0 ) ] 254 ; 265
[ y <- query ( x ) ] 255 ; 266
[ z <- query ( + (x, y) ) ] 256 ; 267
[ w <- query ( * (chi : y : , chi : z :) ) ] 257 ; 268

```

```

[ z <- query ( 0 ) ] 258 ; 269
[ x <- query ( w ) ] 259 ; 270
[ i <- k ] 260 ; 271
[ x <- query ( chi : cs : ) ] 261 ; 272
while [ > (i , 0) ] 262 do { 273
[ i <- - (i, 1) ] 263 ; 274
[ j <- k ] 264 ; 275
[ y <- query ( chi : x : ) ] 265 ; 276
while [ > (j , 0) ] 266 do 277
{ [ j <- - (j, 1) ] 267 ; 278
[ x <- query ( chi : x : ) ] 268 } 279
}; 280
[ x <- query ( cs ) ] 269 ; 281
[ y <- query ( x ) ] 270 ; 282
[ z <- query ( ns ) ] 271 ; 283
[ w <- query ( z ) ] 272 ; 284
[ a <- x ] 273 ; 285
[ c <- z ] 274 ; 286
[ j <- k ] 275 ; 287
while [ < (0, j) ] 276 do { 288
[ da <- query ( * ( a , c ) ) ] 277 ; 289
[ dc <- query ( * ( a , c ) ) ] 278 ; 290
[ a <- - (a, da) ] 279 ; 291
[ c <- - (c, dc) ] 280 ; 292
[ j <- - (j, 1 ) ] 281 293
}; 294
[ x <- query ( a ) ] 282 ; 295
[ y <- query ( cs ) ] 283 ; 296
[ z <- query ( c ) ] 284 ; 297
[ w <- query ( z ) ] 285 ; 298
[ x <- query ( w ) ] 286 ; 299

```

```

[ y <- query ( x ) ] 287 ; 300
[ z <- query ( y ) ] 288 ; 301
[ w <- query ( z ) ] 289 ; 302
[ x <- query ( 0 ) ] 290 ; 303
[ y <- query ( x ) ] 291 ; 304
[ z <- query ( + (x, y) ) ] 292 ; 305
[ w <- query ( * (chi : y : , chi : z :) ) ] 293 ; 306
[ z <- query ( 0 ) ] 294 ; 307
[ x <- query ( w ) ] 295 ; 308
if [ > (x , 0) ] 296 309
then { [ y <- query ( 0 ) ] 297 } 310
else { [ w <- query ( 0 ) ] 298 } ; 311
[ a <- x ] 299 ; 312
[ c <- z ] 300 ; 313
[ j <- k ] 301 ; 314
[ cs <- + ( cs, 0 ) ] 302 ; 315
[ ns <- + ( ns, 0 ) ] 303 ; 316
  [ i <- k ] 304 ; 317
[ x <- query ( 0 ) ] 305 ; 318
[ y <- query ( cs ) ] 306 ; 319
while [ > (i , 0) ] 307 do { 320
[ i <- - (i, 1) ] 308 ; 321
[ j <- k ] 309 ; 322
if [ > (x , 0) ] 310 323
then { [ y <- query ( + ( chi : x : , chi : y : ) ) ] 311 } 324
else { [ y <- query ( chi : x : ) ] 312 } ; 325
while [ > (j , 0) ] 313 do 326
{ [ j <- - (j, 1) ] 314 ; 327
[ x <- query ( + ( x, y) ) ] 315 } 328
  }; 329
[ x <- query ( c ) ] 316 ; 330

```

```

[ y <- query ( cs ) ] 317 ; 331
[ z <- query ( c ) ] 318 ; 332
[ w <- query ( z ) ] 319 ; 333
[ y <- query ( x ) ] 320 ; 334
[ z <- query ( + (x, y) ) ] 321 ; 335
[ w <- query ( * (chi : y : , chi : z :) ) ] 322 ; 336
[ z <- query ( 0 ) ] 323 ; 337
if [ > (x , 0) ] 324 338
then { [ y <- query ( w ) ] 325 } 339
else { [ w <- query ( z ) ] 326 } ; 340
[ y <- query ( cs ) ] 327 ; 341
[ z <- query ( c ) ] 328 ; 342
[ w <- query ( z ) ] 329 ; 343
[ x <- query ( 0 ) ] 330 ; 344
[ y <- query ( x ) ] 331 ; 345
[ z <- query ( y ) ] 332 ; 346
[ w <- query ( z ) ] 333 ; 347
[ x <- query ( y ) ] 334 ; 348
[ y <- query ( x ) ] 335 ; 349
[ z <- query ( + (x, y) ) ] 336 ; 350
[ w <- query ( * (chi : y : , chi : z :) ) ] 337 ; 351
[ z <- query ( 0 ) ] 338 ; 352
[ x <- query ( w ) ] 339 ; 353
[ i <- k ] 340 ; 354
[ x <- query ( chi : cs : ) ] 341 ; 355
while [ > (i , 0) ] 342 do { 356
[ i <- - (i, 1) ] 343 ; 357
[ j <- k ] 344 ; 358
[ y <- query ( chi : x : ) ] 345 ; 359
while [ > (j , 0) ] 346 do 360
{ [ j <- - (j, 1) ] 347 ; 361

```

```

[ x <- query ( chi : x : ) ] 348 }                                362
};                                                                    363
[ x <- query ( cs ) ] 349 ;                                          364
[ y <- query ( x ) ] 350 ;                                          365
[ z <- query ( ns ) ] 351 ;                                          366
[ w <- query ( z ) ] 352 ;                                          367
[ a <- x ] 353 ;                                                    368
[ c <- z ] 354 ;                                                    369
[ j <- k ] 355 ;                                                    370
while [ < (0, j) ] 356 do {                                         371
[ da <- query ( * ( a , c ) ) ] 357 ;                               372
[ dc <- query ( * ( a , c ) ) ] 358 ;                               373
[ a <- - (a, da) ] 359 ;                                           374
[ c <- - (c, dc) ] 360 ;                                           375
[ j <- - (j, 1 ) ] 361                                             376
};                                                                    377
[ x <- query ( 0 ) ] 362 ;                                          378
[ y <- query ( cs ) ] 363 ;                                          379
[ z <- query ( c ) ] 364 ;                                          380
[ w <- query ( z ) ] 365 ;                                          381
[ x <- query ( 0 ) ] 366 ;                                          382
[ y <- query ( x ) ] 367 ;                                          383
[ z <- query ( y ) ] 368 ;                                          384
[ w <- query ( z ) ] 369 ;                                          385
[ x <- query ( 0 ) ] 370 ;                                          386
[ y <- query ( x ) ] 371 ;                                          387
[ z <- query ( + (x, y) ) ] 372 ;                                   388
[ w <- query ( * (chi : y : , chi : z :) ) ] 373 ;               389
[ z <- query ( 0 ) ] 374 ;                                          390
[ x <- query ( w ) ] 375 ;                                          391
if [ > (x , 0) ] 376                                              392

```

```

then { [ y <- query ( x ) ] 377 } 393
else { [ w <- query ( 0 ) ] 378 } ; 394
[ a <- x ] 379 ; 395
[ c <- z ] 380 ; 396
[ j <- k ] 381 ; 397
[ cs <- + ( cs, 0 ) ] 382 ; 398
[ ns <- + ( ns, 0 ) ] 383 ; 399
[ i <- k ] 384 ; 400
[ x <- query ( 0 ) ] 385 ; 401
[ y <- query ( cs ) ] 386 ; 402
while [ > (i , 0) ] 387 do { 403
[ i <- - (i, 1) ] 388 ; 404
[ j <- k ] 389 ; 405
if [ > (x , 0) ] 390 406
then { [ y <- query ( + ( chi : x : , chi : y : ) ) ] 391 } 407
else { [ y <- query ( chi : x : ) ] 392 } ; 408
while [ > (j , 0) ] 393 do 409
{ [ j <- - (j, 1) ] 394 ; 410
[ x <- query ( + ( x, y) ) ] 395 } 411
}; 412
[ x <- query ( 0 ) ] 396 ; 413
[ y <- query ( cs ) ] 397 ; 414
[ z <- query ( c ) ] 398 ; 415
[ w <- query ( z ) ] 399 ; 416
[ y <- query ( x ) ] 400 ; 417
[ z <- query ( + (x, y) ) ] 401 ; 418
[ w <- query ( * (chi : y : , chi : z :) ) ] 402 ; 419
[ z <- query ( 0 ) ] 403 ; 420
if [ > (x , 0) ] 404 421
then { [ y <- query ( w ) ] 405 } 422
else { [ w <- query ( z ) ] 406 } ; 423

```



```

[ y <- query ( cs ) ] 407 ; 424
[ z <- query ( c ) ] 408 ; 425
[ w <- query ( z ) ] 409 ; 426
[ x <- query ( 0 ) ] 410 ; 427
if [ > (w , 0) ] 411 428
then { [ y <- w ] 412 } 429
else { [ w <- query ( z ) ] 413 } ; 430
[ x <- query ( w ) ] 414 ; 431
[ y <- query ( x ) ] 415 ; 432
[ z <- query ( y ) ] 416 ; 433
[ x <- query ( z ) ] 417 ; 434
[ g <- + ( f, w ) ] 418 ; 435
[ h <- + ( c, x ) ] 419 ; 436
[ i <- + ( w, e ) ] 420 ; 437
[ z <- query ( x ) ] 421 ; 438
[ cs <- + ( cs, 0 ) ] 422 ; 439
[ ns <- + ( ns, 0 ) ] 423 ; 440
[ i <- k ] 424 ; 441
[ x <- query ( z ) ] 425 ; 442
[ y <- query ( cs ) ] 426 ; 443
while [ > (i , 0) ] 427 do { 444
[ i <- - (i, 1) ] 428 ; 445
[ j <- k ] 429 ; 446
if [ > (x , 0) ] 430 447
then { [ y <- query ( + ( chi : x : , chi : y : ) ) ] 431 } 448
else { [ y <- query ( chi : x : ) ] 432 } ; 449
while [ > (j , 0) ] 433 do 450
{ [ j <- - (j, 1) ] 434 ; 451
[ x <- query ( + ( x, y) ) ] 435 } 452
}; 453
[ x <- query ( z ) ] 436 ; 454

```

```

[ y <- query ( cs ) ] 437 ; 455
[ z <- query ( c ) ] 438 ; 456
[ w <- query ( z ) ] 439 ; 457
[ y <- query ( w ) ] 440 ; 458
[ z <- query ( + (x, y) ) ] 441 ; 459
[ w <- query ( * (chi : y : , chi : z :) ) ] 442 ; 460
[ z <- query ( 0 ) ] 443 ; 461
if [ > (x , 0) ] 444 462
then { [ y <- query ( x ) ] 445 } 463
else { [ w <- query ( z ) ] 446 } ; 464
[ y <- query ( cs ) ] 447 ; 465
[ z <- query ( c ) ] 448 ; 466
[ w <- query ( z ) ] 449 ; 467
[ x <- query ( w ) ] 450 ; 468
[ y <- query ( x ) ] 451 ; 469
[ z <- query ( w ) ] 452 ; 470
[ w <- query ( z ) ] 453 ; 471
[ x <- query ( 0 ) ] 454 ; 472
[ y <- query ( x ) ] 455 ; 473
[ z <- query ( + (x, y) ) ] 456 ; 474
[ w <- query ( * (chi : y : , chi : z :) ) ] 457 ; 475
[ z <- query ( 0 ) ] 458 ; 476
[ x <- query ( w ) ] 459 ; 477
[ i <- k ] 460 ; 478
[ x <- query ( chi : cs : ) ] 461 ; 479
while [ > (i , 0) ] 462 do { 480
[ i <- - (i, 1) ] 463 ; 481
[ j <- k ] 464 ; 482
[ y <- query ( chi : x : ) ] 465 ; 483
while [ > (j , 0) ] 466 do 484
{ [ j <- - (j, 1) ] 467 ; 485

```

```

[ x <- query ( chi : x : ) ] 468 }                                486
};                                                                    487
[ x <- query ( cs ) ] 469 ;                                          488
[ y <- query ( x ) ] 470 ;                                          489
[ z <- query ( ns ) ] 471 ;                                          490
[ w <- query ( z ) ] 472 ;                                          491
[ a <- x ] 473 ;                                                    492
[ c <- z ] 474 ;                                                    493
[ j <- k ] 475 ;                                                    494
while [ < (0, j) ] 476 do {                                         495
[ da <- query ( * ( a , c ) ) ] 477 ;                               496
[ dc <- query ( * ( a , c ) ) ] 478 ;                               497
[ a <- - (a, da) ] 479 ;                                            498
[ c <- - (c, dc) ] 480 ;                                            499
[ j <- - (j, 1 ) ] 481                                             500
};                                                                    501
[ x <- query ( a ) ] 482 ;                                          502
[ y <- query ( cs ) ] 483 ;                                          503
[ z <- query ( c ) ] 484 ;                                          504
[ w <- query ( z ) ] 485 ;                                          505
[ x <- query ( w ) ] 486 ;                                          506
[ y <- query ( x ) ] 487 ;                                          507
[ z <- query ( y ) ] 488 ;                                          508
[ w <- query ( z ) ] 489 ;                                          509
[ x <- query ( 0 ) ] 490 ;                                          510
[ y <- query ( x ) ] 491 ;                                          511
[ z <- query ( + (x, y) ) ] 492 ;                                   512
[ w <- query ( * (chi : y : , chi : z :) ) ] 493 ;               513
[ z <- query ( 0 ) ] 494 ;                                          514
[ x <- query ( w ) ] 495 ;                                          515
if [ > (x , 0) ] 496                                             516

```

```
then { [ y <- query ( 0 ) ] 497 } 517
else { [ w <- query ( 0 ) ] 498 } ; 518
[ a <- x ] 499 ; 519
[ c <- z ] 500 ; 520
[ r <- query ( c ) ] 501 521
```

APPENDIX B

Supplementary Materials in PART ii

B.1 PROOFS OF LEMMAS IN LANGUAGE MODEL

Lemma (Trace Non-Decreasing). *For any program $c \in \mathcal{C}$ and initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, if there exists $\tau \in \mathcal{T}^{+\infty}$ and $c' \in \mathcal{C}$ such that $\langle c, \tau_0 \rangle \rightarrow \langle c', \tau \rangle$ then there exists a trace $\tau' \in \mathcal{T}^{+\infty}$ such that $\tau_{0++}\tau' = \tau$ formally as follows.*

$$\begin{aligned} & \forall \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^{+\infty}, c, c' \in \mathcal{C} . \langle c, \tau_0 \rangle \rightarrow \langle c', \tau \rangle \\ & \implies \exists \tau' \in \mathcal{T}^{+\infty} . \tau_{0++}\tau' = \tau \end{aligned}$$

Proof. Taking an arbitrary program $c \in \mathcal{C}$, an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, let $\tau \in \mathcal{T}^+$ be the execution trace and c' be a program satisfying

$$\langle c, \tau_0 \rangle \rightarrow \langle c', \tau \rangle.$$

By induction on c , we have the same cases as the Proof of Lemma 4.2.1 in Appendix A.1.1 excepting the case $c = [x \leftarrow \text{query}(\psi)]^{l'}$.

case: $c = [x \leftarrow e]^l$

By the evaluation rule `assn`, we have $\langle [x \leftarrow a]^l, \tau_0 \rangle \rightarrow \langle [\text{skip}]^l, \tau_0 :: (x, l, v) \rangle$, for some $v \in \mathbb{N}$.

Let $\tau = \tau_0 :: (x, l, v)$ and $\tau' = [(x, l, v)]$, then we have $\tau_{0++}\tau' = \tau$ and this case is proved.

case: $c = c_1; c_2$

By the first rule applied to c in operational semantics, there are two cases as follows.

sub-case: seq1

If the first rule applied to c is **seq1**, we have

$$\frac{\langle c_1, \tau_0 \rangle \rightarrow \langle c'_1, \tau_1 \rangle}{\langle c_1; c_2, \tau_0 \rangle \rightarrow \langle c'_1; c_2, \tau_1 \rangle}$$

By induction hypothesis on c_1 , we know

$$\exists \tau'_1 \in \mathcal{T}^{+\infty} . \tau_{0++}\tau'_1 = \tau_1$$

Let $\tau = \tau_1$ and $\tau' = \tau'_1$, then we have $\tau_{0++}\tau' = \tau$ and this case is proved.

sub-case: seq2

If the first rule applied to c is **seq2**, we have

$$\frac{\langle c_2, \tau_0 \rangle \rightarrow \langle c'_2, \tau_2 \rangle}{\langle [\text{skip}]^l; c_2, \tau_0 \rangle \rightarrow \langle c_2, \tau_2 \rangle}$$

By induction hypothesis on c_2 , we know

$$\exists \tau'_2 \in \mathcal{T}^{+\infty} . \tau_{0++}\tau'_2 = \tau_2$$

Let $\tau = \tau_2$ and $\tau' = \tau'_2$, then we have $\tau_{0++}\tau' = \tau$ and this case is proved.

case: while $[b]^{l_w}$ do (c_w)

By the first rule applied to c in operational semantics, there are two cases as follows.

sub-case: while-t

If the first rule applied to c is **while-t**, we have

$$\langle \text{while } [b]^{l_w} \text{ do } (c_w), \tau_0 \rangle \rightarrow \langle c_w; \text{while } [b]^{l_w} \text{ do } (c_w), \tau_0 :: (b, l_w, \text{true}) \rangle.$$

Let $\tau = \tau_0 :: (b, l_w, \text{true})$ and $\tau' = [(b, l_w, \text{true})]$, then we have $\tau_{0++}\tau' = \tau$ and this case is proved.

sub-case: while-f

If the first rule applied to c is **while-f**, we have

$$\langle \text{while } [b]^{l_w} \text{ do } (c_w), \tau_0 \rangle \xrightarrow{\text{while-f}} \langle \text{skip}, \tau_0 :: (b, l_w, \text{false}) \rangle,$$

Let $\tau = \tau_0 :: (b, l_w, \text{false})$ and $\tau' = [(b, l_w, \text{false})]$, then we have $\tau_{0++}\tau' = \tau$ and this case is proved.

case: if $([b]^l, c_t, c_f)$

By the first rule applied to **if** $([b]^l, c_t, c_f)$ in operational semantics, we have two cases **if-t** and **if-f**. Then both of the two cases will append the evaluation result of the guard $[b]^l$, (b, l_w, true) and (b, l_w, false) respectively.

Formally, the two cases will be proved in the same way as **case: c = while** $[b]^l \text{ do } (c_w)$.

□

Lemma (Uniqueness of the Program Labels). *For every program $c \in \mathcal{C}$ and every two labels such that $i, j \in \text{LV}(c)$, then $i \neq j$.*

$$\forall c \in \mathcal{C}, i, j \in \mathcal{L} . i, j \in \text{LV}(c) \implies i \neq j.$$

Proof. The proof is trivially by induction on program c .

□

B.2 SOUNDNESS OF PROGRAM ABSTRACTION

Lemma (Soundness of The Abstract Execution Trace). *For any program c with an arbitrary initial trace $\tau_0 \in \mathcal{T}_0^+(c)$ and a finite execution trace $\tau \in \mathcal{T}^+$ that is generated when c is executing under τ_0 , then for every event $\epsilon \in \tau$ such that $\epsilon = (_, l, _)$ for some label l in program c , there is an abstract event $\hat{\epsilon} = (l, _, _) \in \text{abstrace}(c)$ and $\hat{\epsilon} = (l, _, _)$.*

$$\begin{aligned} \forall c, c' \in \mathcal{C}, \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, \epsilon = (_, l, _) \in \mathcal{E}, l \in \mathbf{L}(c) . (\langle c, \tau_0 \rangle \rightarrow^* \langle c', \tau_0 ++ \tau \rangle) \wedge \epsilon \in \tau \\ \implies \exists \hat{\epsilon} \in \text{abstrace}(c) \wedge \hat{\epsilon} = (l, _, _) \end{aligned}$$

Proof. Taking an arbitrary program c , an intermediate program $c' \in \mathcal{C}$, an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, a finite execution trace $\tau \in \mathcal{T}^+$, a label $l \in \mathbf{L}(c)$ and an event $\epsilon = (_, l, _) \in \mathcal{E}$ such that

$$\langle c, \tau_0 \rangle \rightarrow^* \langle c', \tau_0 ++ \tau \rangle$$

Then it is sufficient to show,

$$\epsilon \in \tau \implies \exists \hat{\epsilon} \in \text{abstrace}(c) \wedge \hat{\epsilon} = (l, _, _)$$

By induction on program c , we have the following cases:

case: $c = [x \leftarrow e]^{l'}$

By the evaluation rule `assn`, we have

$$\langle [x \leftarrow a]^{l'}, \tau_0 \rangle \rightarrow \langle [\text{skip}]^{l'}, \tau_0 :: (x, l', v) \rangle$$

, for some $v \in \mathbb{N}$.

There are 2 cases, where $l' = l$ and $l' \neq l$.

In case of $l' \neq l$, we know $\epsilon \notin [(x, l', v)]$, then this Lemma is vacuously true.

In case of $l' = l$, by the abstract execution trace computation in Definition 56, we have

$$\text{abstrace}(c) = \text{abstrace}'([x \leftarrow a]^{l'}; [\text{skip}]^{\text{ex}}) = \{(l', \text{absexpr}(e), \text{ex})\}$$

Let $\hat{e} = (l', \text{absexpr}(e), \text{ex})$, then we have $\hat{e} \in \text{abstrace}(c)$ and this case is proved.

case: `while` $[b]^{l_w}$ `do` (c_w)

By the first rule applied to c in operational semantics, there are two cases as follows.

sub-case: while-t

In this case, we have

$$\langle \text{while } [b]^{l_w} \text{ do } (c_w), \tau_0 \rangle \rightarrow \langle c_w; \text{while } [b]^{l_w} \text{ do } (c_w), \tau_0 :: (b, l_w, \text{true}) \rangle.$$

In case of $l \neq l_w$, we know $\epsilon \notin [(b, l_w, \text{true})]$, and this Lemma is vacuously true.

In case of $l = l_w$, let $\epsilon = (b, l_w, \text{true})$ and $\tau' = [(b, l_w, \text{true})]$, then it is sufficient to show.

$$\exists \hat{e} \in \text{abstrace}(c) \wedge \hat{e} = (l, _, _)$$

By abstract trace computation in Definition 56, we have

$$\text{abstrace}(c) = \text{abstrace}'(c; [\text{skip}]^{\text{ex}}) = \{(l_w, \top, \text{init}(c_w))\} \cup \dots$$

Let $\hat{e} = (l_w, \top, \text{init}(c_w))$, then we have this case is proved.

sub-case: while-f

In this case, we have

$$\langle \text{while } [b]^{l_w} \text{ do } (c_w), \tau_0 \rangle \rightarrow^{\text{while-f}} \langle \text{skip}, \tau_0 :: (b, l_w, \text{false}) \rangle.$$

In case of $l \neq l_w$, we know $\epsilon \notin [(b, l_w, \text{false})]$, and this Lemma is vacuously true.

In the case where $l = l_w$, let $\tau = [(b, l_w, \text{false})]$ and $\epsilon = (b, l_w, \text{false})$, then it is sufficient to show

$$\epsilon \in [(b, l_w, \text{false})] \implies \exists \hat{\epsilon} = (l, _, _) \in (\mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L}) . \hat{\epsilon} \in \text{abstrace}(c).$$

By abstract trace computation in Definition 56, we have

$$\text{abstrace}(c) = \text{abstrace}'(c; [\text{skip}]^{\text{ex}}) = \{(l_w, \top, \text{init}(c_w))\} \cup \dots$$

Let $\hat{\epsilon} = (l_w, \top, \text{init}(c_w))$, then we have this case proved.

case: if $([b]^l, c_t, c_f)$

By the first rule applied to $\text{if } ([b]^l, c_t, c_f)$ in operational semantics, we have two subcases if-t and if-f.

Then both of the two cases will append the evaluation result of the guard $[b]^l$, i.e., (b, l_w, true) and (b, l_w, false) respectively.

By abstract trace computation in Definition 56, we have

$$\begin{aligned} \text{abstrace}(c) &= \text{abstrace}'(c; [\text{skip}]^{\text{ex}}) \\ &= \text{abstrace}'(\text{if } [b]^l \text{ then } c_t \text{ else } c_f) \cup \text{abstrace}'([\text{skip}]^{\text{ex}}) \\ &\quad \cup \{(l, dc, \text{absinit}(c_2)) \mid (l, dc) \in \text{absfinal}(c_1)\} \\ &= \{(l, \text{absexpr}(b, \top), \text{absinit}(c_t)), (l, \text{absexpr}(\neg b, \top), \text{absinit}(c_f))\} \\ &\quad \cup \text{abstrace}'(c_t) \cup \text{abstrace}'(c_f) \cup \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1)\} \end{aligned}$$

Let $\hat{e} = (l, \text{absexpr}(b, \top), \text{absinit}(c_t))$ or $(l, \text{absexpr}(\neg b, \top), \text{absinit}(c_f))$ respectively in the two subcases.

Then we have both of the two subcases proved.

case: $c = c_1; c_2$

By the first rule applied to c in operational semantics, there are two cases as follows.

sub-case: seq1

In this case, we have the following where $\tau = \tau_1$.

$$\frac{\langle c_1, \tau_0 \rangle \rightarrow \langle c'_1, \tau_{0++\tau_1} \rangle}{\langle c_1; c_2, \tau_0 \rangle \rightarrow \langle c'_1; c_2, \tau_{0++\tau_1} \rangle}$$

By induction hypothesis on c_1 , we know

$$\epsilon \in \tau_1 \implies \exists \hat{e} = (l, _, _) \in (\mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L}) . \hat{e} \in \text{abstrace}(c_1)$$

Let $\epsilon = (_, l_1, _) \in \tau_1$ and $\hat{e}_1 = (l_1, _, _) \in \text{abstrace}(c_1)$, Then it is sufficient to show

$$\exists \hat{e} \in \text{abstrace}(c) \wedge \hat{e} = (l_1, _, _)$$

By abstract trace computation in Definition 56, we have

$$\begin{aligned} \text{abstrace}(c_1) &= \text{abstrace}'(c_1; [\text{skip}]^{\text{ex}}) \\ &= \text{abstrace}'(c_1) \cup \text{abstrace}'([\text{skip}]^{\text{ex}}) \\ &\quad \cup \{(l, dc, \text{absinit}([\text{skip}]^{\text{ex}})) \mid (l, dc) \in \text{absfinal}(c_1)\} \\ &= \text{abstrace}'(c_1) \cup \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1)\} \end{aligned}$$

and

$$\begin{aligned}
\text{abstrace}(c) &= \text{abstrace}'(c; [\text{skip}]^{\text{ex}}) \\
&= \text{abstrace}'(c_1; c_2) \cup \text{abstrace}'([\text{skip}]^{\text{ex}}) \\
&\quad \cup \{(l, dc, \text{absinit}([\text{skip}]^{\text{ex}})) \mid (l, dc) \in \text{absfinal}(c_1; c_2)\} \\
&= \text{abstrace}'(c_1) \cup \text{abstrace}'(c_2) \\
&\quad \cup \{(l, dc, \text{absinit}(c_2)) \mid (l, dc) \in \text{absfinal}(c_1)\} \cup \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1; c_2)\}
\end{aligned}$$

Then we have two subcases where $\hat{e}_1 \in \text{abstrace}'(c_1)$ or $\hat{e}_1 \in \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1; c_2)\}$.

sub-case: $\hat{e}_1 \in \text{abstrace}'(c_1)$

Since $\text{abstrace}'(c_1) \subseteq \text{abstrace}(c)$, then we know $\hat{e}_1 \in \text{abstrace}(c)$.

Let $\hat{e} = \hat{e}_1$, then we have $\hat{e} \in \text{abstrace}(c)$ and this subcase is proved.

sub-case: $\hat{e}_1 \in \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1; c_2)\}$

In this case, we know $(l_1, _) \in \text{absfinal}(c_1)$.

Let $\hat{e} = (l_1, _, \text{absinit}(c_2))$, then we know

$$\hat{e} \in \{(l, dc, \text{absinit}(c_2)) \mid (l, dc) \in \text{absfinal}(c_1)\}.$$

Since $\{(l, dc, \text{absinit}(c_2)) \mid (l, dc) \in \text{absfinal}(c_1)\} \subset \text{abstrace}(c)$, then we have $\hat{e} \in \text{abstrace}(c)$ and this subcase is proved.

sub-case: seq2

In this case, we have the following where $\tau = \tau_2$.

$$\frac{\langle c_2, \tau_0 \rangle \rightarrow \langle c'_2, \tau_0 + \tau_2 \rangle}{\langle [\text{skip}]^l; c_2, \tau_0 \rangle \rightarrow \langle c_2, \tau_0 + \tau_2 \rangle}$$

By induction hypothesis on c_2 , we know

$$\epsilon \in \tau_2 \implies \exists \hat{\epsilon} = (l, _, _) \in (\mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L}) . \hat{\epsilon} \in \text{abstrace}(c_2)$$

Let $\epsilon = (_, l_2, _) \in \tau_2$ and $\hat{\epsilon}_2 = (l_2, _, _) \in \text{abstrace}(c_2)$, we know $\epsilon \in \tau$ as well. Then it is sufficient to show

$$\exists \hat{\epsilon} \in \text{abstrace}(c) \wedge \hat{\epsilon} = (l_2, _, _)$$

By abstract trace computation in Definition 56, we have

$$\begin{aligned} \text{abstrace}(c_2) &= \text{abstrace}'(c_2; [\text{skip}]^{\text{ex}}) \\ &= \text{abstrace}'(c_2) \cup \text{abstrace}'([\text{skip}]^{\text{ex}}) \\ &\quad \cup \{(l, dc, \text{absinit}([\text{skip}]^{\text{ex}})) \mid (l, dc) \in \text{absfinal}(c_2)\} \\ &= \text{abstrace}'(c_2) \cup \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_2)\} \end{aligned}$$

and

$$\begin{aligned} \text{abstrace}(c) &= \text{abstrace}'(c; [\text{skip}]^{\text{ex}}) \\ &= \text{abstrace}'(c_1; c_2) \cup \text{abstrace}'([\text{skip}]^{\text{ex}}) \\ &\quad \cup \{(l, dc, \text{absinit}([\text{skip}]^{\text{ex}})) \mid (l, dc) \in \text{absfinal}(c_1; c_2)\} \\ &= \text{abstrace}'(c_1) \cup \text{abstrace}'(c_2) \\ &\quad \cup \{(l, dc, \text{absinit}(c_2)) \mid (l, dc) \in \text{absfinal}(c_1)\} \cup \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1; c_2)\} \end{aligned}$$

Then we have two subcases where $\hat{\epsilon}_2 \in \text{abstrace}'(c_2)$ or $\hat{\epsilon}_2 \in \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_2)\}$.

sub-case: $\hat{\epsilon}_2 \in \text{abstrace}'(c_2)$

Since $\text{abstrace}'(c_2) \subseteq \text{abstrace}(c)$, then we know $\hat{\epsilon}_2 \in \text{abstrace}(c)$.

Let $\hat{e} = \hat{e}_2$, then we have $\hat{e} \in \text{abstrace}(c)$ and this subcase is proved.

sub-case: $\hat{e}_2 \in \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_2)\}$

In this case, we know $(l_2, _) \in \text{absfinal}(c_2)$. By abstract continuation state computation in Section 12.2.2, we know

$$\text{absfinal}(c_1; c_2) = \text{absfinal}(c_2)$$

Then we know $(l_2, _) \in \text{absfinal}(c_1; c_2)$. Let $\hat{e} = \hat{e}_2$, then we know

$$\hat{e} = \hat{e}_2 \in \{(l, dc, \text{ex}) \mid (l, dc) \in \text{absfinal}(c_1; c_2)\} \subset \text{abstrace}(c).$$

Then we have $\hat{e} \in \text{abstrace}(c)$ and this subcase is proved. $\square$

For every labeled variable in program c , $x^l \in \text{LV}_c$, there is a unique abstract event in program's abstract execution trace $\hat{e} \in \text{abstrace}(c)$ of form $(l, _, _)$.

Lemma (Uniqueness of the Abstract execution trace). *For every program c and a finite execution trace $\tau \in \mathcal{T}^+$ which is generated w.r.t. an initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, there is a unique abstract event $\hat{e} = (l, _, _) \in \text{abstrace}(c)$ for every assignment event $\epsilon \in \mathcal{E}^{\text{asn}}$ in the execution trace having the label l , i.e., $\epsilon = (_, l, _)$ and $\epsilon \in \tau$.*

$$\begin{aligned} \forall c \in \mathcal{C}, \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, \epsilon = (_, l, _) \in \mathcal{E}^{\text{asn}}. \left(\langle c, \tau_0 \rangle \rightarrow^* \langle c', \tau_0 + \tau \rangle \right) \wedge \epsilon \in \tau \\ \implies \exists! \hat{e} = (l, _, _) \in (\mathcal{L} \times \mathcal{DC}^\top \times \mathcal{L}). \hat{e} \in \text{abstrace}(c) \end{aligned}$$

Proof. This is proved trivially by induction on the program c . $\square$

B.3 SOUNDNESS OF RANKING ESTIMATION

Lemma B.3.1 (Soundness of the Ranking Function Estimation). *Given a program c , we have:*

$$\forall \hat{e} = (l, dc, l') . \max\{\text{cnt}(\tau')l \mid \forall \tau \in \mathcal{T} . \langle c, \tau \rangle \rightarrow^* \langle \text{skip}, \tau_{++}\tau' \rangle\} \leq \text{rank}(\hat{e})$$

Proof.

sub-case: $l \notin \text{SCC}(\text{absG}(c))$

In this case, we know variable x^l isn't involved in the body of any `while` command.

Taking an arbitrary $\tau_0 \in \mathcal{T}$, let $\tau \in \mathcal{T}$ be of resulting trace of executing c with τ , i.e., $\langle c, \tau_0 \rangle \rightarrow^* \langle \text{skip}, \tau \rangle$,

we know the assignment command at line l associated with the abstract event $\hat{e}$ will be executed at most once, i.e.,: $\text{cnt}(\tau)l \leq 1$

By rank definition, we know $\text{rank}(\hat{e}) = 1$.

This case is proved.

sub-case: $l \in \text{SCC}(\text{absG}(c)) \wedge \hat{e} \in \text{dec}(x)$

In this case, we know symbol x is decreasing on this transition. So symbol x is a witness of the execution times of this transition.

In this sense, it is safe to assign $\text{rank}(\hat{e}) \triangleq \text{rank}(\hat{e}', c)$.

sub-case: $\exists \hat{e}' \in \text{absG}(c), \text{tp} \in \text{PATH}(\text{absG}(c)) . \hat{e}, \hat{e}' \in \text{tp}$

In this case, we know both $\hat{e}'$ and $\hat{e}$ are on the same simple transition path.

By Definition 60, we know every tp is loop free and interleaving free. Then we know execution of the transitions $\hat{e}'$ and $\hat{e}$ must execute following each other.

In this sense, it is safe to assign $\text{rank}(\hat{e}) \triangleq \text{rank}(\hat{e}')$.

sub-case: $l \in SCC(\text{absG}(c)) \wedge \hat{e} \notin \bigcup_{x \in VAR} \text{dec}(x) \wedge \hat{e} \notin SCC(\text{absG}(c)/\text{dec}(x))$

in this case, we know $\text{rank}(\hat{e}) \triangleq x$.

In the two cases above, the soundness is discussed in Sinn et al. (2017) Section 4 of Paragraph *Discussion on Soundness* in Page 25. $\square$

Theorem (Soundness of the Path-insensitive Transition Bound). *For each program c and an edge $\hat{e} = (l, _, _) \in \text{absG}(c)$, if l is the label of an assignment command, then its path-insensitive transition bound $\text{TB}(\hat{e}, c)$ is a sound upper bound on the execution times of this assignment command in c .*

$$\begin{aligned} \forall \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^{+\infty}, c \in \mathcal{C}, l, l' \in \text{LV}(c) . \\ \left(\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \vee \langle c, \tau_0 \rangle \uparrow^\infty \tau_{0++\tau} \right) \\ \implies \exists \hat{e} = (_, l, _) \in \text{abstrace}(c) \wedge \text{cnt}(\tau, l) \leq \llbracket \text{TB}(\hat{e}, c) \rrbracket(\tau_0) \end{aligned}$$

Proof. Taking arbitrary $\tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^{+\infty}, c, c' \in \mathcal{C}$ and a program point such that

$$\left(\langle c, \tau_0 \rangle \rightarrow^* \langle \text{skip}, \tau_{0++\tau} \rangle \vee \langle c, \tau_0 \rangle \rightarrow^\infty \langle c', \tau_{0++\tau} \rangle \right)$$

Then it is sufficient to show

$$\forall l \in \text{LV}(c) \exists \hat{e} = (_, l, _) \in \text{abstrace}(c) \wedge \text{cnt}(\tau', l) \leq \llbracket \text{TB}(\hat{e}, c) \rrbracket(\tau_0)$$

Let $l \in \text{LV}(c)$ be arbitrary program point, By Lemma 6.3.1, there exists an abstract event in $\text{abstrace}(c)$ of form $(\hat{e}) = (l, _, _)$, corresponding to the assignment command associated to labeled variable x^l .

Let $(\hat{e}) = (l, dc, l') \in \text{abstrace}(c)$ be this event for some dc and l' such that $(\hat{e}) = (l, dc, l') \in \text{abstrace}(c)$, then it is sufficient to show:

$$\text{cnt}(\tau, l) \leq \llbracket \text{TB}(\hat{e}, c) \rrbracket(\tau_0)$$

By definition of $\text{TB}(\hat{e})$:

$$\begin{array}{ll} \text{rank}(\hat{e}) & \text{rank}(\hat{e}) \in \mathcal{SC} \\ \text{inc}(\text{rank}(\hat{e})) + \sum \{\text{TB}(\hat{e}') \times \max(\text{VB}(a) + c, 0) \mid (\hat{e}', a, c) \in \text{re}(\text{rank}(\hat{e}))\} & \text{rank}(\hat{e}) \notin \mathcal{SC} \end{array}$$

case: $\text{rank}(\hat{e}) \in \mathcal{SC}$

Proved by the soundness of Local bound in Lemma B.3.1.

case: $\text{rank}(\hat{e}) \notin \mathcal{SC}$

To show:

$$\begin{aligned} & \max\{\text{cnt}(\tau', l) \mid \forall \tau \in \mathcal{T} . \langle c, \tau \rangle \rightarrow^* \langle \text{skip}, \tau_{++}\tau' \rangle\} \\ & \leq \llbracket \text{inc}(\text{rank}(\hat{e})) \rrbracket(\tau_0) + \sum \{\llbracket \text{TB}(\hat{e}') \times \max(\text{VB}(a) + c, 0) \rrbracket(\tau_0) \mid (\hat{e}', a, c) \in \text{re}(\text{rank}(\hat{e}))\} \end{aligned}$$

Taking an arbitrary initial trace $\tau_0 \in \mathcal{T}$, executing c with τ_0 , let τ be the trace after evaluation, i.e., $\langle c, \tau_0 \rangle \rightarrow^* \langle \text{skip}, \tau \rangle$, it is sufficient to show:

$$\text{cnt}(\tau', l) \leq \llbracket \text{inc}(\text{rank}(\hat{e})) + \sum \{\text{TB}(\hat{e}') \times \max(\text{VB}(a) + c, 0) \mid (\hat{e}', a, c) \in \text{re}(\text{rank}(\hat{e}))\} \rrbracket(\tau_0)$$

By the soundness of the (1) Transition Bound and (2) Variable Bound Invariant in Sinn et al. (2017) Theorem 1, This case is proved. $\square$

B.4 SOUNDNESS OF PATH-SENSITIVE REACHABILITY-BOUNDS ANALYSIS

B.4.1 Loop Bound Soundness (Proof of Lemma 12.6.1)

There are three methods for computing the loop bound, $BD(\text{repeat}(\text{rp}'), c)$ for every loop $\text{repeat}(\text{rp}')$ in a program c 's refined program rp :

1. The first computation fully relies on the soundness of $BOUND(\text{repeat}(\text{rp}'), c)$ from paper Gulwani et al. (2009a), we do not repeat their soundness in this paper.
2. The second computation approach by Equation 12.1 is path-insensitively and sound. Its soundness relies on the soundness of the $TB(\hat{e}, c)$ in Definition 63 in Section 12.4. For a program c and an edge $\hat{e} \in \text{absE}(c)$, $TB(\hat{e}, c)$ is a sound upper bound on the execution times of this transition, formally in Theorem 12.4.1 with proof in Appendix B.3
3. The soundness of our new approach in Definition 70 is proved below.

For every transition path rp , the $BD(\text{rp}, c)$ is a sound upper bound on its execution times.

This bound is sound locally by assuming that all the loops and transition paths where rp is nested execute only once. This assumption comes from the computation of the varGD and the depth first search strategy.

Lemma (Soundness of Loop Bound). *For every program c_r , with a sub program rp in c_r 's refined program, $BD(\text{rp}, c)$ is a sound upper bound on the iterating times of this sub*

refined program rp .

$$\begin{aligned}
& \forall c, c_r \in \mathcal{C}, \tau_0 \in \mathcal{T}_0^+(c_r), \tau_r \in \mathcal{T}_0^+(c_r), \tau \in \mathcal{T}^{+\infty}, l \in \text{LV}(c), \text{rp} . \\
& \text{rp} \subseteq \text{REFINE}(\text{Rewrite}(c)) \wedge \text{rp} = \text{Rewrite}(c_r) \\
& \wedge \left(\langle c_r, \tau_r \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++\tau} \rangle \vee \langle c_r, \tau_r \rangle \uparrow^\infty \tau_{r++\tau} \right) \\
& \implies \llbracket BD(\text{rp}, c) \rrbracket(\tau_0) \\
& \geq \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f}, \tau', \tau_a \in \mathcal{T}^+, \tau_b \in \mathcal{T}^{+\infty} \wedge \tau = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \right. \\
& \quad \wedge \tau_{l_s \rightarrow l_f} = [(-, l_s, -)]_{++\tau'}_{++}[(-, l_f, -)] \\
& \quad \left. \wedge (l_f, -) \in \text{absfinal}(\text{rp}) \wedge l_s = \text{absinit}(\text{rp}) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \right\}.
\end{aligned}$$

Below we prove the soundness of a more general version Loop Bound Computation, $BD(\text{tp}, c)$, for an arbitrary sub program rp in c_r 's refined program

$$\left\{ \begin{array}{ll} 1 & \text{rp} = \text{tp} \\ \min\{BD(\text{rp}_1, c), BD(\text{rp}_2, c)\} & \text{rp} = \text{rp}_1; \text{rp}_2 \\ \max\{BD(\text{rp}_1, c), \dots, BD(\text{rp}_m, c)\} & \text{rp} = \text{choose}\{\text{rp}_1, \dots, \text{rp}_m\} \\ \max_{x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}} \left\{ \frac{a-b}{\text{varGD}(\text{tp}, c)} \right. & \text{rp} = \text{repeat}(\text{tp}) \\ \quad \left. \mid x = a \in \text{rInit}(\text{tp}, c) \wedge x = b \in \text{rFinal}(\text{tp}, c) \right\} & \\ \max_{x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}} \left\{ \frac{\min\{a, a - \text{varGD}(\text{rp}_1, c)\} - b}{\text{varGD}(\text{rp}_1; \text{rp}_2, c)} \right. & \text{rp} = \text{repeat}(\text{rp}_1; \text{rp}_2) \\ \quad \left. \mid x = a \in \text{rInit}(\text{rp}_1, c) \right. & \\ \quad \quad \left. \wedge x = b \in \text{rFinal}(\text{rp}_2; \text{rp}_2, c) \right\} & \\ 0 & o.w. \end{array} \right.$$

Proof. Soundness of the Loop Bound.

Taking an arbitrary global program c , let $\text{rp} \subseteq \text{REFINE}(\text{Rewrite}(c))$ be a sub program of its refined program and program c_r be the while language program where $\text{rp} = \text{Rewrite}(c_r)$

Taking an arbitrary initial trace of c , $\tau_0 \in \mathcal{T}_0^+(c)$ and an arbitrary initial trace of c_r , $\tau_r \in \mathcal{T}_0^+(c_r)$, an execution trace $\tau \in \mathcal{T}^{+\infty}$ and label $l \in \text{LV}(c_r)$ such that

$$\langle c_r, \tau_r \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++\tau} \rangle \vee \langle c_r, \tau_r \rangle \uparrow^\infty \tau_{r++\tau},$$

we have two cases.

- **The execution terminates and** $\langle c_r, \tau_r \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++\tau} \rangle$.

In this case we know $\tau \in \mathcal{T}^+$ and is sufficient to show,

$$\begin{aligned} & \llbracket BD(\text{rp}, c) \rrbracket(\tau_0) \\ & \geq \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f}, \tau_a \in \mathcal{T}^+, \tau_b \in \mathcal{T}^+ \wedge \tau = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \right. \\ & \quad \wedge \tau_{l_s \rightarrow l_f} = [(-, l_s, -)]_{++\tau'++} [(-, l_f, -)] \\ & \quad \left. \wedge (l_f, -) \in \text{absfinal}(\text{rp}) \wedge l_s = \text{absinit}(\text{rp}) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \right\}. \end{aligned}$$

By induction on the refined program, we have the following cases.

case: $\text{rp} = \text{tp}$

In this case, we have $BD(\text{tp}, c) = 1$.

By Algorithm 7 and Definition 60, we know program c_r contains the only one transition path tp .

So we know $\text{absinit}(\text{tp}) = 0$ and $\text{absfinal}(\text{tp}) = \{(l_f, -)\}$ for some $l_f \in \mathbb{N}$.

We also know tp will be executed at most once and $\tau_{0 \rightarrow l_f} = \tau$. So we have

$$\begin{aligned} & \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f}, \tau_a \in \mathcal{T}^+, \tau_b \in \mathcal{T}^+ \wedge \tau = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \right. \\ & \quad \wedge \tau_{l_s \rightarrow l_f} = [(-, 0, -)]_{++\tau'++} [(-, l_f, -)] \\ & \quad \left. \wedge (l_f, -) \in \text{absfinal}(\text{rp}) \wedge \text{cnt}(\tau', 0) = \text{cnt}(\tau', l_f) = 0 \right\} = \# \left\{ \tau \right\} = 1. \end{aligned}$$

Then we have this case proved.

case: $\text{rp} = \text{choose}\{\text{rp}_1, \dots, \text{rp}_m\}$

In this case, we have

$$BD(\text{choose}\{\text{rp}_1, \dots, \text{rp}_m\}, c) = \max\{BD(\text{rp}_1, c), \dots, BD(\text{rp}_m, c)\}.$$

Let $c_1, \dots, c_m \in \mathcal{C}$ be the while language program corresponding to $\text{rp}_1, \dots, \text{rp}_m$ such that $\text{Rewrite}(c_1) = \text{rp}_1, \dots$ and $\text{Rewrite}(c_m) = \text{rp}_m$. According to the Algorithm 7 and the program refinement algorithm REFINER, we know each rp_i is a loop program refined from multiple paths in c_r .

In this sense, each c_i is a loop program and execution of each c_i corresponds to one possible execution of c_r .

According to the operational semantics, let $\tau_1, \dots, \tau_m \in \mathcal{T}^+$ be the execution traces such that

$$\langle c_i, \tau_r \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++\tau_i} \rangle$$

By induction hypothesis, for each c_i, rp_i and the execution trace τ_i , we have

$$\begin{aligned} & \llbracket BD(\text{rp}_i, c) \rrbracket(\tau_0) \\ & \geq \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f}, \tau_a \in \mathcal{T}^+, \tau_b \in \mathcal{T}^+ \wedge \tau_i = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \right. \\ & \quad \wedge \tau_{l_s \rightarrow l_f} = [(_, l_s, _)]_{++\tau'++}[(_, l_f, _)] \\ & \quad \left. \wedge (l_f, _) \in \text{absfinal}(\text{rp}_i) \wedge l_s = \text{absinit}(\text{rp}_i) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \right\}. \end{aligned}$$

Since execution of each c_i corresponds to one possible execution of c_r , we also know $\tau = \tau_i$ where i could be either $1, \dots, m$. So we have for each τ_i if $\tau = \tau_i$,

$$\begin{aligned} & \llbracket BD(\text{rp}_i, c) \rrbracket(\tau_0) \geq \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f}, \tau_a \in \mathcal{T}^+, \tau_b \in \mathcal{T}^+ \wedge \tau = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \right. \\ & \quad \wedge \tau_{l_s \rightarrow l_f} = [(_, l_s, _)]_{++\tau'++}[(_, l_f, _)] \\ & \quad \left. \wedge (l_f, _) \in \text{absfinal}(\text{rp}_i) \wedge l_s = \text{absinit}(\text{rp}_i) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \right\}. \end{aligned}$$

Then we have for all rp_i the following and this case is proved.

$$\begin{aligned} \llbracket BD(\text{rp}, c) \rrbracket(\tau_0) &= \llbracket \max\{BD(\text{rp}_1, c), \dots, BD(\text{rp}_m, c)\} \rrbracket(\tau_0) \\ &\geq \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f}, \tau_a \in \mathcal{T}^+, \tau_b \in \mathcal{T}^+ \wedge \tau = \tau_{a++}\tau_{l_s \rightarrow l_f}++\tau_b \right. \\ &\quad \wedge \tau_{l_s \rightarrow l_f} = [(-, l_s, -)]_{++}\tau'_{++}[(-, l_f, -)] \\ &\quad \left. \wedge (l_f, -) \in \text{absfinal}(\text{rp}) \wedge l_s = \text{absinit}(\text{rp}) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \right\}. \end{aligned}$$

case: $\text{rp} = \text{rp}_1; \text{rp}_2$

In this case, we have

$$BD(\text{rp}_1; \text{rp}_2, c) = \min\{BD(\text{rp}_1, c), BD(\text{rp}_2, c)\}$$

Let $c_1, c_2 \in \mathcal{C}$ be the while language program corresponding to rp_1 and rp_2 such that $\text{Rewrite}(c_1) = \text{rp}_1$ and $\text{Rewrite}(c_2) = \text{rp}_2$.

According to the operational semantics, let $\tau_1, \tau_2 \in \mathcal{T}^+$ be two execution traces such that

$$\langle c_1; c_2, \tau_r \rangle \rightarrow^* \langle c_2, \tau_{r++}\tau_1 \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++}\tau_1++\tau_2 \rangle.$$

By the trace non-decreasing property in Lemma 4.2.1, we have $\tau = \tau_1++\tau_2$.

According to the Algorithm 7 and REFINe algorithm, we have $c_r = c_1; c_2$.

By induction hypothesis on τ_r, c_1, τ_1 and c_2 , we also have

$$\begin{aligned} \llbracket BD(\text{rp}_1, c) \rrbracket(\tau_0) &\geq \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau_a, \tau_b \in \mathcal{T}^+ . \right. \\ &\quad \tau_1 = \tau_{a++}\tau_{l_s \rightarrow l_f}++\tau_b \wedge \tau_{l_s \rightarrow l_f} = [(-, l_s, -)]_{++}\tau'_{++}[(-, l_f, -)] \\ &\quad \left. \wedge (l_f, -) \in \text{absfinal}(c_1) \wedge l_s = \text{absinit}(c_1) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \right\}. \end{aligned}$$

$$\begin{aligned}
& \llbracket BD(\mathbf{rp}_2, c) \rrbracket(\tau_0) \\
& \geq \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau_a, \tau_b \in \mathcal{T}^+ . \right. \\
& \quad \tau_2 = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = [(_, l_s, _)]_{++\tau'_{++}}[(_, l_f, _)] \\
& \quad \left. \wedge (l_f, _) \in \text{absfinal}(\mathbf{rp}_2) \wedge l_s = \text{absinit}(\mathbf{rp}_2) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \right\}.
\end{aligned}$$

By Definition 55, we know $\text{absinit}(\mathbf{rp}) = \text{absinit}(c_1)$, $\text{absfinal}(\mathbf{rp}) = \text{absfinal}(\mathbf{rp}_2)$ and $(\text{absinit}(\mathbf{rp}_2), _) \in \text{absfinal}(c_1)$.

By the label consistency, we know

$$\begin{aligned}
& \forall \tau \in \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau_a, \tau_b \in \mathcal{T}^+ . \right. \\
& \quad \tau = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = [(_, l_s, _)]_{++\tau'_{++}}[(_, l_f, _)] \\
& \quad \wedge (l_f, _) \in \text{absfinal}(\mathbf{rp}_1; \mathbf{rp}_2) \wedge l_s = \text{absinit}(\mathbf{rp}_1; \mathbf{rp}_2) \\
& \quad \left. \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \right\}, \\
& \exists \tau_1 \in \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau_a, \tau_b \in \mathcal{T}^+ . \right. \\
& \quad \tau_1 = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = [(_, l_s, _)]_{++\tau'_{++}}[(_, l_f, _)] \\
& \quad \wedge (l_f, _) \in \text{absfinal}(\mathbf{rp}_1) \wedge l_s = \text{absinit}(\mathbf{rp}_1) \\
& \quad \left. \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \right\}, \\
& \tau_2 \in \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau_a, \tau_b \in \mathcal{T}^+ . \right. \\
& \quad \tau_2 = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = [(_, l_s, _)]_{++\tau'_{++}}[(_, l_f, _)] \\
& \quad \left. \wedge (l_f, _) \in \text{absfinal}(\mathbf{rp}_2) \wedge l_s = \text{absinit}(\mathbf{rp}_2) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \right\} . \\
& \tau = \tau_1++\tau_2.
\end{aligned}$$

Then we have

$$\begin{aligned}
& \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau_a, \tau_b \in \mathcal{T}^+ . \right. \\
& \quad \tau = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = [(-, l_s, -)]_{++\tau'++} [(-, l_f, -)] \\
& \quad \wedge (l_f, -) \in \text{absfinal}(\text{rp}_1; \text{rp}_2) \wedge l_s = \text{absinit}(\text{rp}_1; \text{rp}_2) \\
& \quad \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \Big\} \\
& \leq \min \left\{ \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau_a, \tau_b \in \mathcal{T}^+ . \right. \right. \\
& \quad \tau_1 = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = [(-, l_s, -)]_{++\tau'++} [(-, l_f, -)] \\
& \quad \wedge (l_f, -) \in \text{absfinal}(\text{rp}_1) \wedge l_s = \text{absinit}(\text{rp}_1) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \Big\}, \\
& \quad \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau_a, \tau_b \in \mathcal{T}^+ . \right. \\
& \quad \tau_2 = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = [(-, l_s, -)]_{++\tau'++} [(-, l_f, -)] \\
& \quad \wedge (l_f, -) \in \text{absfinal}(\text{rp}_2) \wedge l_s = \text{absinit}(\text{rp}_2) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \Big\} \Big\} .
\end{aligned}$$

Then we have

$$\begin{aligned}
& \llbracket BD(\text{rp}_1; \text{rp}_2, c) \rrbracket(\tau_0) = \llbracket \min\{BD(\text{rp}_1, c), BD(\text{rp}_2, c)\} \rrbracket(\tau_0) \\
& = \min \left\{ \llbracket BD(\text{rp}_1, c) \rrbracket(\tau_0), \llbracket BD(\text{rp}_2, c) \rrbracket(\tau_0) \right\} \\
& \geq \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau_a, \tau_b \in \mathcal{T}^+ . \right. \\
& \quad \tau = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = [(-, l_s, -)]_{++\tau'++} [(-, l_f, -)] \\
& \quad \wedge (l_f, -) \in \text{absfinal}(\text{rp}_1; \text{rp}_2) \wedge l_s = \text{absinit}(\text{rp}_1; \text{rp}_2) \\
& \quad \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \Big\} .
\end{aligned}$$

This case is proved.

case: $\text{rp} = \text{repeat}(\text{rp}_l)$

In this case, we have $BD(\text{repeat}(\text{rp}_l), c) =$

$$\left\{ \begin{array}{ll} \max_{x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}} \left\{ \frac{a-b}{\text{varGD}(\text{tp}, c)} \right. & \text{rp}_l = (\text{tp}) \\ \left. \mid x = a \in \text{rInit}(\text{tp}, c) \wedge x = b \in \text{rFinal}(\text{tp}, c) \right\} & \\ \frac{\min\{\text{rInit}(\text{rp}_1), \text{rInit}(\text{rp}_1) - \text{varGD}(\text{rp}_1)\} - \text{rFinal}(\text{rp}_2; \text{rp}_2)}{\text{varGD}(\text{rp}_1; \text{rp}_2)} & \text{rp}_l = (\text{rp}_1; \text{rp}_2) \\ 0 & o.w. \end{array} \right.$$

Then we have the soundness guaranteed by the three states, $\text{rInit}(\text{rp})$, $\text{rFinal}(\text{rp})$, $\text{rNext}(\text{rp})$ computed in Definition 73.

For each $\text{tp} \in \text{rp}$ and $x = \text{rank}(\text{tp})$ the ranking function of this path, we prove the soundness of the three states respectively as follows.

- *Initial State Soundness.* By Definition 69, we have $\text{rInit}(\text{rp}) =$

$$\bigcup_{x \in \{\text{rank}(\hat{e}, c) \mid \text{tp} \in \text{rp} \wedge \hat{e} \in \text{tp}\}} \left\{ x = \max_{l_1} \left\{ \text{VB}(y, c) + v \mid \begin{array}{l} (l_1, x' \leq y + v, l_2) \in \text{re}(x, c) \\ \wedge l_1 \leq \text{absinit}(\text{rp}, c). \end{array} \right\} \right\}$$

Taking an arbitrary ranking function $x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}$, let $\tau' \in \mathcal{T}^+$ be the execution trace before first execution of $l_f : \text{rp}_t$, we have

$$\langle c_l, \tau_r \rangle \rightarrow^* \langle c_t; c'_r, \tau_{r++}\tau' \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++}\tau \rangle$$

By the soundness of the rank estimation in Lemma B.3.1 and Theorem 12.4.1, we know for each variable y such that $(l_1, x' \leq y + v, l_2) \in \text{re}(x, c)$, $\text{VB}(y, c)$ is the sound upper bound on its maximum value during the execution. Then we have

$$\llbracket \text{VB}(y, c) \rrbracket(\tau_0) \geq \max\{v \mid \rho(\tau)y = v\} \geq \rho(\tau')y$$

In this sense, we have $\max_{l_1} \left\{ \text{VB}(y, c) + v \mid \begin{array}{l} (l_1, x' \leq y + v, l_2) \in \text{re}(x, c) \\ \wedge l_1 \leq \text{absinit}(\text{rp}) \end{array} \right\}$ is a sound initial value estimation of the value of x before first execution of rp , i.e.,

$$\llbracket \max_{l_1} \left\{ \text{VB}(y, c) + v \mid \begin{array}{l} (l_1, x' \leq y + v, l_2) \in \text{re}(x, c) \\ \wedge l_1 \leq \text{absinit}(\text{rp}) \end{array} \right\} \rrbracket(\tau_0) \geq \rho(\tau')x.$$

- The *Next State*, $\text{rNext}(\text{tp}, c) \in \mathbf{A}_{\text{sc}}(c)$ is a symbolic expression describing how much tp 's ranking function ($\text{rank}(\text{tp})$) is changed after the first execution of rp and before the second execution.

By Definition 73, we have $\text{rNext}(\text{tp}, c) =$

$$\bigcup_{x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}} \left\{ x = \begin{array}{l} \sum_{\hat{e} \in \text{inc}(x, c)} \{v \mid \hat{e} = (l, x' \leq x + v, _) \wedge l \in \text{tp}\} \\ + \max_{l'} \{ \text{VB}(y, c) + v \mid (l, x' \leq y + v, l') \in \text{re}(x, c) \wedge l \in \text{tp} \} \\ - \sum_{\hat{e} \in \text{dec}(x, c)} \{v \mid \hat{e} = (l, x' \leq x - v, _) \wedge l \in \text{tp}\} \end{array} \right\}.$$

Taking an arbitrary ranking function $x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}$, let $\tau' \in \mathcal{T}^+$ be the execution trace before the second execution of tp , we have

$$\langle c_r, \tau_r \rangle \rightarrow^* \langle c'_r, \tau_{r++\tau'} \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++\tau} \rangle$$

By the soundness of the rank estimation in Lemma B.3.1 and Theorem 12.4.1, we know for each variable y such that $(l_1, x' \leq y + v, l_2) \in \text{re}(x, c)$, $\text{VB}(y, c)$ is the sound upper bound on its maximum value during the execution. Then

we have

$$\llbracket \text{VB}(y, c) \rrbracket(\tau_0) \geq \max\{v \mid \pi_3(\epsilon)y = v \wedge \epsilon \in \tau' \wedge \pi_1(\epsilon) = y\} \geq \rho(\tau')y.$$

Same for each abstract event such that $((l', x' \leq x + v, _) \in \text{inc}(x, c)$ and $(l', x' \leq x + v, _) \in \text{dec}(x, c)$, we know

$$\begin{aligned} & \sum_{\hat{e} \in \text{inc}(x, c)} \{v \mid \hat{e} = (l, x' \leq x + v, _) \wedge l \in \text{tp}\} \\ & + \max_{\nu'} \{\text{VB}(y, c) + v \mid (l, x' \leq y + v, l') \in \text{re}(x, c) \wedge l \in \text{tp}\} \text{ bounds the value} \\ & - \sum_{\hat{e} \in \text{dec}(x, c)} \{v \mid \hat{e} = (l, x' \leq x - v, _) \wedge l \in \text{tp}\} \end{aligned}$$

modification of x in one execution of tp . i.e.,

$$\begin{aligned} & \llbracket \text{rInit}(l : \text{rp}) \rrbracket(\tau_0) - \\ & \sum_{\hat{e} \in \text{inc}(x, c)} \{v \mid \hat{e} = (l, x' \leq x + v, _) \wedge l \in \text{tp}\} \\ & \llbracket \quad + \max_{\nu'} \{\text{VB}(y, c) + v \mid (l, x' \leq y + v, l') \in \text{re}(x, c) \wedge l \in \text{tp}\} \rrbracket(\tau_0) \\ & - \sum_{\hat{e} \in \text{dec}(x, c)} \{v \mid \hat{e} = (l, x' \leq x - v, _) \wedge l \in \text{tp}\} \\ & \leq \rho(\tau')x. \end{aligned}$$

- $\text{rFinal}(\text{rp}, c)$ computes the value of each ranking function

$$x \in \{\text{rank}(\hat{e}, c) \mid \text{tp} \in \text{rp} \wedge \hat{e} \in \text{tp}\}$$

after the iteration of $l_f : \text{rp}_t$ finished. Taking an arbitrary ranking function $x \in \{\text{rank}(\hat{e}, c) \mid \text{tp} \in \text{rp} \wedge \hat{e} \in \text{tp}\}$, let $\tau' \in \mathcal{T}^+$ be the execution trace after the execution of rp , then we have

$$\langle c_r, \tau_r \rangle \rightarrow^* \langle c'_r, \tau_{r++\tau'} \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++\tau} \rangle.$$

So we know for each tp in rp with $x = \text{rank}(\text{tp})$, $\text{rFinal}(\text{rp}, c)$ computes the lower bound on the value of x after the execution of rp finished.

Then we know for each $x = a \in \text{rInit}(\text{rp}, c)$ and $x = b \in \text{rFinal}(\text{rp}, c)$ the following holds and this case is proved.

$$\llbracket a - b \rrbracket(\tau_0) \geq \rho(\tau')x.$$

- *Variable Gradient Decent Soundness.* $\text{varGD}(\text{rp}, c)$ contains the equations $x = e$, where $e \in \mathbf{A}_{\text{sc}}(c)$ is a symbolic expression describing the value of x before the second execution of rp , for each transition path tp in this program with x as the ranking function.

By induction on the rp , we have the following subcases.

- **sub-case:** $\text{rp}_l = \text{tp}$

By Definition 69, we have

$$\text{varGD}(\text{tp}, c) = \min\{v \mid x = v \in \text{rNext}(\text{tp}, c)\}.$$

In this case, we have the soundness guaranteed by the soundness of $\text{rNext}(\text{tp}, c)$ for each transition path tp .

- **sub-case:** $\text{rp}_l = \text{rp}_1; \text{rp}_2$

By Definition 69, we have

$$\text{varGD}(\text{rp}_1; \text{rp}_2, c) = \text{varGD}(\text{rp}_1, c) + \text{varGD}(\text{rp}_2, c).$$

By induction hypothesis on rp_1 and rp_2 , we know

for each $\text{tp}_1 \in \text{rp}_1$ with x_1 as its ranking function, $\text{varGD}(\text{rp}_1, c)$ contains the

equations $x_1 = e_1$, where $e_1 \in A_{sc}(c)$ is a symbolic expression describing the value of x_1 before the second execution of rp_1 ;

for each $tp_2 \in rp_2$ with x_2 as its ranking function, $\text{varGD}(rp_2, c)$ contains the equations $x_2 = e_2$, where $e_2 \in A_{sc}(c)$ is a symbolic expression describing the value of x_2 before the second execution of rp_2 .

By the operational semantics, we know the second execution before rp is also the second execution of both rp_1 and rp_2 .

Then we have this case proved by taking the union of them, i.e., $\text{varGD}(rp_1, c) + \text{varGD}(rp_2, c)$.

- **sub-case:** $rp = \text{repeat}(rp')$

By Definition 69, we have

$$\text{varGD}(\text{repeat}(rp'), c) = BD(rp', c) \times \text{varGD}(rp', c).$$

By induction hypothesis on rp' , we know

for each $tp' \in rp'$ with x' as its ranking function, $\text{varGD}(rp', c)$ contains the equations $x' = e'$, where $e' \in A_{sc}(c)$ is a symbolic expression describing the value of x' before the second execution of rp' .

We also know $BD(rp', c)$ computes the maximum iteration times of rp' from $\text{absinit}(c'_r)$ until the l_f where $(l_f, _) \in \text{absfinal}(c'_r)$ where $rp' = \text{REFINE}(c'_r)$.

Then we know $BD(rp', c) \times \text{varGD}(rp', c)$ computes the value of x' before the second execution of $\text{repeat}(rp')$ for each $tp' \in rp'$ with x' as its ranking function.

Then we have this case proved.

Based on the soundness of the three states above, we know that the initial value of the ranking function subtracts its final value divided by the modified value

soundly computes the maximum iteration times of this program from $\text{absinit}(\text{rp})$ until the l_f where $(l_f, _) \in \text{absfinal}(\text{rp})$.

So we have for each $\text{tp} \in \text{rp}$ where $x = \text{rank}(\text{tp})$, and $a, b, a' \in A_{\text{sc}}(c)$ such that $x = a \in \text{rInit}(\text{rp}', c)$ and $x = b \in \text{rFinal}(\text{rp}', c)$, the following holds.

$$\begin{aligned} & \llbracket \frac{a-b}{\text{varGD}(\text{rp}', c)} \rrbracket(\tau_0) \\ & \geq \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau_b, \tau_a \in \mathcal{T}^+ . \right. \\ & \quad \tau = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = [(_, l_s, _)]_{++\tau'++}[(_, l_f, _)] \\ & \quad \wedge (l_f, _) \in \text{absfinal}(\text{rp}_2) \wedge l_s = \text{absinit}(\text{rp}_2) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \Big\}. \end{aligned}$$

Then in the first subcase, $\text{rp}' = \text{repeat}(\text{rp}_l) = \text{tp}$, we have this case proved by

$$\begin{aligned} & \llbracket \max_{x \in \{\text{rank}(\hat{e}, c) \mid \text{tp} \in \text{rp} \wedge \hat{e} \in \text{tp}\}} \left\{ \frac{a-b}{\text{varGD}(\text{tp}, c)} \mid x = a \in \text{rInit}(\text{tp}, c) \wedge x = b \in \text{rFinal}(\text{tp}, c) \right\} \rrbracket(\tau_0) \\ & \geq \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau_b, \tau_a \in \mathcal{T}^+ . \right. \\ & \quad \tau = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = ++\tau'++[(_, l_f, _)] \\ & \quad \wedge (l_f, _) \in \text{absfinal}(\text{tp}) \wedge l_s = \text{absinit}(\text{tp}) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \Big\}. \end{aligned}$$

In the subcase of $\text{rp}_l = \text{rp}_1; \text{rp}_2$, we know for arbitrary $a \in \text{SC}(c)$ such that $x = a \in \text{rInit}(\text{rp}_1, c)$, $\min\{a, a - \text{varGD}(\text{rp}_1, c)\}$ bounds the minimum initial value of the counter during the first loop iteration.

We also have $\text{varGD}(\text{rp}_l) = \text{varGD}(\text{rp}_1; \text{rp}_2)$. Then this subcase is proved by

$$\begin{aligned} & \llbracket \max_{x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}} \left\{ \frac{\min\{a, a - \text{varGD}(\text{rp}_1, c)\} - b}{\text{varGD}(\text{rp}_1; \text{rp}_2, c)} \right. \\ & \quad \left. \mid x = a \in \text{rInit}(\text{rp}_1, c) \wedge x = b \in \text{rFinal}(\text{rp}_2; \text{rp}_2, c) \right\} \rrbracket(\tau_0) \\ & \geq \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau_b, \tau_a \in \mathcal{T}^+ . \right. \\ & \quad \tau = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = ++\tau'++[(_, l_f, _)] \\ & \quad \wedge (l_f, _) \in \text{absfinal}(\text{rp}_l) \wedge l_s = \text{absinit}(\text{rp}_l) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \Big\}. \end{aligned}$$

In the other subcases, since they will not show up by our refine algorithm, we return for these cases all 0.

case: $rp = l : rp_l$

In this case, we have

$$BD(l : rp_l, c) = BD(rp_l, c)$$

Let $c_l \in \mathcal{C}$ be the while language program corresponding to rp_l such that $\text{Rewrite}(c_l) = rp_l$.

According to the Algorithm 7 and the program refinement algorithm REFINES, we know $c_r = c_l$.

Then we have this case proved trivially by induction hypothesis on rp_l and c_l with the same initial and execution trace.

• **The execution is non-terminating and** $\langle c_r, \tau \rangle \uparrow^\infty \tau_{r++}\tau$.

In this case we know $\tau \in \mathcal{T}^\infty$ and is sufficient to show,

$$\begin{aligned} \llbracket BD(rp, c) \rrbracket(\tau_0) &\geq \# \left\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau' \in \mathcal{T}^+, \tau'' \in \mathcal{T}^\infty . \right. \\ &\quad \tau = \tau_{a++}\tau_{l_s \rightarrow l_f}++\tau_b \wedge \tau_{l_s \rightarrow l_f} = [(_, l_s, _)]++\tau'++[(_, l_f, _)] \\ &\quad \left. \wedge (l_f, _) \in \text{absfinal}(rp) \wedge l_s = \text{absinit}(rp) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \right\}. \end{aligned}$$

Let l_f be an arbitrary final continuation point of program rp such that $(l_f, _) \in \text{absfinal}(rp)$, then we have two cases as follows.

case: The execution doesn't reach l_f

In this case, we have

$$\begin{aligned} \sum \Big\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau' \in \mathcal{T}^+, \tau'' \in \mathcal{T}^\infty . \\ \tau = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = [(_, l_s, _)]_{++\tau'++}[(_, l_f, _)] \\ \wedge (l_f, _) \in \text{absfinal}(\text{rp}) \wedge l_s = \text{absinit}(\text{rp}) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \Big\} \\ = \sum \emptyset = 0. \end{aligned}$$

Since $\llbracket BD(\text{rp}, c) \rrbracket(\tau_0) \geq 0$, we have this case proved.

case: The execution reaches l_f

In this case, we know $l_f \in L_{\text{loop}}(c)$, and it is sufficient to show

$$\begin{aligned} \llbracket BD(\text{rp}, c) \rrbracket(\tau_0) \geq \\ \sum \Big\{ \text{lcnt}(\tau, L_{\text{trace}}(\tau_{l_s \rightarrow l_f})) \mid \tau_{l_s \rightarrow l_f} \in \Big\{ \tau_{l_s \rightarrow l_f} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau' \in \mathcal{T}^+, \tau'' \in \mathcal{T}^\infty . \\ \tau = \tau_{a++\tau_{l_s \rightarrow l_f}++\tau_b} \wedge \tau_{l_s \rightarrow l_f} = [(_, l_s, _)]_{++\tau'++}[(_, l_f, _)] \\ \wedge (l_f, _) \in \text{absfinal}(\text{rp}) \wedge l_s = \text{absinit}(\text{rp}) \wedge \text{cnt}(\tau', l_s) = \text{cnt}(\tau', l_f) = 0 \Big\} \Big\}. \end{aligned}$$

By the soundness of ranking function computation in Lemma B.3.1, we know $\text{VB}(x, c)$ will be ∞ for ranking functions of the loop l_f . Then we have

$$\llbracket BD(\text{rp}, c) \rrbracket(\tau_0) = \llbracket \infty \rrbracket(\tau_0) = \infty.$$

We also know $\text{lcnt}(\tau, L_{\text{trace}}(\tau_{l_s \rightarrow l_f})) = \perp$ for every $\tau_{l_s \rightarrow l_f}$.

Since we know $\infty > n$ and $n > \perp$ for every $n \in \mathbb{N}$, then we know $\infty > \perp$ and this case is proved. $\square$

B.4.2 Path Local Reachability-bound Soundness (Proof of Lemma 12.6.2)

Lemma (Soundness of the Path Local Reachability-Bound). *For any program c with its refined program rp and a simple transition path tp in rp , if $l : \text{rp}_l = \text{enclosed}(\text{rp}, \text{tp})$ is the closest loop where tp is nested in this program, then the execution times of tp when executing the rp_l under initial trace $\tau_l \in \mathcal{T}_0^+(c_l)$ is bounded by $\llbracket \text{pathLocRB}(\text{rp}_l, \text{tp}) \rrbracket(\tau_0)$ with any initial trace $\tau_0 \in \mathcal{T}_0^+(c)$.*

$$\forall c, c_l \in \mathcal{C}, \text{tp} \in \text{absG}(c), \tau_l \in \mathcal{T}_0^+(c_l), \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^{+\infty}, l, l' \in \mathcal{L}, \text{rp}.$$

$$\begin{aligned} & \text{rp} = \text{REFINE}(\text{Rewrite}(c)) \wedge l : \text{rp}_l = \text{enclosed}(\text{rp}, \text{tp}) \wedge \text{rp}_l = \text{Rewrite}(c_l) \\ & \wedge \left(\langle c_l, \tau_l \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{l++\tau} \rangle \vee \langle c_l, \tau_l \rangle \uparrow^\infty \tau_{l++\tau} \right) \\ & \implies \llbracket \text{pathLocRB}(\text{rp}_l, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, \text{L}_{\text{path}}(\text{tp})). \end{aligned}$$

Proof. Path Local Reachability-Bound Soundness.

Taking an arbitrary program c , let rp be its refined program and tp be an arbitrary transition path in rp .

Let $l : \text{rp}_l = \text{enclosed}(\text{rp}, \text{tp})$ be the closest loop program where tp is nested and c_l be the while language program of rp_l such that $\text{Rewrite}(c_l) = \text{rp}_l$.

Let $\tau_l \in \mathcal{T}_0^+(c_l)$ be an arbitrary initial trace of the closest loop c_l , and $\tau \in \mathcal{T}^{+\infty}$ be an execution trace such that

$$\langle c_l, \tau_l \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{l++\tau} \rangle \vee \langle c_l, \tau_l \rangle \uparrow^\infty \tau_{l++\tau}$$

- **The execution terminates and** $\langle c_l, \tau_l \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{l++\tau} \rangle$.

In this case we know $\tau \in \mathcal{T}^+$ and is sufficient to show,

$$\llbracket \text{pathLocRB}(\text{rp}_l, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, \text{L}_{\text{path}}(\text{tp})).$$

By induction on the loop program rp_l , we have the following cases:

case: $rp_l = tp$

In this case, we have $\text{pathLocRB}(tp, tp) = 1$ by Definition 71.

Since tp doesn't contain any loop, we know it will be executed at most once, i.e., $\text{lcnt}(\tau, L_{\text{path}}(tp)) \leq 1$.

Then we have the following

$$\llbracket \text{pathLocRB}(rp_l, tp) \rrbracket(\tau_0) = 1 = \text{lcnt}(\tau, L_{\text{path}}(tp))$$

This case is proved.

case: $rp_l = tp' \wedge tp' \neq tp$

In this case, we have $\text{pathLocRB}(tp', tp) = 0$ by Definition 71.

Since $tp' \neq tp$, we know tp isn't executed when executing c_l , i.e., $\text{lcnt}(\tau, L_{\text{path}}(tp)) = 0$.

Then we have

$$\llbracket \text{pathLocRB}(rp_l, tp) \rrbracket(\tau_0) = 0 = \text{lcnt}(\tau, L_{\text{path}}(tp)),$$

and this case is proved.

case: $rp_l = rp_1; rp_2$

In this case, we have $\text{pathLocRB}(rp_1; rp_2, tp) = \text{pathLocRB}(rp_1, tp) + \text{pathLocRB}(rp_2, tp)$ by Definition 7, and it is sufficient to show

$$\llbracket \text{pathLocRB}(rp_1, tp) + \text{pathLocRB}(rp_2, tp) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(tp))$$

Let $c_1, c_2 \in \mathcal{C}$ be the while language program corresponding to rp_1 and rp_2 such that $\text{Rewrite}(c_1) = rp_1$ and $\text{Rewrite}(c_2) = rp_2$. According to the Algorithm 7, we

have $c_l = c_1; c_2$.

According to the operational semantics, let $\tau_1, \tau_2 \in \mathcal{T}^+$ be two execution traces such that

$$\langle c_1; c_2, \tau_l \rangle \rightarrow^* \langle c_2, \tau_{l++\tau_1} \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{l++\tau_1++\tau_2} \rangle$$

By the trace non-decreasing property in Lemma 4.2.1, we have $\tau = \tau_{l++\tau_2}$ and

$$\text{lcnt}(\tau, L_{\text{path}}(\text{tp})) = \text{lcnt}(\tau_1, L_{\text{path}}(\text{tp})) + \text{lcnt}(\tau_2, L_{\text{path}}(\text{tp}))$$

By induction hypothesis on τ_l, c_1, τ_1 and c_2 , we have

$$\llbracket \text{pathLocRB}(\text{rp}_1, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau_1, L_{\text{path}}(\text{tp}))$$

and

$$\llbracket \text{pathLocRB}(\text{rp}_2, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau_2, L_{\text{path}}(\text{tp}))$$

Then we have this case proved as

$$\begin{aligned} & \llbracket \text{pathLocRB}(\text{rp}_1, \text{tp}) + \text{pathLocRB}(\text{rp}_2, \text{tp}) \rrbracket(\tau_0) \\ & \geq \llbracket \text{pathLocRB}(\text{rp}_1, \text{tp}) \rrbracket(\tau_0) + \llbracket \text{pathLocRB}(\text{rp}_2, \text{tp}) \rrbracket(\tau_0) \\ & \geq \text{lcnt}(\tau_1, L_{\text{path}}(\text{tp})) + \text{lcnt}(\tau_2, L_{\text{path}}(\text{tp})) \\ & = \text{lcnt}(\tau, L_{\text{path}}(\text{tp})) \end{aligned}$$

case: $\text{rp}_l = \text{choose}\{\text{rp}_1, \dots, \text{rp}_m\}$

By Definition 7, we know $\text{pathLocRB}(\text{choose}\{\text{rp}_1, \dots, \text{rp}_m\}, \text{tp})$

$= \max\{\text{pathLocRB}(\text{rp}_1, \text{tp}), \dots, \text{pathLocRB}(\text{rp}_m, \text{tp})\}$. Then, it is sufficient to show

$$\llbracket \max\{\text{pathLocRB}(\text{rp}_1, \text{tp}), \dots, \text{pathLocRB}(\text{rp}_m, \text{tp})\} \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp}))$$

Let $c_1, \dots, c_m \in \mathcal{C}$ be the while language program corresponding to $\text{rp}_1, \dots, \text{rp}_m$ such that $\text{Rewrite}(c_1) = \text{rp}_1, \dots$ and $\text{Rewrite}(c_m) = \text{rp}_m$.

Let $\tau_1, \dots, \tau_m \in \mathcal{T}^+$ be the execution traces such that

$$\langle c_i, \tau_i \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++}\tau_i \rangle$$

Then by induction hypothesis on each program rp_i , we know for each rp_i ,

$$\llbracket \text{pathLocRB}(\text{rp}_i, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau_i, \text{L}_{\text{path}}(\text{tp}))$$

Since execution of each rp_i corresponds to one possible execution of c_r as shown in proof of Lemma 12.6.1 in Appendix B.4.1, we know

$$\max\{\text{lcnt}(\tau_0, \text{L}_{\text{path}}(\text{tp})), \dots, \text{lcnt}(\tau_m, \text{L}_{\text{path}}(\text{tp}))\} \geq \text{lcnt}(\tau, \text{L}_{\text{path}}(\text{tp})).$$

Then we have

$$\begin{aligned} & \llbracket \text{pathLocRB}(\text{choose}\{\text{rp}_1, \dots, \text{rp}_m\}, \text{tp}) \rrbracket(\tau_0) \\ &= \llbracket \max\{\text{pathLocRB}(\text{rp}_1, \text{tp}), \dots, \text{pathLocRB}(\text{rp}_m, \text{tp})\} \rrbracket(\tau_0) \\ &= \max\{\llbracket \text{pathLocRB}(\text{rp}_1, \text{tp}) \rrbracket(\tau_0), \dots, \llbracket \text{pathLocRB}(\text{rp}_m, \text{tp}) \rrbracket(\tau_0)\} \\ &\geq \max\{\text{lcnt}(\tau_0, \text{L}_{\text{path}}(\text{tp})), \dots, \text{lcnt}(\tau_m, \text{L}_{\text{path}}(\text{tp}))\} \\ &\geq \text{lcnt}(\tau, \text{L}_{\text{path}}(\text{tp})), \end{aligned}$$

and this case is proved.

case: $\text{rp}_l = \text{repeat}(\text{rp}')$

We have $\text{pathLocRB}(\text{repeat}(\text{rp}'), \text{tp}) = BD(\text{repeat}(\text{rp}'), c) \times \text{pathLocRB}(\text{rp}', \text{tp})$ by

Definition 7, and it is sufficient to show

$$\llbracket BD(\text{repeat}(\text{rp}'), c) \times \text{pathLocRB}(\text{rp}', \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, \text{L}_{\text{path}}(\text{tp}))$$

Let $c' \in \mathcal{C}$ be the while language program corresponding to rp' such that $\text{Rewrite}(c') = \text{rp}'$. According to the Algorithm 7, we have $c_l = \text{while } [b]^l \text{ do } c'$.

According to the operational semantics, let $\tau' \in \mathcal{T}^+$ be the execution trace of executing c' one time with a program $c'' \in \mathcal{C}$ such that

$$\begin{aligned} & \langle \text{while } [b]^l \text{ do } c', \tau_l \rangle \rightarrow \langle c', \tau_{l++}(b, \text{true}, l) \rangle \\ & \rightarrow^* \langle [\text{skip}]^l; \text{while } [b]^l \text{ do } c', \tau_{l++}(b, \text{true}, l)_{++}\tau' \rangle \\ & \rightarrow^* \langle [\text{skip}]^l, \tau_{l++}\tau \rangle \end{aligned}$$

By the trace non-decreasing property in Lemma 4.2.1, we know τ contains multiple execution traces of c' .

By Lemma 12.6.1, we know $BD(\text{repeat}(\text{rp}'), c)$ bounds the iterations of $\text{repeat}(\text{rp}')$.

Then we have

$$\llbracket BD(\text{repeat}(\text{rp}'), c) \rrbracket(\tau_0) \times \text{lcnt}(\tau', \text{L}_{\text{path}}(\text{tp})) \geq \text{lcnt}(\tau, \text{L}_{\text{path}}(\text{tp}))$$

By induction hypothesis on rp' and c' , we also have

$$\llbracket \text{pathLocRB}(\text{rp}', \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau', \text{L}_{\text{path}}(\text{tp}))$$

Then we have this case proved by

$$\begin{aligned}
& \llbracket BD(\text{repeat}(\text{rp}'), c) \times \text{pathLocRB}(\text{rp}', \text{tp}) \rrbracket(\tau_0) \\
&= \llbracket BD(\text{repeat}(\text{rp}'), c) \rrbracket(\tau_0) \times \llbracket \text{pathLocRB}(\text{rp}', \text{tp}) \rrbracket(\tau_0) \\
&\geq \llbracket BD(\text{repeat}(\text{rp}'), c) \rrbracket(\tau_0) \times \text{lcnt}(\tau', L_{\text{path}}(\text{tp})) \\
&\geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp}))
\end{aligned}$$

case: $\text{rp} = l : \text{rp}_l$

In this case, we have

$$\text{pathLocRB}(l : \text{rp}_l, c) = \text{pathLocRB}(\text{rp}_l, c)$$

Let $c_l \in \mathcal{C}$ be the while language program corresponding to rp_l such that $\text{Rewrite}(c_l) = \text{rp}_l$.

According to the Algorithm 7 and the program refinement algorithm REFINER, we know $c_l = c_l$.

Then we have this case proved trivially by induction hypothesis on rp_l and c_l with the same initial and execution trace τ_l .

case: $\text{rp}_l = l' : \text{rp}'$ **and** $l' : \text{rp}' \neq \text{enclosed}(\text{tp})$

In this case, we have $\text{pathLocRB}(l' : \text{rp}', \text{tp}) = 0$ by Definition 71.

Since $l' : \text{rp}' \neq \text{enclosed}(\text{tp})$, we know tp isn't executed when executing c_l , i.e., $\text{lcnt}(\tau, L_{\text{path}}(\text{tp})) = 0$.

Then we have

$$\llbracket \text{pathLocRB}(l' : \text{rp}', \text{tp}) \rrbracket(\tau_0) = 0 = \text{lcnt}(\tau, L_{\text{path}}(\text{tp})),$$

and this case is proved.

- **The execution is non-terminating and $\langle c_l, \tau \rangle \uparrow^\infty \tau_{l++\tau}$.**

In this case we know $\tau \in \mathcal{T}^\infty$ and is sufficient to show,

$$\llbracket \text{pathLocRB}(\text{rp}_l, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp})).$$

By induction on the loop program rp_l , we have the following cases:

case: $\text{rp}_l = \text{tp}$

In this case, we have $\text{pathLocRB}(\text{tp}, \text{tp}) = 1$ by Definition 71.

Since tp doesn't contain any loop, we know it will be executed at most once and terminates.

Then this case is vacuously true.

case: $\text{rp}_l = \text{tp}' \wedge \text{tp}' \neq \text{tp}$

This case is vacuously true as well for the same reason as above.

case: $\text{rp}_l = \text{rp}_1; \text{rp}_2$

In this case, we have $\text{pathLocRB}(\text{rp}_1; \text{rp}_2, \text{tp}) = \text{pathLocRB}(\text{rp}_1, \text{tp}) + \text{pathLocRB}(\text{rp}_2, \text{tp})$ by Definition 7, and it is sufficient to show

$$\llbracket \text{pathLocRB}(\text{rp}_1, \text{tp}) + \text{pathLocRB}(\text{rp}_2, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp}))$$

Let $c_1, c_2 \in \mathcal{C}$ be the while language program corresponding to rp_1 and rp_2 such that $\text{Rewrite}(c_1) = \text{rp}_1$ and $\text{Rewrite}(c_2) = \text{rp}_2$. According to the Algorithm 7, we have $c_l = c_1; c_2$.

According to the operational semantics, we have two possible executions as follows,

sub-case: The execution of c_1 terminates

Then let $\tau_1 \in \mathcal{T}^+$, $\tau_2 \in \mathcal{T}^\infty$ be two execution traces, we have

$$\langle c_1; c_2, \tau_l \rangle \rightarrow^* \langle c_2, \tau_{l++}\tau_1 \rangle \uparrow^\infty \tau_{l++}\tau_1++\tau_2$$

By the trace non-decreasing property in Lemma 4.2.1, we have $\tau = \tau_{l++}\tau_2$ and,

$$\text{lcnt}(\tau, L_{\text{path}}(\text{tp})) = \text{lcnt}(\tau_1, L_{\text{path}}(\text{tp})) + \text{lcnt}(\tau_2, L_{\text{path}}(\text{tp})) = \text{lcnt}(\tau_1, L_{\text{path}}(\text{tp})) + \perp$$

By induction hypothesis on τ_l, c_1, τ_1 and c_2 , we know both

$$\llbracket \text{pathLocRB}(\text{rp}_1, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau_1, L_{\text{path}}(\text{tp})),$$

$$\llbracket \text{pathLocRB}(\text{rp}_2, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau_2, L_{\text{path}}(\text{tp})).$$

Then we have this case proved as

$$\begin{aligned} & \llbracket \text{pathLocRB}(\text{rp}_1, \text{tp}) + \text{pathLocRB}(\text{rp}_2, \text{tp}) \rrbracket(\tau_0) \\ & \geq \llbracket \text{pathLocRB}(\text{rp}_1, \text{tp}) \rrbracket(\tau_0) + \llbracket \text{pathLocRB}(\text{rp}_2, \text{tp}) \rrbracket(\tau_0) \\ & \geq \text{lcnt}(\tau_1, L_{\text{path}}(\text{tp})) + \perp \\ & = \text{lcnt}(\tau, L_{\text{path}}(\text{tp})) \end{aligned}$$

sub-case: The execution of c_1 is non-terminating

In this case, let $\tau_1, \tau_2 \in \mathcal{T}^\infty$ be two execution traces, we have

$$\langle c_1; c_2, \tau_l \rangle \uparrow^\infty \tau_{l++}\tau_1$$

By the trace concatenation in Definition 42 and the non-decreasing property in

Lemma 4.2.1, we know $\tau = \tau_1$ and

$$\text{lcnt}(\tau, L_{\text{path}}(\mathbf{tp})) = \text{lcnt}(\tau_1, L_{\text{path}}(\mathbf{tp})) = \perp$$

By induction hypothesis on τ_l, c_1, τ_1 , we have in the first case

$$\llbracket \text{pathLocRB}(\mathbf{rp}_1, \mathbf{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\mathbf{tp}))$$

Then we have this case proved as

$$\begin{aligned} & \llbracket \text{pathLocRB}(\mathbf{rp}_1, \mathbf{tp}) + \text{pathLocRB}(\mathbf{rp}_2, \mathbf{tp}) \rrbracket(\tau_0) \\ & \geq \llbracket \text{pathLocRB}(\mathbf{rp}_1, \mathbf{tp}) \rrbracket(\tau_0) + \llbracket \text{pathLocRB}(\mathbf{rp}_2, \mathbf{tp}) \rrbracket(\tau_0) \\ & \geq \llbracket \text{pathLocRB}(\mathbf{rp}_1, \mathbf{tp}) \rrbracket(\tau_0) \\ & \geq \text{lcnt}(\tau_1, L_{\text{path}}(\mathbf{tp})) \\ & = \text{lcnt}(\tau, L_{\text{path}}(\mathbf{tp})) = \perp \end{aligned}$$

case: $\mathbf{rp}_l = \text{choose}\{\mathbf{rp}_1, \dots, \mathbf{rp}_m\}$

By Definition 7, we know $\text{pathLocRB}(\text{choose}\{\mathbf{rp}_1, \dots, \mathbf{rp}_m\}, \mathbf{tp})$

$$\max\{\text{pathLocRB}(\mathbf{rp}_1, \mathbf{tp}), \dots, \text{pathLocRB}(\mathbf{rp}_m, \mathbf{tp})\}.$$

Then it is sufficient to show

$$\llbracket \max\{\text{pathLocRB}(\mathbf{rp}_1, \mathbf{tp}), \dots, \text{pathLocRB}(\mathbf{rp}_m, \mathbf{tp})\} \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\mathbf{tp})).$$

Let $c_1, \dots, c_m \in \mathcal{C}$ be the while language program corresponding to $\mathbf{rp}_1, \dots, \mathbf{rp}_m$ such that $\text{Rewrite}(c_1) = \mathbf{rp}_1, \dots$ and $\text{Rewrite}(c_m) = \mathbf{rp}_m$.

Since there exists at least a program c_j whose execution under the initial trace c_r

doesn't terminate, by induction hypothesis on c_j , we know

$$\llbracket \text{pathRB}(\text{rp}_j, \text{tp}) \rrbracket(\tau_0) = \infty.$$

Then by induction hypothesis on all the other programs and taking the maximum value overall $\text{pathLocRB}(\text{rp}_i, \text{tp})$, we get

$$\llbracket \max\{\text{pathLocRB}(\text{rp}_1, \text{tp}), \dots, \text{pathLocRB}(\text{rp}_m, \text{tp})\} \rrbracket(\tau_0) = \infty,$$

and this case is proved.

case: $\text{rp}_l = \text{repeat}(\text{rp}')$

We have $\text{pathLocRB}(\text{repeat}(\text{rp}'), \text{tp}) = BD(\text{repeat}(\text{rp}'), c) \times \text{pathLocRB}(\text{rp}', \text{tp})$ by Definition 7, and it is sufficient to show

$$\llbracket BD(\text{repeat}(\text{rp}'), c) \times \text{pathLocRB}(\text{rp}', \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp}))$$

Since execution of $\text{repeat}(\text{rp}')$ non-terminates, and it is a loop program, by Lemma 12.6.1 second sub0case in the non-terminating case, we know

$$BD(\text{repeat}(\text{rp}'), c) = \infty.$$

Then we have this case proved by

$$\begin{aligned} & \llbracket BD(\text{repeat}(\text{rp}'), c) \times \text{pathLocRB}(\text{rp}', \text{tp}) \rrbracket(\tau_0) \\ &= \llbracket \infty \rrbracket(\tau_0) \\ &\geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp})) \end{aligned}$$

case: $\text{rp} = l : \text{rp}_l$

In this case, we have

$$\text{pathLocRB}(l : \text{rp}_l, c) = \text{pathLocRB}(\text{rp}_l, c)$$

Let $c_l \in \mathcal{C}$ be the while language program corresponding to rp_l such that $\text{Rewrite}(c_l) = \text{rp}_l$.

According to the Algorithm 7 and the program refinement algorithm REFINE, we know $c_l = c_l$.

Then we have this case proved trivially by induction hypothesis on rp_l and c_l with the same initial and execution trace τ_l .

case: $\text{rp}_l = l' : \text{rp}'$ **and** $l' : \text{rp}' \neq \text{enclosed}(\text{tp})$

$\text{pathLocRB}(l' : \text{rp}', \text{tp}) = \text{pathLocRB}(\text{rp}', \text{tp})$ Since $l' : \text{rp}' \neq \text{enclosed}(\text{tp})$, we know tp isn't executed when executing c_l , i.e., $\text{lcnt}(\tau, \text{L}_{\text{path}}(\text{tp})) = 0$.

Then we have

$$\llbracket \text{pathLocRB}(l' : \text{rp}', \text{tp}) \rrbracket(\tau_0) = 0 = \text{lcnt}(\tau, \text{L}_{\text{path}}(\text{tp})),$$

and this case is proved. □

Informal Discussion: For a simple transition path tp only enclosed by one transition path $\text{repeat}(\text{tp})$, we know $\text{pathLocRB}(\text{rp}, \text{tp}) = \text{pathLocRB}(\text{repeat}(\text{tp}), \text{rp})$. Since $\text{pathLocRB}(\text{repeat}(\text{tp}), \text{rp})$ is a sound local bound on the iteration times of $\text{repeat}(\text{tp})$ by assuming all the outside loops executes only once. In this sense, $\text{pathLocRB}(\text{repeat}(\text{tp}), \text{rp})$ is also a sound bound on the iteration times globally. For a simple transition path tp nested in multiple transition paths $\text{rp}_1, \dots, \text{rp}_m$, we know $\text{pathLocRB}(\text{rp}, \text{tp}) = \prod_{i=1, \dots, m} \text{pathLocRB}(\text{rp}_i, \text{rp})$. By the same guarantee from $\text{pathLocRB}(\text{rp}_i, \text{rp})$, it is sound to multiply each of them.

B.4.3 Loop Reachability-bound Soundness (Proof of Lemma 12.7.1)

Lemma (Soundness of the Loop Reachability-Bound). *For any loop $l : \text{rp}$ and a simple transition path tp in a refined program, if $l_t : \text{rp}_t$ is the closest loop enclosing the tp , then the entering times of $l_t : \text{rp}_t$ when executing the $l : \text{rp}$ under initial trace $\tau_0 \in \mathcal{T}_0^+(c)$, is bounded by $\llbracket \text{lpRB}(l : \text{rp}, \text{tp}, c) \rrbracket(\tau_0)$.*

$$\begin{aligned}
& \forall c, c_l \in \mathcal{C}, \text{tp} \in \text{absG}(c), \tau_0 \in \mathcal{T}_0^+(c_l), \tau \in \mathcal{T}^{+\infty}, l, l' \in \mathcal{L}, \text{rp} . \text{rp} = \text{REFINE}(\text{Rewrite}(c)) \\
& \wedge l_t : \text{rp}_t = \text{enclosed}(\text{rp}, \text{tp}) \wedge \text{rp} = \text{Rewrite}(c_l) \\
& \wedge \left(\langle c_l, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \vee \langle c_l, \tau_0 \rangle \uparrow^\infty \tau_{0++\tau} \right) \\
& \implies \llbracket \text{lpRB}(l : \text{rp}, \text{tp}, c) \rrbracket(\tau_0) \\
& \geq \sum \left\{ \text{lcnt}(\tau, \text{L}_{\text{trace}}(\tau_{l \rightarrow l_t})) \mid \tau_{l \rightarrow l_t} \in \left\{ \tau_{l \rightarrow l_t} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau' \in \mathcal{T}^+, \tau'' \in \mathcal{T}^{+\infty} . \right. \right. \\
& \quad \left. \left. \tau = \tau_{l \rightarrow l_t++\tau''} \wedge \tau_{l \rightarrow l_t} = [(_, l, \text{true})]_{++\tau'} :: (_, l_t, \text{true}) \right. \right. \\
& \quad \left. \left. \wedge \text{cnt}(\tau', l) = \text{cnt}(\tau', l_t) = 0 \right\} \right\}.
\end{aligned}$$

Proof. Loop Reachability-Bound Soundness.

Taking an arbitrary program c , let rp be its refined program and tp be an arbitrary transition path in rp .

Let $l : \text{rp}_l$ be a loop program in rp and c_l be the while language program of rp_l such that $\text{Rewrite}(c_l) = \text{rp}_l$.

Let $\tau_l \in \mathcal{T}_0^+(c_l)$ be an arbitrary initial trace of this loop program c_l , and $\tau \in \mathcal{T}^{+\infty}$ be an execution trace such that

$$\langle c_l, \tau_l \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{l++\tau} \rangle \vee \langle c_l, \tau_l \rangle \uparrow^\infty \tau_{l++\tau}$$

Let $l_t : \text{rp}_t = \text{enclosed}(\text{rp}, \text{tp})$ be the closest loop program where tp is located, and $c_t \in \mathcal{C}$ such that $\text{Rewrite}(c_t) = \text{rp}_t$.

To show that $\text{lpRB}(l : \text{rp}, \text{tp})$ bounds the iteration numbers of the loop l w.r.t. the

inner loop $l_t : \text{rp}_t$ such that during the iterations of loop l , the nested loop $l_t : \text{rp}_t$ is executed, it is sufficient to show

$$\begin{aligned} & \llbracket \text{lpRB}(l : \text{rp}, \text{tp}) \rrbracket(\tau_0) \\ & \geq \sum \left\{ \text{lcnt}(\tau, \text{L}_{\text{trace}}(\tau_{l \rightarrow l_t})) \mid \tau_{l \rightarrow l_t} \in \left\{ \tau_{l \rightarrow l_t} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau' \in \mathcal{T}^+, \tau'' \in \mathcal{T}^{+\infty} . \right. \right. \\ & \quad \left. \left. \tau = \tau_{l \rightarrow l_t} \tau'' \wedge \tau_{l \rightarrow l_t} = [(_, l, \text{true})]_{++\tau'} :: (_, l_t, \text{true}) \right. \right. \\ & \quad \left. \left. \wedge \text{cnt}(\tau', l) = \text{cnt}(\tau', l_t) = 0 \right\} \right\}. \end{aligned}$$

- **The execution terminates and** $\langle c_l, \tau_l \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{l++\tau} \rangle$.

In this case we know $\tau \in \mathcal{T}^+$.

By Definition 74, we have

$$\text{lpRB}(l : \text{rp}, \text{tp}, c) \triangleq \max_{x=a \in \text{rFinal}(\text{tp}, c)} \frac{\text{lpInit}(\text{rp}, \text{tp}, c) - a}{\text{lpNext}(\text{rp}, \text{tp}, c)}$$

The soundness is guaranteed by the three states, $\text{lpInit}(\text{rp}, \text{tp}, c)$, $\text{rFinal}(\text{rp}, \text{tp}, c)$, $\text{lpNext}(\text{rp}, \text{tp},)$ computed in Definition 73 as follows.

- *Loop Initial State Soundness.* By Definition 73, we have $\text{lpInit}(\text{rp}, \text{tp}, c) =$

$$\max_{l_1}$$

$$\left\{ \text{VB}(y, c) + v \mid \begin{array}{l} (l_1, x' \leq y + v, l_2) \in \text{re}(x, c) \\ \wedge \text{absinit}(\text{rp}) \leq l_1 \leq \text{absinit}(\text{tp}) \wedge x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\} \end{array} \right\}$$

Let $\tau' \in \mathcal{T}^{+\infty}$ be the execution trace before first execution of $l_t : \text{rp}_t$, then we have

$$\langle c_l, \tau_0 \rangle \rightarrow^* \langle c_t; c', \tau_{0++\tau'} \rangle \rightarrow^* \langle \text{skip}, \tau_{0++\tau} \rangle$$

Let $x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}$ be arbitrary ranking function of an edge on tp , by the soundness of the rank estimation in Lemma B.3.1 and Theorem 12.4.1, we

know for each variable y such that $(l_1, x' \leq y + v, l_2) \in \mathbf{re}(x, c)$, $\mathbf{VB}(y, c)$ is the sound upper bound on its maximum value during the execution. Then we have

$$\llbracket \mathbf{VB}(y, c) \rrbracket(\tau_0) \geq \max\{v \mid \rho(\tau)y = v\} \geq \rho(\tau')y$$

In this sense, we have

$\max_{l_1} \left\{ \mathbf{VB}(y, c) + v \mid \begin{array}{l} (l_1, x' \leq y + v, l_2) \in \mathbf{re}(x, c) \\ \wedge \mathbf{absinit}(\mathbf{rp}) \leq l_1 \leq \mathbf{absinit}(\mathbf{tp}) \end{array} \right\}$ is a sound initial value estimation of the value of x before first execution of $l_t : \mathbf{rp}_t$, i.e.,

$$\llbracket \max_{l_1} \left\{ \mathbf{VB}(y, c) + v \mid \begin{array}{l} (l_1, x' \leq y + v, l_2) \in \mathbf{re}(x, c) \\ \wedge \mathbf{absinit}(\mathbf{rp}) \leq l_1 \leq \mathbf{absinit}(\mathbf{tp}) \end{array} \right\} \rrbracket(\tau_0)$$

$$\geq \max\{v \mid \rho(\tau)x = v\} \geq \rho(\tau')x$$

- $\mathbf{lpNext}(\mathbf{rp}, \mathbf{tp}, c)$ computes the variables states of the $\mathbf{tp}$ after visited the program point l the second time and before visiting any other program point.

By Definition 73, we have $\mathbf{lpNext}(l : \mathbf{rp}, \mathbf{tp}, c)$ as

$$\max_{x \in \{\mathbf{rank}(\hat{e}, c) \mid \hat{e} \in \mathbf{tp}\}} \left\{ \begin{array}{l} \sum_{\hat{e} \in \mathbf{inc}(x, c)} \{v \mid \hat{e} = (l', x' \leq x + v, _) \wedge l' \in \mathbf{rp} \wedge l' \notin \mathbf{tp}\} \\ + \max_{l_2} \{ \mathbf{VB}(y, c) + v \mid (l_1, x' \leq y + v, l_2) \in \mathbf{re}(x, c) \wedge l_1 \in \mathbf{rp} \wedge l_1 \notin \mathbf{tp} \} \\ - \sum_{\hat{e} \in \mathbf{dec}(x, c)} \{v \mid \hat{e} = (l', x' \leq x + v, _) \wedge l' \in \mathbf{rp} \wedge l' \notin \mathbf{tp}\} \\ + BD(\mathbf{enclosed}(\mathbf{tp}), c) \times \mathbf{rNext}(\mathbf{tp}, c) \end{array} \right\}$$

Let $\tau' \in \mathcal{T}^{+\infty}$ be the execution trace before the execution of $l_t : \mathbf{rp}_t$ in the

second iteration of $l : \text{rp}$, then we have

$$\langle c_l, \tau_0 \rangle \rightarrow^* \langle c_t; c', \tau_0 + \tau' \rangle \rightarrow^* \langle \text{skip}, \tau_0 + \tau \rangle$$

Let $x \in \{\text{rank}(\hat{e}, c) \mid \hat{e} \in \text{tp}\}$ be arbitrary ranking function of an edge on tp , by the soundness of the rank estimation in Lemma B.3.1 and Theorem 12.4.1, we know for each variable y such that $(l_1, x' \leq y + v, l_2) \in \text{re}(x, c)$, $\text{VB}(y, c)$ is the sound upper bound on its maximum value during the execution. Then we have

$$\llbracket \text{VB}(y, c) \rrbracket(\tau_0) \geq \max\{v \mid \pi_3(\epsilon)y = v \wedge \epsilon \in \tau' \wedge \pi_1(\epsilon) = y\} \geq \rho(\tau')y$$

Same for each abstract event such that $((l', x' \leq x + v, _) \in \text{inc}(x, c)$ and $(l', x' \leq x + v, _) \in \text{dec}(x, c)$, we know

$$\sum_{\hat{e} \in \text{inc}(x, c)} \{v \mid \hat{e} = (l', x' \leq x + v, _) \wedge l' \in \text{rp} \wedge l' \notin \text{tp}\}$$

$$+ \max_{l_2} \{\text{VB}(y, c) + v \mid (l_1, x' \leq y + v, l_2) \in \text{re}(x, c) \wedge l_1 \in \text{rp} \wedge l_1 \notin \text{tp}\} \text{ is an}$$

$$- \sum_{\hat{e} \in \text{dec}(x, c)} \{v \mid \hat{e} = (l', x' \leq x + v, _) \wedge l' \in \text{rp} \wedge l' \notin \text{tp}\}$$

upper bound on the modified value of x in one iteration of $l : \text{rp}$.

By Lemma 12.6.2 and Lemma 12.6.1, we also know $BD(\text{rp}_t)$ is sound upper bound of l_t 's iterations and $\text{rNext}(\text{tp}, c)$ is the upper bound of x 's value modification in one iteration of $l_t : \text{rp}_t$.

In this sense, we know

$$\sum_{\hat{e} \in \text{inc}(x, c)} \{v \mid \hat{e} = (l', x' \leq x + v, _) \wedge l' \in \text{rp} \wedge l' \notin \text{tp}\}$$

$$+ \max_{l_2} \{\text{VB}(y, c) + v \mid (l_1, x' \leq y + v, l_2) \in \text{re}(x, c) \wedge l_1 \in \text{rp} \wedge l_1 \notin \text{tp}\} \text{ is a}$$

$$- \sum_{\hat{e} \in \text{dec}(x, c)} \{v \mid \hat{e} = (l', x' \leq x + v, _) \wedge l' \in \text{rp} \wedge l' \notin \text{tp}\}$$

$$+ BD(\text{rp}_t) \times \text{rNext}(\text{tp}, c)$$

sound upper bound on the modified value of x from first iteration of $l : \text{rp}$ until the second iteration and before the execution of $l_t : \text{rp}_t$, i.e.,

$$\begin{aligned}
& \llbracket \text{lpInit}(l : \text{rp}, \text{tp}, c) \rrbracket(\tau_0) - \\
& \quad \sum_{\hat{e} \in \text{inc}(x, c)} \{v \mid \hat{e} = (l', x' \leq x + v, _) \wedge l' \in \text{rp} \wedge l' \notin \text{tp}\} \\
& \quad + \max_{l_2} \{ \text{VB}(y, c) + v \mid (l_1, x' \leq y + v, l_2) \in \text{re}(x, c) \wedge l_1 \in \text{rp} \wedge l_1 \notin \text{tp} \} \\
& \quad - \sum_{\hat{e} \in \text{dec}(x, c)} \{v \mid \hat{e} = (l', x' \leq x + v, _) \wedge l' \in \text{rp} \wedge l' \notin \text{tp}\} \\
& \quad + BD(l_t : \text{rp}_t) \times \text{rNext}(\text{tp}, c) \\
& \leq \rho(\tau')x
\end{aligned}$$

- $\text{rFinal}(\text{tp}, c)$ computes the value of $\text{rank}(\text{tp}, c)$ after the iteration of $l_t : \text{rp}_t$ finished.

Let $\tau' \in \mathcal{T}^{+\infty}$ be the execution trace after the execution of $l_t : \text{rp}_t$ in the first iteration of $l : \text{rp}$ and the c' be the program after the execution of c_t in the first iteration of $l : \text{rp}$, then we have

$$\langle c_l, \tau_0 \rangle \rightarrow^* \langle c', \tau_{0++\tau'} \rangle \rightarrow^* \langle \text{skip}, \tau_{0++\tau} \rangle$$

By Lemma 12.6.1, we know $\text{rFinal}(\text{tp}, c)$ computes the lower bound on the value of $\text{rank}(\text{tp}, c)$ after the iteration of $l_t : \text{rp}_t$ finished.

Then we have

$$\text{rFinal}(\text{tp}, c) \leq \rho(\tau')x \text{ and } \text{lpInit}(\text{rp}, \text{tp}, c) - \text{rFinal}(\text{rp}, \text{tp}, c) \geq \rho(\tau')x.$$

Then there are two subcases as follows.

sub-case: $l_t : \text{rp}_t \notin \text{rp}$

In this case, we know loop $l_t : \text{rp}_t$ will never be entered, and this case is proved.

sub-case: $l_t : \text{rp}_t \in \text{rp}$

In this case, based on the soundness of the three states above, we know

$\max_{x=a \in \text{rFinal}(\text{tp}, c)} \frac{\text{lpInit}(\text{rp}, \text{tp}, c) - a}{\text{lpNext}(\text{rp}, \text{tp}, c)}$ soundly computes the maximum number of $l : \text{rp}'s$ iteration times such that during these iterations, the nested loop $l_t : \text{rp}_t$ is executed.

• **The execution is non-terminating and** $\langle c_r, \tau \rangle \uparrow^\infty \tau_{r++}\tau$.

In this case we know $\tau \in \mathcal{T}^\infty$, and it is sufficient to show

$$\begin{aligned} & \llbracket \text{lpRB}(l : \text{rp}, \text{tp}, c) \rrbracket(\tau_0) \\ & \geq \sum \left\{ \text{lcnt}(\tau, \text{L}_{\text{trace}}(\tau_{l \rightarrow l_t})) \mid \tau_{l \rightarrow l_t} \in \{ \tau_{l \rightarrow l_t} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau' \in \mathcal{T}^+, \tau'' \in \mathcal{T}^\infty . \right. \\ & \quad \tau = \tau_{l \rightarrow l_t} ++ \tau'' \wedge \tau_{l \rightarrow l_t} = [(_, l, \text{true})]_{++\tau'} :: (_, l_t, \text{true}) \\ & \quad \left. \wedge \text{cnt}(\tau', l) = \text{cnt}(\tau', l_t) = 0 \right\}. \end{aligned}$$

For each $\tau_{l \rightarrow l_t} \in \left\{ \tau_{l \rightarrow l_t} \mid \tau_{l \rightarrow l_t}, \tau' \in \mathcal{T}^+, \tau'' \in \mathcal{T}^{+\infty} \wedge \tau = \tau_{l \rightarrow l_t} ++ \tau'' \right.$
 $\left. \wedge \tau_{l \rightarrow l_t} = [(_, l, \text{true})]_{++\tau'} :: (_, l_t, \text{true}) \wedge \text{cnt}(\tau', l) = \text{cnt}(\tau', l_t) = 0 \right\}$,
 by Definition 48, we know

$$\text{lcnt}(\tau, \text{L}_{\text{trace}}(\tau_{l \rightarrow l_t})) = \perp.$$

Then we know

$$\begin{aligned} & \sum \left\{ \perp \mid \tau_{l \rightarrow l_t} \in \{ \tau_{l \rightarrow l_t} \mid \tau_{l_s \rightarrow l_f} \in \mathcal{T}^+ \wedge \exists \tau' \in \mathcal{T}^+, \tau'' \in \mathcal{T}^\infty . \right. \\ & \quad \tau = \tau_{l \rightarrow l_t} ++ \tau'' \wedge \tau_{l \rightarrow l_t} = [(_, l, \text{true})]_{++\tau'} :: (_, l_t, \text{true}) \\ & \quad \left. \wedge \text{cnt}(\tau', l) = \text{cnt}(\tau', l_t) = 0 \right\} = \perp, \end{aligned}$$

and we have this case proved. □

B.4.4 Path Reachability-bound Soundness (Proof of Lemma 12.8.1)

Lemma (Soundness of Path Reachability Bound). *For any program c with its refined program rp and a simple transition path tp in this program, the execution times of tp when executing the rp under initial trace $\tau_0 \in \mathcal{T}_0^+(c)$ is bounded by $\llbracket \text{pathRB}(\text{rp}, \text{tp}) \rrbracket(\tau_0)$.*

$$\begin{aligned} & \forall c, c_r \in \mathcal{C}, \text{tp} \in \text{absG}(c), \tau_0 \in \mathcal{T}_0^+(c), \tau_r \in \mathcal{T}_0^+(c_r), \tau \in \mathcal{T}^{+\infty}, l \in \mathcal{L}, \text{rp} . \\ & \text{REFINE}(\text{Rewrite}(c)) = \text{rp} \wedge \text{Rewrite}(c_r) = \text{rp} \wedge \\ & \wedge \left(\langle c_r, \tau_r \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++\tau} \rangle \vee \langle c_r, \tau_r \rangle \uparrow^\infty \tau_{r++\tau} \right) \\ & \implies \llbracket \text{pathRB}(\text{rp}, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, \text{L}_{\text{path}}(\text{tp})). \end{aligned}$$

Proof. Soundness of the Path Reachability Bound.

Taking an arbitrary global program c , let $\text{rp} = \text{REFINE}(\text{Rewrite}(c))$ be its refined program and tp be an arbitrary transition path in this program. Let c_r be the while language program where $\text{rp} = \text{Rewrite}(c_r)$.

Taking an arbitrary initial trace of c , $\tau_0 \in \mathcal{T}_0^+(c)$ and an arbitrary initial trace of c_r , $\tau_r \in \mathcal{T}_0^+(c_r)$, an execution trace $\tau \in \mathcal{T}^{+\infty}$ and label $l \in \text{LV}(c_r)$ such that

$$\langle c_r, \tau_r \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++\tau} \rangle \vee \langle c_r, \tau_r \rangle \uparrow^\infty \tau_{r++\tau},$$

we have two cases.

- **The execution terminates and** $\langle c_r, \tau_r \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++\tau} \rangle$.

In this case we know $\tau \in \mathcal{T}^+$ and is sufficient to show,

$$\llbracket \text{pathRB}(\text{rp}, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, \text{L}_{\text{path}}(\text{tp})).$$

By induction on program rp , we have the same cases as in the proof of *Path Local Reachability-Bound Soundness* in Appendix B.4.2. The proof of the first four cases

are almost the same as the proof of Lemma 12.6.2, I don't unfold and repeat all the detail in this proof.

case: $rp = tp$

By Definition 75, we have $\text{pathRB}(tp, tp) = 1$, which is sound.

case: $rp = tp'$

By Definition 75, we have $\text{pathRB}(tp', tp) = 0$.

Since $tp' \neq tp$, we know tp isn't executed when executing tp' , which is sound.

case: $rp = rp_1; rp_2$

By Definition 75, we have

$$\text{pathRB}(rp_1; rp_2, tp) = \text{pathRB}(rp_1, tp) + \text{pathRB}(rp_2, tp).$$

Then by induction hypothesis on rp_1 and rp_2 , we have this case proved in the same way as the proof of Lemma 12.6.2 in Appendix B.4.2.

case: $rp = \text{choose}\{rp_1, \dots, rp_m\}$

By Definition 75, we have

$$\text{pathRB}(\text{choose}\{rp_1, \dots, rp_m\}, tp) = \max\{\text{pathRB}(rp_1, tp), \dots, \text{pathRB}(rp_m, tp)\}$$

By induction hypothesis on $rp_1, \dots, rp_m$, we have this case proved in the same way as the proof of Lemma 12.6.2 in Appendix B.4.2.

case: $rp = l' : rp'$

By Definition 75, we have

$$\text{pathRB}(l' : \text{rp}', \text{tp}) \triangleq \text{pathLocRB}(\text{rp}', \text{tp}), \quad \text{if } l = \text{enclosed}(\text{tp})$$

$$\text{lpRB}(l' : \text{rp}', \text{tp}) \times \max_{l' : \text{rp}' = \text{enclosed}(l'' : \text{rp}'')} \{\text{pathRB}(l'' : \text{rp}'', \text{tp})\}, \quad o.w.$$

Then, we have the following three cases:

sub-case: $l' : \text{rp}' = \text{enclosed}(\text{tp})$

In this case, we have $\text{pathRB}(l' : \text{rp}', \text{tp}) = \text{pathLocRB}(\text{rp}', \text{tp})$.

Since $l' : \text{rp}' = \text{enclosed}(\text{tp})$, we know $l' : \text{rp}'$ is the closet loop where tp is nested.

By Lemma 12.6.2, we know $\text{pathLocRB}(\text{rp}', \text{tp})$ is the sound upper bound on tp 's reaching times when executing $l' : \text{rp}'$.

So we have this case proved by

$$\llbracket \text{pathRB}(\text{rp}, \text{tp}) \rrbracket(\tau_0) = \llbracket \text{pathLocRB}(\text{rp}, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp})).$$

sub-case: *o.w.*

In this case, we have

$$\text{pathRB}(l' : \text{rp}', \text{tp}) = \text{lpRB}(l' : \text{rp}', \text{tp}) \times \max_{l' : \text{rp}' = \text{enclosed}(l'' : \text{rp}'')} \{\text{pathRB}(l'' : \text{rp}'', \text{tp})\}.$$

Then we need to show:

$$\llbracket \text{lpRB}(l' : \text{rp}', \text{tp}) \times \max_{l' : \text{rp}' = \text{enclosed}(l'' : \text{rp}'')} \{\text{pathRB}(l'' : \text{rp}'', \text{tp})\} \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp})).$$

Since $\text{rp} = l' : \text{rp}'$, according to Algorithm 7, we know c has the form `while  $[b]$  do  $c'$` for some boolean expression b .

For each $l'' : \text{rp}''$ such that $l' : \text{rp}' = \text{enclosed}(l'' : \text{rp}'')$, let $c'' \in \mathcal{C}$ be the while

language program corresponding to rp'' such that $\text{Rewrite}(c'') = \text{rp}''$, $c_1, c_2 \in \mathcal{C}$ such that $c' = c_1; c''; c_2$ $\tau'' \in \mathcal{T}^{+\infty}$ be the execution trace of c'' and $\tau_1, \tau_2 \in \mathcal{T}^{+\infty}$ be the execution traces before and after executing c'' , then we have

$$\begin{aligned} \langle c, \tau_r \rangle &\rightarrow \langle c'; c, \tau_{r++}[(b, l', \text{true})] \rangle \\ &\rightarrow^* \langle c''; c_2; c, \tau_{r++}[(b, l', \text{true})]_{++\tau_1} \rangle \\ &\rightarrow^* \langle c_2; c, \tau_{r++}[(b, l', \text{true})]_{++\tau_1++\tau''} \rangle \\ &\rightarrow^* \langle \text{skip}, \tau_{r++}[(b, l', \text{true})]_{++\tau_1++\tau''++\tau_2} \rangle \end{aligned}$$

By the trace non-decreasing property in Lemma 4.2.1, we know

$$\tau = [(b, l', \text{true})]_{++\tau_1++\tau''++\tau_2}.$$

By induction hypothesis on $\text{pathRB}(l'' : \text{rp}'', \text{tp})$ and c'' , we have

$$\llbracket \text{pathRB}(l'' : \text{rp}'', \text{tp}) \rrbracket(\tau_0) \geq \text{cnt}(\tau'', L(\text{tp})).$$

Then for all $l'' : \text{rp}''$ such that $l' : \text{rp}' = \text{enclosed}(l'' : \text{rp}'')$, we have

$$\llbracket \max_{l' : \text{rp}' = \text{enclosed}(l'' : \text{rp}'') \{ \text{pathRB}(l'' : \text{rp}'', \text{tp}) \} \rrbracket(\tau_0) \geq \text{cnt}(\tau'', L(\text{tp}))$$

Since τ'' is the execution trace for c'' in one iteration of c' , we know τ_2 contains multiple execution traces of c'' in all iterations of c' .

There are two ways to prove the soundness.

By Lemma B.4.1, we know $BD(l' : \text{rp}') \geq$ the iteration numbers of c' . Then we have

$$\begin{aligned} &\llbracket BD(l' : \text{rp}', c) \rrbracket(\tau_0) \times \text{cnt}(\tau'', L(\text{tp})) \\ &\geq \text{cnt}(\tau''_{++\tau_2}, L(\text{tp})) \\ &= \text{cnt}([(b, l', \text{true})]_{++\tau_1++\tau''++\tau_2}) \\ &= \text{lcnt}(\tau, L_{\text{path}}(\text{tp})) \end{aligned}$$

Then we have this case proved as the following.

$$\begin{aligned}
& \llbracket BD(l' : \text{rp}', c) \times \max_{l' : \text{rp}' = \text{enclosed}(l'' : \text{rp}'')} \{\text{pathRB}(l'' : \text{rp}'', \text{tp})\} \rrbracket(\tau_0) \\
& \geq \llbracket BD(l' : \text{rp}') \rrbracket(\tau_0) \times \llbracket \max_{l' : \text{rp}' = \text{enclosed}(l'' : \text{rp}'')} \{\text{pathRB}(l'' : \text{rp}'', \text{tp})\} \rrbracket(\tau_0) \\
& \geq \llbracket BD(l' : \text{rp}') \rrbracket(\tau_0) \times \text{cnt}(\tau'', L(\text{tp})) \\
& \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp}))
\end{aligned}$$

However, this isn't precise enough since tp might not be executed in every iteration of c' because it is located in nested loops.

In this sense, we prove the soundness of the tighter bound based on $\text{lpRB}(l : \text{rp}', \text{tp})$ and Lemma 12.7.1.

By Lemma 12.7.1, we know $\text{lpRB}(l : \text{rp}', \text{tp})$ is a sound upper bound on the maximum number of $l' : \text{rp}'$'s iteration times such that during these iterations, the tp is executed.

In this sense, $\text{lpRB}(l : \text{rp}', \text{tp}) \times \text{pathRB}(\text{rp}, \text{tp})$ is a sound upper bound on tp 's execution times during the execution of $l' : \text{rp}'$.

Then we have

$$\begin{aligned}
& \llbracket \text{lpRB}(l : \text{rp}', \text{tp}) \rrbracket(\tau_0) \times \text{cnt}(\tau'', L(\text{tp})) \\
& \geq \text{cnt}(\tau''_{++\tau_2}, L(\text{tp})) \\
& = \text{cnt}([(b, l', \text{true})]_{++\tau_1} \tau''_{++\tau_2}) \\
& = \text{lcnt}(\tau, L_{\text{path}}(\text{tp}))
\end{aligned}$$

Then we have this case proved as the following.

$$\begin{aligned}
& \text{pathRB}(l' : \text{rp}') \\
&= \llbracket \text{lpRB}(l : \text{rp}', \text{tp}) \times \max_{l' : \text{rp}' = \text{enclosed}(l'' : \text{rp}'') \{\text{pathRB}(l'' : \text{rp}'', \text{tp})\} \rrbracket(\tau_0) \\
&\geq \llbracket \text{lpRB}(l : \text{rp}', \text{tp}) \rrbracket(\tau_0) \times \llbracket \max_{l' : \text{rp}' = \text{enclosed}(l'' : \text{rp}'') \{\text{pathRB}(l'' : \text{rp}'', \text{tp})\} \rrbracket(\tau_0) \\
&\geq \llbracket \text{lpRB}(l : \text{rp}', \text{tp}) \rrbracket(\tau_0) \times \text{cnt}(\tau'', L(\text{tp})) \\
&\geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp}))
\end{aligned}$$

case: $\text{rp} = \text{repeat}(\text{rp}')$

By Definition 75, we have

$$\text{pathRB}(\text{repeat}(\text{rp}'), \text{tp}) = \text{pathRB}(\text{repeat}(\text{rp}'), \text{rp}) \times \text{pathRB}(\text{rp}', \text{tp}).$$

We will never meet this case by the program rewriting in Algorithm 7.

Because $\text{repeat}(\text{rp}')$ only shows up in a loop, we always match the case $l' : \text{repeat}(\text{rp}')$ in the previous case before match this case.

• **The execution is non-terminating and** $\langle c_r, \tau \rangle \uparrow^\infty \tau_{r++\tau}$.

In this case we know $\tau \in \mathcal{T}^\infty$ and is sufficient to show,

$$\llbracket \text{pathRB}(\text{rp}, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp})).$$

By induction on the refined program rp , we have the following cases:

case: $\text{rp} = \text{tp}$

In this case, we have $\text{pathRB}(\text{tp}, \text{tp}) = 1$ by Definition 71.

Since tp doesn't contain any loop, we know it will be executed at most once and terminates.

Then this case is vacuously true.

case: $rp = tp' \wedge tp' \neq tp$

This case is vacuously true as well for the same reason as above.

case: $rp = rp_1; rp_2$

In this case, we have $\text{pathRB}(rp_1; rp_2, tp) = \text{pathRB}(rp_1, tp) + \text{pathRB}(rp_2, tp)$ by Definition 7, and it is sufficient to show

$$\llbracket \text{pathRB}(rp_1, tp) + \text{pathRB}(rp_2, tp) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(tp))$$

Let $c_1, c_2 \in \mathcal{C}$ be the while language program corresponding to rp_1 and rp_2 such that $\text{Rewrite}(c_1) = rp_1$ and $\text{Rewrite}(c_2) = rp_2$. According to the Algorithm 7, we have $c_r = c_1; c_2$.

According to the operational semantics, let $\tau_1 \in \mathcal{T}^{+\infty}, \tau_2 \in \mathcal{T}^\infty$ be two execution traces, we have the two following possible executions,

$$\langle c_1; c_2, \tau_r \rangle \uparrow^\infty \tau_{r++\tau_1}$$

or

$$\langle c_1; c_2, \tau_r \rangle \rightarrow^* \langle c_2, \tau_{r++\tau_1} \rangle \uparrow^\infty \tau_{r++\tau_1++\tau_2}$$

By the trace non-decreasing property in Lemma 4.2.1 and the trace concatenation in Definition 42, we have in the first subcase $\tau = \tau_1$ and

$$\text{lcnt}(\tau, L_{\text{path}}(tp)) = \text{lcnt}(\tau_1, L_{\text{path}}(tp)) = \perp;$$

and in the second subcase $\tau = \tau_1++\tau_2$ and

$$\text{lcnt}(\tau, L_{\text{path}}(tp)) = \text{lcnt}(\tau_1, L_{\text{path}}(tp)) + \text{lcnt}(\tau_2, L_{\text{path}}(tp)) = \text{lcnt}(\tau_1, L_{\text{path}}(tp)) + \perp.$$

By induction hypothesis on τ_r, c_1 and c_2, τ_1 and c_2 , we have either

$$\llbracket \text{pathRB}(\text{rp}_1, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau_1, L_{\text{path}}(\text{tp}))$$

and

$$\llbracket \text{pathRB}(\text{rp}_2, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau_2, L_{\text{path}}(\text{tp}))$$

respectively.

Then we have this case proved as the follows in both subcases.

$$\begin{aligned} & \llbracket \text{pathRB}(\text{rp}_1, \text{tp}) + \text{pathRB}(\text{rp}_2, \text{tp}) \rrbracket(\tau_0) \\ & \geq \llbracket \text{pathRB}(\text{rp}_1, \text{tp}) \rrbracket(\tau_0) + \llbracket \text{pathRB}(\text{rp}_2, \text{tp}) \rrbracket(\tau_0) \\ & \geq \text{lcnt}(\tau_1, L_{\text{path}}(\text{tp})) + \text{lcnt}(\tau_2, L_{\text{path}}(\text{tp})) \\ & \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp})) \end{aligned}$$

case: $\text{rp} = \text{choose}\{\text{rp}_1, \dots, \text{rp}_m\}$

By Definition 7, we know $\text{pathRB}(\text{choose}\{\text{rp}_1, \dots, \text{rp}_m\}, \text{tp})$
 $= \max\{\text{pathRB}(\text{rp}_1, \text{tp}), \dots, \text{pathRB}(\text{rp}_m, \text{tp})\}$ by Definition 7.

Then it is sufficient to show

$$\llbracket \max\{\text{pathRB}(\text{rp}_1, \text{tp}), \dots, \text{pathRB}(\text{rp}_m, \text{tp})\} \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp}))$$

Let $c_1, \dots, c_m \in \mathcal{C}$ be the while language program corresponding to $\text{rp}_1, \dots, \text{rp}_m$ such that $\text{Rewrite}(c_1) = \text{rp}_1, \dots$ and $\text{Rewrite}(c_m) = \text{rp}_m$.

Let $\tau_1, \dots, \tau_m \in \mathcal{T}^{+\infty}$ be the execution traces such that each c_i and τ_i satisfies

$$\langle c_i, \tau_r \rangle \rightarrow^* \langle [\text{skip}]^l, \tau_{r++\tau_i} \rangle \vee \langle c_i, \tau_r \rangle \uparrow^\infty \tau_{r++\tau_i}.$$

Since the execution of each c_i corresponds to one possible execution of c_r as analyzed in Appendix B.4.1, we know there exists at least a sub-program c_j where $\tau_j \in \mathcal{T}^\infty$ such that

$$\langle c_j, \tau_r \rangle \uparrow^\infty \tau_{r++}\tau_j.$$

Then we have $\text{lcnt}(\tau_j, L_{\text{path}}(\text{tp})) = \perp$, and by induction hypothesis on c_j we also know

$$\llbracket \text{pathRB}(\text{rp}_j, \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau_j, L_{\text{path}}(\text{tp}))$$

Then we have this case proved by following

$$\begin{aligned} & \llbracket \max\{\text{pathRB}(\text{rp}_1, \text{tp}), \dots, \text{pathRB}(\text{rp}_m, \text{tp})\} \rrbracket(\tau_0) \\ & \geq \max\{\llbracket \text{pathRB}(\text{rp}_1, \text{tp}) \rrbracket(\tau_0), \dots, \llbracket \text{pathRB}(\text{rp}_2, \text{tp}) \rrbracket(\tau_0)\} \\ & = \infty \\ & \geq \max\{\text{lcnt}(\tau_1, L_{\text{path}}(\text{tp})), \dots, \text{lcnt}(\tau_m, L_{\text{path}}(\text{tp}))\} \\ & \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp})) \end{aligned}$$

case: $\text{rp} = \text{repeat}(\text{rp}')$

We have $\text{pathRB}(\text{repeat}(\text{rp}'), \text{tp}) = BD(\text{repeat}(\text{rp}'), c) \times \text{pathRB}(\text{rp}', \text{tp})$ by Definition 7, and it is sufficient to show

$$\llbracket BD(\text{repeat}(\text{rp}'), c) \times \text{pathRB}(\text{rp}', \text{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp}))$$

Since execution of $\text{repeat}(\text{rp}')$ non-terminates, and it is a loop program, by the second subcase of the non-terminating case in the proof of Lemma 12.6.1, we know

$$BD(\text{repeat}(\text{rp}'), c) = \infty.$$

Then we have this case proved by

$$\begin{aligned}
& \llbracket BD(\text{repeat}(\text{rp}'), c) \times \text{pathRB}(\text{rp}', \text{tp}) \rrbracket(\tau_0) \\
&= \llbracket \infty \rrbracket(\tau_0) \\
&\geq \text{lcnt}(\tau, L_{\text{path}}(\text{tp}))
\end{aligned}$$

case: $\text{rp} = l : \text{rp}_l$

In this case, we have

$$\text{pathRB}(l : \text{rp}_l, c) = \text{pathRB}(\text{rp}_l, c)$$

Let $c_l \in \mathcal{C}$ be the while language program corresponding to rp_l such that $\text{Rewrite}(c_l) = \text{rp}_l$.

According to the Algorithm 7 and the program refinement algorithm REFINER, we know $c_l = c_l$.

Then we have this case proved trivially by induction hypothesis on rp_l and c_l with the same initial and execution trace τ_r .

case: $\text{rp} = l' : \text{rp}'$ **and** $l' : \text{rp}' \neq \text{enclosed}(\text{tp})$

$\text{pathRB}(l' : \text{rp}', \text{tp}) = \text{pathRB}(\text{rp}', \text{tp})$ Since $l' : \text{rp}' \neq \text{enclosed}(\text{tp})$, we know tp isn't executed when executing c_l , i.e., $\text{lcnt}(\tau, L_{\text{path}}(\text{tp})) = 0$.

Then we have

$$\llbracket \text{pathRB}(l' : \text{rp}', \text{tp}) \rrbracket(\tau_0) = 0 = \text{lcnt}(\tau, L_{\text{path}}(\text{tp})),$$

and this case is proved. □

B.4.5 Path-sensitive Reachability-bound Soundness (Proof of Theorem 12.5.1)

Theorem (Soundness of the Path-sensitive Reachability Bound). *For every program c and every label l in this program, $\llbracket \text{psRB}(c, l) \rrbracket$ is a Reachability-Bound for l in c .*

$$\begin{aligned}
& \forall c, c_r \in \mathcal{C}, \text{tp} \in \text{absG}(c), \tau_0 \in \mathcal{T}_0^+(c), \tau_r \in \mathcal{T}_0^+(c_r), \tau \in \mathcal{T}^{+\infty}, l, l' \in \mathcal{L}, \text{rp} . \\
& \text{rp} = \text{REFINE}(\text{Rewrite}(c)) \wedge \text{rp} = \text{Rewrite}(c_r) \wedge \\
& \wedge \left(\langle c_r, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \vee \langle c_r, \tau_0 \rangle \uparrow^\infty \tau_{0++\tau} \right) \\
& \implies \llbracket \text{psRB}(c, l) \rrbracket(\tau_0) \geq \text{cnt}(\tau, l).
\end{aligned}$$

Proof. Path-sensitive Reachability Bound Soundness.

Taking arbitrary program c , a program point $l \in \text{LV}(c)$, Let c_r be the while language program where $\text{rp} = \text{Rewrite}(c_r)$.

Taking an arbitrary initial trace of c , $\tau_0 \in \mathcal{T}_0^+(c)$, an arbitrary initial trace of c_r , $\tau_r \in \mathcal{T}_0^+(c_r)$, and an execution trace $\tau \in \mathcal{T}^{+\infty}$ and label $l' \in \text{LV}(c_r)$ such that

$$\langle c_r, \tau_r \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{r++\tau} \rangle \vee \langle c_r, \tau_r \rangle \uparrow^\infty \tau_{r++\tau},$$

we have two cases.

- **The execution terminates and** $\langle c_r, \tau_r \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{r++\tau} \rangle$.

In this case we know $\tau \in \mathcal{T}^+$ and it is sufficient to show,

$$\llbracket \text{psRB}(c, l) \rrbracket(\tau_0) \geq \text{cnt}(\tau, l).$$

By Definition 68, it is sufficient to show

$$\llbracket \sum \{ \text{pathRB}(\text{rp}, \text{tp}) \mid \text{tp} \in \text{rp} \wedge l \in \text{tp} \} \rrbracket(\tau_0) \geq \text{cnt}(\tau, l)$$

According to whether the program point l is in any loop, there are two cases as

follows.

case: $l \notin SCC(\text{absG}(c))$

In this case, we know l isn't in any loop, and it will be executed at most once. So we have

$$\text{cnt}(\tau, l) \leq 1$$

Since $l \notin SCC(\text{absG}(c))$, we also know there is only one tp contains l by Definition 31.

By Definition 75, we know $\text{pathRB}(\text{rp}, \text{tp}) = 1$.

Then we have

$$\begin{aligned} & \llbracket \sum \{ \text{pathRB}(\text{rp}, \text{tp}) \mid \text{tp} \in \text{rp} \wedge l \in \text{tp} \} \rrbracket(\tau_0) \\ &= \llbracket \sum \{ 1 \}(\tau_0) \rrbracket(\tau_0) \\ &= 1 \geq \text{cnt}(\tau, l). \end{aligned}$$

This case is proved.

case: $l \in SCC(\text{absG}(c))$

In this case, let TP be the set of all transition paths containing label l in program rp , then it is sufficient to show

$$\llbracket \sum \{ \text{pathRB}(\text{rp}, \text{tp}) \mid \text{tp} \in TP \} \rrbracket(\tau_0) \geq \text{cnt}(\tau, l)$$

By Definition 60, we know for each $\text{tp} \in TP$, $l \in L_{\text{path}}(\text{tp})$ and l shows up at most once on $L_{\text{path}}(\text{tp})$.

Then we have

$$\text{cnt}(\tau, l) \leq \sum \{ 1 \text{cnt}(\tau, L_{\text{path}}(\text{tp})) \mid \text{tp} \in TP \}.$$

By the soundness of path reachability-bound in Lemma 12.8.1, we know for each

$\mathbf{tp} \in TP$,

$$\llbracket \text{pathRB}(\mathbf{rp}, \mathbf{tp}) \rrbracket(\tau_0) \geq \text{lcnt}(\tau, L_{\text{path}}(\mathbf{tp})).$$

Then we have the soundness proved by

$$\begin{aligned} & \llbracket \text{psRB}(c, l) \rrbracket(\tau_0) \\ &= \llbracket \sum \{ \text{pathRB}(\mathbf{rp}, \mathbf{tp}) \mid \mathbf{tp} \in TP \} \rrbracket(\tau_0) \\ &\geq \sum \{ \llbracket \text{pathRB}(\mathbf{rp}, \mathbf{tp}) \rrbracket(\tau_0) \mid \mathbf{tp} \in TP \} \\ &\geq \sum \{ \text{lcnt}(\tau, L_{\text{path}}(\mathbf{tp})) \mid \mathbf{tp} \in TP \} \\ &\geq \text{cnt}(\tau, l) \end{aligned}$$

• **The execution is non-terminating and** $\langle c_r, \tau \rangle \uparrow^\infty \tau_{r++}\tau$.

In this case we know $\tau \in \mathcal{T}^\infty$ and is sufficient to show,

$$\llbracket \text{psRB}(c, l) \rrbracket(\tau_0) \geq \text{cnt}(\tau, l).$$

By Definition 47, we know $\text{cnt}(\tau, l) = \perp$ and this case is proved as well by the soundness of path reachability-bound in Lemma 12.8.1. $\square$

BIBLIOGRAPHY

- Aguirre, A., Barthe, G., Gaboardi, M., Garg, D., Katsumata, S., & Sato, T. (2021). Higher-order probabilistic adversarial computations: Categorical semantics and program logics. *CoRR*, *abs/2107.01155*.
- Albert, E., Arenas, P., Genaim, S., & Puebla, G. (2008). Automatic inference of upper bounds for recurrence relations in cost analysis. In M. Alpuente, & G. Vidal (Eds.) *Static Analysis, 15th International Symposium, SAS 2008, Valencia, Spain, July 16-18, 2008. Proceedings*, vol. 5079 of *Lecture Notes in Computer Science*, (pp. 221–237). Springer. https://doi.org/10.1007/978-3-540-69166-2_15.
- Alias, C., Darte, A., Feautrier, P., & Gonnord, L. (2010). Multi-dimensional rankings, program termination, and complexity bounds of flowchart programs. In R. Cousot, & M. Martel (Eds.) *Static Analysis - 17th International Symposium, SAS 2010, Perpignan, France, September 14-16, 2010. Proceedings*, vol. 6337 of *Lecture Notes in Computer Science*, (pp. 117–133). Springer. https://doi.org/10.1007/978-3-642-15769-1_8.
- Arlt, S., Podelski, A., Bertolini, C., Schäfer, M., Banerjee, I., & Memon, A. M. (2012). Lightweight static analysis for GUI testing. In *23rd IEEE International Symposium on Software Reliability Engineering, ISSRE 2012, Dallas, TX, USA, November 27-30, 2012*, (pp. 301–310). IEEE Computer Society. <https://doi.org/10.1109/ISSRE.2012.25>.
- Austin, T. M., & Sohi, G. S. (1992). Dynamic dependency analysis of ordinary programs. In A. Gottlieb (Ed.) *Proceedings of the 19th Annual International Symposium on Computer Architecture. Gold Coast, Australia, May 1992*, (pp. 342–351). ACM. <https://doi.org/10.1145/139669.140395>.
- Balakrishnan, G., Sankaranarayanan, S., Ivancic, F., & Gupta, A. (2009). Refining the control structure of loops using static analysis. In S. Chakraborty, & N. Halbwachs (Eds.) *Proceedings of the 9th ACM & IEEE International conference on Embedded software, EMSOFT 2009, Grenoble, France, October 12-16, 2009*, (pp. 49–58). ACM. <https://doi.org/10.1145/1629335.1629343>.
- Barbosa, M., Barthe, G., Grégoire, B., Koutsos, A., & Strub, P. (2021). Mechanized proofs of adversarial complexity and application to universal composability. *IACR Cryptol. ePrint Arch.*, 2021, 156.
- Bassily, R., Nissim, K., Smith, A. D., Steinke, T., Stemmer, U., & Ullman, J. R. (2016). Algorithmic stability for adaptive data analysis. In D. Wicks, & Y. Mansour

- (Eds.) *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2016, Cambridge, MA, USA, June 18-21, 2016*, (pp. 1046–1059). ACM. <https://doi.org/10.1145/2897518.2897566>.
- Belli, F. (2001). Finite-state testing and analysis of graphical user interfaces. In *12th International Symposium on Software Reliability Engineering (ISSRE 2001)*, 27-30 November 2001, Hong Kong, China, (pp. 34–43). IEEE Computer Society. <https://doi.org/10.1109/ISSRE.2001.989456>.
- Benchmark, A. (2009). <http://mtc.epfl.ch/software-tools/ABC/>.
- Benchmark, C. (2015). <https://www.cs.yale.edu/homes/qcar/aaa/>.
- Benchmark, I. (2020). <https://github.com/icra-team/icra>.
- Benchmark, K. (2018). <https://github.com/s-falke/kittel-koat/tree/master/examples>.
- Benchmark, L. (2017). <https://forsyte.at/static/people/sinn/loopusJAR/index.html>.
- Benchmark, R. (2010). <http://icps.u-strasbg.fr/polylib/>.
- Benchmark, T. (2021). <https://github.com/aeFlores/CoFloCo/tree/master/src>.
- Bilardi, G., & Pingali, K. (1996). A framework for generalized control dependence. In *Proceedings of the ACM SIGPLAN 1996 conference on Programming language design and implementation*, (pp. 291–300).
- Bradley, A. R., Manna, Z., & Sipma, H. B. (2005). Linear ranking with reachability. In K. Etessami, & S. K. Rajamani (Eds.) *Computer Aided Verification, 17th International Conference, CAV 2005, Edinburgh, Scotland, UK, July 6-10, 2005, Proceedings*, vol. 3576 of *Lecture Notes in Computer Science*, (pp. 491–504). Springer. https://doi.org/10.1007/11513988_48.
- Breck, J., Cyphert, J., Kincaid, Z., & Reps, T. W. (2020). Templates and recurrences: better together. In A. F. Donaldson, & E. Torlak (Eds.) *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020*, (pp. 688–702). ACM. <https://doi.org/10.1145/3385412.3386035>.
- Brockschmidt, M., Emmes, F., Falke, S., Fuhs, C., & Giesl, J. (2014). Alternating runtime and size complexity analysis of integer programs. In E. Ábrahám, & K. Havelund (Eds.) *Tools and Algorithms for the Construction and Analysis of Systems - 20th International Conference, TACAS 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April*

- 5-13, 2014. *Proceedings*, vol. 8413 of *Lecture Notes in Computer Science*, (pp. 140–155). Springer. https://doi.org/10.1007/978-3-642-54862-8_10.
- Brockschmidt, M., Emmes, F., Falke, S., Fuhs, C., & Giesl, J. (2016). Analyzing runtime and size complexity of integer programs. *ACM Trans. Program. Lang. Syst.*, 38(4), 13:1–13:50.
- Burke, M. G., & Ryder, B. G. (1990). A critical analysis of incremental iterative data flow analysis algorithms. *IEEE Trans. Software Eng.*, 16(7), 723–728.
- Carbonneaux, Q., Hoffmann, J., & Shao, Z. (2015). Compositional certified resource bounds. In D. Grove, & S. M. Blackburn (Eds.) *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015*, (pp. 467–478). ACM. <https://doi.org/10.1145/2737924.2737955>.
- Cheney, J., Ahmed, A., & Acar, U. A. (2011). Provenance as dependency analysis. *Math. Struct. Comput. Sci.*, 21(6), 1301–1337.
- Çiçek, E., Barthe, G., Gaboardi, M., Garg, D., & Hoffmann, J. (2017). Relational cost analysis. In G. Castagna, & A. D. Gordon (Eds.) *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017, Paris, France, January 18-20, 2017*, (pp. 316–329). ACM. <https://doi.org/10.1145/3009837.3009858>.
- Cook, B., See, A., & Zuleger, F. (2013). Ramsey vs. lexicographic termination proving. In N. Piterman, & S. A. Smolka (Eds.) *Tools and Algorithms for the Construction and Analysis of Systems - 19th International Conference, TACAS 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings*, vol. 7795 of *Lecture Notes in Computer Science*, (pp. 47–61). Springer. https://doi.org/10.1007/978-3-642-36742-7_4.
- Cousot, P. (2019a). Abstract semantic dependency. In B. E. Chang (Ed.) *Static Analysis - 26th International Symposium, SAS 2019, Porto, Portugal, October 8-11, 2019, Proceedings*, vol. 11822 of *Lecture Notes in Computer Science*, (pp. 389–410). Springer. https://doi.org/10.1007/978-3-030-32304-2_19.
- Cousot, P. (2019b). Syntactic and semantic soundness of structural dataflow analysis. In B. E. Chang (Ed.) *Static Analysis - 26th International Symposium, SAS 2019, Porto, Portugal, October 8-11, 2019, Proceedings*, vol. 11822 of *Lecture Notes in Computer Science*, (pp. 96–117). Springer. https://doi.org/10.1007/978-3-030-32304-2_6.
- Cousot, P., & Cousot, R. (1977). Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints.

- In R. M. Graham, M. A. Harrison, & R. Sethi (Eds.) *Conference Record of the Fourth ACM Symposium on Principles of Programming Languages, Los Angeles, California, USA, January 1977*, (pp. 238–252). ACM. <https://doi.org/10.1145/512950.512973>.
- Cousot, P., & Halbwachs, N. (1978). Automatic discovery of linear restraints among variables of a program. In A. V. Aho, S. N. Zilles, & T. G. Szymanski (Eds.) *Conference Record of the Fifth Annual ACM Symposium on Principles of Programming Languages, Tucson, Arizona, USA, January 1978*, (pp. 84–96). ACM Press. <https://doi.org/10.1145/512760.512770>.
- Crary, K., & Weirich, S. (2000). Resource bound certification. In M. N. Wegman, & T. W. Reps (Eds.) *POPL 2000, Proceedings of the 27th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Boston, Massachusetts, USA, January 19-21, 2000*, (pp. 184–198). ACM. <https://doi.org/10.1145/325694.325716>.
- Cyphert, J., Breck, J., Kincaid, Z., & Reps, T. W. (2019). Refinement of path expressions for static analysis. *Proc. ACM Program. Lang.*, 3(POPL), 45:1–45:29.
- Cytron, R., Ferrante, J., Rosen, B. K., Wegman, M. N., & Zadeck, F. K. (1991). Efficiently computing static single assignment form and the control dependence graph. *ACM Trans. Program. Lang. Syst.*, 13(4), 451–490.
- Denning, D. E., & Denning, P. J. (1977). Certification of programs for secure information flow. *Commun. ACM*, 20(7), 504–513.
- Dwork, C., Feldman, V., Hardt, M., Pitassi, T., Reingold, O., & Roth, A. (2015a). Generalization in adaptive data analysis and holdout reuse. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, & R. Garnett (Eds.) *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, (pp. 2350–2358). <https://proceedings.neurips.cc/paper/2015/hash/bad5f33780c42f2588878a9d07405083-Abstract.html>.
- Dwork, C., Feldman, V., Hardt, M., Pitassi, T., Reingold, O., & Roth, A. L. (2015b). Preserving statistical validity in adaptive data analysis. In R. A. Servedio, & R. Rubinfeld (Eds.) *Proceedings of the Forty-Seventh Annual ACM on Symposium on Theory of Computing, STOC 2015, Portland, OR, USA, June 14-17, 2015*, (pp. 117–126). ACM. <https://doi.org/10.1145/2746539.2746580>.
- Falke, S., Kapur, D., & Sinz, C. (2011). Termination analysis of C programs using compiler intermediate languages. In M. Schmidt-Schauß (Ed.) *Proceedings of the 22nd International Conference on Rewriting Techniques and Applications, RTA 2011, May 30 - June 1, 2011, Novi Sad, Serbia*, vol. 10 of LIPIcs, (pp. 41–50). Schloss

- Dagstuhl - Leibniz-Zentrum für Informatik. <https://doi.org/10.4230/LIPIcs.RTA.2011.41>.
- Falke, S., Kapur, D., & Sinz, C. (2012). Termination analysis of imperative programs using bitvector arithmetic. In R. Joshi, P. Müller, & A. Podelski (Eds.) *Verified Software: Theories, Tools, Experiments - 4th International Conference, VSTTE 2012, Philadelphia, PA, USA, January 28-29, 2012. Proceedings*, vol. 7152 of *Lecture Notes in Computer Science*, (pp. 261–277). Springer. https://doi.org/10.1007/978-3-642-27705-4_21.
- Feldman, V., & Steinke, T. (2017). Generalization for adaptively-chosen estimators via stable median. In S. Kale, & O. Shamir (Eds.) *Proceedings of the 30th Conference on Learning Theory, COLT 2017, Amsterdam, The Netherlands, 7-10 July 2017*, vol. 65 of *Proceedings of Machine Learning Research*, (pp. 728–757). PMLR. <http://proceedings.mlr.press/v65/feldman17a.html>.
- Ferrante, J., Ottenstein, K. J., & Warren, J. D. (1987). The program dependence graph and its use in optimization. *ACM Trans. Program. Lang. Syst.*, 9(3), 319–349.
- Flores-Montoya, A. (2016). Upper and lower amortized cost bounds of programs expressed as cost relations. In J. S. Fitzgerald, C. L. Heitmeyer, S. Gnesi, & A. Philippou (Eds.) *FM 2016: Formal Methods - 21st International Symposium, Limassol, Cyprus, November 9-11, 2016, Proceedings*, vol. 9995 of *Lecture Notes in Computer Science*, (pp. 254–273). https://doi.org/10.1007/978-3-319-48989-6_16.
- Flores-Montoya, A. (2017). *Cost Analysis of Programs Based on the Refinement of Cost Relations*. Ph.D. thesis, Darmstadt University of Technology, Germany. <http://tuprints.ulb.tu-darmstadt.de/6746/>.
- Flores-Montoya, A., & Hähnle, R. (2014). Resource analysis of complex programs with cost equations. In J. Garrigue (Ed.) *Programming Languages and Systems - 12th Asian Symposium, APLAS 2014, Singapore, November 17-19, 2014, Proceedings*, vol. 8858 of *Lecture Notes in Computer Science*, (pp. 275–295). Springer. https://doi.org/10.1007/978-3-319-12736-1_15.
- Gaboardi, M., Katsumata, S., Orchard, D., & Sato, T. (2021). Graded hoare logic and its categorical semantics. In N. Yoshida (Ed.) *Programming Languages and Systems - 30th European Symposium on Programming, ESOP 2021, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2021, Luxembourg City, Luxembourg, March 27 - April 1, 2021, Proceedings*, vol. 12648 of *Lecture Notes in Computer Science*, (pp. 234–263). Springer. https://doi.org/10.1007/978-3-030-72019-3_9.
- Gelman, A., & Loken, E. (2014). The statistical crisis in science. *Am Sci*, 102(6), 460.

- Gulwani, S., Jain, S., & Koskinen, E. (2009a). Control-flow refinement and progress invariants for bound analysis. In M. Hind, & A. Diwan (Eds.) *Proceedings of the 2009 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2009, Dublin, Ireland, June 15-21, 2009*, (pp. 375–385). ACM. <https://doi.org/10.1145/1542476.1542518>.
- Gulwani, S., Mehra, K. K., & Chilimbi, T. M. (2009b). SPEED: precise and efficient static estimation of program computational complexity. In Z. Shao, & B. C. Pierce (Eds.) *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2009, Savannah, GA, USA, January 21-23, 2009*, (pp. 127–139). ACM. <https://doi.org/10.1145/1480881.1480898>.
- Gulwani, S., & Zuleger, F. (2010). The reachability-bound problem. In B. G. Zorn, & A. Aiken (Eds.) *Proceedings of the 2010 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2010, Toronto, Ontario, Canada, June 5-10, 2010*, (pp. 292–304). ACM. <https://doi.org/10.1145/1806596.1806630>.
- Gustafsson, J., Ermedahl, A., & Lisper, B. (2005). Towards a flow analysis for embedded system C programs. In *10th IEEE International Workshop on Object-Oriented Real-Time Dependable Systems (WORDS 2005), 2-4 February 2005, Sedona, AZ, USA*, (pp. 287–300). IEEE Computer Society. <https://doi.org/10.1109/WORDS.2005.53>.
- Hammer, C., Grimme, M., & Krinke, J. (2006). Dynamic path conditions in dependence graphs. In J. Hatcliff, & F. Tip (Eds.) *Proceedings of the 2006 ACM SIGPLAN Workshop on Partial Evaluation and Semantics-based Program Manipulation, 2006, Charleston, South Carolina, USA, January 9-10, 2006*, (pp. 58–67). ACM. <https://doi.org/10.1145/1111542.1111552>.
- Hardt, M., & Ullman, J. R. (2014). Preventing false discovery in interactive data analysis is hard. In *55th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2014, Philadelphia, PA, USA, October 18-21, 2014*, (pp. 454–463). IEEE Computer Society. <https://doi.org/10.1109/FOCS.2014.55>.
- Humenberger, A., Jaroschek, M., & Kovács, L. (2018). Invariant generation for multi-path loops with polynomial assignments. In I. Dillig, & J. Palsberg (Eds.) *Verification, Model Checking, and Abstract Interpretation - 19th International Conference, VMCAI 2018, Los Angeles, CA, USA, January 7-9, 2018, Proceedings*, vol. 10747 of *Lecture Notes in Computer Science*, (pp. 226–246). Springer. https://doi.org/10.1007/978-3-319-73721-8_11.
- Ioannidis, J. P. (2005). Why most published research findings are false. *PLoS medicine*, 2(8), e124.

- Israel, D. R. (1956). Simulation techniques for the test and evaluation of real-time computer programs. In *Proceedings of the 11th ACM national meeting, ACM 1956, USA*, (pp. 72–75). ACM. <https://doi.org/10.1145/800258.808952>.
- Jana, A., Halder, R., Abhishekh, K. V., Ganni, S. D., & Cortesi, A. (2020). Extending abstract interpretation to dependency analysis of database applications. *IEEE Trans. Software Eng.*, 46(5), 463–494.
- Jost, S., Hammond, K., Loidl, H., & Hofmann, M. (2010). Static determination of quantitative resource usage for higher-order programs. In M. V. Hermenegildo, & J. Palsberg (Eds.) *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2010, Madrid, Spain, January 17-23, 2010*, (pp. 223–236). ACM. <https://doi.org/10.1145/1706299.1706327>.
- Jung, C., Ligett, K., Neel, S., Roth, A., Sharifi-Malvajerdi, S., & Shenfeld, M. (2021). A new analysis of differential privacy’s generalization guarantees (invited paper). In S. Khuller, & V. V. Williams (Eds.) *STOC ’21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021*, (p. 9). ACM. <https://doi.org/10.1145/3406325.3465358>.
- Kildall, G. A. (1973). A unified approach to global program optimization. In P. C. Fischer, & J. D. Ullman (Eds.) *Conference Record of the ACM Symposium on Principles of Programming Languages, Boston, Massachusetts, USA, October 1973*, (pp. 194–206). ACM Press. <https://doi.org/10.1145/512927.512945>.
- Kincaid, Z., Breck, J., Cyphert, J., & Reps, T. W. (2019). Closed forms for numerical loops. *Proc. ACM Program. Lang.*, 3(POPL), 55:1–55:29.
- Kincaid, Z., Cyphert, J., Breck, J., & Reps, T. W. (2018). Non-linear reasoning for invariant synthesis. *Proc. ACM Program. Lang.*, 2(POPL), 54:1–54:33.
- Lu, T. (2021). <https://zenodo.org/record/5140586#.Y5pBoC-B1QI>.
- Lu, T., Chang, B. E., & Trivedi, A. (2021). Selectively-amortized resource bounding. In C. Dragoi, S. Mukherjee, & K. S. Namjoshi (Eds.) *Static Analysis - 28th International Symposium, SAS 2021, Chicago, IL, USA, October 17-19, 2021, Proceedings*, vol. 12913 of *Lecture Notes in Computer Science*, (pp. 286–307). Springer. https://doi.org/10.1007/978-3-030-88806-0_14.
- Malacaria, P. (2007). Assessing security threats of looping constructs. In M. Hofmann, & M. Felleisen (Eds.) *Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2007, Nice, France, January 17-19, 2007*, (pp. 225–235). ACM. <https://doi.org/10.1145/1190216.1190251>.

- Manolios, P., & Vroon, D. (2006). Termination analysis with calling context graphs. In T. Ball, & R. B. Jones (Eds.) *Computer Aided Verification, 18th International Conference, CAV 2006, Seattle, WA, USA, August 17-20, 2006, Proceedings*, vol. 4144 of *Lecture Notes in Computer Science*, (pp. 401–414). Springer. https://doi.org/10.1007/11817963_36.
- Mastroeni, I., & Zanardini, D. (2008). Data dependencies and program slicing: from syntax to abstract semantics. In R. Glück, & O. de Moor (Eds.) *Proceedings of the 2008 ACM SIGPLAN Symposium on Partial Evaluation and Semantics-based Program Manipulation, PEPM 2008, San Francisco, California, USA, January 7-8, 2008*, (pp. 125–134). ACM. <https://doi.org/10.1145/1328408.1328428>.
- Mastroeni, I., & Zanardini, D. (2017). Abstract program slicing: An abstract interpretation-based approach to program slicing. *ACM Trans. Comput. Log.*, 18(1), 7:1–7:58.
- Memon, A. M. (2007). An event-flow model of gui-based applications for testing. *Softw. Test. Verification Reliab.*, 17(3), 137–157.
- Miné, A. (2006). The octagon abstract domain. *High. Order Symb. Comput.*, 19(1), 31–100.
- Nagar, K., & Jagannathan, S. (2018). Automated detection of serializability violations under weak consistency. In S. Schewe, & L. Zhang (Eds.) *29th International Conference on Concurrency Theory, CONCUR 2018, September 4-7, 2018, Beijing, China*, vol. 118 of *LIPIcs*, (pp. 41:1–41:18). Schloss Dagstuhl - Leibniz-Zentrum für Informatik. <https://doi.org/10.4230/LIPIcs.CONCUR.2018.41>.
- Pollock, L. L., & Soffa, M. L. (1989). An incremental version of iterative data flow analysis. *IEEE Trans. Software Eng.*, 15(12), 1537–1549.
- Potechin, A. (2017). A note on amortized branching program complexity. In R. O'Donnell (Ed.) *32nd Computational Complexity Conference, CCC 2017, July 6-9, 2017, Riga, Latvia*, vol. 79 of *LIPIcs*, (pp. 4:1–4:12). Schloss Dagstuhl - Leibniz-Zentrum für Informatik. <https://doi.org/10.4230/LIPIcs.CCC.2017.4>.
- Qu, W. (2022). *Program-based Analysis for Quantitative Properties*. Ph.D. thesis, Boston University.
- Qu, W., Gaboardi, M., & Garg, D. (2019). Relational cost analysis for functional-imperative programs. *Proc. ACM Program. Lang.*, 3(ICFP), 92:1–92:29.
- Radicek, I., Barthe, G., Gaboardi, M., Garg, D., & Zuleger, F. (2018). Monadic refinements for relational cost analysis. *Proc. ACM Program. Lang.*, 2(POPL), 36:1–36:32.

- Rajani, V., Gaboardi, M., Garg, D., & Hoffmann, J. (2021). A unifying type-theory for higher-order (amortized) cost analysis. *Proc. ACM Program. Lang.*, 5(POPL), 1–28.
- Rogers, R., Roth, A., Smith, A. D., Srebro, N., Thakkar, O., & Woodworth, B. E. (2020). Guaranteed validity for empirical approaches to adaptive data analysis. In S. Chiappa, & R. Calandra (Eds.) *The 23rd International Conference on Artificial Intelligence and Statistics, AISTATS 2020, 26-28 August 2020, Online [Palermo, Sicily, Italy]*, vol. 108 of *Proceedings of Machine Learning Research*, (pp. 2830–2840). PMLR. <http://proceedings.mlr.press/v108/rogers20a.html>.
- Sabelfeld, A., & Myers, A. C. (2003). Language-based information-flow security. *IEEE J. Sel. Areas Commun.*, 21(1), 5–19.
- Sharma, R., Dillig, I., Dillig, T., & Aiken, A. (2011). Simplifying loop invariant generation using splitter predicates. In G. Gopalakrishnan, & S. Qadeer (Eds.) *Computer Aided Verification - 23rd International Conference, CAV 2011, Snowbird, UT, USA, July 14-20, 2011. Proceedings*, vol. 6806 of *Lecture Notes in Computer Science*, (pp. 703–719). Springer. https://doi.org/10.1007/978-3-642-22110-1_57.
- Sinn, M., Zuleger, F., & Veith, H. (2014). A simple and scalable static analysis for bound analysis and amortized complexity analysis. In A. Biere, & R. Bloem (Eds.) *Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings*, vol. 8559 of *Lecture Notes in Computer Science*, (pp. 745–761). Springer. https://doi.org/10.1007/978-3-319-08867-9_50.
- Sinn, M., Zuleger, F., & Veith, H. (2017). Complexity and resource bound analysis of imperative programs using difference constraints. *J. Autom. Reason.*, 59(1), 3–45.
- Steinke, T., & Zakynthinou, L. (2020). Reasoning about generalization via conditional mutual information. In J. D. Abernethy, & S. Agarwal (Eds.) *Conference on Learning Theory, COLT 2020, 9-12 July 2020, Virtual Event [Graz, Austria]*, vol. 125 of *Proceedings of Machine Learning Research*, (pp. 3437–3452). PMLR. <http://proceedings.mlr.press/v125/steinke20a.html>.
- Ullman, J. R., Smith, A. D., Nissim, K., Stemmer, U., & Steinke, T. (2018). The limits of post-selection generalization. In S. Bengio, H. M. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, & R. Garnett (Eds.) *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, (pp. 6402–6411). <https://proceedings.neurips.cc/paper/2018/hash/77ee3bc58ce560b86c2b59363281e914-Abstract.html>.

- Vesga, E. L., Russo, A., & Gaboardi, M. (2021). A programming language for data privacy with accuracy estimations. *ACM Trans. Program. Lang. Syst.*, 43(2), 6:1–6:42.
- Weiser, M. D. (1984). Program slicing. *IEEE Trans. Software Eng.*, 10(4), 352–357.
- Winograd-Cort, D., Haeberlen, A., Roth, A., & Pierce, B. C. (2017). A framework for adaptive differential privacy. *Proc. ACM Program. Lang.*, 1(ICFP), 10:1–10:29.
- Yeom, S., Giacomelli, I., Fredrikson, M., & Jha, S. (2018). Privacy risk in machine learning: Analyzing the connection to overfitting. In *31st IEEE Computer Security Foundations Symposium, CSF 2018, Oxford, United Kingdom, July 9-12, 2018*, (pp. 268–282). IEEE Computer Society. <https://doi.org/10.1109/CSF.2018.00027>.
- Zanioli, M., & Cortesi, A. (2011). Information leakage analysis by abstract interpretation. In I. Cerná, T. Gyimóthy, J. Hromkovic, K. G. Jeffery, R. Královic, M. Vukolic, & S. Wolf (Eds.) *SOFSEM 2011: Theory and Practice of Computer Science - 37th Conference on Current Trends in Theory and Practice of Computer Science, Nový Smokovec, Slovakia, January 22-28, 2011. Proceedings*, vol. 6543 of *Lecture Notes in Computer Science*, (pp. 545–557). Springer. https://doi.org/10.1007/978-3-642-18381-2_45.
- Zuleger, F. (2018). Inductive termination proofs with transition invariants and their relationship to the size-change abstraction. In A. Podelski (Ed.) *Static Analysis - 25th International Symposium, SAS 2018, Freiburg, Germany, August 29-31, 2018, Proceedings*, vol. 11002 of *Lecture Notes in Computer Science*, (pp. 423–444). Springer. https://doi.org/10.1007/978-3-319-99725-4_25.
- Zuleger, F., Gulwani, S., Sinn, M., & Veith, H. (2011). Bound analysis of imperative programs with the size-change abstraction. In E. Yahav (Ed.) *Static Analysis - 18th International Symposium, SAS 2011, Venice, Italy, September 14-16, 2011. Proceedings*, vol. 6887 of *Lecture Notes in Computer Science*, (pp. 280–297). Springer. https://doi.org/10.1007/978-3-642-23702-7_22.

CURRICULUM VITAE

Jiawen Liu
 April 10, 2023

665 Commonwealth Avenue,
 CDS 1002,
 Boston University

Boston, MA 02118
[working email](#)
[personal email](#)

Academic Training:

05/2023(expected)	Ph.D. Boston University, Boston, MA; Computer Science
05/2023(expected)	M.S. Boston University, Boston, MA; Computer Science
06/2017	B.A. Central University of Economics and Finance, Haidian, Beijing; Department of Information Science.

Doctoral Research:

Title:	Program Analysis for Quantitative-reachability Properties
Thesis advisor:	Marco Gaboardi, PhD
Defense date:	March 30, 2023
Summary:	Reasoning about program's properties to control program's execution behavior has great potential in program optimization and program security. This dissertation exploits the program analysis to reason about two <i>quantitative-reachability</i> properties with application in different research areas.